IFM 2.2: Programmiermethoden (Sommer 2023)

Carsten Gips

26. März 2026

Inhaltsverzeichnis

1 Syllabus **5**
1.1 Abkündigung des Moduls “Programmiermethoden” . . . 5
1.1.1 Regelung für noch offene Prüfungen . . . 5
1.1.2 Nächste Klausur . . . 5
1.2 Kursbeschreibung . . . 5
1.3 Überblick Modulinhalte . . . 6
1.4 Team . . . 6
1.5 Kursformat . . . 6
1.5.1 Vorlesung (2 SWS) . . . 6
1.5.2 Praktikum (2+1 SWS) . . . 6
1.6 Fahrplan . . . 7
1.6.1 Vorlesung . . . 7
1.6.2 Praktikum . . . 7
1.7 Prüfungsform, Note und Credits . . . 8
1.8 Materialien . . . 9
1.8.1 Literatur . . . 9
1.8.2 Tools . . . 9
1.9 Förderungen und Kooperationen . . . 9
1.9.1 Förderung durch DH.NRW (Digi Fellowships) . . . 9
1.9.2 Kooperation mit dem DigikoS-Projekt . . . 9
1.10 LICENSE . . . 9

2 Vorlesungsunterlagen **10**
2.1 Versionierung mit Git . . . 10
2.1.1 Intro: Versionskontrolle in der Softwareentwicklung . . . 10
2.1.2 Basics der Versionsverwaltung mit Git (lokale Repos) . . . 16
2.1.3 Git Branches: Features unabhängig entwickeln und mit Git verwalten . . . 24
2.1.4 Branching-Strategien mit Git . . . 31
2.1.5 Arbeiten mit Git Remotes (dezentrale Repos) . . . 36
2.1.6 Zusammenarbeit: Git-Workflows und Merge-/Pull-Requests . . . 43
2.1.7 Git Worktree . . . 51
2.2 Modern Java: Funktionaler Stil und Stream-API . . . 54
2.2.1 Lambda-Ausdrücke und funktionale Interfaces . . . 54
2.2.2 Methoden-Referenzen . . . 62
2.2.3 Stream-API . . . 68
2.2.4 Optional . . . 75
2.2.5 Record-Klassen . . . 86
2.2.6 Interfaces: Default-Methoden . . . 90
2.3 Programmiermethoden und Clean Code . . . 95
2.3.1 Javadoc . . . 95
2.3.2 Logging . . . 99
2.3.3 Code Smells . . . 103

2.3.4 Coding Conventions und Metriken . . . 111
2.3.5 Refactoring . . . 120
2.4 Bauen von Programmen, Automatisierung, Continuous Integration . . . 127
2.4.1 Build-Systeme: Gradle . . . 127
2.4.2 Continuous Integration (CI) . . . 136
2.4.3 Einführung in Docker . . . 147
2.4.4 Build-Systeme: Apache Ant . . . 155
2.4.5 Build-Systeme: Apache Maven . . . 163
2.5 Entwurfsmuster . . . 167
2.5.1 Strategy-Pattern . . . 167
2.5.2 Visitor-Pattern . . . 172
2.5.3 Observer-Pattern . . . 182
2.5.4 Command-Pattern . . . 185
2.5.5 Singleton-Pattern . . . 189
2.5.6 Template-Method-Pattern . . . 192
2.5.7 Factory-Method-Pattern . . . 197
2.5.8 Type-Object-Pattern . . . 200
2.5.9 Flyweight-Pattern . . . 206
2.6 Testen mit JUnit und Mockito . . . 212
2.6.1 Einführung Softwaretest . . . 212
2.6.2 Testen mit JUnit (JUnit-Basics) . . . 220
2.6.3 Testfallermittlung: Wie viel und was muss man testen? . . . 228
2.6.4 Mocking mit Mockito . . . 235
2.7 Fortgeschrittene Java-Themen und Umgang mit JVM . . . 247
2.7.1 Serialisierung von Objekten und Zuständen . . . 247
2.7.2 Java Collections Framework . . . 251
2.7.3 Reguläre Ausdrücke . . . 261
2.7.4 Annotationen . . . 269
2.7.5 Reflection . . . 278
2.7.6 Exception-Handling . . . 284
2.7.7 Aufzählungen (Enumerations) . . . 293
2.7.8 Konfiguration eines Programms . . . 298
2.8 Generics: Umgang mit parametrisierten Typen . . . 303
2.8.1 Generische Klassen & Methoden . . . 303
2.8.2 Bounds & Wildcards . . . 309
2.8.3 Type Erasure . . . 312
2.8.4 Generics und Polymorphie . . . 317
2.9 Multi-Threading: Parallelisierung von Programmen . . . 320
2.9.1 Einführung in die nebenläufige Programmierung mit Threads . . . 320
2.9.2 Synchronisation: Verteilter Zugriff auf gemeinsame Ressourcen . . . 325
2.9.3 High-Level Concurrency . . . 331

3 Organisatorisches **337**
3.1 Prüfungsvorbereitung . . . 337
3.1.1 Elektronische Klausur: Termin, Materialien . . . 337
3.1.2 Technische Vorbereitungen . . . 338
3.1.3 Bearbeitung des E-Assessment . . . 339
3.1.4 Fragetypen-Demo . . . 340
3.1.5 Hinweise zu den Inhalten . . . 340
3.1.6 Beispiele für mögliche Fragen . . . 340

4 Praktikum **343**
4.1 Zyklus 1: Konzept . . . 343
4.1.1 Zyklus 1 (erste Woche): Konzeptphase . . . 343
4.2 Zyklus 1: Implementierung . . . 344
4.2.1 Zyklus 1 (zweite Woche): Implementierungsphase . . . 344
4.3 Zyklus 2: Konzept . . . 344
4.3.1 Zyklus 2 (erste Woche): Konzeptphase . . . 344
4.4 Zyklus 2: Implementierung . . . 344
4.4.1 Zyklus 2 (zweite Woche): Implementierungsphase . . . 344
4.5 Zyklus 3: Konzept . . . 345
4.5.1 Zyklus 3 (erste Woche): Konzeptphase . . . 345
4.6 Zyklus 3: Implementierung . . . 345
4.6.1 Zyklus 3 (zweite Woche): Implementierungsphase . . . 345
4.7 Zyklus 4: Konzept . . . 346
4.7.1 Zyklus 4 (erste Woche): Konzeptphase . . . 346
4.8 Zyklus 4: Implementierung . . . 347
4.8.1 Zyklus 4 (zweite Woche): Implementierungsphase . . . 347
4.9 Zyklus 5: Konzept . . . 347
4.9.1 Zyklus 5 (erste Woche): Konzeptphase . . . 347
4.10 Zyklus 5: Implementierung . . . 348
4.10.1 Zyklus 5 (zweite Woche): Implementierungsphase . . . 348
4.11 Sonderabgabe . . . 348
4.11.1 Sondertermin (bei sonst ‘nicht bestanden’ oder Krankheit) . . . 348
4.12 Praktikum . . . 349
4.12.1 Gruppe “Basics” . . . 349
4.12.2 Gruppe “Monster” . . . 349
4.12.3 Gruppe “Loot” . . . 350
4.12.4 Gruppe “Bounty” . . . 350
4.13 Praktikumsaufgaben Gruppe Basics . . . 351
4.13.1 Fallen . . . 351
4.13.2 Fähigkeiten . . . 351
4.13.3 Charakterklassen . . . 352
4.13.4 Speichern und Laden . . . 352
4.13.5 JUnit (Gruppe Basics) . . . 353
4.13.6 Freie Aufgabe . . . 353

4.14 Praktikumsaufgaben Gruppe Monster . . . 353
4.14.1 Nahkampf . . . 353
4.14.2 Fernkampf . . . 354
4.14.3 Monster . . . 354
4.14.4 Boss-Monster . . . 355
4.14.5 Game-Over . . . 355
4.14.6 JUnit (Gruppe Monster) . . . 355
4.14.7 Freie Aufgabe . . . 356
4.15 Praktikumsaufgaben Gruppe Loot . . . 356
4.15.1 Items . . . 356
4.15.2 Freundlicher NPC-Geist . . . 357
4.15.3 Dialogsystem . . . 357
4.15.4 Shop . . . 357
4.15.5 Monster-Schatzkisten . . . 358
4.15.6 Quests . . . 358
4.15.7 Lockpicking . . . 359
4.15.8 JUnit (Gruppe Loot) . . . 359
4.15.9 Freie Aufgabe . . . 359

1 Syllabus

1.1 Abkündigung des Moduls “Programmiermethoden”

Das Modul “Programmiermethoden” wurde im Rahmen der Reakkreditierung im Herbst 2023 durch das kleinere Modul “Programmieren 2” ersetzt und kann nicht mehr belegt werden (es werden keine Vorlesungen und keine Praktika mehr für “Programmiermethoden” angeboten).
Die Inhalte von “Programmiermethoden” werden hier für Wiederholende weiterhin gepflegt. Das zugehörige Lernmodul finden Sie im offenen ILIAS-Bereich der HSBI.
Bis zum Auslaufen der zugehörigen Prüfungsordnungen PO10 und PO18 finden auch weiterhin Prüfungen für Wiederholende statt.

1.1.1 Regelung für noch offene Prüfungen

Wer in “Programmiermethoden” noch eine Prüfung benötigt, kann diese in den Prüfungszeiträumen des Sommersemesters (und ggf. des Wintersemesters, falls angeboten) als Klausur nachholen.
Dabei gelten folgende Regelungen (vgl. Mail von Frau Seele vom 28.09.2023) für offene Prüfungsverfahren:

- Parcoursprüfung: Bei einer offenen Parcoursprüfung bildet die Klausur die gesamte Prüfung.
- Performanzprüfung:
  - Der praktische Teil der Performanzprüfung entfällt, die Note und CP werden über eine Klausur bestimmt.
  - Wenn Sie aber bereits eine Note bzw. CP für den praktischen Teil haben (und nur die theoretische Leistung noch fehlt), dann nehmen Sie an der weiterhin angebotenen Klausur teil und die Gesamtnote wird sich wie bisher aus den beiden Teilnoten zusammensetzen.

1.1.2 Nächste Klausur

Der Termin für die nächste Klausur für “Programmiermethoden” wird über den Prüfplan vom Studierendenservice bekanntgegeben. (vgl. LSF und Ankündigungen)

1.2 Kursbeschreibung

Sie haben letztes Semester in **OOP** die *wichtigsten* Elemente und Konzepte der Programmiersprache Java kennen gelernt.

In diesem Modul geht es darum, diese Kenntnisse sowohl auf der Java- als auch auf der Methoden-Seite so zu erweitern, dass Sie gemeinsam größere Anwendungen erstellen und pflegen können. Sie werden fortgeschrittene Konzepte in Java kennenlernen und sich mit etablierten Methoden in der Softwareentwicklung wie Versionierung von Code, Einhaltung von Coding Conventions, Grundlagen des Softwaretests, Anwendung von Refactoring, Einsatz von Build-Tools und Logging auseinander setzen. Wenn uns dabei ein Entwurfsmuster “über den Weg läuft”, werden wir die Gelegenheit nutzen und uns dieses genauer anschauen.

1.3 Überblick Modulinhalte

1. Fortgeschrittene Konzepte in Java
    - Funktionale Programmierung: Default-Methoden, Funktionsinterfaces, Methodenreferenzen, Lambdas, Stream-API
    - Generische Programmierung: Generics
    - Parallele Programmierung: Threads
    - Reguläre Ausdrücke, Annotationen, Reflection
    - CLI, Konfiguration, Fremde APIs nutzen
2. Fortgeschrittenes OO-Design
    - Entwurfsmuster: Strategy, Template-Method, Factory-Method, Singleton, Observer, Visitor, Command, …
3. Programmiermethoden
    - Versionskontrolle: Git
    - Testen, Coding Conventions, Refactoring
    - Logging, Build-Tools, CI

1.4 Team

- Carsten Gips
- Tutoren (siehe ILIAS-Mitgliederliste)

1.5 Kursformat

1.5.1 Vorlesung (2 SWS)

Fr, 16:30 - 18:00 Uhr (online/J104)

1.5.2 Praktikum (2+1 SWS)

Praktikumsgruppe	Zeit	Raum
Gruppe 1	Fr, 09:00 - 10:30 Uhr	online/J104
Gruppe 2	Fr, 13:30 - 15:00 Uhr	online/J104
Gruppe 3	Fr, 15:00 - 16:30 Uhr	online/J104
Gruppe 4	Fr, 10:45 - 12:15 Uhr	online/J104

Durchführung als **Flipped Classroom**: Alle Sitzungen online/per Zoom (**Zugangsdaten siehe ILIAS**)

1.6 Fahrplan

1.6.1 Vorlesung

Woche	Datum Vorlesung	Themen	Bemerkungen
15	Fr, 14.04.	Orga (**Zoom**) || Einführung Versionierung | Git Basics || Lambda-Ausdrücke || Javadoc	
16	Fr, 21.04.	Git: Branches | Branching-Strategies || Gradle || Methodenreferenzen || Strategy-Pattern	
17	Fr, 28.04.	Git: Remotes | Workflows || Stream-API | Optional	
18	Fr, 05.05.	Testen: Einführung | JUnit-Basics | Testfallermittlung | Mocking || Logging	
19	Fr, 12.05.	Code-Smells | Coding-Rules | Refactoring || CI	**Gastvortrag zu JUnit/Mocking in der Praxis von Daniel Rosowski (Smartsquare GmbH, Bielefeld)**
20	Fr, 19.05.	Serialisierung || Records | Default-Methoden || Docker || Git-Worktree	
21	Fr, 26.05.	Generics: Klassen und Methoden | Bounds und Wildcards | Type Erasure | Polymorphie || Collections	
22	Fr, 02.06.	RegExp || Pattern: Visitor | Observer | Command	
23	Fr, 09.06.	Annotationen | Reflection | Exception-Handling || Singleton-Pattern	
24	Fr, 16.06.	Enumerationen | Konfiguration | ANT || Template-Method-Pattern | Factory-Method-Pattern	
25	Fr, 23.06.	Multi-Threading: Intro Threads | Synchronisierung | Highlevel Threadkonzepte || Maven || Type-Object-Pattern	
26	Fr, 30.06.	Rückblick (**Zoom**) | Prüfungsvorbereitung	
27	*Mi, 05.07.* (Start: 09:00 und 11:00 Uhr)	Klausur (Campus Minden, B40)	

1.6.2 Praktikum

Woche	Blatt	Abgabe ILIAS und Peer-Feedback (ILIAS)	Vorstellung Praktikum
16	B01a	Abgabe: Do, 20.04., 08 Uhr; Peer-Feedback: Fr, 21.04., 08 Uhr	Praktikum: Fr, 21.04.
17	B01b	Abgabe: Do, 27.04., 08 Uhr; Peer-Feedback: Fr, 28.04., 08 Uhr	Praktikum: Fr, 28.04.
18	B02a	Abgabe: Do, 04.05., 08 Uhr; Peer-Feedback: Fr, 05.05., 08 Uhr	Praktikum: Fr, 05.05.
19	B02b	Abgabe: Do, 11.05., 08 Uhr; Peer-Feedback: Fr, 12.05., 08 Uhr	Praktikum: Fr, 12.05.
20	B03a	Abgabe: Do, 18.05., 08 Uhr; Peer-Feedback: Fr, 19.05., 08 Uhr	Praktikum: Fr, 19.05.
21	B03b	Abgabe: Do, 25.05., 08 Uhr; Peer-Feedback: Fr, 26.05., 08 Uhr	Praktikum: Fr, 26.05.
22	B04a	Abgabe: Do, 01.06., 08 Uhr; Peer-Feedback: Fr, 02.06., 08 Uhr	Praktikum: Fr, 02.06.
23	B04b	Abgabe: Do, 08.06., 08 Uhr; Peer-Feedback: Fr, 09.06., 08 Uhr	Praktikum: Fr, 09.06.
24	B05a	Abgabe: Do, 15.06., 08 Uhr; Peer-Feedback: Fr, 16.06., 08 Uhr	Praktikum: Fr, 16.06.
25	B05b	Abgabe: Do, 22.06., 08 Uhr; Peer-Feedback: Fr, 23.06., 08 Uhr	Praktikum: Fr, 23.06.
26	B06	Abgabe: Fr, 30.06., 08 Uhr	Praktikum: Fr, 30.06.

1.7 Prüfungsform, Note und Credits

Performanzprüfung, 7 ECTS

- **Praktische Teilleistung**: Regelmäßige Bearbeitung der Praktikumsaufgaben, fristgerechte Abgabe der Lösungen (PDF, ZIP, Link) im ILIAS, Erstellung von Peer-Feedback im ILIAS, Vorstellung der Lösungen im Praktikum => Punkte

  Notenspiegel:

  - 90 Punkte gesamt erreichbar: Zyklus 1 und 2 je 15 Punkte, Zyklus 3 bis 5 je 15+5 Punkte
  - 4.0: ab 50% (45.0 Punkte), alle 5% nächste Teilnote, 1.0: ab 95% (85.5 Punkte)

- **Theoretische Teilleistung**: Digitale Klausur ("**Klausur**") in den Prüfungszeiträumen; Prüfungsvorbereitung.

- **Gesamtnote**: 50% Praxis, 50% Theorie

Wiederholer mit bereits begonnener Parcours-Prüfung absolvieren stattdessen eine Parcours-Prüfung. Bitte melden Sie sich vor Beginn der Praktika per E-Mail beim Dozenten.

1.8 Materialien

1.8.1 Literatur

1. “**Java ist auch eine Insel**”. Ullenboom, C., Rheinwerk-Verlag, 2021. ISBN 978-3-8362-8745-6.
2. “**Pro Git** (Second Edition)”. Chacon, S. und Straub, B., Apress, 2014. ISBN 978-1-4842-0077-3.
3. “The Java Tutorials”. Oracle Corporation, 2023.
4. “Learn Java”. Oracle Corporation, 2023.

1.8.2 Tools

- JDK: Java SE 21 (LTS) (Oracle oder Alternativen, bitte 64-bit Version nutzen)
- IDE: Eclipse IDE for Java Developers oder IntelliJ IDEA (Community Edition) oder Visual Studio Code oder Vim oder …
- Git

1.9 Förderungen und Kooperationen

1.9.1 Förderung durch DH.NRW (Digi Fellowships)

Die Überarbeitung dieser Lehrveranstaltung wurde vom Ministerium für Kultur und Wissenschaft (MKW) in NRW im Einvernehmen mit der Digitalen Hochschule NRW (DH.NRW) gefördert: “Fellowships für Innovationen in der digitalen Hochschulbildung” (*Digi Fellowships*).

1.9.2 Kooperation mit dem DigikoS-Projekt

Diese Vorlesung wird zudem vom Projekt “Digitalbaukasten für kompetenzorientiertes Selbststudium” (*DigikoS*) unterstützt. Ein vom DigikoS-Projekt ausgebildeter Digital Learning Scout hat insbesondere die Koordination der digitalen Gruppenarbeiten, des Peer-Feedbacks und der Postersessions in ILIAS technisch und inhaltlich begleitet. DigikoS wird als Verbundprojekt von der Stiftung Innovation in der Hochschullehre gefördert.

1.10 LICENSE



Unless otherwise noted, this work by Carsten Gips and contributors is licensed under CC BY-SA 4.0. See the credits for a detailed list of contributing projects.

2 Vorlesungsunterlagen

2.1 Versionierung mit Git

2.1.1 Intro: Versionskontrolle in der Softwareentwicklung

TL;DR
In der Softwareentwicklung wird häufig ein Versionsmanagementsystem (VCS) eingesetzt, welches die Verwaltung von Versionsständen und Änderungen ermöglicht. Ein Repository sammelt dabei die verschiedenen Änderungen (quasi wie eine Datenbank der Software-Versionsstände). Die Software *Git* ist verbreiteter Vertreter und arbeitet mit dezentralen Repositories.
Ein neues lokales Repository kann man mit `git init` anlegen. Der Befehl legt den Unterordner `.git/` im aktuellen Ordner an, darin befindet sich das lokale Repository und weitere von Git benötigte Dateien (FINGER WEG!). Die Dateien und anderen Unterordner im aktuellen Ordner können nun der Versionskontrolle hinzugefügt werden.
Den lokal vorliegenden (Versions-) Stand der Dateien im aktuellen Ordner nennt man auch "Workingcopy".
Ein bereits existierendes Repo kann mit `git clone <url>` geklont werden.
GitHub ist nicht Git, sondern ein kommerzieller Anbieter, der das Hosten von Git-Repositories und weitere Features anbietet.

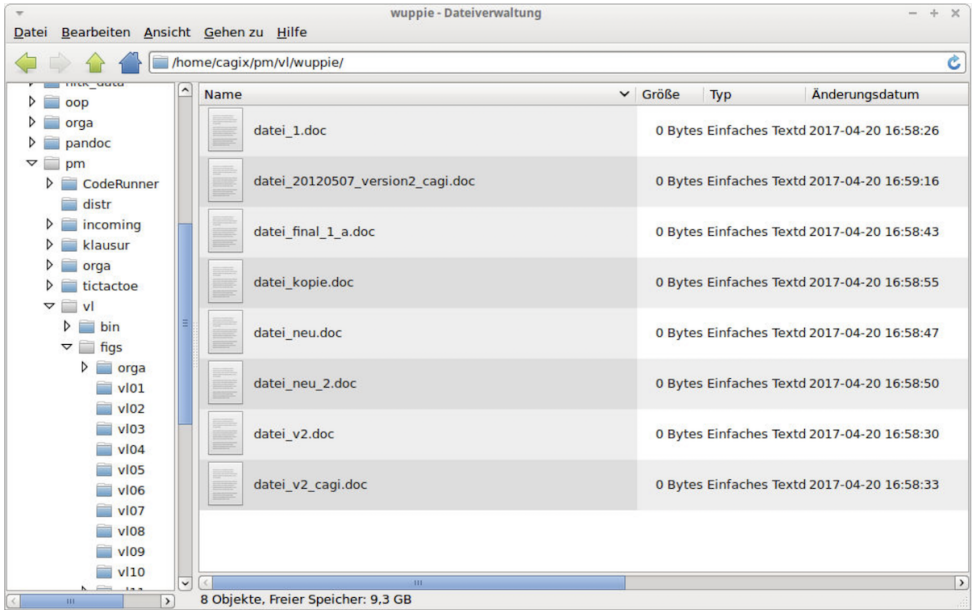
Videos

- VL Git Intro
- Demo Config
- Demo Repo

2.1.1.1 Typische Probleme bei SW-Entwicklung

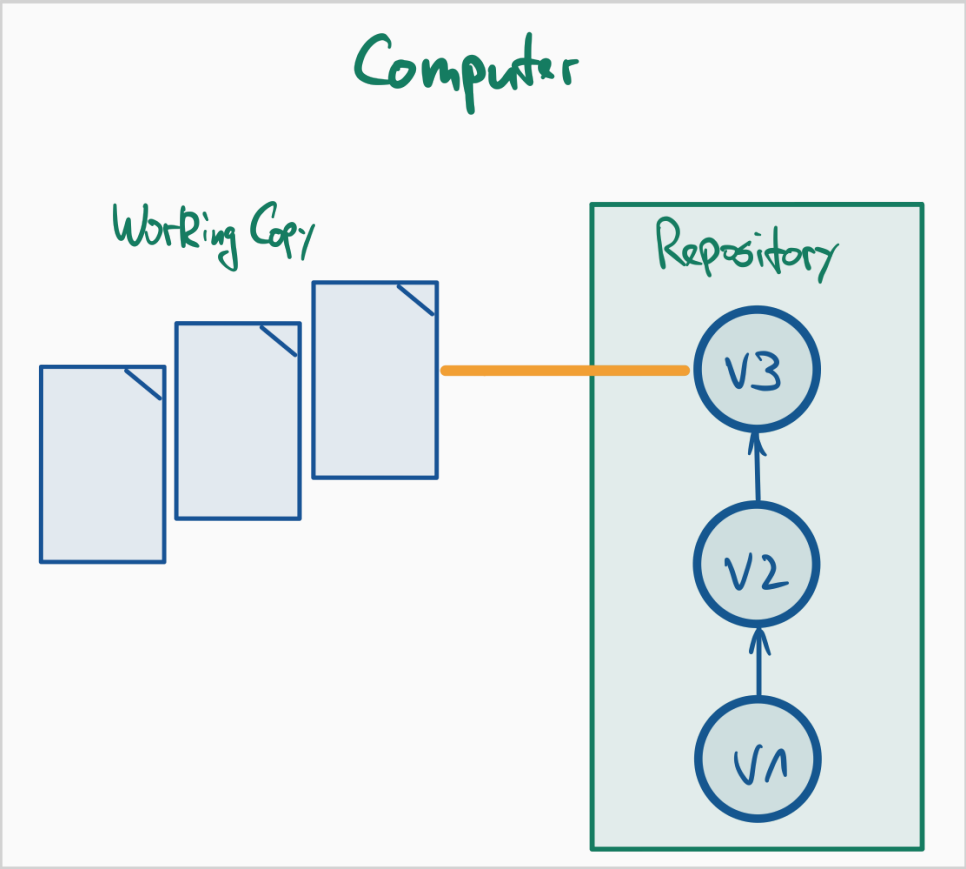
- Was hat wer wann (und wo) geändert? Und warum?
- Ich brauche den Stand von gestern/letzter Woche/…
- Ich will schnell mal eine neue Idee ausprobieren …
- Ich arbeite an mehreren Rechnern (Synchronisation)
- Wir müssen gemeinsam an der gleichen Codebasis arbeiten.
- Wir arbeiten am Release v42, aber Kunde braucht schnell einen Fix für v40

2.1.1.2 Folgen SW-Entwicklung ohne Versionsverwaltung



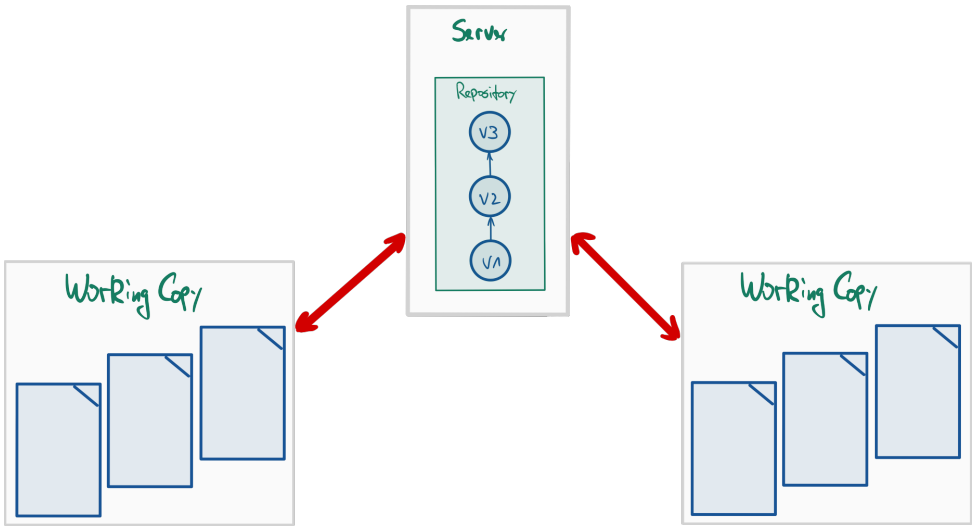
- Filesystem müllt voll mit manuell versionierten Dateien/Sicherungen ala `file_20120507_version2_cagi.txt`
- Ordner/Projekte müssen dupliziert werden für neue Ideen
- Code müllt voll mit auskommentierten Zeilen (“Könnte ja noch gebraucht werden”)
- Unklar, wann welche Änderung von wem warum eingeführt wurde
- Unbeabsichtigtes Überschreiben mit älteren Versionen beim Upload in gemeinsamen Filesharing-Bereich

2.1.1.3 Prinzip Versionsverwaltung



- **Repository: Datenbank** mit verschiedenen Versionsständen, Kommentaren, Tags etc.
- **Workingcopy: Arbeitskopie** eines bestimmten Versionsstandes

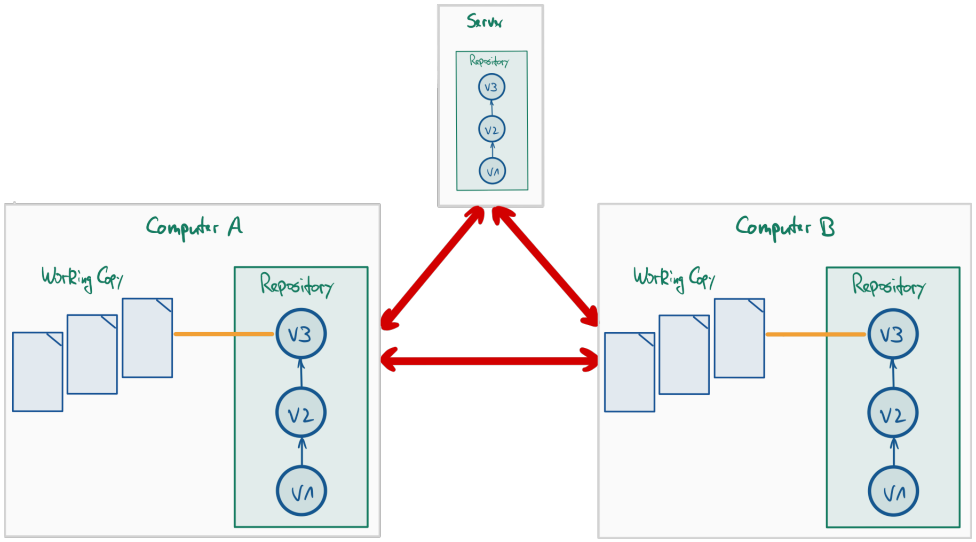
2.1.1.4 Varianten: Zentrale Versionsverwaltung (Beispiel SVN)



Es gibt ein zentrales Repository (typischerweise auf einem Server), von dem die Developer einen bestimmten Versionsstand “auschecken” (sich lokal kopieren) und in welches sie Änderungen wieder zurück “pushen”.

Zur Abfrage der Historie und zum Veröffentlichen von Änderungen benötigt man entsprechend immer eine Verbindung zum Server.

2.1.1.5 Varianten: Verteilte Versionsverwaltung (Beispiel Git)



In diesem Szenario hat jeder Developer nicht nur die Workingcopy, sondern auch noch eine Kopie des Repositories. Zusätzlich kann es einen oder mehrere Server geben, auf denen dann nur das Repository vorgehalten wird, d.h. dort gibt es normalerweise keine Workingcopy. Damit kann unabhängig voneinander gearbeitet werden.

Allerdings besteht nun die Herausforderung, die geänderten Repositories miteinander abzugleichen. Das kann zwischen dem lokalen Rechner und dem Server passieren, aber auch zwischen zwei “normalen” Rechnern (also zwischen den Developern).

Hinweis: *GitHub ain't no Git!* Git ist eine Technologie zur Versionsverwaltung. Es gibt verschiedene Implementierungen und Plugins für IDEs und Editoren. GitHub ist dagegen *ein* Dienstleister, wo man Git-Repositories ablegen kann und auf diese mit Git (von der Konsole oder aus der IDE) zugreifen kann. Darüber hinaus bietet der Service aber zusätzliche Features an, beispielsweise ein Issue-Management oder sogenannte *Pull-Requests*. Dies hat aber zunächst mit Git nichts zu tun. Weitere populäre Anbieter sind beispielsweise Bitbucket oder Gitlab oder Gitea, wobei einige auch selbst gehostet werden können.

2.1.1.6 Versionsverwaltung mit Git: Typische Arbeitsschritte

1. Repository anlegen (oder clonen)

2. Dateien neu erstellen (und löschen, umbenennen, verschieben)
3. Änderungen einpflegen ("committen")
4. Änderungen und Logs betrachten
5. Änderungen rückgängig machen
6. Projektstand markieren ("taggen")

7. Entwicklungszweige anlegen ("branchen")
8. Entwicklungszweige zusammenführen ("mergen")

9. Änderungen verteilen (verteiltes Arbeiten, Workflows)

2.1.1.7 (Globale) Konfiguration **Minimum**:

- `git config --global user.name <name>`
- `git config --global user.email <email>`

Diese Konfiguration muss man nur einmal machen.

Wenn man den Schalter `--global` weglässt, gelten die Einstellungen nur für das aktuelle Projekt/Repo.

Zumindest Namen und EMail-Adresse **muss** man setzen, da Git diese Information beim Anlegen der Commits speichert (== benötigt!).

Aliase:

- `git config --global alias.ci commit`
- `git config --global alias.co checkout`
- `git config --global alias.br branch`
- `git config --global alias.st status`
- `git config --global alias.ll 'log --all --graph --decorate --oneline'`

Zusätzlich kann man weitere Einstellungen vornehmen, etwa auf bunte Ausgabe umschalten: `git config --global color.ui auto` oder Abkürzungen (Aliase) für Befehle definieren: `git config --global alias.ll 'log --all --oneline --graph --decorate'` ...

Git (und auch GitHub) hat kürzlich den Namen des Default-Branches von `master` auf `main` geändert. Dies kann man in Git ebenfalls selbst einstellen: `git config --global init.defaultBranch <name>`.

Anschauen kann man sich die Einstellungen in der Textdatei `~/.gitconfig` oder per Befehl `git config --global -l`.

2.1.1.8 Neues Repo anlegen

- `git init`

  => Erzeugt neues Repository im akt. Verzeichnis

- `git clone <url>`

  => Erzeugt (verlinkte) Kopie des Repos unter `<url>`

2.1.1.9 Wrap-Up

- Git: Versionsmanagement mit dezentralen Repositories
- Anlegen eines lokalen Repos mit `git init`
- Clonen eines existierenden Repos mit `git clone <url>`

Zum Nachlesen

- Chacon und Straub (2014, Kap. 1 und 2)
- Atlassian Pty Ltd (2022)
- Github Inc. (2022)

Lernziele

- k1: Varianten der Versionierung
- k1: Begriffe Workingcopy und Repository
- k2: Github ist nicht Git
- k2: Erstellung von lokalen Git-Repositories
- k3: Umgang mit entsprechenden Git-Befehlen auf der Konsole

Quizzes

- Quiz Git Intro (ILIAS)

2.1.2 Basics der Versionsverwaltung mit Git (lokale Repos)

TL;DR
Änderungen an Dateien (in der Workingcopy) werden mit `git add` zum "Staging" (Index) hinzugefügt. Dies ist eine Art Sammelbereich für Änderungen, die mit dem nächsten Commit in das Repository überführt werden. Neue (bisher nicht versionierte Dateien) müssen ebenfalls zunächst mit `git add` zum Staging hinzugefügt werden.
Änderungen kann man mit `git log` betrachten, dabei erhält man u.a. eine Liste der Commits und der jeweiligen Commmit-Messages.
Mit `git diff` kann man gezielt Änderungen zwischen Commits oder Branches betrachten.
Mit `git tag` kann man bestimmte Commits mit einem "Stempel" (zusätzlicher Name) versehen, um diese leichter finden zu können.
Wichtig sind die Commit-Messages: Diese sollten eine kurze Zusammenfassung haben, die **aktiv** formuliert wird (was ändert dieser Commit: "Formatiere den Java-Code entsprechend Style"; nicht aber "Java-Code nach Style formatiert"). Falls der Kommentar länger sein soll, folgt eine Leerzeile auf die erste Zeile (Zusammenfassung) und danach ein Block mit der längeren Erklärung.

Videos

- VL Git Basics
- Demo New Files
- Demo Arbeitsablauf: Datei ändern - stagen - committen
- Demo Amend
- Demo Log
- Demo Diff
- Demo Tag

2.1.2.1 Versionsverwaltung mit Git: Typische Arbeitsschritte

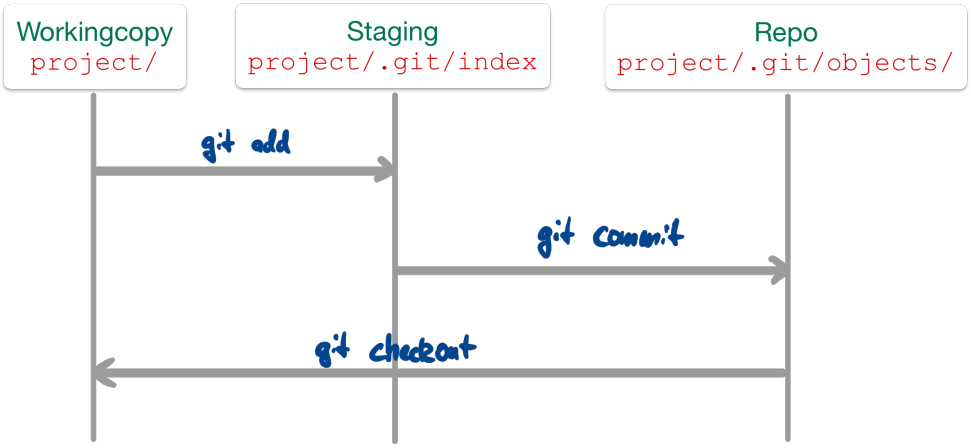
1. Repository anlegen (oder clonen)

2. Dateien neu erstellen (und löschen, umbenennen, verschieben)
3. Änderungen einpflegen ("committen")
4. Änderungen und Logs betrachten
5. Änderungen rückgängig machen
6. Projektstand markieren ("taggen")

7. Entwicklungszweige anlegen ("branchen")
8. Entwicklungszweige zusammenführen ("mergen")

9. Änderungen verteilen (verteiltes Arbeiten, Workflows)

2.1.2.2 Dateien unter Versionskontrolle stellen



1. `git add .` (oder `git add <file>`)

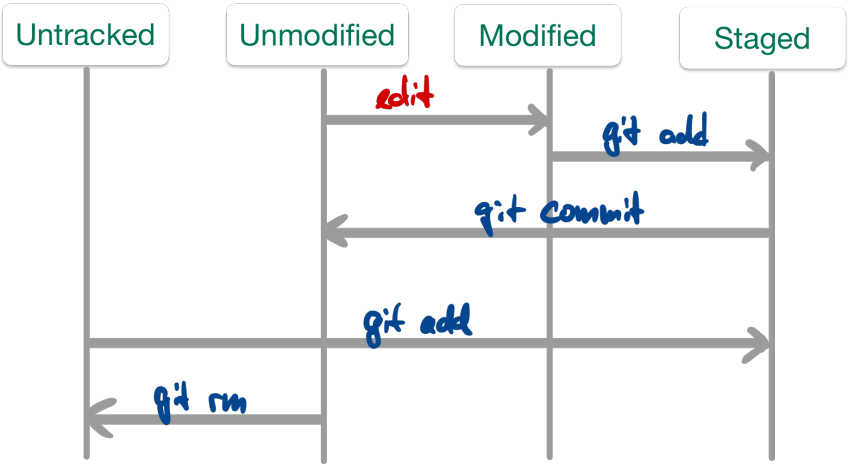
   => Stellt alle Dateien (bzw. die Datei `<file>`) im aktuellen Verzeichnis unter Versionskontrolle

2. `git commit`

   => Fügt die Dateien dem Repository hinzu

Abfrage mit `git status`

2.1.2.3 Änderungen einpflegen



- Abfrage mit: `git status`
- "Staging" von modifizierten Dateien: `git add <file>`
- Committen der Änderungen im Stage: `git commit`

Anmerkung: Alternativ auch mit `git commit -m "Kommentar"`, um das Öffnen des Editors zu vermeiden … geht einfach schneller ;)

Das "staging area" stellt eine Art Zwischenebene zwischen Working Copy und Repository dar: Die Änderungen sind temporär "gesichert", aber noch nicht endgültig im Repository eingepflegt ("committed").

Man kann den Stage dazu nutzen, um Änderungen an einzelnen Dateien zu sammeln und diese dann (in einem Commit) gemeinsam einzuchecken.

Man kann den Stage in der Wirkung umgehen, indem man alle in der Working Copy vorliegenden Änderungen per `git commit -a -m "Kommentar"` eincheckt. Der Schalter "`-a`" nimmt alle vorliegenden Änderungen an **bereits versionierten** Dateien, fügt diese dem Stage hinzu und führt dann den Commit durch. Das ist das von SVN bekannte Verhalten. Achtung: Nicht versionierte Dateien bleiben dabei außen vor!

2.1.2.4 Letzten Commit ergänzen

- `git commit --amend -m "Eigentlich wollte ich das so sagen"`

  Wenn keine Änderungen im Stage sind, wird so die letzte Commit-Message geändert.

- `git add <file>; git commit --amend`

  Damit können vergessene Änderungen an der Datei `<file>` zusätzlich im letzten Commit aufgezeichnet werden.

  In beiden Fällen ändert sich die Commit-ID!

2.1.2.5 Weitere Datei-Operationen: hinzufügen, umbenennen, löschen

- Neue (unversionierte) Dateien und Änderungen an versionierten Dateien zum Staging hinzufügen: `git add <file>`
- Löschen von Dateien (Repo+Workingcopy): `git rm <file>`
- Löschen von Dateien (nur Repo): `git rm --cached <file>`
- Verschieben/Umbenennen: `git mv <fileAlt> <fileNeu>`

Aus Sicht von Git sind zunächst alle Dateien "untracked", d.h. stehen nicht unter Versionskontrolle.

Mit `git add <file>` (und `git commit`) werden Dateien in den Index (den Staging-Bereich, d.h. nach dem Commit letztlich in das Repository) aufgenommen. Danach stehen sie unter "Beobachtung" (Versionskontrolle). So lange, wie eine Datei identisch zur Version im Repository ist, gilt sie als unverändert ("unmodified"). Eine Änderung führt entsprechend zum Zustand "modified", und ein `git add <file>` speichert die Änderungen im Stage. Ein Commit überführt die im Stage vorgemerkte Änderung in das Repo, d.h. die Datei gilt wieder als "unmodified".

Wenn eine Datei nicht weiter versioniert werden soll, kann sie aus dem Repo entfernt werden. Dies kann mit `git rm <file>` geschehen, wobei die Datei auch aus der Workingcopy gelöscht wird. Wenn die Datei erhalten bleiben soll, aber nicht versioniert werden soll (also als "untracked" markiert werden soll), dann muss sie mit `git rm --cached <file>` aus der Versionskontrolle gelöscht werden. Achtung: Die Datei ist dann nur ab dem aktuellen Commit gelöscht, d.h. frühere Revisionen enthalten die Datei noch!

Wenn eine Datei umbenannt werden soll, geht das mit `git mv <fileAlt> <fileNeu>`. Letztlich ist dies nur eine Abkürzung für die Folge `git rm --cached <fileAlt>`, manuelles Umbenennen der Datei in der Workingcopy und `git add <fileNeu>`.

2.1.2.6 Commits betrachten

- Liste aller Commits: `git log`
  - `git log -<n>` oder `git log --since="3 days ago"` Meldungen eingrenzen …
  - `git log --stat` Statistik …
  - `git log --author="pattern"` Commits eines Autors
  - `git log <file>` Änderungen einer Datei

- Inhalt eines Commits: `git show`

2.1.2.7 Änderungen und Logs betrachten

- `git diff [<file>]`

  Änderungen zwischen Workingcopy und letztem Commit (ohne Stage)

  Das "staging area" wird beim Diff von Git behandelt, als wären die dort hinzugefügten Änderungen bereits eingecheckt (genauer: als letzter Commit im aktuellen Branch im Repo vorhanden). D.h. wenn Änderungen in einer Datei mittels `git add <datei>` dem Stage hinzugefügt wurden, zeigt `git diff <datei>` keine Änderungen an!

- `git diff commitA commitB`

  Änderungen zwischen Commits

- Blame: `git blame <file>`

  Wer hat was wann gemacht?

2.1.2.8 Dateien ignorieren: *.gitignore*

- Nicht alle Dateien gehören ins Repo:
  - generierte Dateien: `.class`
  - temporäre Dateien
- Datei `.gitignore` anlegen und committen
  - Wirkt auch für Unterordner
  - Inhalt: Reguläre Ausdrücke für zu ignorierende Dateien und Ordner

```
# Compiled source #
*.class
*.o
*.so

# Packages #
*.zip

# All directories and files in a directory #
bin/**/*
```

man 5 gitignore

2.1.2.9 Zeitmaschine

- Änderungen in Workingcopy rückgängig machen
  - Änderungen nicht in Stage: `git checkout <file>` oder `git restore <file>`
  - Änderungen in Stage: `git reset HEAD <file>` oder `git restore --staged <file>`

  => Hinweise von `git status` beachten!

- Datei aus altem Stand holen:
  - `git checkout <commit> <file>`, oder
  - `git restore --source <commit> <file>`
- Commit verwerfen, Geschichte neu: `git revert <commit>`

Hinweis: In den neueren Versionen von Git ist der Befehl `git restore` hinzugekommen, mit dem Änderungen rückgängig gemacht werden können. Der bisherige Befehl `git checkout` steht immer noch zur Verfügung und bietet über `git restore` hinaus weitere Anwendungsmöglichkeiten.

- Stempel (Tag) vergeben: `git tag <tagname> <commit>`
- Tags anzeigen: `git tag` und `git show <tagname>`

2.1.2.10 Wann und wie committen?

Jeder Commit stellt einen Rücksetzpunkt dar!

Typische Regeln:

- Kleinere “Häppchen” einchecken: ein Feature oder Task (das nennt man auch *atomic commit*: das kleinste Set an Änderungen, die gemeinsam Sinn machen und die ggf. gemeinsam zurückgesetzt werden können)

- Logisch zusammenhängende Änderungen gemeinsam einchecken
- Projekt muss nach Commit compilierbar sein
- Projekt sollte nach Commit lauffähig sein

Ein Commit sollte in sich geschlossen sein, d.h. die kleinste Menge an Änderungen enthalten, die gemeinsam einen Sinn ergeben und die (bei Bedarf) gemeinsam zurückgesetzt oder verschoben werden können. Das nennt man auch **atomic commit**.

Wenn Sie versuchen, die Änderungen in Ihrem Commit zu beschreiben (siehe nächste Folie "Commit-Messages"), dann werden Sie einen *atomic commit* mit einem kurzen Satz (natürlich im Imperativ!) beschreiben können. Wenn Sie mehr Text brauchen, haben Sie wahrscheinlich keinen *atomic commit* mehr vor sich.

Lesen Sie dazu auch How atomic Git commits dramatically increased my productivity - and will increase yours too.

2.1.2.11 Schreiben von Commit-Messages: WARUM?! Schauen Sie sich einmal einen Screenshot eines `git log --oneline 61e48f0..e2c8076` im Dungeon-CampusMinden/Dungeon an:

```
e2c8076 [Toolchain] Re-rename groupID (#374)
a991cf3 Repository umbenennen (#370)
8a3f213 Fix Typo in graphics Template (#360)
4cd9463 Fix für Issue 357 Pathfinding Probleme mit ID 0  (#359)
a21cd66 Merge frame methods (#353)
719836d [GITHUB] New Issue-Templates (#336)
20caad9 Update: github_ci.yml, cache gradle wrapper (#350)
0ed33af Add dungeon-starter to core (#348)
8c73ab2 [GENERATOR] Perlin Noise #332 (#334)
9c2c611 [FEATURE] Refactor Level (#329)
144d7b4 Update gson version and add dependencies checker (#344)
a75d101 [ENHANCEMENT] Add List for AbstractController (#337)
8170ade [BUG] add Javadoc for beginFrame/endFrame/setup (#339)
4348a13 Remove Xms and Xmx options (#330)
0b18d69 rename Tile.texturePath to Tile.texture (#322)
46530b6 Layer system 5 (#300)
d5c20f5 rename getTexture in getTexturePath (#312)
cb24ce5 Levelgenerierung: Remove Wildcards and Replacement (#310)
763c02e Remove version number (master, 5.x) (#301)
d73aa0c Door bug (#298)
637acbe Pausefunktion für Gameloop (#299)
1a9326c Update workflows and buildscripts (develop) (#293)
2ccce6d konstante Laufzeit der getTileAt(coordinate) Methode in Level  (#266)
0b565d4 Better start and exit (#258)
```

Nun stellen Sie sich vor, Sie sind auf der Suche nach Informationen, suchen einen bestimmten Commit oder wollen eine bestimmte Änderung finden …

Wenn man das genauer analysiert, dann stören bestimmte Dinge:

- Mischung aus Deutsch und Englisch
- "Vor-sich-hin-Murmeln": "Layer system 5"
- Teileweise werden Tags genutzt wie `[BUG]`, aber nicht durchgängig
- Mischung zwischen verschiedenen Formen: "Repo umbenennen", "Benenne Repo um", "Repo umbenannt"

- Unterschiedliche Groß- und Kleinschreibung
- Sehr unterschiedlich lange Zeilen/Kommentare

Das Beachten einheitlicher Regeln ist enorm wichtig!

Leider sagt sich das so leicht - in der Praxis macht man es dann doch schnell wieder unsauber. Dennoch, auch im Dungeon-Repo gibt es einen positiven Trend (`git log --oneline 8039d6c..7f49e89`):

```
7f49e89 [GAME] add better level size generation options (#493)
c64f7c1 [GAME] add fireball textures (#477)
f645815 [GAME] remove old logo files, use new logo (#478)
3f1f2c9 new logo with the red cat in the dimensions 35x35, 64x64 and 128x128 (#476)
c6f0fc1 [Game]  Schatztruhe (#463)
803e2d0 [GAME] Change Exit/StartTile (#459)
8483714 [Game] Fix Exception in Configuration on Game Startup (#475)
cb8017f [GAME] Erweiterung des Collision checks um onEnter und onLeave (#452)
382e541 [GAME] Add fight skill (#469)
9c429af [GAME] SkillSystem (#395)
6d76e79 [GAME] add Interaktionssystem & Interaktionscomponent (#389)
a90dfe6 [GAME] add Skill Cool Down (#394)
e08c3b3 [GAME] add  Basis Logging (#455)
7290999 [GAME] fix getRandomTile mögliche Endlosschleife (#458)
13f460d [GAME] Items (Active & Passive) + Stats (#450)
5f0459f [GAME] onDeath drop Loot (#396)
03e1ff0 [GAME] Funktionales Handling von Optionals (#400)
4bb8b9b [GAME] Configuration File (#443)
2e0caaf [GAME] Inventory Component (#388)
195c686 [GAME] add XPComponent & -System
3d342bb [GAME] Merge `pm-dungeon` and `game` Projects and create `api`Project (#401)
64a0fce [GAME] Build w/ local code, i.e. do NOT use Maven Central, and remove duplicated assets (#399)
5c11d7d [GAME] IIdle: PatrouilleWalk (#352)
d0da90f [GAME] add ECS Dokumentation (#211)
```

Typische Regeln und Konventionen tauchen überall auf, beispielsweise in Chacon und Straub (2014) oder bei Tim Pope (siehe nächstes Beispiel) oder bei "How to Write a Git Commit Message".

```
Short (50 chars or less) summary of changes

More detailed explanatory text, if necessary.  Wrap it to about
72 characters or so.  In some contexts, the first line is treated
as the subject of an email and the rest of the text as the body.
The blank line separating the summary from the body is critical
(unless you omit the body entirely); tools like rebase can get
confused if you run the two together.

Further paragraphs come after blank lines.

 - Bullet points are okay, too
 - Typically a hyphen or asterisk is used for the bullet, preceded
   by a single space, with blank lines in between, but conventions
   vary here
```

Quelle: "A Note About Git Commit Messages" by Tim Pope on tbaggery.com

Denken Sie sich die Commit-Message als E-Mail an einen zukünftigen Entwickler, der das in fünf Jahren liest!

Vom Aufbau her hat eine E-Mail auch eine Summary und dann den eigentlichen Inhalt … Erklären Sie das **"WARUM"** der Änderung! (Das "WER", "WAS", "WANN" wird bereits automatisch von Git aufgezeichnet …)

Lesen (und beachten) Sie unbedingt auch "How to Write a Git Commit Message"!

2.1.2.12 Wrap-Up

- Änderungen einpflegen zweistufig (`add`, `commit`)
- Status der Workingcopy mit `status` ansehen
- Logmeldungen mit `log` ansehen
- Änderungen auf einem File mit `diff` bzw. `blame` ansehen
- Projektstand markieren mit `tag`
- Ignorieren von Dateien/Ordnern: Datei `.gitignore`

Zum Nachlesen

- Chacon und Straub (2014, Kap. 2)
- Atlassian Pty Ltd (2022)
- Github Inc. (2022)

Lernziele

- k3: Umgang mit Dateien: Hinzufügen zum und Löschen aus Repo
- k3: Umgang mit Änderungen: Hinzufügen zum Staging und Commit
- k3: Herausfinden von Unterschieden, Ansehen der Historie
- k3: Ignorieren von Dateien und Ordnern

Quizzes

- Quiz Git Basics (ILIAS)

Challenges
Versionierung 101

1. Legen Sie ein Repository an.
2. Fügen Sie Dateien dem Verzeichnis hinzu und stellen Sie *einige* davon unter Versionskontrolle.
3. Ändern Sie eine Datei und versionieren Sie die Änderung.
4. Was ist der Unterschied zwischen "`git add .; git commit`" und "`git commit -a`"?
5. Wie finden Sie heraus, welche Dateien geändert wurden?
6. Entfernen Sie eine Datei aus der Versionskontrolle, aber nicht aus dem Verzeichnis!
7. Entfernen Sie eine Datei komplett (Versionskontrolle und Verzeichnis).
8. Ändern Sie eine Datei und betrachten die Unterschiede zum letzten Commit.
9. Fügen Sie eine geänderte Datei zum Index hinzu. Was erhalten Sie bei `git diff <datei>`?
10. Wie können Sie einen früheren Stand einer Datei wiederherstellen? Wie finden Sie überhaupt den Stand?
11. Legen Sie sich ein Java-Projekt in Ihrer IDE an an. Stellen Sie dieses Projekt unter Git-Versionskontrolle. Führen Sie die vorigen Schritte mit Ihrer IDE durch.

Interaktive Git-Tutorials: Schaffen Sie die Rätsel?

- Learn Git Branching
- Oh My Git!
- Git Time

2.1.3 Git Branches: Features unabhängig entwickeln und mit Git verwalten

TL;DR
Die Commits in Git bauen aufeinander auf und bilden dabei eine verkettete "Liste". Diese "Liste" nennt man auch *Branch* (Entwicklungszweig). Beim Initialisieren eines Repositories wird automatisch ein Default-Branch angelegt, auf dem die Commits dann eingefügt werden.
Weitere Branches kann man mit `git branch` anlegen, und die Workingcopy kann mit `git switch` oder `git checkout` auf einen anderen Branch umgeschaltet werden. Auf diese Weise kann man an mehreren Features parallel arbeiten, ohne dass die Arbeiten sich gegenseitig stören.
Zum Mergen (Vereinigen) von Branches gibt es `git merge`. Dabei werden die Änderungen im angegebenen Branch in den aktuell in der Workingcopy ausgecheckten Branch integriert und hier ggf. ein neuer Merge-Commit erzeugt. Falls es in beiden Branches inkompatible Änderungen an der selben Stelle gab, entsteht beim Mergen ein Merge-Konflikt. Dabei zeigt Git in den betroffenen Dateien jeweils an, welche Änderung aus welchem Branch stammt und man muss diesen Konflikt durch Editieren der Stellen manuell beheben.
Mit `git rebase` kann die Wurzel eines Branches an eine andere Stelle verschoben werden. Dies wird später bei Workflows eine Rolle spielen.

Videos

- VL Git Branches
- Demo Anlegen und Mergen von Branches
- Demo Auflösen von Merge-Konflikten
- Demo HEAD

2.1.3.1 Neues Feature entwickeln/ausprobieren

```
A---B---C  master
```

- Bisher nur lineare Entwicklung: Commits bauen aufeinander auf (lineare Folge von Commits)
- `master` ist der (Default-) Hauptentwicklungszweig
  - Pointer auf letzten Commit
  - Default-Name: "`master`" (muss aber nicht so sein bzw. kann geändert werden)

Anmerkung: Git und auch Github haben den Namen für den Default-Branch von `master` auf `main`geändert. Der Name an sich ist aber für Git bedeutungslos und kann mittels `git config --global init.defaultBranch <name>` geändert werden. In Github hat der Default-Branch eine gewisse Bedeutung, beispielsweise ist der Default-Branch das automatische Ziel beim Anlegen von Pull-Requests. In Github kann man den Default-Namen global in den User-Einstellungen (Abschnitt "Repositories") und für jedes einzelne Repository in den Repo-Einstellungen (Abschnitt "Branches") ändern.

Entwicklung des neuen Features soll stabilen `master`-Branch nicht beeinflussen => Eigenen Entwicklungszweig für die Entwicklung des Features anlegen:

1. Neuen Branch erstellen: `git branch wuppie`
2. Neuen Branch auschecken: `git checkout wuppie` oder `git switch wuppie`

Alternativ: `git checkout -b wuppie` oder `git switch -c wuppie` (neuer Branch und auschecken in einem Schritt)

```
A---B---C  master, wuppie
```

Startpunkt: prinzipiell beliebig (jeder Commit in der Historie möglich).

Die gezeigten Beispiel zweigen den neuen Branch direkt vom aktuell ausgecheckten Commit/Branch ab. Also aufpassen, was gerade in der Workingcopy los ist!

Alternativ nutzen Sie die Langform: `git branch wuppie master` (mit `master` als Startpunkt; hier kann jeder beliebige Branch, Tag oder Commit genutzt werden).

Nach Anlegen des neuen Branches zeigen beide Pointer auf den selben Commit.

Anmerkung: In neueren Git-Versionen wurde der Befehl "**`switch`**" eingeführt, mit dem Sie in der Workingcopy auf einen anderen Branch wechseln können. Der bisherige Befehl "`checkout`" funktioniert aber weiterhin.

2.1.3.2 Arbeiten im Entwicklungszweig

```
          D  wuppie
         /
A---B---C  master
```

- Entwicklung des neuen Features erfolgt im eigenen Branch: beeinflusst den stabilen `master`-Branch nicht
- Wenn in der Workingcopy der Feature-Branch ausgecheckt ist, gehen die Commits in den Feature-Branch; der `master` bleibt auf dem alten Stand
- Wenn der `master` ausgecheckt wäre, würden die Änderungen in den `master` gehen, d.h. der `master` würde sich ab Commit `C` parallel zu `wuppie` entwickeln

2.1.3.3 Problem: Fehler im ausgelieferten Produkt

```
          D  wuppie
         /
A---B---C  master
```

Fix für `master` nötig:

1. `git checkout master`
2. `git checkout -b fix`
3. Änderungen in `fix` vornehmen …

Das führt zu dieser Situation:

```
          D  wuppie
         /
A---B---C  master
         \
          E  fix
```

`git checkout` <branchname> holt den aktuellen Stand des jeweiligen Branches in die Workingcopy. (Das geht in neueren Git-Versionen auch mit `git` **`switch`** <branchname>.)

Man kann weitere Branches anlegen, d.h. hier im Beispiel ein neuer Feature-Branch `fix`, der auf dem `master` basiert. Analog könnte man auch Branches auf der Basis von `wuppie` anlegen …

2.1.3.4 Fix ist stabil: Integration in *master*

```
          D  wuppie
         /
A---B---C  master
         \
          E  fix
```

1. `git checkout master`
2. `git merge fix` => **fast forward** von `master`
3. `git branch -d fix`

Der letzte Schritt entfernt den Branch `fix`.

```
          D  wuppie
         /
A---B---C---E  master
```

- Allgemein: `git merge` <branchname> führt die Änderungen im angegebenen Branch <branchname> in den aktuell in der Workingcopy ausgecheckten Branch ein. Daraus resultiert für den aktuell ausgecheckten Branch ein neuer Commit, der Branch <branchname> bleibt dagegen auf seinem bisherigen Stand.

  Beispiel:

  - Die Workingcopy ist auf A
  - `git merge B` führt A und B zusammen: B wird **in** A gemergt
  - Wichtig: Der Merge-Commit (sofern nötig) findet hierbei in A statt!

  In der Abbildung ist A der `master` und B der `fix`.

- Nach dem Merge existieren beide Branches weiter (sofern sie nicht explizit gelöscht werden)
- Hier im Beispiel findet ein sogenannter "Fast forward" statt.

  "Fast forward" ist ein günstiger Spezialfall beim Merge: Beide Branches liegen in einer direkten Kette, d.h. der Zielbranch kann einfach "weitergeschaltet" werden. Ein Merge-Commit ist in diesem Fall nicht notwendig und wird auch nicht angelegt.

2.1.3.5 Feature weiter entwickeln

```
          D---F  wuppie
         /
A---B---C---E  master
```

1. `git switch wuppie`
2. Weitere Änderungen im Branch `wuppie` ...

`git switch <branchname>` holt den aktuellen Stand des jeweiligen Branches in die Workingcopy. Man kann also jederzeit in der Workingcopy die Branches wechseln und entsprechend weiterarbeiten.

Hinweis: Während der neue `git switch`-Befehl nur Branches umschalten kann, funktioniert `git checkout` sowohl mit Branchnamen und Dateinamen - damit kann man also auch eine andere Version einer Datei in der Workingcopy "auschecken". Falls gleiche Branch- und Dateinamen existieren, muss man für das Auschecken einer Datei noch "`--`" nutzen: `git checkout -- <dateiname>`.

2.1.3.6 Feature ist stabil: Integration in *master*

```
          D---F  wuppie                          D---F  wuppie
         /                    =>                /     \
A---B---C---E  master                  A---B---C---E---G  master
```

1. `git checkout master`
2. `git merge wuppie`
   => Kein *fast forward* möglich: Git sucht nach gemeinsamen Vorgänger

Hier im Beispiel ist der Standardfall beim Mergen dargestellt: Die beiden Branches liegen nicht in einer direkten Kette von Commits, d.h. hier wurde parallel weitergearbeitet.

Git sucht in diesem Fall nach dem gemeinsamen Vorgänger beider Branches und führt die jeweiligen Änderungen (Differenzen) seit diesem Vorgänger in einem Merge-Commit zusammen.

Im `master` entsteht ein neuer Commit, da kein *fast forward* beim Zusammenführen der Branches möglich!

Anmerkung: `git checkout wuppie; git merge master` würde den `master` in den `wuppie` mergen, d.h. der Merge-Commit wäre dann in `wuppie`.

Beachten Sie dabei die "Merge-Richtung":

- Die Workingcopy ist auf A
- `git merge B` führt A und B zusammen: B wird **in** A gemergt
- Wichtig: Der Merge-Commit (sofern nötig) findet hierbei in A statt!

In der Abbildung ist A der `master` und B der `wuppie`.

Achtung: Richtung beachten! `git checkout A; git merge B` führt beide Branches zusammen, genauer: führt die Änderungen von B in A ein, d.h. der entsprechende Merge-Commit ist in A!

2.1.3.7 Konflikte beim Mergen

(Parallele) Änderungen an selber Stelle => Merge-Konflikte

```
$ git merge wuppie
Auto-merging hero.java
CONFLICT (content): Merge conflict in hero.java
Automatic merge failed; fix conflicts and then commit the result.
```

Git fügt Konflikt-Marker in die Datei ein:

```
<<<<<<< HEAD:hero.java
public void getActiveAnimation() {
    return null;
=======
public Animation getActiveAnimation() {
    return this.idleAnimation;
>>>>>>> wuppie:hero.java
```

- Der Teil mit HEAD ist aus dem aktuellen Branch in der Workingcopy
- Der Teil aus dem zu mergenden Branch ist unter `wuppie` notiert
- Das `=======` trennt beide Bereiche

2.1.3.8 Merge-Konflikte auflösen

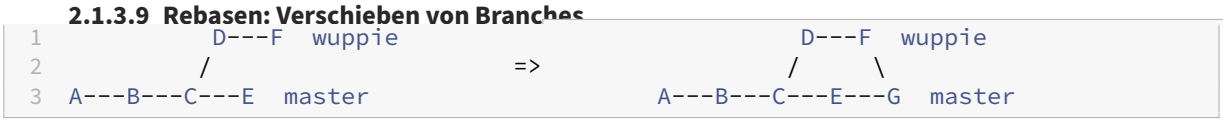
Manuelles Editieren nötig (Auflösung des Konflikts):

1) Entfernen der Marker
2) Hinzufügen der Datei zum Index
3) Analog für restliche Dateien mit Konflikt
4) Commit zum Abschließen des Merge-Vorgangs

Alternativ: Nutzung graphischer Oberflächen mittels `git mergetool`

Konsole: Branchen und Mergen

2.1.3.9 Rebasen: Verschieben von Branches



Bisher haben wir Branches durch Mergen zusammengeführt. Dabei entsteht in der Regel ein extra Merge-Commit (im Beispiel G), außer es handelt sich um ein *fast forward*. Außerdem erkennt man in der Historie sehr gut, dass hier in einem separaten Branch gearbeitet wurde, der irgendwann in den `master` gemergt wurde.

Leider wird dieses Vorgehen in großen Projekten recht schnell sehr unübersichtlich. Außerdem werden Merges in der Regeln nur von besonders berechtigten Personen (Manager) durchgeführt, die im Falle von Merge-Konflikten diese dann selbst auflösen müssten (ohne aber die fachliche Befähigung zu haben). Hier greift man dann häufig zur Alternative *Rebase*. Dabei wird der Ursprung eines Branches auf einen bestimmten Commit verschoben. Im Anschluss ist dann ein Merge mit *fast forward*, also ohne die typischen rautenförmigen Ketten in der Historie und ohne extra Merge-Commit möglich. Dies kann aber auch als Nachteil gesehen werden, da man in der Historie den früheren Branch nicht mehr erkennt! Ein weiterer schwerwiegender Nachteil ist, dass alle Commits im verschobenen Branch umgeschrieben werden und damit neue Commit-IDs bekommen. Das verursacht bei der Zusammenarbeit in Projekten massive Probleme! Als Vorteil gilt, dass man mögliche Merge-Konflikte bereits beim Rebasen auflösen muss, d.h. hier muss derjenige, der den Merge "beantragt", durch einen vorherigen Rebase den konfliktfreien Merge sicherstellen. Mehr dazu in "Branching-Strategien" und "Workflows".

```
1 git rebase master wuppie
```

führt zu

```
1               D'---F'  wuppie
2              /
3 A---B---C---E  master
```

Nach dem Rebase von `wuppie` auf `master` sieht es so aus, als ob der Branch `wuppie` eben erst vom `master` abgezweigt wurde. Damit ist dann ein *fast forward* Merge von `wuppie` in den `master` möglich, d.h. es gibt keine Raute und auch keinen extra Merge-Commit (hier nicht gezeigt).

Man beachte aber die Änderung der Commit-IDs von `wuppie`: Aus `D` wird `D'`! (Datum, Ersteller und Message bleiben aber erhalten.)

2.1.3.10 Don't lose your HEAD

- Branches sind wie Zeiger auf letzten Stand (Commit) eines Zweiges
- `HEAD`: Spezieller Pointer
  - Zeigt auf den aktuellen Branch der Workingcopy
- Früheren Commit auschecken (ohne Branch): "headless state"
  - Workingcopy ist auf früherem Commit
  - Kein Branch => Änderungen gehen verloren!

    Eventuelle Änderungen würden ganz normal als Commits auf dem `HEAD`-Branch aufgezeichnet. Sobald man aber einen anderen Branch auscheckt, wird der `HEAD` auf diesen anderen Branch gesetzt, so dass die eben gemachten Commits "in der Luft hängen". Sofern man die SHA's kennt, kommt man noch auf die Commits zurück. Allerdings laufen von Zeit zu Zeit interne Aufräum-Aktionen, so dass die Chance gut steht, dass die "kopflosen" Commits irgendwann tatsächlich verschwinden.

2.1.3.11 Wrap-Up

- Anlegen von Branches mit `git branch`
- Umschalten der Workingcopy auf anderen Branch: `git checkout` oder `git switch`
- Mergen von Branches und Auflösen von Konflikten: `git merge`
- Verschieben von Branches mit `git rebase`

Zum Nachlesen

- Chacon und Straub (2014, Kap. 3)
- Atlassian Pty Ltd (2022)
- Github Inc. (2022)

Lernziele

- k3: Erzeugen von Branches
- k3: Mergen von Branches, Auflösen möglicher Konflikte
- k3: Rebasen von Branches

Quizzes

- Quiz Git Branches (ILIAS)
- Tutorial: Welcome to Learn Git Branching
- Tutorial: A Visual Git Reference

Challenges
Branches und Merges

1. Legen Sie in Ihrem Projekt einen Branch an. Ändern Sie einige Dateien und committen Sie die Änderungen. Checken Sie den Master-Branch aus und mergen Sie die Änderungen. Was beobachten Sie?
2. Legen Sie einen weiteren Branch an. Ändern Sie einige Dateien und committen Sie die Änderungen. Checken Sie den Master-Branch aus und ändern Sie dort ebenfalls:
   - Ändern Sie eine Datei an einer Stelle, die nicht bereits im Branch modifiziert wurde.
   - Ändern Sie eine Datei an einer Stelle, die bereits im Branch manipuliert wurde.

   Committen Sie die Änderungen.

   Mergen Sie den Branch jetzt in den Master-Branch. Was beobachten Sie? Wie lösen Sie Konflikte auf?

Interaktive Git-Tutorials: Schaffen Sie die Rätsel?

- Learn Git Branching
- Oh My Git!
- Git Time

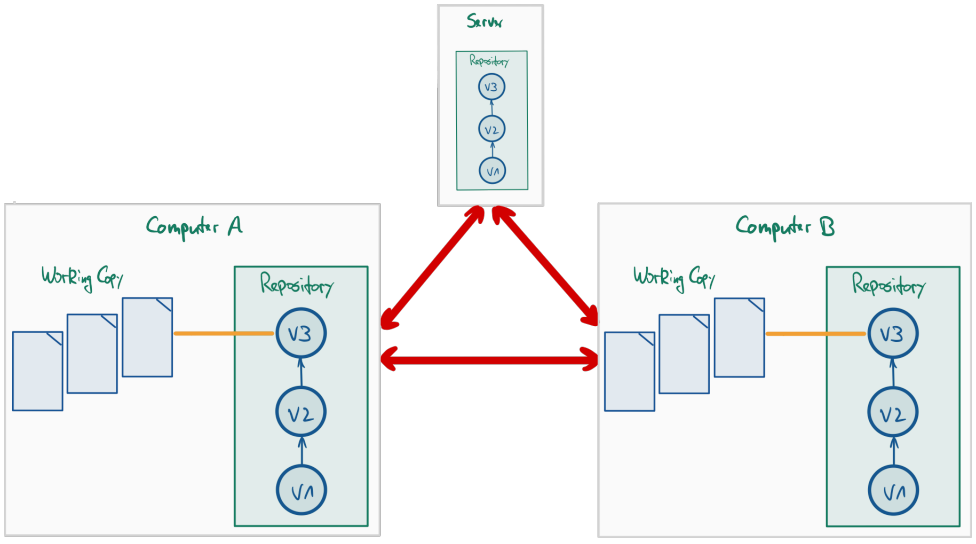
2.1.4 Branching-Strategien mit Git

TL;DR
Das Erstellen und Mergen von Branches ist in Git besonders einfach. Dies kann man sich in der Entwicklung zunutze machen und die einzelnen Features unabhängig voneinander in eigenen Hilfs-Branches ausarbeiten.
Es haben sich zwei grundlegende Modelle etabliert: “Git-Flow” und “GitHub Flow”.
In **Git-Flow** gibt es ein umfangreiches Konzept mit verschiedenen Branches für feste Aufgaben, welches sich besonders gut für Entwicklungmodelle mit festen Releases eignet. Es gibt zwei langlaufende Branches: `master` enthält den stabilen veröffentlichten Stand, in `develop` werden die Ergebnisse der Entwicklung gesammelt. Features werden in kleinen Feature-Branches entwickelt, die von `develop` abzweigen und dort wieder hineinmünden. Für Releases wird von `develop` ein eigener Release-Branch angelegt und nach Finalisierung in den `master` und in `develop` gemergt. Fixes werden vom `master` abgezweigt, und wieder in den `master` und auch nach `develop` integriert. Dadurch stehen auf dem `master` immer die stabilen Release-Stände zur Verfügung, und im `develop` sammeln sich die Entwicklungsergebnisse.
Der **GitHub Flow** basiert auf einem deutlich schlankeren Konzept und passt gut für die kontinuierliche Entwicklung ohne echte Releases. Hier hat man auch wieder einen `master` als langlaufenden Branch, der die stabilen Release-Stände enthält. Vom `master` zweigen direkt die kleinen Feature-Branches ab und werden auch wieder direkt in den `master` integriert.

Videos

- VL Git Branching-Strategien

2.1.4.1 Nutzung von Git in Projekten: Verteiltes Git (und Workflows)



Git ermöglicht ein einfaches und schnelles Branchen. Dies kann man mit entsprechenden Branching-Strategien sinnvoll für die SW-Entwicklung einsetzen.

Im Folgenden sollen also die Frage betrachtet werden: **Wie setze ich Branches sinnvoll ein?**

2.1.4.2 Umgang mit Branches: Themen-Branches

```
                 I---J---K  wuppieV1
                /
          D---F  wuppie
         /
A---B---C---E  master
             \
              G---H  test
```

Branchen ist in Git sehr einfach und schnell. Deshalb wird (gerade auch im Vergleich mit SVN) gern und viel gebrancht.

Ein häufiges anzutreffendes Modell ist dabei die Nutzung von **Themen-Branches**: Man hat einen Hauptzweig (`master`). Wann immer eine neue Idee oder ein Baustein unabhängig entwickelt werden soll/kann, wird ein entsprechender Themen-Branch aufgemacht. Dabei handelt es sich normalerweise um **kleine Einheiten**!

Themenbranches haben in der Regel eine **kurze Lebensdauer**: Wenn die Entwicklung abgeschlossen ist, wird die Idee bzw. der Baustein in den Hauptzweig integriert und der Themenbranch gelöscht.

- Vorteil: Die Entwicklung im Themenbranch ist in sich gekapselt und stört nicht die Entwicklung in anderen Branches (und diese stören umgekehrt nicht die Entwicklung im Themenbranch).
- Nachteil:
  - Mangelnder Überblick durch viele Branches
  - Ursprung der Themenbranches muss überlegt gewählt werden, d.h. alle dort benötigten Features müssen zu dem Zeitpunkt im Hauptzweig vorhanden sein

2.1.4.3 Umgang mit Branches: Langlaufende Branches

```
A---B---D  master
     \
      C---E---I  develop
           \
            F---G---H  topic
```

Häufig findet man in (größeren) Projekten Branches, die über die gesamte Lebensdauer des Projekts existieren, sogenannte “langlaufende Branches”.

Normalerweise gibt es einen Branch, in dem stets der stabile Stand des Projekts enthalten ist. Dies ist häufig der `master`. In diesem Branch gibt es nur sehr wenige Commits: normalerweise nur Merges aus dem `develop`-Branch (etwa bei Fertigstellung einer Release-Version) und ggf. Fehlerbehebungen.

Die aktive Entwicklung findet in einem separaten Branch statt: `develop`. Hier nutzt man zusätzlich Themen-Branches für die Entwicklung einzelner Features, die nach Fertigstellung in den `develop` gemergt werden.

Kleinere Projekte kommen meist mit den zwei langlaufenden Branches in der obigen Darstellung aus. Bei größeren Projekten finden sich häufig noch etliche weitere langlaufende Branches, beispielsweise “Proposed Updates” etc. beim Linux-Kernel.

- Vorteile:
  - Mehr Struktur im Projekt durch in ihrer Semantik wohldefinierte Branches

  - Durch weniger Commits pro Branch lässt sich die Historie leichter verfolgen (u.a. auch aus bestimmter Rollen-Perspektive: Entwickler, Manager, …)
- Nachteile: Bestimmte “ausgezeichnete” Branches; zusätzliche Regeln zum Umgang mit diesen beachten

2.1.4.4 Komplexe Branching-Strategie: Git-Flow



Das Git-Flow-Modell von Vincent Driessen (nvie.com/posts/a-successful-git-branching-model) zeigt einen in der Praxis überaus bewährten Umgang mit Branches. Lesen Sie an der angegebenen Stelle nach, besser kann man die Nutzung dieses eleganten Modells eigentlich nicht erklären :-)

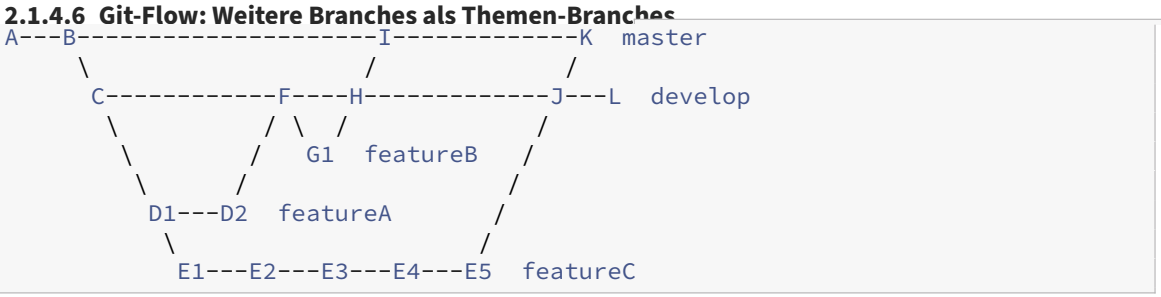
2.1.4.5 Git-Flow: Hauptzweige *master* und *develop*



Bei Git-Flow gibt es zwei langlaufende Branches: Den `master`, der immer den stabilen Stand enthält und in den *nie* ein direkter Commit gemacht wird, sowie den `develop`, wo letztlich (ggf. über Themenbranches) die eigentliche Entwicklung stattfindet.

Änderungen werden zunächst im `develop` erstellt und getestet. Wenn die Features stabil sind, erfolgt ein Merge von `develop` in den `master`. Hier kann noch der Umweg über einen `release`-Branch genommen werden: Als “Feature-Freeze” wird vom `develop` ein `release`-Branch abgezweigt. Darin wird das Release dann aufpoliert, d.h. es erfolgen nur noch kleinere Korrekturen und Änderungen, aber keine echte Entwicklungsarbeit mehr. Nach Fertigstellung wird der `release` dann sowohl in den `master` als auch `develop` gemergt.

2.1.4.6 Git-Flow: Weitere Branches als Themen-Branches



Für die Entwicklung eigenständiger Features bietet es sich auch im Git-Flow an, vom `develop` entsprechende Themenbranches abzuzweigen und darin jeweils isoliert die Features zu entwickeln. Wenn diese Arbeiten eine gewisse Reife haben, werden die Featurebranches in den `develop` integriert.

2.1.4.7 Git-Flow: Merging-Detail

```
---C--------E  develop
    \      /                git merge --no-ff
     D1---D2  featureA
```

vs.

```
---C---D1---D2  develop      git merge
```

Wenn beim Mergen ein "*fast forward*" möglich ist, würde Git beim Mergen eines (Feature-) Branches in den `develop` (oder allgemein in einen anderen Branch) *keinen* separaten Commit erzeugen (Situation rechts in der Abbildung).

Damit erscheint der `develop`-Branch wie eine lineare Folge von Commits. In manchen Projekten wird dies bevorzugt, weil die Historie sehr übersichtlich aussieht.

Allerdings verliert man die Information, dass hier ein Feature entwickelt wurde und wann es in den `develop` integriert wurde (linke Seite in obiger Abbildung). Häufig wird deshalb ein extra Merge-Commit mit `git merge --no-ff <branch>` (extra Schalter "`--no-ff`") erzwungen, obwohl ein "*fast forward*" möglich wäre.

Anmerkung: Man kann natürlich auch über Konventionen in den Commit-Kommentaren eine gewisse Übersichtlichkeit erzwingen. Beispielsweise könnte man vereinbaren, dass alle Commit-Kommentare zu einem Feature "A" mit "`feature a:`" starten müssen.

2.1.4.8 Git-Flow: Umgang mit Fehlerbehebung

```
A---B---D--------F1  master
     \   \      /
      \   E1---E2  fix
       \         \
        C1-------F2  develop
```

Wenn im stabilen Branch (also dem `master`) ein Problem bekannt wird, darf man es nicht einfach im `master` fixen. Stattdessen wird ein extra Branch vom `master` abgezweigt, in dem der Fix entwickelt wird. Nach Fertigstellung wird dieser Branch sowohl in den `master` als auch den `develop` gemergt, damit auch im Entwicklungszweig der Fehler behoben ist.

Dadurch entspricht jeder Commit im `master` einem Release.

2.1.4.9 Vereinfachte Braching-Strategie: GitHub Flow

```
A---B---C----D----------E  master
     \   \  /          /
      \   ta1  topicA  /
       \             /
        tb1---tb2---tb3  topicB
```

Github verfolgt eine deutlich vereinfachte Strategie: "GitHub Flow" (vgl. "GitHub Flow" (S. Chacon) bzw. "GitHub flow" (GitHub, Inc.)).

Hier ist der stabile Stand ebenfalls immer im `master`. Features werden ebenso wie im Git-Flow-Modell in eigenen Feature-Branches entwickelt.

Allerdings zweigen Feature-Branches *immer direkt* vom `master` ab und werden nach dem Test auch immer dort wieder direkt integriert (es gibt also keine weiteren langlaufenden Branches wie `develop` oder `release`).

In der obigen Abbildung ist zu sehen, dass für die Entwicklung eines Features ein entsprechender Themenbranch vom `master` abgezweigt wird. Darin erfolgt dann die Entwicklung des Features, d.h. mehrere Commits. Das Mergen des Features in den `master` erfolgt dann aber nicht lokal, sondern mit einem "Pull-Request" auf dem Server: Sobald man im Feature-Branch einen "diskussionswürdigen" Stand hat, wird ein **Pull-Request** (*PR*) über die Weboberfläche aufgemacht (streng genommen gehört dies in die Kategorie "Zusammenarbeit" bzw. "Workflows"; außerdem gehört ein PR nicht zu Git selbst, sondern zum Tooling von Github). In einem PR können andere Entwickler den Code kommentieren und ergänzen. Jeder weitere Commit auf dem Themenbranch wird ebenfalls Bestandteil des Pull-Requests. Parallel laufen ggf. automatisierte Tests etc. und durch das Akzeptieren des PR in der Weboberfläche erfolgt schließlich der Merge des Feature-Branches in den `master`.

2.1.4.10 Diskussion: Git-Flow vs. GitHub Flow

In der Praxis zeigt sich, dass das Git-Flow-Modell besonders gut geeignet ist, wenn man tatsächlich so etwas wie "Releases" hat, die zudem nicht zu häufig auftreten.

Das GitHub-Flow-Vorgehen bietet sich an, wenn man entweder keine Releases hat oder diese sehr häufig erfolgen (typisch bei agiler Vorgehensweise). Zudem vermeidet man so, dass die Feature-Branches zu lange laufen, womit normalerweise die Wahrscheinlichkeit von Merge-Konflikten stark steigt. **Achtung**: Da die Feature-Branches direkt in den `master`, also den stabilen Produktionscode gemergt werden, ist es hier besonders wichtig, *vor* dem Merge entsprechende Tests durchzuführen und den Merge erst zu machen, wenn alle Tests "grün" sind.

Hier ein paar Einstiegsseiten für die Diskussion, die teilweise sehr erbittert (und mit ideologischen Zügen) geführt wird (erinnert an die Diskussionen, welche Linux-Distribution die bessere sei):

- Git-Flow-Modell von Vincent Driessen
- Kurzer Überblick über das GitHub-Flow-Modell
- Diskussion des GitHub-Flow-Modells (Github)
- Luca Mezzalira: "Git-Flow vs Github Flow"
- Scott Schacon, Autor des Pro-Git-Buchs
- Noch eine (längere) Betrachtung (Robin Daugherty)

2.1.4.11 Wrap-Up

- Einsatz von Themenbranches für die Entwicklung
- Unterschiedliche Modelle:
  - Git-Flow: umfangreiches Konzept, gut für Entwicklung mit festen Releases
  - GitHub Flow: deutlich schlankeres Konzept, passend für kontinuierliche Entwicklung ohne echte Releases

Zum Nachlesen

- Chacon und Straub (2014, Kap. 3)
- Driessen (2010)
- Chacon (2013)
- GitHub Inc. (2022b)
- Atlassian Pty Ltd (2022)
- Github Inc. (2022)

Lernziele

- k3: Einsatz von Themenbranches in der Entwicklung
- k3: Git-Flow-Modell anwenden
- k3: GitHub Flow-Modell anwenden

Quizzes

- Quiz Git Branching-Strategien (ILIAS)

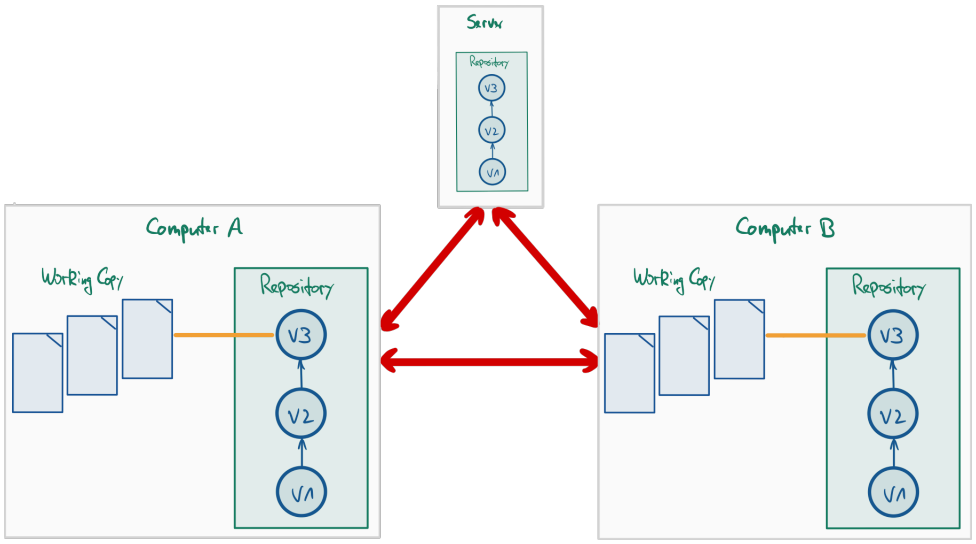
2.1.5 Arbeiten mit Git Remotes (dezentrale Repos)

TL;DR
Eine der Stärken von Git ist das Arbeiten mit verteilten Repositories. Zu jeder Workingcopy gehört eine Kopie des Repositories, wodurch jederzeit alle Informationen einsehbar sind und auch offline gearbeitet werden kann. Allerdings muss man für die Zusammenarbeit mit anderen Entwicklern die lokalen Repos mit den "entfernten" Repos (auf dem Server oder anderen Entwicklungsrechnern) synchronisieren.
Beim Klonen eines Repositories mit `git clone <url>` wird das fremde Repo mit dem Namen `origin` im lokalen Repo festgehalten. Dieser Name wird auch als Präfix für die Branches in diesem Repo genutzt, d.h. die Branches im Remote-Repo tauchen als `origin`/`<branch>` im lokalen Repo auf. Diese Remote-Branches kann man nicht direkt bearbeiten, sondern man muss diese Remote-Branches in einem lokalen Branch auschecken und dann darin weiterarbeiten. Es können beliebig viele weitere Remotes dem eigenen Repository hinzugefügt werden.
Änderungen aus einem Remote-Repo können mit `git fetch <remote>` in das lokale Repo geholt werden. Dies aktualisiert **nur** die Remote-Branches `<remote>`/`<branch>`! Die Änderungen können anschließend mit `git merge <remote>`/`<branch>` in den aktuell in der Workingcopy ausgecheckten Branch gemergt werden. (*Anmerkung*: Wenn mehrere Personen an einem Branch arbeiten, will man die eigenen Arbeiten in dem Branch vermutlich eher auf den aktuellen Stand des Remote **rebasen** statt mergen!) Eigene Änderungen können mit `git push <remote> <branch>` in das Remote-Repo geschoben werden.
Um den Umgang mit den Remote-Branches und den davon abgeleiteten lokalen Branches zu vereinfachen, gibt es das Konzept der "Tracking Branches". Dabei "folgt" ein lokaler Branch einem Remote-Branch. Ein einfaches `git pull` oder `git push` holt dann Änderungen aus dem Remote-Branch in den ausgecheckten lokalen Branch bzw. schiebt Änderungen im lokalen Branch in den Remote-Branch.

Videos

- VL Git Remotes
- Demo Fetch, Pull und Push
- Demo Tracking-Branches
- Demo Verknüpfen weiterer Remotes

2.1.5.1 Nutzung von Git in Projekten: Verteiltes Git (und Workflows)



Git ermöglicht eine einfaches Zusammenarbeit in verteilten Teams. Nachdem wir die verschiedenen Branching-Strategien betrachtet haben, soll im Folgenden die Frage betrachtet werden: **Wie arbeite ich sinnvoll über Git mit anderen Kollegen und Teams zusammen? Welche Modelle haben sich etabliert?**

2.1.5.2 Clonen kann sich lohnen

```
https://github.com/Programmiermethoden-CampusMinden/PM-Lecture

---C---D---E  master
```

=> `git clone https://github.com/Programmiermethoden-CampusMinden/PM-Lecture`

```
./PM-Lecture/  (lokaler Rechner)

---C---D---E  master
           ^origin/master
```

Git-Repository mit der URL <URL-Repo> in lokalen Ordner <directory> auschecken:

- `git clone <URL-Repo> [<directory>]`
- Workingcopy ist automatisch über den Namen `origin` mit dem remote Repo auf dem Server verbunden
- Lokaler Branch `master` ist mit dem remote Branch `origin/master` verbunden (“Tracking Branch”, s.u.), der den Stand des `master`-Branches auf dem Server spiegelt

Für die URL sind verschiedene Protokolle möglich, beispielsweise:

- “`file://`” für über das Dateisystem erreichbare Repositories (ohne Server)
- “`https://`” für Repo auf einem Server: Authentifikation mit Username und Passwort (!)
- “`git@`” für Repo auf einem Server: Authentifikation mit **SSH-Key** (diese Variante wird im Praktikum im Zusammenspiel mit dem Gitlab-Server im SW-Labor verwendet)

2.1.5.3 Eigener und entfernter *master* entwickeln sich weiter

```
https://github.com/Programmiermethoden-CampusMinden/PM-Lecture

---C---D---E---F---G  master
```

```
./PM-Lecture/  (lokaler Rechner)

---C---D---E---H  master
           ^origin/master
```

Nach dem Auschecken liegen (in diesem Beispiel) drei `master`-Branches vor:

1. Der `master` auf dem Server,
2. der lokale `master`, und
3. die lokale Referenz auf den `master`-Branch auf dem Server: `origin/master`.

Der lokale `master` ist ein normaler Branch und kann durch Commits verändert werden.

Der `master` auf dem Server kann sich ebenfalls ändern, beispielsweise weil jemand anderes seine lokalen Änderungen mit dem Server abgeglichen hat (`git push`, s.u.).

Der Branch `origin/master` lässt sich nicht direkt verändern! Das ist lediglich eine lokale Referenz auf den `master`-Branch auf dem Server und zeigt an, welchen Stand man bei der letzten Synchronisierung hatte. D.h. erst mit dem nächsten Abgleich wird sich dieser Branch ändern (sofern sich der entsprechende Branch auf dem Server verändert hat).

Anmerkung: Dies gilt analog für alle anderen Branches. Allerdings wird nur der `origin/master` beim Clonen automatisch als lokaler Branch ausgecheckt.

Zur Abbildung: Während man lokal arbeitet (Commit `H` auf dem lokalen `master`), kann es passieren, dass sich auch das remote Repo ändert. Im Beispiel wurden dort die beiden Commits `F` und `G` angelegt (durch `git push`, s.u.).

Wichtig: Da in der Zwischenzeit das lokale Repo nicht mit dem Server abgeglichen wurde, zeigt der remote Branch `origin/master` immer noch auf den Commit `E`!

2.1.5.4 Änderungen im Remote holen und Branches zusammenführen

```
https://github.com/Programmiermethoden-CampusMinden/PM-Lecture

---C---D---E---F---G  master
```

=> `git fetch origin`

```
./PM-Lecture/  (lokaler Rechner)

---C---D---E---H  master
            \
             F---G  origin/master
```

2.1.5.4.1 Änderungen auf dem Server mit dem eigenen Repo abgleichen Mit `git fetch origin` alle Änderungen holen

- Alle remote Branches werden aktualisiert und entsprechen den jeweiligen Branches auf dem Server: Im Beispiel zeigt jetzt `origin/master` ebenso wie der `master` auf dem Server auf den Commit `G`.
- Neue Branches auf dem Server werden ebenfalls "geholt", d.h. sie liegen nach dem Fetch als entsprechende remote Branches vor
- Auf dem Server gelöschte Branches werden nicht automatisch lokal gelöscht; dies kann man mit `git fetch --prune origin` automatisch erreichen

Wichtig: Es werden nur die remote Branches aktualisiert, nicht die lokalen Branches!

2.1.5.4.2 *master*-Branch nach "git fetch origin" zusammenführen

1. Mit `git checkout master` Workingcopy auf eigenen `master` umstellen
2. Mit `git merge origin/master` Änderungen am `origin/master` in eigenen `master` mergen
3. Mit `git push origin master` eigene Änderungen ins remote Repo pushen

```
https://github.com/Programmiermethoden-CampusMinden/PM-Lecture

---C---D---E---H---I  master
            \     /
             F---G
```

```
./PM-Lecture/  (lokaler Rechner)

---C---D---E---H---I  master
            \     /^origin/master
             F---G
```

Anmerkung: Schritt (2) kann man auch per `git pull origin master` erledigen … Ein `pull` fasst `fetch` und `merge` zusammen (s.u.).

Anmerkung Statt dem `merge` in Schritt (2) kann man auch den lokalen `master` auf den aktualisierten `origin/master` rebasen und vermeidet damit die "Raute". Der `pull` kann mit der Option "`--rebase`" auf "rebase" umgestellt werden (per Default wird bei `pull` ein "merge" ausgeführt).

2.1.5.4.3 Auf dem Server ist nur ein *fast forward merge* möglich Sie können Ihre Änderungen in Ihrem lokalen `master` auch direkt in das remote Repo pushen, solange auf dem Server ein **fast forward merge** möglich ist.

Wenn aber (wie in der Abbildung) der lokale und der remote `master` divergieren, müssen Sie den Merge wie beschrieben lokal durchführen (`fetch`/`merge` oder `pull`) und das Ergebnis wieder in das remote Repo pushen (dann ist ja wieder ein *fast forward merge* möglich, es sei denn, jemand hat den remote `master` in der Zwischenzeit weiter geschoben - dann muss die Aktualisierung erneut durchgeführt werden).

Beispiel für Zusammenführen (merge und push), Anmerkung zu fast forward merge

2.1.5.5 Branches und Remotes

- Eigenen (neuen) lokalen Branch ins remote Repo schicken
  - `git push <remote> <branch>`

- Neuer Branch im remote Repo
  - `git fetch <remote>` holt (auch) alle neuen Branches
  - Lokale Änderungen an remote Branches nicht möglich!
    => **Remote Branch in lokalen Branch mergen** (oder auschecken)

2.1.5.6 Zusammenfassung: Arbeiten mit Remotes

1. Änderungen vom Server holen: `git fetch <remote>`
   => Holt alle Änderungen vom Repo `<remote>` ins eigene Repo (Workingcopy bleibt unangetastet!)

2. Aktuellen lokalen Branch auffrischen: `git merge <remote>/<branch>`
   (oder alternativ `git pull <remote> <branch>`)

3. Eigene Änderungen hochladen: `git push <remote> <branch>`

2.1.5.6.1 Anmerkung: *push* geht nur, wenn

1. Ziel ein "bare"-Repository ist, **und**
2. keine Konflikte entstehen

=> im remote Repo nur "fast forward"-Merge möglich

=> bei Konflikten erst `fetch` und `merge`, danach `push`

Anmerkung: Ein "bare"-Repository enthält keine Workingcopy, sondern nur das Repo an sich. Die ist bei Repos, die Sie auf einem Server wie Gitlab oder Github anlegen, automatisch der Fall. Sie können aber auch lokal ein solches "bare"-Repo anlegen, indem Sie beim Initialisieren den Schalter `--bare` mitgeben: `git init --bare` ...

2.1.5.6.2 Beispiel

```
git fetch origin           # alle Änderungen vom Server holen
git checkout master        # auf lokalen Master umschalten
git merge origin/master    # lokalen Master aktualisieren

... # Herumspielen am lokalen Master

git push origin master     # lokalen Master auf Server schicken
```

2.1.5.7 Vereinfachung: Tracking Branches

- **Tracking Branch**: lokaler Branch, der remote Branch "verfolgt"
  - Beispiel: lokaler `master`-Branch folgt `origin/master` per Default

- **Vereinfachung im Workflow**:
  - `git pull` entspricht
    1. `git fetch <remote>` **plus**
    2. `git merge <remote>/<branch>`
  - `git push` entspricht `git push <remote> <branch>`

Vorsicht: `pull` und `push` beziehen sich nur auf ausgecheckten Tracking Branch

2.1.5.8 Einrichten von Tracking Branches

- `git clone`: lokaler `master` trackt automatisch `origin/master`

- Remote Branch als Tracking Branch einrichten:
  1. Änderungen aus remote Repo holen: `git fetch <remote>`
  2. Tracking Branch anlegen: `git checkout -t <remote>/<branch>` (=> Option `-t` richtet den remote Branch als Tracking Branch ein)

- Lokalen neuen Branch ins remote Repo schicken und als Tracking Branch einrichten:
  1. Lokalen Branch erzeugen: `git checkout -b <branch>`
  2. Lokalen Branch ins Repo schicken: `git push -u <remote> <branch>` (=> Option `-u` richtet den lokalen Branch als Tracking Branch ein)

2.1.5.9 Hinzufügen eines (weiteren) Remote Repository



Sie können einem Repo beliebig viele Remotes hinzufügen:

`git remote add <name> <url>`

Beispiel: `git remote add andi git@github.com:andi/repo.git`

- Remote `origin` wird bei `clone` automatisch angelegt
- Ansehen der Remotes mit `git remote -v`
- `fetch`, `push` und `pull` jeweils über den vergebenen Namen

Beispiel: `git fetch andi` oder `git push origin master`

2.1.5.10 Wrap-Up

- Synchronisierung des lokalen Repos mit anderen Repos
  - Repo kopieren: `git clone <url>`
  - Interner Name fürs fremde Repo: `origin`
  - Änderungen vom fremden Repo holen: `git fetch <remote>`
  - Änderungen in lokalen Branch einpflegen: `git merge <remote>/<branch>`
  - Eigene Änderungen ins fremde Repo schieben: `git push <remote> <branch>`

- Tracking Branches (Konzept, Anwendung)
  - Remote Branches können lokal nicht verändert werden:
    * In lokale Branches mergen, oder
    * Tracking Branches anlegen => einfaches `pull` und `push` nutzen
  - Tracking Branches sind lokale Branches, die remote Branches verfolgen (“tracken”)

Zum Nachlesen

- Chacon und Straub (2014, Kap. 3)
- Atlassian Pty Ltd (2022)
- Github Inc. (2022)

Lernziele

- k3: Erzeugen eines Clones von fremden Git-Repositories
- k3: Holen der Änderungen vom fremden Repo
- k3: Aktualisierung der lokalen Branches
- k3: Pushen der lokalen Änderungen ins fremde Repo
- k3: Anlegen von lokalen Branches vs. Anlegen von entfernten Branches
- k3: Anlegen eines Tracking Branches zum Vereinfachen der Arbeit

Quizzes

- Quiz Git Remotes (ILIAS)

Challenges
Interaktive Git-Tutorials: Schaffen Sie die Rätsel?

- Learn Git Branching
- Oh My Git!
- Git Time

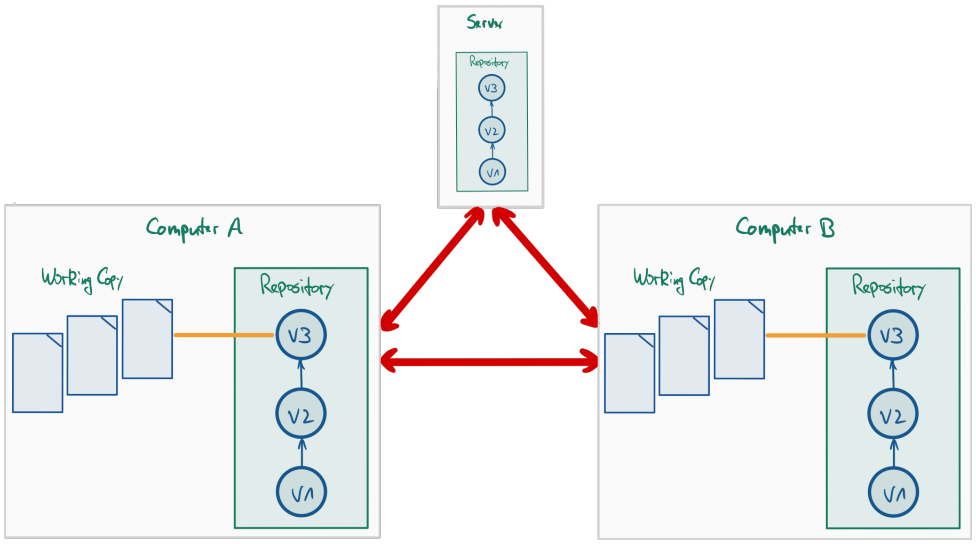
2.1.6 Zusammenarbeit: Git-Workflows und Merge-/Pull-Requests

TL;DR
Git erlaubt unterschiedliche Formen der Zusammenarbeit.
Bei kleinen Teams kann man einen einfachen zentralen Ansatz einsetzen. Dabei gibt es ein zentrales Repo auf dem Server, und alle Team-Mitglieder dürfen direkt in dieses Repo pushen. Hier muss man sich gut absprechen und ein vernünftiges Branching-Schema ist besonders wichtig.
In größeren Projekten gibt es oft ein zentrales öffentliches Repo, wo aber nur wenige Personen Schreibrechte haben. Hier forkt man sich dieses Repo, erstellt also eine öffentliche Kopie auf dem Server. Diese Kopie klont man lokal und arbeitet hier und pusht die Änderungen in den eigenen öffentlich sichtbaren Fork. Um die Änderungen ins Projekt-Repo integrieren zu lassen, wird auf dem Server ein sogenannter Merge-Request (Gitlab) bzw. Pull-Request (GitHub) erstellt. Dies erlaubt zusätzlich ein Review und eine Diskussion direkt am Code. Damit man die Änderungen im Hauptprojekt in den eigenen Fork bekommt, trägt man das Hauptprojekt als weiteres Remote in die Workingcopy ein und aktualisiert regelmäßig die Hauptbranches, von denen dann auch die eigenen Feature-Branches ausgehen sollten.

Videos

- VL Git Workflows
- Demo Anlegen eines Forks, Erstellen eines Pull-Requests (PR)
- Demo Arbeiten mit einem PR, Review eines PR

2.1.6.1 Nutzung von Git in Projekten: Verteiltes Git (und Workflows)

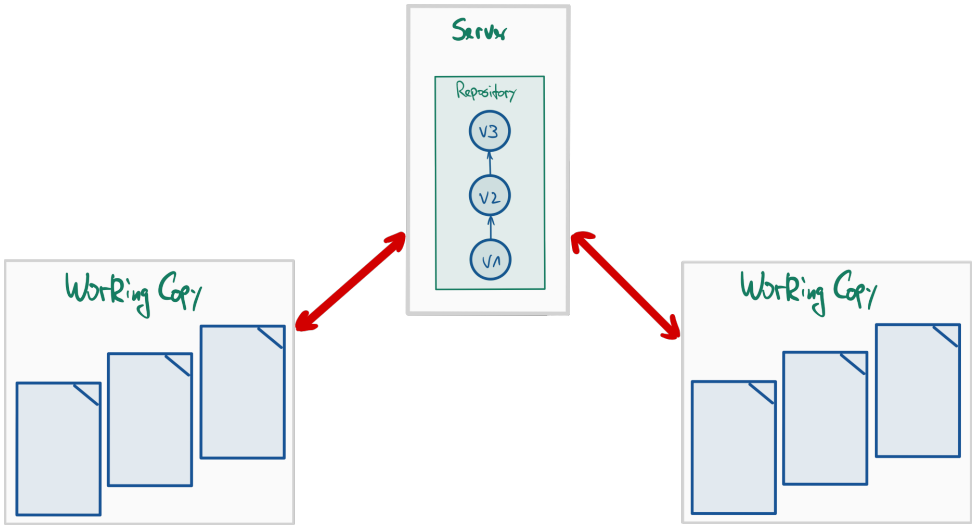


Git ermöglicht ein einfaches und schnelles Branchen. Dies kann man mit entsprechenden Branching-Strategien sinnvoll für die SW-Entwicklung einsetzen.

Auf der anderen Seite ermöglicht Git ein sehr einfaches verteiltes Arbeiten. Auch hier ergeben sich verschiedene Workflows, wie man mit anderen Entwicklern an einem Projekt arbeiten will/kann.

Im Folgenden sollen also die Frage betrachtet werden: **Wie gestalte ich die Zusammenarbeit?** Antwort: Workflows mit Git …

2.1.6.2 Zusammenarbeit: Zentraler Workflow mit Git (analog zu SVN)

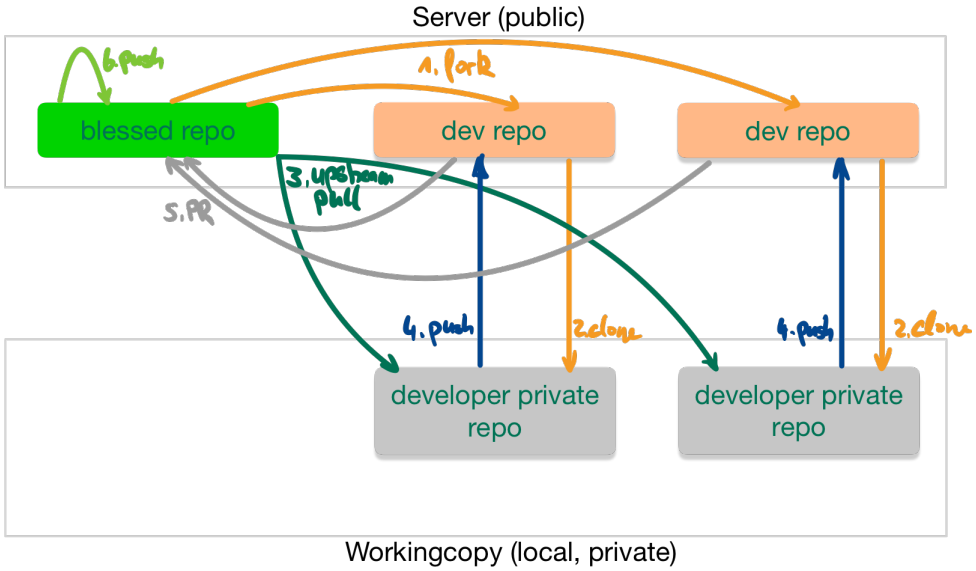


In kleinen Projektgruppen wie beispielsweise Ihrer Arbeitsgruppe wird häufig ein einfacher zentralisierter Workflow bei der Versionsverwaltung genutzt. Im Mittelpunkt steht dabei ein zentrales Repository, auf dem alle Teammitglieder gleichberechtigt und direkt pushen dürfen.

- Vorteile:
  - Einfachstes denkbares Modell
  - Ein gemeinsames Repo (wie bei SVN)
  - Alle haben Schreibzugriff auf ein gemeinsames Repo

- Nachteile:
  - Definition und Umsetzung von Rollen mit bestimmten Rechten (“Manager”, “Entwickler”, “Gast-Entwickler”, …) schwierig bis unmöglich (das ist kein Git-Thema, sondern hängt von der Unterstützung durch den Anbieter des Servers ab)
  - Jeder darf überall pushen: Enge und direkte Abstimmung nötig
  - Modell funktioniert meist nur in sehr kleinen Teams (2..3 Personen)

2.1.6.3 Zusammenarbeit: Einfacher verteilter Workflow mit Git



In großen und/oder öffentlichen Projekten wird üblicherweise ein Workflow eingesetzt, der auf den Möglichkeiten von verteilten Git-Repositories basiert.

Dabei wird zwischen verschiedenen Rollen (“Integrationsmanager”, “Entwickler”) unterschieden.

Sie finden dieses Vorgehen beispielsweise beim Linux-Kernel und auch häufig bei Projekten auf Github.

- Es existiert ein geschütztes (“blessed”) Master-Repo
  - Stellt die Referenz für das Projekt dar
  - Push-Zugriff nur für ausgewählte Personen (“Integrationsmanager”)

- Entwickler
  - Forken das Master-Repo auf dem Server und klonen ihren Fork lokal

  - Arbeiten auf lokalem Klon: Unabhängige Entwicklung eines Features
  - Pushen ihren Stand in ihren Fork (ihr eigenes öffentliches Repo): Veröffentlichung des Beitrags zum Projekt (sobald fertig bzw. diskutierbar)
  - Lösen Pull- bzw. Merge-Request gegen das Master-Repo aus: Beitrag soll geprüft und ins Projekt aufgenommen werden (Merge ins Master-Repo durch den Integrationsmanager)

- Integrationsmanager
  - Prüft die Änderungen im Pull- bzw. Merge-Request und fordert ggf. Nacharbeiten an bzw. lehnt Integration ab (technische oder politische Gründe)
  - Führt Merge der Entwickler-Zweige mit den Hauptzweigen durch Akzeptieren der Pull- bzw. Merge-Requests durch: Beitrag der Entwickler ist im Projekt angekommen und ist beim nächsten Pull in deren lokalen Repos vorhanden

Den hier gezeigten Zusammenhang kann man auf weitere Ebenen verteilen, vgl. den im Linux-Kernel gelebten "Dictator and Lieutenants Workflow" (siehe Literatur).

Hinweis: Hier wird nur die Zusammenarbeit im verteilten Team dargestellt. Dazu kommt noch das Arbeiten mit verschiedenen Branches!

Anmerkung: In der Workingcopy wird das eigene (öffentliche) Repo oft als `origin` und das geschützte ("blessed") Master-Repo als `upstream` referenziert.

2.1.6.3.1 Anmerkungen zum Forken

Sie könnten auch das Original-Repo direkt clonen. Allerdings würden dann die `push` dort aufschlagen, was in der Regel nicht erwünscht ist (und auch nicht erlaubt ist).

Deshalb forkt man das Original-Repo auf dem Server, d.h. auf dem Server wird eine Kopie des Original-Repos im eigenen Namespace angelegt. Auf diese Kopie hat man dann uneingeschränkten Zugriff.

2.1.6.3.2 Anmerkungen zu den Namen für die Remotes: `origin` und `upstream`

Üblicherweise checkt man die *Kopie* lokal aus (d.h. erzeugt einen Clone). In der Workingcopy verweist dann `origin` auf die Kopie. Um Änderungen am Original-Repo zu erhalten, fügt man dieses unter dem Namen `upstream` als weiteres Remote-Repo hinzu. Dies ist eine nützliche *Konvention*.

Um Änderungen aus dem Original-Repo in den eigenen Fork (und die Workingcopy) zu bringen, führt man dann einfach folgendes aus (im Beispiel für den `master`):

```
1 $ git checkout master          # Workingcopy auf master
2 $ git pull upstream master     # Aktualisiere lokalen master mit master aus
    Original-Repo
3 $ git push origin master       # Pushe lokalen master in den Fork
```

2.1.6.4 Feature-Branches aktualisieren: Mergen mit *master* vs. Rebase auf *master*

Im Netz finden sich häufig Anleitungen, wonach man Änderungen im `master` mit einem Merge in den Feature-Branch holt, also sinngemäß:

```
$ git checkout master          # Workingcopy auf master
$ git pull upstream master     # Aktualisiere lokalen master mit master aus
    Vorgabe-Repo
$ git checkout feature         # Workingcopy auf feature
$ git merge master             # Aktualisiere feature: Merge master in feature
$ git push origin feature      # Push aktuellen feature ins Team-Repo
```

Das funktioniert rein technisch betrachtet.

Allerdings spielt in den meisten Git-Projekten der `master` üblicherweise eine besondere Rolle (vgl. Branching-Strategien) und ist üblicherweise stets das **Ziel** eines Merge, aber nie die Quelle! D.h. per Konvention geht der Fluß von Änderungen stets **in** den `master` (und nicht heraus).

Wenn man sich nicht an diese Konvention hält, hat man später möglicherweise Probleme, die Merge-Historie zu verstehen (welche Änderung kam von woher)!

Um die Änderungen im `master` in einen Feature-Branch zu bekommen, sollte deshalb ein **Rebase** (des Feature-Branches auf den `master`) vor einem Merge (des `master` in den Feature-Branch) bevorzugt werden.

Merk-Regel: Merge niemals nie den `master` in Feature-Branches!

Achtung: Ein Rebase bei veröffentlichten Branches ist problematisch, da Dritte auf diesem Branch arbeiten könnten und entsprechend auf die Commit-IDs angewiesen sind. Nach einem Rebase stimmen diese Commit-IDs nicht mehr, was normalerweise mindestens zu Verärgerung führt … Die Dritten müssten ihre Arbeit dann auf den neuen Feature-Branch (d.h. den Feature-Branch nach dessen Rebase) rebasen … vgl. auch “The Perils of Rebasing” in Abschnitt “3.6 Rebasing” in (Chacon und Straub 2014).

2.1.6.4.1 Mögliches Szenario im Praktikum

Im Praktikum haben Sie das Vorgabe-Repo. Dieses könnten Sie als `upstream` in Ihre lokale Workingcopy einbinden.

Mit Ihrem Team leben Sie vermutlich einen zentralen Workflow, d.h. Sie binden Ihr gemeinsames Repo als `origin` in Ihre lokale Workingcopy ein.

Dann müssen Sie den lokalen `master` aus *beiden* Remotes aktualisieren. Zusätzlich wollen Sie Ihren aktuellen Themenbranch auf den aktuellen `master` rebasen.

```
$ git checkout master          # Workingcopy auf master
$ git pull upstream master     # Aktualisiere lokalen master mit master aus
    Vorgabe-Repo
$ git pull origin master       # Aktualisiere lokalen master mit master aus Team
    -Repo
$ git push origin master       # Pushe lokalen master in das Team-Repo zurück
$ git rebase master feature    # Rebase feature auf den aktuellen lokalen master
$ git push -f origin feature   # Push aktuellen feature ins Team-Repo ("-f" wg.
    geänderter IDs durch rebase)
```

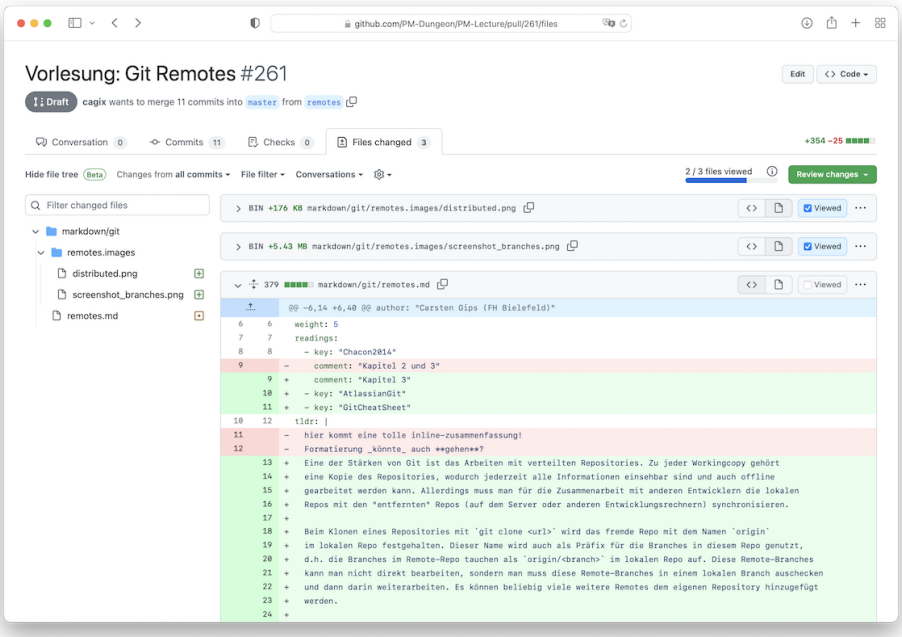
Anmerkung: Dabei können in Ihrem `master` die unschönen “Rauten” entstehen. Wenn Sie das vermeiden wollen, tauschen Sie den zweiten und den dritten Schritt und führen den Pull gegen den Upstream-`master` als `pull --rebase` durch. Dann müssen Sie Ihren lokalen `master` allerdings auch force-pushen in Ihr Team-Repo und die anderen Team-Mitglieder sollten darüber informiert werden, dass sich der `master` auf inkompatible Weise geändert hat …

2.1.6.5 Kommunikation: Merge- bzw. Pull-Requests Mergen kann man auf der Konsole (oder in der IDE) und anschließend die (neuen) Branches auf den Server pushen.

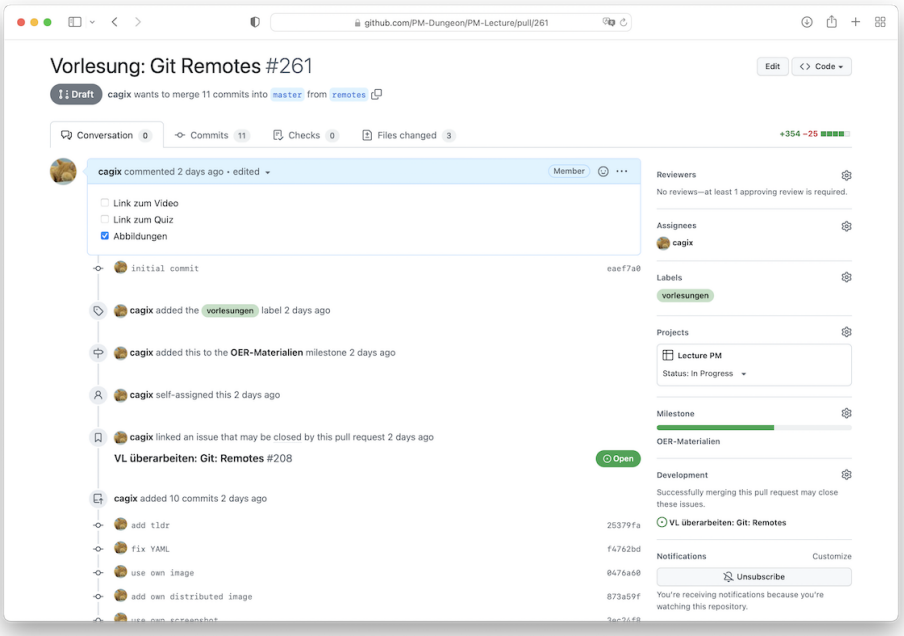
Die verschiedenen Git-Server erlauben ebenfalls ein GUI-gestütztes Mergen von Branches: “Merge-Requests” (*MR*, Gitlab) bzw. “Pull-Requests” (*PR*, Github). Das hat gegenüber dem lokalen Mergen wichtige Vorteile: Andere Entwickler sehen den beabsichtigten Merge (frühzeitig) und können direkt den Code kommentieren und die vorgeschlagenen Änderungen diskutieren, aber auch allgemeine Kommentare abgeben.

Falls möglich, sollte man einen MR/PR immer dem Entwickler zuweisen, der sich weiter um diesen MR/PR kümmern wird (also zunächst ist man das erstmal selbst). Zusätzlich kann man einen Reviewer bestimmen, d.h. wer soll sich den Code ansehen.

Hier ein Screenshot der Änderungsansicht unseres Gitlab-Servers (SW-Labor):



Nachfolgend für den selben MR aus der letzten Abbildung noch die reine Diskussionsansicht:



2.1.6.6 Best Practices bei Merge-/Pull-Requests

1. MR/PR so zeitig wie möglich aufmachen
   - Am besten sofort, wenn ein neuer Branch auf den Server gepusht wird!
   - Ggf. mit dem Präfix “WIP” im Titel gegen unbeabsichtigtes vorzeitiges Mergen sperren … (bei GitHub als “Draft”-PR öffnen)
2. Auswahl Start- und Ziel-Branch (und ggf. Ziel-Repo)
   - Es gibt verschiedene Stellen, um einen MR/PR zu erstellen. Manchmal kann man nur noch den Ziel-Branch einstellen, manchmal beides.
   - Bitte auch darauf achten, welches Ziel-Repo eingestellt ist! Bei Forks wird hier immer das Original-Repo voreingestellt!
   - Den Ziel-Branch kann man ggf. auch nachträglich durch Editieren des MR/PR anpassen (Start-Branch und Ziel-Repo leider nicht, also beim Erstellen aufpassen!).
3. Titel (*Summary*): Das ist das, was man in der Übersicht sieht!
   - Per Default wird die letzte Commit-Message eingesetzt.
   - Analog zur Commit-Message: Bitte hier unbedingt einen sinnvollen Titel einsetzen: Was macht der MR/PR (kurz)?
4. Beschreibung: Was passiert, wenn man diesen MR/PR akzeptiert (ausführlicher)?
   - Analog zur Commit-Message sollte hier bei Bedarf die Summary ausformuliert werden und beschreiben, was der MR/PR ändert.
5. Assignee: Wer soll sich drum kümmern?

- Ein MR/PR sollte immer jemanden zugewiesen sein, d.h. nicht “unassigned” sein. Ansonsten ist nicht klar, wer den Request durchführen/akzeptieren soll.
- Außerdem taucht ein nicht zugewiesener MR/PR nicht in der Übersicht “meiner” MR/PR auf, d.h. diese MR/PR haben die Tendenz, vergessen zu werden!

6. Diskussion am (und neben) dem Code
   - Nur die vorgeschlagenen Code-Änderungen diskutieren!
   - Weitergehende Diskussionen (etwa über Konzepte o.ä.) besser in separaten Issues erledigen, da sonst die Anzeige des MR/PR langsam wird (ist beispielsweise ein Problem bei Gitlab).
7. Weitere Commits auf dem zu mergenden Branch gehen automatisch mit in den Request
8. Weitere Entwickler kann man mit “`@username`” in einem Kommentar auf “CC” setzen und in die Diskussion einbinden

Anmerkung: Bei Gitlab (d.h. auch bei dem Gitlab-Server im SW-Labor) gibt es “*Merge-Requests*” (MR). Bei Github gibt es “*Pull-Requests*” (PR) …

2.1.6.7 Wrap-Up

- Git-Workflows für die Zusammenarbeit:
  - einfacher zentraler Ansatz für kleine Arbeitsgruppen vs.
  - einfacher verteilter Ansatz mit einem “blessed” Repo (häufig in Open-Source-Projekten zu finden)
- Aktualisieren Ihres Clones und Ihres Forks mit Änderungen aus dem “blessed” Repo
- Unterschied zwischen einem Pull/Merge und einem Pull/Rebase
- Erstellen von Beiträgen zu einem Projekt über Merge-Requests
  - Welche Commits werden Bestandteil eines Merge-Requests (und warum)
  - Diskussion über den Code in Merge-Requests

Zum Nachlesen

- Chacon und Straub (2014, 5, 4.8, 6)
- Atlassian Pty Ltd (2022)
- Github Inc. (2022)

Lernziele

- k2: Git-Workflows für die Zusammenarbeit
- k2: Unterschied zwischen einem Pull/Merge und einem Pull/Rebase
- k2: Welche Commits werden Bestandteil eines Merge-Requests (und warum)
- k3: Einsatz des zentralisierten Workflows
- k3: Einsatz des einfachen verteilten Workflows mit unterschiedlichen Repos
- k3: Aktualisieren Ihres Clones und Ihres Forks mit Änderungen aus dem blessed Repo
- k3: Erstellen von Beiträgen zu einem Projekt über Merge-Requests
- k3: Aktualisierung von Merge-Requests
- k3: Diskussion über den Code in Merge-Requests

Quizzes

- Quiz Git Workflows (ILIAS)

2.1.7 Git Worktree

TL;DR
Git Worktree erlaubt es, Branches in separaten Ordnern auszuchecken. Diese Ordner sind mit der Workingcopy verknüpft, d.h. alle Änderungen über Git-Befehle werden automatisch mit der Workingcopy "synchronisiert". Im Unterschied zum erneuten Clonen hat man in den verknüpften Ordnern aber nicht die gesamte Historie noch einmal neu als `.git`-Ordner, sondern nur den Link auf die Workingcopy, wodurch viel Platz gespart wird. Damit bilden Git Worktrees eine elegante Möglichkeit, parallel an verschiedenen Branches zu arbeiten.

Videos

- VL Git Worktree
- Demo Git Worktree

2.1.7.1 Git Worktree - Mehrere Branches parallel auschecken

2.1.7.1.1 Szenario

- Sie arbeiten an einem Projekt
- Großes Repo mit vielen Versionen und Branches
- Ungesicherte Änderungen im Featurebranch
- Wichtige Bugfixes an alter Version nötig

2.1.7.1.2 Lösungsansätze

- `git stash` nutzen und Branch wechseln
- Repo erneut in anderem Ordner auschecken

2.1.7.1.3 Probleme

1. `git stash` und `git switch`

   Funktioniert für die meisten Fälle relativ gut und ist daher die "Lösung to go".

   Aber Sie müssen später aufpassen, dass Sie auch wirklich wieder im richtigen Branch sind, wenn Sie die Änderungen im Stash anwenden (`git stash pop`)! Und wenn Sie mehrere Einträge in der Stash-Liste haben, kann es recht schnell recht unübersichtlich werden - zu welchem Branch gehören welche Einträge in der Stash-Liste?

Außerdem kann es gerade in größeren Projekten passieren, dass sich die Konfiguration zwischenzeitlich ändert. Wenn Sie jetzt in der IDE einfach auf einen alten Stand mit einer anderen Konfiguration wechseln, kann es schnell passieren, dass sich die IDE “verschluckt” und Sie dadurch viel Arbeit haben.

2. Nochmal woanders auschecken

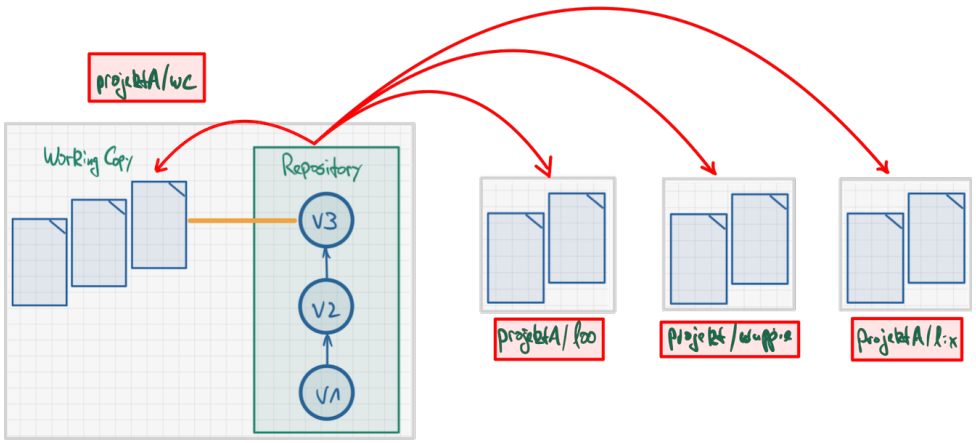
   Im Prinzip ist das eine Möglichkeit. Sie können dann den anderen Ordner in Ihrer IDE als neues Projekt öffnen und sofort starten.

   Aber: Sie benötigen noch einmal den Platz auf der Festplatte/SSD/… wie für die ursprüngliche Workingcopy! Das kann bei alten/großen Projekten schnell recht groß werden und Probleme verursachen.

   Außerdem ist die Synchronisierung zwischen den beiden Workingcopies (der ursprünglichen und der neuen) nicht vorhanden bzw. das müssen Sie manuell per `git push` und `git pull` (in jeder Kopie des Repos!) erledigen!

2.1.7.1.4 Git Worktree kann helfen! => Mehrere Branches gleichzeitig auschecken (als neue Ordner im Dateisystem)

2.1.7.2 How to use Git Worktree



2.1.7.3 Worktree anlegen

```
git worktree add <path> <branch>
```

Legt neuen Ordner <path> an und checkt darin <branch> als “linked worktree” aus.

Mit `git worktree add ../wuppie foo` würden Sie also parallel zum aktuellen Ordner (wo Ihre Workingcopy enthalten ist) einen neuen Ordner `wuppie/` anlegen und darin den Branch `foo` auschecken.

Wenn Sie in den Ordner `wuppie` wechseln, finden Sie auch eine *Datei* `.git`. Darin ist lediglich der Pfad zur Workingcopy vermerkt, damit Git Änderungen auch in die eigentliche Workingcopy spiegeln kann. Dies ist der sogenannte "linked worktree".

Im Vergleich dazu finden Sie in der eigentlichen Workingcopy einen *Ordner* `.git`, der üblicherweise die gesamte Historie etc. enthält und entsprechend groß werden kann.

Den Befehl `git worktree add` gibt es in verschiedenen Versionen. In der Kurzform `git worktree add <path>` würde ein neuer Branch angelegt und ausgecheckt, der der letzten Komponente von `<path>` entspricht …

Warnung: Nicht in selben Ordner oder in Unterordner auschecken!

Die neuen Worktrees sollten immer **außerhalb** der Workingcopy liegen! Sie können Git sehr schnell sehr gründlich durcheinanderbringen, wenn Sie einen Worktree im selben Ordner oder in einem Unterordner anlegen.

`git worktree` sollte nach Möglichkeit nicht zusammen mit Git Submodules eingesetzt werden (unstabiles Verhalten)!

2.1.7.4 Worktree wechseln

- Worktrees anzeigen: `git worktree list`
- Worktree wechseln: Ordner wechseln (IDE: neues Projekt)

Die Worktrees sind aus Sicht des Dateisystems einfach Ordner. Die `.git`-Datei verlinkt für Git den Ordner mit der ursprünglichen Workingcopy.

Um also mit einem Worktree arbeiten zu können, wechseln Sie einfach das Verzeichnis. In einer IDE würden Sie entsprechend ein neues Projekt anlegen. So können Sie gleichzeitig in verschiedenen Branches arbeiten.

Änderungen in einem Worktree werden automatisch in die ursprüngliche Workingcopy gespiegelt. Analog können Sie in einem Worktree auf die aktuelle Historie aus der ursprünglichen Workingcopy zugreifen.

Hinweis: Sie können in den Ordnern zwar Branches wechseln, aber nicht auf einen Branch, der bereits in einem anderen Ordner (Worktree) ausgecheckt ist. Es ist gute Praxis, dass die Ordnernamen dem ausgecheckten Branch (linked Worktree) entsprechen, um Verwirrungen zu vermeiden.

2.1.7.5 Worktree löschen

```
git worktree remove <worktree>
```

Sofern der Worktree "clean" ist, es also keine nicht comitteten Änderungen gibt, können Sie mit `git worktree remove <worktree>` einen Worktree `<worktree>` wieder löschen.

Dabei bleibt der Ordner erhalten - Sie können ihn selbst löschen oder später wiederverwenden.

2.1.7.6 Wrap-Up Git Worktree: Auschecken von Branches in separate Ordner

- Anlegen: `git worktree add <path> <branch>`
- Anschauen: `git worktree list`
- Löschen: `git worktree remove <worktree>`

- Dokumentation: https://git-scm.com/docs/git-worktree

Lernziele

- k2: Vorteile von Git Worktree
- k2: Prinzipielle Arbeitsweise von Git Worktree
- k3: Anlegen von Worktrees
- k3: Anzeigen von Worktrees
- k3: Löschen von Worktrees

2.2 Modern Java: Funktionaler Stil und Stream-API

2.2.1 Lambda-Ausdrücke und funktionale Interfaces

TL;DR
Mit einer anonymen inneren Klasse erstellt man gewissermaßen ein Objekt einer "Wegwerf"-Klasse: Man leitet *on-the-fly* von einem Interface ab oder erweitert eine Klasse und implementiert die benötigten Methoden und erzeugt von dieser Klasse sofort eine Instanz (Objekt). Diese neue Klasse ist im restlichen Code nicht sichtbar.
Anonyme innere Klassen sind beispielsweise in Swing recht nützlich, wenn man einer Komponente einen Listener mitgeben will: Hier erzeugt man eine anonyme innere Klasse basierend auf dem passenden Listener-Interface, implementiert die entsprechenden Methoden und übergibt das mit dieser Klasse erzeugte Objekt als neuen Listener der Swing-Komponente.
Mit Java 8 können unter gewissen Bedingungen diese anonymen inneren Klassen zu Lambda-Ausdrücken (und Methoden-Referenzen) vereinfacht werden. Dazu muss die anonyme innere Klasse ein sogenanntes **funktionales Interface** implementieren.
Funktionale Interfaces sind Interfaces mit *genau einer abstrakten Methode*. Es können beliebig viele Default-Methoden im Interface enthalten sein, und es können `public` sichtbare abstrakte Methoden von `java.lang.Object` geerbt/überschrieben werden.
Die Lambda-Ausdrücke entsprechen einer anonymen Methode: Die Parameter werden aufgelistet (in Klammern), und hinter einem Pfeil kommt entweder *ein* Ausdruck (Wert - gleichzeitig Rückgabewert des Lambda-Ausdrucks) oder beliebig viele Anweisungen (in geschweiften Klammern, mit Semikolon):

- Form 1: `(parameters)-> expression`
- Form 2: `(parameters)-> { statements; }`

Der Lambda-Ausdruck muss von der Signatur her genau der einen abstrakten Methode im unterliegenden funktionalen Interface entsprechen.

Videos

- VL Lambda-Ausdrücke und funktionale Interfaces
- Demo Anonyme innere Klasse
- Demo Lambda-Ausdruck
- Demo Funktionale Interfaces selbst definiert
- Demo Vordefinierte funktionale Interfaces im JDK

2.2.1.1 Problem: Sortieren einer Studi-Liste

```
List<Studi> sl = new ArrayList<>();

// Liste sortieren?
sl.sort(???);  // Parameter: java.util.Comparator<Studi>
```

```
public class MyCompare implements Comparator<Studi> {
    @Override  public int compare(Studi o1, Studi o2) {
        return o1.getCredits() - o2.getCredits();
    }
}
```

```
// Liste sortieren?
MyCompare mc = new MyCompare();
sl.sort(mc);
```

Da `Comparator<T>` ein Interface ist, muss man eine extra Klasse anlegen, die die abstrakte Methode aus dem Interface implementiert und ein Objekt von dieser Klasse erzeugen und dieses dann der `sort()`-Methode übergeben.

Die Klasse bekommt wie in Java üblich eine eigene Datei und ist damit in der Package-Struktur offen sichtbar und "verstopft" mir damit die Strukturen: Diese Klasse ist doch nur eine Hilfsklasse … Noch schlimmer: Ich brauche einen Namen für diese Klasse!

Den ersten Punkt könnte man über verschachtelte Klassen lösen: Die Hilfsklasse wird innerhalb der Klasse definiert, die das Objekt benötigt. Für den zweiten Punkt brauchen wir mehr Anlauf …

2.2.1.2 Erinnerung: Verschachtelte Klassen ("*Nested Classes*")

Man kann Klassen innerhalb von Klassen definieren: Verschachtelte Klassen.

- Implizite Referenz auf Instanz der äußeren Klasse, Zugriff auf **alle** Elemente
- **Begriffe**:
  - "normale" innere Klassen: "*inner classes*"
  - statische innere Klassen: "*static nested classes*"
- Einsatzzweck:
  - Hilfsklassen: Zusätzliche Funktionalität kapseln; Nutzung **nur** in äußerer Klasse
  - Kapselung von Rückgabewerten

Sichtbarkeit: Wird u.U. von äußerer Klasse "überstimmt"

2.2.1.2.1 Innere Klassen (“*Inner Classes*”)

- Objekt der äußeren Klasse muss existieren
- Innere Klasse ist normales Member der äußeren Klasse
- Implizite Referenz auf Instanz äußerer Klasse
- Zugriff auf **alle** Elemente der äußeren Klasse
- Sonderfall: Definition innerhalb von Methoden (“local classes”)
  - Nur innerhalb der Methode sichtbar
  - Kennt zusätzlich `final` Attribute der Methode

Beispiel:

```
public class Outer {
    ...
    private class Inner {
        ...
    }

    Outer.Inner inner = new Outer().new Inner();
}
```

Beispiel mit Iterator als innere Klasse: nested.StudiListNested

2.2.1.2.2 Statische innere Klassen (“*Static Nested Classes*”)

- Keine implizite Referenz auf Objekt
- Nur Zugriff auf Klassenmethoden und -attribute

Beispiel:

```
class Outer {
    ...
    static class StaticNested {
        ...
    }
}

Outer.StaticNested nested = new Outer.StaticNested();
```

2.2.1.3 Lösung: Comparator als anonyme innere Klasse

```
List<Studi> sl = new ArrayList<>();

// Parametrisierung mit anonymer Klasse
sl.sort(
        new Comparator<Studi>() {
            @Override
            public int compare(Studi o1, Studi o2) {
                return o1.getCredits() - o2.getCredits();
            }
        });  // Semikolon nicht vergessen!!!
```

=> Instanz einer anonymen inneren Klasse, die das Interface `Comparator<Studi>` implementiert

- Für spezielle, einmalige Aufgabe: nur eine Instanz möglich
- Kein Name, kein Konstruktor, oft nur eine Methode
- Müssen Interface implementieren oder andere Klasse erweitern
  - Achtung Schreibweise: ohne **`implements`** oder **`extends`**!
- Konstruktor kann auch Parameter aufweisen
- Zugriff auf alle Attribute der äußeren Klasse plus alle **`final`** lokalen Variablen
- Nutzung typischerweise bei GUIs: Event-Handler etc.

Demo: nested.DemoAnonymousInnerClass

2.2.1.4 Vereinfachung mit Lambda-Ausdruck

```
1  List<Studi> sl = new ArrayList<>();
2
3  // Parametrisierung mit anonymer Klasse
4  sl.sort(
5          new Comparator<Studi>() {
6              @Override
7              public int compare(Studi o1, Studi o2) {
8                  return o1.getCredits() - o2.getCredits();
9              }
10         });  // Semikolon nicht vergessen!!!
11
12
13 // Parametrisierung mit Lambda-Ausdruck
14 sl.sort( (Studi o1, Studi o2) -> o1.getCredits() - o2.getCredits() );
```

Anmerkung: Damit für den Parameter alternativ auch ein Lambda-Ausdruck verwendet werden kann, muss der erwartete Parameter vom Typ her ein "**funktionales Interface**" (s.u.) sein!

Demo: nested.DemoLambda

2.2.1.5 Syntax für Lambdas

```
1  (Studi o1, Studi o2)  ->  o1.getCredits() - o2.getCredits()
```

Ein Lambda-Ausdruck ist eine Funktion ohne Namen und besteht aus drei Teilen:

1. Parameterliste (in runden Klammern),
2. Pfeil
3. Funktionskörper (rechte Seite)

Falls es *genau einen* Parameter gibt, *können* die runden Klammern um den Parameter entfallen.

Dabei kann der Funktionskörper aus *einem Ausdruck* ("*expression*") bestehen oder einer *Menge von Anweisungen* ("*statements*"), die dann in geschweifte Klammern eingeschlossen werden müssen (Block mit Anweisungen).

Der Wert des Ausdrucks ist zugleich der Rückgabewert des Lambda-Ausdrucks.

Varianten:

- **`(parameters)-> expression`**
- **`(parameters)-> { statements; }`**

2.2.1.6 Quiz: Welches sind keine gültigen Lambda-Ausdrücke?

1. `() -> {}`
2. `() -> "wuppie"`
3. `() -> { return "fluppie"; }`
4. `(Integer i)-> return i + 42;`
5. `(String s)-> { "foo"; }`
6. `(String s)-> s.length()`
7. `(Studi s)-> s.getCredits()> 300`
8. `(List<Studi> sl)-> sl.isEmpty()`
9. `(int x, int y)-> { System.out.println("Erg: "); System.out.println(x+y); }`
10. `() -> new Studi()`
11. `s -> s.getCps()> 100 && s.getCps()< 300`
12. `s -> { return s.getCps()> 100 && s.getCps()< 300; }`

Auflösung:

(4) und (5): **`return`** ist eine Anweisung, d.h. bei (4) fehlen die geschweiften Klammern. `"foo"` ist ein String und als solcher ein Ausdruck, d.h. hier sind die geschweiften Klammern zu viel (oder man ergänze den String mit einem **`return`**, also **`return`** `"foo";` …).

2.2.1.7 Definition "Funktionales Interface" ("*functional interface*")

```java
@FunctionalInterface
public interface Wuppie<T> {
    int wuppie(T obj);
    boolean equals(Object obj);
    default int fluppie() { return 42; }
}
```

`Wuppie<T>` ist ein **funktionales Interface** ("*functional interface*") (seit Java 8)

- Hat **genau *eine* abstrakte Methode**
- Hat evtl. weitere Default-Methoden
- Hat evtl. weitere abstrakte Methoden, die **`public`** Methoden von `java.lang.Object` überschreiben

Die Annotation `@FunctionalInterface` selbst ist nur für den Compiler: Falls das Interface *kein* funktionales Interface ist, würde er beim Vorhandensein dieser Annotation einen Fehler werfen. Oder anders herum: Allein durch das Annotieren mit `@FunctionalInterface` wird aus einem Interface noch kein funktionales Interface! Vergleichbar mit `@Override` …

Während man für eine anonyme Klasse lediglich ein "normales" Interface (oder eine Klasse) benötigt, braucht man für Lambda-Ausdrücke zwingend ein passendes funktionales Interface!

Anmerkung: Es scheint keine einheitliche deutsche Übersetzung für den Begriff *functional interface* zu geben. Es wird häufig mit "funktionales Interface", manchmal aber auch mit "Funktionsinterface" übersetzt.

Das in den obigen Beispielen eingesetzte Interface `java.util.Comparator<T>` ist also ein funktionales Interface: Es hat nur *eine* eigene abstrakte Methode **`int`** `compare(T o1, T o2);`.

Im Package java.util.function sind einige wichtige funktionale Interfaces bereits vordefiniert, beispielsweise `Predicate` (Test, ob eine Bedingung erfüllt ist) und `Function` (verarbeite einen Wert und liefere einen passenden Ergebniswert). Diese kann man auch in eigenen Projekten nutzen!

2.2.1.8 Quiz: Welches ist kein funktionales Interface?

```java
public interface Wuppie {
    int wuppie(int a);
}

public interface Fluppie extends Wuppie {
    int wuppie(double a);
}

public interface Foo {
}

public interface Bar extends Wuppie {
    default int bar() { return 42; }
}
```

Auflösung:

- `Wuppie` hat *genau eine* abstrakte Methode => funktionales Interface
- `Fluppie` hat zwei abstrakte Methoden => **kein** funktionales Interface
- `Foo` hat gar keine abstrakte Methode => **kein** funktionales Interface
- `Bar` hat *genau eine* abstrakte Methode (und eine Default-Methode) => funktionales Interface

2.2.1.9 Lambdas und funktionale Interfaces: Typprüfung

```java
interface java.util.Comparator<T> {
    int compare(T o1, T o2);    // abstrakte Methode
}
```

```java
// Verwendung ohne weitere Typinferenz
Comparator<Studi> c1 = (Studi o1, Studi o2) -> o1.getCredits() - o2.getCredits
    ();

// Verwendung mit Typinferenz
Comparator<Studi> c2 = (o1, o2) -> o1.getCredits() - o2.getCredits();
```

Der Compiler prüft in etwa folgende Schritte, wenn er über einen Lambda-Ausdruck stolpert:

1. In welchem Kontext habe ich den Lambda-Ausdruck gesehen?
2. OK, der Zieltyp ist hier `Comparator<Studi>`.
3. Wie lautet die **eine** abstrakte Methode im `Comparator<T>`-Interface?
4. OK, das ist `int compare(T o1, T o2);`
5. Da `T` hier an `Studi` gebunden ist, muss der Lambda-Ausdruck der Methode `int compare(Studi o1, Studi o2);` entsprechen: 2x `Studi` als Parameter und als Ergebnis ein `int`
6. Ergebnis:
   a) Cool, passt zum Lambda-Ausdruck `c1`. Fertig.
   b) D.h. in `c2` müssen `o1` und `o2` vom Typ `Studi` sein. Cool, passt zum Lambda-Ausdruck `c2`. Fertig.

2.2.1.10 Wrap-Up

- Anonyme Klassen: “Wegwerf”-Innere Klassen
  - Müssen Interface implementieren oder Klasse erweitern
- Java8: **Lambda-Ausdrücke** statt anonymer Klassen (**funktionales Interface nötig**)
  - Zwei mögliche Formen:
    - Form 1: `(parameters)-> expression`
    - Form 2: `(parameters)-> { statements; }`
  - Im jeweiligen Kontext muss ein **funktionales Interface** verwendet werden, d.h. ein Interface mit **genau** einer abstrakten Methode
  - Der Lambda-Ausdruck muss von der Signatur her dieser einen abstrakten Methode entsprechen

Zum Nachlesen

- Oracle Corporation (2022c)
- Urma u. a. (2014, Kap. 3)
- Ullenboom (2021, Kap. 12)

Lernziele

- k2: Funktionales Interfaces (Definition)
- k3: Einsatz innerer und anonymer Klassen
- k3: Erstellen eigener funktionaler Interfaces
- k3: Einsatz von Lambda-Ausdrücken

Quizzes

- Quiz Lambda-Ausdrücke und funktionale Interfaces (ILIAS)

Challenges

Beispiel aus einem Code-Review im Dungeon-CampusMinden/Dungeon

Erklären Sie folgenden Code:

```java
public interface IFightAI {
    void fight(Entity entity);
}

public class AIComponent extends Component {
    private final IFightAI fightAI;

    fightAI =
                entity1 -> {
                    System.out.println("TIME TO FIGHT!");
                    // todo replace with melee skill
                };
}
```

Sortieren mit Lambdas und funktionalen Interfaces

In den Vorgaben finden Sie die Klassen `Student` und `StudentSort` mit vorgefertigten Methoden zu den Teilaufgaben sowie eine Testsuite `SortTest` mit einzelnen Testfälllen zu den Teilaufgaben, mit der Ihre Implementierung aufgerufen und getestet wird.
Ziel dieser Aufgabe ist es, eine Liste von Studierenden mithilfe verschiedener syntaktischer Strukturen (Lambda-Ausdrücke, Methoden-Referenzen) zu sortieren. Dabei soll bei allen Teilaufgaben die Methode java.util.List#sort für das eigentliche Sortieren verwendet werden.

1. In dieser Teilaufgabe sollen Sie der Methode `List#sort` das Sortierkriterium mithilfe eines **Lambda-Ausdrucks** übergeben. Greifen Sie im Lambda-Ausdruck für den Vergleich der Objekte auf die Getter der Objekte zu.

   Hinweis: Erstellen Sie hierzu keine neuen Methoden, sondern verwenden Sie nur Lambda-Ausdrücke innerhalb des Aufrufs von `List#sort`.

   1a Sortieren Sie die Studierendenliste aufsteigend nach dem Geburtsdatum (`sort_1a()`).

   1b Sortieren Sie die Studierendenliste absteigend nach dem Namen (`sort_1b()`).

2. Erweitern Sie die Klasse `Student` um eine *statische* Methode, die zwei `Student`-Objekte anhand des Alters miteinander vergleicht. Die Methode soll die Signatur `static int compareByAge(Student a, Student b)` besitzen und die folgenden Werte zurückliefern:

   - a > b -> -1
   - a < b -> 1
   - a == b -> 0

   Verwenden Sie die neue statische Methode `compareByAge` zum Sortieren der Liste in `sort_2a()`. Nutzen Sie dabei einen **Lambda-Ausdruck**.

3. Erweitern Sie die Klasse `Student` um eine Instanz-Methode, die das `Student`-Objekt mit einem anderen (als Parameter übergebenen) `Student`-Objekt vergleicht. Die Methode soll die Signatur `int compareByName(Student other)` besitzen und die folgenden Werte zurückliefern:

   - self > other -> -1
   - self < other -> 1
   - self == other -> 0

   Verwenden Sie die neue Methode `compareByName` zum Sortieren der Liste in `sort_3a()`. Nutzen Sie dabei einen **Lambda-Ausdruck**.

4. Erstellen Sie ein generisches Funktionsinterface, dass die Methode `compare` definiert und zum Vergleichen von zwei Objekten mit generischen Typen dient.

   Erzeugen Sie mithilfe eines **Lambda-Ausdrucks** eine **Instanz** Ihres Interfaces, um damit zwei Objekte vom Typ `Student` in Bezug auf ihr Alter vergleichen zu können. Verwenden Sie die erzeugte Instanz, um die Studierendenliste absteigend zu sortieren (`sort_4a()`).

2.2.2 Methoden-Referenzen

TL;DR
Seit Java8 können **Referenzen auf Methoden** statt anonymer Klassen eingesetzt werden (**funktionales Interface nötig**).
Dabei gibt es drei mögliche Formen:

- Form 1: Referenz auf eine statische Methode: `ClassName::staticMethodName` (wird verwendet wie `(args)-> ClassName.staticMethodName(args)`)
- Form 2: Referenz auf eine Instanz-Methode eines Objekts: `objectref::instanceMethodName` (wird verwendet wie `(args)-> objectref.instanceMethodName(args)`)
- Form 3: Referenz auf eine Instanz-Methode eines Typs: `ClassName::instanceMethodName` (wird verwendet wie `(o1, args)-> o1.instanceMethodName(args)`)

Im jeweiligen Kontext muss ein passendes funktionales Interface verwendet werden, d.h. ein Interface mit **genau** einer abstrakten Methode. Die Methoden-Referenz muss von der Syntax her dieser einen abstrakten Methode entsprechen (bei der dritten Form wird die Methode auf dem ersten Parameter aufgerufen).

Videos

- VL Methoden-Referenzen
- Demo Referenz auf statische Methode
- Demo Referenz auf Instanz-Methode (Objekt)
- Demo Referenz auf Instanz-Methode (Typ)

2.2.2.1 Beispiel: Sortierung einer Liste

```java
List<Studi> sl = new ArrayList<Studi>();

// Anonyme innere Klasse
Collections.sort(sl, new Comparator<Studi>() {
    @Override public int compare(Studi o1, Studi o2) {
        return Studi.cmpCpsClass(o1, o2);
    }
});


// Lambda-Ausdruck
Collections.sort(sl, (o1, o2) -> Studi.cmpCpsClass(o1, o2));

// Methoden-Referenz
Collections.sort(sl, Studi::cmpCpsClass);
```

2.2.2.1.1 Anmerkung Für das obige Beispiel wird davon ausgegangen, dass in der Klasse `Studi` eine statische Methode `cmpCpsClass()` existiert:

```java
public static int cmpCpsClass(Studi s1, Studi s2) {
    return s1.getCps() - s2.getCps();
}
```

Wenn man im Lambda-Ausdruck nur Methoden der eigenen Klasse aufruft, kann man das auch direkt per *Methoden-Referenz* abkürzen!

- Erinnerung: `Comparator<T>` ist ein funktionales Interface
- Instanzen können wie üblich durch Ableiten bzw. anonyme Klassen erzeugt werden
- Alternativ kann seit Java8 auch ein passender Lambda-Ausdruck verwendet werden
- Ab Java8: Referenzen auf passende Methoden (Signatur!) können ein funktionales Interface "implementieren"
  - Die statische Methode `static int cmpCpsClass(Studi s1, Studi s2)` hat die selbe Signatur wie `int compare(Studi s1, Studi s2)` aus `Comparator<Studi>`
  - Kann deshalb wie eine Instanz von `Comparator<Studi>` genutzt werden
  - Name der Methode spielt dabei keine Rolle

2.2.2.2 Überblick: Arten von Methoden-Referenzen

1. Referenz auf eine statische Methode
   - Form: `ClassName::staticMethodName`
   - Wirkung: Aufruf mit `(args)-> ClassName.staticMethodName(args)`
2. Referenz auf Instanz-Methode eines bestimmten Objekts
   - Form: `objectref::instanceMethodName`
   - Wirkung: Aufruf mit `(args)-> objectref.instanceMethodName(args)`
3. Referenz auf Instanz-Methode eines bestimmten Typs
   - Form: `ClassName::instanceMethodName`
   - Wirkung: Aufruf mit `(arg0, rest)-> arg0.instanceMethodName(rest)` (`arg0` ist vom Typ `ClassName`)

Anmerkung: Analog zur Referenz auf eine statische Methode gibt es noch die Form der Referenz auf einen Konstruktor: `ClassName::new`. Für Referenzen auf Konstruktoren mit mehr als 2 Parametern muss ein eigenes passendes funktionales Interface mit entsprechend vielen Parametern definiert werden …

2.2.2.3 Methoden-Referenz 1: Referenz auf statische Methode

```java
public class Studi {
    public static int cmpCpsClass(Studi s1, Studi s2) {
        return s1.getCredits() - s2.getCredits();
    }

    public static void main(String... args) {
        List<Studi> sl = new ArrayList<Studi>();

        // Referenz auf statische Methode
        Collections.sort(sl, Studi::cmpCpsClass);

        // Entsprechender Lambda-Ausdruck
        Collections.sort(sl, (o1, o2) -> Studi.cmpCpsClass(o1, o2));
    }
}
```

Demo: methodreferences.DemoStaticMethodReference

`Collections.sort()` erwartet in diesem Szenario als zweiten Parameter eine Instanz von `Comparator<Studi>` mit einer Methode `int compare(Studi o1, Studi o2)`.

Die übergebene Referenz auf die **statische Methode `cmpCpsClass` der Klasse `Studi`** hat die **selbe Signatur** und wird deshalb von `Collections.sort()` genauso genutzt wie die eigentlich erwartete Methode `Comparator<Studi>#compare(Studi o1, Studi o2)`, d.h. statt `compare(o1, o2)` wird nun für jeden Vergleich **`Studi.cmpCpsClass(o1, o2)`** aufgerufen.

2.2.2.4 Methoden-Referenz 2: Referenz auf Instanz-Methode (Objekt)

```java
public class Studi {
    public int cmpCpsInstance(Studi s1, Studi s2) {
        return s1.getCredits() - s2.getCredits();
    }

    public static void main(String... args) {
        List<Studi> sl = new ArrayList<Studi>();
        Studi holger = new Studi("Holger", 42);

        // Referenz auf Instanz-Methode eines Objekts
        Collections.sort(sl, holger::cmpCpsInstance);

        // Entsprechender Lambda-Ausdruck
        Collections.sort(sl, (o1, o2) -> holger.cmpCpsInstance(o1, o2));
    }
}
```

Demo: methodreferences.DemoInstanceMethodReferenceObject

`Collections.sort()` erwartet in diesem Szenario als zweites Argument wieder eine Instanz von `Comparator<Studi>` mit einer Methode `int compare(Studi o1, Studi o2)`.

Die übergebene Referenz auf die **Instanz-Methode `cmpCpsInstance` des Objekts `holger`** hat die selbe Signatur und wird entsprechend von `Collections.sort()` genauso genutzt wie die eigentlich erwartete Methode `Comparator<Studi>#compare(Studi o1, Studi o2)`, d.h. statt `compare(o1, o2)` wird nun für jeden Vergleich **`holger.cmpCpsInstance(o1, o2)`** aufgerufen.

2.2.2.5 Methoden-Referenz 3: Referenz auf Instanz-Methode (Typ)

```java
public class Studi {
    public int cmpCpsInstance(Studi studi) {
        return this.getCredits() - studi.getCredits();
    }

    public static void main(String... args) {
        List<Studi> sl = new ArrayList<Studi>();

        // Referenz auf Instanz-Methode eines Typs
        Collections.sort(sl, Studi::cmpCpsInstance);

        // Entsprechender Lambda-Ausdruck
        Collections.sort(sl, (o1, o2) -> o1.cmpCpsInstance(o2));
    }
}
```

Demo: methodreferences.DemoInstanceMethodReferenceType

`Collections.sort()` erwartet in diesem Szenario als zweites Argument wieder eine Instanz von `Comparator<Studi>` mit einer Methode `int compare(Studi o1, Studi o2)`.

Die übergebene Referenz auf die **Instanz-Methode `cmpCpsInstance` des Typs `Studi`** hat die Signatur `int cmpCpsInstance(Studi studi)` und wird von `Collections.sort()` so genutzt: Statt `compare(o1, o2)` wird nun für jeden Vergleich **`o1.cmpCpsInstance(o2)`** aufgerufen.

2.2.2.6 Ausblick: Threads

Erinnerung an bzw. Vorgriff auf "Threads: Intro":

```
1  public interface Runnable {
2      void run();
3  }
```

Damit lassen sich Threads auf verschiedene Arten erzeugen:

```
1  public class ThreadStarter {
2      public static void wuppie() { System.out.println("wuppie(): wuppie"); }
3  }
4
5
6  Thread t1 = new Thread(new Runnable() {
7      public void run() {
8          System.out.println("t1: wuppie");
9      }
10 });
11
12 Thread t2 = new Thread(() -> System.out.println("t2: wuppie"));
13
14 Thread t3 = new Thread(ThreadStarter::wuppie);
```

Beispiel: methodreferences.ThreadStarter

2.2.2.7 Ausblick: Datenstrukturen als Streams

Erinnerung an bzw. Vorgriff auf "Stream-API":

```
1  class X {
2      public static boolean gtFour(int x) { return (x > 4) ? true : false; }
3  }
4
5  List<String> words = Arrays.asList("Java8", "Lambdas", "PM",
6          "Dungeon", "libGDX", "Hello", "World", "Wuppie");
7
8  List<Integer> wordLengths = words.stream()
9          .map(String::length)
10         .filter(X::gtFour)
11         .sorted()
12         .collect(toList());
```

Beispiel: methodreferences.CollectionStreams

- Collections können als Datenstrom betrachtet werden: `stream()`
  - Iteration über die Collection, analog zu externer Iteration mit `foreach`

- Daten aus dem Strom filtern: `filter`, braucht Prädikat
- Auf alle Daten eine Funktion anwenden: `map`
- Daten im Strom sortieren: `sort` (auch mit Comparator)
- Daten wieder einsammeln mit `collect`

=> Typische Elemente **funktionaler Programmierung**

=> Verweis auf Wahlfach “Spezielle Methoden der Programmierung”

2.2.2.8 Wrap-Up

Seit Java8: **Methoden-Referenzen** statt anonymer Klassen (**funktionales Interface nötig**)

- Drei mögliche Formen:
  - Form 1: Referenz auf statische Methode: `ClassName::staticMethodName` (verwendet wie `(args)-> ClassName.staticMethodName(args)`)
  - Form 2: Referenz auf Instanz-Methode eines Objekts: `objectref::instanceMethodName` (verwendet wie `(args)-> objectref.instanceMethodName(args)`)
  - Form 3: Referenz auf Instanz-Methode eines Typs: `ClassName::instanceMethodName` (verwendet wie `(o1, args)-> o1.instanceMethodName(args)`)
- Im jeweiligen Kontext muss ein passendes funktionales Interface verwendet werden (d.h. ein Interface mit **genau** einer abstrakten Methode)

Zum Nachlesen

- Oracle Corporation (2022c)
- Urma u. a. (2014, Kap. 3)

Lernziele

- k2: Funktionales Interfaces (Definition)
- k3: Einsatz von Methoden-Referenzen

Quizzes

- Quiz Methoden-Referenzen (ILIAS)

Challenges

In den Vorgaben finden Sie die Klassen `Student` und `StudentSort` mit vorgefertigten Methoden zu den Teilaufgaben sowie eine Testsuite `SortTest` mit einzelnen Testfälllen zu den Teilaufgaben, mit der Ihre Implementierung aufgerufen und getestet wird.
Ziel dieser Aufgabe ist es, eine Liste von Studierenden mithilfe verschiedener syntaktischer Strukturen (Lambda-Ausdrücke, Methoden-Referenzen) zu sortieren. Dabei soll bei allen Teilaufgaben die Methode java.util.List#sort für das eigentliche Sortieren verwendet werden.

1. Erweitern Sie die Klasse `Student` um eine *statische* Methode, die zwei `Student`-Objekte anhand des Alters miteinander vergleicht. Die Methode soll die Signatur `static int compareByAge(Student a, Student b)` besitzen und die folgenden Werte zurückliefern:
   - a > b -> -1
   - a < b -> 1
   - a == b -> 0

   Verwenden Sie die neue statische Methode `compareByAge` zum Sortieren der Liste in `sort_2b()`. Nutzen Sie dabei eine **Methodenreferenz**.
2. Erweitern Sie die Klasse `Student` um eine Instanz-Methode, die das `Student`-Objekt mit einem anderen (als Parameter übergebenen) `Student`-Objekt vergleicht. Die Methode soll die Signatur `int compareByName(Student other)` besitzen und die folgenden Werte zurückliefern:
   - self > other -> -1
   - self < other -> 1
   - self == other -> 0

   Verwenden Sie die neue Methode `compareByName` zum Sortieren der Liste in `sort_3b()`. Nutzen Sie dabei eine **Methodenreferenz**.

2.2.3 Stream-API

TL;DR
Mit der Collection-API existiert in Java die Möglichkeit, Daten auf verschiedenste Weisen zu speichern (`Collection<T>`). Mit der Stream-API gibt es die Möglichkeit, diese Daten in einer Art Pipeline zu verarbeiten. Ein `Stream<T>` ist eine Folge von Objekten vom Typ `T`. Die Verarbeitung der Daten ist "lazy", d.h. sie erfolgt erst auf Anforderung (durch die terminale Operation).
Ein Stream hat eine Datenquelle und kann beispielsweise über `Collection#stream()` oder `Stream.of()` angelegt werden. Streams speichern keine Daten. Die Daten werden aus der verbundenen Datenquelle geholt.
Auf einem Stream kann man eine Folge von intermediären Operationen wie `peek()`, `map()`, `flatMap()`, `filter()`, `sorted()` … durchführen. Alle diese Operationen arbeiten auf dem Stream und erzeugen einen neuen Stream als Ergebnis. Dadurch kann die typische Pipeline-artige Verkettung der Operationen ermöglicht werden. Die intermediären Operationen werden erst ausgeführt, wenn der Stream durch eine terminale Operation geschlossen wird.
Terminale Operationen wie `count()`, `forEach()`, `allMatch()` oder `collect()`

- `collect(Collectors.toList())` (bzw. direkt mit `stream.toList()` (ab Java16))
- `collect(Collectors.toSet())`
- `collect(Collectors.toCollection(LinkedList::new))` (als `Supplier<T>`)

stoßen die Verarbeitung des Streams an und schließen den Stream damit ab.
Wir können hier nur die absoluten Grundlagen betrachten. Die Stream-API ist sehr groß und mächtig und lohnt die weitere selbstständige Auseinandersetzung :-)

Videos

- VL Stream-API
- Demo Stream-API
- Demo Vordefinierte funktionale Interfaces im JDK

2.2.3.1 Motivation Es wurden Studis, Studiengänge und Fachbereiche modelliert (aus Gründen der Übersichtlichkeit einfach als Record-Klassen).

Nun soll pro Fachbereich die Anzahl der Studis ermittelt werden, die bereits 100 ECTS oder mehr haben. Dazu könnte man über alle Studiengänge im Fachbereich iterieren, und in der inneren Schleife über alle Studis im Studiengang. Dann filtert man alle Studis, deren ECTS größer 100 sind und erhöht jeweils den Zähler:

```java
public record Studi(String name, int credits) {}
public record Studiengang(String name, List<Studi> studis) {}
public record Fachbereich(String name, List<Studiengang> studiengaenge) {}

private static long getCountFB(Fachbereich fb) {
    long count = 0;
    for (Studiengang sg : fb.studiengaenge()) {
        for (Studi s : sg.studis()) {
            if (s.credits() > 100) count += 1;
        }
    }
    return count;
}
```

Dies ist ein Beispiel, welches klassisch in OO-Manier als Iteration über Klassen realisiert ist. (Inhaltlich ist es vermutlich nicht sooo sinnvoll.)

2.2.3.2 Innere Schleife mit Streams umgeschrieben

```
1  private static long getCountSG(Studiengang sg) {
2      return sg.studis().stream()
3                        .map(Studi::credits)
4                        .filter(c -> c > 100)
5                        .count();
6  }
7
8  private static long getCountFB2(Fachbereich fb) {
9      long count = 0;
10     for (Studiengang sg : fb.studiengaenge()) {
11         count += getCountSG(sg);
12     }
13     return count;
14 }
```

2.2.3.2.1 Erklärung des Beispiels Im Beispiel wurde die innere Schleife in einen Stream ausgelagert.

Mit der Methode `Collection#stream()` wird aus der Collection ein neuer Stream erzeugt. Auf diesem wird für jedes Element durch die Methode `map()` die Methode `Studi#credits()` angewendet, was aus einem Strom von `Studi` einen Strom von `Integer` macht. Mit `filter()` wird auf jedes Element das Prädikat `c -> c > 100` angewendet und alle Elemente aus dem Strom entfernt, die der Bedingung nicht entsprechen. Am Ende wird mit `count()` gezählt, wie viele Elemente im Strom enthalten sind.

2.2.3.2.2 Was ist ein Stream? Ein “Stream” ist ein Strom (Folge) von Daten oder Objekten. In Java wird die Collections-API für die Speicherung von Daten (Objekten) verwendet. Die Stream-API dient zur Iteration über diese Daten und entsprechend zur Verarbeitung der Daten. In Java speichert ein Stream keine Daten.

Das Konzept kommt aus der funktionalen Programmierung und wurde in Java nachträglich eingebaut (wobei dieser Prozess noch lange nicht abgeschlossen zu sein scheint).

In der funktionalen Programmierung kennt man die Konzepte “map”, “filter” und “reduce”: Die Funktion “map()” erhält als Parameter eine Funktion und wendet diese auf alle Elemente eines Streams an. Die Funktion “filter()” bekommt ein Prädikat als Parameter und prüft jedes Element im Stream, ob es dem Prädikat genügt (also ob das Prädikat mit dem jeweiligen Element zu **`true`** evaluiert - die anderen Objekte werden entfernt). Mit “reduce()” kann man Streams zu einem einzigen Wert zusammenfassen (denken Sie etwa an das Aufsummieren aller Elemente eines Integer-Streams). Zusätzlich kann man in der funktionalen Programmierung ohne Probleme unendliche Ströme darstellen: Die Auswertung erfolgt nur bei Bedarf und auch dann auch nur so weit wie nötig. Dies nennt man auch “*lazy evaluation*”.

Die Streams in Java versuchen, diese Konzepte aus der funktionalen Programmierung in die objektorientierte Programmierung zu übertragen. Ein Stream in Java hat eine Datenquelle, von wo die Daten gezogen werden - ein Stream speichert selbst keine Daten. Es gibt “intermediäre Operationen” auf einem Stream, die die Elemente verarbeiten und das Ergebnis als Stream zurückliefern. Daraus ergibt sich typische Pipeline-artige Verkettung

der Operationen. Allerdings werden diese Operationen erst durchgeführt, wenn eine “terminale Operation” den Stream “abschließt”. Ein Stream ohne eine terminale Operation macht also tatsächlich *nichts*.

Die Operationen auf dem Stream sind üblicherweise zustandslos, können aber durchaus auch einen Zustand haben. Dies verhindert üblicherweise die parallele Verarbeitung der Streams. Operationen sollten aber nach Möglichkeit keine *Seiteneffekte* haben, d.h. keine Daten außerhalb des Streams modifizieren. Operationen dürfen auf keinen Fall die Datenquelle des Streams modifizieren!

2.2.3.3 Erzeugen von Streams

```java
List<String> l1 = List.of("Hello", "World", "foo", "bar", "wuppie");
Stream<String> s1 = l1.stream();

Stream<String> s2 = Stream.of("Hello", "World", "foo", "bar", "wuppie");

Random random = new Random();
Stream<Integer> s3 = Stream.generate(random::nextInt);

Pattern pattern = Pattern.compile(" ");
Stream<String> s4 = pattern.splitAsStream("Hello world! foo bar wuppie!");
```

Dies sind möglicherweise die wichtigsten Möglichkeiten, in Java einen Stream zu erzeugen.

Ausgehend von einer Klasse aus der Collection-API kann man die Methode `Collection#stream()` aufrufen und bekommt einen seriellen Stream.

Alternativ bietet das Interface `Stream` verschiedene statische Methoden wie `Stream.of()` an, mit deren Hilfe Streams angelegt werden können. Dies funktioniert auch mit Arrays …

Und schließlich kann man per `Stream.generate()` einen Stream anlegen, wobei als Argument ein “Supplier” (Interface `java.util.function.Supplier<T>`) übergeben werden muss. Dieses Argument wird dann benutzt, um die Daten für den Stream zu generieren.

Wenn man aufmerksam hinschaut, findet man an verschiedensten Stellen die Möglichkeit, die Daten per Stream zu verarbeiten, u.a. bei regulären Ausdrücken.

Man kann per `Collection#parallelStream()` auch parallele Streams erzeugen, die intern das “Fork&Join-Framework” nutzen. Allerdings sollte man nur dann parallele Streams anlegen, wenn dadurch tatsächlich Vorteile durch die Parallelisierung zu erwarten sind (Overhead!).

2.2.3.4 Intermediäre Operationen auf Streams

```java
private static void dummy(Studiengang sg) {
    sg.studis().stream()
            .peek(s -> System.out.println("Looking at: " + s.name()))
            .map(Studi::credits)
            .peek(c -> System.out.println("This one has: " + c + " ECTS"))
            .filter(c -> c > 5)
            .peek(c -> System.out.println("Filtered: " + c))
            .sorted()
            .forEach(System.out::println);
}
```

An diesem (weitestgehend sinnfreien) Beispiel werden einige intermediäre Operationen demonstriert.

Die Methode `peek()` liefert einen Stream zurück, die aus den Elementen des Eingabestroms bestehen. Auf jedes Element wird die Methode `void accept(T)` des `Consumer<T>` angewendet (Argument der Methode), was aber nicht zu einer Änderung der Daten führt. **Hinweis**: Diese Methode dient vor allem zu Debug-Zwecken! Durch den Seiteneffekt kann die Methode eine schlechtere Laufzeit zur Folge haben oder sogar eine sonst mögliche parallele Verarbeitung verhindern oder durch eine parallele Verarbeitung verwirrende Ergebnisse zeigen!

Die Methode `map()` liefert ebenfalls einen Stream zurück, der durch die Anwendung der Methode `R apply(T)` der als Argument übergebenen `Function<T,R>` auf jedes Element des Eingabestroms entsteht. Damit lassen sich die Elemente des ursprünglichen Streams verändern; für jedes Element gibt es im Ergebnis-Stream ebenfalls ein Element (der Typ ändert sich, aber nicht die Anzahl der Elemente).

Mit der Methode `filter()` wird ein Stream erzeugt, der alle Objekte des Eingabe-Streams enthält, auf denen die Anwendung der Methode `boolean test(T)` des Arguments `Predicate<T>` zu `true` evaluiert (der Typ und Inhalt der Elemente ändert sich nicht, aber die Anzahl der Elemente).

Mit `sorted()` wird ein Stream erzeugt, der die Elemente des Eingabe-Streams sortiert (existiert auch mit einem `Comparator<T>` als Parameter).

Diese Methoden sind alles **intermediäre** Operationen. Diese arbeiten auf einem Stream und erzeugen einen neuen Stream und werden erst dann ausgeführt, wenn eine terminale Operation den Stream abschließt.

Dabei sind die gezeigten intermediären Methoden bis auf `sorted()` ohne inneren Zustand. `sorted()` ist eine Operation mit innerem Zustand (wird für das Sortieren benötigt). Dies kann ordentlich in Speicher und Zeit zuschlagen und u.U. nicht/nur schlecht parallelisierbar sein. Betrachten Sie den fiktiven parallelen Stream `stream.parallel().sorted().skip(42)`: Hier müssen erst *alle* Elemente sortiert werden, bevor mit `skip(42)` die ersten 42 Elemente entfernt werden. Dies kann auch nicht mehr parallel durchgeführt werden.

Die Methode `forEach()` schließlich ist eine **terminale** Operation, die auf jedes Element des Eingabe-Streams die Methode `void accept(T)` des übergebenen `Consumer<T>` anwendet. Diese Methode ist eine **terminale Operation**, d.h. sie führt zur Auswertung der anderen *intermediären* Operationen und schließt den Stream ab.

2.2.3.5 Was tun, wenn eine Methode Streams zurückliefert

Wir konnten vorhin nur die innere Schleife in eine Stream-basierte Verarbeitung umbauen. Das Problem ist: Die äußere Schleife würde einen Stream liefern (Stream von Studiengängen), auf dem wir die `map`-Funktion anwenden müssten und darin dann für jeden Studiengang einen (inneren) Stream mit den Studis eines Studiengangs verarbeiten müssten.

```
1  private static long getCountSG(Studiengang sg) {
2      return sg.studis().stream().map(Studi::credits).filter(c -> c > 100).count
           ();
3  }
4
5  private static long getCountFB2(Fachbereich fb) {
6      long count = 0;
7      for (Studiengang sg : fb.studiengaenge()) {
8          count += getCountSG(sg);
9      }
10     return count;
11 }
```

Dafür ist die Methode `flatMap()` die Lösung. Diese Methode bekommt als Argument ein Objekt vom Typ

`Function<? super T, ? extends Stream<? extends R>>` mit einer Methode `Stream<? extends R> apply(T)`. Die Methode `flatMap()` verarbeitet den Stream in zwei Schritten:

1. Mappe über alle Elemente des Eingabe-Streams mit der Funktion. Im Beispiel würde also aus einem `Stream<Studiengang>` jeweils ein `Stream<Stream<Studi>>`, also alle `Studiengang`-Objekte werden durch je ein `Stream<Studi>`-Objekt ersetzt. Wir haben jetzt also einen Stream von `Stream<Studi>`-Objekten.
2. "Klopfe den Stream wieder flach", d.h. nimm die einzelnen `Studi`-Objekte aus den `Stream<Studi>`-Objekten und setze diese stattdessen in den Stream. Das Ergebnis ist dann wie gewünscht ein `Stream<Studi>` (Stream mit `Studi`-Objekten).

```
private static long getCountFB3(Fachbereich fb) {
    return fb.studiengaenge().stream()
            .flatMap(sg -> sg.studis().stream())
            .map(Studi::credits)
            .filter(c -> c > 100)
            .count();
}
```

Zum direkten Vergleich hier noch einmal der ursprüngliche Code mit zwei verschachtelten Schleifen und entsprechenden Hilfsvariablen:

```
private static long getCountFB(Fachbereich fb) {
    long count = 0;
    for (Studiengang sg : fb.studiengaenge()) {
        for (Studi s : sg.studis()) {
            if (s.credits() > 100) count += 1;
        }
    }
    return count;
}
```

2.2.3.6 Streams abschließen: Terminale Operationen

```
Stream<String> s = Stream.of("Hello", "World", "foo", "bar", "wuppie");

long count = s.count();
s.forEach(System.out::println);
String first = s.findFirst().get();
Boolean b = s.anyMatch(e -> e.length() > 3);

List<String> s1 = s.collect(Collectors.toList());
List<String> s2 = s.toList();   // ab Java16
Set<String> s3 = s.collect(Collectors.toSet());
List<String> s4 = s.collect(Collectors.toCollection(LinkedList::new));
```

Streams müssen mit ***einer* terminalen Operation** abgeschlossen werden, damit die Verarbeitung tatsächlich angestoßen wird (*lazy evaluation*).

Es gibt viele verschiedene terminale Operationen. Wir haben bereits `count()` und `forEach()` gesehen. In der Sitzung zu "Optionals" werden wir noch `findFirst()` näher kennenlernen.

Daneben gibt es beispielsweise noch `allMatch()`, `anyMatch()` und `noneMatch()`, die jeweils ein Prädikat

testen und einen Boolean zurückliefern (matchen alle, mind. eines oder keines der Objekte im Stream).

Mit `min()` und `max()` kann man sich das kleinste und das größte Element des Streams liefern lassen. Beide Methoden benötigen dazu einen `Comparator<T>` als Parameter.

Mit der Methode `collect()` kann man eine der drei Methoden aus `Collectors` über den Stream laufen lassen und eine `Collection` erzeugen lassen:

1. `toList()` sammelt die Elemente in ein `List`-Objekt (bzw. direkt mit `stream.toList()` (ab Java16))
2. `toSet()` sammelt die Elemente in ein `Set`-Objekt
3. `toCollection()` sammelt die Elemente durch Anwendung der Methode `T get()` des übergebenen `Supplier<T>`-Objekts auf

Die ist nur die sprichwörtliche "Spitze des Eisbergs"! Es gibt viele weitere Möglichkeiten, sowohl bei den intermediären als auch den terminalen Operationen. Schauen Sie in die Dokumentation!

Demo: streams.Demo

2.2.3.7 Spielregeln

- Operationen dürfen nicht die Stream-Quelle modifizieren
- Operationen können die Werte im Stream ändern (`map`) oder die Anzahl (`filter`)
- Keine Streams in Attributen/Variablen speichern oder als Argumente übergeben: Sie könnten bereits "gebraucht" sein!

  => Ein Stream sollte immer sofort nach der Erzeugung benutzt werden
- Operationen auf einem Stream sollten keine Seiteneffekte (Veränderungen von Variablen/Attributen außerhalb des Streams) haben (dies verhindert u.U. die parallele Verarbeitung)

2.2.3.8 Wrap-Up `Stream<T>`: Folge von Objekten vom Typ `T`, Verarbeitung "lazy" (Gegenstück zu `Collection<T>`: Dort werden Daten **gespeichert**, hier werden Daten **verarbeitet**)

- Neuen Stream anlegen: `Collection#stream()` oder `Stream.of()` …
- Intermediäre Operationen: `peek()`, `map()`, `flatMap()`, `filter()`, `sorted()` …
- Terminale Operationen: `count()`, `forEach()`, `allMatch()`, `collect()` …
  - `collect(Collectors.toList())`
  - `collect(Collectors.toSet())`
  - `collect(Collectors.toCollection())` (mit `Supplier<T>`)
- Streams speichern keine Daten
- Intermediäre Operationen laufen erst bei Abschluss des Streams los
- Terminale Operation führt zur Verarbeitung und Abschluss des Streams

Schöne Doku: “The Stream API”, und auch “Package java.util.stream”.

Zum Nachlesen

- Oracle Corporation (2022b)
- Ullenboom (2021, 17.3–17.6)

Lernziele

- k2: Streams speichern keine Daten
- k2: Streams verarbeiten die Daten lazy
- k2: map() ändert den Typ (und Inhalt) von Objekten im Stream, aber nicht die Anzahl
- k2: filter() ändert die Anzahl der Objekte im Stream, aber nicht deren Typ (und Inhalt)
- k2: Streams machen ausführlich Gebrauch von den funktionalen Interfaces in java.util.function
- k2: Streams sollten nicht in Attributen gehalten oder als Argument von Methoden herumgereicht werden
- k3: Anlegen eines Streams
- k3: Verkettung von intermediären Operationen
- k3: Durchführung der Berechnung und Abschluss des Streams mit einer terminalen Operation
- k3: Einsatz von flatMap()

Quizzes

- Quiz Stream-API (ILIAS)

Challenges

In den Vorgaben finden Sie die Klasse `Main`, in der die Methoden `Main#a`, `Main#b` und `Main#c` “klassisch” mit **`for`**-Schleifen implementiert wurden.
Führen Sie für die drei Methoden `Main#a`, `Main#b` und `Main#c` ein Refactoring durch, so dass in diesen Methoden jeweils die Java Stream-API genutzt wird und es keine **`for`**-/`foreach`-/**`while`**-Schleifen mehr gibt.

2.2.4 Optional

TL;DR
Häufig hat man in Methoden den Fall, dass es keinen Wert gibt, und man liefert dann `null` als "kein Wert vorhanden" zurück. Dies führt dazu, dass die Aufrufer eine entsprechende `null`-Prüfung für die Rückgabewerte durchführen müssen, bevor sie das Ergebnis nutzen können.
`Optional` schließt elegant den Fall "kein Wert vorhanden" ein: Es kann mit der Methode `Optional.ofNullable()` das Argument in ein Optional verpacken (Argument != `null`) oder ein `Optional.empty()` zurückliefern ("leeres" Optional, wenn Argument == `null`).
Man kann Optionals prüfen mit `isEmpty()` und `ifPresent()` und dann direkt mit `ifPresent()`, `orElse()` und `orElseThrow()` auf den verpackten Wert zugreifen. Besser ist aber der Zugriff über die Stream-API von `Optional`: `map()`, `filter`, `flatMap()`, ...
`Optional` ist vor allem für Rückgabewerte gedacht, die den Fall "kein Wert vorhanden" einschließen sollen. Attribute, Parameter und Sammlungen sollten nicht `Optional`-Referenzen speichern, sondern "richtige" (unverpackte) Werte (und eben zur Not `null`). `Optional` ist kein Ersatz für `null`-Prüfung von Methoden-Parametern (nutzen Sie hier beispielsweise passende Annotationen). `Optional` ist auch kein Ersatz für vernünftiges Exception-Handling im Fall, dass etwas Unerwartetes passiert ist. Liefern Sie **niemals** `null` zurück, wenn der Rückgabetyp der Methode ein `Optional` ist!

Videos

- VL Optional
- Demo Optional
- Demo Optional: Beispiel aus der Praxis im PM-Dungeon

2.2.4.1 Motivation

```java
public class LSF {
    private Set<Studi> sl;

    public Studi getBestStudi() {
        if (sl == null) return null;  // Fehler: Es gibt noch keine Sammlung

        Studi best = null;
        for (Studi s : sl) {
            if (best == null) best = s;
            if (best.credits() < s.credits()) best = s;
        }
        return best;
    }
}

public static void main(String... args) {
    LSF lsf = new LSF();

    Studi best = lsf.getBestStudi();
    if (best != null) {
        String name = best.name();
        if (name != null) {
            // mach was mit dem Namen ...
        }
```

```
25          }
26      }
```

2.2.4.1.1 Problem: `null` wird an (zu) vielen Stellen genutzt

- Es gibt keinen Wert (“not found”)
- Felder wurden (noch) nicht initialisiert
- Es ist ein Problem oder etwas Unerwartetes aufgetreten

=> Parameter und Rückgabewerte müssen stets auf **`null`** geprüft werden (oder Annotationen wie `@NotNull` eingesetzt werden …)

2.2.4.1.2 Lösung

- `Optional<T>` für Rückgabewerte, die “kein Wert vorhanden” mit einschließen (statt **`null`** bei Abwesenheit von Werten)
- `@NotNull`/`@Nullable` für Parameter einsetzen (oder separate Prüfung)
- Exceptions werfen in Fällen, wo ein Problem aufgetreten ist

2.2.4.1.3 Anmerkungen

- Verwendung von **`null`** auf Attribut-Ebene (Klassen-interne Verwendung) ist okay!
- `Optional<T>` ist **kein** Ersatz für **`null`**-Checks!
- **`null`** ist **kein** Ersatz für vernünftiges Error-Handling! Das häufig zu beobachtende “Irgendwas Unerwartetes ist passiert, hier ist **`null`**” ist ein **Anti-Pattern**!

2.2.4.1.4 Beispiel aus der Praxis im PM-Dungeon Schauen Sie sich einmal das Review zu den `ecs.components.ai.AITools` in https://github.com/Dungeon-CampusMinden/Dungeon/pull/128#pullrequestreview-1254025874 an.

133 game/src/ecs/components/ai/AITools.java Viewed

```
+
+    /**
+     * Finds the path to a random (accessible) tile in the given radius, starting from the position
+     * of the given entity.
+     *
+     * @param entity Entity whose position is the center point
+     * @param radius Search radius
+     * @return Path from the position of the entity to the randomly selected tile
+     */
+    public static GraphPath<Tile> calculateNewPath(Entity entity, float radius) {
+        PositionComponent pc = (PositionComponent) entity.getComponent(PositionComponent.name);
+        VelocityComponent vc = (VelocityComponent) entity.getComponent(VelocityComponent.name);
+        if (pc != null && vc != null) {
+            ILevel level = ECS.currentLevel;
+            Point position = pc.getPosition();
+            List<Tile> tiles = new ArrayList<>();
+            for (float x = position.x - radius; x <= position.x + radius; x++) {
+                for (float y = position.y - radius; y <= position.y + radius; y++) {
+                    tiles.add(level.getTileAt(new Point(x, y).toCoordinate()));
+                }
+            }
+            tiles.removeIf(Objects::isNull);
+            tiles.removeIf(tile -> !tile.isAccessible());
+            Coordinate newPosition = tiles.get(random.nextInt(tiles.size())).getCoordinate();
+            return level.findPath(
+                    level.getTileAt(position.toCoordinate()), level.getTileAt(newPosition));
+        }
+        return null;
```

Die Methode `AITools#calculateNewPath` soll in der Umgebung einer als Parameter übergebenen Entität nach einem Feld (`Tile`) suchen, welches für die Entität betretbar ist und einen Pfad von der Position der Entität zu diesem Feld an den Aufrufer zurückliefern.

Zunächst wird in der Entität nach einer `PositionComponent` und einer `VelocityComponent` gesucht. Wenn es (eine) diese(r) Components nicht in der Entität gibt, wird der Wert **`null`** an den Aufrufer von `AITools#calculateNewPath` zurückgeliefert. (*Anmerkung*: Interessanterweise wird in der Methode nicht mit der `VelocityComponent` gearbeitet.)

Dann wird in der `PositionComponent` die Position der Entität im aktuellen Level abgerufen. In einer Schleife werden alle Felder im gegebenen Radius in eine Liste gespeichert. (*Anmerkung*: Da dies über die **`float`**-Werte passiert und nicht über die Feld-Indizes wird ein `Tile` u.U. recht oft in der Liste abgelegt. Können Sie sich hier einfache Verbesserungen überlegen?)

Da `level.getTileAt()` offenbar als Antwort auch **`null`** zurückliefern kann, werden nun zunächst per `tiles.removeIf(Objects::isNull);` all diese **`null`**-Werte wieder aus der Liste entfernt. Danach erfolgt die Prüfung, ob die verbleibenden Felder betretbar sind und nicht-betretbare Felder werden entfernt.

Aus den verbleibenden (betretbaren) Feldern in der Liste wird nun eines zufällig ausgewählt und per `level.findPath()` ein Pfad von der Position der Entität zu diesem Feld berechnet und zurückgeliefert. (*Anmerkung*: Hier wird ein zufälliges Tile in der Liste der umgebenden Felder gewählt, von diesem die Koordinaten bestimmt, und dann noch einmal aus dem Level das dazugehörige Feld geholt - dabei hatte man die Referenz auf das Feld bereits in der Liste. Können Sie sich hier eine einfache Verbesserung überlegen?)

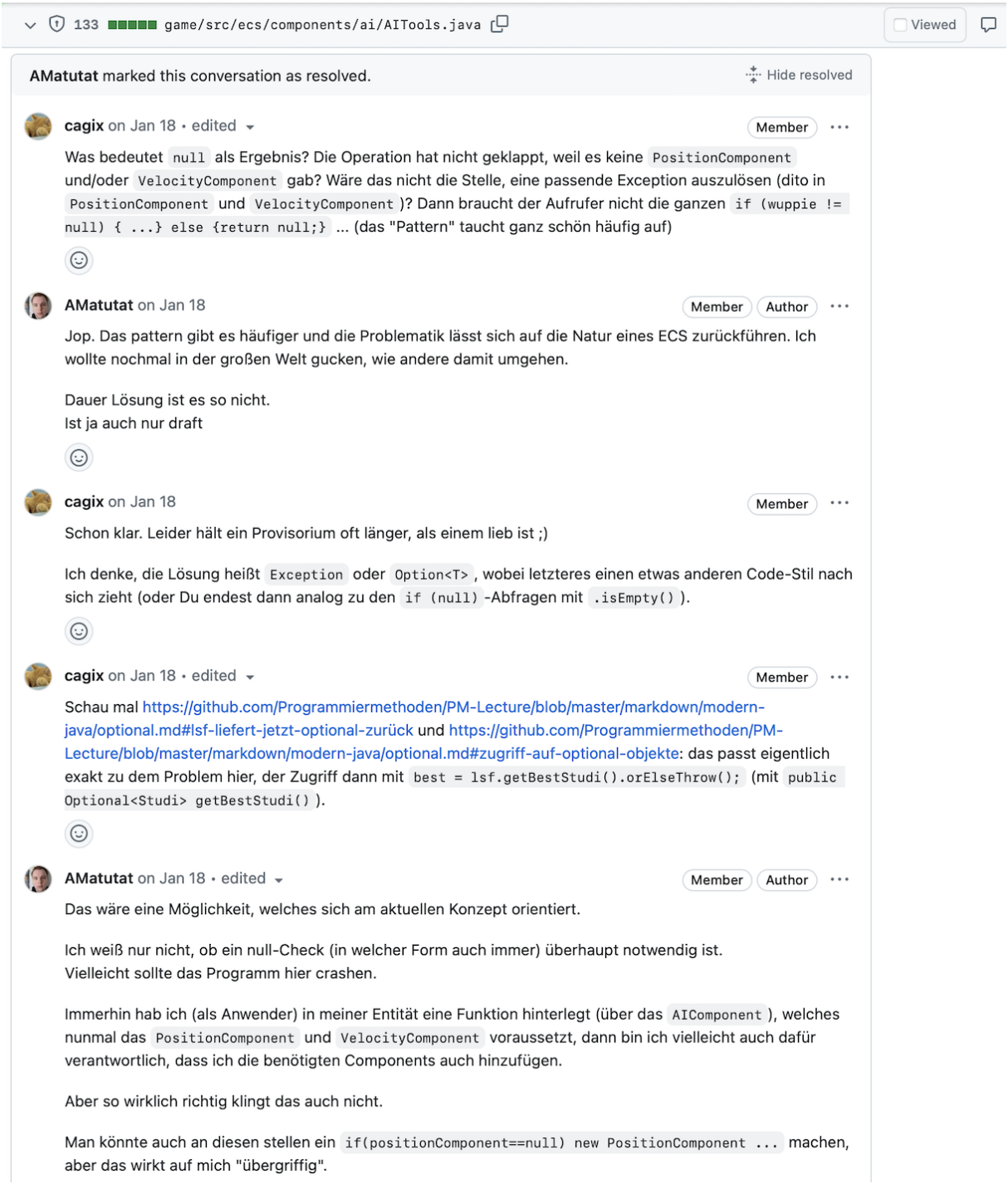
Zusammengefasst:

- Die als Parameter `entity` übergebene Referenz darf offenbar *nicht* **`null`** sein. Die ersten beiden Statements in der Methode rufen auf dieser Referenz Methoden auf, was bei einer **`null`**-Referenz zu einer `NullPointer`-Exception führen würde. Hier wäre **`null`** ein Fehlerzustand.
- `entity.getComponent()` kann offenbar **`null`** zurückliefern, wenn die gesuchte Component nicht vorhanden ist. Hier wird **`null`** als "kein Wert vorhanden" genutzt, was dann nachfolgende **`null`**-Checks notwendig macht.
- Wenn es die gewünschten Components nicht gibt, wird dem Aufrufer der Methode **`null`** zurückgeliefert. Hier ist nicht ganz klar, ob das einfach nur "kein Wert vorhanden" ist oder eigentlich ein Fehlerzustand?
- `level.getTileAt()` kann offenbar **`null`** zurückliefern, wenn kein Feld an der Position vorhanden ist. Hier wird **`null`** wieder als "kein Wert vorhanden" genutzt, was dann nachfolgende **`null`**-Checks notwendig macht (Entfernen aller **`null`**-Referenzen aus der Liste).
- `level.findPath()` kann auch wieder **`null`** zurückliefern, wenn kein Pfad berechnet werden konnte. Hier ist wieder nicht ganz klar, ob das einfach nur "kein Wert vorhanden" ist oder eigentlich ein Fehlerzustand? Man könnte beispielsweise in diesem Fall ein anderes Feld probieren?

Der Aufrufer bekommt also eine `NullPointer`-Exception, wenn der übergebene Parameter `entity` nicht vorhanden ist oder den Wert **`null`**, wenn in der Methode etwas schief lief oder schlicht kein Pfad berechnet werden konnte oder tatsächlich einen Pfad. Damit wird der Aufrufer gezwungen, den Rückgabewert vor der Verwendung zu untersuchen.

Allein in dieser einen kurzen Methode macht `null` so viele extra Prüfungen notwendig und den Code dadurch schwerer lesbar und fehleranfälliger! `null` wird als (unvollständige) Initialisierung und als Rückgabewert und für den Fehlerfall genutzt, zusätzlich ist die Semantik von `null` nicht immer klar. (*Anmerkung*: Der Gebrauch von **`null`** hat nicht wirklich etwas mit "der Natur eines ECS" zu tun. Die Methode wurde mittlerweile komplett überarbeitet und ist in der hier gezeigten Form glücklicherweise nicht mehr zu finden.)

Entsprechend hat sich in diesem Review die nachfolgende Diskussion ergeben:



133 game/src/ecs/components/ai/AITools.java

Viewed

AMatutat marked this conversation as resolved. Hide resolved

cagix on Jan 18 • edited — Member

Was bedeutet `null` als Ergebnis? Die Operation hat nicht geklappt, weil es keine `PositionComponent` und/oder `VelocityComponent` gab? Wäre das nicht die Stelle, eine passende Exception auszulösen (dito in `PositionComponent` und `VelocityComponent`)? Dann braucht der Aufrufer nicht die ganzen `if (wuppie != null) { ...} else {return null;}` ... (das "Pattern" taucht ganz schön häufig auf)

AMatutat on Jan 18 — Member, Author

Jop. Das pattern gibt es häufiger und die Problematik lässt sich auf die Natur eines ECS zurückführen. Ich wollte nochmal in der großen Welt gucken, wie andere damit umgehen.

Dauer Lösung ist es so nicht.
Ist ja auch nur draft

cagix on Jan 18 — Member

Schon klar. Leider hält ein Provisorium oft länger, als einem lieb ist ;)

Ich denke, die Lösung heißt `Exception` oder `Option<T>`, wobei letzteres einen etwas anderen Code-Stil nach sich zieht (oder Du endest dann analog zu den `if (null)`-Abfragen mit `.isEmpty()`).

cagix on Jan 18 • edited — Member

Schau mal https://github.com/Programmiermethoden/PM-Lecture/blob/master/markdown/modern-java/optional.md#lsf-liefert-jetzt-optional-zurück und https://github.com/Programmiermethoden/PM-Lecture/blob/master/markdown/modern-java/optional.md#zugriff-auf-optional-objekte: das passt eigentlich exakt zu dem Problem hier, der Zugriff dann mit `best = lsf.getBestStudi().orElseThrow();` (mit `public Optional<Studi> getBestStudi()`).

AMatutat on Jan 18 • edited — Member, Author

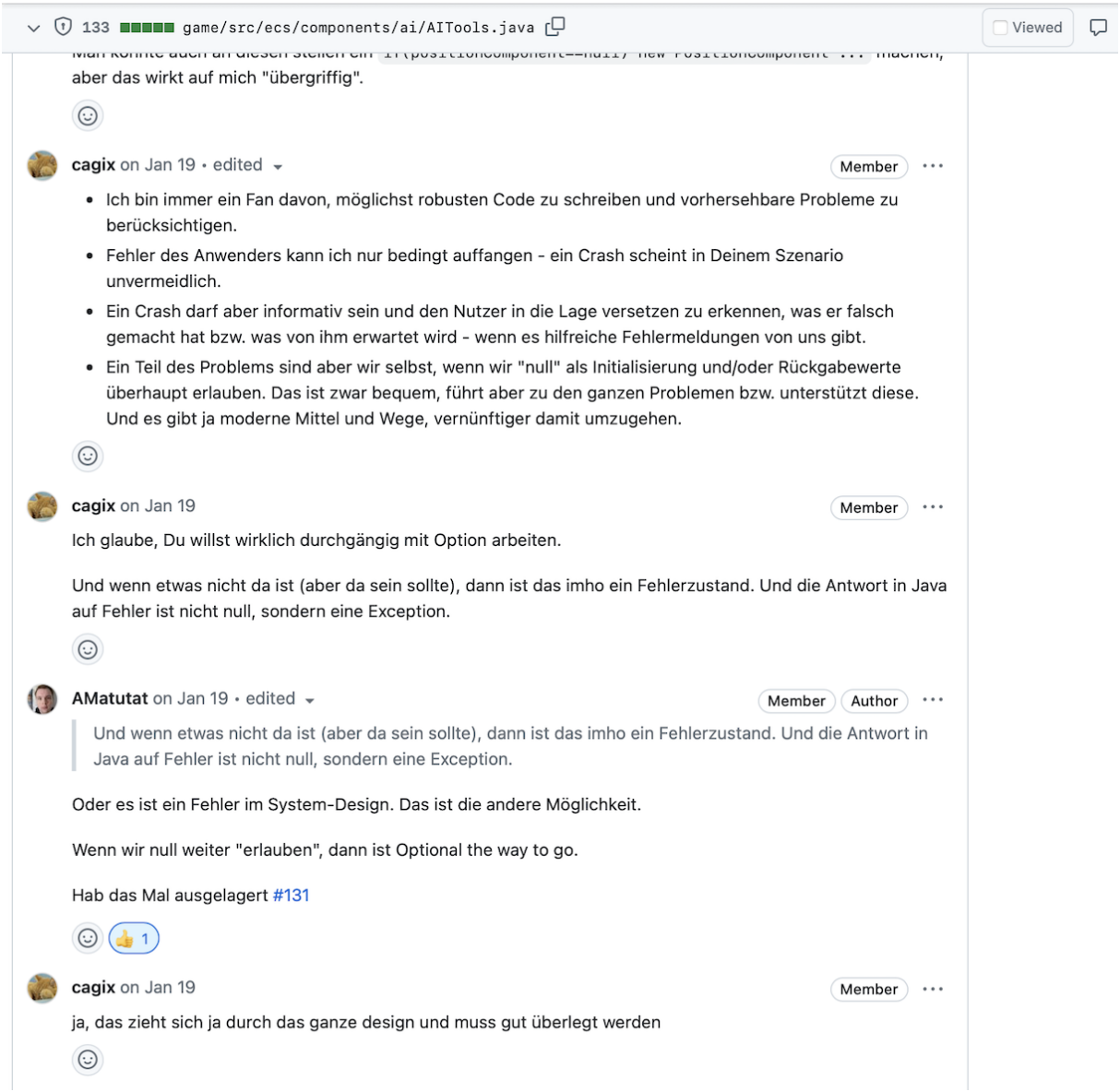
Das wäre eine Möglichkeit, welches sich am aktuellen Konzept orientiert.

Ich weiß nur nicht, ob ein null-Check (in welcher Form auch immer) überhaupt notwendig ist.
Vielleicht sollte das Programm hier crashen.

Immerhin hab ich (als Anwender) in meiner Entität eine Funktion hinterlegt (über das `AIComponent`), welches nunmal das `PositionComponent` und `VelocityComponent` voraussetzt, dann bin ich vielleicht auch dafür verantwortlich, dass ich die benötigten Components auch hinzufügen.

Aber so wirklich richtig klingt das auch nicht.

Man könnte auch an diesen stellen ein `if(positionComponent==null) new PositionComponent ...` machen, aber das wirkt auf mich "übergriffig".



2.2.4.2 Erzeugen von *Optional*-Objekten

Konstruktor ist `private` …

- “Kein Wert”: `Optional.empty()`
- Verpacken eines non-`null` Elements: `Optional.of()`
  (`NullPointerException` wenn Argument `null`!)

- Verpacken eines “unsicheren”/beliebigen Elements: `Optional.ofNullable()`
  - Liefert verpacktes Element, oder
  - `Optional.empty()`, falls Element `null` war

Es sollte in der Praxis eigentlich nur wenige Fälle geben, wo ein Aufruf von `Optional.of()` sinnvoll ist. Ebenso ist `Optional.empty()` nur selten sinnvoll.

Stattdessen sollte stets `Optional.ofNullable()` verwendet werden.

`null` kann nicht nicht in `Optional<T>` verpackt werden! (Das wäre dann eben `Optional.empty()`.)

2.2.4.3 LSF liefert jetzt *Optional* zurück

```java
public class LSF {
    private Set<Studi> sl;

    public Optional<Studi> getBestStudi() throws NullPointerException {
        // Fehler: Es gibt noch keine Sammlung
        if (sl == null) throw new NullPointerException("There ain't any
            collection");

        Studi best = null;
        for (Studi s : sl) {
            if (best == null) best = s;
            if (best.credits() < s.credits()) best = s;
        }

        // Entweder Optional.empty() (wenn best==null) oder Optional.of(best)
            sonst
        return Optional.ofNullable(best);
    }
}
```

Das Beispiel soll verdeutlichen, dass man im Fehlerfall nicht einfach **`null`** oder `Optional.empty()` zurückliefern soll, sondern eine passende Exception werfen soll.

Wenn die Liste aber leer ist, stellt dies keinen Fehler dar! Es handelt sich um den Fall “kein Wert vorhanden”. In diesem Fall wird statt **`null`** nun ein `Optional.empty()` zurückgeliefert, also ein Objekt, auf dem der Aufrufer die üblichen Methoden aufrufen kann.

2.2.4.4 Zugriff auf *Optional*-Objekte In der funktionalen Programmierung gibt es schon lange das Konzept von `Optional`, in Haskell ist dies beispielsweise die Monade `Maybe`. Allerdings ist die Einbettung in die Sprache von vornherein mit berücksichtigt worden, insbesondere kann man hier sehr gut mit *Pattern Matching* in der Funktionsdefinition auf den verpackten Inhalt reagieren.

In Java gibt es die Methode `Optional#isEmpty()`, die einen Boolean zurückliefert und prüft, ob es sich um ein leeres `Optional` handelt oder ob hier ein Wert "verpackt" ist.

Für den direkten Zugriff auf die Werte gibt es die Methoden `Optional#orElseThrow()` und `Optional#orElse()`. Damit kann man auf den verpackten Wert zugreifen, oder es wird eine Exception geworfen bzw. ein Ersatzwert geliefert.

Zusätzlich gibt es `Optional#isPresent()`, die als Parameter ein `java.util.function.Consumer` erwartet, also ein funktionales Interface mit einer Methode `void accept(T)`, die das Objekt verarbeitet.

```
Studi best;

// Testen und dann verwenden
if (!lsf.getBestStudi().isEmpty()) {
    best = lsf.getBestStudi().get();
    // mach was mit dem Studi ...
}

// Arbeite mit Consumer
lsf.getBestStudi().ifPresent(studi -> {
    // mach was mit dem Studi ...
});

// Studi oder Alternative (wenn Optional.empty())
best = lsf.getBestStudi().orElse(anne);

// Studi oder NoSuchElementException (wenn Optional.empty())
best = lsf.getBestStudi().orElseThrow();
```

Es gibt noch eine Methode `get()`, die so verhält wie `orElseThrow()`. Da man diese Methode vom Namen her schnell mit einem Getter verwechselt, ist sie mittlerweile *deprecated*.

Anmerkung: Da `getBestStudi()` eine `NullPointerException` werfen kann, sollte der Aufruf möglicherweise in ein `try`/`catch` verpackt werden. Dito für `orElseThrow()`.

Beispiel: optional.traditional.Demo

2.2.4.5 Einsatz mit Stream-API

```
public class LSF {
    ...
    public Optional<Studi> getBestStudi() throws NullPointerException {
        if (sl == null) throw new NullPointerException("There ain't any
            collection");
        return sl.stream()
                 .sorted((s1, s2) -> s2.credits() - s1.credits())
                 .findFirst();
    }
}
```

```
11
12 public static void main(String... args) {
13     ...
14     String name = lsf.getBestStudi()
15                      .map(Studi::name)
16                      .orElseThrow();
17 }
```

Beispiel: optional.streams.Demo

Im Beispiel wird in `getBestStudi()` die Sammlung als Stream betrachtet, über die Methode `sorted()` und den Lamda-Ausdruck für den `Comparator` sortiert ("falsch" herum: absteigend in den Credits der Studis in der Sammlung), und `findFirst()` ist die terminale Operation auf dem Stream, die ein `Optional<Studi>` zurückliefert: entweder den Studi mit den meisten Credits (verpackt in `Optional<Studi>`) oder `Optional.empty()`, wenn es überhaupt keine Studis in der Sammlung gab.

In `main()` wird dieses `Optional<Studi>` mit den Stream-Methoden von `Optional<T>` bearbeitet, zunächst mit `Optional#map()`. Man braucht nicht selbst prüfen, ob das von `getBestStudi()` erhaltene Objekt leer ist oder nicht, da dies von `Optional#map()` erledigt wird: Es wendet die Methodenreferenz auf den verpackten Wert an (sofern dieser vorhanden ist) und liefert damit den Namen des Studis als `Optional<String>` verpackt zurück. Wenn es keinen Wert, also nur `Optional.empty()` von `getBestStudi()` gab, dann ist der Rückgabewert von `Optional#map()` ein `Optional.empty()`. Wenn der Name, also der Rückgabewert von `Studi::name`, **`null`** war, dann wird ebenfalls ein `Optional.empty()` zurückgeliefert. Dadurch wirft `orElseThrow()` dann eine `NoSuchElementException`. Man kann also direkt mit dem String `name` weiterarbeiten ohne extra **`null`**-Prüfung - allerdings will man noch ein Exception-Handling einbauen (dies fehlt im obigen Beispiel aus Gründen der Übersicht) …

2.2.4.6 Weitere *Optionals*

Für die drei primitiven Datentypen **`int`**, **`long`** und **`double`** gibt es passende Wrapper-Klassen von `Optional<T>`: `OptionalInt`, `OptionalLong` und `OptionalDouble`.

Diese verhalten sich analog zu `Optional<T>`, haben aber keine Methode `ofNullable()`, da dies hier keinen Sinn ergeben würde: Die drei primitiven Datentypen repräsentieren Werte - diese können nicht **`null`** sein.

2.2.4.7 Regeln für *Optional*

1. Nutze `Optional` nur als Rückgabe für "kein Wert vorhanden"

   `Optional` ist nicht als Ersatz für eine **`null`**-Prüfung o.ä. gedacht, sondern als Repräsentation, um auch ein "kein Wert vorhanden" zurückliefern zu können.

2. Nutze nie **`null`** für eine `Optional`-Variable oder einen `Optional`-Rückgabewert

   Wenn man ein `Optional` als Rückgabe bekommt, sollte das niemals selbst eine **`null`**-Referenz sein. Das macht das gesamte Konzept kaputt!

   Nutzen Sie stattdessen `Optional.empty()`.

3. Nutze `Optional.ofNullable()` zum Erzeugen eines `Optional`

Diese Methode verhält sich "freundlich" und erzeugt automatisch ein `Optional.empty()`, wenn das Argument **`null`** ist. Es gibt also keinen Grund, dies mit einer Fallunterscheidung selbst erledigen zu wollen.

Bevorzugen Sie `Optional.ofNullable()` vor einer manuellen Fallunterscheidung und dem entsprechenden Einsatz von `Optional.of()` und `Optional.empty()`.

4. Erzeuge keine `Optional` als Ersatz für die Prüfung auf **`null`**

   Wenn Sie auf **`null`** prüfen müssen, müssen Sie auf **`null`** prüfen. Der ersatzweise Einsatz von `Optional` macht es nur komplexer - prüfen müssen Sie hinterher ja immer noch.

5. Nutze `Optional` nicht in Attributen, Methoden-Parametern und Sammlungen

   Nutzen Sie `Optional` vor allem für Rückgabewerte.

   Attribute sollten immer direkt einen Wert haben oder **`null`**, analog Parameter von Methoden o.ä. … Hier hilft `Optional` nicht, Sie müssten ja trotzdem eine **`null`**-Prüfung machen, nur eben dann über den `Optional`, wodurch dies komplexer und schlechter lesbar wird.

   Aus einem ähnlichen Grund sollten Sie auch in Sammlungen keine `Optional` speichern!

6. Vermeide den direkten Zugriff (`ifPresent()`, `orElseThrow()` …)

   Der direkte Zugriff auf ein `Optional` entspricht dem Prüfen auf **`null`** und dann dem Auspacken. Dies ist nicht nur Overhead, sondern auch schlechter lesbar.

   Vermeiden Sie den direkten Zugriff und nutzen Sie `Optional` mit den Stream-Methoden. So ist dies von den Designern gedacht.

2.2.4.8 Interessante Links

- "Using Optionals"
- "What You Might Not Know About Optional"
- "Experienced Developers Use These 7 Java Optional Tips to Remove Code Clutter"
- "Code Smells: Null"
- "Class Optional"

2.2.4.9 Wrap-Up `Optional` als Rückgabe für "kein Wert vorhanden"

- `Optional.ofNullable()`: Erzeugen eines `Optional`
  - Entweder Objekt "verpackt" (Argument != **`null`**)
  - Oder `Optional.empty()` (Argument == **`null`**)
- Prüfen mit `isEmpty()` und `ifPresent()`
- Direkter Zugriff mit `ifPresent()`, `orElse()` und `orElseThrow()`
- Stream-API: `map()`, `filter()`, `flatMap()`, …

- Attribute, Parameter und Sammlungen: nicht `Optional` nutzen

- Kein Ersatz für `null`-Prüfung!

Schöne Doku: "Using Optionals".

Zum Nachlesen

- Oracle Corporation (2022b)
- Ullenboom (2021, 12.6)

Lernziele

- k2: Optionals sind kein Ersatz für null-Prüfung!
- k2: Optionals sollen nicht für Attribute oder Parameter genutzt werden
- k2: Es darf kein null zurückgeliefert werden, wenn der Rückgabetyp ein Optional ist
- k2: Optionals und null sind kein Ersatz für Exception-Handling
- k3: Einsatz von Optional in Rückgabewerten
- k3: Erzeugen von Optionals mit Optional.ofNullable()
- k3: Zugriff auf Optionals entweder direkt oder per Stream-API

Quizzes

- Quiz Optional (ILIAS)

Challenges

Katzen-Café

In den Vorgaben finden Sie eine Implementierung für ein Katzencafé.
Verändern Sie die Vorgaben so, dass möglich wenig `null` verwendet wird. Setzen Sie dazu gezielt und sinnvoll Exception-Handling und `Optional<T>` ein. Ergänzen Sie die Vorgaben um ein ausführliches Beispiel und bevölkern Sie das Café mit verschiedenen Katzen und geben Sie diese mit Hilfe der verschiedenen Methoden aus.
Begründen Sie die Relevanz der verbleibenden `null`-Vorkommen im Code.

String-Handling

Können Sie den folgenden Code so umschreiben, dass Sie statt der `if`-Abfragen und der einzelnen direkten Methodenaufrufe die Stream-API und `Optional<T>` nutzen?

```
1  String format(final String text, String replacement) {
2      if (text.isEmpty()) {
3          return "";
4      }
5
6      final String trimmed = text.trim();
7      final String withSpacesReplaced = trimmed.replaceAll(" +", replacement
           );
8
9      return replacement + withSpacesReplaced + replacement;
10 }
```

Ein Aufruf `format("Hello World ... ", "_");` liefert den String "_Hello_World_..._".

2.2.5 Record-Klassen

TL;DR

Häufig schreibt man relativ viel *Boiler Plate Code*, um einfach ein paar Daten plus den Konstruktor und die Zugriffsmethoden zu kapseln. Und selbst wenn die IDE dies zum Teil abnehmen kann - lesen muss man diesen Overhead trotzdem noch.

Für den Fall von Klassen mit `final` Attributen wurden in Java14 die **Record-Klassen** eingeführt. Statt dem Schlüsselwort `class` wird das neue Schlüsselwort `record` verwendet. Nach dem Klassennamen kommen in runden Klammern die "Komponenten" - eine Auflistung der Parameter für den Standardkonstruktor (Typ, Name). Daraus wird automatisch ein "kanonischer Konstruktor" mit exakt diesen Parametern generiert. Es werden zusätzlich `private final` Attribute generiert für jede Komponente, und diese werden durch den kanonischen Konstruktor gesetzt. Außerdem wird für jedes Attribut automatisch ein Getter mit dem Namen des Attributs generiert (also ohne den Präfix "get").

Beispiel:

```
1  public record StudiR(String name, int credits) {}
```

Der Konstruktor und die Getter können überschrieben werden, es können auch eigene Methoden definiert werden (eigene Konstruktoren *müssen* den kanonischen Konstruktor aufrufen). Es gibt außer den über die Komponenten definierten Attribute keine weiteren Attribute. Da eine Record-Klasse intern von `java.lang.Record` ableitet, kann eine Record-Klasse nicht von weiteren Klassen ableiten (erben). Man kann aber beliebig viele Interfaces implementieren. Record-Klassen sind implizit final, d.h. man nicht von Record-Klassen erben.

Videos

- VL Record-Klassen
- Demo Record-Klassen

2.2.5.1 Motivation; Klasse Studi

```
1  public class Studi {
2      private final String name;
3      private final int credits;
4
5      public Studi(String name, int credits) {
6          this.name = name;
7          this.credits = credits;
8      }
9
10     public String getName() {
11         return name;
12     }
13
14     public int getCredits() {
15         return credits;
16     }
17 }
```

2.2.5.2 Klasse Studi als Record

```
1  public record StudiR(String name, int credits) {}
```

- Immutable Klasse mit Feldern `String name` und `int credits`
  => "`(String name, int credits)`" werden "Komponenten" des Records genannt
- Standardkonstruktor setzt diese Felder ("Kanonischer Konstruktor")
- Getter für beide Felder:

```
1  public String name() { return this.name; }
2  public int credits() { return this.credits; }
```

Record-Klassen wurden in Java14 eingeführt und werden immer wieder in neuen Releases erweitert/ergänzt.

Der kanonische Konstruktor hat das Aussehen wie die Record-Deklaration, im Beispiel also `public StudiR(String name, int credits)`. Dabei werden die Komponenten über eine Kopie der Werte initialisiert.

Für die Komponenten werden automatisch private Attribute mit dem selben Namen angelegt.

Für die Komponenten werden automatisch Getter angelegt. Achtung: Die Namen entsprechen denen der Komponenten, es fehlt also der übliche "get"-Präfix!

2.2.5.3 Eigenschaften und Einschränkungen von Record-Klassen

- Records erweitern implizit die Klasse `java.lang.Record`:
  Keine andere Klassen mehr erweiterbar! (Interfaces kein Problem)
- Record-Klassen sind implizit final
- Keine weiteren (Instanz-) Attribute definierbar (nur die Komponenten)
- Keine Setter definierbar für die Komponenten: Attribute sind final
- Statische Attribute mit Initialisierung erlaubt

2.2.5.4 Records: Prüfungen im Konstruktor

Der Konstruktor ist erweiterbar:

```
1  public record StudiS(String name, int credits) {
2      public StudiS(String name, int credits) {
3          if (name == null) { throw new IllegalArgumentException("Name cannot be
               null!"); }
4          else { this.name = name; }
5
6          if (credits < 0) { this.credits = 0; }
7          else { this.credits = credits; }
8      }
9  }
```

In dieser Form muss man die Attribute selbst setzen.

Alternativ kann man die "kompakte" Form nutzen:

```
1  public record StudiT(String name, int credits) {
2      public StudiT {
3          if (name == null) { throw new IllegalArgumentException("Name cannot be
               null!"); }
```

```
4
5           if (credits < 0) { credits = 0; }
6       }
7   }
```

In der kompakten Form kann man nur die Werte der Parameter des Konstruktors ändern. Das Setzen der Attribute ergänzt der Compiler nach dem eigenen Code.

Es sind weitere Konstruktoren definierbar, diese *müssen* den kanonischen Konstruktor aufrufen:

```
1  public StudiT() {
2      this("", 42);
3  }
```

2.2.5.5 Getter und Methoden Getter werden vom Compiler automatisch generiert. Dabei entsprechen die Methoden-Namen den Namen der Attribute:

```
1  public record StudiR(String name, int credits) {}
2
3  public static void main(String... args) {
4      StudiR r = new StudiR("Sabine", 75);
5
6      int x = r.credits();
7      String y = r.name();
8  }
```

Getter überschreibbar und man kann weitere Methoden definieren:

```
1  public record StudiT(String name, int credits) {
2      public int credits() { return credits + 42; }
3      public void wuppie() { System.out.println("WUPPIE"); }
4  }
```

Die Komponenten/Attribute sind aber `final` und können nicht über Methoden geändert werden!

2.2.5.6 Beispiel aus den Challenges In den Challenges zum Thema Optional gibt es die Klasse `Katze` in den Vorgaben.

Die Katze wurde zunächst “klassisch” modelliert: Es gibt drei Eigenschaften `name`, `gewicht`und `lieblingsBox`. Ein Konstruktor setzt diese Felder und es gibt drei Getter für die einzelnen Eigenschaften. Das braucht 18 Zeilen Code (ohne Kommentare Leerzeilen). Zudem erzeugt der Boilerplate-Code relativ viel “visual noise”, so dass der eigentliche Kern der Klasse schwerer zu erkennen ist.

In einem Refactoring wurde diese Klasse durch eine äquivalente Record-Klasse ersetzt, die nur noch 2 Zeilen Code (je nach Code-Style auch nur 1 Zeile) benötigt. Gleichzeitig wurde die Les- und Wartbarkeit deutlich verbessert.

33 markdown/modern-java/src/challenges/optional/Katze.java Viewed

```
@@ -1,30 +1,9 @@
 package challenges.optional;

-public class Katze {
-    private String name;
-    private float gewicht;
-    private Box lieblingsBox;
-
-    /**
-     * @param name Name der Katze
-     * @param gewicht Gewicht der Katze in kg
-     * @param lieblingsBox Lieblingsbox der Katze
-     */
-    public Katze(String name, float gewicht, Box lieblingsBox) {
-        this.name = name;
-        this.gewicht = gewicht;
-        this.lieblingsBox = lieblingsBox;
-    }
-
-    public String name() {
-        return name;
-    }
-
-    public float gewicht() {
-        return gewicht;
-    }
-
-    public Box lieblingsBox() {
-        return lieblingsBox;
-    }
+/**
+ * @param name         Name der Katze
+ * @param gewicht      Gewicht der Katze in kg
+ * @param lieblingsBox Lieblingsbox der Katze
+ */
+public record Katze(String name, float gewicht, Box lieblingsBox) {
 }
```

2.2.5.7 Wrap-Up

- Records sind immutable Klassen:
  - `final` Attribute (entsprechend den Komponenten)
  - Kanonischer Konstruktor
  - Automatische Getter (Namen wie Komponenten)
- Konstruktoren und Methoden können ergänzt/überschrieben werden
- Keine Vererbung von Klassen möglich (kein `extends`)

Schöne Doku: “Using Record to Model Immutable Data”.

Zum Nachlesen

- Oracle Corporation (2022b)

Lernziele

- k2: Record-Klassen sind final
- k2: Record-Klassen haben einen kanonischen Konstruktor
- k2: Die Attribute von Record-Klassen sind final und werden automatisch angelegt und über den Konstruktor gesetzt
- k2: Die Getter in Record-Klassen haben die Namen und Typen der Komponenten, also keinen Präfix 'get'
- k2: Der kanonische Konstruktor kann ergänzt werden
- k2: Es können weitere Methoden definiert werden
- k2: Record-Klassen können nicht von anderen Klassen erben, können aber Interfaces implementieren
- k3: Einsatz von Record-Klassen

Quizzes

- Quiz Record-Klassen (ILIAS)

2.2.6 Interfaces: Default-Methoden

TL;DR
Seit Java8 können Methoden in Interfaces auch fertig implementiert sein: Sogenannte **Default-Methoden**.
Dazu werden die Methoden mit dem neuen Schlüsselwort `default` gekennzeichnet. Die Implementierung wird an die das Interface implementierenden Klassen (oder Interfaces) vererbt und kann bei Bedarf überschrieben werden.
Da eine Klasse von einer anderen Klasse erben darf, aber mehrere Interfaces implementieren kann, könnte es zu einer Mehrfachvererbung einer Methode kommen: Eine Methode könnte beispielsweise in verschiedenen Interfaces als Default-Methode angeboten werden, und wenn eine Klasse diese Interfaces implementiert, steht eine Methode mit der selben Signatur auf einmal mehrfach zur Verfügung. Dies muss (u.U. manuell) aufgelöst werden.
Auflösung von Mehrfachvererbung:

- Regel 1: Klassen gewinnen
- Regel 2: Sub-Interfaces gewinnen
- Regel 3: Methode explizit auswählen

Aktuell ist der Unterschied zu abstrakten Klassen: Interfaces können **keinen Zustand** haben, d.h. keine Attribute/Felder.

Videos

- VL Default-Methoden
- Demo Regel 1
- Demo Regel 2
- Demo Regel 3

2.2.6.1 Problem: Etablierte API (Interfaces) erweitern

```java
interface Klausur {
    void anmelden(Studi s);
    void abmelden(Studi s);
}
```

=> Nachträglich noch `void schreiben(Studi s);` ergänzen?

Wenn ein Interface nachträglich erweitert wird, müssen alle Kunden (also alle Klassen, die das Interface implementieren) auf die neuen Signaturen angepasst werden. Dies kann viel Aufwand verursachen und API-Änderungen damit unmöglich machen.

2.2.6.2 Default-Methoden: Interfaces mit Implementierung

Seit Java8 können Interfaces auch Methoden implementieren. Es gibt zwei Varianten: Default-Methoden und statische Methoden.

```
1  interface Klausur {
2      void anmelden(Studi s);
3      void abmelden(Studi s);
4
5      default void schreiben(Studi s) {
6          ...     // Default-Implementierung
7      }
8
9      default void wuppie() {
10         throw new java.lang.UnsupportedOperationException();
11     }
12 }
```

Methoden können in Interfaces seit Java8 implementiert werden. Für Default-Methoden muss das Schlüsselwort `default` vor die Signatur gesetzt werden. Klassen, die das Interface implementieren, können diese Default-Implementierung erben oder selbst neu implementieren (überschreiben). Alternativ kann die Klasse eine Default-Methode neu *deklarieren* und wird damit zur abstrakten Klasse.

Dies ähnelt abstrakten Klassen. Allerdings kann in abstrakten Klassen neben dem Verhalten (implementierten Methoden) auch Zustand über die Attribute gespeichert werden.

2.2.6.3 Problem: Mehrfachvererbung

Drei Regeln zum Auflösen bei Konflikten:

1. **Klassen gewinnen**: Methoden aus Klasse oder Superklasse haben höhere Priorität als Default-Methoden
2. **Sub-Interfaces gewinnen**: Methode aus am meisten spezialisiertem Interface mit Default-Methode wird gewählt Beispiel: Wenn `B extends A` dann ist `B` spezialisierter als `A`
3. Sonst: Klasse muss **Methode explizit auswählen**: Methode überschreiben und gewünschte (geerbte) Variante aufrufen: `X.super.m(...)` (`X` ist das gewünschte Interface)

Auf den folgenden Folien wird dies anhand kleiner Beispiele verdeutlicht.

2.2.6.4 Auflösung Mehrfachvererbung: 1. Klassen gewinnen

```
1  interface A {
2      default String hello() { return "A"; }
3  }
4  class C {
5      public String hello() { return "C"; }
6  }
7  class E extends C implements A {}
8
9
10 /** Mehrfachvererbung: 1. Klassen gewinnen */
```

```java
public class DefaultTest1 {
    public static void main(String... args) {
        String e = new E().hello();
    }
}
```

Demo: defaultmethods.rule1.DefaultTest1

Die Klasse E erbt sowohl von Klasse C als auch vom Interface A die Methode `hello()` (Mehrfachvererbung). In diesem Fall "gewinnt" die Implementierung aus Klasse C.

1. Regel: Klassen gewinnen immer. Deklarationen einer Methode in einer Klasse oder einer Oberklasse haben Vorrang von allen Default-Methoden.

2.2.6.5 Auflösung Mehrfachvererbung: 2. Sub-Interfaces gewinnen

```java
interface A {
    default String hello() { return "A"; }
}
interface B extends A {
    @Override default String hello() { return "B"; }
}
class D implements A, B {}


/** Mehrfachvererbung: 2. Sub-Interfaces gewinnen */
public class DefaultTest2 {
    public static void main(String... args) {
        String e = new D().hello();
    }
}
```

Demo: defaultmethods.rule2.DefaultTest2

Die Klasse D erbt sowohl vom Interface A als auch vom Interface B die Methode `hello()` (Mehrfachvererbung). In diesem Fall "gewinnt" die Implementierung aus Klasse B: Interface B ist spezialisierter als A.

2. Regel: Falls Regel 1 nicht zutrifft, gewinnt die Default-Methode, die am meisten spezialisiert ist.

2.2.6.6 Auflösung Mehrfachvererbung: 3. Methode explizit auswählen

```java
interface A {
    default String hello() { return "A"; }
}
interface B {
    default String hello() { return "B"; }
}
class D implements A, B {
    @Override public String hello() { return A.super.hello(); }
}


/** Mehrfachvererbung: 3. Methode explizit auswählen */
public class DefaultTest3 {
    public static void main(String... args) {
        String e = new D().hello();
    }
```

```
17 }
```

Demo: defaultmethods.rule3.DefaultTest3

Die Klasse D erbt sowohl vom Interface A als auch vom Interface B die Methode `hello()` (Mehrfachvererbung). In diesem Fall *muss* zur Auflösung die Methode in D neu implementiert werden und die gewünschte geerbte Methode explizit aufgerufen werden. (Wenn dies unterlassen wird, führt das selbst bei Nicht-Nutzung der Methode `hello()` zu einem Compiler-Fehler!)

Achtung: Der Aufruf der Default-Methode aus Interface A erfolgt mit `A.super.hello();` (nicht einfach durch `A.hello();`)!

3. Regel: Falls weder Regel 1 noch 2 zutreffen bzw. die Auflösung noch uneindeutig ist, muss man manuell durch die explizite Angabe der gewünschten Methode auflösen.

2.2.6.7 Quiz: Was kommt hier raus?

```
1  interface A {
2      default String hello() { return "A"; }
3  }
4  interface B extends A {
5      @Override default String hello() { return "B"; }
6  }
7  class C implements B {
8      @Override public String hello() { return "C"; }
9  }
10 class D extends C implements A, B {}
11
12
13 /** Quiz Mehrfachvererbung */
14 public class DefaultTest {
15     public static void main(String... args) {
16         String e = new D().hello(); // ???
17     }
18 }
```

Die Klasse D erbt sowohl von Klasse C als auch von den Interfaces A und B die Methode `hello()` (Mehrfachvererbung). In diesem Fall "gewinnt" die Implementierung aus Klasse C: Klassen gewinnen immer (Regel 1).

Beispiel: defaultmethods.quiz.DefaultTest

2.2.6.8 Statische Methoden in Interfaces

```
1  public interface Collection<E> extends Iterable<E> {
2      boolean add(E e);
3      ...
4  }
5  public class Collections {
6      private Collections() { }
7      public static <T> boolean addAll(Collection<? super T> c, T... elements)
           {...}
8      ...
9  }
```

Typisches Pattern in Java: Interface plus Utility-Klasse (Companion-Klasse) mit statischen Hilfsmethoden zum einfacheren Umgang mit Instanzen des Interfaces (mit Objekten, deren Klasse das Interface implementiert).

Beispiel: `Collections` ist eine Hilfs-Klasse zum Umgang mit `Collection`-Objekten.

Seit Java8 können in Interfaces neben Default-Methoden auch statische Methoden implementiert werden.

Die Hilfsmethoden können jetzt ins Interface wandern => Utility-Klassen werden obsolet … Aus Kompatibilitätsgründen würde man die bisherige Companion-Klasse weiterhin anbieten, wobei die Implementierungen auf die statischen Methoden im Interface verweisen (*SKIZZE, nicht real!*):

```
1  public interface CollectionX<E> extends Iterable<E> {
2      boolean add(E e);
3      static <T> boolean addAll(CollectionX<? super T> c, T... elements) { ... }
4      ...
5  }
6  public class CollectionsX {
7      public static <T> boolean addAll(CollectionX<? super T> c, T... elements) {
8          return CollectionX.addAll(c, elements);  // Verweis auf Interface
9      }
10     ...
11 }
```

2.2.6.9 Interfaces vs. Abstrakte Klassen

- **Abstrakte Klassen**: Schnittstelle und Verhalten und Zustand
- **Interfaces**:
  - vor Java 8 nur Schnittstelle
  - ab Java 8 Schnittstelle und Verhalten

  Unterschied zu abstrakten Klassen: Kein Zustand, d.h. keine Attribute

- Design:
  - Interfaces sind beinahe wie abstrakte Klassen, nur ohne Zustand
  - Klassen können nur von **einer** (abstrakten) Klasse erben, aber **viele** Interfaces implementieren

2.2.6.10 Wrap-Up Seit Java8: Interfaces mit Implementierung: **Default-Methoden**

- Methoden mit dem Schlüsselwort **`default`** können Implementierung im Interface haben
- Die Implementierung wird vererbt und kann bei Bedarf überschrieben werden
- Auflösung von Mehrfachvererbung:
  - Regel 1: Klassen gewinnen
  - Regel 2: Sub-Interfaces gewinnen
  - Regel 3: Methode explizit auswählen
- Unterschied zu abstrakten Klassen: **Kein Zustand**

Zum Nachlesen

- Oracle Corporation (2022c)
- Urma u. a. (2014, Kap. 9)

Lernziele

- k2: Interfaces mit Default-Methoden, Unterschied zu abstrakten Klassen
- k2: Problem der Mehrfachvererbung
- k3: Erstellen von Interfaces mit Default-Methoden
- k3: Regeln zum Auflösen der Mehrfachvererbung

Quizzes

- Quiz Default-Methoden (ILIAS)

Challenges
Erklären Sie die Code-Schnipsel in der Vorgabe und die jeweils entstehenden Ausgaben.

2.3 Programmiermethoden und Clean Code

2.3.1 Javadoc

TL;DR
Mit Javadoc kann aus speziell markierten Block-Kommentaren eine externe Dokumentation im HTML-Format erzeugt werden. Die Block-Kommentare, auf die das im JDK enthaltene Programm `javadoc` reagiert, beginnen mit `/**` (also einem zusätzlichen Stern, der für den Java-Compiler nur das erste Kommentarzeichen ist).
Die erste Zeile eines Javadoc-Kommentars ist eine "Zusammenfassung" und an fast allen Stellen der generierten Doku sichtbar. Diese Summary sollte kurz gehalten werden und eine Idee vermitteln, was die Klasse oder die Methode oder das Attribut macht.
Für die Dokumentation von Parametern, Rückgabetypen, Exceptions und veralteten Elementen existieren spezielle Annotationen: `@param`, `@return`, `@throws` und `@deprecated`.
Als Faustregel gilt: Es werden **alle** `public` und `protected` Elemente (Klassen, Methoden, Attribute) mit Javadoc kommentiert. Alle nicht-öffentlichen Elemente bekommen normale Java-Kommentare (Zeilen- oder Blockkommentare).

Videos

- VL Javadoc

2.3.1.1 Dokumentation mit Javadoc

```
1 /**
2  * Beschreibung Beschreibung (Summary).
3  *
4  * <p>Hier kommt dann ein laengerer Text, der die Dinge
```

```
5   * bei Bedarf etwas ausfuehrlicher erklaert.
6   */
7  public void wuppie() {}
```

Javadoc-Kommentare sind (aus Java-Sicht) normale Block-Kommentare, wobei der Beginn mit `/**` eingeleitet wird. Dieser Beginn ist für das Tool `javadoc` (Bestandteil des JDK, genau wie `java` und `javac`) das Signal, dass hier ein Kommentar anfängt, den das Tool in eine HTML-Dokumentation übersetzen soll.

Typischerweise wird am Anfang jeder Kommentarzeile ein * eingefügt; dieser wird von Javadoc ignoriert.

Sie können neben normalem Text und speziellen Annotationen auch HTML-Elemente wie <p> und <code> oder <ul> nutzen.

Mit `javadoc *.java` können Sie in der Konsole aus den Java-Dateien die Dokumentation generieren lassen. Oder Sie geben das in Ihrer IDE in Auftrag … (die dann diesen Aufruf gern für Sie tätigt).

2.3.1.2 Standard-Aufbau

```
1  /**
2   * Beschreibung Beschreibung (Summary).
3   *
4   * <p> Hier kommt dann ein laengerer Text, der die Dinge
5   * bei Bedarf etwas ausfuehrlicher erklaert.
6   *
7   * @param   date  Tag, Wert zw. 1 .. 31
8   * @return  Anzahl der Sekunden seit 1.1.1970
9   * @throws  NumberFormatException
10  * @deprecated As of JDK version 1.1
11  */
12 public int setDate(int date) {
13     setField(Calendar.DATE, date);
14 }
```

- Erste Zeile bei Methoden/Attributen geht in die generierte “Summary” in der Übersicht, der Rest in die “Details”
  - Die “Summary” sollte kein kompletter Satz sein, wird aber wie ein Satz geschrieben (Groß beginnen, mit Punkt beenden). Es sollte nicht beginnen mit “Diese Methode macht …” oder “Diese Klasse ist …”. Ein gutes Beispiel wäre “Berechnet die Steuerrückerstattung.”
  - Danach kommen die Details, die in der generierten Dokumentation erst durch Aufklappen der Elemente sichtbar sind. Erklären Sie, wieso der Code was machen soll und welche Designentscheidungen getroffen wurden (und warum).
- Leerzeilen gliedern den Text in Absätze. Neue Absätze werden mit einem <p> eingeleitet. (Ausnahmen: Wenn der Text mit <ul> o.ä. beginnt oder der Absatz mit den Block-Tags.)
- Die “Block-Tags” `@param`, `@return`, `@throws`, `@deprecated` werden durch einen Absatz von der restlichen Beschreibung getrennt und tauchen in exakt dieser Reihenfolge auf. Die Beschreibung dieser Tags ist nicht leer - anderenfalls lässt man das Tag weg. Falls die Zeile für die Beschreibung nicht reicht, wird umgebrochen und die Folgezeile mit vier Leerzeichen (beginnend mit dem `@`) eingerückt.
  - Mit `@param` erklären Sie die Bedeutung eines Parameters (von links nach rechts) einer Methode. Beispiel: `@param date Tag, Wert zw. 1 .. 31`. Wiederholen Sie dies für jeden Parameter.

  - Mit `@return` beschreiben Sie den Rückgabetyp/-wert. Beispiel: `@return Anzahl der Sekunden seit 1.1.1970`. Bei Rückgabe von **`void`** wird diese Beschreibung weggelassen (die Beschreibung wäre dann ja leer).
  - Mit `@throws` geben Sie an, welche "checked" Exceptions die Methode wirft.
  - Mit `@deprecated` können Sie im Kommentar sagen, dass ein Element veraltet ist und möglicherweise mit der nächsten Version o.ä. entfernt wird. (siehe nächste Folie)

=> Dies sind die Basis-Regeln aus dem populären Google-Java-Style (Google Open Source 2022).

2.3.1.3 Veraltete Elemente

```
/**
 * Beschreibung Beschreibung Beschreibung.
 *
 * @deprecated As of v102, replaced by <code>Foo.fluppie()</code>.
 */
@Deprecated
public void wuppie() {}
```

- Annotation zum Markieren als "veraltet" (in der generierten Dokumentation): `@deprecated`
- Für Sichtbarkeit zur Laufzeit bzw. im Tooling/IDE: normale Code-Annotation `@Deprecated`

Dies ist ein guter Weg, um Elemente einer öffentlichen API als "veraltet" zu kennzeichnen. Üblicherweise wird diese Kennzeichnung für einige wenige Releases beibehalten und danach das veraltete Element aus der API entfernt.

2.3.1.4 Autoren, Versionen, ...

```
/**
 * Beschreibung Beschreibung Beschreibung.
 *
 * @author  Dagobert Duck
 * @version V1
 * @since   schon immer
 */
```

- Annotationen für Autoren und Version: `@author`, `@version`, `@since`

Diese Annotationen finden Sie vor allem in Kommentaren zu Packages oder Klassen.

2.3.1.5 Was muss kommentiert werden?

- Alle **`public`** Klassen
- Alle **`public`** und **`protected`** Elemente der Klassen

- Ausnahme: `@Override` (An diesen Methoden *kann*, aber *muss* nicht kommentiert werden.)

Alle anderen Elemente bei Bedarf mit *normalen* Kommentaren versehen.

2.3.1.5.1 Beispiel aus dem JDK: ArrayList Schauen Sie sich gern mal Klassen aus der Java-API an, beispielsweise eine `java.util.ArrayList`:

- Generierte Dokumentation: zu "ArrayList" runterscrollen bzw. direkt
- Quellcode: ArrayList.java

2.3.1.5.2 Best Practices: Was beschreibe ich eigentlich? Unter Documentation Best Practices finden Sie eine sehr gute Beschreibung, was das Ziel der Dokumentation sein sollte. Versuchen Sie, dieses zu erreichen!

2.3.1.6 Wrap-Up

- Javadoc-Kommentare sind normale Block-Kommentare beginnend mit `/**`
- Generierung der HTML-Dokumentation mit `javadoc *.java`
- Erste Zeile ist eine Zusammenfassung (fast immer sichtbar)
- Längerer Text danach als "Description" einer Methode/Klasse
- Annotationen für besondere Elemente: `@param`, `@return`, `@throws`, `@deprecated`

- Faustregel: Alle **`public`** und **`protected`** Elemente mit Javadoc kommentieren!

Zum Nachlesen

- Ullenboom (2021, 23.4)
- Google Open Source (2022, Kap. 7)

Lernziele

- k2: Ziel der Javadoc-Dokumentation verstehen
- k2: Typischen Aufgabe von Javadoc-Kommentaren verstehen
- k3: Dokumentation öffentlich sichtbarer Elemente mit Javadoc
- k3: Schreiben einer sinnvollen Summary
- k3: Einsatz von Annotationen zur Dokumentation von Parametern, Rückgabetypen, Exceptions, veralteten Elementen

Quizzes

- Quiz Javadoc (ILIAS)

2.3.2 Logging

TL;DR
Im Paket `java.util.logging` findet sich eine einfache Logging-API.
Über die Methode `getLogger()` der Klasse `Logger` (*Factory-Method-Pattern*) kann ein (neuer) Logger erzeugt werden, dabei wird über den String-Parameter eine Logger-Hierarchie aufgebaut analog zu den Java-Package-Strukturen. Der oberste Logger (der "Root-Logger") hat den leeren Namen.
Jeder Logger kann mit einem Log-Level (Klasse `Level`) eingestellt werden; Log-Meldungen unterhalb des eingestellten Levels werden verworfen.
Vom Logger nicht verworfene Log-Meldungen werden an den bzw. die Handler des Loggers und (per Default) an den Eltern-Logger weiter gereicht. Die Handler haben ebenfalls ein einstellbares Log-Level und verwerfen alle Nachrichten unterhalb der eingestellten Schwelle. Zur tatsächlichen Ausgabe gibt man einem Handler noch einen Formatter mit. Defaultmäßig hat nur der Root-Logger einen Handler.
Der Root-Logger (leerer String als Name) hat als Default-Level (wie auch sein Console-Handler) "`Info`" eingestellt.
Nachrichten, die durch Weiterleitung nach oben empfangen wurden, werden nicht am Log-Level des empfangenden Loggers gemessen, sondern akzeptiert und an die Handler des Loggers und (sofern nicht deaktiviert) an den Elternlogger weitergereicht.

Videos

- VL Logging
- Demo Logging (Überblick)
- Demo Log-Level
- Demo Logging: Handler und Formatter
- Demo Weiterleitung an den Elternlogger

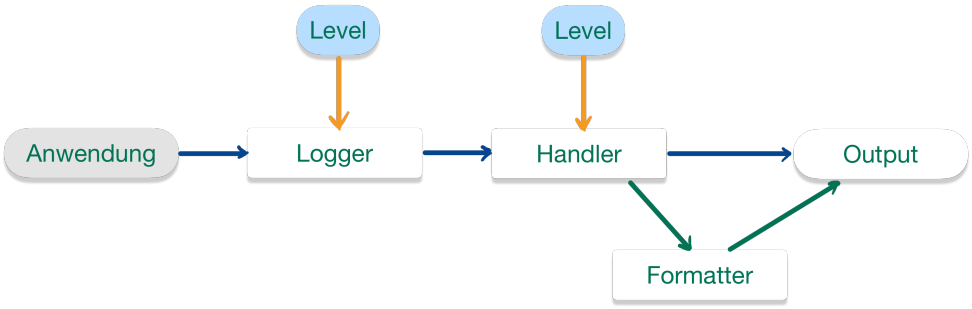
2.3.2.1 Wie prüfen Sie die Werte von Variablen/Objekten?

1. Debugging
   - Beeinflusst Code nicht
   - Kann schnell komplex und umständlich werden
   - Sitzung transient - nicht wiederholbar

2. "Poor-man's-debugging" (Ausgaben mit `System.out.println`)
   - Müssen irgendwann entfernt werden
   - Ausgabe nur auf einem Kanal (Konsole)
   - Keine Filterung nach Problemgrad - keine Unterscheidung zwischen Warnungen, einfachen Informationen, …

3. **Logging**
   - Verschiedene (Java-) Frameworks:
     `java.util.logging` (JDK), *log4j* (Apache), *SLF4J*, *Logback*, …

2.3.2.2 Java Logging API - Überblick Paket `java.util.logging`



Eine Applikation kann verschiedene Logger instanziieren. Die Logger bauen per Namenskonvention hierarchisch aufeinander auf. Jeder Logger kann selbst mehrere Handler haben, die eine Log-Nachricht letztlich auf eine bestimmte Art und Weise an die Außenwelt weitergeben.

Log-Meldungen werden einem Level zugeordnet. Jeder Logger und Handler hat ein Mindest-Level eingestellt, d.h. Nachrichten mit einem kleineren Level werden verworfen.

Zusätzlich gibt es noch Filter, mit denen man Nachrichten (zusätzlich zum Log-Level) nach weiteren Kriterien filtern kann.

Konsole: logging.LoggingDemo

2.3.2.3 Erzeugen neuer Logger

```
1 import java.util.logging.Logger;
2 Logger l = Logger.getLogger(MyClass.class.getName());
```

- **Factory-Methode** der Klasse `java.util.logging.Logger`

  ```
  1 public static Logger getLogger(String name);
  ```

  => Methode liefert bereits **vorhandenen Logger** mit diesem Namen (sonst neuen Logger)

- **Best Practice**:
  Nutzung des voll-qualifizierten Klassennamen: `MyClass.class.getName()`
  - Leicht zu implementieren
  - Leicht zu erklären
  - Spiegelt modulares Design
  - Ausgaben enthalten automatisch Hinweis auf Herkunft (Lokalität) der Meldung
  - **Alternativen**: Funktionale Namen wie “XML”, “DB”, “Security”

2.3.2.4 Ausgabe von Logmeldungen

```
1 public void log(Level level, String msg);
```

- Diverse Convenience-Methoden (Auswahl):

```
1 public void warning(String msg)
2 public void info(String msg)
3 public void entering(String srcClass, String srcMethod)
4 public void exiting(String srcClass, String srcMethod)
```

- Beispiel

```
1 import java.util.logging.Logger;
2 Logger l = Logger.getLogger(MyClass.class.getName());
3 l.info("Hello World :-)");
```

2.3.2.5 Wichtigkeit von Logmeldungen: Stufen

- `java.util.logger.Level` definiert 7 Stufen:
  - `SEVERE`, `WARNING`, `INFO`, `CONFIG`, `FINE`, `FINER`, `FINEST`
    (von höchster zu niedrigster Prio)
  - Zusätzlich `ALL` und `OFF`

- Nutzung der Log-Level:
  - Logger hat Log-Level: Meldungen mit kleinerem Level werden verworfen
  - Prüfung mit `public boolean isLoggable(Level)`
  - Setzen mit `public void setLevel(Level)`

Konsole: logging.LoggingLevel

=> Warum wird im Beispiel nach `log.setLevel(Level.ALL);` trotzdem nur ab `INFO` geloggt? Wer erzeugt eigentlich die Ausgaben?!

2.3.2.6 Jemand muss die Arbeit machen ...

- Pro Logger **mehrere** Handler möglich
  - Logger übergibt nicht verworfene Nachrichten an Handler
  - Handler haben selbst ein Log-Level (analog zum Logger)
  - Handler verarbeiten die Nachrichten, wenn Level ausreichend

- Standard-Handler: `StreamHandler`, `ConsoleHandler`, `FileHandler`

- Handler nutzen zur Formatierung der Ausgabe einen `Formatter`
- Standard-Formatter: `SimpleFormatter` und `XMLFormatter`

Konsole: logging.LoggingHandler

=> Warum wird im Beispiel nach dem Auskommentieren von `log.setUseParentHandlers(false);` immer noch eine zusätzliche Ausgabe angezeigt (ab `INFO` aufwärts)?!

2.3.2.7 Ich ... bin ... Dein ... Vater ...

- Logger bilden **Hierarchie** über Namen
  - Trenner für Namenshierarchie: "`.`" (analog zu Packages) => mit jedem "`.`" wird eine weitere Ebene der Hierarchie aufgemacht ...
  - Jeder Logger kennt seinen Eltern-Logger: `Logger#getParent()`
  - Basis-Logger: leerer Name (`""`)
    - Voreingestelltes Level des Basis-Loggers: `Level.INFO` (!)

- Weiterleiten von Nachrichten
  - Nicht verworfene Log-Aufrufe werden an Eltern-Logger weitergeleitet (Default)
    - Abschalten mit `Logger#setUseParentHandlers(false);`
  - Diese leiten an ihre Handler sowie an ihren Eltern-Logger weiter (unabhängig von Log-Level!)

Konsole: logging.LoggingParent; Tafel: Skizze Logger-Baum

2.3.2.8 Wrap-Up

- Java Logging API im Paket `java.util.logging`
- Neuer Logger über **Factory-Methode** der Klasse `Logger`
  - Einstellbares Log-Level (Klasse `Level`)
  - Handler kümmern sich um die Ausgabe, nutzen dazu Formatter
  - Mehrere Handler je Logger registrierbar
  - Log-Level auch für Handler einstellbar (!)
  - Logger (und Handler) "interessieren" sich nur für Meldungen ab bestimmter Wichtigkeit
  - Logger reichen nicht verworfene Meldungen defaultmäßig an Eltern-Logger weiter (rekursiv)

Zum Nachlesen

- Oracle Corporation (2022a, Kap. 8)

Lernziele

- k3: Nutzung der Java Logging API im Paket java.util.logging
- k3: Erstellung eigener Handler und Formatter

Quizzes

- Quiz Logging (ILIAS)

Challenges

1. Schreiben Sie einen Formatter, welcher die Meldungen in folgendem Format auf der *Konsole* ausgibt. Bauen Sie diesen Formatter in alle Logger ein.

```
------------
Logger: record.getLoggerName()
Level: record.getLevel()
Class: record.getSourceClassName()
Method: record.getSourceMethodName()
Message: record.getMessage()
------------
```

2. Schreiben Sie einen weiteren Formatter, welcher die Daten als Komma-separierte Werte (CSV-Format) mit der folgenden Reihenfolge in eine *Datei* ausgibt (durch Anfügen einer neuen Zeile an bereits bestehenden Inhalt). Bauen Sie diesen Formatter in den Logger für den Ringpuffer ein.

```
record.getLoggerName(),record.getLevel(),record.getSourceMethodName()
    ,record.getSourceClassName(),record.getMessage()
```

3. Ersetzen Sie in einem Beispielprogramm sämtliche Konsolenausgaben (`System.out.println` und `System.err.println`) in der Vorgabe durch geeignete Logger-Aufrufe mit passendem Log-Level.

   Alle Warnungen und Fehler sollen zusätzlich in eine `.csv`-Datei geschrieben werden. Auf der Konsole sollen alle Log-Meldungen ausgegeben werden.

2.3.3 Code Smells

TL;DR

Code entsteht nicht zum Selbstzweck, er muss von anderen Menschen leicht verstanden und gewartet werden können: Entwickler verbringen einen wesentlichen Teil ihrer Zeit mit dem **Lesen** von (fremdem) Code. Dabei helfen "Coding Conventions", die eine gewisse einheitliche äußerliche Erscheinung des Codes vorgeben (Namen, Einrückungen, …). Die Beachtung von grundlegenden Programmierprinzipien hilft ebenso, die Lesbarkeit und Verständlichkeit zu verbessern.
Code, der diese Konventionen und Regeln verletzt, zeigt sogenannte "**Code Smells**" oder "Bad Smells". Das sind Probleme im Code, die noch nicht direkt zu einem Fehler führen, die aber im Laufe der Zeit die Chance für echte Probleme deutlich erhöht.

Videos

- VL Code Smells

2.3.3.1 Code Smells: Ist das Code oder kann das weg?

```
class checker {
    static public void CheckANDDO(DATA1 inp, int c, FH.Studi
    CustD, int x, int y, int in, int out,int c1, int c2, int c3 = 4)
{
    public int i; // neues i
for(i=0;i<10;i++) // fuer alle i
{
```

```
8           inp.kurs[0] = 10; inp.kurs[i] = CustD.cred[i]/c;
9   }
10        SetDataToPlan(  CustD  );
11      public double myI = in*2.5; // myI=in*2.5
12      if (c1)
13          out = myI; //OK
14      else if(  c3 == 4  )
15      {
16          myI = c2 * myI;
17      if (c3 != 4 || true ) { // unbedingt beachten!
18          //System.out.println("x:"+(x++));
19          System.out.println("x:"+(x++)); // x++
20          System.out.println("out: "+out);
21      } }}   }
```

Der Code im obigen Beispiel lässt sich möglicherweise kompilieren. Und möglicherweise tut er sogar das, was er tun soll.

Dennoch: **Der Code "stinkt"** (zeigt **Code Smells**):

- Nichtbeachtung üblicher Konventionen (Coding Rules)
- Schlechte Kommentare
- Auskommentierter Code
- Fehlende Datenkapselung
- Zweifelhafte Namen
- Duplizierter Code
- "Langer" Code: Lange Methoden, Klassen, Parameterlisten, tief verschachtelte `if`/`then`-Bedingungen, …
- Feature Neid
- `switch`/`case` oder `if`/`else` statt Polymorphie
- Globale Variablen, lokale Variablen als Attribut
- Magic Numbers

Diese Liste enthält die häufigsten "Smells" und ließe sich noch beliebig fortsetzen. Schauen Sie mal in die unten angegebene Literatur :-)

Stinkender Code führt zu möglichen (späteren) Problemen.

2.3.3.2 Was ist guter ("sauberer") Code ("Clean Code")?

Im Grunde bezeichnet "sauberer Code" ("Clean Code") die Abwesenheit von Smells. D.h. man könnte Code als "sauberen" Code bezeichnen, wenn die folgenden Eigenschaften erfüllt sind (keine vollständige Aufzählung!):

- Gut ("angenehm") lesbar
- Schnell verständlich: Geeignete Abstraktionen
- Konzentriert sich auf **eine** Aufgabe
- So einfach und direkt wie möglich
- Ist gut getestet

In (Martin 2009) lässt der Autor Robert Martin verschiedene Ikonen der SW-Entwicklung zu diesem Thema zu Wort kommen - eine sehr lesenswerte Lektüre!

=> Jemand kümmert sich um den Code; solides Handwerk

2.3.3.3 Warum ist guter ("sauberer") Code so wichtig?

> Any fool can write code that a computer can understand. Good programmers write code that humans can understand.
>
> Quelle: "*Any fool…*": (Fowler 2011, p. 15)

Auch wenn das zunächst seltsam klingt, aber Code muss auch von Menschen gelesen und verstanden werden können. Klar, der Code muss inhaltlich korrekt sein und die jeweilige Aufgabe erfüllen, er muss kompilieren etc. … aber er muss auch von anderen Personen weiter entwickelt werden und dazu gelesen und verstanden werden. Guter Code ist nicht einfach nur inhaltlich korrekt, sondern kann auch einfach verstanden werden.

Code, der nicht einfach lesbar ist oder nur schwer verständlich ist, wird oft in der Praxis später nicht gut gepflegt: Andere Entwickler haben (die berechtigte) Angst, etwas kaputt zu machen und arbeiten "um den Code herum". Nur leider wird das Konstrukt dann nur noch schwerer verständlich …

2.3.3.3.1 Code Smells Verstöße gegen die Prinzipien von *Clean Code* nennt man auch *Code Smells*: Der Code "stinkt" gewissermaßen. Dies bedeutet nicht unbedingt, dass der Code nicht funktioniert (d.h. er kann dennoch compilieren und die Anforderungen erfüllen). Er ist nur nicht sauber formuliert, schwer verständlich, enthält Doppelungen etc., was im Laufe der Zeit die Chance für tatsächliche Probleme deutlich erhöht.

Und weil es so wichtig ist, hier gleich noch einmal:

Stinkender Code führt zu möglichen (späteren) Problemen.

2.3.3.3.2 "Broken Windows" Phänomen Wenn ein Gebäude leer steht, wird es eine gewisse Zeit lang nur relativ langsam verfallen: Die Fenster werden nicht mehr geputzt, es sammelt sich Graffiti, Gras wächst in der Dachrinne, Putz blättert ab …

Irgendwann wird dann eine Scheibe eingeworfen. Wenn dieser Punkt überschritten ist, beschleunigt sich der Verfall rasant: Über Nacht werden alle erreichbaren Scheiben eingeworfen, Türen werden zerstört, es werden sogar Brände gelegt …

Das passiert auch bei Software! Wenn man als Entwickler das Gefühl bekommt, die Software ist nicht gepflegt, wird man selbst auch nur relativ schlechte Arbeit abliefern. Sei es, weil man nicht versteht, was der Code macht und sich nicht an die Überarbeitung der richtigen Stellen traut und stattdessen die Änderungen als weiteren "Erker" einfach dran pappt. Seit es, weil man keine Lust hat, Zeit in ordentliche Arbeit zu investieren, weil der Code ja eh schon schlecht ist … Das wird mit der Zeit nicht besser …

"Broken Windows" Phänomen

2.3.3.3.3 Maßeinheit für Code-Qualität ;-) Es gibt eine “praxisnahe” (und nicht ganz ernst gemeinte) Maßeinheit für Code-Qualität: Die “WTF/m” (*What the Fuck per minute*): Thom Holwerda: www.osnews.com/story/19266/WTFs_.

Wenn beim Code-Review durch Kollegen viele “WTF” kommen, ist der Code offenbar nicht in Ordnung …

2.3.3.4 Code Smells: Nichtbeachtung von Coding Conventions

- Richtlinien für einheitliches Aussehen => Andere Programmierer sollen Code schnell lesen können
  - Namen, Schreibweisen
  - Kommentare (Ort, Form, Inhalt)
  - Einrückungen und Spaces vs. Tabs
  - Zeilenlängen, Leerzeilen
  - Klammern
- Beispiele: Sun Code Conventions, Google Java Style
- *Hinweis*: Betrifft vor allem die (äußere) Form!

2.3.3.5 Code Smells: Schlechte Kommentare I

- Ratlose Kommentare

```
1  /* k.A. was das bedeutet, aber wenn man es raus nimmt, geht's nicht mehr */
2  /* TODO: was passiert hier, und warum? */
```

  Der Programmierer hat selbst nicht verstanden (und macht sich auch nicht die Mühe zu verstehen), was er da tut! Fehler sind vorprogrammiert!

- Redundante Kommentare: Erklären Sie, was der Code **inhaltlich** tun sollte (und warum)!

```
1  public int i; // neues i
2  for(i=0;i<10;i++)
3  // fuer alle i
```

  Was würden Sie Ihrem Kollegen erklären (müssen), wenn Sie ihm/ihr den Code vorstellen?

  Wiederholen Sie nicht, was der Code tut (das kann ich ja selbst lesen), sondern beschreiben Sie, was der Code tun *sollte* und *warum*.

  Beschreiben Sie dabei auch das Konzept hinter einem Codebaustein.

2.3.3.6 Code Smells: Schlechte Kommentare II

- Veraltete Kommentare

  Hinweis auf unsauberes Arbeiten: Oft wird im Zuge der Überarbeitung von Code-Stellen vergessen, auch den Kommentar anzupassen! Sollte beim Lesen extrem misstrauisch machen.

- Auskommentierter Code

  Da ist jemand seiner Sache unsicher bzw. hat eine Überarbeitung nicht abgeschlossen. Die Chance, dass sich der restliche Code im Laufe der Zeit so verändert, dass der auskommentierte Code nicht mehr (richtig) läuft, ist groß! Auskommentierter Code ist gefährlich und dank Versionskontrolle absolut überflüssig!

- Kommentare erscheinen zwingend nötig

  Häufig ein Hinweis auf ungeeignete Wahl der Namen (Klassen, Methoden, Attribute) und/oder auf ein ungeeignetes Abstraktionsniveau (beispielsweise Nichtbeachtung des Prinzips der "*Single Responsibility*")!

  Der Code soll im **Normalfall** für sich selbst sprechen: **WAS** wird gemacht. Der Kommentar erklärt im Normalfall, **WARUM** der Code das machen soll.

- Unangemessene Information, z.B. Änderungshistorien

  Hinweise wie "wer hat wann was geändert" gehören in das Versionskontroll- oder ins Issue-Tracking-System. Die Änderung ist im Code sowieso nicht mehr sichtbar/nachvollziehbar!

2.3.3.7 Code Smells: Schlechte Namen und fehlende Kapselung

```
1  public class Studi extends Person {
2      public String n;
3      public int c;
4
5      public void prtIf() { ... }
6  }
```

Nach drei Wochen fragen Sie sich, was `n` oder `c` oder `Studi#prtIf()` wohl sein könnte! (Ein anderer Programmierer fragt sich das schon beim **ersten** Lesen.) Klassen und Methoden sollten sich erwartungsgemäß verhalten.

Wenn Dinge öffentlich angeboten werden, muss man damit rechnen, dass andere darauf zugreifen. D.h. man kann nicht mehr so einfach Dinge wie die interne Repräsentation oder die Art der Berechnung austauschen! Öffentliche Dinge gehören zur Schnittstelle und damit Teil des "Vertrags" mit den Nutzern!

- Programmierprinzip "**Prinzip der minimalen Verwunderung**"
  - Klassen und Methoden sollten sich erwartungsgemäß verhalten
  - Gute Namen ersparen das Lesen der Dokumentation
- Programmierprinzip "**Kapselung/Information Hiding**"
  - Möglichst schlanke öffentliche Schnittstelle
  - => "Vertrag" mit Nutzern der Klasse!

2.3.3.8 Code Smells: Duplizierter Code

```
1  public class Studi {
2      public String getName() { return name; }
3      public String getAddress() {
4          return strasse+", "+plz+" "+stadt;
5      }
6
```

```
7       public String getStudiAsString() {
8           return name+" ("+strasse+", "+plz+" "+stadt+")";
9       }
10  }
```

- Programmierprinzip "**DRY**" => "Don't repeat yourself!"

Im Beispiel wird das Formatieren der Adresse mehrfach identisch implementiert, d.h. duplizierter Code. Auslagern in eigene Methode und aufrufen!

Kopierter/duplizierter Code ist problematisch:

- Spätere Änderungen müssen an mehreren Stellen vorgenommen werden
- Lesbarkeit/Orientierung im Code wird erschwert (Analogie: Reihenhaussiedlung)
- Verpasste Gelegenheit für sinnvolle Abstraktion!

2.3.3.9 Code Smells: Langer Code

- Lange Klassen
  - Faustregel: 5 Bildschirmseiten sind viel
- Lange Methoden
  - Faustregel: 1 Bildschirmseite
  - (Martin 2009): deutlich weniger als 20 Zeilen
- Lange Parameterlisten
  - Faustregel: max. 3 … 5 Parameter
  - (Martin 2009): 0 Parameter ideal, ab 3 Parameter gute Begründung nötig
- Tief verschachtelte `if`/`then`-Bedingungen
  - Faustregel: 2 … 3 Einrückungsebenen sind viel

- Programmierprinzip "**Single Responsibility**"

  Jede Klasse ist für genau einen Aspekt des Gesamtsystems verantwortlich

2.3.3.9.1 Lesbarkeit und Übersichtlichkeit leiden

- Der Mensch kann sich nur begrenzt viele Dinge im Kurzzeitgedächtnis merken
- Klassen, die länger als 5 Bildschirmseiten sind, erfordern viel Hin- und Her-Scrollen, dito für lange Methoden
- Lange Methoden sind schwer verständlich (erledigen viele Dinge?)
- Mehr als 3 Parameter kann sich kaum jemand merken, vor allem beim Aufruf von Methoden
- Die Testbarkeit wird bei zu komplexen Methoden/Klassen und vielen Parametern sehr erschwert
- Große Dateien verleiten (auch mangels Übersichtlichkeit) dazu, neuen Code ebenfalls schluderig zu gliedern

2.3.3.9.2 Langer Code deutet auch auf eine Verletzung des Prinzips der Single Responsibility hin

- Klassen fassen evtl. nicht zusammengehörende Dinge zusammen

```java
public class Student {
    private String name;
    private String phoneAreaCode;
    private String phoneNumber;

    public void printStudentInfo() {
        System.out.println("name:    " + name);
        System.out.println("contact: " + phoneAreaCode + "/" + phoneNumber)
            ;
    }
}
```

Warum sollte sich die Klasse `Student` um die Einzelheiten des Aufbaus einer Telefonnummer kümmern? Das Prinzip der "*Single Responsibility*" wird hier verletzt!

- Methoden erledigen vermutlich mehr als nur eine Aufgabe

```java
public void credits() {
    for (Student s : students) {
        if (s.hasSemesterFinished()) {
            ECTS c = calculateEcts(s);
            s.setEctsSum(c);
        }
    }
}

// Diese Methode erledigt 4 Dinge: Iteration, Abfrage, Berechnung, Setzen
    ...
```

=> Erklären Sie die Methode jemandem. Wenn dabei das Wort "und" vorkommt, macht die Methode höchstwahrscheinlich zu viel!

- Viele Parameter bedeuten oft fehlende Datenabstraktion

```java
Circle makeCircle(int x, int y, int radius);
Circle makeCircle(Point center, int radius);  // besser!
```

2.3.3.10 Code Smells: Feature Neid

```java
public class CreditsCalculator {
    public ECTS calculateEcts(Student s) {
        int semester = s.getSemester();
        int workload = s.getCurrentWorkload();
        int nrModuls = s.getNumberOfModuls();
        int total = Math.min(30, workload);
        int extra = Math.max(0, total - 30);
        if (semester < 5) {
            extra = extra * nrModuls;
        }
        return new ECTS(total + extra);
    }
}
```

- Zugriff auf (viele) Interna der anderen Klasse! => Hohe Kopplung der Klassen!
- Methode `CreditsCalculator#calculateEcts()` "möchte" eigentlich in `Student` sein …

2.3.3.11 Weiterführende Links

- "Foundations: Clean Code" (The Odin Project)
- "Documentation Best Practices" (Google Styleguide)

2.3.3.12 Wrap-Up

- Code entsteht nicht zum Selbstzweck => Lesbarkeit ist wichtig

- Code Smells: Code führt zu möglichen (späteren) Problemen
  - Richtiges Kommentieren und Dokumentieren

    In dieser Sitzung haben wir vor allem auf Kommentare geschaut. Zum Thema Dokumentieren siehe die Einheit zu "Javadoc".
  - Einhalten von Coding Conventions
    * Regeln zu Schreibweisen und Layout
    * Leerzeichen, Einrückung, Klammern
    * Zeilenlänge, Umbrüche
    * Kommentare
  - Einhalten von Prinzipien des objektorientierten Programmierens
    * Jede Klasse ist für genau **einen** Aspekt des Systems verantwortlich. (*Single Responsibility*)
    * Keine Code-Duplizierung! (*DRY* - Don't repeat yourself)
    * Klassen und Methoden sollten sich erwartungsgemäß verhalten
    * Kapselung: Möglichst wenig öffentlich zugänglich machen

Zum Nachlesen

- Martin (2009)
- Passig und Jander (2013)
- Inden (2013, Kap. 10)

Lernziele

- k3: Erkennen und Vermeiden von Code Smells
- k3: Unterscheiden von leicht lesbarem und schwer lesbarem Code
- k3: Programmierprinzipien anwenden, um den Code sauberer zu gestalten
- k3: Bessere Kommentare schreiben

Quizzes

- Quiz Code Smells (ILIAS)

2.3.4 Coding Conventions und Metriken

TL;DR
Code entsteht nicht zum Selbstzweck, er muss von anderen Menschen leicht verstanden und gewartet werden können: Entwickler verbringen einen wesentlichen Teil ihrer Zeit mit dem **Lesen** von (fremdem) Code.
Dabei helfen "Coding Conventions", die eine gewisse einheitliche äußerliche Erscheinung des Codes vorgeben (Namen, Einrückungen, …). Im Java-Umfeld ist der "Google Java Style" bzw. der recht ähnliche "AOSP Java Code Style for Contributors" häufig anzutreffen. Coding Conventions beinhalten typischerweise Regeln zu

- Schreibweisen und Layout
- Leerzeichen, Einrückung, Klammern
- Zeilenlänge, Umbrüche
- Kommentare

Die Beachtung von grundlegenden Programmierprinzipien hilft ebenso, die Lesbarkeit und Verständlichkeit zu verbessern.
Metriken sind Kennzahlen, die aus dem Code berechnet werden, und können zur Überwachung der Einhaltung von Coding Conventions und anderen Regeln genutzt werden. Nützliche Metriken sind dabei NCSS (*Non Commenting Source Statements*), McCabe (*Cyclomatic Complexity*), BEC (*Boolean Expression Complexity*) und DAC (*Class Data Abstraction Coupling*).
Für die Formatierung des Codes kann man die IDE nutzen, muss dort dann aber die Regeln detailliert manuell einstellen. Das Tool **Spotless** lässt sich dagegen in den Build-Prozess einbinden und kann die Konfiguration über ein vordefiniertes Regelset passend zum Google Java Style/AOSP automatisiert vornehmen.
Die Prüfung der Coding Conventions und Metriken kann durch das Tool **Checkstyle** erfolgen. Dieses kann beispielsweise als Plugin in der IDE oder direkt in den Build-Prozess eingebunden werden und wird mit Hilfe einer XML-Datei konfiguriert.
Um typische Anti-Pattern zu vermeiden, kann man den Code mit sogenannten *Lintern* prüfen. Ein Beispiel für die Java-Entwicklung ist **SpotBugs**, welches sich in den Build-Prozess einbinden lässt und über 400 typische problematische Muster im Code erkennt.
Für die Praktika in der Veranstaltung Programmiermethoden wird der Google Java Style oder AOSP genutzt. Für die passende Checkstyle-Konfiguration wird eine minimale checkstyle.xml bereitgestellt (vgl. Folie "Konfiguration für das PM-Praktikum").

Videos

- VL Coding Conventions
- Demo Formatter und Spotless
- Demo Checkstyle
- Demo Checkstyle: Konfiguration mit Eclipse-CS
- Demo SpotBugs

2.3.4.1 Coding Conventions: Richtlinien für einheitliches Aussehen von Code

=> Ziel: Andere Programmierer sollen Code schnell lesen können

- **Namen, Schreibweisen**: UpperCamelCase vs. lowerCamelCase vs. UPPER_SNAKE_CASE
- **Kommentare** (Ort, Form, Inhalt): Javadoc an allen `public` und `protected` Elementen

- **Einrückungen und Spaces vs. Tabs**: 4 Spaces
- **Zeilenlängen**: 100 Zeichen
- **Leerzeilen**: Leerzeilen für Gliederung
- **Klammern**: Auf selber Zeile wie Code

Beispiele: Sun Code Conventions, Google Java Style, AOSP Java Code Style for Contributors

2.3.4.2 Beispiel nach Google Java Style/AOSP formatiert

```java
package wuppie.deeplearning.strategy;

/**
 * Demonstriert den Einsatz von AOSP/Google Java Style ..............
     Umbruch nach 100 Zeichen |
 */
public class MyWuppieStudi implements Comparable<MyWuppieStudi> {
    private static String lastName;
    private static MyWuppieStudi studi;

    private MyWuppieStudi() {}

    /** Erzeugt ein neues Exemplar der MyWuppieStudi-Spezies (max. 40 Zeilen)
        */
    public static MyWuppieStudi getMyWuppieStudi(String name) {
        if (studi == null) {
            studi = new MyWuppieStudi();
        }
        if (lastName == null) lastName = name;

        return studi;
    }

    @Override
    public int compareTo(MyWuppieStudi o) {
        return lastName.compareTo(lastName);
    }
}
```

Dieses Beispiel wurde nach Google Java Style/AOSP formatiert.

Die Zeilenlänge beträgt max. 100 Zeichen. Pro Methode werden max. 40 Zeilen genutzt. Zwischen Attributen, Methoden und Importen wird jeweils eine Leerzeile eingesetzt (zwischen den einzelnen Attributen *muss* aber keine Leerzeile genutzt werden). Zur logischen Gliederung können innerhalb von Methoden weitere Leerzeilen eingesetzt werden, aber immer nur eine.

Klassennamen sind UpperCamelCase, Attribute und Methoden und Parameter lowerCamelCase, Konstanten (im Beispiel nicht vorhanden) UPPER_SNAKE_CASE. Klassen sind Substantive, Methoden Verben.

Alle `public` und `protected` Elemente werden mit einem Javadoc-Kommentar versehen. Überschriebene Methoden müssen nicht mit Javadoc kommentiert werden, müssen aber mit `@Override` markiert werden.

Geschweifte Klammern starten immer auf der selben Codezeile. Wenn bei einem `if` nur ein Statement vorhanden ist und dieses auf die selbe Zeile passt, kann auf die umschließenden geschweiften Klammern ausnahmsweise verzichtet werden.

Es wird mit Leerzeichen eingerückt. Google Java Style arbeitet mit 2 Leerzeichen, während AOSP hier 4 Leerzeichen vorschreibt. Im Beispiel wurde nach AOSP eingerückt.

Darüber hinaus gibt es vielfältige weitere Regeln für das Aussehen des Codes. Lesen Sie dazu entsprechend auf Google Java Style und auch auf AOSP nach.

2.3.4.3 Formatieren Sie Ihren Code (mit der IDE) Sie können den Code manuell formatieren, oder aber (sinnvollerweise) über Tools formatieren lassen. Hier einige Möglichkeiten:

- IDE: Code-Style einstellen und zum Formatieren nutzen
- google-java-format: `java -jar google-java-format.jar --replace *.java` (auch als IDE-Plugin)
- **Spotless** in Gradle:

```
plugins {
    id "java"
    id "com.diffplug.spotless" version "6.5.0"
}

spotless {
    java {
        // googleJavaFormat()
        googleJavaFormat().aosp()  // indent w/ 4 spaces
    }
}
```

Prüfen mit `./gradlew spotlessCheck` (Teil von `./gradlew check`) und Formatieren mit `./gradlew spotlessApply`

2.3.4.3.1 Einstellungen der IDE's

- Eclipse:
  - `Project` > `Properties` > `Java Code Style` > `Formatter`: Coding-Style einstellen/einrichten
  - Code markieren, `Source` > `Format`
  - Komplettes Aufräumen: `Source` > `Clean Up` (Formatierung, Importe, Annotationen, ...) Kann auch so eingestellt werden, dass ein "Clean Up" immer beim Speichern ausgeführt wird!
- IntelliJ verfügt über ähnliche Fähigkeiten:
  - Einstellen über `Preferences` > `Editor` > `Code Style` > `Java`
  - Formatieren mit `Code` > `Reformat Code` oder `Code` > `Reformat File`

Die Details kann/muss man einzeln einstellen. Für die "bekannten" Styles (Google Java Style) bringen die IDE's oft aber schon eine Gesamtkonfiguration mit.

Achtung: Zumindest in Eclipse gibt es mehrere Stellen, wo ein Code-Style eingestellt werden kann ("Clean Up", "Formatter", ...). Diese sollten dann jeweils auf den selben Style eingestellt werden, sonst gibt es unter

Umständen lustige Effekte, da beim Speichern ein anderer Style angewendet wird als beim “Clean Up” oder beim “Format Source” …

Analog sollte man bei der Verwendung von Checkstyle auch in der IDE im Formatter die entsprechenden Checkstyle-Regeln (s.u.) passend einstellen, sonst bekommt man durch Checkstyle Warnungen angezeigt, die man durch ein automatisches Formatieren *nicht* beheben kann.

2.3.4.3.2 Google Java Style und google-java-format Wer direkt den Google Java Style nutzt, kann auch den dazu passenden Formatter von Google einsetzen: google-java-format. Diesen kann man entweder als Plugin für IntelliJ/Eclipse einsetzen oder als Stand-alone-Tool (Kommandozeile oder Build-Skripte) aufrufen. Wenn man sich noch einen entsprechenden Git-Hook definiert, wird vor jedem Commit der Code entsprechend den Richtlinien formatiert :)

2.3.4.3.3 Spotless und google-java-format in Gradle *Hinweis*: Bei Spotless in Gradle müssen je nach den Versionen von Spotless/google-java-format bzw. des JDK noch Optionen in der Datei `gradle.properties` eingestellt werden (siehe Demo und Spotless > google-java-format (Web)).

Tipp: Die Formatierung über die IDE ist angenehm, aber in der Praxis leider oft etwas hakelig: Man muss alle Regeln selbst einstellen (und es gibt *einige* dieser Einstellungen), und gerade IntelliJ “greift” manchmal nicht alle Code-Stellen beim Formatieren. Nutzen Sie Spotless und bauen Sie die Konfiguration in Ihr Build-Skript ein und konfigurieren Sie über den Build-Prozess.

Demo: Konfiguration Formatter (IDE), Spotless/Gradle

2.3.4.4 Metriken: Kennzahlen für verschiedene Aspekte zum Code Metriken messen verschiedene Aspekte zum Code und liefern eine Zahl zurück. Mit Metriken kann man beispielsweise die Einhaltung der Coding Rules (Formate, …) prüfen, aber auch die Einhaltung verschiedener Regeln des objektorientierten Programmierens.

2.3.4.4.1 Beispiele für wichtige Metriken (jeweils Max-Werte für PM) Die folgenden Metriken und deren Maximal-Werte sind gute Erfahrungswerte aus der Praxis und helfen, den Code Smell “Langer Code” (vgl. “Code Smells”) zu erkennen und damit zu vermeiden. Über die Metriken *BEC*, *McCabe* und *DAC* wird auch die Einhaltung elementarer Programmierregeln gemessen.

- **NCSS** (*Non Commenting Source Statements*)
  - Zeilen pro Methode: 40; pro Klasse: 250; pro Datei: 300
    Annahme: Eine Anweisung je Zeile …
- **Anzahl der Methoden** pro Klasse: 10
- **Parameter** pro Methode: 3
- **BEC** (*Boolean Expression Complexity*)
  Anzahl boolescher Ausdrücke in `if` etc.: 3
- **McCabe** (*Cyclomatic Complexity*)
  - Anzahl der möglichen Verzweigungen (Pfade) pro Methode + 1
  - 1-4 gut, 5-7 noch OK

- **DAC** (*Class Data Abstraction Coupling*)
  - Anzahl der genutzten (instantiierten) “Fremdklassen”
  - Werte kleiner 7 werden i.A. als normal betrachtet

Die obigen Grenzwerte sind typische Standardwerte, die sich in der Praxis allgemein bewährt haben (vergleiche u.a. (Martin 2009) oder auch in AOSP: Write short methods und AOSP: Limit line length).

Dennoch sind das keine absoluten Werte an sich. Ein Übertreten der Grenzen ist ein **Hinweis** darauf, dass **höchstwahrscheinlich** etwas nicht stimmt, muss aber im konkreten Fall hinterfragt und diskutiert und begründet werden!

2.3.4.4.2 Metriken im Beispiel von oben

```
1     private static String lastName;
2     private static MyWuppieStudi studi;
3
4     public static MyWuppieStudi getMyWuppieStudi(String name) {
5         if (studi == null) {
6             studi = new MyWuppieStudi();
7         }
8         if (lastName == null) lastName = name;
9
10        return studi;
11    }
```

- BEC: 1 (nur ein boolescher Ausdruck im `if`)
- McCabe: 3 (es gibt zwei mögliche Verzweigungen in der Methode plus die Methode selbst)
- DAC: 1 (eine “Fremdklasse”: `String`)

Anmerkung: In Checkstyle werden für einige häufig verwendete Standard-Klassen Ausnahmen definiert, d.h. `String` würde im obigen Beispiel *nicht* bei DAC mitgezählt/angezeigt.

=> Verweis auf LV Softwareengineering

2.3.4.5 Tool-Support: Checkstyle Metriken und die Einhaltung von Coding-Conventions werden sinnvollerweise nicht manuell, sondern durch diverse Tools erfasst, etwa im Java-Bereich mit Hilfe von **Checkstyle**.

Das Tool lässt sich Standalone über CLI nutzen oder als Plugin für IDE’s (Eclipse oder IntelliJ) einsetzen. Gradle bringt ein eigenes Plugin mit.

- IDE: diverse Plugins: Eclipse-CS, CheckStyle-IDEA
- CLI: `java -jar checkstyle-10.2-all.jar -c google_checks.xml *.java`
- Plugin “**checkstyle**” in Gradle:

```
1  plugins {
2      id "java"
3      id "checkstyle"
4  }
5
6  checkstyle {
```

```
7       configFile file('checkstyle.xml')
8       toolVersion '10.2'
9   }
```

- Aufruf: Prüfen mit `./gradlew checkstyleMain` (Teil von `./gradlew check`)
- Konfiguration: `<projectDir>/config/checkstyle/checkstyle.xml` (Default) bzw. mit der obigen Konfiguration direkt im Projektordner
- Report: `<projectDir>/build/reports/checkstyle/main.html`

Demo: IntelliJ, Checkstyle/Gradle

2.3.4.6 Checkstyle: Konfiguration Die auszuführenden Checks lassen sich über eine XML-Datei konfigurieren. In Eclipse-CS kann man die Konfiguration auch in einer GUI bearbeiten.

Das Checkstyle-Projekt stellt eine passende Konfiguration für den Google Java Style bereit. Diese ist auch in den entsprechenden Plugins oft bereits enthalten und kann direkt ausgewählt oder als Startpunkt für eigene Konfigurationen genutzt werden.

Der Startpunkt für die Konfigurationsdatei ist immer das Modul “Checker”. Darin können sich “FileSetChecks” (Module, die auf einer Menge von Dateien Checks ausführen), “Filters” (Module, die Events bei der Prüfung von Regeln filtern) und “AuditListeners” (Module, die akzeptierte Events in einen Report überführen) befinden. Der “TreeWalker” ist mit der wichtigste Vertreter der FileSetChecks-Module und transformiert die zu prüfenden Java-Sourcen in einen *Abstract Syntax Tree*, also eine Baumstruktur, die dem jeweiligen Code unter der Java-Grammatik entspricht. Darauf können dann wiederum die meisten Low-Level-Module arbeiten.

Eine Reihe von Standard-Checks sind bereits in Checkstyle implementiert und benötigen keine weitere externe Abhängigkeiten. Man kann aber zusätzliche Regeln aus anderen Projekten beziehen (etwa via Gradle/Maven) oder sich eigene zusätzliche Regeln in Java schreiben. Die einzelnen Checks werden in der Regel als “Modul” dem “TreeWalker” hinzugefügt und über die jeweiligen Properties näher konfiguriert.

Sie finden in der Doku zu jedem Check das entsprechende Modul, das Eltern-Modul (also wo müssen Sie das Modul im XML-Baum einfügen) und auch die möglichen Properties und deren Default-Einstellungen.

```
<module name="Checker">
    <module name="LineLength">
        <property name="max" value="100"/>
    </module>

    <module name="TreeWalker">
        <module name="AvoidStarImport"/>
        <module name="MethodCount">
            <property name="maxPublic" value="10"/>
            <property name="maxTotal" value="40"/>
        </module>
    </module>
</module>
```

Alternativen/Ergänzungen: beispielsweise MetricsReloaded.

Demo: Konfiguration mit Eclipse-CS, Hinweis auf Formatter

2.3.4.7 SpotBugs: Finde Anti-Pattern und potentielle Bugs (Linter)

- **SpotBugs** sucht nach über 400 potentiellen Bugs im Code
  - Anti-Pattern (schlechte Praxis, "dodgy" Code)
  - Sicherheitsprobleme
  - Korrektheit
- CLI: `java -jar spotbugs.jar options ...`
- IDE: IntelliJ SpotBugs plugin, SpotBugs Eclipse plugin
- Gradle: SpotBugs Gradle Plugin

```
plugins {
    id "java"
    id "com.github.spotbugs" version "5.0.6"
}
spotbugs {
    ignoreFailures = true
    showStackTraces = false
}
```

Prüfen mit `./gradlew spotbugsMain` (in `./gradlew check`)

Demo: SpotBugs/Gradle

2.3.4.8 Konfiguration für das PM-Praktikum (Format, Metriken, Checkstyle, SpotBugs) Im PM-Praktikum beachten wir die obigen Coding Conventions und Metriken mit den dort definierten Grenzwerten. Diese sind bereits in der bereit gestellten Minimal-Konfiguration für Checkstyle (s.u.) konfiguriert.

2.3.4.8.1 Formatierung

- Google Java Style/AOSP: **Spotless**

Zusätzlich wenden wir den Google Java Style an. Statt der dort vorgeschriebenen Einrückung mit 2 Leerzeichen (und 4+ Leerzeichen bei Zeilenumbruch in einem Statement) können Sie auch mit 4 Leerzeichen einrücken (8 Leerzeichen bei Zeilenumbruch) (AOSP). Halten Sie sich in Ihrem Team an eine einheitliche Einrückung (Google Java Style *oder* AOSP).

Formatieren Sie Ihren Code vor den Commits mit **Spotless** (über Gradle) oder stellen Sie den Formatter Ihrer IDE entsprechend ein.

2.3.4.8.2 Checkstyle

- Minimal-Konfiguration für **Checkstyle** (Coding Conventions, Metriken)

Nutzen Sie die folgende **Minimal-Konfiguration** für **Checkstyle** für Ihre Praktikumsaufgaben. Diese beinhaltet die Prüfung der wichtigsten Formate nach Google Java Style/AOSP sowie der obigen Metriken. Halten Sie diese Regeln ein.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE module PUBLIC "-//Checkstyle//DTD Checkstyle Configuration 1.3//EN" "
    https://checkstyle.org/dtds/configuration_1_3.dtd">

<module name="Checker">
  <property name="severity" value="warning"/>

  <module name="TreeWalker">
    <module name="JavaNCSS">
      <property name="methodMaximum" value="40"/>
      <property name="classMaximum" value="250"/>
      <property name="fileMaximum" value="300"/>
    </module>
    <module name="BooleanExpressionComplexity"/>
    <module name="CyclomaticComplexity">
      <property name="max" value="7"/>
    </module>
    <module name="ClassDataAbstractionCoupling">
      <property name="max" value="6"/>
    </module>
    <module name="MethodCount">
      <property name="maxTotal" value="10"/>
      <property name="maxPrivate" value="10"/>
      <property name="maxPackage" value="10"/>
      <property name="maxProtected" value="10"/>
      <property name="maxPublic" value="10"/>
    </module>
    <module name="ParameterNumber">
      <property name="max" value="3"/>
    </module>
    <module name="MethodLength">
      <property name="max" value="40"/>
    </module>
    <module name="Indentation">
      <property name="basicOffset" value="4"/>
      <property name="lineWrappingIndentation" value="8"/>
      <property name="caseIndent" value="4"/>
      <property name="throwsIndent" value="4"/>
      <property name="arrayInitIndent" value="4"/>
    </module>
    <module name="TypeName"/>
    <module name="MethodName"/>
    <module name="MemberName"/>
    <module name="ParameterName"/>
    <module name="ConstantName"/>
    <module name="OneStatementPerLine"/>
    <module name="MultipleVariableDeclarations"/>
    <module name="MissingOverride"/>
    <module name="MissingJavadocMethod"/>
    <module name="AvoidStarImport"/>
  </module>

  <module name="LineLength">
    <property name="max" value="100"/>
  </module>
  <module name="FileTabCharacter">
```

```
56      <property name="eachLine" value="true"/>
57    </module>
58    <module name="NewlineAtEndOfFile"/>
59  </module>
```

Sie können diese Basis-Einstellungen auch aus dem Programmiermethoden-CampusMinden/PM-Lecture-Repo direkt herunterladen: checkstyle.xml.

Sie können zusätzlich gern noch die weiteren (und strengeren) Regeln aus der vom Checkstyle-Projekt bereitgestellten Konfigurationsdatei für den Google Java Style nutzen. *Hinweis*: Einige der dort konfigurierten Checkstyle-Regeln gehen allerdings über den Google Java Style hinaus.

2.3.4.8.3 Linter: SpotBugs

- Vermeiden von Anti-Pattern mit **SpotBugs**

Setzen Sie zusätzlich **SpotBugs** mit ein. Ihre Lösungen dürfen keine Warnungen oder Fehler beinhalten, die SpotBugs melden würde.

2.3.4.9 Wrap-Up

- Code entsteht nicht zum Selbstzweck => Regeln nötig!
  - Coding Conventions
    - Regeln zu Schreibweisen und Layout
    - Leerzeichen, Einrückung, Klammern
    - Zeilenlänge, Umbrüche
    - Kommentare
  - Formatieren mit **Spotless**
  - Prinzipien des objektorientierten Programmierens (vgl. “Code Smells”)
    - Jede Klasse ist für genau **einen** Aspekt des Systems verantwortlich. (*Single Responsibility*)
    - Keine Code-Duplizierung! (*DRY* - Don’t repeat yourself)
    - Klassen und Methoden sollten sich erwartungsgemäß verhalten
    - Kapselung: Möglichst wenig öffentlich zugänglich machen
- Metriken: Einhaltung von Regeln in Zahlen ausdrücken
- Prüfung manuell durch Code Reviews oder durch Tools wie **Checkstyle** oder **SpotBugs**
- Definition des “PM-Styles” (siehe Folie “Konfiguration für das PM-Praktikum”)

Zum Nachlesen

- Martin (2009)
- Inden (2013, Kap. 13)
- Google Open Source (2022)

Lernziele

- k2: Erklären verschiedener Coding Conventions
- k2: Erklären wichtiger Grundregeln des objektorientierten Programmierens
- k2: Erklären der Metriken NCSS, McCabe, BEC, DAC
- k3: Einhalten der wichtigsten Grundregeln des objektorientierten Programmierens
- k3: Einhalten der wichtigsten Coding Conventions (Formatierung, Namen, Metriken)
- k3: Nutzung des Tools Spotless (Formatierung des Codes)
- k3: Nutzung des Tools Checkstyle (Coding Conventions und Metriken)
- k3: Nutzung des Tools SpotBugs (Vermeiden von Anti-Pattern)

2.3.5 Refactoring

TL;DR
Refactoring bedeutet Änderung der inneren Struktur des Codes ohne Beeinflussung äußeren Verhaltens.
Mit Hilfe von Refactoring kann man Code Smells beheben, und Lesbarkeit, Verständlichkeit und Wartbarkeit von Software verbessern.
Es ist wichtig, immer nur einzelne Schritte zu machen und anschließend die Testsuite laufen zu lassen, damit nicht versehentlich Fehler oder Verhaltensänderungen beim Refactoring eingebaut werden.
Prinzipiell kann man Refactoring manuell mit Search&Replace durchführen, aber es bietet sich an, hier die IDE-Unterstützung zu nutzen. Es stehen verschiedene Methoden zur Verfügung, die nicht unbedingt einheitlich benannt sein müssen oder in jeder IDE vorkommen. Zu den häufig genutzten Methoden zählen *Rename*, *Extract*, *Move* und *Push Up/Pull Down*.

Videos

- VL Refactoring
- Demo Refactoring: Rename
- Demo Refactoring: Encapsulate
- Demo Refactoring: Extract Method
- Demo Refactoring: Move Method
- Demo Refactoring: Pull up

2.3.5.1 Was ist Refactoring?

> Refactoring ist, wenn einem auffällt, daß der Funktionsname `foobar` ziemlich bescheuert ist, und man die Funktion in `sinus` umbenennt.
>
> Quelle: "356: Refactoring" by Andreas Bogk on Lutz Donnerhacke: "Fachbegriffe der Informatik"

> Refactoring (noun): a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behaviour.

> Quelle: "*Refactoring*": (Fowler 2011, p. 53)

Refactoring: Änderungen an der **inneren Struktur** einer Software

- Beobachtbares (äußeres) Verhalten ändert sich dabei **nicht**
  - Keine neuen Features einführen
  - Keine Bugs fixen
  - Keine öffentliche Schnittstelle ändern (*Anmerkung*: Bis auf Umbenennungen oder Verschiebungen von Elementen innerhalb der Software)
- Ziel: Verbesserung von Verständlichkeit und Änderbarkeit

2.3.5.2 Anzeichen, dass Refactoring jetzt eine gute Idee wäre

- Code "stinkt" (zeigt/enthält *Code Smells*)

  Code Smells sind strukturelle Probleme, die im Laufe der Zeit zu Problemen führen können. Refactoring ändert die innere Struktur des Codes und kann entsprechend genutzt werden, um die Smells zu beheben.

- Schwer erklärbarer Code

  Könnten Sie Ihren Code ohne Vorbereitung in der Abgabe erklären? In einer Minute? In fünf Minuten? In zehn? Gar nicht?

  In den letzten beiden Fällen sollten Sie definitiv über eine Vereinfachung der Strukturen nachdenken.

- Verständnisprobleme, Erweiterungen

  Sie grübeln in der Abgabe, was Ihr Code machen sollte?

  Sie überlegen, was Ihr Code bedeutet, um herauszufinden, wo Sie die neue Funktionalität anbauen können?

  Sie suchen nach Codeteilen, finden diese aber nicht, da die sich in anderen (falschen?) Stellen/Klassen befinden?

  Nutzen Sie die (neuen) Erkenntnisse, um den Code leichter verständlich zu gestalten.

> "Three strikes and you refactor."
>
> Quelle: "*Three strikes…*": (Fowler 2011, p. 58): "The Rule of Three"

Wenn Sie sich zum dritten Mal über eine suboptimale Lösung ärgern, dann werden Sie sich vermutlich noch öfter darüber ärgern. Jetzt ist der Zeitpunkt für eine Verbesserung.

Schauen Sie sich die entsprechenden Kapitel in (Passig und Jander 2013) und (Fowler 2011) an, dort finden Sie noch viele weitere Anhaltspunkte, ob und wann Refactoring sinnvoll ist.

2.3.5.3 Bevor Sie loslegen ...

1. **Unit Tests** schreiben
   - Normale und ungültige Eingaben
   - Rand- und Spezialfälle
2. **Coding Conventions** einhalten
   - Sourcecode formatieren (lassen)
3. Haben Sie die fragliche Codestelle auch wirklich verstanden?!

2.3.5.4 Vorgehen beim Refactoring

2.3.5.4.1 Überblick über die Methoden des Refactorings Die Refactoring-Methoden sind nicht einheitlich definiert, es existiert ein großer und uneinheitlicher "Katalog" an möglichen Schritten. Teilweise benennt jede IDE die Schritte etwas anders, teilweise werden unterschiedliche Möglichkeiten angeboten.

Zu den am häufigsten genutzten Methoden zählen

- Rename Method/Class/Field
- Encapsulate Field
- Extract Method/Class
- Move Method
- Pull Up, Push Down (Field, Method)

2.3.5.4.2 Best Practice Eine Best Practice (oder nennen Sie es einfach eine wichtige Erfahrung) ist, beim Refactoring langsam und gründlich vorzugehen. Sie ändern die Struktur der Software und können dabei leicht Fehler oder echte Probleme einbauen. Gehen Sie also langsam und sorgsam vor, machen Sie einen Schritt nach dem anderen und sichern Sie sich durch eine gute Testsuite ab, die Sie nach jedem Schritt erneut ausführen: Das Verhalten der Software soll sich ja nicht ändern, d.h. die Tests müssen nach jedem einzelnen Refactoring-Schritt immer grün sein (oder Sie haben einen Fehler gemacht).

- Kleine Schritte: immer nur **eine** Änderung zu einer Zeit
- Nach **jedem** Refactoring-Schritt **Testsuite** laufen lassen

  => Nächster Refactoring-Schritt erst, wenn alle Tests wieder "grün"
- Versionskontrolle nutzen: **Jeden** Schritt **einzeln** committen

2.3.5.5 Refactoring-Methode: Rename Method/Class/Field

2.3.5.5.1 Motivation Name einer Methode/Klasse/Attributs erklärt nicht ihren Zweck.

2.3.5.5.2 Durchführung Name selektieren, “`Refactor > Rename`”

2.3.5.5.3 Anschließend ggf. prüfen Aufrufer? Superklassen?

2.3.5.5.4 Beispiel Vorher

```
1 public String getTeN() {}
```

Nachher

```
1 public String getTelefonNummer() {}
```

2.3.5.6 Refactoring-Methode: Encapsulate Field

2.3.5.6.1 Motivation Sichtbarkeit von Attributen reduzieren.

2.3.5.6.2 Durchführung Attribut selektieren, “`Refactor > Encapsulate Field`”

2.3.5.6.3 Anschließend ggf. prüfen Superklassen? Referenzen? (Neue) JUnit-Tests?

2.3.5.6.4 Beispiel Vorher

```
1 int cps;
2
3 public void printDetails() {
4     System.out.println("Credits: " + cps);
5 }
```

Nachher

```
1 private int cps;
2
3 int getCps() { return cps; }
4 void setCps(int cps) {  this.cps = cps;  }
5
6 public void printDetails() {
7     System.out.println("credits: " + getCps());
8 }
```

2.3.5.7 Refactoring-Methode: Extract Method/Class

2.3.5.7.1 Motivation

- Codefragment stellt eigenständige Methode dar
- “Überschriften-Code”
- Code-Duplizierung
- Code ist zu “groß”
- Klasse oder Methode erfüllt unterschiedliche Aufgaben

2.3.5.7.2 Durchführung Codefragment selektieren, “`Refactor > Extract Method`” bzw. “`Refactor > Extract Class`”

2.3.5.7.3 Anschließend ggf. prüfen

- Aufruf der neuen Methode? Nutzung der neuen Klasse?
- Neue JUnit-Tests nötig? Veränderung bestehender Tests nötig?
- Speziell bei Methoden:
  - Nutzung lokaler Variablen: Übergabe als Parameter!
  - Veränderung lokaler Variablen: Rückgabewert in neuer Methode und Zuweisung bei Aufruf; evtl. neue Typen nötig!

2.3.5.7.4 Beispiel Vorher

```java
public void printInfos() {
    printHeader();
    // Details ausgeben
    System.out.println("name:    " + name);
    System.out.println("credits: " + cps);
}
```

Nachher

```java
public void printInfos() {
    printHeader();
    printDetails();
}
private void printDetails() {
    System.out.println("name:    " + name);
    System.out.println("credits: " + cps);
}
```

2.3.5.8 Refactoring-Methode: Move Method

2.3.5.8.1 Motivation Methode nutzt (oder wird genutzt von) mehr Eigenschaften einer fremden Klasse als der eigenen Klasse.

2.3.5.8.2 Durchführung Methode selektieren, “`Refactor` > `Move`” (ggf. “Keep original method as delegate to moved method” aktivieren)

2.3.5.8.3 Anschließend ggf. prüfen

- Aufruf der neuen Methode (Delegation)?
- Neue JUnit-Tests nötig? Veränderung bestehender Tests nötig?
- Nutzung lokaler Variablen: Übergabe als Parameter!
- Veränderung lokaler Variablen: Rückgabewert in neuer Methode und Zuweisung bei Aufruf; evtl. neue Typen nötig!

2.3.5.8.4 Beispiel Vorher

```java
public class Kurs {
    int cps;
    String descr;
}

public class Studi extends Person {
    String name;
    int cps;
    Kurs kurs;

    public void printKursInfos() {
        System.out.println("Kurs:    " + kurs.descr);
        System.out.println("Credits: " + kurs.cps);
    }
}
```

Nachher

```java
public class Kurs {
    int cps;
    String descr;

    public void printKursInfos() {
        System.out.println("Kurs:    " + descr);
        System.out.println("Credits: " + cps);
    }
}

public class Studi extends Person {
    String name;
    int cps;
    Kurs kurs;

    public void printKursInfos() { kurs.printKursInfos(); }
}
```

2.3.5.9 Refactoring-Methode: Pull Up, Push Down (Field, Method)

2.3.5.9.1 Motivation

- Attribut/Methode nur für die Oberklasse relevant: **Pull Up**
- Subklassen haben identische Attribute/Methoden: **Pull Up**
- Attribut/Methode nur für eine Subklasse relevant: **Push Down**

2.3.5.9.2 Durchführung Name selektieren, “`Refactor > Pull Up`” oder “`Refactor > Push Down`”

2.3.5.9.3 Anschließend ggf. prüfen Referenzen/Aufrufer? JUnit-Tests?

2.3.5.9.4 Beispiel Vorher

```
1  public class Person { }
2
3  public class Studi extends Person {
4      String name;
5      public void printDetails() { System.out.println("name:    " + name); }
6  }
```

Nachher

```
1  public class Person { protected String name; }
2
3  public class Studi extends Person {
4      public void printDetails() { System.out.println("name:    " + name); }
5  }
```

2.3.5.10 Wrap-Up Behebung von **Bad Smells** durch **Refactoring**

=> Änderung der inneren Struktur ohne Beeinflussung des äußeren Verhaltens

- Verbessert Lesbarkeit, Verständlichkeit, Wartbarkeit
- Immer nur kleine Schritte machen
- Nach jedem Schritt Testsuite laufen lassen
- Katalog von Maßnahmen, beispielsweise *Rename*, *Extract*, *Move*, *Push Up/Pull Down*, …
- Unterstützung durch IDEs wie Eclipse, Idea, …

Zum Nachlesen

- Fowler (2011)
- Inden (2013, Kap. 11)

Lernziele

- k2: Begriff, Notwendigkeit und Vorgehen des/beim Refactoring
- k2: Bedeutung kleiner Schritte beim Refactoring
- k2: Bedeutung einer sinnvollen Testsuite beim Refactoring
- k2: Refactoring: Nur innere Struktur ändern, nicht äußeres Verhalten!
- k3: Anwendung der wichtigsten Refactoring-Methoden: Rename, Extract, Move, Push Up/Pull Down

Quizzes

- Quiz Refactoring (ILIAS)

Challenges

In der Vorgabe finden Sie einige Klassen mit unübersichtlichem und schlecht strukturierten Code.
Welche *Bad Smells* können Sie hier identifizieren?
Beheben Sie die Smells durch die *schrittweise Anwendung* von den aus der Vorlesung bekannten Refactoring-Methoden. Wenden Sie dabei *mindestens* die unten genannten Methoden an. Wenn Sie keinen passenden Smell identifizieren können, suchen Sie sich eine geeignete Stelle, um die jeweilige Methode anzuwenden. Denken Sie auch daran, dass Refactoring immer durch eine entsprechende Testsuite abgesichert sein muss.
Ergänzend zu der Übersicht aus der Vorlesung finden sie unter Refactoring Guru eine erweiterte Auflistung der gängigen Refactoring-Techniken.

1. Extract Method/Class
2. Move Method/Field
3. Encapsulate Method/Field
4. Pull Up oder Push Down

2.4 Bauen von Programmen, Automatisierung, Continuous Integration

2.4.1 Build-Systeme: Gradle

TL;DR

Um beim Übersetzen und Testen von Software von den spezifischen Gegebenheiten auf einem Entwicklerrechner unabhängig zu werden, werden häufig sogenannte Build-Tools eingesetzt. Mit diesen konfiguriert man sein Projekt abseits einer IDE und übersetzt, testet und baut seine Applikation damit entsprechend unabhängig. In der Java-Welt sind aktuell die Build-Tools Ant, Maven und Gradle weit verbreitet.
In Gradle ist ein Java-Entwicklungsmodell quasi eingebaut. Über die Konfigurationsskripte müssen nur noch bestimmte Details wie benötigte externe Bibliotheken oder die Hauptklasse und sonstige Projektbesonderheiten konfiguriert werden. Über "Tasks" wie `build`, `test` oder `run` können Java-Projekte übersetzt, getestet und ausgeführt werden. Dabei werden die externen Abhängigkeiten (Bibliotheken) aufgelöst (soweit konfiguriert) und auch abhängige Tasks mit erledigt, etwa muss zum Testen vorher der Source-Code übersetzt werden.
Gradle bietet eine Fülle an Plugins für bestimmte Aufgaben an, die jeweils mit neuen Tasks einher kommen. Beispiele sind das Plugin `java`, welches weitere Java-spezifische Tasks wie `classes` mitbringt, oder das Plugin `checkstyle` zum Überprüfen von Coding-Style-Richtlinien.

Videos

- VL Build-Systeme: Gradle
- Demo Gradle

2.4.1.1 Automatisieren von Arbeitsabläufen

Works on my machine …

Einen häufigen Ausspruch, den man bei der Zusammenarbeit in Teams zu hören bekommt, ist “Also, bei mir läuft der Code.” …

Das Problem dabei ist, dass jeder Entwickler eine andere Maschine hat, oft ein anderes Betriebssystem oder eine andere OS-Version. Dazu kommen noch eine andere IDE und/oder andere Einstellungen und so weiter.

Wie bekommt man es hin, dass Code zuverlässig auch auf anderen Rechnern baut? Ein wichtiger Baustein dafür sind sogenannte “Build-Systeme”, also Tools, die unabhängig von der IDE (und den IDE-Einstellungen) für das Übersetzen der Software eingesetzt werden und deren Konfiguration dann mit im Repo eingecheckt wird. Damit kann die Software dann auf allen Rechnern und insbesondere dann auch auf dem Server (Stichwort “Continuous Integration”) unabhängig von der IDE o.ä. automatisiert gebaut und getestet werden.

- Build-Tools:
  - Apache Ant
  - Apache Maven
  - **Gradle**

Das sind die drei am häufigsten anzutreffenden Build-Tools in der Java-Welt.

Ant ist von den drei genannten Tools das älteste und setzt wie Maven auf XML als Beschreibungssprache. In Ant müssen dabei alle Regeln stets explizit formuliert werden, die man benutzen möchte.

In Maven wird dagegen von einem bestimmten Entwicklungsmodell ausgegangen, hier müssen nur noch die Abweichungen zu diesem Modell konfiguriert werden.

In Gradle wird eine DSL basierend auf der Skriptsprache Groovy (läuft auf der JVM) eingesetzt, und es gibt hier wie in Maven ein bestimmtes eingebautes Entwicklungsmodell. Gradle bringt zusätzlich noch einen Wrapper mit, d.h. es wird eine Art Gradle-Starter im Repo konfiguriert, der sich quasi genauso verhält wie ein fest installiertes Gradle (s.u.).

Achtung: Während Ant und Maven relativ stabil in der API sind, verändert sich Gradle teilweise deutlich zwischen den Versionen. Zusätzlich sind bestimmte Gradle-Versionen oft noch von bestimmten JDK-Versionen abhängig. In der Praxis bedeutet dies, dass man Gradle-Skripte im Laufe der Zeit relativ oft überarbeiten muss (einfach nur, damit das Skript wieder läuft - ohne dass man dabei irgendwelche neuen Features oder sonstige Vorteile erzielen würde). Ein großer Vorteil ist aber der Gradle-Wrapper (s.u.).

2.4.1.2 Gradle: Eine DSL in Groovy

DSL: *Domain Specific Language*

```
// build.gradle
plugins {
    id 'java'
    id 'application'
}

repositories {
    mavenCentral()
}

dependencies {
    testImplementation 'junit:junit:4.13.2'
}

application {
    mainClass = 'fluppie.App'
}
```

Dies ist mit die einfachste Build-Datei für Gradle.

Über Plugins wird die Unterstützung für Java und das Bauen von Applikationen aktiviert, d.h. es stehen darüber entsprechende spezifische Tasks zur Verfügung.

Abhängigkeiten sollen hier aus dem Maven-Repository MavenCentral geladen werden. Zusätzlich wird hier als Abhängigkeit für den Test (`testImplementation`) die JUnit-Bibliothek in einer Maven-artigen Notation angegeben (vgl. mvnrepository.com). (Für nur zur Übersetzung der Applikation benötigte Bibliotheken verwendet man stattdessen das Schlüsselwort `implementation`.)

Bei der Initialisierung wurde als Package `fluppie` angegeben. Gradle legt darunter per Default die Klasse `App` mit einer `main()`-Methode an. Entsprechend kann man über den Eintrag `application` den Einsprungpunkt in die Applikation konfigurieren.

2.4.1.3 Gradle-DSL

Ein Gradle-Skript ist letztlich ein in Groovy geschriebenes Skript. Groovy ist eine auf Java basierende und auf der JVM ausgeführte Skriptsprache. Seit einigen Versionen kann man die Gradle-Build-Skripte auch in der Sprache Kotlin schreiben.

2.4.1.4 Dateien

Für das Bauen mit Gradle benötigt man drei Dateien im Projektordner:

- `build.gradle`: Die auf der Gradle-DSL beruhende Definition des Builds mit den Tasks (und ggf. Abhängigkeiten) eines Projekts.

  Ein Multiprojekt hat pro Projekt eine solche Build-Datei. Dabei können die Unterprojekte Eigenschaften der Eltern-Buildskripte “erben” und so relativ kurz ausfallen.

- `settings.gradle`: Eine optionale Datei, in der man beispielsweise den Projektnamen oder bei einem Multiprojekt die relevanten Unterprojekte festlegt.

- `gradle.properties`: Eine weitere optionale Datei, in der projektspezifische Properties für den Gradle-Build spezifizieren kann.

2.4.1.5 Gradle Init Um eine neue Gradle-Konfiguration anlegen zu lassen, geht man in einen Ordner und führt darin `gradle init` aus. Gradle fragt der Reihe nach einige Einstellungen ab:

```
$ gradle init

Select type of project to generate:
  1: basic
  2: application
  3: library
  4: Gradle plugin
Enter selection (default: basic) [1..4] 2

Select implementation language:
  1: C++
  2: Groovy
  3: Java
  4: Kotlin
  5: Scala
  6: Swift
Enter selection (default: Java) [1..6] 3

Split functionality across multiple subprojects?:
  1: no - only one application project
  2: yes - application and library projects
Enter selection (default: no - only one application project) [1..2] 1

Select build script DSL:
  1: Groovy
  2: Kotlin
Enter selection (default: Groovy) [1..2] 1

Select test framework:
  1: JUnit 4
  2: TestNG
  3: Spock
  4: JUnit Jupiter
Enter selection (default: JUnit Jupiter) [1..4] 1

Project name (default: tmp): wuppie
Source package (default: tmp): fluppie
```

Typischerweise möchte man eine Applikation bauen (Auswahl 2 bei der ersten Frage). Als nächstes wird nach der Sprache des Projekts gefragt sowie nach der Sprache für das Gradle-Build-Skript (Default ist Groovy) sowie nach dem Testframework, welches verwendet werden soll.

Damit wird die eingangs gezeigte Konfiguration angelegt.

2.4.1.6 Ordner Durch `gradle init` wird ein neuer Ordner `wuppie/` mit folgender Ordnerstruktur angelegt:

```
drwxr-xr-x 4 cagix cagix 4096 Apr  8 11:43 ./
drwxrwxrwt 1 cagix cagix 4096 Apr  8 11:43 ../
-rw-r--r-- 1 cagix cagix  154 Apr  8 11:43 .gitattributes
-rw-r--r-- 1 cagix cagix  103 Apr  8 11:43 .gitignore
drwxr-xr-x 3 cagix cagix 4096 Apr  8 11:43 app/
```

```
drwxr-xr-x 3 cagix cagix 4096 Apr  8 11:42 gradle/
-rwxr-xr-x 1 cagix cagix 8070 Apr  8 11:42 gradlew*
-rw-r--r-- 1 cagix cagix 2763 Apr  8 11:42 gradlew.bat
-rw-r--r-- 1 cagix cagix  370 Apr  8 11:43 settings.gradle
```

Es werden Einstellungen für Git erzeugt (`.gitattributes` und `.gitignore`).

Im Ordner `gradle/` wird der Gradle-Wrapper abgelegt (s.u.). Dieser Ordner wird normalerweise mit ins Repo eingecheckt. Die Skripte `gradlew` und `gradlew.bat` sind die Startskripte für den Gradle-Wrapper (s.u.) und werden normalerweise ebenfalls ins Repo mit eingecheckt.

Der Ordner `.gradle/` (erscheint ggf. nach dem ersten Lauf von Gradle auf dem neuen Projekt) ist nur ein Hilfsordner ("Cache") von Gradle. Hier werden heruntergeladene Dateien etc. abgelegt. Dieser Order sollte **nicht** ins Repo eingecheckt werden und ist deshalb auch per Default im generierten `.gitignore` enthalten. (Zusätzlich gibt es im User-Verzeichnis auch noch einen Ordner `.gradle/` mit einem globalen Cache.)

In `settings.gradle` finden sich weitere Einstellungen. Die eigentliche Gradle-Konfiguration befindet sich zusammen mit dem eigentlichen Projekt im Unterordner `app/`:

```
drwxr-xr-x 4 root root 4096 Apr  8 11:50 ./
drwxr-xr-x 5 root root 4096 Apr  8 11:49 ../
drwxr-xr-x 5 root root 4096 Apr  8 11:50 build/
-rw-r--r-- 1 root root  852 Apr  8 11:43 build.gradle
drwxr-xr-x 4 root root 4096 Apr  8 11:43 src/
```

Die Datei `build.gradle` ist die durch `gradle init` erzeugte (und eingangs gezeigte) Konfigurationsdatei, vergleichbar mit `build.xml` für Ant oder `pom.xml` für Maven. Im Unterordner `build/` werden die generierten `.class`-Dateien etc. beim Build-Prozess abgelegt.

Unter `src/` findet sich dann eine Maven-typische Ordnerstruktur für die Sourcen:

```
$ tree src/
src/
|-- main
|   |-- java
|   |   `-- fluppie
|   |       `-- App.java
|   `-- resources
`-- test
    |-- java
    |   `-- fluppie
    |       `-- AppTest.java
    `-- resources
```

Unterhalb von `src/` ist ein Ordner `main/` für die Quellen der Applikation (Sourcen und Ressourcen). Für jede Sprache gibt es einen eigenen Unterordner, hier entsprechend `java/`. Unterhalb diesem folgt dann die bei der Initialisierung angelegte Package-Struktur (hier `fluppie` mit der Default-Main-Klasse `App` mit einer `main()`-Methode). Diese Strukturen wiederholen sich für die Tests unterhalb von `src/test/`.

Wer die herkömmlichen, deutlich flacheren Strukturen bevorzugt, also unterhalb von `src/` direkt die Java-Package-Strukturen für die Sourcen der Applikation und unterhalb von `test/` entsprechend die Strukturen für die JUnit-Test, der kann dies im Build-Skript einstellen:

```
sourceSets {
    main {
        java {
            srcDirs = ['src']
        }
        resources {
            srcDirs = ['res']
        }
    test {
        java {
            srcDirs = ['test']
        }
    }
}
```

2.4.1.7 Ablauf eines Gradle-Builds Ein Gradle-Build hat zwei Hauptphasen: Konfiguration und Ausführung.

Während der Konfiguration wird das gesamte Skript durchlaufen (vgl. Ausführung der direkten Anweisungen eines Tasks). Dabei wird ein Graph erzeugt: welche Tasks hängen von welchen anderen ab etc.

Anschließend wird der gewünschte Task ausgeführt. Dabei werden zuerst alle Tasks ausgeführt, die im Graphen auf dem Weg zu dem gewünschten Task liegen.

Mit `gradle tasks` kann man sich die zur Verfügung stehenden Tasks ansehen. Diese sind der Übersicht halber noch nach "Themen" sortiert.

Für eine Java-Applikation sind die typischen Tasks `gradle build` zum Bauen der Applikation (inkl. Ausführen der Tests) sowie `gradle run` zum Starten der Anwendung. Wer nur die Java-Sourcen compilieren will, würde den Task `gradle compileJava` nutzen. Mit `gradle check` würde man compilieren und die Tests ausführen sowie weitere Checks durchführen (`gradle test` würde nur compilieren und die Tests ausführen), mit `gradle jar` die Anwendung in ein `.jar`-File packen und mit `gradle javadoc` die Javadoc-Dokumentation erzeugen und mit `gradle clean` die generierten Hilfsdateien aufräumen (löschen).

2.4.1.8 Plugin-Architektur Für bestimmte Projekttypen gibt es immer wieder die gleichen Aufgaben. Um hier Schreibaufwand zu sparen, existieren verschiedene Plugins für verschiedene Projekttypen. In diesen Plugins sind die entsprechenden Tasks bereits mit den jeweiligen Abhängigkeiten formuliert. Diese Idee stammt aus Maven, wo dies für Java-basierte Projekte umgesetzt ist.

Beispielsweise erhält man über das Plugin `java` den Task `clean` zum Löschen aller generierten Build-Artefakte, den Task `classes`, der die Sourcen zu `.class`-Dateien kompiliert oder den Task `test`, der die JUnit-Tests ausführt …

Sie können sich Plugins und weitere Tasks relativ leicht auch selbst definieren.

2.4.1.9 Auflösen von Abhängigkeiten Analog zu Maven kann man Abhängigkeiten (etwa in einer bestimmten Version benötigte Bibliotheken) im Gradle-Skript angeben. Diese werden (transparent für den User) von einer ebenfalls angegeben Quelle, etwa einem Maven-Repository, heruntergeladen und für den Build genutzt. Man muss also nicht mehr die benötigten `.jar`-Dateien der Bibliotheken mit ins Projekt einchecken. Analog zu Maven können erzeugte Artefakte automatisch publiziert werden, etwa in einem Maven-Repository.

Für das Projekt benötigte Abhängigkeiten kann man über den Eintrag `dependencies` spezifizieren. Dabei unterscheidet man u.a. zwischen Applikation und Tests: `implementation` und `testImplementation` für das Compilieren und Ausführen von Applikation bzw. Tests. Diese Abhängigkeiten werden durch Gradle über die im Abschnitt `repositories` konfigurierten Repositories aufgelöst und die entsprechenden `.jar`-Files geladen (in den `.gradle`/-Ordner).

Typische Repos sind das Maven-Repo selbst (`mavenCentral()`) oder das Google-Maven-Repo (`google()`).

Die Einträge in `dependencies` erfolgen dabei in einer Maven-Notation, die Sie auch im Maven-Repo mvnrepository.com finden.

2.4.1.10 Beispiel mit weiteren Konfigurationen (u.a. Checkstyle und Javadoc)

```
plugins {
    id 'java'
    id 'application'
    id 'checkstyle'
}


repositories {
    mavenCentral()
}

application {
    mainClass = 'hangman.Main'
}

java {
    sourceCompatibility = JavaVersion.VERSION_11
    targetCompatibility = JavaVersion.VERSION_11
}

run {
    standardInput = System.in
}

sourceSets {
    main {
        java {
            srcDirs = ['src']
        }
        resources {
            srcDirs = ['res']
        }
    }
}

checkstyle {
    configFile = file("${rootDir}/google_checks.xml")
    toolVersion = '8.32'
}

dependencies {
    implementation group: 'org.apache.poi', name: 'poi', version: '4.1.2'
}
```

```
44
45 javadoc {
46     options.showAll()
47 }
```

Hier sehen Sie übrigens noch eine weitere mögliche Schreibweise für das Notieren von Abhängigkeiten: `implementation group: 'org.apache.poi', name: 'poi', version: '4.1.2'` und `implementation 'org.apache.poi:poi:4.1.2'` sind gleichwertig, wobei die letztere Schreibweise sowohl in den generierten Builds-Skripten und in der offiziellen Dokumentation bevorzugt wird.

2.4.1.11 Gradle und Ant (und Maven)

Vorhandene Ant-Buildskripte kann man nach Gradle importieren und ausführen lassen. Über die DSL kann man auch direkt Ant-Tasks aufrufen. Siehe auch "Using Ant from Gradle".

2.4.1.12 Gradle-Wrapper

```
1 project
2 |-- app/
3 |-- build.gradle
4 |-- gradlew
5 |-- gradlew.bat
6 `-- gradle/
7     `-- wrapper/
8         |-- gradle-wrapper.jar
9         `-- gradle-wrapper.properties
```

Zur Ausführung von Gradle-Skripten benötigt man eine lokale Gradle-Installation. Diese sollte für i.d.R. alle User, die das Projekt bauen wollen, identisch sein. Leider ist dies oft nicht gegeben bzw. nicht einfach lösbar.

Zur Vereinfachung gibt es den Gradle-Wrapper `gradlew` (bzw. `gradlew.bat` für Windows). Dies ist ein kleines Shellskript, welches zusammen mit einigen kleinen `.jar`-Dateien im Unterordner `gradle/` mit ins Repo eingecheckt wird und welches direkt die Rolle des `gradle`-Befehls einer Gradle-Installation übernehmen kann. Man kann also in Konfigurationskripten, beispielsweise für Gitlab CI, alle Aufrufe von `gradle` durch Aufrufe von `gradlew` ersetzen.

Beim ersten Aufruf lädt `gradlew` dann die spezifizierte Gradle-Version herunter und speichert diese in einem lokalen Ordner `.gradle/`. Ab dann greift `gradlew` auf diese lokale (nicht "installierte") `gradle`-Version zurück.

`gradle init` erzeugt den Wrapper automatisch in der verwendeten Gradle-Version mit. Alternativ kann man den Wrapper nachträglich über `gradle wrapper --gradle-version 6.5` in einer bestimmten (gewünschten) Version anlegen lassen.

Da der Gradle-Wrapper im Repository eingecheckt ist, benutzen alle Entwickler damit automatisch die selbe Version, ohne diese auf ihrem System zuvor installieren zu müssen. Deshalb ist der Einsatz des Wrappers einem fest installierten Gradle vorzuziehen!

2.4.1.13 Wrap-Up

- Automatisieren von Arbeitsabläufen mit Build-Tools/-Skripten

- Einstieg in **Gradle** (DSL zur Konfiguration)
  - Typisches Java-Entwicklungsmodell eingebaut
  - Konfiguration der Abweichungen (Abhängigkeiten, Namen, …)
  - Gradle-Wrapper: Ersetzt eine feste Installation

2.4.1.14 Link-Sammlung Gradle

- "Getting Started"
- "Building Java Applications Sample"
- "Building Java Applications with libraries Sample"
- "Building Java Libraries Sample"
- "Building Java & JVM projects"

Zum Nachlesen

- Gradle Inc. (2022)
- Ullenboom (2021)
- Inden (2013)

Lernziele

- k3: Schreiben und Verstehen einfacher Gradle-Skripte

Quizzes

- Quiz Gradle (ILIAS)

Challenges

Betrachten Sie das Buildskript `gradle.build` aus Dungeon-CampusMinden/Dungeon.
Erklären Sie, in welche Abschnitte das Buildskript unterteilt ist und welche Aufgaben diese Abschnitte jeweils erfüllen.
Gehen Sie dabei im *Detail* auf das Plugin `java` und die dort bereitgestellten Tasks und deren Abhängigkeiten untereinander ein.

2.4.2 Continuous Integration (CI)

TL;DR
In größeren Projekten mit mehreren Teams werden die Beteiligten i.d.R. nur noch "ihre" Codestellen compilieren und testen. Dennoch ist es wichtig, das gesamte Projekt regelmäßig zu "bauen" und auch umfangreichere Testsuiten regelmäßig laufen zu lassen. Außerdem ist es wichtig, das in einer definierten Umgebung zu tun und nicht auf einem oder mehreren Entwicklerrechnern, die i.d.R. (leicht) unterschiedlich konfiguriert sind, um zuverlässige und nachvollziehbare Ergebnisse zu bekommen. Weiterhin möchte man auf bestimmte Ereignisse reagieren, wie etwa neue Commits im Git-Server, oder bei Pull-Requests möchte man vor dem Merge automatisiert sicherstellen, dass damit die vorhandenen Tests alle "grün" sind und auch die Formatierung etc. stimmt.
Dafür hat sich "Continuous Integration" etabliert. Hier werden die angesprochenen Prozesse regelmäßig auf einem dafür eingerichteten System durchgeführt. Aktivitäten wie Übersetzen, Testen, Style-Checks etc. werden in sogenannten "Pipelines" oder "Workflows" zusammengefasst und automatisiert durch Commits, Pull-Requests oder Merges auf dem Git-Server ausgelöst. Die Aktionen können dabei je nach Trigger und Branch unterschiedlich sein, d.h. man könnte etwa bei PR gegen den Master umfangreichere Tests laufen lassen als bei einem PR gegen einen Develop-Branch. In einem Workflow oder einer Pipeline können einzelne Aktionen wiederum von anderen Aktionen abhängen. Das Ergebnis kann man dann auf dem Server einsehen oder bekommt man komfortabel als Report per Mail zugeschickt.
Wir schauen uns hier exemplarisch GitHub Actions und GitLab CI/CD an. Um CI sinnvoll einsetzen zu können, benötigt man Kenntnisse über Build-Tools. "CI" tritt üblicherweise zusammen mit "CD" (Continuous Delivery) auf, also als "CI/CD". Der "CD"-Teil ist nicht Gegenstand der Betrachtung in dieser Lehrveranstaltung.

Videos

- VL Continuous Integration
- Demo GitHub Actions
- Demo Demo GitLab CI/CD

2.4.2.1 Motivation: Zusammenarbeit in Teams

2.4.2.1.1 Szenario

- Projekt besteht aus diversen Teilprojekten
- Verschiedene Entwicklungs-Teams arbeiten (getrennt) an verschiedenen Projekten
- Tester entwickeln Testsuiten für die Teilprojekte
- Tester entwickeln Testsuiten für das Gesamtprojekt

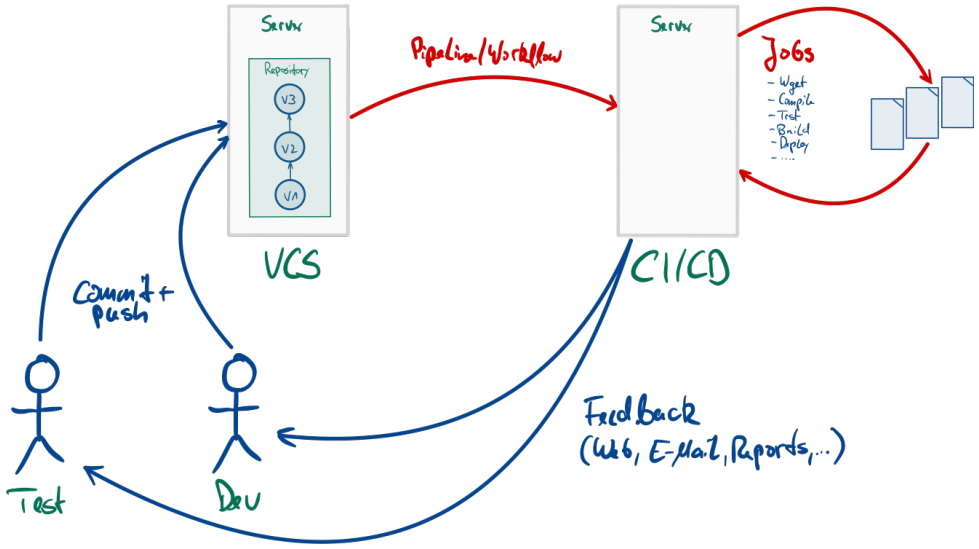
2.4.2.1.2 Manuelle Ausführung der Testsuiten reicht nicht

- Belastet den Entwicklungsprozess
- Keine (einheitliche) Veröffentlichung der Ergebnisse
- Keine (einheitliche) Eskalation bei Fehlern
- Keine regelmäßige Integration in Gesamtprojekt

2.4.2.1.3 Continuous Integration

- Regelmäßige, automatische Ausführung: Build und Tests
- Reporting
- Weiterführung der Idee: Regelmäßiges Deployment (*Continuous Deployment*)

2.4.2.2 Continuous Integration (CI)



2.4.2.2.1 Vorgehen

- Entwickler und Tester committen ihre Änderungen regelmäßig (Git, SVN, …)
- CI-Server arbeitet Build-Skripte ab, getriggert durch Events: Push-Events, Zeit/Datum, …
  - Typischerweise wird dabei:
    * Das Gesamtprojekt übersetzt (“gebaut”)
    * Die Unit- und die Integrationstests abgearbeitet
    * Zu festen Zeiten werden zusätzlich Systemtests gefahren
  - Typische weitere Builds: “Nightly Build”, Release-Build, …
  - Ergebnisse jeweils auf der Weboberfläche einsehbar (und per E-Mail)

2.4.2.2.2 Einige Vorteile

- Tests werden regelmäßig durchgeführt (auch wenn sie lange dauern oder die Maschine stark belasten)
- Es wird regelmäßig ein Gesamt-Build durchgeführt
- Alle Teilnehmer sind über aktuellen Projekt(-zu-)stand informiert

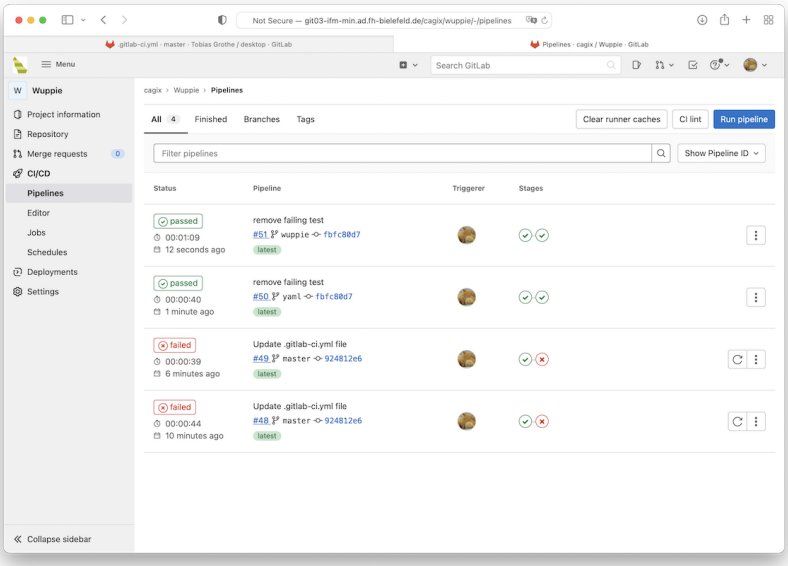
2.4.2.2.3 Beispiele für verbreitete CI-Umgebungen

- Jenkins
- GitLab CI/CD
- GitHub Actions und GitHub CI/CD
- Bamboo
- Travis CI

2.4.2.3 GitLab CI/CD

Siehe auch "Get started with Gitlab CI/CD". (Für den Zugriff wird VPN benötigt!)

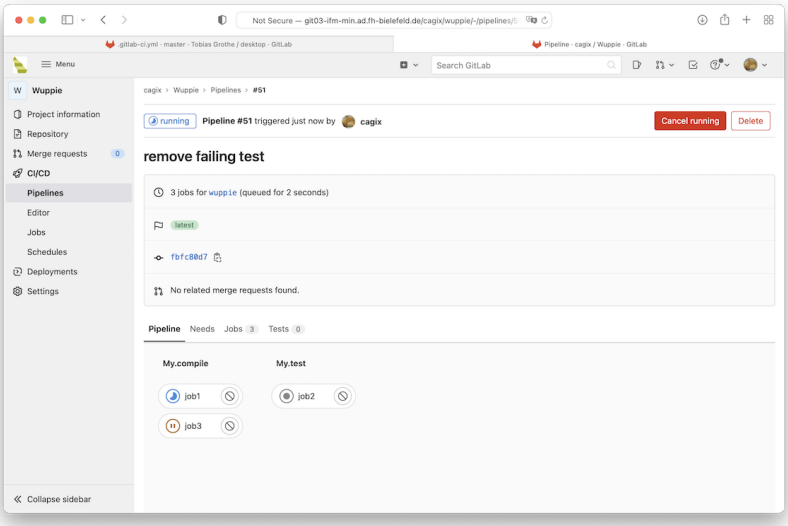
2.4.2.3.1 Übersicht über Pipelines



- In Spalte "Status" sieht man das Ergebnis der einzelnen Pipelines: "pending" (die Pipeline läuft gerade), "cancelled" (Pipeline wurde manuell abgebrochen), "passed" (alle Jobs der Pipeline sind sauber durchgelaufen), "failed" (ein Job ist fehlgeschlagen, Pipeline wurde deshalb abgebrochen)
- In Spalte "Pipeline" sind die Pipelines eindeutig benannt aufgeführt, inkl. Trigger (Commit und Branch)
- In Spalte "Stages" sieht man den Zustand der einzelnen Stages

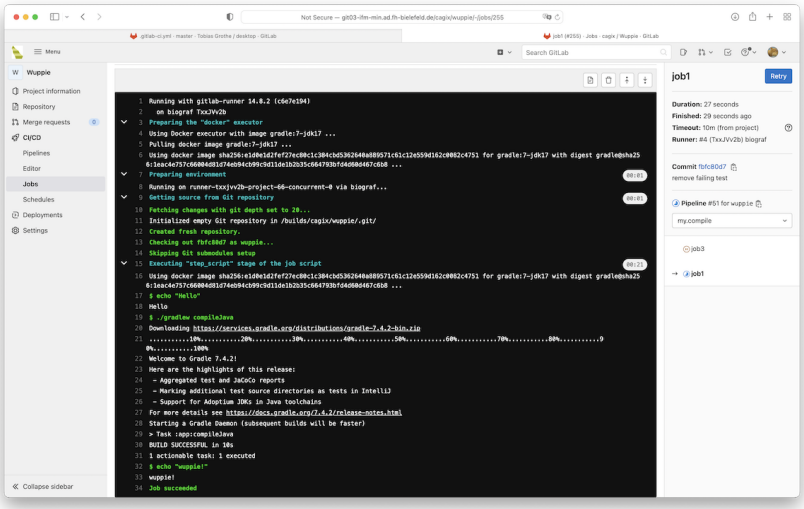
Wenn man mit der Maus auf den Status oder die Stages geht, erfährt man mehr bzw. kann auf eine Seite mit mehr Informationen kommen.

2.4.2.3.2 Detailansicht einer Pipeline



Wenn man in eine Pipeline in der Übersicht klickt, werden die einzelnen Stages dieser Pipeline genauer dargestellt.

2.4.2.3.3 Detailansicht eines Jobs



Wenn man in einen Job einer Stage klickt, bekommt man quasi die Konsolenausgabe dieses Jobs. Hier kann man ggf. Fehler beim Ausführen der einzelnen Skripte oder die Ergebnisse beispielsweise der JUnit-Läufe anschauen.

2.4.2.3.4 GitLab CI/CD: Konfiguration mit YAML-Datei

Datei `.gitlab-ci.yml` im Projekt-Ordner:

```yaml
stages:
    - my.compile
    - my.test

job1:
    script:
        - echo "Hello"
        - ./gradlew compileJava
        - echo "wuppie!"
    stage: my.compile
    only:
        - wuppie

job2:
    script: "./gradlew test"
    stage: my.test

job3:
    script:
        - echo "Job 3"
    stage: my.compile
```

Stages

Unter `stages` werden die einzelnen Stages einer Pipeline definiert. Diese werden in der hier spezifizierten Reihenfolge durchgeführt, d.h. zuerst würde `my.compile` ausgeführt, und erst wenn alle Jobs in `my.compile` erfolgreich ausgeführt wurden, würde anschließend `my.test` ausgeführt.

Dabei gilt: Die Jobs einer Stage werden (potentiell) parallel zueinander ausgeführt, und die Jobs der nächsten Stage werden erst dann gestartet, wenn alle Jobs der aktuellen Stage erfolgreich beendet wurden.

Wenn keine eigenen `stages` definiert werden, kann man (lt. Doku) auf die Default-Stages `build`, `test` und `deploy` zurückgreifen. **Achtung**: Sobald man eigene Stages definiert, stehen diese Default-Stages *nicht* mehr zur Verfügung!

Jobs

`job1`, `job2` und `job3` definieren jeweils einen Job.

- `job1` besteht aus mehreren Befehlen (unter `script`). Alternativ kann man die bei `job2` gezeigte Syntax nutzen, wenn nur ein Befehl zu bearbeiten ist.

  Die Befehle werden von GitLab CI/CD in einer Shell ausgeführt.

- Die Jobs `job1` und `job2` sind der Stage `my.compile` zugeordnet (Abschnitt `stage`). Einer Stage können mehrere Jobs zugeordnet sein, die dann parallel ausgeführt werden.

  Wenn ein Job nicht explizit einer Stage zugeordnet ist, wird er (lt. Doku) zur Default-Stage `test` zugewiesen. (Das geht nur, wenn es diese Stage auch gibt!)

- Mit `only` und `except` kann man u.a. Branches oder Tags angeben, für die dieser Job ausgeführt (bzw. nicht ausgeführt) werden soll.

Durch die Kombination von Jobs mit der Zuordnung zu Stages und Events lassen sich unterschiedliche Pipelines für verschiedene Zwecke definieren.

2.4.2.3.5 Hinweise zur Konfiguration von GitLab CI/CD Im Browser in den Repo-Einstellungen arbeiten:

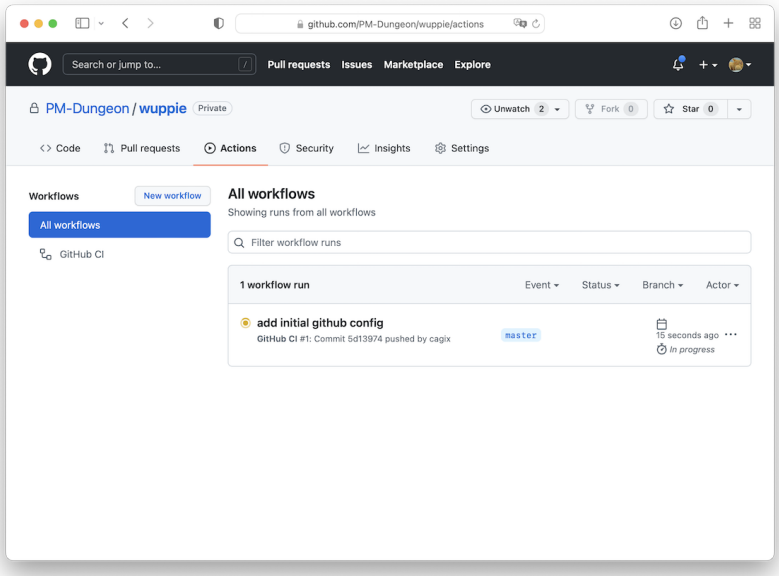
1. Unter `Settings` > `General` > `Visibility, project features, permissions` das `CI/CD` aktivieren
2. Prüfen unter `Settings` > `CI/CD` > `Runners`, dass unter `Available shared Runners` mind. ein shared Runner verfügbar ist (mit grün markiert ist)
3. Unter `Settings` > `CI/CD` > `General pipelines` einstellen:
   - `Git strategy`: `git clone`
   - `Timeout`: `10m`
   - `Public pipelines`: **`false`** (nicht angehakt)
4. YAML-File (`.gitlab-ci.yml`) in Projektwurzel anlegen, Aufbau siehe oben
5. Build-Skript erstellen, **lokal** lauffähig bekommen, dann in Jobs nutzen
6. Im `.gitlab-ci.yml` die relevanten Branches einstellen (s.o.)
7. Pushen, und unter `CI/CD` > `Pipelines` das Builden beobachten
   - in Status reinklicken und schauen, ob und wo es hakt
8. `README.md` anlegen in Projektwurzel (neben `.gitlab-ci.yml`), Markdown-Schnipsel aus `Settings` > `CI/CD` > `General pipelines` > `Pipeline status` auswählen und einfügen ....

Optional:

9. Ggf. Schedules unter `CI/CD` > `Schedules` anlegen
10. Ggf. extra Mails einrichten: `Settings` > `Integrations` > `Pipeline status emails`

2.4.2.4 GitHub Actions Siehe "GitHub Actions: Automate your workflow from idea to production" und auch "GitHub: CI/CD explained".

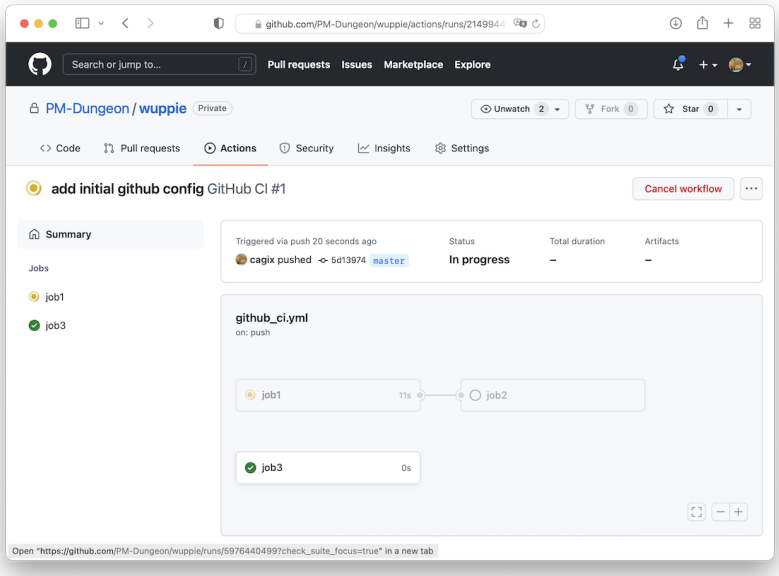
2.4.2.4.1 Übersicht über Workflows



Hier sieht man das Ergebnis der letzten Workflows. Dazu sieht man den Commit und den Branch, auf dem der Workflow gelaufen ist sowie wann er gelaufen ist. Über die Spalten kann man beispielsweise nach Status oder Event filtern.

In der Abbildung ist ein Workflow mit dem Namen “GitHub CI” zu sehen, der aktuell noch läuft.

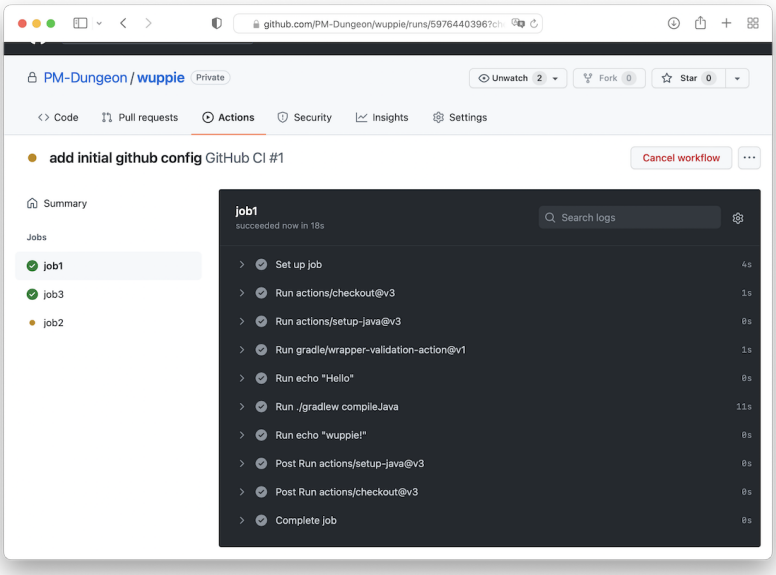
2.4.2.4.2 Detailansicht eines Workflows



Wenn man in einen Workflow in der Übersicht anklickt, werden die einzelnen Jobs dieses Workflows genauer

dargestellt. “job3” ist erfolgreich gelaufen, “job1” läuft gerade, und “job2” hängt von “job1” ab, d.h. kann erst nach dem erfolgreichen Lauf von “job2” starten.

2.4.2.4.3 Detailansicht eines Jobs



Wenn man in einen Job anklickt, bekommt man quasi die Konsolenausgabe dieses Jobs. Hier kann man ggf. Fehler beim Ausführen der einzelnen Skripte oder die Ergebnisse beispielsweise der JUnit-Läufe anschauen.

2.4.2.4.4 GitHub Actions: Konfiguration mit YAML-Datei Workflows werden als YAML-Dateien im Ordner `.github/workflows/` angelegt.

```yaml
name: GitHub CI

on:
  # push on master branch
  push:
    branches: [master]
  # manually triggered
  workflow_dispatch:

jobs:

  job1:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
```

```yaml
      - uses: gradle/wrapper-validation-action@v1
      - run: echo "Hello"
      - run: ./gradlew compileJava
      - run: echo "wuppie!"

  job2:
    needs: job1
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
      - uses: gradle/wrapper-validation-action@v1
      - run: ./gradlew test

  job3:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Job 3"
```

Workflowname und Trigger-Events

Der Name des Workflows wird mit dem Eintrag `name` spezifiziert und sollte sich im Dateinamen widerspiegeln, also im Beispiel `.github/workflows/github_ci.yml`.

Im Eintrag `on` können die Events definiert werden, die den Workflow triggern. Im Beispiel ist ein Push-Event auf dem `master`-Branch definiert sowie mit `workflow_dispatch:` das manuelle Triggern (auf einem beliebigen Branch) freigeschaltet.

Jobs

Die Jobs werden unter dem Eintrag `jobs` definiert: `job1`, `job2` und `job3` definieren jeweils einen Job.

- `job1` besteht aus mehreren Befehlen (unter `steps`), die auf einem aktuellen virtualisierten Ubuntu-Runner ausgeführt werden.

  Es wird zunächst das Repo mit Hilfe der Checkout-Action ausgecheckt (`uses: actions/checkout@v5`), das JDK eingerichtet/installiert (`uses: actions/setup-java@v3`) und der im Repo enthaltene Gradle-Wrapper auf Unversehrtheit geprüft (`uses: gradle/wrapper-validation-action@v1`).

  Die Actions sind vordefinierte Actions und im Github unter `github.com/` + Action zu finden, d.h. `actions/checkout` oder `actions/setup-java`. Actions können von jedermann definiert und bereitgestellt werden, in diesem Fall handelt es sich um von GitHub selbst im Namespace “actions” bereit gestellte direkt nutzbare Actions. Man kann Actions auch selbst im Ordner `.github/actions/` für das Repo definieren (Beispiel: plfa.github.io).

  Mit `run` werden Befehle in der Shell auf dem genutzten Runner (hier Ubuntu) ausgeführt.

- Die Jobs `job2` ist von `job1` abhängig und wird erst gestartet, wenn `job1` erfolgreich abgearbeitet ist.

  Ansonsten können die Jobs prinzipiell parallel ausgeführt werden.

Durch die Kombination von Workflows mit verschiedenen Jobs und Abhängigkeiten zwischen Jobs lassen sich unterschiedliche Pipelines (“Workflows”) für verschiedene Zwecke definieren.

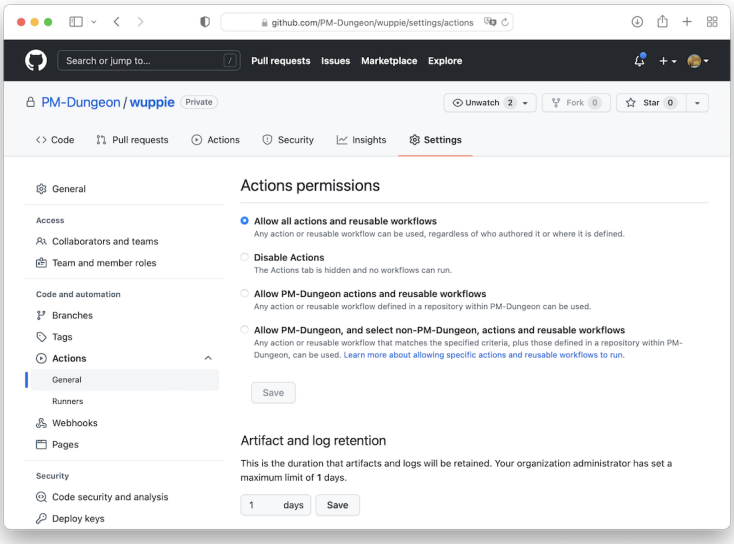
Es lassen sich auch andere Runner benutzen, etwa ein virtualisiertes Windows oder macOS. Man kann auch über einen “Matrix-Build” den Workflow auf mehreren Betriebssystemen gleichzeitig laufen lassen.

Man kann auch einen Docker-Container benutzen. Dabei muss man beachten, dass dieser am besten aus einer Registry (etwa von Docker-Hub oder aus der GitHub-Registry) “gezogen” wird, weil das Bauen des Docker-Containers aus einem Docker-File in der Action u.U. relativ lange dauert.

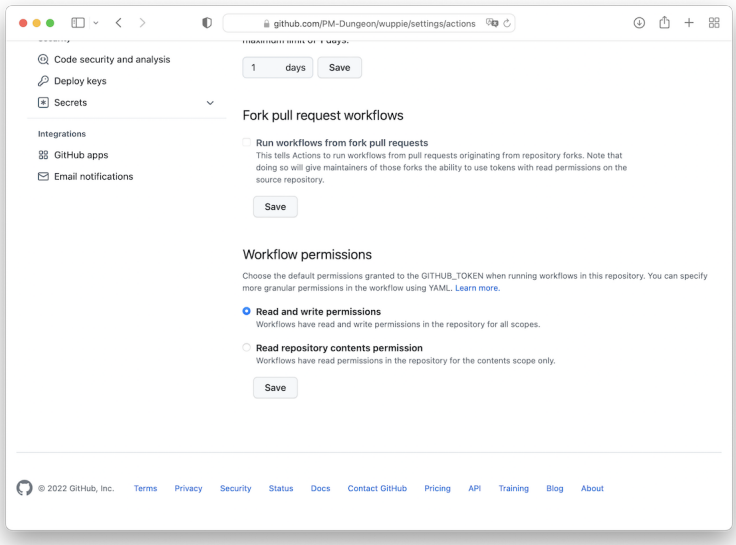
2.4.2.4.5 Hinweise zur Konfiguration von GitHub Actions

Im Browser in den Repo-Einstellungen arbeiten:

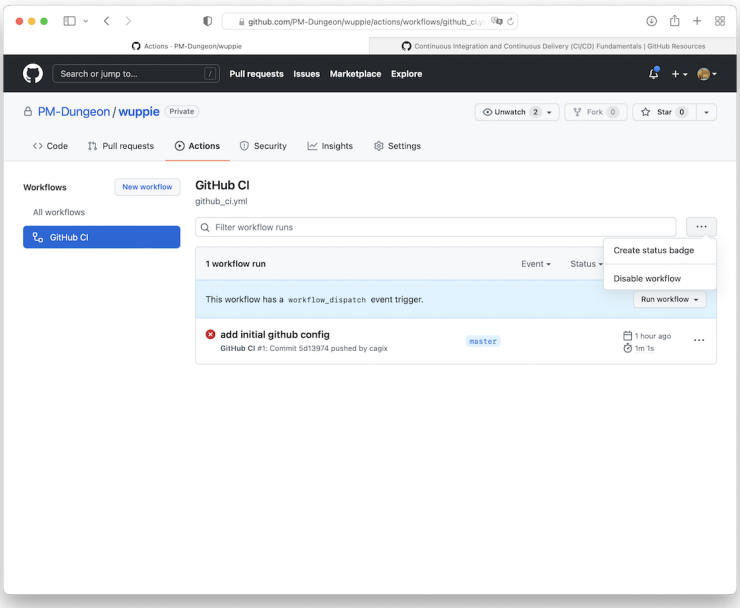
1. Unter `Settings` > `Actions` > `General` > `Actions permissions` die Actions aktivieren (Auswahl, welche Actions erlaubt sind)



2. Unter `Settings` > `Actions` > `General` > `Workflow permissions` ggf. bestimmen, ob die Actions das Repo nur lesen dürfen oder auch zusätzlich schreiben dürfen



3. Unter `Actions` > <`WORKFLOW`> den Workflow ggf. deaktivieren:



2.4.2.5 Wrap-Up Überblick über Continuous Integration:

- Konfigurierbare Aktionen, die auf dem Gitlab-/GitHub-Server ausgeführt werden
- Unterschiedliche Trigger: Commit, Merge, …
- Aktionen können Branch-spezifisch sein
- Aktionen können von anderen Aktionen abhängen

Zum Nachlesen

- „Documentation Gitlab CI“ (o. J.)
- GitHub Inc. (2022a)

Lernziele

- k2: Arbeitsweise von/mit CI

Quizzes

- Quiz Continuous Integration (ILIAS)

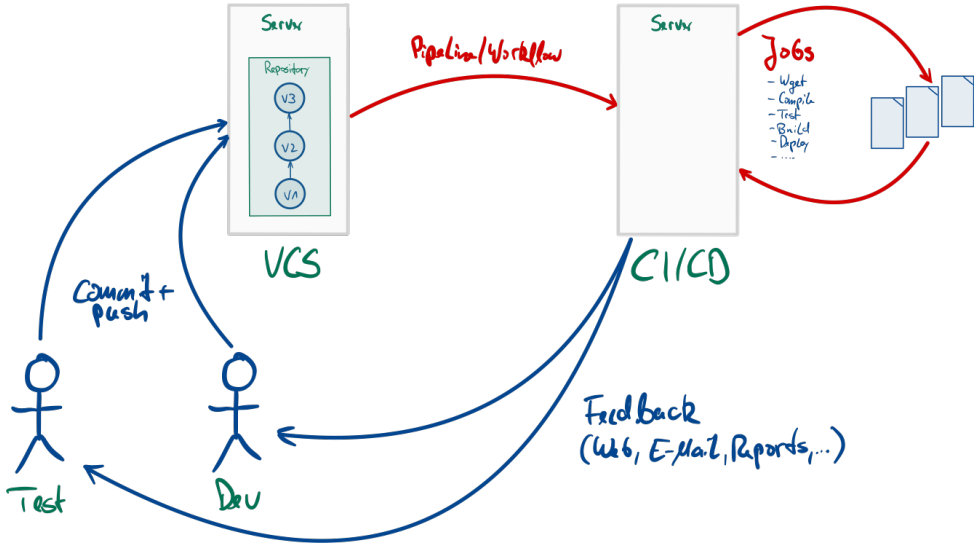
2.4.3 Einführung in Docker

TL;DR
Container sind im Gegensatz zu herkömmlichen VMs eine schlanke Virtualisierungslösung. Dabei laufen die Prozesse direkt im Kernel des Host-Betriebssystems, aber abgeschottet von den anderen Prozessen durch Linux-Techniken wie `cgroups` und `namespaces` (unter Windows kommt dafür der WSL2 zum Einsatz, unter macOS wird eine kleine Virtualisierung genutzt).
Container sind sehr nützlich, wenn man an mehreren Stellen eine identische Arbeitsumgebung benötigt. Man kann dabei entweder die Images (fertige Dateien) oder die Dockerfiles (Anweisungen zum Erzeugen eines Images) im Projekt verteilen. Tatsächlich ist es nicht unüblich, ein Dockerfile in das Projekt-Repo mit einzuchecken.
Durch Container hat man allerdings im Gegensatz zu herkömmlichen VMs keinen Sicherheitsgewinn, da die im Container laufende Software ja direkt auf dem Host-Betriebssystem ausgeführt wird.
Es gibt auf DockerHub fertige Images, die man sich ziehen und starten kann. Ein solches gestartetes Image nennt sich dann Container und enthält beispielsweise Dateien, die in den Container gemountet oder kopiert werden. Man kann auch eigene Images bauen, indem man eine entsprechende Konfiguration (Dockerfile) schreibt. Jeder Befehl bei der Erstellung eines Images erzeugt einen neuen Layer, die sich dadurch mehrere Images teilen können.
In der Konfiguration einer Gitlab-CI-Pipeline kann man mit `image` ein Docker-Image angeben, welches dann in der Pipeline genutzt wird.
VSCode kann über das Remote-Plugin sich (u.a.) mit Containern verbinden und dann im Container arbeiten (editieren, compilieren, debuggen, testen, …).
In dieser kurzen Einheit kann ich Ihnen nur einen ersten Einstieg in das Thema geben. Wir haben uns beispielsweise nicht Docker Compose oder Kubernetes angeschaut, und auch die Themen Netzwerk (zwischen Containern oder zwischen Containern und anderen Rechnern) und Volumnes habe ich außen vor gelassen. Dennoch kommt man in der Praxis bereits mit den hier vermittelten Basiskenntnissen erstaunlich weit …

Videos

- VL Einführung in Docker
- Demo Container in der Konsole
- Demo GitLab CI/CD und Docker
- Demo GitHub Actions und Docker
- Demo VSCode und Docker

2.4.3.1 Motivation CI/CD: WFM (*Works For Me*)



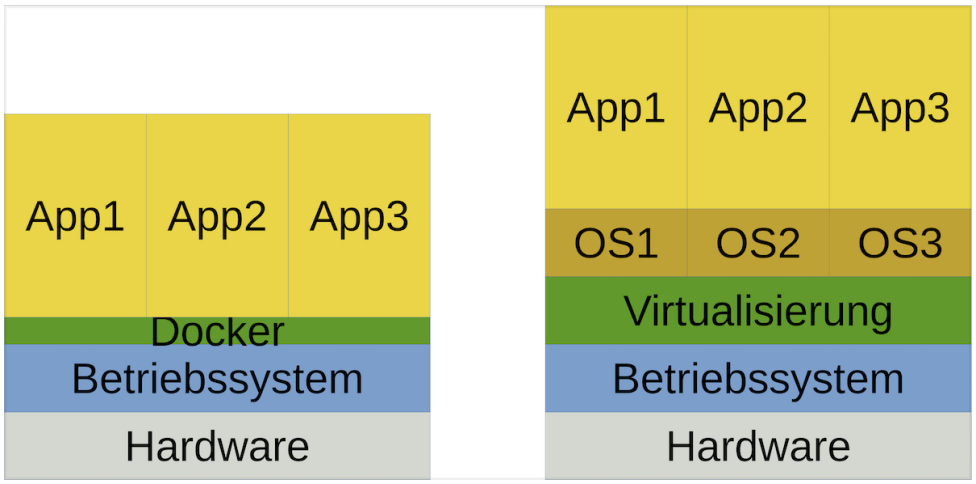
Auf dem CI-Server muss man eine Arbeitsumgebung konfigurieren und bereitstellen, für Java-basierte Projekte muss beispielsweise ein JDK existieren und man benötigt Tools wie Maven oder Gradle, um die Buildskripte auszuführen. Je nach Projekt braucht man dann noch weitere Tools und Bibliotheken. Diese Konfigurationen sind unabhängig vom CI-Server und werden üblicherweise nicht direkt installiert, sondern über eine Virtualisierung bereitgestellt.

Selbst wenn man keine CI-Pipelines einsetzt, hat man in Projekten mit mehreren beteiligten Personen häufig das Problem "*WFM*" ("works for me"). Jeder Entwickler hat sich auf ihrem Rechner eine Entwicklungsumgebung aufgesetzt und nutzt in der Regel seine bevorzugte IDE oder sogar unterschiedliche JDK-Versionen … Dadurch kann es schnell passieren, dass Probleme oder Fehler auftreten, die sich nicht von allen Beteiligten immer nachvollziehen lassen. Hier wäre eine einheitliche Entwicklungsumgebung sinnvoll, die in einer "schlanken" Virtualisierung bereitgestellt wird.

Als Entwickler kann man zeitgleich in verschiedenen Projekten beteiligt sein, die unterschiedliche Anforderungen an die Entwicklungstools mit sich bringen. Es könnte beispielsweise passieren, dass man zeitgleich drei bestimmte Python-Versionen benötigt. In den meisten Fällen schafft man es (mit ein wenig Aufwand), diese Tools nebeneinander zu installieren. Oft ist das in der Praxis aber schwierig und fehleranfällig.

In diesen Fällen kann eine Virtualisierung helfen.

2.4.3.2 Virtualisierung: Container vs. VM



Wenn man über Virtualisierung auf dem Desktop spricht, kann man grob zwei Varianten unterscheiden. In beiden Fällen ist die Basis die Hardware (Laptop, Desktop-Rechner) und das darauf laufende (Host-) Betriebssystem (Linux, FreeBSD, macOS, Windows, …). Darauf läuft dann wiederum die Virtualisierung.

Im rechten Bild wird eine herkömmliche Virtualisierung mit virtuellen Maschinen (*VM*) dargestellt. Dabei wird in der VM ein komplettes Betriebssystem (das "Gast-Betriebssystem") installiert und darin läuft dann die gewünschte Anwendung. Die Virtualisierung (VirtualBox, VMware, …) läuft dabei als Anwendung auf dem Host-Betriebssystem und stellt dem Gast-Betriebssystem in der VM einen Rechner mit CPU, RAM, … zur Verfügung und übersetzt die Systemaufrufe in der VM in die entsprechenden Aufrufe im Host-Betriebssystem. Dies benötigt in der Regel entsprechende Ressourcen: Durch das komplette Betriebssystem in der VM ist eine VM (die als Datei im Filesystem des Host-Betriebssystems liegt) oft mehrere 10GB groß. Für die Übersetzung werden zusätzlich Hardwareressourcen benötigt, d.h. hier gehen CPU-Zyklen und RAM "verloren" … Das Starten einer VM dauert entsprechend lange, da hier ein komplettes Betriebssystem hochgefahren werden muss. Dafür sind die Prozesse in einer VM relativ stark vom Host-Betriebssystem abgekapselt, so dass man hier von einer "Sandbox" sprechen kann: Viren o.ä. können nicht so leicht aus einer VM "ausbrechen" und auf das Host-Betriebssystem zugreifen (quasi nur über Lücken im Gast-Betriebssystem kombiniert mit Lücken in der Virtualisierungssoftware).

Im linken Bild ist eine schlanke Virtualisierung auf Containerbasis dargestellt. Die Anwendungen laufen direkt als Prozesse im Host-Betriebssystem, ein Gast-Betriebssystem ist nicht notwendig. Durch den geschickten Einsatz von `namespaces` und `cgroups` und anderen in Linux und FreeBSD verfügbaren Techniken werden die Prozesse abgeschottet, d.h. der im Container laufende Prozess "sieht" die anderen Prozesse des Hosts nicht. Die Erstellung und Steuerung der Container übernimmt hier beispielsweise Docker. Die Container sind dabei auch wieder Dateien im Host-Filesystem. Dadurch benötigen Container wesentlich weniger Platz als herkömmliche VMs, der Start einer Anwendung geht deutlich schneller und die Hardwareressourcen (CPU, RAM, …) werden effizient genutzt. Nachteilig ist, dass hier in der Regel ein Linux-Host benötigt wird (für Windows wird mittlerweile der Linux-Layer (*WSL*) genutzt; für macOS wurde bisher eine Linux-VM im Hintergrund hochgefahren, mittlerweile wird aber eine eigene schlanke Virtualisierung eingesetzt). Außerdem steht im Container üblicherweise kein graphisches Benutzerinterface zur Verfügung. Da die Prozesse direkt im Host-Betriebssystem laufen, stellen Container keine Sicherheitsschicht ("Sandboxen") dar!

In allen Fällen muss die Hardwarearchitektur beachtet werden: Auf einer Intel-Maschine können normalerweise keine VMs/Container basierend auf ARM-Architektur ausgeführt werden und umgekehrt.

2.4.3.3 Getting started

- DockerHub: fertige Images => hub.docker.com/search
- Image downloaden: `docker pull <IMAGE>`
- Image starten: `docker run <IMAGE>`

2.4.3.3.1 Begriffe

- **Docker-File**: Beschreibungsdatei, wie Docker ein Image erzeugen soll.
- **Image**: Enthält die Dinge, die lt. dem Docker-File in das Image gepackt werden sollen. Kann gestartet werden und erzeugt damit einen Container.
- **Container**: Ein laufendes Images (genauer: eine laufende Instanz eines Images). Kann dann auch zusätzliche Daten enthalten.

2.4.3.3.2 Beispiele

```
1 docker pull debian:stable-slim
2 docker run  --rm -it  debian:stable-slim  /bin/sh
```

`debian` ist ein fertiges Images, welches über DockerHub bereit gestellt wird. Mit dem Postfix `stable-slim` wird eine bestimmte Version angesprochen.

Mit `docker run debian:stable-slim` startet man das Image, es wird ein Container erzeugt. Dieser enthält den aktuellen Datenstand, d.h. wenn man im Image eine Datei anlegt, wäre diese dann im Container enthalten.

Mit der Option `--rm` wird der Container nach Beendigung automatisch wieder gelöscht. Da jeder Aufruf von `docker run <IMAGE>` einen neuen Container erzeugt, würden sich sonst recht schnell viele Container auf dem Dateisystem des Hosts ansammeln, die man dann manuell aufräumen müsste. Man kann aber einen beendeten Container auch erneut laufen lassen … (vgl. Dokumentation von `docker`). Mit der Option `--rm` sind aber auch im Container angelegte Daten wieder weg! Mit der Option `-it` wird der Container interaktiv gestartet und man landet in einer Shell.

Bei der Definition eines Images kann ein "*Entry Point*" definiert werden, d.h. ein Programm, welches automatisch beim Start des Container ausgeführt wird. Häufig erlauben Images aber auch, beim Start ein bestimmtes auszuführendes Programm anzugeben. Im obigen Beispiel ist das `/bin/sh`, also eine Shell …

```
1 docker pull openjdk:latest
2 docker run  --rm  -v "$PWD":/data -w /data  openjdk:latest  javac Hello.java
3 docker run  --rm  -v "$PWD":/data -w /data  openjdk:latest  java Hello
```

Auch für Java gibt es vordefinierte Images mit einem JDK. Das Tag "`latest`" zeigt dabei auf die letzte stabile Version des `openjdk`-Images. Üblicherweise wird "`latest`" von den Entwicklern immer wieder weiter geschoben, d.h. auch bei anderen Images gibt es ein "`latest`"-Tag. Gleichzeitig ist es die Default-Einstellung für die Docker-Befehle, d.h. es kann auch weggelassen werden: `docker run openjdk:latest` und `docker run openjdk` sind gleichwertig. Alternativ kann man hier auch hier wieder eine konkrete Version angeben.

Über die Option `-v` wird ein Ordner auf dem Host (hier durch `"$PWD"` dynamisch ermittelt) in den Container eingebunden ("gemountet"), hier auf den Ordner `/data`. Dort sind dann die Dateien sichtbar, die im Ordner `"$PWD"` enthalten sind. Über die Option `-w` kann ein Arbeitsverzeichnis definiert werden.

Mit `javac Hello.java` wird `javac` im Container aufgerufen auf der Datei `/data/Hello.java` im Container, d.h. die Datei `Hello.java`, die im aktuellen Ordner des Hosts liegt (und in den Container gemountet wurde). Das Ergebnis (`Hello.class`) wird ebenfalls in den Ordner `/data/` im Container geschrieben und erscheint dann im Arbeitsverzeichnis auf dem Host … Analog kann dann mit `java Hello` die Klasse ausgeführt werden.

Demo: Container in der Konsole

2.4.3.4 Images selbst definieren

```
1  FROM debian:stable-slim
2
3  ARG USERNAME=pandoc
4  ARG USER_UID=1000
5  ARG USER_GID=1000
6
7  RUN apt-get update && apt-get install -y --no-install-recommends            \
8          apt-utils bash wget make graphviz biber                             \
9          texlive-base texlive-plain-generic texlive-latex-base               \
10     #
11     && groupadd --gid $USER_GID $USERNAME                                   \
12     && useradd -s /bin/bash --uid $USER_UID --gid $USER_GID -m $USERNAME     \
13     #
14     && apt-get autoremove -y && apt-get clean -y && rm -rf /var/lib/apt/lists/*
15
16 WORKDIR /pandoc
17 USER $USERNAME
```

`docker build -t <NAME> -f <DOCKERFILE> .`

`FROM` gibt die Basis an, d.h. hier ein Image von Debian in der Variante `stable-slim`, d.h. das ist der Basis-Layer für das zu bauende Docker-Image.

Über `ARG` werden hier Variablen gesetzt.

`RUN` ist der Befehl, der im Image (hier Debian) ausgeführt wird und einen neuen Layer hinzufügt. In diesen Layer werden alle Dateien eingefügt, die bei der Ausführung des Befehls erzeugt oder angelegt werden. Hier im Beispiel wird das Debian-Tool `apt-get` gestartet und weitere Debian-Pakete installiert.

Da jeder `RUN`-Befehl einen neuen Layer anlegt, werden die restlichen Konfigurationen ebenfalls in diesem Lauf durchgeführt. Insbesondere wird ein nicht-Root-User angelegt, der von der UID und GID dem Default-User in Linux entspricht. Die gemounteten Dateien haben die selben Rechte wie auf dem Host, und durch die Übereinstimmung von UID/GID sind die Dateien problemlos zugreifbar und man muss nicht mit dem Root-User arbeiten (dies wird aus offensichtlichen Gründen als Anti-Pattern angesehen). Bevor der `RUN`-Lauf abgeschlossen wird, werden alle temporären und später nicht benötigten Dateien von `apt-get` entfernt, damit diese nicht Bestandteil des Layers werden.

Mit `WORKDIR` und `USER` wird das Arbeitsverzeichnis gesetzt und auf den angegebenen User umgeschaltet. Damit muss der User nicht mehr beim Aufruf von außen gesetzt werden.

Über `docker build -t <NAME> -f <DOCKERFILE> .` wird aus dem angegebenen Dockerfile und dem Inhalt des aktuellen Ordners (“`.`”) ein neues Image erzeugt und mit dem angegebenen Namen benannt.

Hinweis zum Umgang mit Containern und Updates: Bei der Erstellung eines Images sind bestimmte Softwareversionen Teil des Images geworden. Man kann prinzipiell in einem Container die Software aktualisieren, aber dies geht in dem Moment wieder verloren, wo der Container beendet und gelöscht wird. Außerdem widerspricht dies dem Gedanken, dass mehrere Personen mit dem selben Image/Container arbeiten und damit auch die selben Versionsstände haben. In der Praxis löscht man deshalb das alte Image einfach und erstellt ein neues, welches dann die aktualisierte Software enthält.

Beispiel: debian-latex.df

2.4.3.5 CI-Pipeline (GitLab)

```yaml
default:
    image: openjdk:17

job1:
    stage: build
    script:
        - java -version
        - javac Hello.java
        - java Hello
        - ls -lags
```

In den Gitlab-CI-Pipelines (analog wie in den GitHub-Actions) kann man Docker-Container für die Ausführung der Pipeline nutzen.

Mit `image: openjdk:17` wird das Docker-Image `openjdk:17` vom DockerHub geladen und durch den Runner für die Stages als Container ausgeführt. Die Aktionen im `script`-Teil, wie beispielsweise `javac Hello.java` werden vom Runner an die Standard-Eingabe der Shell des Containers gesendet. Im Prinzip entspricht das dem Aufruf auf dem lokalen Rechner: `docker run openjdk:17 javac Hello.java`.

Demo: GitLab CI/CD und Docker

2.4.3.6 CI-Pipeline (GitHub)

```yaml
name: demo
on:
    push:
        branches: [master]
    workflow_dispatch:

jobs:
    job1:
        runs-on: ubuntu-latest
        container: docker://openjdk:17
        steps:
            - uses: actions/checkout@v5
            - run: java -version
            - run: javac Hello.java
            - run: java Hello
            - run: ls -lags
```

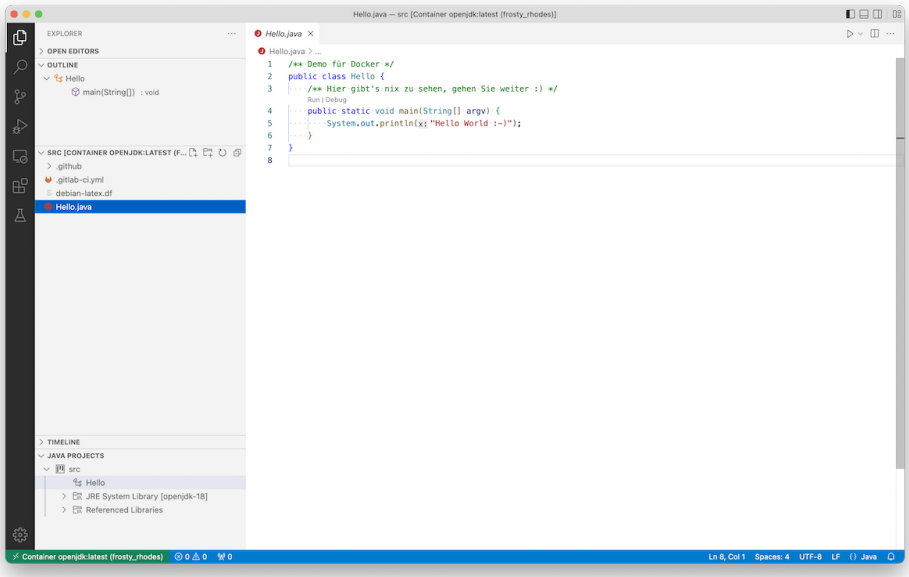
https://stackoverflow.com/questions/71283311/run-github-workflow-on-docker-image-with-a-dockerfile
https://docs.github.com/en/actions/using-jobs/running-jobs-in-a-container

In den GitHub-Actions kann man Docker-Container für die Ausführung der Pipeline nutzen.

Mit `docker://openjdk:17` wird das Docker-Image `openjdk:17` vom DockerHub geladen und auf dem Ubuntu-Runner als Container ausgeführt. Die Aktionen im `steps`-Teil, wie beispielsweise `javac Hello.java` werden vom Runner an die Standard-Eingabe der Shell des Containers gesendet. Im Prinzip entspricht das dem Aufruf auf dem lokalen Rechner: `docker run openjdk:17 javac Hello.java`.

Demo: GitHub Actions und Docker

2.4.3.7 VSCode und das Plugin “Remote - Containers”



1. VSCode (Host): Plugin “Remote - Containers” installieren
2. Docker (Host): Container starten mit Workspace gemountet
3. VSCode (Host): Attach to Container => neues Fenster (Container)
4. VSCode (Container): Plugin “Java Extension Pack” installieren
5. VSCode (Container): Dateien editieren, kompilieren, debuggen, ...

Mit Visual Studio Code (VSC) kann man über SSH oder in einem Container arbeiten. Dazu installiert man sich VSC lokal auf dem Host und installiert dort das Plugin “Remote - Containers”. VSC kann darüber vordefinierte Docker-Images herunterladen und darin arbeiten oder man kann alternativ einen Container selbst starten und diesen mit VSC verbinden (“attachen”).

Beim Verbinden öffnet VSC ein neues Fenster, welches mit dem Container verbunden ist. Nun kann man in diesem neuen Fenster ganz normal arbeiten, allerdings werden alle Dinge in dem Container erledigt. Man öffnet also Dateien in diesem Container, editiert sie im Container, übersetzt und testet im Container und nutzt dabei die im Container installierten Tools. Sogar die entsprechenden VSC-Plugins kann man im Container installieren.

Damit benötigt man auf einem Host eigentlich nur noch VSC und Docker, aber keine Java-Tools o.ä. und kann diese über einen im Projekt definierten Container (über ein mit versioniertes Dockerfile) nutzen.

Anmerkung: IntelliJ kann remote nur debuggen, d.h. das Editieren, Übersetzen, Testen läuft lokal auf dem Host (und benötigt dort den entsprechenden Tool-Stack). Für das Debuggen kann Idea das übersetzte Projekt auf ein Remote (SSH, Docker) schieben und dort debuggen.

Noch einen Schritt weiter geht das Projekt code-server: Dieses stellt u.a. ein Docker-Image codercom/code-server bereit, welches einen Webserver startet und über diesen kann man ein im Container laufendes (angepasstes) VSC erreichen. Man braucht also nur noch Docker und das Image und kann dann über den Webbrowser programmieren. Der Projektordner wird dabei in den Container gemountet, so dass die Dateien entsprechend zur Verfügung stehen:

```
1 docker run -it --name code-server -p 127.0.0.1:8080:8080 -v "$HOME/.config:/
      home/coder/.config" -v "$PWD:/home/coder/project" codercom/code-server:
      latest
```

Auf diesem Konzept setzt auch der kommerzielle Service GitHub Codespaces von GitHub auf.

Demo: VSCode und Docker

2.4.3.8 Link-Sammlung

- Wikipedia: Docker
- Wikipedia: Virtuelle Maschinen
- Docker: Überblick, Container
- Docker: HowTo
- DockerHub: Suche nach fertigen Images
- Docker und Java
- Dockerfiles: Best Practices
- Gitlab, Docker
- VSCode: Entwickeln in Docker-Containern
- Nickoloff (2019) und Miell und Sayers (2019)

2.4.3.9 Wrap-Up

- Schlanke Virtualisierung mit Containern (kein eigenes OS)
- *Kein* Sandbox-Effekt

- Begriffe: Docker-File vs. Image vs. Container
- Ziehen von vordefinierten Images
- Definition eines eigenen Images
- Arbeiten mit Containern: lokal, CI/CD, VSCode …

Zum Nachlesen

- Ullenboom (2021)
- Inden (2013)

Lernziele

- k2: Unterschied zwischen Containern und VMs
- k2: Einsatzgebiete für Container
- k2: Container laufen als abgeschottete Prozesse auf dem Host - kein Sandbox-Effekt
- k3: Container von DockerHub ziehen
- k3: Container starten
- k3: Eigene Container definieren und bauen
- k3: Einsatz von Containern in GitLab CI/CD und GitHub Actions
- k3: Einsatz von VSCode und Containern

Quizzes

- Quiz Docker (ILIAS)

2.4.4 Build-Systeme: Apache Ant

TL;DR
Zum Automatisieren von Arbeitsabläufen (Kompilieren, Testen, …) stehen in der Java-Welt verschiedene Tools zur Verfügung: Apache Ant, Apache Maven und Gradle sind sicher die am bekanntesten darunter.
In Apache Ant werden die Build-Skripte in XML definiert. Die äußere Klammer ist dabei das `<project>`. In einem Projekt kann es ein oder mehrere Teilziele (*Targets*) geben, die untereinander abhängig sein können. Die Targets können quasi "aufgerufen" werden bzw. in der IDE selektiert und gestartet werden.
In einem Target kann man schließlich mit *Tasks* Aufgaben wie Kompilieren, Testen, Aufräumen, … erledigen lassen. Dazu gibt es eine breite Palette an vordefinierten Tasks. Zusätzlich sind umfangreiche Operationen auf dem Filesystem möglich (Ordner erstellen, löschen, Dinge kopieren, …).
Über *Properties* können Werte und Namen definiert werden, etwa für bestimmte Ordner. Die Properties sind unveränderliche Variablen (auch wenn man sie im Skript scheinbar neu setzen kann).
Über Apache Ivy können analog zu Maven und Gradle definierte Abhängigkeiten aus Maven-Central aufgelöst werden.
Im Unterschied zu Maven und Gradle ist in Ant *kein* Java-Entwicklungsmodell eingebaut. Man muss sämtliche Targets selbst definieren.

Videos

- VL Apache Ant
- Demo Aufruf von Ant (Konsole, IDE: hello.xml)
- Demo Properties, Targets, Dependencies (build.xml)
- Demo Abhängigkeiten mit Ivy auflösen (ivydemo.xml)

2.4.4.1 Automatisieren von Arbeitsabläufen

Works on my machine …

- Build-Tools:
  - **Apache Ant**
  - Apache Maven
  - Gradle

- Aufgaben:
  - Übersetzen des Quellcodes
  - Ausführen der Unit-Tests
  - Generieren der Dokumentation
  - Packen der Distribution
  - Aufräumen temporärer Dateien
  - …

=> Automatisieren mit Apache Ant: ant.apache.org

2.4.4.2 Aufbau von Ant-Skripten: Projekte und Targets

```
<project name="Vorlesung" default="clean" basedir=".">
    <target name="init" />
    <target name="compile" depends="init" />
    <target name="test" depends="compile" />
    <target name="dist" depends="compile,test" />
    <target name="clean" />
</project>
```

- Ein Hauptziel: `project`
- Ein oder mehrere Teilziele: `target`
  - Abhängigkeiten der Teilziele untereinander möglich: `depends`
  - Vorbedingung für Targets mit `if` oder `unless`

2.4.4.3 Aufgaben erledigen: Tasks

- **Tasks**: Aufgaben bzw. Befehle (“Methodenaufruf”), in Targets auszuführen
- Struktur: `<taskname attr1="value1"attr2="value2"/>`

```
<target name="simplecompile" depends="clean,init">
    <javac srcdir="src" destdir="build" classpath="." />
</target>
```

- Beispiele: `echo`, `javac`, `jar`, `javadoc`, `junit`, …
- Je Target mehrere Tasks möglich
- Quellen:
  - Eingebaute Tasks
  - Optionale Task-Bibliotheken
  - Selbst definierte Tasks

=> Überblick: ant.apache.org/manual/tasksoverview.html

Konsole/IDE: ant -f hello.xml

2.4.4.4 Properties: Name-Wert-Paare

```
<property name="app"       value="MyProject" />
<property name="build.dir" location="build" />

<target name="init">
    <mkdir dir="${build.dir}" />
</target>
```

- Setzen von Eigenschaften: <property name="wuppie"value="fluppie"/>
  - Properties lassen sich nur einmal setzen (“immutable”)
    - Erneute Definition ist wirkungslos, erzeugt aber leider keinen Fehler
- Nutzung von Properties: <property name="db"value="${wuppie}.db"/>
- Pfade: “location” statt “value” nutzen: <property name="ziel"location="${p}/bla/blub"/>

- Properties beim Aufruf setzen mit Option “-D”:
  ant -Dwuppie=fluppie

Beispiel build.xml, Properties

2.4.4.5 Tasks zum Umgang mit Dateien und Ordnern

```
<target name="demo">
    <mkdir dir="${build.dir}/lib" />

    <delete dir="${build.dir}" />
    <delete file="${dist.dir}/wuppie.jar" />

    <copy file="myfile.txt" tofile="../bak/mycopy.txt" />
    <move file="src/file.orig" tofile="bak/file.moved" />
</target>
```

- <mkdir dir="${dist}/lib"/>
  - Legt auch direkte Unterverzeichnisse an
  - Keine Aktion, falls Verzeichnis existiert
- <delete dir="${builddir}"/>
  - Löscht eine Datei (“file”) oder ein Verzeichnis (“dir”) (rekursiv!)
- <copy file="myfile.txt"tofile="../bak/mycopy.txt"/>
- <move file="src/file.orig"tofile="bak/file.moved"/>

Beispiel build.xml, init und clean

2.4.4.6 Nutzung von Filesets in Tasks

```
<copy todir="archive">
    <fileset dir="src">
        <include name="**/*.java" />
        <exclude name="**/*.ba?" />
    </fileset>
</copy>

<delete>
    <fileset dir="." includes="**/*.ba?" />
</delete>
```

- "*" für beliebig viele Zeichen
- "?" für genau ein Zeichen
- "**" alle Unterverzeichnisse

Es gibt auch die Variante <dirset dir="...">, um Verzeichnisse zu gruppieren.

2.4.4.7 Pfade und externe Bibliotheken

- Als Element direkt im Task:

  ```
  <classpath>
      <pathelement location="${lib}/helper.jar" />
      <pathelement path="${project.classpath}" />
  </classpath>
  ```

  D.h. die Einbettung in den `javac`-Task würde etwa so erfolgen:

  ```
  <target ... >
      <javac ...>
          <classpath>
              <pathelement location="${lib}/helper.jar" />
              <pathelement path="${project.classpath}" />
          </classpath>
      </javac>
  </target>
  ```

  Anmerkung: Neben dem `pathelement` können Sie hier auch (wie im nächsten Beispiel gezeigt) ein oder mehrere `fileset` nutzen.

- Wiederverwendbar durch ID und "`refid`":

  ```
  <path id="java.class.path">
      <fileset dir="${lib}">
          <include name="**/*.jar" />
      </fileset>
  </path>

  <classpath refid="java.class.path" />
  ```

  Die Einbettung in den `javac`-Task würde hier etwa so erfolgen:

```
<path id="java.class.path">
    <fileset dir="${lib}">
        <include name="**/*.jar" />
    </fileset>
</path>

<target ... >
    <javac ...>
        <classpath refid="java.class.path" />
    </javac>
</target>
```

Anmerkung: Neben dem `fileset` können Sie hier auch (wie oben gezeigt) ein oder mehrere `pathelement` nutzen.

Anmerkung: Laut ant.apache.org/manual/Tasks/junit.html benötigt man neben der aktuellen `junit.jar` noch die `ant-junit.jar` im Classpath, um mit dem `junit`-Ant-Task entsprechende JUnit4-Testfälle ausführen zu können.

Für JUnit5 gibt es einen neuen Task `JUnitLauncher` (vgl. ant.apache.org/manual/Tasks/junitlauncher.html).

2.4.4.8 Beispiele

2.4.4.8.1 Beispiel-Task: Kompilieren

```
<path id="project.classpath">
    <fileset dir="${lib.dir}" includes="**/*.jar" />
</path>

<target name="compile" depends="init" description="compile the source " >
    <javac srcdir="${src.dir}" destdir="${build.dir}">
        <classpath refid="project.classpath" />
    </javac>
</target>
```

2.4.4.8.2 Beispiel-Task: Packen

```
<target name="dist" depends="compile" description="generate the distribution" >
    <mkdir dir="${dist.dir}" />
    <jar jarfile="${dist.dir}/${app}.jar" basedir="${build.dir}">
        <manifest>
            <attribute name="Main-Class"  value="${app}" />
        </manifest>
    </jar>
</target>
```

2.4.4.8.3 Beispiel-Task: Testen

- Tests einer Testklasse ausführen:

```
<junit>
    <test name="my.test.TestCase" />
</junit>
```

- Test ausführen und XML mit Ergebnissen erzeugen:

```
<junit printsummary="yes" fork="yes" haltonfailure="yes">
    <formatter type="xml" />
    <test name="my.test.TestCase" />
</junit>
```

- Verschiedene Tests als Batch ausführen:

```
<junit printsummary="yes" haltonfailure="yes">
    <classpath>
        <pathelement location="${build.tests}" />
        <pathelement path="${java.class.path}" />
    </classpath>

    <formatter type="plain"/>

    <test name="my.test.TestCase" haltonfailure="no" outfile="result">
        <formatter type="xml" />
    </test>

    <batchtest fork="yes" todir="${reports.tests}">
        <fileset dir="${src.tests}">
            <include name="**/*Test*.java" />
            <exclude name="**/AllTests.java" />
        </fileset>
    </batchtest>
</junit>
```

- Aus Testergebnis (XML) einen Report generieren:

```
<junitreport todir="${reportdir}">
    <fileset dir="...">
        <include name="TEST-*.xml" />
    </fileset>
    <report format="frames" todir="..." />
</junitreport>
```

- Abbruch bei Fehler:

```
<junit ... failureproperty="tests.failed" ... >
    <fail if="tests.failed" />
</junit>
```

=> `junit.jar` und `ant-junit.jar` (JUnit4.x) im Pfad!

2.4.4.8.4 Programme ausführen

```
<target name="run">
    <java jar="build/jar/HelloWorld.jar" fork="true" classname ="test.Main">
        <arg value ="-h" />
        <classpath>
            <pathelement location="./lib/test.jar" />
        </ classpath>
    </ java>
</target>
```

2.4.4.9 Ausblick: Laden von Abhängigkeiten mit Apache Ivy

Apache Ivy: Dependency Manager für Ant

```
<!-- build.xml -->
<project xmlns:ivy="antlib:org.apache.ivy.ant">
    <target name="resolve">
        <ivy:retrieve/>
    </target>
</project>
```

Wenn Ivy installiert ist, kann man durch den Eintrag `xmlns:ivy="antlib:org.apache.ivy.ant"` in der Projekt-Deklaration im Ant-Skript die Ivy-Tasks laden. Der wichtigste Task ist dabei `ivy:retrieve`, mit dem externe Projektabhängigkeiten heruntergeladen werden können.

```
<!-- ivy.xml -->
<ivy-module version="2.0">
    <dependencies>
        <dependency org="commons-cli" name="commons-cli" rev="1.5.0" />
        <dependency org="junit" name="junit" rev="4.13.2" />
    </dependencies>
</ivy-module>
```

Zur Steuerung von Ivy legt man eine weitere Datei `ivy.xml` an. Das Wurzelelement ist `ivy-module`, wobei die `version` die niedrigste kompatible Ivy-Version angibt.

Der `dependencies`-Abschnitt definiert dann die Abhängigkeiten, die Ivy auflösen muss. Die Schreibweise ist dabei wie im Maven2 Repository (mvnrepository.com) angelegt. Dort findet man beispielsweise für Apache Commons CLI den Eintrag für Maven (“POM”-Datei):

```
<!-- https://mvnrepository.com/artifact/commons-cli/commons-cli -->
<dependency>
    <groupId>commons-cli</groupId>
    <artifactId>commons-cli</artifactId>
    <version>1.4</version>
</dependency>
```

Für die Ivy-Konfiguration übernimmt man die `groupId` als `org`, die `artifactId` als `name` und die `version` als `rev` im Eintrag `dependency`.

Damit kann Ivy diese Bibliothek über den Ant-Task `ivy:retrieve` vor dem Bauen herunterladen, sofern die Bibliothek noch nicht lokal vorhanden ist. Eventuelle Abhängigkeiten werden dabei ebenfalls aufgelöst.

Im Detail: Der Ant-Task `ivy:retrieve` löst zunächst die Abhängigkeiten auf und lädt die Dateien (sofern sie noch nicht vorhanden oder veraltet sind) in den Ivy-Cache (per Default: `~/.ivy2/cache/`). Danach werden die Dateien in den Default-Library-Order im Projekt kopiert (per Defaul: `./lib/`). Die Ordner kann man über Optionen im `ivy:retrieve`-Task einstellen.

Demo: ivydemo.xml

2.4.4.10 Ausblick: Weitere Build-Systeme

- Maven

  - War als Nachfolger von Ant gedacht
  - Statt wie bei Ant explizit Targets zu formulieren, geht Maven von einem Standardprojekt aus - nur noch Abweichungen müssen formuliert werden
  - Zieht Abhängigkeiten in zentralen `.maven`-Ordner

- Gradle
  - Eine Art Mischung aus Ant und Maven unter Nutzung der Sprache Groovy

- Make
  - DER Klassiker, stammt aus der C-Welt. Kann aber natürlich auch Java.
  - Analog zu Ant: Aktionen und Ziele müssen explizit definiert werden

2.4.4.11 Wrap-Up Apache Ant: ant.apache.org

- Automatisieren von Arbeitsabläufen
- Apache Ant: Targets, Tasks, Properties
  - Targets sind auswählbare Teilziele
  - Abhängigkeiten zwischen Targets möglich
  - Tasks erledigen Aufgaben (innerhalb Targets)
  - Properties sind nicht änderbare Variablen
  - Umfangreiche Operationen auf Filesystem möglich

Zum Nachlesen

- Inden (2013, Abschn. 2.5.2)

Lernziele

- k3: Schreiben einfacher Ant-Skripte mit Abhängigkeiten zwischen den Targets
- k3: Nutzung von Ant-Filesets (Dateisystemoperationen, Classpath)
- k3: Nutzung von Ant-Properties
- k3: Ausführen von Ant-Targets aus der IDE heraus

Quizzes

- Quiz Apache Ant (ILIAS)

2.4.5 Build-Systeme: Apache Maven

TL;DR
Zum Automatisieren von Arbeitsabläufen (Kompilieren, Testen, …) stehen in der Java-Welt verschiedene Tools zur Verfügung: Apache Ant, Apache Maven und Gradle sind sicher die am bekanntesten darunter.
In Apache Maven ist bereits der typische Java-Standard-Lebenszyklus eingebaut und es müssen nur noch Abweichungen davon und Festlegung von Versionen und Dependencies in XML formuliert werden. Dies nennt man auch "*Convention over Configuration*".
Die Maven-Goals sind auswählbare Ziele und werden durch Plugins bereitgestellt. Zwischen den Goals sind Abhängigkeiten möglich (und bereits eingebaut). Über Properties kann man noch Namen und Versionsnummern o.ä. definieren.
Abhängigkeiten zu externen Bibliotheken werden als Dependencies formuliert: Am besten den Abschnitt von Maven-Central kopieren.

Videos

- VL Apache Maven
- Demo Maven-Projekt

2.4.5.1 Build-Tool Maven: Alternative zu Ant oder Gradle

```
mvn archetype:generate -DgroupId=de.hsbi.pm -DartifactId=my-project
                       -DarchetypeArtifactId=maven-archetype-quickstart
```

```
my-project
├── pom.xml
└── src
    ├── main
    │   └── java
    │       └── de
    │           └── fhb
    │               └── pm
    │                   └── App.java
    └── test
        └── java
            └── de
                └── fhb
                    └── pm
                        └── AppTest.java
```

Von der zeitlichen Entstehung her kommt Maven nach Ant, aber vor Gradle. Wie in Ant sind auch die Maven-Buildskripte XML-basierte Textdateien (Gradle nutzt eine Groovy-basierte DSL).

Allerdings hat Maven im Gegensatz zu Ant bereits ein Modell des Java-Entwicklungsprozess "eingebaut": Im Ant-Skript muss alles, was man tun möchte, explizit als Target formuliert werden, d.h. auch ein Kompilieren der Sourcen oder Ausführen der Tests muss extra als Target ins Ant-Skript geschrieben werden, um benutzbar zu sein. In Maven ist dieses Modell bereits implementiert, d.h. hier muss man lediglich zusätzliche oder abweichende Dinge im XML-File konfigurieren. Das nennt man auch "*convention over configuration*".

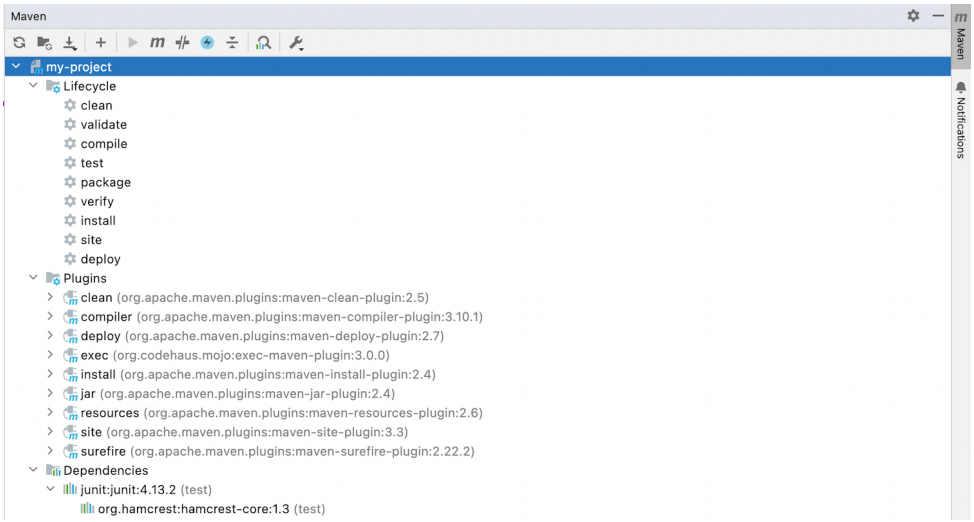
Der Maven-Aufruf `mvn archetype:generate -DgroupId=de.hsbi.pm -DartifactId=my-project -DarchetypeArtifactId=maven-archetype-quickstart` erzeugt mit Hilfe des Plugins

`archetype`, welches das Ziel (engl.: "*Maven goal*") `generate` bereitstellt, ein neues Projekt mit dem Namen `my-project` und der initialen Package-Struktur `de.hsbi.pm`. Das von Maven für die Projekterstellung genutzte Plugin ist unter der ID `maven-archetype-quickstart` in den Maven-Repositories (etwa Maven-Central) verfügbar, hier kann man mit der zusätzlichen Option `-DarchetypeVersion=1.4` auf die letzte Version schalten.

Die erzeugte Ordnerstruktur entspricht der Standardstruktur von Gradle (Gradle hat diese quasi von Maven übernommen). Die Konfigurationsdatei für Maven hat den Namen `pom.xml`.

Hinweis: Die `groupId` und `artifactId` werden auch für eine Veröffentlichung des Jar-Files des Projekts auf dem zentralen Maven-Repository Maven-Central genutzt. Von hier würde Maven auch als Abhängigkeit konfigurierte Bibliotheken herunterladen.

2.4.5.2 Lebenszyklus (eingebaut in Maven)



In Maven ist das typische Java-Entwicklungsmodell als "Lebenszyklus" implementiert.

Entsprechende Plugins stellen die jeweiligen "*Goals*" (Ziele) bereit. Dabei sind auch die Abhängigkeiten berücksichtigt, d.h. das Ziel `test` erfordert ein `compile` …

2.4.5.3 Project Object Model: *pom.xml*

```xml
<project>
    <!-- aktuelle Version für Maven 2.x-->
    <modelVersion>4.0.0</modelVersion>

    <!-- Basisinformationen -->
    <groupId>de.hsbi.pm</groupId>
    <artifactId>my-project</artifactId>
    <version>1.0-SNAPSHOT</version>

    <!-- Eigenschaften, vergleichbar zu den Properties in Ant -->
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
13          <maven.compiler.release>17</maven.compiler.release>
14      </properties>
15
16      <!-- Abhängigkeiten zu externen Bibliotheken -->
17      <dependencies>
18          <dependency>
19              <groupId>junit</groupId>
20              <artifactId>junit</artifactId>
21              <version>4.13.2</version>
22              <scope>test</scope>
23          </dependency>
24      </dependencies>
25  </project>
```

Die Konfigurationsdatei `pom.xml` stellt die Konfiguration für das Maven-Projekt bereit (“Project Object Model”).

Es werden mindestens der Name des Projekts sowie die Abhängigkeiten definiert.

Die `groupId` ist ein eindeutiger Bezeichner für die Organisation oder den Autor des Projekts. Oft wird hier einfach wie im obigen Beispiel eine Package-Struktur genutzt, aber wie im Fall von JUnit kann dies auch ein einfacher String (dort “`junit`”) sein.

Die `artifactId` ist der eindeutige Name für das Projekt, d.h. unter diesem Namen wird das generierte Jar-File im Maven-Repository zu finden sein (sofern es denn veröffentlicht wird).

Über `dependencies` kann man benötigte Abhängigkeiten definieren, hier als Beispiel JUnit in der 4.x Variante … Diese werden bei Bedarf von Maven vom Maven-Repository heruntergeladen. Die Einträge für die Dependencies findet man ebenfalls auf MavenCentral.

2.4.5.4 Project Object Model: Plugins

```
1   <project>
2       ...
3
4       <!-- Plugins: Stellen eigene "Goals" zur Verfügung -->
5       <build>
6           <plugins>
7               <plugin>
8                   <!-- https://mvnrepository.com/artifact/org.apache.maven.
    plugins/maven-compiler-plugin -->
9                   <groupId>org.apache.maven.plugins</groupId>
10                  <artifactId>maven-compiler-plugin</artifactId>
11                  <version>3.10.1</version>
12              </plugin>
13
14              <plugin>
15                  <!-- https://mvnrepository.com/artifact/org.apache.maven.
    plugins/maven-surefire-plugin -->
16                  <groupId>org.apache.maven.plugins</groupId>
17                  <artifactId>maven-surefire-plugin</artifactId>
18                  <version>2.22.2</version>
19              </plugin>
20          </plugins>
21      </build>
22  </project>
```

Zusätzlich können die Phasen des Build-Prozesses konfiguriert werden, d.h. für die entsprechenden Plugins finden sich Abschnitte unter `<build><plugins>` in der `pom.xml`.

Auf maven.apache.org/plugins/index.html finden Sie eine Übersicht über häufig benutzte Plugins sowie die von den Plugins bereitgestellten Goals sowie Konfigurationsmöglichkeiten.

Die entsprechenden POM-Einträge finden Sie analog zu den Dependencies ebenfalls auf MavenCentral (Tag "plugin" statt "dependency").

Plugins können aber auch selbst erstellt werden und in das Projekt eingebunden werden, ein erster Einstieg ist die Plugin-API.

2.4.5.5 Und wie lasse ich jetzt eine Anwendung mal laufen?

- `mvn clean`: Lösche alle generierten Artefakte, beispielsweise `.class`-Dateien.
- `mvn compile` => `mvn compiler:compile`: Übersetze die Sourcen und schiebe die generierten `.class`-Dateien in den Ordner `target/classes/` (Default). Dazu werden alle Abhängigkeiten aufgelöst und bei Bedarf (neu) heruntergeladen (Default: Userverzeichnis, Ordner `.m2/`).
- `mvn test` => `mvn surefire:test`: Lasse die Tests laufen. Hängt von `compile` ab. Namenskonvention: Alle Klassen mit `*Test.java` und `Test*.java` im Standard-Testordner `src/test/java/` werden betrachtet (und weitere, vgl. maven.apache.org/surefire/maven-surefire-plugin/examples/junit-platform.html).
- `mvn package`: Hängt von `compile` ab und erzeugt ein Jar-File mit dem Namen "artifactId-version.jar" im Ordner `target/`. Mit `mvn install` kann man dieses Jar-File dann auch dem lokalen Repository im Home-Verzeichnis des Users (`.m2/`) hinzufügen.
- `mvn exec:java -Dexec.mainClass="de.hsbi.pm.Main"`: Hängt von `compile` ab und führt die Klasse `de.hsbi.pm.Main` aus.

Demo: pom.xml

2.4.5.6 Wrap-Up

Apache Maven: maven.apache.org, Maven Getting Started Guide

- Automatisieren von Arbeitabläufen
- Apache Maven: Goals, Properties, Dependencies => "*Convention over Configuration*", Java-Standard-Lebenszyklus eingebaut
  - Goals sind auswählbare Ziele, bereitgestellt durch Plugins
  - Abhängigkeiten zwischen Goals möglich
  - Properties agieren wie Variablen, etwa für Versionsnummern
  - Abhängigkeiten zu externen Bibliotheken werden als Dependencies formuliert: Abschnitt von Maven-Central kopieren

Zum Nachlesen

- Ullenboom (2021)
- Inden (2013)

Lernziele

- k3: Schreiben einfacher Maven-Skripte zu Übersetzen des Projekts, zum Testen und zum Erzeugen von Jar-Files
- k3: Nutzung von Maven-Properties
- k3: Einbinden externer Bibliotheken als Dependencies
- k3: Ausführen von Maven-Goals aus IDE heraus und Einbindung als Builder

Quizzes

- Quiz Apache Maven (ILIAS)

2.5 Entwurfsmuster

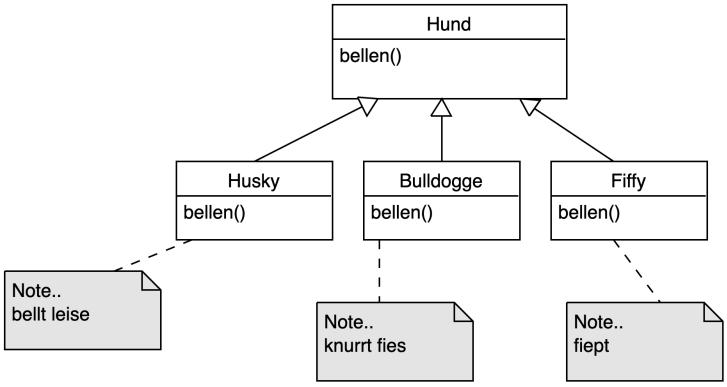
2.5.1 Strategy-Pattern

TL;DR
Das Verhalten von Klassen kann über Vererbungshierarchien weitergegeben und durch Überschreiben in den erbenden Klassen verändert werden. Dies führt häufig schnell zu breiten und tiefen Vererbungsstrukturen.
Das Strategy-Pattern ist ein Entwurfsmuster, in dem Verhalten stattdessen an passende Klassen/Objekte ausgelagert (delegiert) wird.
Es wird eine Schnittstelle benötigt (Interface oder abstrakte Klasse), in dem Methoden zum Abrufen des gewünschten Verhaltens definiert werden. Konkrete Klassen leiten davon ab und implementieren das gewünschte konkrete Verhalten.
In den nutzenden Klassen wird zur Laufzeit eine passende Instanz der (Strategie-) Klassen übergeben (Konstruktor, Setter, …) und beispielsweise über ein Attribut referenziert. Das gewünschte Verhalten muss nun nicht mehr in der nutzenden Klasse selbst implementiert werden, stattdessen wird einfach auf dem übergebenen Objekt die Methode aus der Schnittstelle aufgerufen. Dies nennt man auch “Delegation”, weil die Aufgabe (das Verhalten) an ein anderes Objekt (hier das Strategie-Objekt) weiter gereicht (delegiert) wurde.

Videos

- VL Strategy-Pattern
- Demo Strategy-Pattern

2.5.1.1 Wie kann man das Verhalten einer Klasse dynamisch ändern?

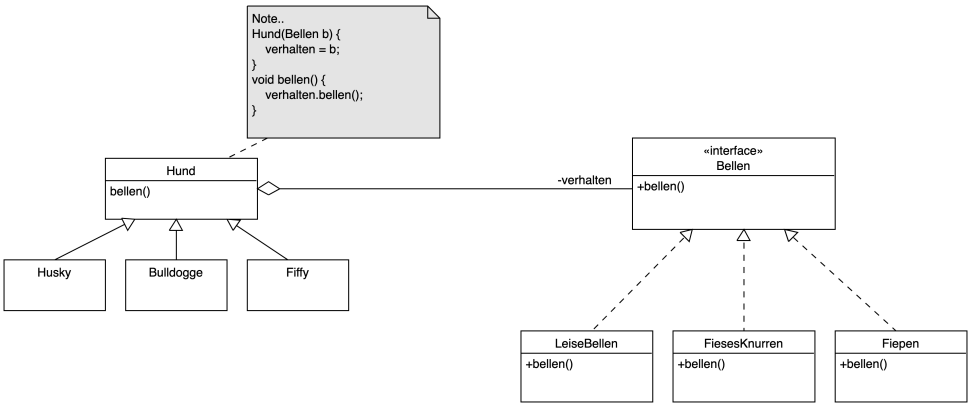


Modellierung unterschiedlicher Hunderassen: Jede Art bellt anders.

Es bietet sich an, die Hunderassen von einer gemeinsamen Basisklasse `Hund` abzuleiten, um die Hundeartigkeit allgemein sicherzustellen.

Da jede Rasse anders bellen soll, muss jedes Mal die Methode `bellen` überschrieben werden. Das ist relativ aufwändig und fehleranfällig. Außerdem kann man damit nicht modellieren, dass es beispielsweise auch konkrete Bulldoggen geben mag, die nur leise fiepen …

2.5.1.2 Lösung: Delegation der Aufgabe an geeignetes Objekt



Der `Hund` delegiert das Verhalten beim Bellen an ein Objekt, welches beispielsweise bei der Instantiierung der Klasse übergeben wurde (oder später über einen Setter). D.h. die Methode `Hund#bellen` bellt nicht mehr selbst, sondern ruft auf einem passenden Objekt eine vereinbarte Methode auf.

Dieses passende Objekt ist hier im Beispiel vom Typ `Bellen` und hat eine Methode `bellen` (Interface). Die verschiedenen Bell-Arten kann man über eigene Klassen implementieren, die das Interface einhalten.

Damit braucht man in den Klassen für die Hunderassen die Methode `bellen` nicht jeweils neu überschreiben, sondern muss nur bei der Instantiierung eines Hundes ein passendes `Bellen`-Objekt mitgeben.

Als netten Nebeneffekt kann man so auch leicht eine konkrete Bulldogge realisieren, die eben nicht fies knurrt, sondern leise fiept …

Entwurfsmuster: **Strategy Pattern**

2.5.1.3 Exkurs UML: Assoziation vs. Aggregation vs. Komposition

Eine **Assoziation** beschreibt eine Beziehung zwischen zwei (oder mehr) UML-Elementen (etwa Klassen oder Interfaces).

Eine **Aggregation** (leere Raute) ist eine Assoziation, die eine *Teil-Ganzes-Beziehung* hervorhebt. Teile können dabei ohne das Ganze existieren (Beispiel: Personen als Partner in einer Ehe-Beziehung). D.h. auf der einbindenden Seite (mit der leeren Raute) hat man implizit `0..*` stehen.

Eine **Komposition** (volle Raute) ist eine Assoziation, die eine Teil-Ganzes-Beziehung hervorhebt. Teile können aber nicht ohne das Ganze existieren (Beispiel: Gebäude und Stockwerke: Ein Gebäude besteht aus Stockwerken, die ohne das Gebäude aber nicht existieren.). D.h. auf der einbindenden Seite (mit der vollen Raute) steht implizit eine 1 (ein Stockwerk gehört genau zu einem Gebäude, ein Gebäude besteht aber aus mehreren Stockwerken).

Siehe auch Aggregation, Assoziation und Klassendiagramm.

2.5.1.4 Zweites Beispiel: Sortieren einer Liste von Studis

Sortieren einer Liste von Studis: `Collections.sort` kann eine Liste nach einem Default-Kriterium sortieren oder aber über einen extra `Comparator` nach benutzerdefinierten Kriterien … Das Verhalten der Sortiermethode wird also quasi an dieses Comparator-Objekt delegiert …

```java
public class Studi {
    private String name;
    public Studi(String name) { this.name = name; }

    public static void main(String[] args) {
        List<Studi> list = new ArrayList<Studi>();
        list.add(new Studi("Klaas"));
        list.add(new Studi("Hein"));
        list.add(new Studi("Pit"));

        // Sortieren der Liste (Standard-Reihenfolge)?!
        // Sortieren der Liste (eigene Reihenfolge)?!
    }
}
```

Konsole strategy.SortDefault, strategy.SortOwnCrit

Anmerkung: Die Interfaces `Comparable` und `Comparator` und deren Nutzung wurde(n) in OOP besprochen. Anonyme Klassen wurden ebenfalls in OOP besprochen. Bitte lesen Sie dies noch einmal in der Semesterliteratur nach, wenn Sie hier unsicher sind!

2.5.1.5 Hands-On: Strategie-Muster

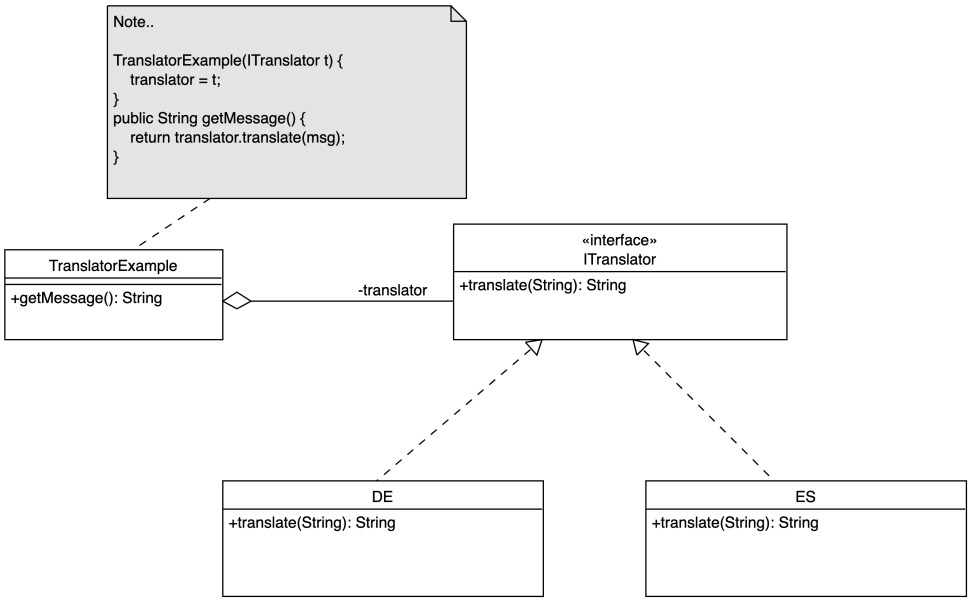
Implementieren Sie das Strategie-Muster für eine Übersetzungsfunktion:

- Eine Klasse liefert eine Nachricht (`String`) mit `getMessage()` zurück.
- Diese Nachricht ist in der Klasse in Englisch implementiert.
- Ein passendes Übersetzerobjekt soll die Nachricht beim Aufruf der Methode `getMessage()` in die Ziel-Sprache übersetzen.

Fragen:

1. Wie muss das Pattern angepasst werden?
2. Wie sieht die Implementierung aus?

2.5.1.6 Auflösung



Konsole strategy.TranslatorExample

2.5.1.7 Wrap-Up Strategy-Pattern: Verhaltensänderung durch Delegation an passendes Objekt

- Interface oder abstrakte Klasse als Schnittstelle
- Konkrete Klassen implementieren Schnittstelle => konkrete Strategien
- Zur Laufzeit Instanz dieser Klassen übergeben (Aggregation) …
- … und nutzen (Delegation)

Zum Nachlesen

- Eilebrecht und Starke (2013)
- Gamma u. a. (2011)
- Kleuker (2018)

Lernziele

- k3: Strategie-Entwurfsmuster praktisch anwenden

Quizzes

- Quiz Strategy-Pattern (ILIAS)

Challenges

Implementieren Sie das Spiel “Schere,Stein,Papier” (Spielregeln vergleiche wikipedia.org/wiki/Schere,Stein,Papier) in Java.

Nutzen Sie das Strategy-Pattern, um den Spielerinstanzen zur Laufzeit eine konkrete Spielstrategie mitzugeben, nach denen die Spieler ihre Züge *berechnen*. Implementieren Sie mindestens drei unterschiedliche konkrete Strategien.

Hinweis: Eine mögliche Strategie könnte sein, den Nutzer via Tastatureingabe nach dem nächsten Zug zu fragen.

Gehen Sie bei der Lösung der Aufgabe methodisch vor:

1. Stellen Sie sich eine Liste mit relevanten Anforderungen zusammen.
2. Erstellen Sie (von Hand) ein Modell (UML-Klassendiagramm):
   - Welche Klassen und Interfaces werden benötigt?
   - Welche Aufgaben sollen die Klassen haben?
   - Welche Attribute und Methoden sind nötig?
   - Wie sollen die Klassen interagieren, wer hängt von wem ab?
3. Implementieren Sie Ihr Modell in Java. Schreiben Sie ein Hauptprogramm, welches das Spiel startet, die Spieler ziehen lässt und dann das Ergebnis ausgibt.
4. Überlegen Sie, wie Sie Ihr Programm sinnvoll manuell testen können und tun Sie das.

2.5.2 Visitor-Pattern

TL;DR
Häufig bietet es sich bei Datenstrukturen an, die Traversierung nicht direkt in den Klassen der Datenstrukturen zu implementieren, sondern in Hilfsklassen zu verlagern. Dies gilt vor allem dann, wenn die Datenstruktur aus mehreren Klassen besteht (etwa ein Baum mit verschiedenen Knotentypen) und/oder wenn man nicht nur eine Traversierungsart ermöglichen will oder/und wenn man immer wieder neue Arten der Traversierung ergänzen will. Das würde nämlich bedeuten, dass man für jede weitere Form der Traversierung in *allen* Klassen eine entsprechende neue Methode implementieren müsste.
Das Visitor-Pattern lagert die Traversierung in eigene Klassenstruktur aus.
Die Klassen der Datenstruktur bekommen nur noch eine `accept()`-Methode, in der ein Visitor übergeben wird und rufen auf diesem Visitor einfach dessen `visit()`-Methode auf (mit einer Referenz auf sich selbst als Argument).
Der Visitor hat für jede Klasse der Datenstruktur eine Überladung der `visit()`-Methode. In diesen kann er je nach Klasse die gewünschte Verarbeitung vornehmen. Üblicherweise gibt es ein Interface oder eine abstrakte Klasse für die Visitoren, von denen dann konkrete Visitoren ableiten.
Bei Elementen mit "Kindern" muss man sich entscheiden, wie die Traversierung implementiert werden soll. Man könnte in der `accept()`-Methode den Visitor an die Kinder weiter reichen (also auf den Kindern `accept()` mit dem Visitor aufrufen), bevor man die `visit()`-Methode des Visitors mit sich selbst als Referenz aufruft. Damit ist die Form der Traversierung in den Klassen der Datenstruktur fest verankert und über den Visitor findet "nur" noch eine unterschiedliche Form der Verarbeitung statt. Alternativ überlässt man es dem Visitor, die Traversierung durchzuführen: Hier muss in den `visit()`-Methoden für die einzelnen Elemente entsprechend auf mögliche Kinder reagiert werden.
In diesem Pattern findet ein sogenannter "Double-Dispatch" statt: Zur Laufzeit wird ein konkreter Visitor instantiiert und über `accept()` an ein Element der Datenstruktur übergeben. Dort ist zur Compile-Zeit aber nur der Obertyp der Visitoren bekannt, d.h. zur Laufzeit wird hier der konkrete Typ bestimmt und entsprechend die richtige `visit()`-Methode auf der "echten" Klasse des Visitors aufgerufen (erster Dispatch). Da im Visitor die `visit()`-Methoden für jeden Typ der Datenstrukur überladen sind, findet nun zur Laufzeit die Auflösung der korrekten Überladung statt (zweiter Dispatch).

Videos

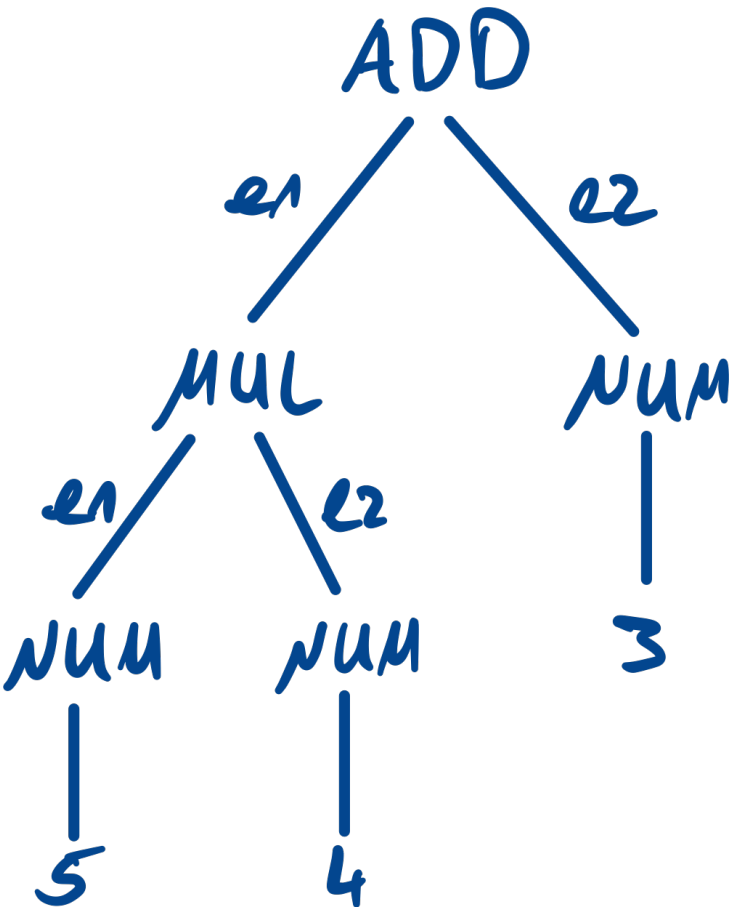
- VL Visitor-Pattern
- Demo Visitor-Pattern (Part I: Traversierung ohne Visitor)
- Demo Visitor-Pattern (Part II: Traversierung mit Visitor)

2.5.2.1 Motivation: Parsen von "5*4+3"

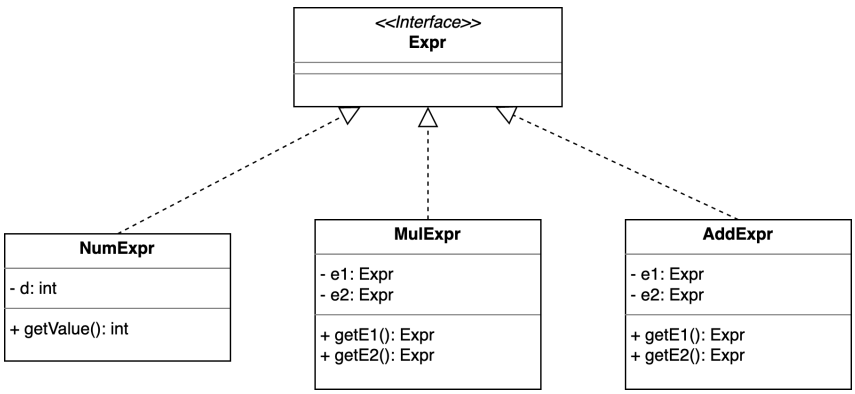
Zum Parsen von Ausdrücken (*Expressions*) könnte man diese einfache Grammatik einsetzen. Ein Ausdruck ist dabei entweder ein einfacher Integer oder eine Addition oder Multiplikation zweier Ausdrücke.

```
1 expr : e1=expr '*' e2=expr      # MUL
2      | e1=expr '+' e2=expr      # ADD
3      | INT                      # NUM
4      ;
```

Beim Parsen von "5*4+3" würde dabei der folgende Parsetree entstehen:



2.5.2.2 Strukturen für den Parsetree



Der Parsetree für diese einfache Grammatik ist ein Binärbaum. Die Regeln werden auf Knoten im Baum zurückgeführt. Es gibt Knoten mit zwei Kindknoten, und es gibt Knoten ohne Kindknoten (“Blätter”).

Entsprechend kann man sich einfache Klassen definieren, die die verschiedenen Knoten in diesem Parsetree repräsentieren. Als Obertyp könnte es ein (noch leeres) Interface `Expr` geben.

```java
public interface Expr {}

public class NumExpr implements Expr {
    private final int d;

    public NumExpr(int d) { this.d = d; }
}

public class MulExpr implements Expr {
    private final Expr e1;
    private final Expr e2;

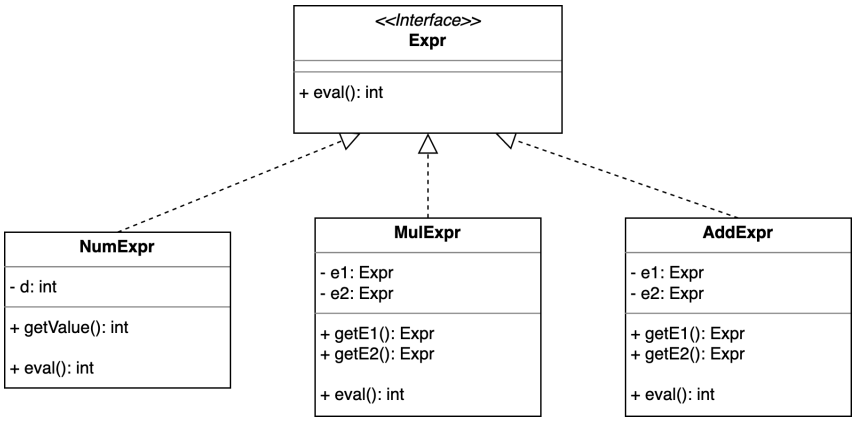
    public MulExpr(Expr e1, Expr e2) {
        this.e1 = e1;  this.e2 = e2;
    }
}

public class AddExpr implements Expr {
    private final Expr e1;
    private final Expr e2;

    public AddExpr(Expr e1, Expr e2) {
        this.e1 = e1;  this.e2 = e2;
    }
}


public class DemoExpr {
    public static void main(final String... args) {
        // 5*4+3
        Expr e = new AddExpr(new MulExpr(new NumExpr(5), new NumExpr(4)), new
            NumExpr(3));
    }
}
```

2.5.2.3 Ergänzung I: Ausrechnen des Ausdrucks Es wäre nun schön, wenn man mit dem Parsetree etwas anfangen könnte. Vielleicht möchte man den Ausdruck ausrechnen?



Zum Ausrechnen des Ausdrucks könnte man dem Interface eine `eval()`-Methode spendieren. Jeder Knoten

kann für sich entscheiden, wie die entsprechende Operation ausgewertet werden soll: Bei einer `NumExpr` ist dies einfach der gespeicherte Wert, bei Addition oder Multiplikation entsprechend die Addition oder Multiplikation der Auswertungsergebnisse der beiden Kindknoten.

```java
public interface Expr {
    int eval();
}

public class NumExpr implements Expr {
    private final int d;

    public NumExpr(int d) { this.d = d; }
    public int eval() { return d; }
}

public class MulExpr implements Expr {
    private final Expr e1;
    private final Expr e2;

    public MulExpr(Expr e1, Expr e2) {
        this.e1 = e1;  this.e2 = e2;
    }
    public int eval() { return e1.eval() * e2.eval(); }
}

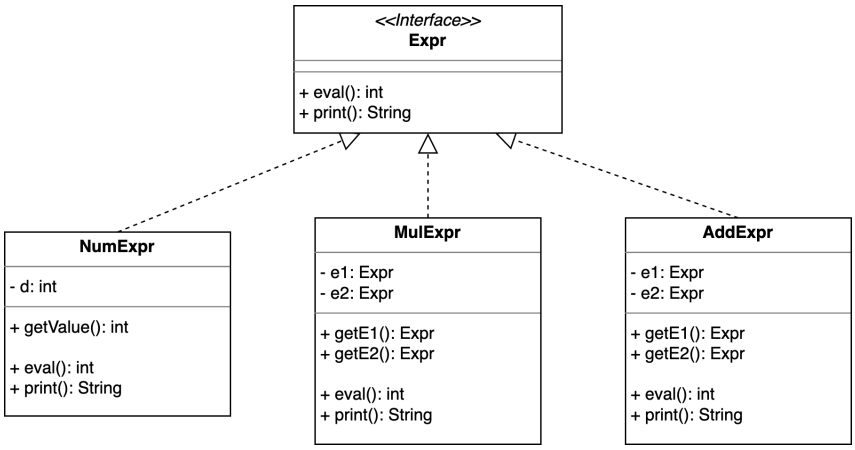
public class AddExpr implements Expr {
    private final Expr e1;
    private final Expr e2;

    public AddExpr(Expr e1, Expr e2) {
        this.e1 = e1;  this.e2 = e2;
    }
    public int eval() { return e1.eval() + e2.eval(); }
}


public class DemoExpr {
    public static void main(final String... args) {
        // 5*4+3
        Expr e = new AddExpr(new MulExpr(new NumExpr(5), new NumExpr(4)), new
            NumExpr(3));

        int erg = e.eval();
    }
}
```

2.5.2.4 Ergänzung II: Pretty-Print des Ausdrucks Nachdem das Ausrechnen so gut geklappt hat, will der Chef nun noch flink eine Funktion, mit der man den Ausdruck hübsch ausgeben kann:

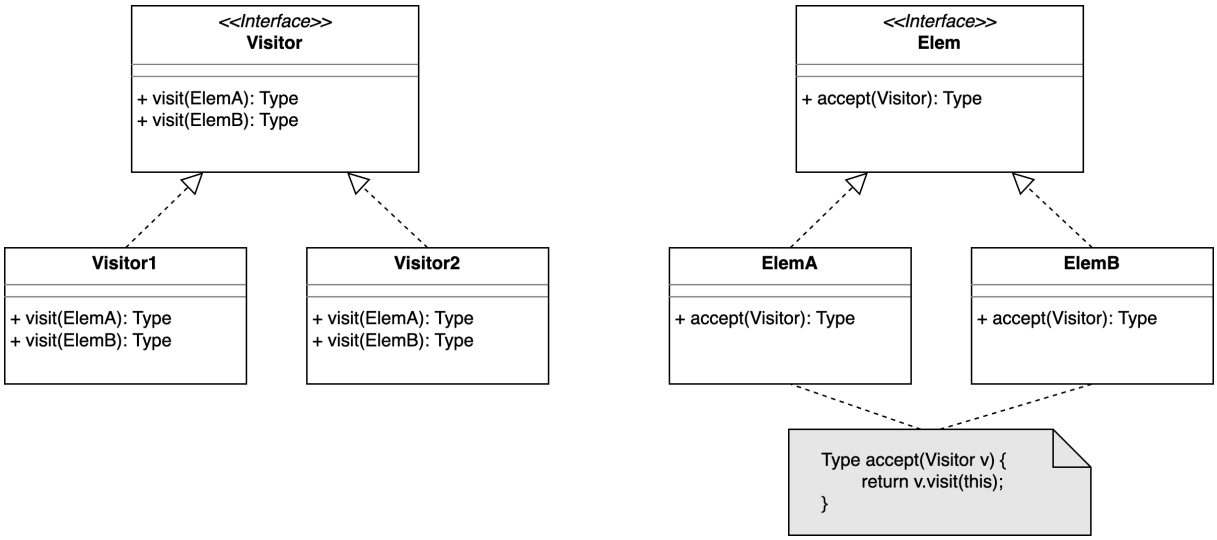


Das fängt an, sich zu wiederholen. Wir implementieren immer wieder ähnliche Strukturen, mit denen wir diesen Parsetree traversieren … Und wir müssen für *jede* Erweiterung immer *alle* Expression-Klassen anpassen!

Beispiel: direct.DemoExpr

Das geht besser.

2.5.2.5 Visitor-Pattern (Besucher-Entwurfsmuster)



Das Entwurfsmuster “Besucher” (*Visitor Pattern*) lagert die Aktion beim Besuchen eines Knotens in eine separate Klasse aus.

Dazu bekommt jeder Knoten im Baum eine neue Methode, die einen Besucher akzeptiert. Dieser Besucher kümmert sich dann um die entsprechende Verarbeitung des Knotens, also um das Auswerten oder Ausgeben im obigen Beispiel.

Die Besucher haben eine Methode, die für jeden zu bearbeitenden Knoten überladen wird. In dieser Methode findet dann die eigentliche Verarbeitung statt: Auswerten des Knotens oder Ausgeben des Knotens …

```
1  public interface Expr {
2      void accept(ExprVisitor v);
3  }
4
5  public class NumExpr implements Expr {
6      private final int d;
7
8      public NumExpr(int d) { this.d = d; }
9      public int getValue() { return d; }
10
11     public void accept(ExprVisitor v) { v.visit(this); }
12 }
13
14 public class MulExpr implements Expr {
15     private final Expr e1;
16     private final Expr e2;
17
18     public MulExpr(Expr e1, Expr e2) {
19         this.e1 = e1;  this.e2 = e2;
20     }
21     public Expr getE1() { return e1; }
22     public Expr getE2() { return e2; }
23
```

```
24      public void accept(ExprVisitor v) { v.visit(this); }
25  }
26
27  public class AddExpr implements Expr {
28      private final Expr e1;
29      private final Expr e2;
30
31      public AddExpr(Expr e1, Expr e2) {
32          this.e1 = e1;  this.e2 = e2;
33      }
34      public Expr getE1() { return e1; }
35      public Expr getE2() { return e2; }
36
37      public void accept(ExprVisitor v) { v.visit(this); }
38  }
39
40
41  public interface ExprVisitor {
42      void visit(NumExpr e);
43      void visit(MulExpr e);
44      void visit(AddExpr e);
45  }
46
47  public class EvalVisitor implements ExprVisitor {
48      private final Stack<Integer> erg = new Stack<>();
49
50      public void visit(NumExpr e) { erg.push(e.getValue()); }
51      public void visit(MulExpr e) {
52          e.getE1().accept(this);  e.getE1().accept(this);
53          erg.push(erg.pop() * erg.pop());
54      }
55      public void visit(AddExpr e) {
56          e.getE1().accept(this);  e.getE1().accept(this);
57          erg.push(erg.pop() + erg.pop());
58      }
59      public int getResult() { return erg.peek(); }
60  }
61
62  public class PrintVisitor implements ExprVisitor {
63      private final Stack<String> erg = new Stack<>();
64
65      public void visit(NumExpr e) { erg.push("NumExpr(" + e.getValue() + ")"); }
66      public void visit(MulExpr e) {
67          e.getE1().accept(this);  e.getE1().accept(this);
68          erg.push("MulExpr(" + erg.pop() + ", " + erg.pop() + ")");
69      }
70      public void visit(AddExpr e) {
71          e.getE1().accept(this);  e.getE1().accept(this);
72          erg.push("AddExpr(" + erg.pop() + ", " + erg.pop() + ")");
73      }
74      public String getResult() { return erg.peek(); }
75  }
76
77
78  public class DemoExpr {
79      public static void main(final String... args) {
80          // 5*4+3
```

```
81          Expr e = new AddExpr(new MulExpr(new NumExpr(5), new NumExpr(4)), new
                NumExpr(3));
82
83          EvalVisitor v1 = new EvalVisitor();
84          e.accept(v1);
85          int erg = v1.getResult();
86
87          PrintVisitor v2 = new PrintVisitor();
88          e.accept(v2);
89          String s = v2.getResult();
90      }
91  }
```

2.5.2.5.1 Implementierungsdetail In den beiden Klasse `AddExpr` und `MulExpr` müssen auch die beiden Kindknoten besucht werden, d.h. hier muss der Baum weiter traversiert werden.

Man kann sich überlegen, diese Traversierung in den Klassen `AddExpr` und `MulExpr` selbst anzustoßen.

Alternativ könnte auch der Visitor die Traversierung vornehmen. Gerade bei der Traversierung von Datenstrukturen ist diese Variante oft von Vorteil, da man hier unterschiedliche Traversierungsarten haben möchte (Breitensuche vs. Tiefensuche, Pre-Order vs. Inorder vs. Post-Order, …) und diese elegant in den Visitor verlagern kann.

Beispiel Traversierung intern (in den Knotenklassen): visitor.visit.intrav.DemoExpr

Beispiel Traversierung extern (im Visitor): visitor.visit.extrav.DemoExpr

2.5.2.5.2 (Double-) Dispatch Zur Laufzeit wird in `accept()` der Typ des Visitors aufgelöst und dann in `visit()` der Typ der zu besuchenden Klasse. Dies nennt man auch “Double-Dispatch”.

2.5.2.5.3 Hinweis I Man könnte versucht sein, die `accept()`-Methode aus den Knotenklassen in die gemeinsame Basisklasse zu verlagern: Statt

```
1       public void accept(ExprVisitor v) {
2           v.visit(this);
3       }
```

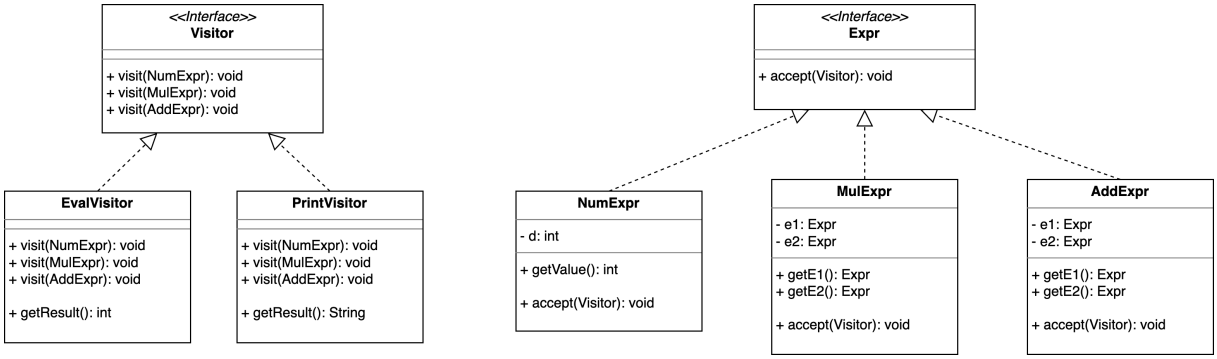
in *jeder* Knotenklasse einzeln zu definieren, könnte man das doch *einmalig* in der Basisklasse definieren:

```
1   public abstract class Expr {
2       /** Akzeptiere einen Visitor für die Verarbeitung */
3       public void accept(ExprVisitor v) {
4           v.visit(this);
5       }
6   }
```

Dies wäre tatsächlich schön, weil man so Code-Duplizierung vermeiden könnte. Aber es funktioniert in Java leider nicht. (Warum?)

2.5.2.5.4 Hinweis II Während die `accept()`-Methode nicht in die Basisklasse der besuchten Typen (im Bild oben die Klasse `Elem` bzw. im Beispiel oben die Klasse `Expr`) verlagert werden kann, kann man aber die `visit()`-Methoden im Interface `Visitor` durchaus als Default-Methoden im Interface implementieren.

2.5.2.6 Ausrechnen des Ausdrucks mit einem Visitor



Demo: visitor.visit.extrav.DemoExpr

2.5.2.7 Wrap-Up **Visitor-Pattern**: Auslagern der Traversierung in eigene Klassenstruktur

- Klassen der Datenstruktur
  - bekommen eine `accept()`-Methode für einen Visitor
  - rufen den Visitor mit sich selbst als Argument auf
- Visitor
  - hat für jede Klasse eine Überladung der `visit()`-Methode
  - Rückgabewerte schwierig: Intern halten oder per **`return`** (dann aber unterschiedliche `visit()`-Methoden für die verschiedenen Rückgabetypen!)
- (Double-) Dispatch: Zur Laufzeit wird in `accept()` der Typ des Visitors und in `visit()` der Typ der zu besuchenden Klasse aufgelöst

Zum Nachlesen

- Eilebrecht und Starke (2013)
- Gamma u. a. (2011)

Lernziele

- k2: Aufbau des Visitor-Patterns (Besucher-Entwurfsmusters)
- k3: Anwendung des Visitor-Patterns auf konkrete Beispiele, etwa den PM-Dungeon

Quizzes

- Quiz Visitor-Pattern (ILIAS)

Challenges

In den Vorgaben finden Sie Code zur Realisierung von (rudimentären) binären Suchbäumen.

1. Betrachten Sie die Klassen `BinaryNode` und `Main`. Die Klasse `BinaryNode` dient zur einfachen Repräsentierung von binären Suchbäumen, in `Main` ist ein Versuchsaufbau vorbereitet.
   - Implementieren Sie das Visitor-Pattern für den Binärbaum (in den Klassen `BinaryNode` und `Main`). Der `nodeVisitor` soll einen Binärbaum *inorder* traversieren.
   - Führen Sie in `Main` die Aufrufe auf `binaryTree` aus (3a).
   - Worin besteht der Unterschied zwischen den Aufrufen `binaryTree.accept(nodeVisitor)` und `nodeVisitor.visit(binaryTree)` (3a)?
2. In `BinaryNode` wird ein Blatt aktuell durch einen Knoten repräsentiert, der für beide Kindbäume den Wert **`null`** hat. Um Blätter besser zu repräsentieren, gibt es die Klasse `UnaryNode`.
   - Passen Sie `BinaryNode` so an, dass die Kindbäume auch `UnaryNode` sein können.
   - Entfernen Sie in `Main` die Auskommentierung um die Definition von `mixedTree`.
   - Führen Sie in `Main` die Aufrufe auf `mixedTree` aus (3b). Passen Sie dazu ggf. Ihre Implementierung des Visitor-Patterns an.
   - Worin besteht der Unterschied zwischen den Aufrufen `mixedTree.accept(nodeVisitor)` und `nodeVisitor.visit(mixedTree)` (3b)?
3. Sowohl `binaryTree` als auch `mixedTree` werden in `Main` als `BinaryNode<String>` deklariert. Das ist eine unschöne Praxis: Es soll nach Möglichkeit der Obertyp genutzt werden. Dies ist in diesem Fall `Node<String>`.
   - Entfernen Sie in `Main` die Auskommentierung um die Definition von `tree`.
   - Führen Sie in `Main` die Aufrufe auf `tree` aus (3c). Passen Sie dazu ggf. Ihre Implementierung des Visitor-Patterns an.
   - Worin besteht der Unterschied zwischen den Aufrufen `tree.accept(nodeVisitor)` und `nodeVisitor.visit(tree)` (3c)?
4. Implementieren Sie analog zu `nodeVisitor` einen weiteren Visitor, der die Bäume *postorder* traversiert und wiederholen Sie für diesen neuen Visitor die Aufrufe in (3a) bis (3c).
5. Erklären Sie, wieso im Visitor-Pattern für den Start der Traversierung statt `visitor.visit(tree)` der Aufruf `tree.accept(visitor)` genutzt wird.
6. Erklären Sie, wieso im Visitor-Pattern in der `accept`-Methode der Knoten der Aufruf `visitor.visit(this)` genutzt wird. Erklären Sie, wieso dieser Aufruf nicht in der Oberklasse bzw. im gemeinsamen Interface der Knoten implementiert werden kann.
7. Erklären Sie, wieso im Visitor-Pattern in der `visit`-Methode der Visitoren statt `visit(node.left())` der Aufruf `node.left().accept(this)` genutzt wird.

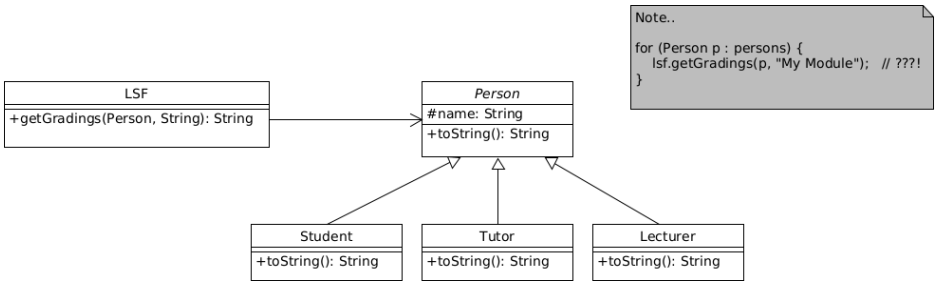
2.5.3 Observer-Pattern

TL;DR
Eine Reihe von Objekten möchte über eine Änderung in einem anderen (“zentralen”) Objekt informiert werden. Dazu könnte das “zentrale” Objekt eine Zugriffsmethode anbieten, die die anderen Objekte regelmäßig abrufen (“pollen”).
Mit dem Observer-Pattern kann man das aktive Polling vermeiden. Die interessierten Objekte “registrieren” sich beim “zentralen” Objekt. Sobald dieses eine Änderung erfährt oder Informationen bereitstehen o.ä., wird das “zentrale” Objekt alle registrierten Objekte über den Aufruf einer Methode benachrichtigen. Dazu müssen diese eine gemeinsame Schnittstelle implementieren.
Das “zentrale” Objekt, welches abgefragt wird, nennt man “*Observable*” oder “*Subject*”. Die Objekte, die die Information abfragen möchten, nennt man “*Observer*”.

Videos

- VL Observer-Pattern
- Demo Observer-Pattern

2.5.3.1 Verteilung der Prüfungsergebnisse

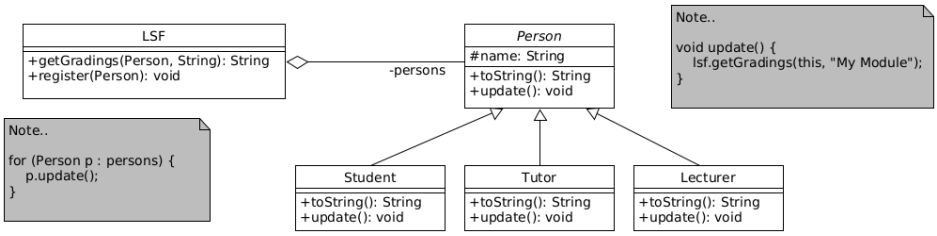


Die Studierenden möchten nach einer Prüfung wissen, ob für einen bestimmten Kurs die/ihre Prüfungsergebnisse im LSF bereit stehen.

Dazu modelliert man eine Klasse `LSF` und implementiert eine Abfragemethode, die dann alle Objekte regelmäßig aufrufen können. Dies sieht dann praktisch etwa so aus:

```
1  final Person[] persons = { new Lecturer("Frau Holle"),
2                             new Student("Heinz"),
3                             new Student("Karla"),
4                             new Tutor("Kolja"),
5                             new Student("Wuppie") };
6  final LSF lsf = new LSF();
7
8  for (Person p : persons) {
9      lsf.getGradings(p, "My Module");   // ???!
10 }
```

2.5.3.2 Elegantere Lösung: Observer-Entwurfsmuster

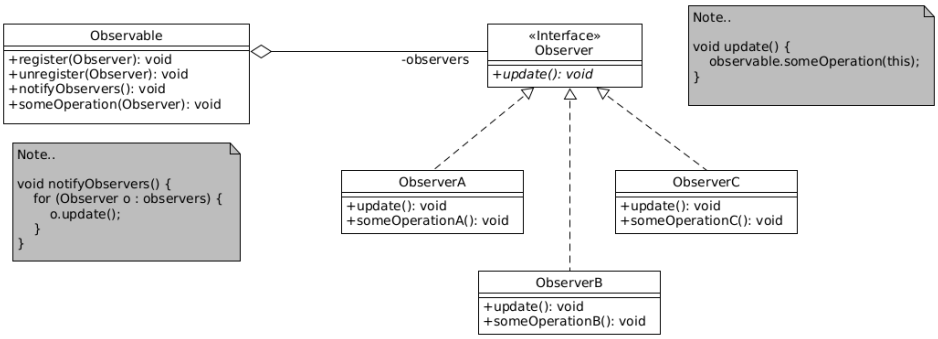


Sie erstellen im `LSF` eine Methode `register()`, mit der sich interessierte Objekte beim `LSF` registrieren können.

Zur Benachrichtigung der registrierten Objekte brauchen diese eine geeignete Methode, die traditionell `update()` genannt wird.

Demo: observer

2.5.3.3 Observer-Pattern verallgemeinert



Im vorigen Beispiel wurde die Methode `update()` einfach der gemeinsamen Basisklasse `Person` hinzugefügt. Normalerweise möchte man die Aspekte `Person` und `Observer` aber sauber trennen und definiert sich dazu ein *separates* Interface `Observer` mit der Methode `update()`, die dann alle "interessierten" Klassen (zusätzlich zur bestehenden Vererbungshierarchie) implementieren.

Die Klasse für das zu beobachtende Objekt benötigt dann eine Methode `register()`, mit der sich Observer registrieren können. Die Objektreferenzen werden dabei einfach einer internen Sammlung hinzugefügt.

Häufig findet sich dann noch eine Methode `unregister()`, mit der sich bereits registrierte Beobachter wieder abmelden können. Weiterhin findet man häufig eine Methode `notifyObservers()`, die man von außen auf dem beobachteten Objekt aufrufen kann und die dann auf allen registrierten Beobachtern deren Methoden `update()` aufruft. (Dieser Vorgang kann aber auch durch eine sonstige Zustandsänderung im beobachteten Objekt durchgeführt werden.)

In der Standarddefinition des Observer-Patterns nach (Gamma u. a. 2011) werden beim Aufruf der Methode `update()` keine Werte an die Beobachter mitgegeben. Der Beobachter muss sich entsprechend eine eigene

Referenz auf das beobachtete Objekt halten, um dort dann weitere Informationen erhalten zu können. Dies kann vereinfacht werden, indem das beobachtete Objekt beim Aufruf der `update()`-Methode die Informationen als Parameter mitgibt, beispielsweise eine Referenz auf sich selbst o.ä. … Dies muss dann natürlich im `Observer`-Interface nachgezogen werden.

Hinweis: Es gibt in Swing bereits die Interfaces `Observer` und `Observable`, die aber als "deprecated" gekennzeichnet sind. Sinnvollerweise nutzen Sie nicht diese Interfaces aus Swing, sondern implementieren Ihre eigenen Interfaces, wenn Sie das Observer-Pattern einsetzen wollen!

2.5.3.4 Wrap-Up Observer-Pattern: Benachrichtige registrierte Objekte über Statusänderungen

- Interface `Observer` mit Methode `update()`
- Interessierte Objekte
  1. implementieren das Interface `Observer`
  2. registrieren sich beim zu beobachtenden Objekt (`Observable`)
- Beobachtetes Objekt ruft auf allen registrierten Objekten `update()` auf
- `update()` kann auch Parameter haben

Zum Nachlesen

- Nystrom (2014, Kap. 4)
- Gamma u. a. (2011)

Lernziele

- k2: Aufbau des Observer-Patterns (Beobachter-Entwurfsmusters)
- k3: Anwendung des Observer-Patterns auf konkrete Beispiele, etwa den PM-Dungeon

Quizzes

- Quiz Observer-Pattern (ILIAS)

Challenges
In den Vorgaben finden Sie ein Modell für eine Lieferkette zwischen Großhandel und Einzelhandel.
Wenn beim Einzelhändler eine Bestellung von einem Kunden eingeht (`Einzelhandel#bestellen`), speichert dieser den `Auftrag` zunächst in einer Liste ab. In regelmäßigen Abständen (`Einzelhandel#loop`) sendet der Einzelhändler die offenen Bestellungen an seinen Großhändler (`Grosshandel#bestellen`). Hat der Großhändler die benötigte Ware vorrätig, sendet er diese an den Einzelhändler (`Einzelhandel#empfangen`). Dieser kann dann den Auftrag gegenüber seinem Kunden erfüllen (keine Methode vorgesehen).
Anders als der Einzelhandel speichert der Großhandel keine Aufträge ab. Ist die benötigte Ware bei einer Bestellung also nicht oder nicht in ausreichender Zahl auf Lager, wird diese nicht geliefert und der Einzelhandel muss (später) eine neue Bestellung aufgeben.
Der Großhandel bekommt regelmäßig (`Grosshandel#loop`) neue Ware für die am wenigsten vorrätigen Positionen.
Im aktuellen Modell wird der Einzelhandel nicht über den neuen Lagerbestand des Großhändlers informiert und kann daher nur "zufällig" neue Bestellanfragen an den Großhändler senden.
Verbessern Sie das Modell, indem Sie das Observer-Pattern integrieren. Wer ist Observer? Wer ist Observable? Welche Informationen werden bei einem `update` mitgeliefert?
Bauen Sie in alle Aktionen vom Einzelhändler und vom Großhändler passendes Logging ein.
Anmerkung: Sie dürfen nur die Vorgaben-Klassen `Einzelhandel` und `Grosshandel` verändern, die anderen Vorgaben-Klassen dürfen Sie nicht bearbeiten. Sie können zusätzlich benötigte eigene Klassen/Interfaces implementieren.

2.5.4 Command-Pattern

TL;DR
Das **Command-Pattern** ist die objektorientierte Antwort auf Callback-Funktionen: Man kapselt Befehle in einem Objekt.

1. Die `Command`-Objekte haben eine Methode `execute()` und führen dabei Aktion auf einem bzw. "ihrem" Receiver aus.
2. `Receiver` sind Objekte, auf denen Aktionen ausgeführt werden, im Dungeon könnten dies etwa Hero, Monster, … sein. Receiver müssen keine der anderen Akteure in diesem Pattern kennen.
3. Damit die `Command`-Objekte aufgerufen werden, gibt es einen `Invoker`, der `Command`-Objekte hat und zu gegebener Zeit auf diesen die Methode `execute()` aufruft. Der Invoker muss dabei die konkreten Kommandos und die Receiver nicht kennen (nur die `Command`-Schnittstelle).
4. Zusätzlich gibt es einen `Client`, der die anderen Akteure kennt und alles zusammen baut.

Videos

- VL Command-Pattern

2.5.4.1 Motivation Irgendwo im Dungeon wird es ein Objekt einer Klasse ähnlich wie `InputHandler` geben mit einer Methode ähnlich zu `handleInput()`:

```
1 public class InputHandler {
```

```
    public void handleInput() {
        switch (keyPressed()) {
            case BUTTON_W -> hero.jump();
            case BUTTON_A -> hero.moveX();
            case ...
            default -> { ... }
        }
    }
}
```

Diese Methode wird je Frame einmal aufgerufen, um auf eventuelle Benutzereingaben reagieren zu können. Je nach gedrücktem Button wird auf dem Hero eine bestimmte Aktion ausgeführt …

Das funktioniert, ist aber recht unflexibel. Die Aktionen sind den Buttons fest zugeordnet und erlauben keinerlei Konfiguration.

2.5.4.2 Auflösen der starren Zuordnung über Zwischenobjekte

```
public interface Command { void execute(); }

public class Jump implements Command {
    private Entity e;
    public void execute() { e.jump(); }
}

public class InputHandler {
    private final Command wbutton = new Jump(hero);  // Über Ctor/Methoden
        setzen!
    private final Command abutton = new Move(hero);  // Über Ctor/Methoden
        setzen!

    public void handleInput() {
        switch (keyPressed()) {
            case BUTTON_W -> wbutton.execute();
            case BUTTON_A -> abutton.execute();
            case ...
            default -> { ... }
        }
    }
}
```

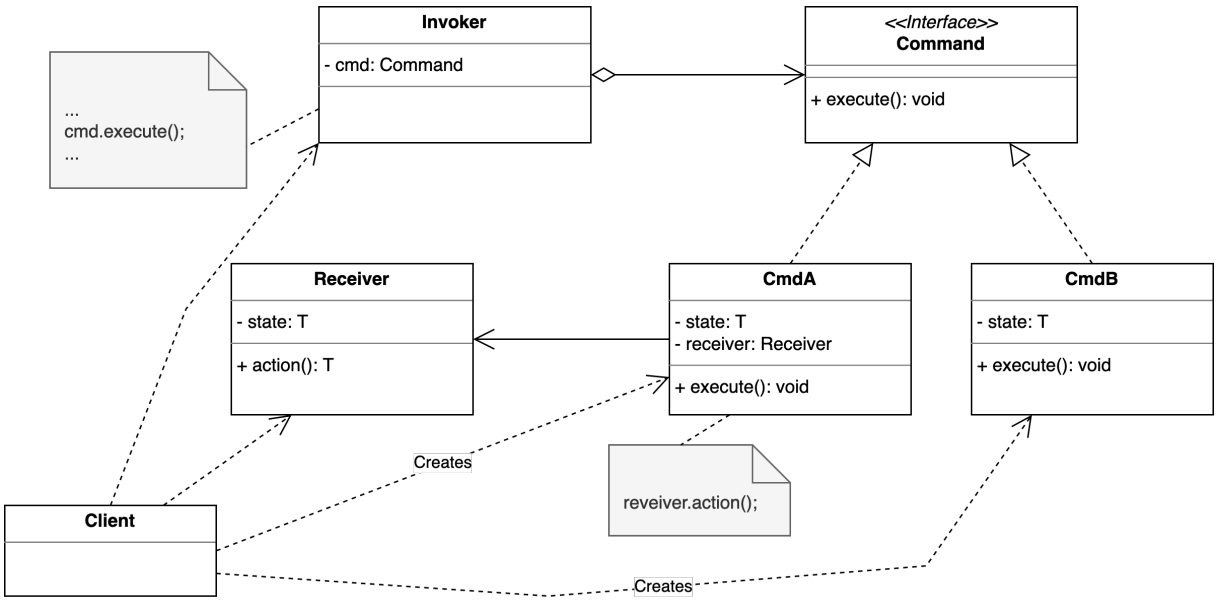
Die starre Zuordnung “Button : Aktion” wird aufgelöst und über Zwischenobjekte konfigurierbar gemacht.

Für die Zwischenobjekte wird ein Typ `Command` eingeführt, der nur eine `execute()`-Methode hat. Für jede gewünschte Aktion wird eine Klasse davon abgeleitet, diese Klassen können auch einen Zustand pflegen.

Den Buttons wird nun an geeigneter Stelle (Konstruktor, Methoden, …) je ein Objekt der jeweiligen Command-Unterklassen zugeordnet. Wenn ein Button betätigt wird, wird auf dem Objekt die Methode `execute()` aufgerufen.

Damit die Kommandos nicht nur auf den Helden wirken können, kann man den Kommando-Objekten beispielsweise noch eine Entität mitgeben, auf der das Kommando ausgeführt werden soll. Im Beispiel oben wurde dafür der `hero` genutzt.

2.5.4.3 Command: Objektorientierte Antwort auf Callback-Funktionen



Im Command-Pattern gibt es vier beteiligte Parteien: Client, Receiver, Command und Invoker.

Ein Command ist die objektorientierte Abstraktion eines Befehls. Es hat möglicherweise einen Zustand, und und kennt "seinen" Receiver und kann beim Aufruf der `execute()`-Methode eine vorher verabredete Methode auf diesem Receiver-Objekt ausführen.

Ein Receiver ist eine Klasse, die Aktionen durchführen kann. Sie kennt die anderen Akteure nicht.

Der Invoker (manchmal auch "Caller" genannt) ist eine Klasse, die Commands aggregiert und die die Commandos "ausführt", indem hier die `execute()`-Methode aufgerufen wird. Diese Klasse kennt nur das `Command`-Interface und keine spezifischen Kommandos (also keine der Sub-Klassen). Es kann zusätzlich eine gewisse Buchführung übernehmen, etwa um eine Undo-Funktionalität zu realisieren.

Der Client ist ein Programmteil, der ein Command-Objekt aufbaut und dabei einen passenden Receiver übergibt und der das Command-Objekt dann zum Aufruf an den Invoker weiterreicht.

In unserem Beispiel lassen sich die einzelnen Teile so sortieren:

- Client: Klasse `InputHandler` (erzeugt neue `Command`-Objekte im obigen Code) bzw. `main()`, wenn man die `Command`-Objekte dort erstellt und an den Konstruktor von `InputHandler` weiterreicht
- Receiver: Objekt `hero` der Klasse `Hero` (auf diesem wird eine Aktion ausgeführt)
- Command: `Jump` und `Move`
- Invoker: `InputHandler` (in der Methode `handleInput()`)

2.5.4.4 Undo

Wir könnten das `Command`-Interface um ein paar Methoden erweitern:

```
1  public interface Command {
2      void execute();
3      void undo();
4      Command newCommand(Entity e);
```

```
5 }
```

Jetzt kann jedes Command-Objekt eine neue Instanz erzeugen mit der Entity, die dann dieses Kommando empfangen soll:

```
1  public class Move implements Command {
2      private Entity e;
3      private int x, y, oldX, oldY;
4
5      public void execute() { oldX = e.getX();  oldY = e.getY();  x = oldX + 42;
           y = oldY;  e.moveTo(x, y); }
6      public void undo() { e.moveTo(oldX, oldY); }
7      public Command newCommand(Entity e) { return new Move(e); }
8  }
9
10 public class InputHandler {
11     private final Command wbutton;
12     private final Command abutton;
13     private final Stack<Command> s = new Stack<>();
14
15     public void handleInput() {
16         Entity e = getSelectedEntity();
17         switch (keyPressed()) {
18             case BUTTON_W -> { s.push(wbutton.newCommand(e)); s.peek().execute
                   (); }
19             case BUTTON_A -> { s.push(abutton.newCommand(e)); s.peek().execute
                   (); }
20             case BUTTON_U -> s.pop().undo();
21             case ...
22             default -> { ... }
23         }
24     }
25 }
```

Über den Konstruktor von `InputHandler` (im Beispiel nicht gezeigt) würde man wie vorher die `Command`-Objekte für die Buttons setzen. Es würde aber in jedem Aufruf von `handleInput()` abgefragt, was gerade die selektierte Entität ist und für diese eine neue Instanz des zur Tastatureingabe passenden `Command`-Objekts erzeugt. Dieses wird nun in einem Stack gespeichert und danach ausgeführt.

Wenn der Button “U” gedrückt wird, wird das letzte `Command`-Objekt aus dem Stack genommen (Achtung: Im echten Leben müsste man erst einmal schauen, ob hier noch was drin ist!) und auf diesem die Methode `undo()` aufgerufen. Für das Kommando `Move` ist hier skizziert, wie ein Undo aussehen könnte: Man muss einfach bei jedem `execute()` die alte Position der Entität speichern, dann kann man sie bei einem `undo()` wieder auf diese Position verschieben. Da für jeden Move ein neues Objekt angelegt wird und dieses nur einmal benutzt wird, braucht man keine weitere Buchhaltung …

2.5.4.5 Wrap-Up **Command-Pattern**: Kapsele Befehle in ein Objekt

- `Command`-Objekte haben eine Methode `execute()` und führen darin Aktion auf Receiver aus
- `Receiver` sind Objekte, auf denen Aktionen ausgeführt werden (Hero, Monster, …)
- `Invoker` hat `Command`-Objekte und ruft darauf `execute()` auf

- `Client` kennt alle und baut alles zusammen

Objektorientierte Antwort auf Callback-Funktionen

Zum Nachlesen

- Gamma u. a. (2011)
- Nystrom (2014, Kap. 2)

Lernziele

- k2: Aufbau des Command-Patterns
- k3: Anwendung des Command-Patterns auf konkrete Beispiele, etwa den PM-Dungeon

Quizzes

- Quiz Command-Pattern (ILIAS)

Challenges

Schreiben Sie für den `Dwarf` in den Vorgaben einen Controller, welcher das Command-Pattern verwendet.

- “W” führt Springen aus
- “A” bewegt den Zwerg nach links
- “D” bewegt den Zwerg nach rechts
- “S” führt Ducken aus

Schreiben Sie zusätzlich für den `Cursor` einen Controller, welcher das Command-Pattern mit Historie erfüllt (ebenfalls über die Tasten “W”, “A”, “S” und “D”).
Schreiben Sie eine Demo, um die Funktionalität Ihres Programmes zu demonstrieren.

2.5.5 Singleton-Pattern

TL;DR

Wenn von einer Klasse nur genau ein Objekt angelegt werden kann, nennt man dies auch das “Singleton-Pattern”. Dazu muss verhindert werden, dass der Konstruktor aufgerufen werden kann. Üblicherweise “versteckt” man diesen einfach (Sichtbarkeit auf `private` setzen). Für den Zugriff auf die Instanz bietet man eine statische Methode an. Im Prinzip kann man die Instanz direkt beim Laden der Klasse anlegen (“Eager”) oder abwarten, bis die Instanz über die statische Methode angefordert wird, und das Objekt erst dann anlegen (“Lazy”).

Videos

- VL Singleton-Pattern

2.5.5.1 Motivation

```
public enum Fach { IFM, ELM, ARC }
```

```
Logger l = Logger.getLogger(MyClass.class.getName());
```

Von den Enum-Konstanten soll es nur genau eine Instantiierung, also jeweils nur genau ein Objekt geben. Ähnlich war es beim Logging: Für jeden Namen soll/darf es nur einen tatsächlichen Logger (== Objekt) geben.

Dies nennt man "**Singleton Pattern**".

Anmerkung: Im Logger-Fall handelt es sich streng genommen nicht um ein Singleton, da es vom Logger mehrere Instanzen geben kann (wenn der Name sich unterscheidet). Aber jeden Logger mit einem bestimmten Namen gibt es nur einmal im ganzen Programm, insofern ist es doch wieder ein Beispiel für das Singleton-Pattern …

2.5.5.2 Umsetzung: "Eager" Singleton Pattern Damit man von "außen" keine Instanzen einer Klasse anlegen kann, versteckt man den Konstruktor, d.h. man setzt die Sichtbarkeit auf `private`. Zusätzlich benötigt man eine Methode, die das Objekt zurückliefern kann. Beim Logger war dies beispielsweise der Aufruf `Logger.getLogger("name")`.

Man kann verschiedene Ausprägungen bei der Umsetzung des Singleton Patterns beobachten. Die beiden wichtigsten sind das "Eager Singleton Pattern" und das "Lazy Singleton Pattern". Der Unterschied liegt darin, wann genau das Objekt erzeugt wird: Beim "Eager Singleton Pattern" wird es direkt beim Laden der Klasse erzeugt.

```
public class SingletonEager {
    private static final SingletonEager inst = new SingletonEager();

    // Privater Constructor: Niemand kann Objekte außerhalb der Klasse anlegen
    private SingletonEager() {}

    public static SingletonEager getInst() {
        return inst;
    }
}
```

Beispiel: singleton.SingletonEager

2.5.5.3 Umsetzung: "Lazy" Singleton Pattern Beim "Lazy Singleton Pattern" wird das Objekt erst erzeugt, wenn die Instanz tatsächlich benötigt wird (also erst beim Aufruf der `get`-Methode).

```
public class SingletonLazy {
    private static SingletonLazy inst = null;

    // Privater Constructor: Niemand kann Objekte außerhalb der Klasse anlegen
    private SingletonLazy() {}

    public static SingletonLazy getInst() {
        // Thread-safe. Kann weggelassen werden bei Single-Threaded-Gebrauch
        synchronized (SingletonLazy.class) {
            if (inst == null) {
```

```
11                  inst = new SingletonLazy();
12              }
13          }
14          return inst;
15      }
16  }
```

Beispiel: singleton.SingletonLazy

2.5.5.4 Vorsicht!

Sie schaffen damit eine globale Variable!

Da es von der Klasse nur eine Instanz gibt, und Sie sich diese dank der statischen Methode an jeder Stelle im Programm "geben" lassen können, haben Sie in der Praxis eine globale Variable geschaffen. Das kann direkt zu schlechter Programmierung (ver-) führen. Zudem wird der Code schwerer lesbar/navigierbar, da diese Singletons nicht über die Schnittstellen von Methoden übergeben werden müssen.

Nutzen Sie das Pattern **sparsam**.

2.5.5.5 Wrap-Up Singleton-Pattern: Klasse, von der nur genau ein Objekt instantiiert werden kann

1. Konstruktor "verstecken" (Sichtbarkeit auf `private` setzen)
2. Methode zum Zugriff auf die eine Instanz
3. Anlegen der Instanz beispielsweise beim Laden der Klasse ("Eager") oder beim Aufruf der Zugriffsmethode ("Lazy")

Zum Nachlesen

- Nystrom (2014, Kap. 6)

Lernziele

- k2: Was ist ein Singleton? Was ist der Unterschied zw. einem Lazy und einem Eager Singleton?
- k3: Anwendung des Singleton-Patterns

Quizzes

- Quiz Singleton-Pattern (ILIAS)

2.5.6 Template-Method-Pattern

TL;DR
Das Template-Method-Pattern ist ein Entwurfsmuster, bei dem ein gewisses Verhalten in einer Methode implementiert wird, die wie eine Schablone agiert, der sogenannten "Template-Methode". Darin werden dann u.a. Hilfsmethoden aufgerufen, die in der Basisklasse entweder als **`abstract`** markiert sind oder mit einem leeren Body implementiert sind ("Hook-Methoden"). Über diese Template-Methode legt also die Basisklasse ein gewisses Verhaltensschema fest ("Template") - daher auch der Name.
In den ableitenden Klassen werden dann die abstrakten Methoden und/oder die Hook-Methoden implementiert bzw. überschrieben und damit das Verhalten verfeinert.
Zur Laufzeit ruft man auf den Objekten die Template-Methode auf. Dabei wird von der Laufzeitumgebung der konkrete Typ der Objekte bestimmt (auch wenn man sie unter dem Typ der Oberklasse führt) und die am tiefsten in der Vererbungshierarchie implementierten Methoden aufgerufen. D.h. die Aufrufe der Hilfsmethoden in der Template-Methode führen zu den in der jeweiligen ableitenden Klasse implementierten Varianten.

Videos

- VL Template-Method-Pattern

2.5.6.1 Motivation: Syntax-Highlighting im Tokenizer In einem Compiler ist meist der erste Arbeitsschritt, den Eingabestrom in einzelne Token aufzubrechen. Dies sind oft die verschiedenen Schlüsselwörter, Operationen, Namen von Variablen, Methoden, Klassen etc. … Aus der Folge von Zeichen (also dem eingelesenen Programmcode) wird ein Strom von Token, mit dem die nächste Stufe im Compiler dann weiter arbeiten kann.

```java
public class Lexer {
    private final List<Token> allToken;  // alle verfügbaren Token-Klassen

    public List<Token> tokenize(String string) {
        List<Token> result = new ArrayList<>();

        while (string.length() > 0) {
            for (Token t : allToken) {
                Token token = t.match(string);
                if (token != null) {
                    result.add(token);
                    string = string.substring(token.getContent().length(),
                        string.length());
                }
            }
        }

        return result;
    }
}
```

Dazu prüft man jedes Token, ob es auf den aktuellen Anfang des Eingabestroms passt. Wenn ein Token passt, erzeugt man eine Instanz dieser Token-Klasse und speichert darin den gematchten Eingabeteil, den man dann vom Eingabestrom entfernt. Danach geht man in die Schleife und prüft wieder alle Token … bis irgendwann der

Eingabestrom leer ist und man den gesamten eingelesenen Programmcode in eine dazu passende Folge von Token umgewandelt hat.

Anmerkung: Abgesehen von fehlenden Javadoc etc. hat das obige Code-Beispiel mehrere Probleme: Man würde im realen Leben nicht mit `String`, sondern mit einem Zeichenstrom arbeiten. Außerdem fehlt noch eine Fehlerbehandlung, wenn nämlich keines der Token in der Liste `allToken` auf den aktuellen Anfang des Eingabestroms passt.

2.5.6.2 Token-Klassen mit formatiertem Inhalt

Um den eigenen Tokenizer besser testen zu können, wurde beschlossen, dass jedes Token seinen Inhalt als formatiertes HTML-Schnipsel zurückliefern soll. Damit kann man dann alle erkannten Token formatiert ausgeben und erhält eine Art Syntax-Highlighting für den eingelesenen Programmcode.

```
1  public abstract class Token {
2      protected String content;
3
4      abstract protected String getHtml();
5  }
6  public class KeyWord extends Token {
7      @Override
8      protected String getHtml() {
9          return "<font color=\"red\"><b>" +  this.content + "</b></font>";
10     }
11 }
12 public class StringContent extends Token {
13     @Override
14     protected String getHtml() {
15         return "<font color=\"green\">" +  this.content + "</font>";
16     }
17 }
18
19
20 Token t = new KeyWord();
21 LOG.info(t.getHtml());
```

In der ersten Umsetzung erhält die Basisklasse `Token` eine weitere abstrakte Methode, die jede Token-Klasse implementieren muss und in der die Token-Klassen einen String mit dem Token-Inhalt und einer Formatierung für HTML zurückgeben.

Dabei fällt auf, dass der Aufbau immer gleich ist: Es werden ein oder mehrere Tags zum Start der Format-Sequenz mit dem Token-Inhalt verbunden, gefolgt mit einem zum verwendeten startenden HTML-Format-Tag passenden End-Tag.

Auch wenn die Inhalte unterschiedlich sind, sieht das stark nach einer Verletzung von DRY aus …

2.5.6.3 Don’t call us, we’ll call you

```
1  public abstract class Token {
2      protected String content;
3
4      public final String getHtml() {
5          return htmlStart() + this.content + htmlEnd();
6      }
```

```
7
8       abstract protected String htmlStart();
9       abstract protected String htmlEnd();
10  }
11  public class KeyWord extends Token {
12      @Override protected String htmlStart() { return "<font color=\"red\"><b>";
            }
13      @Override protected String htmlEnd() { return "</b></font>"; }
14  }
15  public class StringContent extends Token {
16      @Override protected String htmlStart() { return "<font color=\"green\">"; }
17      @Override protected String htmlEnd() { return "</font>"; }
18  }
19
20
21  Token t = new KeyWord();
22  LOG.info(t.getHtml());
```

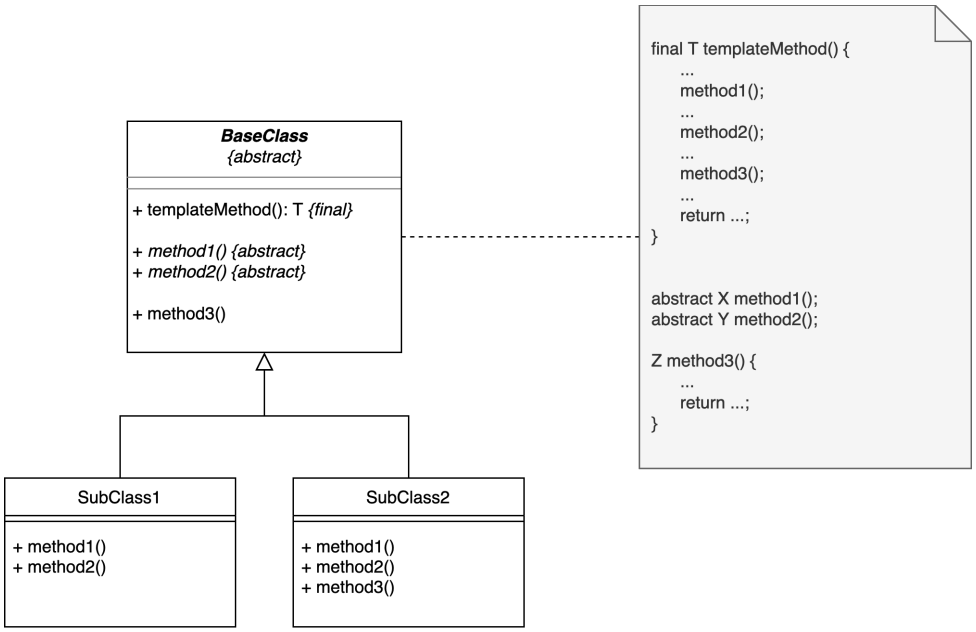
Wir können den Spaß einfach umdrehen (“inversion of control”) und die Methode zum Zusammenbasteln des HTML-Strings bereits in der Basisklasse implementieren. Dazu “rufen” wir dort drei Hilfsmethoden auf, die die jeweiligen Bestandteile des Strings (Format-Start, Inhalt, Format-Ende) erzeugen und deren konkrete Implementierung wir in der Basisklasse nicht kennen. Dies ist dann Sache der ableitenden konkreten Token-Klassen.

Objekte vom Typ `KeyWord` sind dank der Vererbungsbeziehung auch `Token` (Vererbung: *is-a-Beziehung*). Wenn man nun auf einem `Token t` die Methode `getHtml()` aufruft, wird zur Laufzeit geprüft, welchen Typ `t` tatsächlich hat (im Beispiel `KeyWord`). Methodenaufrufe werden dann mit den am tiefsten in der vorliegenden Vererbungshierarchie implementierten Methoden durchgeführt: Hier wird also die von `Token` geerbte Methode `getHtml()` in `KeyWord` aufgerufen, die ihrerseits die Methoden `htmlStart()` und `htmlEnd()` aufruft. Diese sind in `KeyWord` implementiert und liefern nun die passenden Ergebnisse.

Die Methode `getHtml()` wird auch als “*Template-Methode*” bezeichnet. Die beiden darin aufgerufenen Methoden `htmlStart()` und `htmlEnd()` in `Token` werden auch als “Hilfsmethoden” (oder “*Helper Methods*”) bezeichnet.

Dies ist ein Beispiel für das **Template-Method-Pattern**.

2.5.6.4 Template-Method-Pattern



2.5.6.4.1 Aufbau Template-Method-Pattern In der Basisklasse implementiert man eine Template-Methode (in der Skizze `templateMethod`), die sich auf anderen in der Basisklasse deklarierten (Hilfs-) Methoden "abstützt" (diese also aufruft; in der Skizze `method1`, `method2`, `method3`). Diese Hilfsmethoden können als **`abstract`** markiert werden und müssen dann von den ableitenden Klassen implementiert werden (in der Skizze `method1` und `method2`). Man kann aber auch einige/alle dieser aufgerufenen Hilfsmethoden in der Basisklasse implementieren (beispielsweise mit einem leeren Body - sogenannte "Hook"-Methoden) und die ableitenden Klassen *können* dann diese Methoden überschreiben und das Verhalten so neu formulieren (in der Skizze `method3`).

Damit werden Teile des Verhaltens an die ableitenden Klassen ausgelagert.

2.5.6.4.2 Verwandtschaft zum Strategy-Pattern Das Template-Method-Pattern hat eine starke Verwandtschaft zum Strategy-Pattern.

Im Strategy-Pattern haben wir Verhalten komplett an andere Objekte *delegiert*, indem wir in einer Methode einfach die passende Methode auf dem übergebenen Strategie-Objekt aufgerufen haben.

Im Template-Method-Pattern nutzen wir statt Delegation die Mechanismen Vererbung und dynamische Polymorphie und definieren in der Basis-Klasse abstrakte oder Hook-Methoden, die wir bereits in der Template-Methode der Basis-Klasse aufrufen. Damit ist das grobe Verhalten in der Basis-Klasse festgelegt, wird aber in den ableitenden Klassen durch das dortige Definieren oder Überschreiben der Hilfsmethoden verfeinert. Zur Laufzeit werden dann durch die dynamische Polymorphie die tatsächlich implementierten Hilfsmethoden in den ableitenden Klassen aufgerufen. Damit lagert man im Template-Method-Pattern gewissermaßen nur Teile des Verhaltens an die ableitenden Klassen aus.

2.5.6.5 Wrap-Up Template-Method-Pattern: Verhaltensänderung durch Vererbungsbeziehungen

- Basis-Klasse:
  - Template-Methode, die Verhalten definiert und Hilfsmethoden aufruft
  - Hilfsmethoden: Abstrakte Methoden (oder “Hook”: Basis-Implementierung)
- Ableitende Klassen: Verfeinern Verhalten durch Implementieren der Hilfsmethoden
- Zur Laufzeit: Dynamische Polymorphie: Aufruf der Template-Methode nutzt die im tatsächlichen Typ des Objekts implementierten Hilfsmethoden

Zum Nachlesen

- Eilebrecht und Starke (2013)
- Gamma u. a. (2011)

Lernziele

- k3: Template-Method-Entwurfsmuster praktisch anwenden

Quizzes

- Quiz Template-Method-Pattern (ILIAS)

Challenges

Schreiben Sie eine abstrakte Klasse Drucker. Implementieren Sie die Funktion `kopieren`, bei der zuerst die Funktion `scannen` und dann die Funktion `drucken` aufgerufen wird. Der Kopiervorgang ist für alle Druckertypen identisch, das Scannen und Drucken ist abhängig vom Druckertyp.
Implementieren Sie zusätzlich zwei unterschiedliche Druckertypen.

- `Tintendrucker extends Drucker`
- `Laserdrucker extends Drucker`
- `Tintendrucker#scannen` loggt den Text “Scanne das Dokument mit dem Tintendrucker.”
- `Laserdrucker#scannen` loggt den Text “Scanne das Dokument mit dem Laserdrucker.”
- `Tintendrucker#drucken` loggt den Text “Drucke das Dokument auf dem Tintendrucker.”
- `Laserdrucker#drucken` loggt den Text “Drucke das Dokument auf dem Laserdrucker.”

Nutzen Sie das Template-Method-Pattern.

2.5.7 Factory-Method-Pattern

TL;DR
Oft ist es wünschenswert, dass Nutzer nicht direkt Objekte von bestimmten Klassen anlegen (können). Hier kann eine "Fabrik-Methode" (**Factory-Method**) helfen, der man die gewünschten Parameter übergibt und die daraus dann das passende Objekt (der richtigen Klasse) erzeugt und zurückliefert.
Dadurch erreicht man eine höhere Entkoppelung, die Nutzer müssen nur noch das Interface oder die abstrakte Klasse, also den Obertyp des Ergebnisses kennen. Außerdem lassen sich so leicht die konkreten Klassen austauschen.
Dieses Entwurfsmuster kommt häufig zusammen mit dem *Singleton-Pattern* vor, wo es nur eine einzige Instanz einer Klasse geben soll. Über eine Fabrik-Methode kann man diese Instanz ggf. erzeugen und dann die Referenz darauf zurückliefern.

Videos

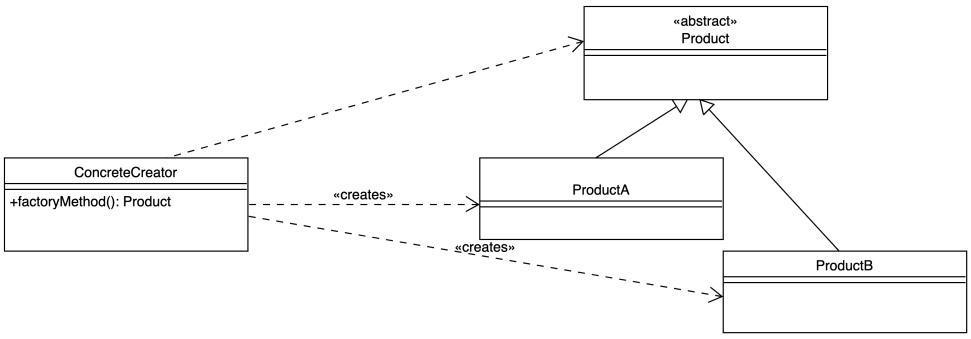
- VL Factory-Method-Pattern
- Demo Factory-Method-Pattern

2.5.7.1 Motivation: Ticket-App

- Nutzer geben Fahrtziel an (und nicht die Ticketart!)
- Ticket-App bucht passendes Ticket
  - User muss nicht die konkreten Ticketarten kennen
  - Ticketarten lassen sich leicht austauschen

=> **Factory-Method-Pattern**: Objekte sollen nicht direkt durch den Nutzer erzeugt werden

2.5.7.2 Factory-Method-Pattern



2.5.7.3 Hands-On: Ticket-App Implementieren Sie eine Ticket-App, die verschiedene Tickets mit Hilfe des Factory-Method Entwurfsmusters generiert.

UML; Konsole: factory.FactoryBeispiel

2.5.7.4 Wrap-Up

- Konkrete Objekte sollen nicht direkt über Konstruktor erzeugt werden
- (Statische) Hilfsmethode, die aus Parameter das “richtige” Objekte erzeugt

- Vorteil:
  - Nutzer kennt nur das Interface
  - Konkrete Klassen lassen sich leicht austauschen

Zum Nachlesen

- Eilebrecht und Starke (2013)
- Gamma u. a. (2011)
- Kleuker (2018)

Lernziele

- k3: Entwurfsmuster Factory-Methode anwenden

Quizzes

- Quiz Factory-Method-Pattern (ILIAS)

Challenges

Ein Kunde kommt in unser Computergeschäft und möchte bei uns einen Computer bestellen. Dabei gibt er an, wie er diesen vorwiegend nutzen möchte bzw. für welchen Zweck er gedacht ist (“*stationär*” oder “*mobil*”). Nach reichlicher Überlegung, ob er den neuen Rechner zu Hause stehen haben möchte oder lieber keinen weiteren Rechner, egal ob “*mobil*” oder “*stationär*”, bei sich im Weg herumstehen haben will, teilt er Ihnen seine Entscheidung darüber mit (“*stationär*” oder “*mobil*” vs. “*nicht daheim*”). Bei diesem Gespräch merkt er beiläufig an, dass es ein Rechner mit “*viel Wumms*” sein könnte oder vielleicht doch besser etwas Kleines, was leise vor sich hin schnurrt (“*viel Wumms*” vs. “*leise schnurrend*”).

Je nach gewünschter Konfiguration soll ein den oben genannten Auswahlkriterien entsprechender Rechner mit den aus der unten stehenden Konfigurationsmatrix zu entnehmenden Eigenschaften automatisch erzeugt werden. Die Größe des installierten RAM, die Anzahl der eingebauten CPU-Kerne mit ihrer jeweiligen Taktrate, sowie die Art und Größe der installierten Festplatte (HDD oder SSD) sollte dabei zu dem gewählten Paket passend gesetzt werden.

Implementieren Sie eine “Computerfabrik” (Klasse `ComputerFactory`), die Ihnen den richtig konfigurierten Rechner zusammenbaut. Nutzen Sie dabei das “Factory-Method-Pattern” zum Erzeugen der Objekte der einzelnen Subklassen. Dabei soll Ihre Computerfabrik anhand der ihr übergebenen Konfiguration eigenständig entscheiden, welche Art von Computer dabei erstellt werden soll.

Implementieren Sie dazu in Ihrer Factory die Factory-Methode `buildComputer`, welche das jeweils passend konfigurierte Objekt zurückgibt.

```
1  public class ComputerFactory {
2      ...
3
4      public static Computer buildComputer(..."stationär",..."viel Wumms") {
5          ...
6          return myComputer;
7      }
8  }
```

Konfigurationsmatrix

	“stationär” (`DesktopComputer`)	“mobil” (`LaptopComputer`)	“nicht daheim” (`CloudComputer`)
“leise schnurrend”	8 Cores, 1.21GHZ, 16GB RAM, 256GB HDD	4 Cores, 1.21GHZ, 8GB RAM, 256GB HDD	8 Cores, 1.21GHZ, 24GB RAM, 1000GB HDD
“viel Wumms”	16 Cores, 4.2GHZ, 32GB RAM, 2000GB SSD	8 Cores, 2.4GHZ, 16GB RAM, 256GB SSD	42 Cores, 9.001GHZ, 128GB RAM, 10000GB SSD

2.5.8 Type-Object-Pattern

TL;DR
Das Type-Object-Pattern dient dazu, die Anzahl der Klassen auf Code-Ebene zu reduzieren und durch eine Konfiguration zu ersetzen und damit eine höhere Flexibilität zu erreichen.
Dazu werden sogenannte Type-Objects definiert: Sie enthalten genau die Eigenschaften, die in verschiedenen (Unter-) Klassen gemeinsam vorkommen. Damit können diese Eigenschaften aus den ursprünglichen Klassen entfernt und durch eine Referenz auf ein solches Type-Object ersetzt werden. In den Klassen muss man dann nur noch die für die einzelnen Typen individuellen Eigenschaften implementieren. Zusätzlich kann man nun verschiedene (Unter-) Klassen zusammenlegen, da der Typ über das geteilte Type-Object definiert wird (zur Laufzeit) und nicht mehr durch eine separate Klasse auf Code-Ebene repräsentiert werden muss.
Die Type-Objects werden zur Laufzeit mit den entsprechenden Ausprägungen der früheren (Unter-) Klassen angelegt und dann über den Konstruktor in die nutzenden Objekte übergeben. Dadurch teilen sich alle Objekte einer früheren (Unter-) Klasse das selbe Type-Objekt und zeigen nach außen das selbe Verhalten. Die Type-Objects werden häufig über eine entsprechende Konfiguration erzeugt, so dass man beispielsweise unterschiedliche Monsterklassen und -eigenschaften ausprobieren kann, ohne den Code neu kompilieren zu müssen. Man kann sogar eine Art "Vererbung" unter den Type-Objects implementieren.

Videos

- VL Type-Object-Pattern

2.5.8.1 Motivation: Monster und spezialisierte Monster

```java
public abstract class Monster {
    protected int attackDamage;
    protected int movementSpeed;

    public Monster(int attackDamage, int movementSpeed) { ... }
    public void attack(Monster m)  { ... }
}

public class Rat extends Monster {
    public Rat() { super(10, 10); }  // Ratten haben 10 Damage und 10 Speed
    @Override public void attack(Monster m)  { ... }
}

public class Gnoll extends Monster { ... }


public static void main(String[] args) {
    Monster harald = new Rat();
    Monster eve = new Gnoll();
    ...
}
```

Sie haben sich eine Monster-Basisklasse geschrieben. Darin gruppieren Sie typische Eigenschaften eines Monsters: Es kann sich mit einer bestimmten Geschwindigkeit bewegen und es kann anderen Monstern bei einem Angriff einen bestimmten Schaden zufügen.

Um nun andere Monstertypen zu erzeugen, greifen Sie zur Vererbung und leiten von der Basisklasse Ihre speziali-

sierten Monster ab und überschreiben die Defaultwerte und bei Bedarf auch das Verhalten (die Methoden).

Damit entsteht aber recht schnell eine tiefe und verzweigte Vererbungshierarchie, Sie müssen ja für jede Variation eine neue Unterklasse anlegen. Außerdem müssen für jede (noch so kleine) Änderung an den Monster-Eigenschaften viele Klassen editiert und das gesamte Projekt neu kompiliert werden.

Es würde auch nicht wirklich helfen, die Eigenschaften der Unterklassen über deren Konstruktor einstellbar zu machen (die `Rat` könnte in ihrem Konstruktor beispielsweise noch die Werte für Damage und Speed übergeben bekommen). Dann würden die Eigenschaften an allen Stellen im Programm verstreut, wo Sie den Konstruktor aufrufen.

2.5.8.2 Vereinfachen der Vererbungshierarchie (mit Enums als Type-Object)

```
1  public enum Species { RAT, GNOLL, ... }
2
3  public final class Monster {
4      private final Species type;
5      private int attackDamage;
6      private int movementSpeed;
7
8      public Monster(Species type) {
9          switch (type) {
10             case RAT: attackDamage = 10; movementSpeed = 10; break;
11             ...
12         }
13     }
14     public void attack(Monster m)  { ... }
15 }
16
17
18 public static void main(String[] args) {
19     Monster harald = new Monster(Species.RAT);
20     Monster eve = new Monster(Species.GNOLL);
21     ...
22 }
```

Die Lösung für die Vermeidung der Vererbungshierarchie: Die Monster-Basisklasse bekommt ein Attribut, welches den Typ des Monsters bestimmt (das sogenannte “Type-Object”). Das könnte wie im Beispiel ein einfaches Enum sein, das in den Methoden des Monsters abgefragt wird. So kann zur Laufzeit bei der Erzeugung der Monster-Objekte durch Übergabe des Enums bestimmt werden, was genau dieses konkrete Monster genau ist bzw. wie es sich verhält.

Im obigen Beispiel wird eine Variante gezeigt, wo das Enum im Konstruktor ausgewertet wird und die Attribute entsprechend gesetzt werden. Man könnte das auch so implementieren, dass man auf die Attribute verzichtet und stattdessen stets das Enum auswertet.

Allerdings ist das Hantieren mit den Enums etwas umständlich: Man muss an allen Stellen, wo das Verhalten der Monster unterschiedlich ist, ein **`switch`**/**`case`** einbauen und den Wert des Type-Objects abfragen. Das bedeutet einerseits viel duplizierten Code und andererseits muss man bei Erweiterungen des Enums auch *alle* **`switch`**/**`case`**-Blöcke anpassen.

2.5.8.3 Monster mit Strategie

```
1  public final class Species {
```

```
    private final int attackDamage;
    private final int movementSpeed;
    private final int xp;

    public Species(int attackDamage, int movementSpeed, int xp) { ... }
    public void attack(Monster m)  { ... }
}

public final class Monster {
    private final Species type;
    private int xp;

    public Monster(Species type) { this.type = type;  xp = type.xp(); }
    public int movementSpeed() { return type.movementSpeed(); }
    public void attack(Monster m)  { type.attack(m); }
}


public static void main(String[] args) {
    final Species RAT = new Species(10, 10, 4);
    final Species GNOLL = new Species(...);

    Monster harald = new Monster(RAT);
    Monster eve = new Monster(GNOLL);
}
```

Statt des Enums nimmt man eine “echte” Klasse mit Methoden für die Type-Objects. Davon legt man zur Laufzeit Objekte an (das sind dann die möglichen Monster-Typen) und bestückt damit die zu erzeugenden Monster.

Im Monster selbst rufen die Monster-Methoden dann einfach nur die Methoden des Type-Objects auf (Delegation => Strategie-Pattern). Man kann aber auch Attribute im Monster selbst pflegen und durch das Type-Object nur passend initialisieren.

Vorteil: Änderungen erfolgen bei der Parametrisierung der Objekte (an **einer** Stelle im Code, vermutlich `main()` oder beispielsweise durch Einlesen einer Konfig-Datei).

2.5.8.4 Fabrikmethode für die Type-Objects

```
public final class Species {
    ...

    public Monster newMonster() {
        return new Monster(this);
    }
}


public static void main(String[] args) {
    final Species RAT = new Species(10, 10, 4);
    final Species GNOLL = new Species(...);

    Monster harald = RAT.newMonster();
    Monster eve = GNOLL.newMonster();
}
```

Das Hantieren mit den Type-Objects und den Monstern ist nicht so schön. Deshalb kann man in der Klasse für die

Type-Objects noch eine Fabrikmethode (=> Factory-Method-Pattern) mit einbauen, über die dann die Monster erzeugt werden.

2.5.8.5 Vererbung unter den Type-Objects

```
public final class Species {
    ...

    public Species(int attackDamage, int movementSpeed, int xp) {
        this.attackDamage = attackDamage;  this.movementSpeed = movementSpeed;
            this.xp = xp;
    }
    public Species(Species parent, int attackDamage) {
        this.attackDamage = attackDamage;
        movementSpeed = parent.movementSpeed;  xp = parent.xp;
    }
}


public static void main(String[] args) {
    final Species RAT = new Species(10, 10, 4);
    final Species BOSS_RAT = new Species(RAT, 100);
    final Species GNOLL = new Species(...);

    Monster harald = RAT.newMonster();
    Monster eve = GNOLL.newMonster();
}
```

Es wäre hilfreich, wenn die Type-Objects Eigenschaften untereinander teilen/weitergeben könnten. Damit man aber jetzt nicht hier eine tiefe Vererbungshierarchie aufbaut und damit wieder am Anfang des Problems wäre, baut man die Vererbung quasi selbst ein über eine Referenz auf ein Eltern-Type-Object. Damit kann man zur Laufzeit einem Type-Object sagen, dass es bestimmte Eigenschaften von einem anderen Type-Object übernehmen soll.

Im Beispiel werden die Eigenschaften `movementSpeed` und `xp` "vererbt" und entsprechend aus dem Eltern-Type-Object übernommen (sofern dieses übergeben wird).

2.5.8.6 Erzeugen der Type-Objects dynamisch über eine Konfiguration

```
{
    "Rat": {
        "attackDamage": 10,
        "movementSpeed": 10,
        "xp": 4
    },
    "BossRat": {
        "parent": "Rat",
        "attackDamage": 100
    },
    "Gnoll": {
        "attackDamage": ...,
        "movementSpeed": ...,
        "xp": ...
    }
}
```

Jetzt kann man die Konfiguration der Type-Objects in einer Konfig-Datei ablegen und einfach an einer passenden Stelle im Programm einlesen. Dort werden dann damit die Type-Objects angelegt und mit Hilfe dieser dann die passend konfigurierten Monster (und deren Unterarten) erzeugt.

2.5.8.7 Vor- und Nachteile des Type-Object-Pattern

2.5.8.7.1 Vorteil Es gibt nur noch wenige Klassen auf Code-Ebene (im Beispiel: 2), und man kann über die Konfiguration beliebig viele Monster-Typen erzeugen.

2.5.8.7.2 Nachteil Es werden zunächst nur Daten "überschrieben", d.h. man kann nur für die einzelnen Typen spezifische Werte mitgeben/definieren.

Bei Vererbung kann man in den Unterklassen nahezu beliebig das Verhalten durch einfaches Überschreiben der Methoden ändern. Das könnte man in diesem Entwurfsmuster erreichen, in dem man beispielsweise eine Reihe von vordefinierten Verhaltensarten implementiert, die dann anhand von Werten ausgewählt und anhand anderer Werte weiter parametrisiert werden.

2.5.8.7.3 Verwandtschaft zum Flyweight-Pattern Das Type-Object-Pattern ist keines der "klassischen" Design-Pattern der "Gang of Four" (Gamma u. a. 2011). Dennoch ist es gerade in der Spiele-Entwicklung häufig anzutreffen.

Das Type-Object-Pattern ist sehr ähnlich zum Flyweight-Pattern. In beiden Pattern teilen sich mehrere Objekte gemeinsame Daten, die über Referenzen auf gemeinsame Hilfsobjekte eingebunden werden. Die Zielrichtung unterscheidet sich aber deutlich:

- Beim Flyweight-Pattern ist das Ziel vor allem die Erhöhung der Speichereffizienz, und die dort geteilten Daten müssen nicht unbedingt den "Typ" des nutzenden Objekts definieren.
- Beim Type-Objekt-Pattern ist das Ziel die Flexibilität auf Code-Ebene, indem man die Anzahl der Klassen minimiert und die Typen in ein eigenes Objekt-Modell verschiebt. Das Teilen von Speicher ist hier nur ein Nebeneffekt.

2.5.8.8 Wrap-Up Type-Object-Pattern: Implementierung eines eigenen Objekt-Modells

- Ziel: Minimierung der Anzahl der Klassen
- Ziel: Erhöhung der Flexibilität

- Schiebe "Typen" in ein eigenes Objekt-Modell
- Type-Objects lassen sich dynamisch über eine Konfiguration anlegen
- Objekte erhalten eine Referenz auf "ihr" Type-Object
- "Vererbung" unter den Type-Objects möglich

Zum Nachlesen

- Nystrom (2014, Kap. 13)

Lernziele

- k2: Verschieben des Typ-definierenden Teils der Eigenschaften in ein Type-Object
- k2: Erklären der Ähnlichkeit zum Flyweight-Pattern
- k3: Praktischer Einsatz des Type-Object-Patterns

Quizzes

- Quiz Type-Object-Pattern (ILIAS)

Challenges

Betrachten Sie das folgende `IMonster`-Interface:

```
1 public interface IMonster {
2     String getVariety();
3     int getXp();
4     int getMagic();
5     String makeNoise();
6 }
```

Leiten Sie von diesem Interface eine Klasse `Monster` ab. Nutzen Sie das Type-Object-Pattern und erzeugen Sie verschiedene "Klassen" von Monstern, die sich in den Eigenschaften `variety`, `xp` und `magic` unterscheiden und in der Methode `makeNoise()` entsprechend unterschiedlich verhalten. Die Eigenschaft `xp` wird dabei von jedem Monster während seiner Lebensdauer selbst verwaltet, die anderen Eigenschaften bleiben während der Lebensdauer eines Monsters konstant (ebenso wie die Methode `makeNoise()`).

1. Was wird Bestandteil des Type-Objects? Begründen Sie Ihre Antwort.
2. Implementieren Sie das Type-Object und integrieren Sie es in die Klasse `Monster`.
3. Implementieren Sie eine Factory-Methode in der Klasse für die Type-Objects, um ein neues Monster mit diesem Type-Objekt erzeugen zu können.
4. Implementieren Sie einen "Vererbungs"-Mechanismus für die Type-Objects (nicht Vererbung im Java-/OO-Sinn!). Dabei soll eine Eigenschaft überschrieben werden können.
5. Erzeugen Sie einige Monstertypen und jeweils einige Monster und lassen Sie diese ein Geräusch machen (`makeNoise()`).
6. Ersetzen Sie das Type-Object durch ein selbst definiertes (komplexes) Enum.

2.5.9 Flyweight-Pattern

TL;DR
Das Flyweight-Pattern dient der Steigerung der (Speicher-) Effizienz, indem gemeinsame Daten durch gemeinsam genutzte Objekte repräsentiert werden.
Den sogenannten *Intrinsic State*, also die Eigenschaften, die sich alle Objekte teilen, werden in gemeinsam genutzte Objekte ausgelagert, und diese werden in den ursprünglichen Klassen bzw. Objekten nur referenziert. So werden diese Eigenschaften nur einmal in den Speicher geladen.
Den sogenannten *Extrinsic State*, also alle individuellen Eigenschaften, werden entsprechend individuell je Objekt modelliert/eingestellt.

Videos

- VL Flyweight-Pattern

2.5.9.1 Motivation: Modellierung eines Levels

2.5.9.1.1 Variante I: Einsatz eines Enums für die Felder

```
1  public enum Tile { WATER, FLOOR, WALL, ... }
2
3  public class Level {
4      private Tile[][] tiles;
5
6      public Level() {
7          tiles[0][0] = Tile.WALL;  tiles[1][0] = Tile.WALL;   tiles[2][0] = Tile
               .WALL;  ...
8          tiles[0][1] = Tile.WALL;  tiles[1][1] = Tile.FLOOR;  tiles[2][1] = Tile
               .FLOOR; ...
9          tiles[0][2] = Tile.WALL;  tiles[1][2] = Tile.WATER;  tiles[2][2] = Tile
               .FLOOR; ...
10         ...
11     }
12
13     public boolean isAccessible(int x, int y) {
14         switch (tiles[x][y]) {
15             case: WATER: return false;
16             case: FLOOR: return true;
17             ...
18         }
19     }
20     ...
21 }
```

Ein Level kann als Array mit Feldern modelliert werden. Die Felder selbst könnten mit Hilfe eines Enums repräsentiert werden.

Allerdings muss dann bei jedem Zugriff auf ein Feld und dessen Eigenschaften eine entsprechende `switch`/`case`-Fallunterscheidung eingebaut werden. Damit verstreut man die Eigenschaften über die gesamte Klasse, und bei jeder Änderung am Enum für die Tiles müssen *alle* `switch`/`case`-Blöcke entsprechend angepasst werden.

2.5.9.1.2 Variante II: Einsatz einer Klasse/Klassenhierarchie für die Felder

```
1  public abstract class Tile {
2      protected boolean isAccessible;
3      protected Texture texture;
4      public boolean isAccessible() { return isAccessible; }
5  }
6  public class Floor extends Tile {
7      public Floor() { isAccessible = true;  texture = Texture.loadTexture("path/
           to/floor.png"); }
8  }
9  ...
10
11 public class Level {
12     private final Tile[][] tiles;
13
14     public Level() {
15         tiles[0][0] = new Wall();  tiles[1][0] = new Wall();   tiles[2][0] =
               new Wall();  ...
16         tiles[0][1] = new Wall();  tiles[1][1] = new Floor();  tiles[2][1] =
               new Floor(); ...
17         tiles[0][2] = new Wall();  tiles[1][2] = new Water();  tiles[2][2] =
               new Floor(); ...
18         ...
19     }
20     public boolean isAccessible(int x, int y) { return tiles[x][y].isAccessible
           (); }
21 }
```

Hier werden die Felder über eine Klassenhierarchie mit gemeinsamer Basisklasse modelliert.

Allerdings wird hier die Klassenhierarchie unter Umständen sehr schnell sehr umfangreich. Außerdem werden Eigenschaften wie Texturen beim Anlegen der Tile-Objekte immer wieder neu geladen und entsprechend mehrfach im Speicher gehalten (großer Speicherbedarf).

2.5.9.2 Flyweight: Nutze gemeinsame Eigenschaften gemeinsam Idee: Eigenschaften, die nicht an einem konkreten Objekt hängen, werden in gemeinsam genutzte Objekte ausgelagert (Shared Objects/Memory).

Ziel: Erhöhung der Speichereffizienz (geringerer Bedarf an Hauptspeicher, geringere Bandbreite bei der Übertragung der Daten/Objekt an die GPU, …).

2.5.9.2.1 Lösungsvorschlag

```
1  public final class Tile {
2      private final boolean isAccessible;
3      private final Texture texture;
4      public boolean isAccessible() { return isAccessible; }
5  }
6
7  public class Level {
8      private static final Tile FLOOR = new Tile(true,  Texture.loadTexture("path
           /to/floor.png"));
9      private static final Tile WALL  = new Tile(false, Texture.loadTexture("path
           /to/wall.png"));
10     private static final Tile WATER = new Tile(false, Texture.loadTexture("path
           /to/water.png"));
```

```
11
12      private final Tile[][] tiles;
13
14      public Level() {
15          tiles[0][0] = WALL;  tiles[1][0] = WALL;   tiles[2][0] = WALL;  ...
16          tiles[0][1] = WALL;  tiles[1][1] = FLOOR;  tiles[2][1] = FLOOR; ...
17          tiles[0][2] = WALL;  tiles[1][2] = WATER;  tiles[2][2] = FLOOR; ...
18          ...
19      }
20      public boolean isAccessible(int x, int y) { return tiles[x][y].isAccessible
            (); }
21  }
```

Man legt die verschiedenen Tiles nur je *einmal* an und nutzt dann Referenzen auf diese Objekte. Dadurch werden die speicherintensiven Elemente wie Texturen o.ä. nur je einmal geladen und im Speicher vorgehalten.

Bei dieser Modellierung können die einzelnen Felder aber keine individuellen Eigenschaften haben, wie etwa, ob ein Feld bereits durch den Helden untersucht/betreten wurde o.ä. …

2.5.9.2.2 Lösungsvorschlag II

```
1   public final class TileModel {
2       private final boolean isAccessible;
3       private final Texture texture;
4       public boolean isAccessible() { return isAccessible; }
5   }
6   public final class Tile {
7       private boolean wasEntered;
8       private final TileModel model;
9       public boolean isAccessible() { return model.isAccessible(); }
10      public boolean wasEntered() { return wasEntered; }
11  }
12
13  public class Level {
14      private static final TileModel FLOOR = new TileModel(true,  Texture.
            loadTexture("path/to/floor.png"));
15      ...
16
17      private final Tile[][] tiles;
18
19      public Level() {
20          tiles[0][0] = new Tile(WALL);  tiles[1][0] = new Tile(WALL);   tiles
                [2][0] = new Tile(WALL);  ...
21          tiles[0][1] = new Tile(WALL);  tiles[1][1] = new Tile(FLOOR);  tiles
                [2][1] = new Tile(FLOOR); ...
22          tiles[0][2] = new Tile(WALL);  tiles[1][2] = new Tile(WATER);  tiles
                [2][2] = new Tile(FLOOR); ...
23          ...
24      }
25      public boolean isAccessible(int x, int y) { return tiles[x][y].isAccessible
            (); }
26  }
```

In dieser Variante werden die Eigenschaften eines `Tile` in Eigenschaften aufgeteilt, die von den Tiles geteilt werden können (im Beispiel Textur und Betretbarkeit) und in Eigenschaften, die je Feld individuell modelliert werden müssen (im Beispiel: wurde das Feld bereits betreten?).

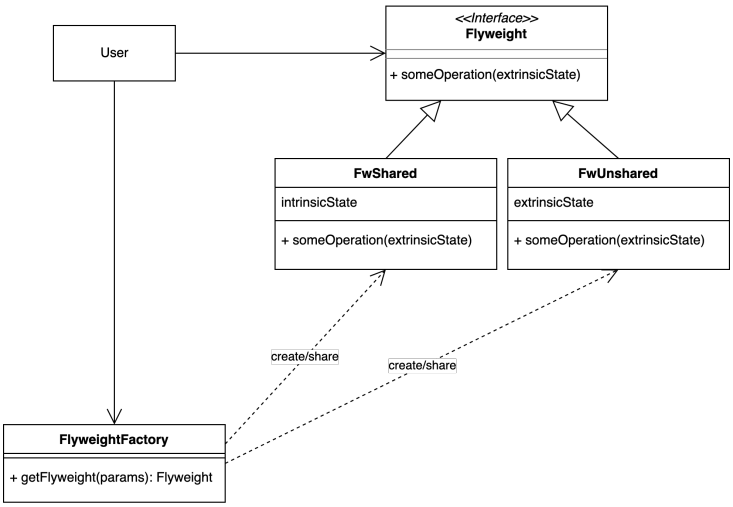
Entsprechend könnte man für das Level-Beispiel ein `TileModel` anlegen, welches die gemeinsamen Eigenschaften verwaltet. Man erzeugt dann im Level die nötigen Modelle je genau einmal und nutzt sie, um damit dann die konkreten Felder zu erzeugen und im Level-Array zu referenzieren. Damit werden Tile-Modelle von Tiles der gleichen "Klasse" gemeinsam genutzt und die Texturen u.ä. nur je einmal im Speicher repräsentiert.

2.5.9.3 Flyweight-Pattern: Begriffe

- **Intrinsic** State: invariant, Kontext-unabhängig, gemeinsam nutzbar
  => auslagern in gemeinsame Objekte

- **Extrinsic** State: variant, Kontext-abhängig und kann nicht geteilt werden
  => individuell modellieren

2.5.9.4 Flyweight-Pattern: Klassische Modellierung



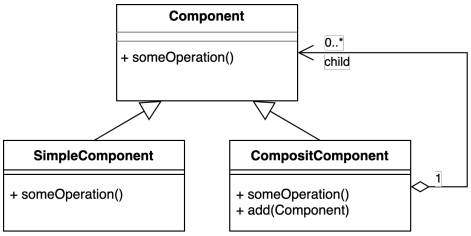
Im klassischen Flyweight-Pattern der "Gang of Four" (Gamma u. a. 2011) wird ein gemeinsames Interface erstellt, von dem die einzelnen Fliegengewicht-Klassen ableiten. Der Nutzer kennt nur dieses Interface und nicht direkt die implementierenden Klassen.

Das Interface wird von zwei Arten von Klassen implementiert: Klassen, die nur intrinsischen Zustand modellieren, und Klassen, die extrinsischen Zustand modellieren.

Für die Klassen, die den intrinsischen Zustand modellieren, werden die Objekte gemeinsam genutzt (nicht im Diagramm darstellbar) und deshalb eine Factory davor geschaltet, die die Objekte der entsprechenden Fliegengewicht-Klassen erzeugt und dabei darauf achtet, dass diese Objekte nur einmal angelegt und bei erneuter Anfrage einfach nur wieder zurückgeliefert werden.

Zusätzlich gibt es Klassen, die extrinsischen Zustand modellieren und deshalb nicht unter den Nutzern geteilt werden können und deren Objekte bei jeder Anfrage neu erstellt werden. Aber auch diese werden von der Factory erzeugt/verwaltet.

2.5.9.4.1 Kombination mit dem Composite-Pattern In der Praxis kann man das Pattern so direkt meist nicht einsetzen, sondern verbindet es mit dem Composite-Pattern:



Ein Element kann eine einfache Komponente sein (im obigen Beispiel war das die Klasse `TileModel`) oder eine zusammengesetzte Komponente, die ihrerseits andere Komponenten speichert (im obigen Beispiel war das die Klasse `Tile`, die ein Objekt vom Typ `TileModel` referenziert - allerdings fehlt im obigen Beispiel das gemeinsame Interface …).

2.5.9.4.2 Level-Beispiel mit Flyweight (vollständig) und Composite Im obigen Beispiel wurde zum Flyweight-Pattern noch das Composite-Pattern hinzugenommen, aber es wurde aus Gründen der Übersichtlichkeit auf ein gemeinsames Interface und auf die Factory verzichtet. Wenn man es anpassen würde, dann würde das Beispiel ungefähr so aussehen:

```
1  public interface ITile {
2      public boolean isAccessible();
3  }
4
5  public final class TileModel implements ITile {
6      private final boolean isAccessible;
7      private final Texture texture;
8
9      public boolean isAccessible() { return isAccessible; }
10 }
11
12 public final class Tile implements ITile {
13     private boolean wasEntered;
14     private final TileModel model;
15
16     public boolean isAccessible() { return model.isAccessible(); }
17
18     public boolean wasEntered() { return wasEntered; }
19 }
20
21 public final class TileFactory {
22     private static final TileModel FLOOR = new TileModel(true,  Texture.
           loadTexture("path/to/floor.png"));
23     ...
24
25     public static final ITile getTile(String tile) {
26         switch (tile) {
27             case "WALL": return new Tile(WALL);
28             case "FLOOR": return new Tile(FLOOR);
29             case "WATER": return new Tile(WATER);
```

```
30              ...
31          }
32      }
33  }
34
35  public class Level {
36      private ITile[][] tiles;
37
38      public Level() {
39          tiles[0][0] = TileFactory.getTile("WALL");
40          tiles[1][0] = TileFactory.getTile("WALL");
41          tiles[2][0] = TileFactory.getTile("WALL");
42          ...
43
44          tiles[0][1] = TileFactory.getTile("WALL");
45          tiles[1][1] = TileFactory.getTile("FLOOR");
46          tiles[2][1] = TileFactory.getTile("FLOOR");
47          ...
48
49          tiles[0][2] = TileFactory.getTile("WALL");
50          tiles[1][2] = TileFactory.getTile("WATER");
51          tiles[2][2] = TileFactory.getTile("FLOOR");
52          ...
53
54          ...
55      }
56
57      public boolean isAccessible(int x, int y) { return tiles[x][y].isAccessible
            (); }
58  }
```

2.5.9.5 Verwandtschaft zum Type-Object-Pattern Das Flyweight-Pattern ist sehr ähnlich zum Type-Object-Pattern. In beiden Pattern teilen sich mehrere Objekte gemeinsame Daten, die über Referenzen auf gemeinsame Hilfsobjekte eingebunden werden. Die Zielrichtung unterscheidet sich aber deutlich:

- Beim Flyweight-Pattern ist das Ziel vor allem die Erhöhung der Speichereffizienz, und die dort geteilten Daten müssen nicht unbedingt den “Typ” des nutzenden Objekts definieren.
- Beim Type-Objekt-Pattern ist das Ziel die Flexibilität auf Code-Ebene, indem man die Anzahl der Klassen minimiert und die Typen in ein eigenes Objekt-Modell verschiebt. Das Teilen von Speicher ist hier nur ein Nebeneffekt.

2.5.9.6 Wrap-Up Flyweight-Pattern: Steigerung der (Speicher-) Effizienz durch gemeinsame Nutzung von Objekten

- Lagere *Intrinsic State* in gemeinsam genutzte Objekte aus
- Modelliere *Extrinsic State* individuell

Zum Nachlesen

- Nystrom (2014, Kap. 3)

Lernziele

- k2: Unterscheiden von Intrinsic State und Extrinsic State
- k2: Verschieben des Intrinsic States in gemeinsam genutzte Objekte
- k2: Erklären der Ähnlichkeit zum Type-Object-Pattern
- k3: Praktischer Einsatz des Flyweight-Patterns

Quizzes

- Quiz Flyweight-Pattern (ILIAS)

Challenges

In den Vorgaben finden Sie ein Modellierung eines Schachspiels.
Identifizieren Sie die Stellen im Vorgabe-Code, wo Sie das Flyweight-Pattern sinnvoll anwenden können und bauen Sie dieses Pattern über ein Refactoring ein. Begründen Sie, wie Sie das Pattern eingesetzt haben und warum Sie welche Elemente *immutable* oder *mutable* deklariert haben.
Wieso eignet sich das Flyweight-Pattern besonders im Bereich von Computerspielen? Geben Sie mögliche Vor- und Nachteile an und begründen Sie Ihre Antwort.

2.6 Testen mit JUnit und Mockito

2.6.1 Einführung Softwaretest

TL;DR

Fehler schleichen sich durch Zeitdruck und hohe Komplexität schnell in ein Softwareprodukt ein. Die Folgen können von “ärgerlich” über “teuer” bis hin zu (potentiell) “tödlich” reichen. Richtiges Testen ist also ein wichtiger Aspekt bei der Softwareentwicklung!
JUnit ist ein Java-Framework, mit dem Unit-Tests (aber auch andere Teststufen) implementiert werden können. In JUnit 4 und 5 zeichnet man eine Testmethode mit Hilfe der Annotation `@Test` an der entsprechenden Methode aus. Dadurch kann man Produktiv- und Test-Code prinzipiell mischen; Best Practice ist aber das Anlegen eines weiteren Ordners `test/` und das Spiegeln der Package-Strukturen. Für die zu testende Klasse wird eine korrespondierende Testklasse mit dem Suffix “Test” (Konvention) angelegt und dort die Testmethoden implementiert. Der IDE muss der neue `test/`-Ordner noch als Ordner für Sourcen bzw. Tests bekannt gemacht werden. In den Testmethoden baut man den Test auf, führt schließlich den Testschritt durch (beispielsweise konkreter Aufruf der zu testenden Methode) und prüft anschließend mit einem `assert*()`, ob das erzielte Ergebnis dem erwarteten Ergebnis entspricht. Ist alles OK, ist der Test “grün”, sonst “rot”.
Da ein fehlschlagendes `assert*()` den Test abbricht, werden eventuell danach folgende Prüfungen **nicht** mehr durchgeführt und damit ggf. weitere Fehler maskiert. Deshalb ist es gute Praxis, in einer Testmethode nur einen Testfall zu implementieren und i.d.R. auch nur ein (oder wenige) Aufrufe von `assert*()` pro Testmethode zu haben.

Videos

- VL Einführung Softwaretest
- Demo Anlegen von Testfällen mit JUnit

2.6.1.1 Software-Fehler und ihre Folgen

1982: Absturz eines F117 Kampfjets: in der Softwaresteuerung Höhen- und Seitenruder vertauscht
http://de.wikipedia.org/wiki/Programmfehler

2005: TOKIOTER BÖRSE: Chef tritt ab wg. Softwarefehler (mehrere 100 Millionen Dollar Verlust)
http://www.manager-magazin.de/unternehmen/karriere/0,2828,druck-391434,00.html

2006: VW ruft 3500 Passat-Modelle zurück. Ein Softwarefehler kann zum Absterben des Motors führen.
http://auto.t-online.de/volkswagen-rueckruf-fuer-den-passat-wegen-softwarefehler/id_12821896/index

2007: Defektes Computersystem für den Tod von zehn Soldaten verantwortlich
http://www.heise.de/newsticker/meldung/Defektes-Computersystem-fuer-den-Tod-von-zehn-Soldaten-verantwortlich-186999.html?view=print

2008: Volvo-Rückruf: XC90 mit Software-Problem (Zündung vs. Klimaanlage)
http://www.auto-motor-und-sport.de/news/volvo-rueckruf-xc90-mit-software-problem-711983.html

2010: Softwarefehler in EC-Karten-Sicherheitschip: Kunden bekommen kein Geld am Automaten
http://www.tagesspiegel.de/wirtschaft/ec-karten-panne-zum-jahreswechsel/1658102.html

2010: Rückrufe kosten Toyota mehr als eine Milliarde Euro (Softwareprobleme)
http://www.spiegel.de/wirtschaft/unternehmen/0,1518,druck-675874,00.html

2010: Ford: Rückrufe wg. Softwareproblemen im Bremssystem
http://www.automobil-produktion.de/2010/02/ruckruf-jetzt-auch-ford/

2011: Honda startet umfangreiche Rückrufaktion, weltweit etwa 2,5 Millionen Autos betroffen (Softwareprobleme können zu Schäden am Getriebe führen)
http://www.handelsblatt.com/unternehmen/industrie/softwareprobleme-honda-startet-umfangreiche-rueckrufaktion/v_detail_tab_print/4470752.html

2012: Report "Softwareentwicklung 2012" (Accso): "Softwarefehler kosten die deutsche Volkswirtschaft Milliarden"
http://www.elektronikpraxis.vogel.de/index.cfm?pid=5416&pk=361330&print=true&printtype=article

2.6.1.2 (Einige) Ursachen für Fehler

- Zeit- und Kostendruck
- Mangelhafte Anforderungsanalyse
- Hohe Komplexität
- Mangelhafte Kommunikation
- Keine/schlechte Teststrategie
- Mangelhafte Beherrschung der Technologie
- …

2.6.1.3 Irgendjemand muss mit Deinen Bugs leben! Leider gibt es im Allgemeinen keinen Weg zu zeigen, dass eine Software korrekt ist. Man kann (neben formalen Beweisansätzen) eine Software nur unter möglichst vielen Bedingungen ausprobieren, um zu schauen, wie sie sich verhält, und um die dabei zu Tage tretenden Bugs zu fixen.

Mal abgesehen von der verbesserten *User-Experience* führt weniger fehlerbehaftete Software auch dazu, dass man seltener mitten in der Nacht geweckt wird, weil irgendwo wieder ein Server gecrasht ist … Weniger fehlerbehaftete Software ist auch leichter zu ändern und zu pflegen! In realen Projekten macht Maintenance den größten Teil an der Softwareentwicklung aus … Während Ihre Praktikumsprojekte vermutlich nach der Abgabe

nie wieder angeschaut werden, können echte Projekte viele Jahre bis Jahrzehnte leben! D.h. irgendwer muss sich dann mit Ihren Bugs herumärgern - vermutlich sogar Sie selbst ;)

> Always code as if the guy who ends up maintaining your code will be a violent psychopath who knows where you live. Code for readability.
>
> – John F. Woods

Dieses Zitat taucht immer mal wieder auf, beispielsweise auf der OSCON 2014 … Es scheint aber tatsächlich, dass John F. Woods die ursprüngliche Quelle war (vgl. Stackoverflow: 876089).

Da wir nur wenig Zeit haben und zudem vergesslich sind und obendrein die Komplexität eines Projekts mit der Anzahl der Code-Zeilen i.d.R. nicht-linear ansteigt, müssen wir das Testen automatisieren. Und hier kommt JUnit ins Spiel :)

2.6.1.4 Was wann testen? Wichtigste Teststufen

- **Modultest**
  - Testen einer Klasse und ihrer Methoden
  - Test auf gewünschtes Verhalten (Parameter, Schleifen, …)
- **Integrationstest**
  - Test des korrekten Zusammenspiels mehrerer Komponenten
  - Konzentration auf Schnittstellentests
- **Systemtest**
  - Test des kompletten Systems unter produktiven Bedingungen
  - Orientiert sich an den aufgestellten Use Cases
  - Funktionale und nichtfunktionale Anforderungen testen

=> Verweis auf Wahlfach “Softwarequalität”

2.6.1.5 JUnit: Test-Framework für Java

JUnit — Open Source Java Test-Framework zur Erstellung und Durchführung wiederholbarer Tests

- JUnit 3
  - Tests müssen in eigenen Testklassen stehen
  - Testklassen müssen von Klasse `TestCase` erben
  - Testmethoden müssen mit dem Präfix "`test`" beginnen

- **JUnit 4**
  - Annotation `@Test` für Testmethoden
  - Kein Zwang zu spezialisierten Testklassen (insbesondere kein Zwang mehr zur Ableitung von `TestCase`)
  - Freie Namenswahl für Testmethoden (benötigen nicht mehr Präfix "`test`")

  Damit können prinzipiell auch direkt im Source-Code Methoden als JUnit-Testmethoden ausgezeichnet werden … (das empfiehlt sich in der Regel aber nicht)

- *JUnit 5 = JUnit Platform + JUnit Jupiter + JUnit Vintage*
  - Erweiterung um mächtigere Annotationen
  - Aufteilung in spezialisierte Teilprojekte

  Das Teilprojekt "JUnit Platform" ist die Grundlage für das JUnit-Framework. Es bietet u.a. einen Console-Launcher, um Testsuiten manuell in der Konsole zu starten oder über Builder wie Ant oder Gradle.
  Das Teilprojekt "JUnit Jupiter" ist das neue Programmiermodell zum Schreiben von Tests in JUnit 5. Es beinhaltet eine TestEngine zum Ausführen der in Jupiter geschriebenen Tests.
  Das Teilprojekt "JUnit Vintage" beinhaltet eine TestEngine zum Ausführen von Tests, die in JUnit 3 oder JUnit 4 geschrieben sind.

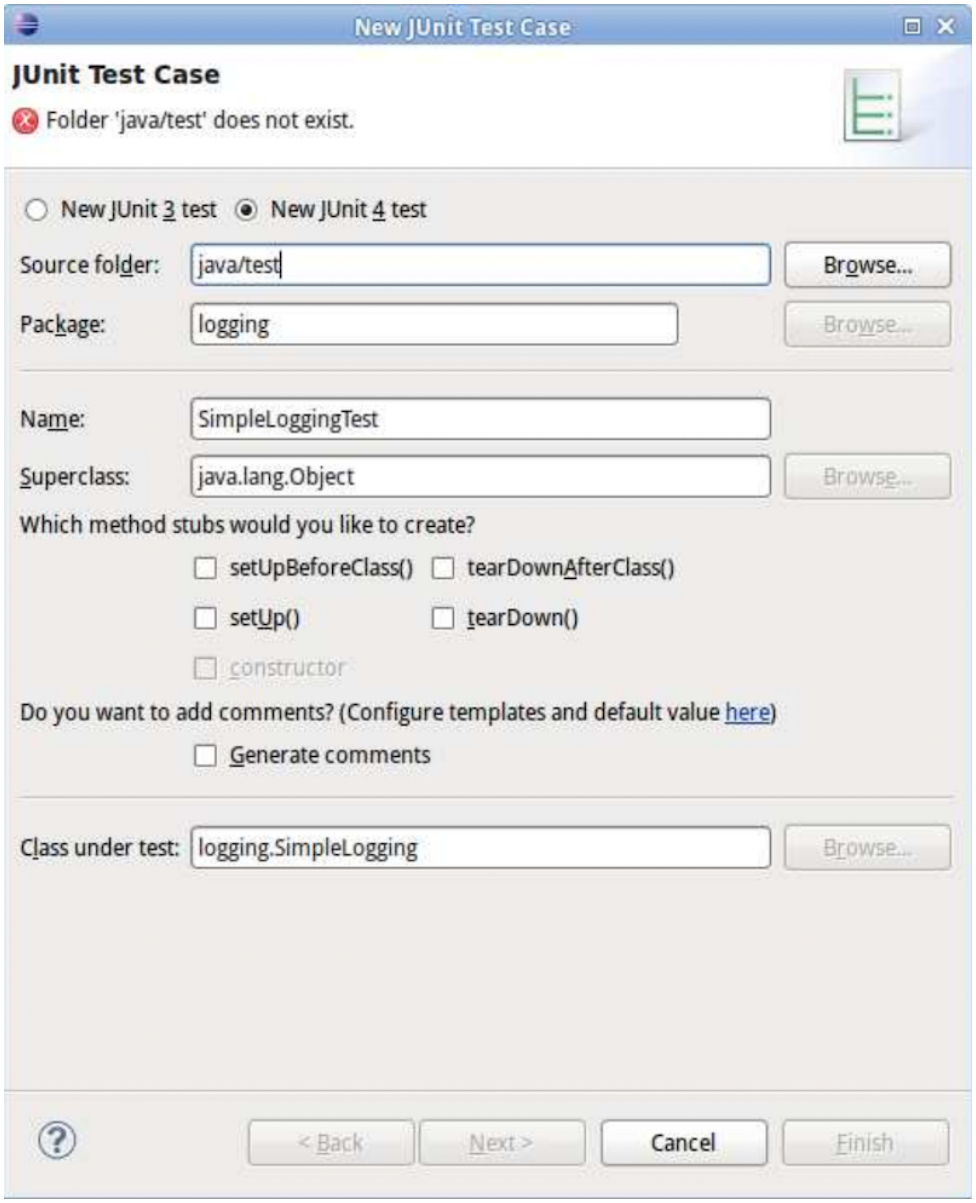
Anmerkung: Wie der Name schon sagt, ist das Framework für Modultests ("Unit-Tests") gedacht. Man kann damit aber auch auf anderen Teststufen arbeiten!

Anmerkung: Im Folgenden besprechen wir JUnit am Beispiel **JUnit 4**, da diese Version des Frameworks besonders stark verbreitet ist und JUnit 5 (trotz offiziellem Release) immer noch stellenweise unfertig wirkt. Auf Unterschiede zu JUnit 5 wird an geeigneter Stelle hingewiesen (abgesehen von Import-Statements). Mit JUnit 3 sollte nicht mehr aktiv gearbeitet werden, d.h. insbesondere keine neuen Tests mehr erstellt werden, da diese Version nicht mehr weiterentwickelt wird.

2.6.1.6 Anlegen und Organisation der Tests mit JUnit

- Anlegen neuer Tests: Klasse auswählen, Kontextmenü `New` > `JUnit Test Case`

- **Best Practice**:
  Spiegeln der Paket-Hierarchie
  - Toplevel-Ordner `test` (statt `src`)
  - Package-Strukturen spiegeln
  - Testklassen mit Suffix "`Test`"



Vorteile dieses Vorgehens:

- Die Testklassen sind aus Java-Sicht im selben Package wie die Source-Klassen, d.h. Zugriff auf Package-sichtbare Methoden etc. ist gewährleistet
- Durch die Spiegelung der Packages in einem separaten Testordner erhält man eine gute getrennte Übersicht über jeweils die Tests und die Sourcen

- Die Wiederverwendung des Klassennamens mit dem Anhang “Test” erlaubt die schnelle Erkennung, welche Tests hier vorliegen

In der Paketansicht liegen dann die Source- und die Testklassen immer direkt hintereinander (da sie im selben Paket sind und mit dem selben Namen anfangen) => besserer Überblick!

2.6.1.7 Anmerkung: Die (richtige) JUnit-Bibliothek muss im Classpath liegen!

Eclipse bringt für JUnit 4 und JUnit 5 die nötigen Jar-Dateien mit und fragt beim erstmaligen Anlegen einer neuen Testklasse, ob die für die ausgewählte Version **passenden JUnit-Jars zum Build-Path hinzugefügt** werden sollen.

IntelliJ bringt ebenfalls eine JUnit 4 Bibliothek mit, die zum Projekt als Abhängigkeit hinzugefügt werden muss. Für JUnit 5 bietet IntelliJ an, die Jar-Dateien herunterzuladen und in einem passenden Ordner abzulegen.

Alternativ lädt man die Bibliotheken entsprechend der Anleitung unter junit.org herunter und bindet sie in das Projekt ein.

2.6.1.8 JUnit 4+5: Definition von Tests

Annotation `@Test` vor Testmethode schreiben

```java
import org.junit.Test;
import static org.junit.Assert.*;

public class FactoryBeispielTest4 {
    @Test
    public void testGetTicket() {
        fail("not implemented");
    }
}
```

Für JUnit 5 muss statt `org.junit.Test` entsprechend `org.junit.jupiter.api.Test` importiert werden.

Während in JUnit 4 die Testmethoden mit der Sichtbarkeit **`public`** versehen sein müssen und keine Parameter haben (dürfen), spielt die Sichtbarkeit in JUnit 5 keine Rolle (und die Testmethoden dürfen Parameter aufweisen => vgl. Abschnitt “Dependency Injection for Constructors and Methods” in der JUnit-Doku).

2.6.1.9 JUnit 4: Ergebnis prüfen

Klasse **`org.junit.Assert`** enthält diverse **statische** Methoden zum Prüfen:

```java
// Argument muss true bzw. false sein
void assertTrue(boolean);
void assertFalse(boolean);

// Gleichheit im Sinne von equals()
void assertEquals(Object, Object);

// Test sofort fehlschlagen lassen
void fail();

...
```

Für JUnit 5 finden sich die Assert-Methoden im Package `org.junit.jupiter.api.Assertions`.

2.6.1.10 Anmerkung zum statischen Import

Bei normalem Import der Klasse `Assert` muss man jeweils den voll qualifizierten Namen einer statischen Methode nutzen: `Assert.fail()`.

Alternative statischer Import: `import static org.junit.Assert.fail;` => *Statische Member* der importierten Klasse (oder Interface) werden über ihre *unqualifizierten Namen* zugreifbar. Achtung: Namenskollisionen möglich!

```
1 // nur bestimmtes Member importieren
2 import static packageName.className.staticMemberName;
3 // alle statischen Member importieren
4 import static packageName.className.*;
```

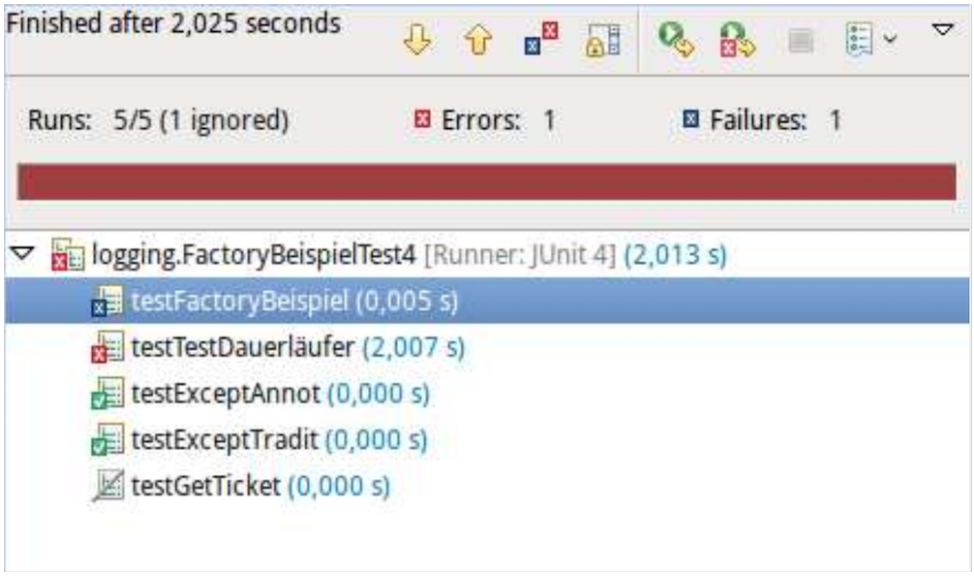
- Beispiel normaler Import:

```
1 import org.junit.Assert;
2 Assert.fail("message");
```

- Beispiel statischer Import:

```
1 import static org.junit.Assert.fail;
2 fail("message");
```

2.6.1.11 Mögliche Testausgänge bei JUnit



1. **Error**: Fehler im Programm (Test)
   - Unbehandelte Exception
   - Abbruch (Timeout)

2. **Failure**: Testausgang negativ
    - Assert fehlgeschlagen
    - `Assert.fail()` aufgerufen

3. **OK**

2.6.1.12 Anmerkungen zu Assert

- Pro Testmethode möglichst nur **ein** Assert verwenden!
- Anderenfalls: Schlägt ein Assert fehl, wird der Rest nicht mehr überprüft …

2.6.1.13 Wrap-Up

- Testen ist genauso wichtig wie Coden
- Richtiges Testen spart Geld, Zeit, …
- Tests auf verschiedenen Abstraktionsstufen

- JUnit als Framework für (Unit-) Tests; hier JUnit 4 (mit Ausblick auf JUnit 5)
    - Testmethoden mit Annotation `@Test`
    - Testergebnis mit `assert*` prüfen

Zum Nachlesen

- vogella GmbH (2021)
- The JUnit Team (2022)
- Kleuker (2019)
- Osherove (2014)
- Spillner und Linz (2012)
- Thies u. a. (o. J.)

Lernziele

- k2: Ursachen von Softwarefehlern
- k3: Aufbauen von Tests mit JUnit 4 und 5 unter Nutzung der Annotation `@Test`

Quizzes

- Quiz JUnit-Basics - nur erster Teil (ILIAS)

2.6.2 Testen mit JUnit (JUnit-Basics)

TL;DR
In JUnit 4 und 5 werden Testmethoden mit Hilfe der Annotation `@Test` ausgezeichnet. Über die verschiedenen `assert*()`-Methoden kann das Testergebnis mit dem erwarteten Ergebnis verglichen werden und entsprechend ist der Test "grün" oder "rot". Mit den verschiedenen `assume*()`-Methoden kann dagegen geprüft werden, ob eventuelle Vorbedingungen für das Ausführen eines Testfalls erfüllt sind - anderenfalls wird der Testfall dann übersprungen.
Mit Hilfe von `@Before` und `@After` können Methoden gekennzeichnet werden, die jeweils vor jeder Testmethode und nach jeder Testmethode aufgerufen werden. Damit kann man seine Testumgebung auf- und auch wieder abbauen (JUnit 4).
Erwartete Exceptions lassen sich in JUnit 4 mit einem Parameter `expected` in der Annotation `@Test` automatisch prüfen: `@Test(expected=package.Exception.class)`. In JUnit 4 besteht die Möglichkeit, Testklassen zu Testsuiten zusammenzufassen und gemeinsam laufen zu lassen.

Videos

- VL JUnit Basics
- Demo assume() vs. assert()
- Demo Parametrisierte Tests mit JUnit4
- Demo Parametrisierte Tests mit JUnit5

2.6.2.1 JUnit: Ergebnis prüfen

Klasse `org.junit.Assert` enthält diverse **statische** Methoden zum Prüfen:

```
1  // Argument muss true bzw. false sein
2  void assertTrue(boolean);
3  void assertFalse(boolean);
4
5  // Gleichheit im Sinne von equals()
6  void assertEquals(Object, Object);
7
8  // Test sofort fehlschlagen lassen
9  void fail();
10
11 ...
```

2.6.2.2 To "assert" or to "assume"?

- Mit `assert*` werden Testergebnisse geprüft
  - Test wird ausgeführt
  - Ergebnis: OK, Failure, Error
- Mit `assume*` werden Annahmen über den Zustand geprüft
  - Test wird abgebrochen, wenn Annahme nicht erfüllt

- Prüfen von Vorbedingungen: Ist der Test hier ausführbar/anwendbar?

Beispiel: junit4.TestAssume

2.6.2.3 Setup und Teardown: Testübergreifende Konfiguration

```java
private Studi x;

@Before
public void setUp() { x = new Studi(); }

@Test
public void testToString() {
    // Studi x = new Studi();
    assertEquals(x.toString(), "Heinz (15cps)");
}
```

`@Before` wird **vor jeder** Testmethode aufgerufen
`@BeforeClass` wird **einmalig** vor allen Tests aufgerufen (**`static`**!)
`@After` wird **nach jeder** Testmethode aufgerufen
`@AfterClass` wird **einmalig** nach allen Tests aufgerufen (**`static`**!)

In JUnit 5 wurden die Namen dieser Annotationen leicht geändert:

JUnit 4	JUnit 5
`@Before`	`@BeforeEach`
`@After`	`@AfterEach`
`@BeforeClass`	`@BeforeAll`
`@AfterClass`	`@AfterAll`

2.6.2.4 Beispiel für den Einsatz von `@Before`

Annahme: **alle/viele** Testmethoden brauchen **neues** Objekt `x` vom Typ `Studi`

```java
private Studi x;

@Before
public void setUp() {
    x = new Studi("Heinz", 15);
}

@Test
public void testToString() {
    // Studi x = new Studi("Heinz", 15);
    assertEquals(x.toString(), "Name: Heinz, credits: 15");
}

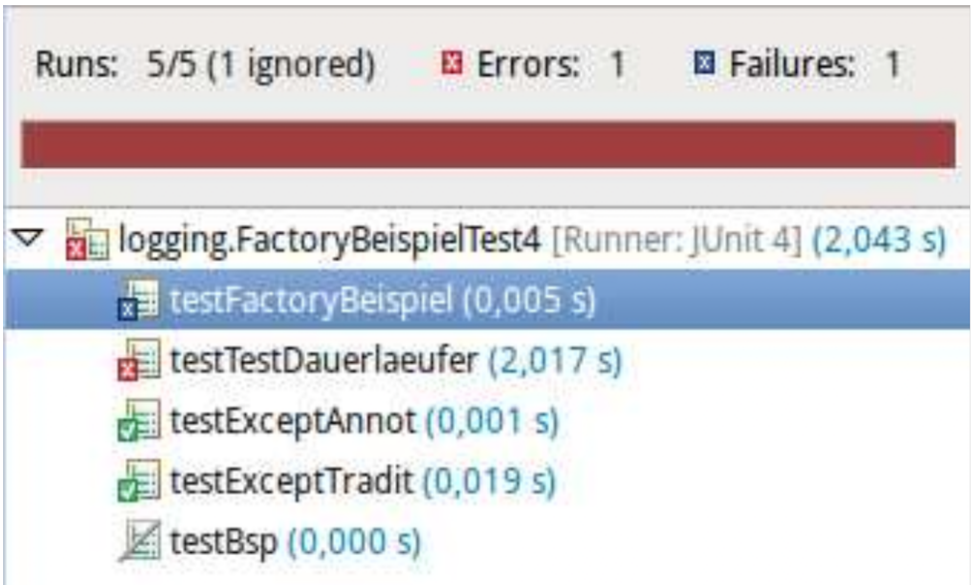
@Test
public void testGetName() {
    // Studi x = new Studi("Heinz", 15);
```

```
17      assertEquals(x.getName(), "Heinz");
18  }
```

2.6.2.5 Ignorieren von Tests

- Hinzufügen der Annotation `@Ignore`
- Alternativ mit Kommentar: `@Ignore("Erst im nächsten Release")`

```
1  @Ignore("Warum ignoriert")
2  @Test
3  public void testBsp() {
4      Bsp x = new Bsp();
5      assertTrue(x.isTrue());
6  }
```

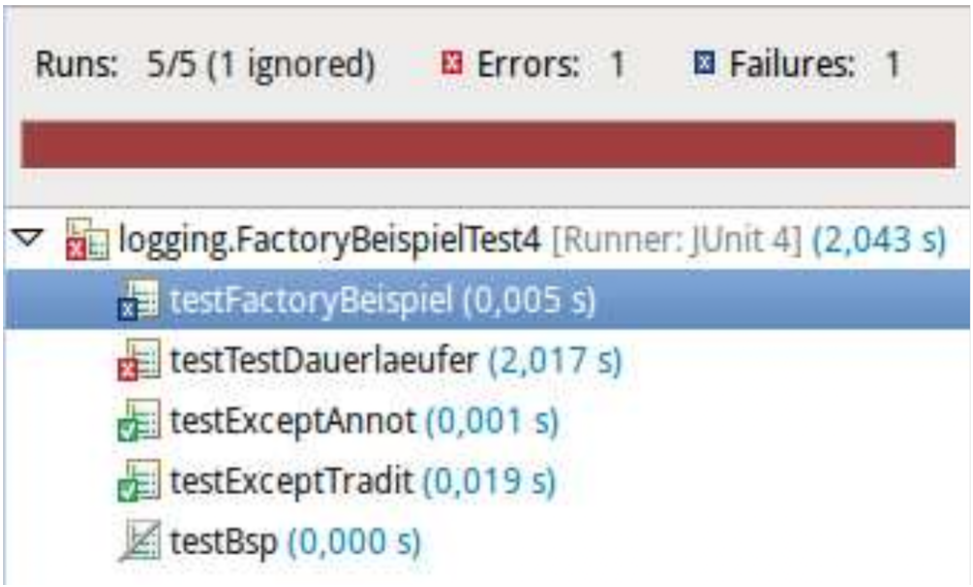


In JUnit 5 wird statt der Annotation `@Ignore` die Annotation `@Disabled` mit der selben Bedeutung verwendet. Auch hier lässt sich als Parameter ein String mit dem Grund für das Ignorieren des Tests hinterlegen.

2.6.2.6 Vermeidung von Endlosschleifen: Timeout

- Testfälle werden nacheinander ausgeführt
- Test mit Endlosschleife würde restliche Tests blockieren
- Erweitern der `@Test`-Annotation mit Parameter "`timeout`":
  => `@Test(timeout=2000)` (Zeitangabe in Millisekunden)

```
1  @Test(timeout = 2000)
2  void testTestDauerlaeufer() {
3      while (true) { ; }
4  }
```



In JUnit 5 hat die Annotation `@Test` keinen `timeout`-Parameter mehr. Als Alternative bietet sich der Einsatz von `org.junit.jupiter.api.Assertions.assertTimeout` an. Dabei benötigt man allerdings *Lambda-Ausdrücke* (Verweis auf spätere VL):

```
@Test
void testTestDauerlaeufer() {
    assertTimeout(ofMillis(2000), () -> {
        while (true) { ; }
    });
}
```

(Beispiel von oben mit Hilfe von JUnit 5 formuliert)

2.6.2.7 Test von Exceptions: Expected

Traditionelles Testen von Exceptions mit **`try`** und **`catch`**:

```
@Test
public void testExceptTradit() {
    try {
        int i = 0 / 0;
        fail("keine ArithmeticException ausgeloest");
    } catch (ArithmeticException aex) {
        assertNotNull(aex.getMessage());
    } catch (Exception e) {
        fail("falsche Exception geworfen");
    }
}
```

Der `expected`-Parameter für die `@Test`-Annotation in JUnit 4 macht dies deutlich einfacher: `@Test(expected = MyException.class)` => Test scheitert, wenn diese Exception **nicht** geworfen wird

```
@Test(expected = java.lang.ArithmeticException.class)
public void testExceptAnnot() {
```

```
3       int i = 0 / 0;
4   }
```

In JUnit 5 hat die Annotation `@Test` keinen `expected`-Parameter mehr. Als Alternative bietet sich der Einsatz von `org.junit.jupiter.api.Assertions.assertThrows` an. Dabei benötigt man allerdings *Lambda-Ausdrücke* (Verweis auf spätere VL):

```
1   @Test
2   public void testExceptAnnot() {
3       assertThrows(java.lang.ArithmeticException.class, () -> {
4           int i = 0 / 0;
5       });
6   }
```

(Beispiel von oben mit Hilfe von JUnit 5 formuliert)

2.6.2.8 Parametrisierte Tests Manchmal möchte man den selben Testfall mehrfach mit anderen Werten (Parametern) durchführen.

```
1   class Sum {
2       public int sum(int i, int j) {
3           return i + j;
4       }
5   }
6
7   class SumTest {
8       @Test
9       public void testSum() {
10          Sum s = new Sum();
11          assertEquals(s.sum(1, 1), 2);
12      }
13      // und mit (2,2, 4), (2,2, 5), ...????
14  }
```

Prinzipiell könnte man dafür entweder in einem Testfall eine Schleife schreiben, die über die verschiedenen Parameter iteriert. In der Schleife würde dann jeweils der Aufruf der zu testenden Methode und das gewünschte Assert passieren. Alternativ könnte man den Testfall entsprechend oft duplizieren mit jeweils den gewünschten Werten.

Beide Vorgehensweisen haben Probleme: Im ersten Fall würde die Schleife bei einem Fehler oder unerwarteten Ergebnis abbrechen, ohne dass die restlichen Tests (Werte) noch durchgeführt würden. Im zweiten Fall bekommt man eine unnötig große Anzahl an Testmethoden, die bis auf die jeweiligen Werte identisch sind (Code-Duplizierung).

2.6.2.8.1 Parametrisierte Tests mit JUnit 4 JUnit 4 bietet für dieses Problem sogenannte “parametrisierte Tests” an. Dafür muss eine Testklasse in JUnit 4 folgende Bedingungen erfüllen:

1. Die Testklasse wird mit der Annotation `@RunWith(Parameterized.class)` ausgezeichnet.
2. Es muss eine öffentliche statische Methode geben mit der Annotation `@Parameters`. Diese Methode liefert eine Collection zurück, wobei jedes Element dieser Collection ein Array mit den Parametern für einen Durchlauf der Testmethoden ist.

3. Die Parameter müssen gesetzt werden. Dafür gibt es zwei Varianten:
   a) Für jeden Parameter gibt es ein öffentliches Attribut. Diese Attribute müssen mit der Annotation `@Parameter` markiert sein und können in den Testmethoden normal genutzt werden. JUnit sorgt dafür, dass für jeden Eintrag in der Collection aus der statischen `@Parameters`-Methode diese Felder gesetzt werden und die Testmethoden aufgerufen werden.
   b) Alternativ gibt es einen Konstruktor, der diese Werte setzt. Die Anzahl der Parameter im Konstruktor muss dabei exakt der Anzahl (und Reihenfolge) der Werte in jedem Array in der von der statischen `@Parameters`-Methode gelieferten Collection entsprechen. Der Konstruktor wird für jeden Parametersatz einmal aufgerufen und die Testmethoden einmal durchgeführt.

Letztlich wird damit das Kreuzprodukt aus Testmethoden und Testdaten durchgeführt.

Beispiel: junit4.SumTestConstructor, junit4.SumTestParameters

2.6.2.8.2 Parametrisierte Tests mit JUnit 5

In JUnit 5 werden parametrisierte Tests mit der Annotation `@ParameterizedTest` gekennzeichnet (statt mit `@Test`).

Mit Hilfe von `@ValueSource` kann man ein einfaches Array von Werten (Strings oder primitive Datentypen) angeben, mit denen der Test ausgeführt wird. Dazu bekommt die Testmethode einen entsprechenden passenden Parameter:

```java
@ParameterizedTest
@ValueSource(strings = {"wuppie", "fluppie", "foo"})
void testWuppie(String candidate) {
    assertTrue(candidate.equals("wuppie"));
}
```

Alternativ lassen sich als Parameterquelle u.a. Aufzählungen (`@EnumSource`) oder Methoden (`@MethodSource`) angeben.

Hinweis: Parametrisierte Tests werden in JUnit 5 derzeit noch als "*experimentell*" angesehen!

Beispiel: junit5.TestValueSource, junit5.TestMethodSource

2.6.2.9 Testsuiten: Tests gemeinsam ausführen (JUnit 4)

Eclipse: `New` > `Other` > `Java` > `JUnit` > `JUnit Test Suite`

```java
import org.junit.runner.RunWith;
import org.junit.runners.Suite;
import org.junit.runners.Suite.SuiteClasses;

@RunWith(Suite.class)
@SuiteClasses({
    // Hier kommen alle Testklassen rein
    PersonTest.class,
    StudiTest.class
})

public class MyTestSuite {
    // bleibt leer!!!
}
```

2.6.2.10 Testsuiten mit JUnit 5

In JUnit 5 gibt es zwei Möglichkeiten, Testsuiten zu erstellen:

- `@SelectPackages`: Angabe der Packages, die für die Testsuite zusammengefasst werden sollen
- `@SelectClasses`: Angabe der Klassen, die für die Testsuite zusammengefasst werden sollen

```
1 @RunWith(JUnitPlatform.class)
2 @SelectClasses({StudiTest5.class, WuppieTest5.class})
3 public class MyTestSuite5 {
4     // bleibt leer!!!
5 }
```

Zusätzlich kann man beispielsweise mit `@IncludeTags` oder `@ExcludeTags` Testmethoden mit bestimmten Tags einbinden oder ausschließen. Beispiel: Schließe alle Tests mit Tag "develop" aus: `@ExcludeTags("develop")`. Dabei wird an den Testmethoden zusätzlich das Tag `@Tag` verwendet, etwas `@Tag("develop")`.

Achtung: Laut der offiziellen Dokumentation (Abschnitt "4.4.4. Test Suite") gilt zumindest bei der Selection über `@SelectPackages` der Zwang zu einer Namenskonvention: Es werden dabei nur Klassen gefunden, deren Name mit `Test` beginnt oder endet! Weiterhin werden Testsuites mit der Annotation `@RunWith(JUnitPlatform.class)` **nicht** auf der "JUnit 5"-Plattform ausgeführt, sondern mit der JUnit 4-Infrastuktur!

2.6.2.11 Wrap-Up

JUnit als Framework für (Unit-) Tests; hier JUnit 4 (mit Ausblick auf JUnit 5)

- Testmethoden mit Annotation `@Test`
- `assert` (Testergebnis) vs. `assume` (Testvorbedingung)
- Aufbau der Testumgebung `@Before`
- Abbau der Testumgebung `@After`
- Steuern von Tests mit `@Ignore` oder `@Test(timout=XXX)`
- Exceptions einfordern mit `@Test(expected=package.Exception.class)`
- Tests zusammenfassen zu Testsuiten

Zum Nachlesen

- vogella GmbH (2021)
- The JUnit Team (2022)
- Kleuker (2019)
- Osherove (2014)
- Spillner und Linz (2012)
- Thies u. a. (o. J.)

Lernziele

- k3: Steuern von Tests (ignorieren, zeitliche Begrenzung)
- k3: Prüfung von Exceptions
- k3: Aufbau von Testsuiten mit JUnit

Quizzes

- Quiz JUnit-Basics (ILIAS)

Challenges

Schreiben Sie eine JUnit-Testklasse (JUnit 4.x oder 5.x) und testen Sie eine `ArrayList<String>`. Prüfen Sie dabei, ob das Einfügen und Entfernen wie erwartet funktioniert.

1. Initialisieren Sie in einer `setUp()`-Methode das Testobjekt und fügen Sie zwei Elemente ein. Stellen Sie mit einer passenden `assume*`-Methode sicher, dass die Liste genau diese beiden Elemente enthält. Die `setUp()`-Methode soll vor jedem Testfall ausgeführt werden.
2. Setzen Sie in einer `tearDown()`-Methode das Testobjekt wieder auf **`null`** und stellen Sie mit einer passenden `assume*`-Methode sicher, dass das Testobjekt tatsächlich **`null`** ist. Die `tearDown()`-Methode soll nach jedem Testfall ausgeführt werden.
3. Schreiben Sie eine Testmethode `testAdd()`. Fügen Sie ein weiteres Element zum Testobjekt hinzu und prüfen Sie mit einer passenden `assert*`-Methode, ob die Liste nach dem Einfügen den gewünschten Zustand hat: Die Länge der Liste muss 3 Elemente betragen und alle Elemente müssen in der richtigen Reihenfolge in der Liste stehen.
4. Schreiben Sie eine Testmethode `testRemoveObject()`. Entfernen Sie ein vorhandenes Element (über die Referenz auf das Objekt) aus dem Testobjekt und prüfen Sie mit einer passenden `assert*`-Methode, ob die Liste nach dem Entfernen den gewünschten Zustand hat: Die Liste darf nur noch das verbleibende Element enthalten.
5. Schreiben Sie eine Testmethode `testRemoveIndex()`. Entfernen Sie ein vorhandenes Element über dessen *Index* in der Liste und prüfen Sie mit einer passenden `assert*`-Methode, ob die Liste nach dem Entfernen den gewünschten Zustand hat: Die Liste darf nur noch das verbleibende Element enthalten. (Nutzen Sie zum Entfernen die `remove(int)`-Methode der Liste.)
6. Schreiben Sie zusätzlich einen **parametrisierten JUnit-Test** für die folgende Klasse:

```java
import java.util.ArrayList;

public class SpecialArrayList extends ArrayList<String> {
    public void concatAddStrings(String a, String b) {
        this.add(a + b);
    }
}
```

Testen Sie, ob die Methode `concatAddStrings` der Klasse `SpecialArrayList` die beiden übergebenen Strings korrekt konkateniert und das Ergebnis richtig in die Liste einfügt. Testen Sie dabei mit mindestens den folgenden Parameter-Tripeln:

a	b	expected
“”	“”	“”
“”	“a”	“a”
“a”	“”	“a”
“abc”	“123”	“abc123”

2.6.3 Testfallermittlung: Wie viel und was muss man testen?

TL;DR
Mit Hilfe der Äquivalenzklassenbildung kann man Testfälle bestimmen. Dabei wird der Eingabebereich für jeden Parameter einer Methode in Bereiche mit gleichem Verhalten der Methode eingeteilt (die sogenannten "Äquivalenzklassen"). Dabei können einige Äquivalenzklassen (ÄK) gültigen Eingabebereichen entsprechen ("gültige ÄK"), also erlaubten/erwarteten Eingaben (die zum gewünschten Verhalten führen), und die restlichen ÄK entsprechen dann ungültigen Eingabebereichen ("ungültige ÄK"), also nicht erlaubten Eingaben, die von der Methode zurückgewiesen werden sollten. Jede dieser ÄK muss in mindestens einem Testfall vorkommen, d.h. man bestimmt einen oder mehrere zufällige Werte in den ÄK. Dabei können über mehrere Parameter hinweg verschiedene gültige ÄK in einem Testfall kombiniert werden. Bei den ungültigen ÄK kann dagegen immer nur ein Parameter eine ungültige ÄK haben, für die restlichen Parameter müssen gültige ÄK genutzt werden, und diese werden dabei als durch diesen Testfall "nicht getestet" betrachtet.
Zusätzlich entstehen häufig Fehler bei den Grenzen der Bereiche, etwa in Schleifen. Deshalb führt man zusätzlich noch eine Grenzwertanalyse durch und bestimmt für jede ÄK den unteren und den oberen Grenzwert und erzeugt aus diesen Werten zusätzliche Testfälle.
Wenn in der getesteten Methode der Zustand des Objekts eine Rolle spielt, wird dieser wie ein weiterer Eingabeparameter für die Methode betrachtet und entsprechend in die ÄK-Bildung bzw. GW-Analyse einbezogen.
Wenn ein Testfall sich aus den gültigen ÄK/GW speist, spricht man auch von einem "Positiv-Test"; wenn ungültige ÄK/GW genutzt werden, spricht man auch von einem "Negativ-Test".

Videos

- VL Testfallermittlung

2.6.3.1 Hands-On (10 Minuten): Wieviel und was muss man testen?

```java
public class Studi {
    private int credits = 0;

    public void addToCredits(int credits) {
        if (credits < 0) {
            throw new IllegalArgumentException("Negative Credits!");
        }
        if (this.credits + credits > 210) {
            throw new IllegalArgumentException("Mehr als 210 Credits!");
        }
        this.credits += credits;
    }
}
```

2.6.3.1.1 *JEDE* Methode mindestens testen mit/auf:

- Positive Tests: Gutfall (Normalfall) => "gültige ÄK/GW"
- Negativ-Tests (Fehlbedienung, ungültige Werte) => "ungültige ÄK/GW"
- Rand- bzw. Extremwerte => GW
- Exceptions

=> Anforderungen abgedeckt (Black-Box)?

=> Wichtige Pfade im Code abgedeckt (White-Box)?

2.6.3.1.2 Praxis

- Je kritischer eine Klasse/Methode/Artefakt ist, um so intensiver testen!
- Suche nach Kompromissen: Testkosten vs. Kosten von Folgefehlern; beispielsweise kein Test generierter Methoden

=> "Erzeugen" der Testfälle über die Äquivalenzklassenbildung und Grenzwertanalyse (siehe nächste Folien). Mehr dann später im Wahlfach "Softwarequalität" …

2.6.3.2 Äquivalenzklassenbildung Beispiel: Zu testende Methode mit Eingabewert *x*, der zw. 10 und 100 liegen soll



- Zerlegung der Definitionsbereiche in Äquivalenzklassen (ÄK):
  - Disjunkte Teilmengen, wobei
  - Werte *einer* ÄK führen zu *gleichartigem* Verhalten
- Annahme: Eingabeparameter sind untereinander unabhängig
- Unterscheidung gültige und ungültige ÄK

2.6.3.2.1 Bemerkungen

Hintergrund: Da die Werte einer ÄK zu gleichartigem Verhalten führen, ist es egal, *welchen* Wert man aus einer ÄK für den Test nimmt.

Formal hat man *eine* ungültige ÄK (d.h. die Menge aller ungültigen Werte). In der Programmierpraxis macht es aber einen Unterschied, ob es sich um Werte unterhalb oder oberhalb des erlaubten Wertebereichs handelt (Fallunterscheidung). Beispiel: Eine Funktion soll Werte zwischen 10 und 100 verarbeiten. Dann sind alle Werte kleiner 10 oder größer 100 mathematisch gesehen in der selben ÄK "ungültig". Praktisch macht es aber Sinn, eine ungültige ÄK für "kleiner 10" und eine weitere ungültige ÄK für "größer 100" zu betrachten …

Traditionell betrachtet man nur die Eingabeparameter. Es kann aber Sinn machen, auch die Ausgabeseite zu berücksichtigen (ist aber u.U. nur schwierig zu realisieren).

2.6.3.2.2 Faustregeln bei der Bildung von ÄK

- Falls eine Beschränkung einen Wertebereich spezifiziert: Aufteilung in eine gültige und zwei ungültige ÄK

  Beispiel: Eingabewert *x* soll zw. 10 und 100 liegen

  - Gültige ÄK: $[10, 100]$
  - Ungültige ÄKs: $x < 10$ und $100 < x$

- Falls eine Beschränkung eine minimale und maximale Anzahl von Werten spezifiziert: Aufteilung in eine gültige und zwei ungültige ÄK

  Beispiel: Jeder Studi muss pro Semester an mindestens einer LV teilnehmen, maximal sind 5 LVs erlaubt.

  - Gültige ÄK: $1 \leq x \leq 5$
  - Ungültige ÄKs: $x = 0$ (keine Teilnahme) und $5 < x$ (mehr als 5 Kurse)

- Falls eine Beschränkung eine Menge von Werten spezifiziert, die möglicherweise unterschiedlich behandelt werden: Für jeden Wert dieser Menge eine eigene gültige ÄK erstellen und zusätzlich insgesamt eine ungültige ÄK

  Beispiel: Das Hotel am Urlaubsort ermöglicht verschiedene Freizeitaktivitäten: Segway-fahren, Tauchen, Tennis, Golf

  - Gültige ÄKs:
    * Segway-fahren
    * Tauchen
    * Tennis
    * Golf
  - Ungültige ÄK: “alles andere”

- Falls eine Beschränkung eine Situation spezifiziert, die zwingend erfüllt sein muss: Aufteilung in eine gültige und eine ungültige ÄK

Hinweis: Werden Werte einer ÄK vermutlich nicht gleichwertig behandelt, dann erfolgt die Aufspaltung der ÄK in kleinere ÄKs. Das ist im Grunde die analoge Überlegung zu mehreren ungültigen ÄKs.

ÄKs sollten für die weitere Arbeit einheitlich und eindeutig benannt werden. Typisches Namensschema: “gÄKn” und “uÄKn” für gültige bzw. ungültige ÄKs mit der laufenden Nummer n.

2.6.3.3 ÄK: Erstellung der Testfälle

- Jede ÄK durch *mindestens* **einen TF** abdecken

- Dabei pro Testfall

  - *mehrere gültige ÄKs* kombinieren, oder
  - genau *eine ungültige ÄK* untersuchen (restl. Werte aus gültigen ÄK auffüllen; diese gelten dann aber nicht als getestet!)

Im Prinzip muss man zur Erstellung der Testfälle (TF) eine paarweise vollständige Kombination über die ÄK bilden, d.h. jede ÄK kommt mit jeder anderen ÄK in einem TF zur Ausführung.

Erinnerung: Annahme: Eingabeparameter sind untereinander unabhängig! => Es reicht, wenn *jede* gültige ÄK *einmal* in einem TF zur Ausführung kommt. => Kombination verschiedener gültiger ÄK in *einem TF*.

Achtung: Dies gilt nur für die **gültigen** ÄK! Bei den ungültigen ÄKs dürfen diese nicht miteinander in einem TF kombiniert werden! Bei gleichzeitiger Behandlung verschiedener ungültiger ÄK bleiben u.U. Fehler unentdeckt, da sich die Wirkungen der ungültigen ÄK überlagern!

Für jeden Testfall (TF) wird aus den zu kombinierenden ÄK ein zufälliger Repräsentant ausgewählt.

2.6.3.4 ÄK: Beispiel: Eingabewert *x* soll zw. 10 und 100 liegen

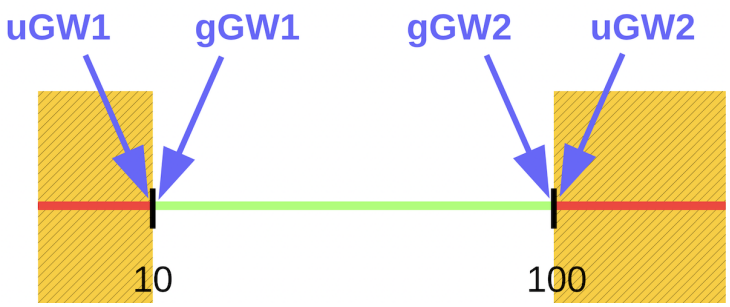
2.6.3.4.1 Äquivalenzklassen

Eingabe	gültige ÄK	ungültige ÄK
x	gÄK1: $[10, 100]$	uÄK2: $x < 10$
		uÄK3: $100 < x$

2.6.3.4.2 Tests

Testnummer	1	2	3
geprüfte ÄK	gÄK1	uÄK2	uÄK3
x	42	7	120
Erwartetes Ergebnis	OK	Exception	Exception

2.6.3.5 Grenzwertanalyse



Beobachtung: Grenzen in Verzweigungen/Schleifen kritisch

- Grenzen der ÄK (kleinste und größte Werte) **zusätzlich** testen
  - "gültige Grenzwerte" (*gGW*): Grenzwerte von gültigen ÄK
  - "ungültige Grenzwerte" (*uGW*): Grenzwerte von ungültigen ÄK

Zusätzlich sinnvoll: Weitere grenznahe Werte, d.h. weitere Werte "rechts" und "links" der Grenze nutzen.

Bildung der Testfälle:

- Jeder GW muss in mind. einem TF vorkommen

Pro TF darf ein GW (gültig oder ungültig) verwendet werden, die restlichen Parameter werden (mit zufälligen Werten) aus gültigen ÄK aufgefüllt, um mögliche Grenzwertprobleme nicht zu überlagern.

2.6.3.6 GW: Beispiel: Eingabewert *x* soll zw. 10 und 100 liegen

2.6.3.6.1 Äquivalenzklassen

Eingabe	gültige ÄK	ungültige ÄK
x	gÄK1: $[10, 100]$	uÄK2: $x < 10$
		uÄK3: $100 < x$

2.6.3.6.2 Grenzwertanalyse Zusätzliche Testdaten: 9 (uÄK2o) und 10 (gÄK1u) sowie 100 (gÄK1o) und 101 (uÄK3u)

2.6.3.6.3 Tests

Testnummer	4	5	6	7
geprüfter GW	gÄK1u	gÄK1o	uÄK2o	uÄK3u
x	10	100	9	101
Erwartetes Ergebnis	OK	OK	Exception	Exception

Hinweis: Die Ergebnisse der GW-Analyse werden **zusätzlich** zu den Werten aus der ÄK-Analyse eingesetzt. Für das obige Beispiel würde man also folgende Tests aus der kombinierten ÄK- und GW-Analyse erhalten:

Testnummer	1	2	3	4	5	6	7
geprüfte(r) ÄK/GW	gÄK1	uÄK2	uÄK3	gÄK1u	gÄK1o	uÄK2o	uÄK3u

Testnummer	1	2	3	4	5	6	7
x	42	7	120	10	100	9	101
Erwartetes Ergebnis	OK	Exception	Exception	OK	OK	Exception	Exception

2.6.3.7 Anmerkung: Analyse abhängiger Parameter Wenn das Ergebnis von der Kombination der Eingabewerte abhängt, dann sollte man dies bei der Äquivalenzklassenbildung berücksichtigen: Die ÄK sind in diesem Fall in Bezug auf die Kombinationen zu bilden!

Schauen Sie sich dazu das Beispiel im Kleuker (2019), Abschnitt “4.3 Analyse abhängiger Parameter” an.

Die einfache ÄK-Bildung würde in diesem Fall versagen, da die Eingabewerte nicht unabhängig sind. Leider ist die Betrachtung der möglichen Kombinationen u.U. eine sehr komplexe Aufgabe …

Analoge Überlegungen gelten auch für die ÄK-Bildung im Zusammenhang mit objektorientierter Programmierung. Die Eingabewerte und der Objektzustand müssen dann *gemeinsam* bei der ÄK-Bildung betrachtet werden!

Vergleiche Kleuker (2019), Abschnitt “4.4 Äquivalenzklassen und Objektorientierung”.

2.6.3.8 Wrap-Up

- Gründliches Testen ist ebenso viel Aufwand wie Coden
- Äquivalenzklassenbildung und Grenzwertanalyse

Zum Nachlesen

- vogella GmbH (2021)
- The JUnit Team (2022)
- Kleuker (2019)
- Osherove (2014)
- Spillner und Linz (2012)
- Thies u. a. (o. J.)

Lernziele

- k2: Merkmale schlecht testbaren Codes erklären
- k2: Merkmale guter Unit-Tests erklären
- k3: Erstellen von Testfällen mittels Äquivalenzklassenbildung und Grenzwertanalyse

Quizzes

- Quiz Testfallermittlung (ILIAS)

Challenges

Der RSV Flotte Speiche hat in seiner Mitgliederverwaltung (`MitgliederVerwaltung`) die Methode `testBeitritt` implementiert. Mit dieser Methode wird geprüft, ob neue Mitglieder in den Radsportverein aufgenommen werden können.

```java
public class MitgliederVerwaltung {

    /**
     * Testet, ob ein Mitglied in den Verein aufgenommen werden kann.
     *
     * @param alter       Alter in Lebensjahren, Bereich [0, 99]
     * @param motivation  Motivation auf einer Scala von 0 bis 10
     * @return <code>true</code>, wenn das Mitglied aufgenommen werden
         kann, sonst <code>false</code>
     */
    public boolean testBeitritt(int alter, int motivation) {
        if (alter < 0 || alter > 99 || motivation < 0 || motivation > 10)
            {
            throw new IllegalArgumentException("Alter oder/und Motivation
                ungültig");
        }
        if (alter < 16) {
            return false;
        }
        return motivation >= 4 && motivation <= 7;
    }
}
```

1. Führen Sie eine Äquivalenzklassenbildung durch und geben Sie die gefundenen Äquivalenzklassen (*ÄK*) an: laufende Nummer, Definition (Wertebereiche o.ä.), kurze Beschreibung (gültige/ungültige ÄK, Bedeutung).
2. Führen Sie zusätzlich eine Grenzwertanalyse durch und geben Sie die jeweiligen Grenzwerte (*GW*) an.
3. Erstellen Sie aus den ÄK und GW wie in der Vorlesung diskutiert Testfälle. Geben Sie pro Testfall (*TF*) an, welche ÄK und/oder GW abgedeckt sind, welche Eingaben Sie vorsehen und welche Ausgabe Sie erwarten.

   Hinweis: Erstellen Sie separate (zusätzliche) TF für die GW, d.h. integrieren Sie diese *nicht* in die ÄK-TF.
4. Implementieren Sie die Testfälle in JUnit (JUnit 4 oder 5). Fassen Sie die Testfälle der gültigen ÄK in einem parametrisierten Test zusammen. Für die ungültigen ÄKs erstellen Sie jeweils eine eigene JUnit-Testmethode. Beachten Sie, dass Sie auch die Exceptions testen müssen.

2.6.4 Mocking mit Mockito

TL;DR
Häufig hat man es in Softwaretests mit dem Problem zu tun, dass die zu testenden Klassen von anderen, noch nicht implementierten Klassen oder von zufälligen oder langsamen Operationen abhängen.
In solchen Situationen kann man auf "Platzhalter" für diese Abhängigkeiten zurückgreifen. Dies können einfache Stubs sein, also Objekte, die einfach einen festen Wert bei einem Methodenaufruf zurückliefern oder Mocks, wo man auf die Argumente eines Methodenaufrufs reagieren kann und passende unterschiedliche Rückgabewerte zurückgeben kann.
Mockito ist eine Java-Bibliothek, die zusammen mit JUnit das Mocking von Klassen in Java erlaubt. Man kann hier zusätzlich auch die Interaktion mit dem gemockten Objekt überprüfen und testen, ob eine bestimmte Methode mit bestimmten Argumenten aufgerufen wurde und wie oft.

Videos

- VL Mocking
- Demo Mocking: Stubs
- Demo Mocking: Mocks
- Demo Mocking: Spy
- Demo Mocking: verify()
- Demo Mocking: Matcher

2.6.4.1 Motivation: Entwicklung einer Studi-/Prüfungsverwaltung

2.6.4.1.1 Szenario Zwei Teams entwickeln eine neue Studi-/Prüfungsverwaltung für die Hochschule. Ein Team modelliert dabei die Studierenden, ein anderes Team modelliert die Prüfungsverwaltung LSF.

- Team A:

```
1  public class Studi {
2      String name;  LSF lsf;
3
4      public Studi(String name, LSF lsf) {
5          this.name = name;  this.lsf = lsf;
6      }
7
8      public boolean anmelden(String modul) { return lsf.anmelden(name, modul
           ); }
9      public boolean einsicht(String modul) { return lsf.ergebnis(name, modul
           ) > 50; }
10 }
```

- Team B:

```
1  public class LSF {
2      public boolean anmelden(String name, String modul) { throw new
           UnsupportedOperationException(); }
3      public int ergebnis(String name, String modul) { throw new
           UnsupportedOperationException(); }
```

```
4 }
```

Team B kommt nicht so recht vorwärts, Team A ist fertig und will schon testen.

Wie kann Team A seinen Code testen?

Optionen:

- Gar nicht testen?!
- Das LSF selbst implementieren? Wer pflegt das dann? => manuell implementierte Stubs
- Das LSF durch einen Mock ersetzen => Einsatz der Bibliothek “mockito”

2.6.4.1.2 Motivation Mocking und Mockito

Mockito ist ein Mocking-Framework für JUnit. Es simuliert das Verhalten eines realen Objektes oder einer realen Methode.

Wofür brauchen wir denn jetzt so ein Mocking-Framework überhaupt?

Wir wollen die Funktionalität einer Klasse isoliert vom Rest testen können. Dabei stören uns aber bisher so ein paar Dinge:

- Arbeiten mit den echten Objekten ist langsam (zum Beispiel aufgrund von Datenbankenzugriffen)
- Objekte beinhalten oft komplexe Abhängigkeiten, die in Tests schwer abzudecken sind
- Manchmal existiert der zu testende Teil einer Applikation auch noch gar nicht, sondern es gibt nur die Interfaces.
- Oder es gibt unschöne Seiteneffekte beim Arbeiten mit den realen Objekten. Zum Beispiel könnte es sein, das immer eine E-Mail versendet wird, wenn wir mit einem Objekt interagieren.

In solchen Situationen wollen wir eine Möglichkeit haben, das Verhalten eines realen Objektes bzw. der Methoden zu simulieren, ohne dabei die originalen Methoden aufrufen zu müssen. (Manchmal möchte man das dennoch, aber dazu später mehr…)

Und genau hier kommt Mockito ins Spiel. Mockito hilft uns dabei, uns von den externen Abhängigkeiten zu lösen, indem es sogenannte Mocks, Stubs oder Spies anbietet, mit denen sich das Verhalten der realen Objekte simulieren/überwachen und testen lässt.

2.6.4.1.3 Aber was genau ist denn jetzt eigentlich Mocking?

Ein Mock-Objekt (“etwas vortäuschen”) ist im Software-Test ein Objekt, das als Platzhalter (Attrappe) für das echte Objekt verwendet wird.

Mocks sind in JUnit-Tests immer dann nützlich, wenn man externe Abhängigkeiten hat, auf die der eigene Code zugreift. Das können zum Beispiel externe APIs sein oder Datenbanken etc. … Mocks helfen einem beim Testen nun dabei, sich von diesen externen Abhängigkeiten zu lösen und seine Softwarefunktionalität dennoch schnell und effizient testen zu können ohne evtl. auftretende Verbindungsfehler oder andere mögliche Seiteneffekte der externen Abhängigkeiten auszulösen.

Dabei simulieren Mocks die Funktionalität der externen APIs oder Datenbankzugriffe. Auf diese Weise ist es möglich Softwaretests zu schreiben, die scheinbar die gleichen Methoden aufrufen, die sie auch im regulären Softwarebetrieb nutzen würden, allerdings werden diese wie oben erwähnt allerdings für die Tests nur simuliert.

Mocking ist also eine Technik, die in Softwaretests verwendet wird, in denen die gemockten Objekte anstatt der realen Objekte zu Testzwecken genutzt werden. Die gemockten Objekte liefern dabei bei einem vom Programmierer bestimmten (Dummy-) Input, einen dazu passenden gelieferten (Dummy-) Output, der durch seine vorhersagbare Funktionalität dann in den eigentlichen Testobjekten gut für den Test nutzbar ist.

Dabei ist es von Vorteil die drei Grundbegriffe “Mock”, “Stub” oder “Spy”, auf die wir in der Vorlesung noch häufiger treffen werden, voneinander abgrenzen und unterscheiden zu können.

2.6.4.1.4 Dabei bezeichnet ein

- **Stub**: Ein Stub ist ein Objekt, dessen Methoden nur mit einer minimalen Logik für den Test implementiert wurden. Häufig werden dabei einfach feste (konstante) Werte zurückgeliefert, d.h. beim Aufruf einer Methode wird unabhängig von der konkreten Eingabe immer die selbe Ausgabe zurückgeliefert.
- **Mock**: Ein Mock ist ein Objekt, welches im Gegensatz zum Stub bei vorher definierten Funktionsaufrufen mit vorher definierten Argumente eine definierte Rückgabe liefert.
- **Spy**: Ein Spy ist ein Objekt, welches Aufrufe und übergebene Werte protokolliert und abfragbar macht. Es ist also eine Art Wrapper um einen Stub oder einen Mock.

2.6.4.1.5 Mockito Setup

- Gradle: `build.gradle`

```
dependencies {
    implementation 'junit:junit:4.13.2'
    implementation 'org.mockito:mockito-core:4.5.1'
}
```

- Maven: `pom.xml`

```
<dependencies>
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
    </dependency>
        <groupId>org.mockito</groupId>
        <artifactId>mockito-core</artifactId>
        <version>4.5.1</version>
    </dependency>
</dependencies>
```

2.6.4.2 Manuell Stubs implementieren Team A könnte manuell das LSF rudimentär implementieren (nur für die Tests, einfach mit festen Rückgabewerten): **Stubs**

```
public class StudiStubTest {
    Studi studi;  LSF lsf;

    @Before
    public void setUp() { lsf = new LsfStub();  studi = new Studi("Harald", lsf
        ); }
```

```
6
7       @Test
8       public void testAnmelden() { assertTrue(studi.anmelden("PM-Dungeon")); }
9       @Test
10      public void testEinsicht() { assertTrue(studi.einsicht("PM-Dungeon")); }
11
12
13      // Stub für das noch nicht fertige LSF
14      class LsfStub extends LSF {
15          public boolean anmelden(String name, String modul) { return true; }
16          public int ergebnis(String name, String modul) { return 80; }
17      }
18  }
```

Problem: Wartung der Tests (wenn das richtige LSF fertig ist) und Wartung der Stubs (wenn sich die Schnittstelle des LSF ändert, muss auch der Stub nachgezogen werden).

Problem: Der Stub hat nur eine Art minimale Default-Logik (sonst könnte man ja das LSF gleich selbst implementieren). Wenn man im Test andere Antworten braucht, müsste man einen weiteren Stub anlegen …

Demo hsbi.StudiStubTest

2.6.4.3 Mockito: Mocking von ganzen Klassen

Lösung: Mocking der Klasse `LSF` mit Mockito für den Test von `Studi`: `mock()`.

```
1   public class StudiMockTest {
2       Studi studi;  LSF lsf;
3
4       @Before
5       public void setUp() { lsf = mock(LSF.class);  studi = new Studi("Harald",
            lsf); }
6
7       @Test
8       public void testAnmelden() {
9           when(lsf.anmelden(anyString(), anyString())).thenReturn(true);
10          assertTrue(studi.anmelden("PM-Dungeon"));
11      }
12
13      @Test
14      public void testEinsichtI() {
15          when(lsf.ergebnis("Harald", "PM-Dungeon")).thenReturn(80);
16          assertTrue(studi.einsicht("PM-Dungeon"));
17      }
18      @Test
19      public void testEinsichtII() {
20          when(lsf.ergebnis("Harald", "PM-Dungeon")).thenReturn(40);
21          assertFalse(studi.einsicht("PM-Dungeon"));
22      }
23  }
```

Der Aufruf `mock(LSF.class)` erzeugt einen Mock der Klasse (oder des Interfaces) `LSF`. Dabei wird ein Objekt vom Typ `LSF` erzeugt, mit dem man dann wie mit einem normalen Objekt weiter arbeiten kann. Die Methoden sind allerdings nicht implementiert …

Mit Hilfe von `when().thenReturn()` kann man definieren, was genau beim Aufruf einer bestimmten Methode auf dem Mock passieren soll, d.h. welcher Rückgabewert entsprechend zurückgegeben werden soll. Hier kann man dann für bestimmte Argumentwerte andere Rückgabewerte definieren. `when(lsf.ergebnis("Harald", "PM-Dungeon")).thenReturn(80)` gibt also für den Aufruf von `ergebnis` mit den Argumenten `"Harald"` und `"PM-Dungeon"` auf dem Mock `lsf` den Wert 80 zurück.

Dies kann man in weiten Grenzen flexibel anpassen.

Mit Hilfe der Argument-Matcher `anyString()` wird jedes String-Argument akzeptiert.

Demo hsbi.StudiMockTest

2.6.4.4 Mockito: Spy = Wrapper um ein Objekt Team B hat das `LSF` nun implementiert und Team A kann es endlich für die Tests benutzen. Aber das `LSF` hat eine Zufallskomponente (`ergebnis()`). Wie kann man nun die Reaktion des Studis testen (`einsicht()`)?

Lösung: Mockito-Spy als partieller Mock einer Klasse (Wrapper um ein Objekt): `spy()`.

```
1  public class StudiSpyTest {
2      Studi studi;  LSF lsf;
3
4      @Before
5      public void setUp() { lsf = spy(LSF.class);  studi = new Studi("Harald",
           lsf); }
6
7      @Test
8      public void testAnmelden() { assertTrue(studi.anmelden("PM-Dungeon")); }
9
10     @Test
11     public void testEinsichtI() {
12         doReturn(80).when(lsf).ergebnis("Harald", "PM-Dungeon");
13         assertTrue(studi.einsicht("PM-Dungeon"));
14     }
15     @Test
16     public void testEinsichtII() {
17         doReturn(40).when(lsf).ergebnis("Harald", "PM-Dungeon");
18         assertFalse(studi.einsicht("PM-Dungeon"));
19     }
20 }
```

Der Aufruf `spy(LSF.class)` erzeugt einen Spy um ein Objekt der Klasse `LSF`. Dabei bleiben zunächst die Methoden in `LSF` erhalten und können aufgerufen werden, sie können aber auch mit einem (partiellen) Mock überlagert werden. Der Spy zeichnet wie der Mock die Interaktion mit dem Objekt auf.

Mit Hilfe von `doReturn().when()` kann man definieren, was genau beim Aufruf einer bestimmten Methode auf dem Spy passieren soll, d.h. welcher Rückgabewert entsprechend zurückgegeben werden soll. Hier kann man analog zum Mock für bestimmte Argumentwerte andere Rückgabewerte definieren. `doReturn(40).when(lsf).ergebnis("Harald", "PM-Dungeon")` gibt also für den Aufruf von `ergebnis` mit den Argumenten `"Harald"` und `"PM-Dungeon"` auf dem Spy `lsf` den Wert 40 zurück.

Wenn man die Methoden nicht mit einem partiellen Mock überschreibt, dann wird einfach die originale Methode aufgerufen (Beispiel: In `studi.anmelden("PM-Dungeon")` wird `lsf.anmelden("Harald", "PM-Dungeon")` aufgerufen.).

Auch hier können Argument-Matcher wie `anyString()` eingesetzt werden.

Demo hsbi.StudiSpyTest

2.6.4.5 Wurde eine Methode aufgerufen?

```java
public class VerifyTest {
    @Test
    public void testAnmelden() {
        LSF lsf = mock(LSF.class);  Studi studi = new Studi("Harald", lsf);

        when(lsf.anmelden("Harald", "PM-Dungeon")).thenReturn(true);

        assertTrue(studi.anmelden("PM-Dungeon"));


        verify(lsf).anmelden("Harald", "PM-Dungeon");
        verify(lsf, times(1)).anmelden("Harald", "PM-Dungeon");

        verify(lsf, atLeast(1)).anmelden("Harald", "PM-Dungeon");
        verify(lsf, atMost(1)).anmelden("Harald", "PM-Dungeon");

        verify(lsf, never()).ergebnis("Harald", "PM-Dungeon");

        verifyNoMoreInteractions(lsf);
    }
}
```

Mit der Methode `verify()` kann auf einem Mock oder Spy überprüft werden, ob und wie oft und in welcher Reihenfolge Methoden aufgerufen wurden und mit welchen Argumenten. Auch hier lassen sich wieder Argument-Matcher wie `anyString()` einsetzen.

Ein einfaches `verify(mock)` prüft dabei, ob die entsprechende Methode exakt einmal vorher aufgerufen wurde. Dies ist äquivalent zu `verify(mock, times(1))`. Analog kann man mit den Parametern `atLeast()` oder `atMost` bestimmte Unter- oder Obergrenzen für die Aufrufe angeben und mit `never()` prüfen, ob es gar keinen Aufruf vorher gab.

`verifyNoMoreInteractions(lsf)` ist interessant: Es ist genau dann **`true`**, wenn es außer den vorher abgefragten Interaktionen keinerlei sonstigen Interaktionen mit dem Mock oder Spy gab.

```java
LSF lsf = mock(LSF.class);
Studi studi = new Studi("Harald", lsf);

when(lsf.anmelden("Harald", "PM-Dungeon")).thenReturn(true);

InOrder inOrder = inOrder(lsf);

assertTrue(studi.anmelden("PM-Dungeon"));
studi.anmelden("Wuppie");

inOrder.verify(lsf).anmelden("Harald", "Wuppie");
inOrder.verify(lsf).anmelden("Harald", "PM-Dungeon");
```

Mit `InOrder` lassen sich Aufrufe auf einem Mock/Spy oder auch auf verschiedenen Mocks/Spies in eine zeitliche Reihenfolge bringen und so überprüfen.

Demo hsbi.VerifyTest

2.6.4.6 Fangen von Argumenten

```java
public class MatcherTest {
    @Test
    public void testAnmelden() {
        LSF lsf = mock(LSF.class);  Studi studi = new Studi("Harald", lsf);

        when(lsf.anmelden(anyString(), anyString())).thenReturn(false);
        when(lsf.anmelden("Harald", "PM-Dungeon")).thenReturn(true);

        assertTrue(studi.anmelden("PM-Dungeon"));
        assertFalse(studi.anmelden("Wuppie?"));

        verify(lsf, times(1)).anmelden("Harald", "PM-Dungeon");
        verify(lsf, times(1)).anmelden("Harald", "Wuppie?");

        verify(lsf, times(2)).anmelden(anyString(), anyString());
        verify(lsf, times(1)).anmelden(eq("Harald"), eq("Wuppie?"));
        verify(lsf, times(2)).anmelden(argThat(new MyHaraldMatcher()),
            anyString());
    }


    class MyHaraldMatcher implements ArgumentMatcher<String> {
        public boolean matches(String s) { return s.equals("Harald"); }
    }
}
```

Sie können die konkreten Argumente angeben, für die der Aufruf gelten soll. Alternativ können Sie mit vordefinierten `ArgumentMatchers` wie `anyString()` beispielsweise auf beliebige Strings reagieren oder selbst einen eigenen `ArgumentMatcher<T>` für Ihren Typ `T` erstellen und nutzen.

Wichtig: Wenn Sie für einen Parameter einen `ArgumentMatcher` einsetzen, müssen Sie für die restlichen Parameter der Methode dies ebenfalls tun. Sie können keine konkreten Argumente mit `ArgumentMatcher` mischen.

Sie finden viele weitere vordefinierte Matcher in der Klasse `ArgumentMatchers`. Mit der Klasse `ArgumentCaptor<T>` finden Sie eine alternative Möglichkeit, auf Argumente in gemockten Methoden zu reagieren. Schauen Sie sich dazu die Javadoc von Mockito an.

Demo hsbi.MatcherTest

2.6.4.7 Ausblick: PowerMock

Mockito sehr mächtig, aber unterstützt (u.a.) keine

- Konstruktoren
- private Methoden
- final Methoden
- static Methoden (ab Version 3.4.0 scheint auch Mockito statische Methoden zu unterstützen)

=> Lösung: PowerMock

2.6.4.8 Ausführlicheres Beispiel: WuppiWarenlager

Credits: Der Dank für die Erstellung des nachfolgenden Beispiels und Textes geht an @jedi101.

Demo: WuppiWarenlager (wuppie.stub)

Bei dem gezeigten Beispiel unseres `WuppiStores` sieht man, dass dieser normalerweise von einem fertigen Warenlager die Wuppis beziehen möchte. Da dieses Lager aber noch nicht existiert, haben wir uns kurzerhand einfach einen Stub von unserem `IWuppiWarenlager`-Interface erstellt, in dem wir zu Testzwecken händisch ein Paar Wuppis ins Lager geräumt haben.

Das funktioniert in diesem Mini-Testbeispiel ganz gut aber, wenn unsere Stores erst einmal so richtig Fahrt aufnehmen und wir irgendwann weltweit Wuppis verkaufen, wird der Code des `IWuppiWarenlagers` wahrscheinlich sehr schnell viel komplexer werden, was unweigerlich dann zu Maintenance-Problemen unserer händisch angelegten Tests führt. Wenn wir zum Beispiel einmal eine Methode hinzufügen wollen, die es uns ermöglicht, nicht immer alle Wuppis aus dem Lager zu ordern oder vielleicht noch andere Methoden, die Fluppis orderbar machen, hinzufügen, müssen wir immer dafür sorgen, dass wir die getätigten Änderungen händisch in den Stub des Warenlagers einpflegen.

Das will eigentlich niemand…

2.6.4.8.1 Einsatz von Mockito

Aber es gibt da einen Ausweg. Wenn es komplexer wird, verwenden wir Mocks.

Bislang haben wir noch keinen Gebrauch von Mockito gemacht. Das ändern wir nun.

Demo: WuppiWarenlager (wuppie.mock)

Wie in diesem Beispiel gezeigt, müssen wir nun keinen Stub mehr von Hand erstellen, sondern überlassen dies Mockito.

```
1 IWuppiWarenlager lager = mock(IWuppiWarenlager.class);
```

Anschließend können wir, ohne die Methode `getAllWuppis()` implementiert zu haben, dennoch so tun als, ob die Methode eine Funktionalität hätte.

```
1 // Erstellen eines imaginären Lagerbestands.
2 List<String> wuppisImLager = Arrays.asList("GruenerWuppi","RoterWuppi");
3 when(lager.getAlleWuppis()).thenReturn(wuppisImLager);
```

Wann immer nun die Methode `getAlleWuppis()` des gemockten Lagers aufgerufen wird, wird dieser Aufruf von Mockito abgefangen und wie oben definiert verändert. Das Ergebnis können wir abschließend einfach in unserem Test testen:

```
1 // Erzeugen des WuppiStores.
2 WuppiStore wuppiStore = new WuppiStore(lager);
3
4 // Bestelle alle Wuppis aus dem gemockten Lager List<String>
5 bestellteWuppis = wuppiStore.bestelleAlleWuppis(lager);
6
7 // Hat die Bestellung geklappt?
8 assertEquals(2,bestellteWuppis.size());
```

2.6.4.8.2 Mockito Spies Manchmal möchten wir allerdings nicht immer gleich ein ganzes Objekt mocken, aber dennoch Einfluss auf die aufgerufenen Methoden eines Objekts haben, um diese testen zu können. Vielleicht gibt es dabei ja sogar eine Möglichkeit unsere JUnit-Tests, mit denen wir normalerweise nur Rückgabewerte von Methoden testen können, zusätzlich auch das Verhalten also die Interaktionen mit einem Objekt beobachtbar zu machen. Somit wären diese Interaktionen auch testbar.

Und genau dafür bietet Mockito eine Funktion: der sogenannte “Spy”.

Dieser Spion erlaubt es uns nun zusätzlich das Verhalten zu testen. Das geht in die Richtung von BDD - Behavior Driven Development.

Demo: WuppiWarenlager (wuppie.spy)

```
1 // Spion erstellen, der unser wuppiWarenlager überwacht.
2 this.wuppiWarenlager = spy(WuppiWarenlager.class);
```

Hier hatten wir uns einen Spion erzeugt, mit dem sich anschließend das Verhalten verändern lässt:

```
1 when(wuppiWarenlager.getAlleWuppis()).thenReturn(Arrays.asList(new Wuppi("
      Wuppi007")));
```

Aber auch der Zugriff lässt sich kontrollieren/testen:

```
1 verify(wuppiWarenlager).addWuppi(normalerWuppi);
2 verifyNoMoreInteractions(wuppiWarenlager);
```

Die normalen Testmöglichkeiten von JUnit runden unseren Test zudem ab.

```
1 assertEquals(1,wuppiWarenlager.lager.size());
```

2.6.4.9 Mockito und Annotationen In Mockito können Sie wie oben gezeigt mit `mock()` und `spy()` neue Mocks bzw. Spies erzeugen und mit `verify()` die Interaktion überprüfen und mit `ArgumentMatcher<T>` bzw. den vordefinierten `ArgumentMatchers` auf Argumente zuzugreifen bzw. darauf zu reagieren.

Zusätzlich/alternativ gibt es in Mockito zahlreiche Annotationen, die **ersatzweise** statt der genannten Methoden genutzt werden können. Hier ein kleiner Überblick über die wichtigsten in Mockito verwendeten Annotation:

- `@Mock` wird zum Markieren des zu mockenden Objekts verwendet.

  ```
  1 @Mock
  2 WuppiWarenlager lager;
  ```

- `@RunWith(MockitoJUnitRunner.class)` ist der entsprechende JUnit-Runner, wenn Sie Mocks mit `@Mock` anlegen.

  ```
  1 @RunWith(MockitoJUnitRunner.class)
  2 public class ToDoBusinessMock {...}
  ```

- `@Spy` erlaubt das Erstellen von partiell gemockten Objekten. Dabei wird eine Art Wrapper um das zu mockende Objekt gewickelt, der dafür sorgt, dass alle Methodenaufrufe des Objekts an den Spy delegiert werden. Diese können über den Spion dann abgefangen/verändert oder ausgewertet werden.

```
@Spy
ArrayList<Wuppi> arrayListenSpion;
```

- `@InjectMocks` erlaubt es, Parameter zu markieren, in denen Mocks und/oder Spies injiziert werden. Mockito versucht dann (in dieser Reihenfolge) per Konstruktorinjektion, Setterinjektion oder Propertyinjektion die Mocks zu injizieren. Weitere Informationen darüber findet man hier: Mockito Dokumentation

  Anmerkung: Es ist aber nicht ratsam "Field- oder Setterinjection" zu nutzen, da man nur bei der Verwendung von "Constructorinjection" sicherstellen kann, das eine Klasse nicht ohne die eigentlich notwendigen Parameter instanziiert wurde.

```
@InjectMocks
Wuppi fluppi;
```

- `@Captor` erlaubt es, die Argumente einer Methode abzufangen/auszuwerten. Im Zusammenspiel mit Mockitos `verify()`-Methode kann man somit auch die einer Methode übergebenen Argumente verifizieren.

```
@Captor
ArgumentCaptor<String> argumentCaptor;
```

- `@ExtendWith(MockitoExtension.class)` wird in JUnit5 verwendet, um die Initialisierung von Mocks zu vereinfachen. Damit entfällt zum Beispiel die noch unter JUnit4 nötige Initialisierung der Mocks durch einen Aufruf der Methode `MockitoAnnotations.openMocks()` im Setup des Tests (`@Before` bzw. `@BeforeEach`).

2.6.4.9.1 Prüfen der Interaktion mit *verify()*

Mit Hilfe der umfangreichen `verify()`-Methoden, die uns Mockito mitliefert, können wir unseren Code unter anderem auf unerwünschte Seiteneffekte testen. So ist es mit `verify` zum Beispiel möglich abzufragen, ob mit einem gemockten Objekt interagiert wurde, wie damit interagiert wurde, welche Argumente dabei übergeben worden sind und in welcher Reihenfolge die Interaktionen damit erfolgt sind.

Hier nur eine kurze Übersicht über das Testen des Codes mit Hilfe von Mockitos `verify()`-Methoden.

```
@Test
public void testVerify_DasKeineInteraktionMitDerListeStattgefundenHat() {
    // Testet, ob die spezifizierte Interaktion mit der Liste nie stattgefunden
        hat.
    verify(fluppisListe, never()).clear();
}
```

```
@Test
public void testVerify_ReihenfolgeDerInteraktionenMitDerFluppisListe() {
    // Testet, ob die Reihenfolge der spezifizierten Interaktionen mit der
       Liste eingehalten wurde.
    fluppisListe.clear();
    InOrder reihenfolge = inOrder(fluppisListe);
    reihenfolge.verify(fluppisListe).add("Fluppi001");
    reihenfolge.verify(fluppisListe).clear();
}
```

```java
@Test
public void testVerify_FlexibleArgumenteBeimZugriffAufFluppisListe() {
    // Testet, ob schon jemals etwas zu der Liste hinzugefügt wurde.
    // Dabei ist es egal welcher String eingegeben wurde.
    verify(fluppisListe).add(anyString());
}
```

```java
@Test
public void testVerify_InteraktionenMitHilfeDesArgumentCaptor() {
    // Testet, welches Argument beim Methodenaufruf übergeben wurde.
    fluppisListe.addAll(Arrays.asList("BobDerBaumeister"));
    ArgumentCaptor<List> argumentMagnet = ArgumentCaptor.forClass(FluppisListe.
        class);
    verify(fluppisListe).addAll(argumentMagnet.capture());
    List<String> argumente = argumentMagnet.getValue();
    assertEquals("BobDerBaumeister", argumente.get(0));
}
```

Demo: WuppiWarenlager (wuppie.verify)

2.6.4.10 Wrap-Up

- Gründliches Testen ist ebenso viel Aufwand wie Coden!

- Mockito ergänzt JUnit:
  - Mocken ganzer Klassen (`mock()`, `when().thenReturn()`)
  - Wrappen von Objekten (`spy()`, `doReturn().when()`)
  - Auswerten, wie häufig Methoden aufgerufen wurden (`verify()`)
  - Auswerten, mit welchen Argumenten Methoden aufgerufen wurden (`anyString`)

Zum Nachlesen

- S. Faber and B. Dutheil and R. Winterhalter and T.v.d. Lippe (2022)

Lernziele

- k2: Begriffe: Mocking, Mock, Stub, Spy
- k3: Erzeugen eines Mocks in Mockito
- k3: Erzeugen eines Spies in Mockito
- k3: Prüfen von Interaktion mit verify()
- k3: Einsatz von ArgumentMatcher

Quizzes

- Quiz Mocking (ILIAS)

Challenges

Betrachten Sie die drei Klassen `Utility.java`, `Evil.java` und `UtilityTest.java`:

```java
public class Utility {
    private int intResult = 0;
    private Evil evilClass;

    public Utility(Evil evilClass) {
        this.evilClass = evilClass;
    }

    public void evilMethod() {
        int i = 2 / 0;
    }

    public int nonEvilAdd(int a, int b) {
        return a + b;
    }

    public int evilAdd(int a, int b) {
        evilClass.evilMethod();
        return a + b;
    }

    public void veryEvilAdd(int a, int b) {
        evilMethod();
        evilClass.evilMethod();
        intResult = a + b;
    }

    public int getIntResult() {
        return intResult;
    }
}

public class Evil {
    public void evilMethod() {
        int i = 3 / 0;
    }
}

public class UtilityTest {
    private Utility utilityClass;
    // Initialisieren Sie die Attribute entsprechend vor jedem Test.

    @Test
    void test_nonEvilAdd() {
        Assertions.assertEquals(10, utilityClass.nonEvilAdd(9, 1));
    }

    @Test
    void test_evilAdd() {
        Assertions.assertEquals(10, utilityClass.evilAdd(9, 1));
    }

    @Test
    void test_veryEvilAdd() {
        utilityClass.veryEvilAdd(9, 1);
        Assertions.assertEquals(10, utilityClass.getIntResult());
    }
}
```

Testen Sie die Methoden `nonEvilAdd`, `evilAdd` und `veryEvilAdd` der Klasse `Utility.java` mit dem JUnit- und dem Mockito-Framework.

Vervollständigen Sie dazu die Klasse `UtilityTest.java` und nutzen Sie Mocking mit Mockito, um die Tests zum Laufen zu bringen. Die Tests dürfen Sie entsprechend verändern, aber die Aufrufe aus der Vorgabe müssen erhalten

2.7 Fortgeschrittene Java-Themen und Umgang mit JVM

2.7.1 Serialisierung von Objekten und Zuständen

TL;DR
Objekte lassen sich mit der Methode `void writeObject(Object)` in `ObjectOutputStream` einfach in einen Datenstrom schreiben. Dies kann beispielsweise eine Datei o.ä. sein. Mit Hilfe von `Object readObject()` in `ObjectInputStream` lassen sich Objekte aus dem Datenstrom auch wieder herstellen. Dies nennt man Serialisierung und De-Serialisierung.
Um Objekte einer Klasse serialisieren zu können, muss diese das leere Interface `Serializable` implementieren ("Marker-Interface"). Damit wird quasi die Unterstützung in `Object*Stream` freigeschaltet.
Wenn ein Objekt serialisiert wird, werden alle Attribute in den Datenstrom geschrieben, d.h. die Typen der Attribute müssen ihrerseits serialisierbar sein. Dies gilt für alle primitiven Typen und die meisten eingebauten Typen. Die Serialisierung erfolgt ggf. rekursiv, Zirkelreferenzen werden erkannt und aufgebrochen.
`static` und `transient` Attribute werden nicht serialisiert.
Beim De-Serialisieren wird das neue Objekt von der Laufzeitumgebung aus dem Datenstrom rekonstruiert. Dies geschieht direkt, es wird kein Konstruktor involviert.
Beim Serialisieren wird für die Klasse des zu schreibenden Objekts eine `serialVersionUID` berechnet und mit gespeichert. Beim Einlesen wird dann geprüft, ob die serialisierten Daten zur aktuellen Version der Klasse passen. Da dies relativ empfindlich gegenüber Änderungen an einer Klasse ist, wird empfohlen, selbst eine `serialVersionUID` pro Klasse zu definieren.

Videos

- VL Serialisierung
- Demo Serialisierung

2.7.1.1 Motivation: Persistierung von Objekten und Spielzuständen

```java
public class Studi {
    private final int credits = 42;
    private String name = "Hilde";

    ...
}
```

Wie kann ich Objekte speichern und wieder laden?

Ich möchte ein Spiel (einen Lauf) im Dungeon abspeichern, um es später fortsetzen zu können. Wie kann ich den aktuellen Zustand (also Level, Monster, Held, Inventar, XP/Health/…) so speichern, dass ich später das Spiel nach einem Neustart einfach fortsetzen kann?

2.7.1.2 Serialisierung von Objekten

- Klassen müssen Marker-Interface `Serializable` implementieren

"Marker-Interface": Interface ohne Methoden. Ändert das Verhalten des Compilers, wenn eine Klasse dieses Interface implementiert: Weitere Funktionen werden "freigeschaltet", beispielsweise die Fähigkeit, Klone zu erstellen (`Cloneable`) oder bei `Serializable` Objekte serialisierbar zu machen.

Das ist in meinen Augen eine "Design-Sünde" in Java (neben der Einführung von **`null`**): Normalerweise definieren Interfaces eine Schnittstelle, die eine das Interface implementierende Klasse dann erfüllen muss. Damit agiert das Interface wie ein Typ. Hier ist das Interface aber leer, es wird also keine Schnittstelle definiert. Aber es werden damit stattdessen Tooling-Optionen aktiviert, was Interfaces vom Konzept her eigentlich nicht machen sollten/dürften - dazu gibt es Annotationen!

- Schreiben von Objekten (samt Zustand) in Streams

```
1  ObjectOutputStream: void writeObject(Object)
```

Die Serialisierung erfolgt dabei für alle Attribute (außer **`static`** und **`transient`**, s.u.) rekursiv.

Dabei werden auch Zirkelreferenzen automatisch aufgelöst/unterbrochen.

- Lesen und "Wiedererwecken" der Objekte aus Streams

```
1  ObjectInputStream: Object readObject()
```

Dabei erfolgt KEIN Konstruktor-Aufruf!

2.7.1.3 Einfaches Beispiel

```
1  public class Studi implements Serializable {
2      private final int credits = 42;
3      private String name = "Hilde";
4
5      public static void writeObject(Studi studi, String filename) {
6          try (FileOutputStream fos = new FileOutputStream(filename);
7              ObjectOutputStream oos = new ObjectOutputStream(fos)) {
8              oos.writeObject(studi);    oos.close();
9          } catch (IOException ex) {}
10     }
11
12     public static Studi readObject(String filename) {
13         Studi studi = null;
14         try (FileInputStream fis = new FileInputStream(filename);
15             ObjectInputStream ois = new ObjectInputStream(fis)) {
16             studi = (Studi) ois.readObject();    ois.close();
17         } catch (IOException | ClassNotFoundException ex) {}
18         return studi;
19     }
20 }
```

2.7.1.4 Bedingungen für Objekt-Serialisierung

- Klassen implementieren Marker-Interface `Serializable`
- Alle Attribute müssen ebenfalls serialisierbar sein (oder Deklaration "**`transient`**")
- Alle primitiven Typen sind per Default serialisierbar
- Es wird automatisch rekursiv serialisiert, aber jedes Objekt nur einmal (bei Mehrfachreferenzierung)
- Serialisierbarkeit vererbt sich

2.7.1.5 Ausnahmen

- Als `static` deklarierte Attribute werden nicht serialisiert
- Als `transient` deklarierte Attribute werden nicht serialisiert
- Nicht serialisierbare Attribut-Typen führen zu `NotSerializableException`

2.7.1.6 Version-UID

```
1 static final long serialVersionUID = 42L;
```

- Dient zum Vergleich der serialisierten Version und der aktuellen Klasse
- Über IDE generieren oder manuell vergeben
- Wenn das Attribut fehlt, wird eine Art Checksumme von der Runtime-Umgebung berechnet (basierend auf diversen Eigenschaften der Klasse)

Dieser Wert wird beim Einlesen verglichen: Das Objekt wird nur dann wieder de-serialisiert, wenn die `serialVersionUID` mit der einzulesenden Klasse übereinstimmt!

Bei automatischer Berechnung der `serialVersionUID` durch die JVM kann jede kleine Änderung an der Klasse (beispielsweise Refactoring: Änderung der Methodennamen) eine neue `serialVersionUID` zur Folge haben. Das würde bedeuten, dass bereits serialisierte Objekte nicht mehr eingelesen werden können, auch wenn sich nur Methoden o.ä. verändert haben und die Attribute noch so vorhanden sind. Deshalb bietet es sich an, hier selbst eine `serialVersionUID` zu definieren - dann muss man aber auch selbst darauf achten, diese zu verändern, wenn sich wesentliche strukturelle Änderungen an der Klasse ergeben!

2.7.1.7 Bemerkungen Es existieren diverse weitere Fallstricke und Probleme, siehe (Bloch 2018) Kapitel 11 “Serialization”.

Man kann in den `ObjectOutputStream` nicht nur ein Objekt schreiben, sondern mehrere Objekte und Variablen schreiben lassen. In dieser Reihenfolge muss man diese dann aber auch wieder aus dem Stream herauslesen (vgl. Object Streams).

Man kann die zu serialisierenden Attribute mit der Annotation `@Serial` markieren. Dies ist in der Wirkung ähnlich zu `@Override`: Der Compiler prüft dann, ob die markierten Attribute wirklich serialisierbar sind und würde sonst zur Compile-Zeit einen Fehler werfen.

Weitere Links:

- Tutorials:
  - https://docs.oracle.com/en/java/javase/17/docs/specs/serialization/input.html
  - https://www.baeldung.com/java-serialization
- API:
  - https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/ObjectOutputStream.html
  - https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/ObjectInputStream.html
  - https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/Serializable.html

Demo: serial.SerializableStudi

2.7.1.8 Wrap-Up

- Markerinterface `Serializable` schaltet Serialisierbarkeit frei

- Objekte schreiben: `ObjectOutputStream`: `void writeObject(Object)`
- Objekte lesen: `ObjectInputStream`: `Object readObject()`

- Wichtigste Eigenschaften:
  - Attribute müssen serialisierbar sein
  - **`transient`** und **`static`** Attribute werden nicht serialisiert
  - De-Serialisierung: KEIN Konstruktor-Aufruf!
  - Serialisierbarkeit vererbt sich
  - Objekt-Referenz-Graph wird automatisch beachtet

Zum Nachlesen

- Oracle Corporation (2022c)

Lernziele

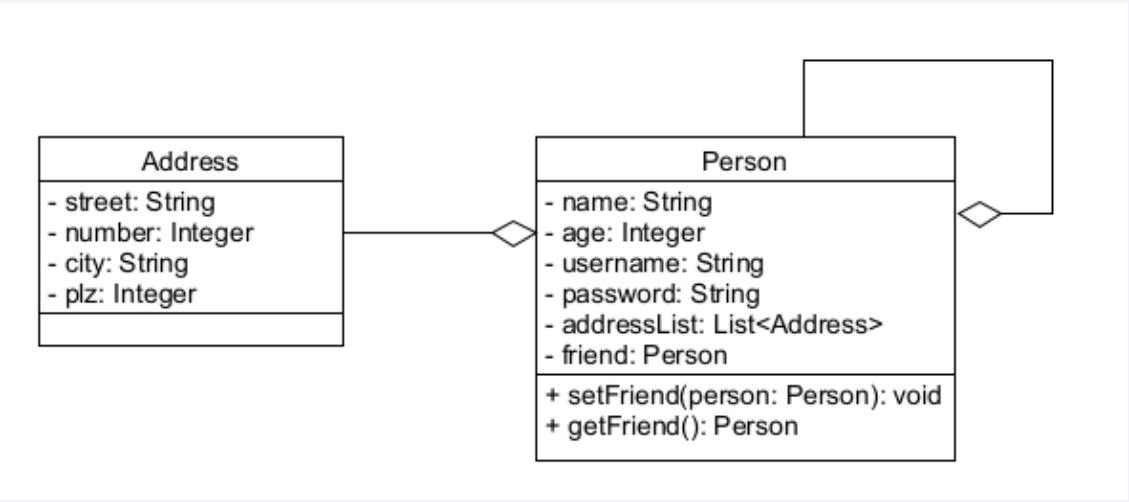
- k2: Was ist ein Marker-Interface und warum ist dies eine der großen Design-Sünden in Java?
- k2: Erklären Sie den Prozess der Serialisierung und De-Serialisierung. Worauf müssen Sie achten?
- k3: Serialisierung von Objekten und Programmzuständen
- k3: Serialisierung eigener Klassen und Typen

Quizzes

- Quiz Serialisierung (ILIAS)

Challenges

Implementieren Sie die beiden Klassen entsprechend dem UML-Diagram:



Objekte vom Typ `Person` sowie `Address` sollen serialisierbar sein (vgl. Vorlesung). Dabei soll das Passwort nicht serialisiert bzw. gespeichert werden, alle anderen Eigenschaften von `Person` sollen serialisierbar sein.
Hinweis: Verwenden Sie zur Umsetzung java.io.Serializable.
Erstellen Sie in Ihrem `main()` einige Instanzen von Person und speichern Sie diese in serialisierter Form und laden (deserialisieren) Sie diese anschließend in neue Variablen.
Betrachten Sie die ursprünglichen und die wieder deserialisierten Objekte mit Hilfe des Debuggers. Alternativ können Sie die Objekte auch in übersichtlicher Form über den Logger ausgeben.

2.7.2 Java Collections Framework

TL;DR

Die Collection-API bietet verschiedene Sammlungen an, mit denen man Objekte speichern kann: Listen, Queues, Mengen, … Für diese Typen gibt es jeweils verschiedene Implementierungen mit einem spezifischen Verhalten. Zusätzlich gibt es noch Maps für das Speichern von Key/Value-Paaren, dabei wird für die Keys eine Hash-Tabelle eingesetzt.
Die Hilfs-Klasse `Collections` bietet statische Hilfs-Methoden, die auf `Collection<T>`s anwendbar sind.
Wenn man eigene Klassen in der Collection-API oder in Map benutzen möchte, sollte man den “equals-hashCode-Contract” berücksichtigen.

Videos

- VL Collections

2.7.2.1 Motivation: Snippet aus einer Klasse im PM-Dungeon

```
1 private List<Entity> entities = new ArrayList<>();
2
```

```java
public void add(Entity e){
    if (!entities.contains(e)) entities.add(e);
}
```

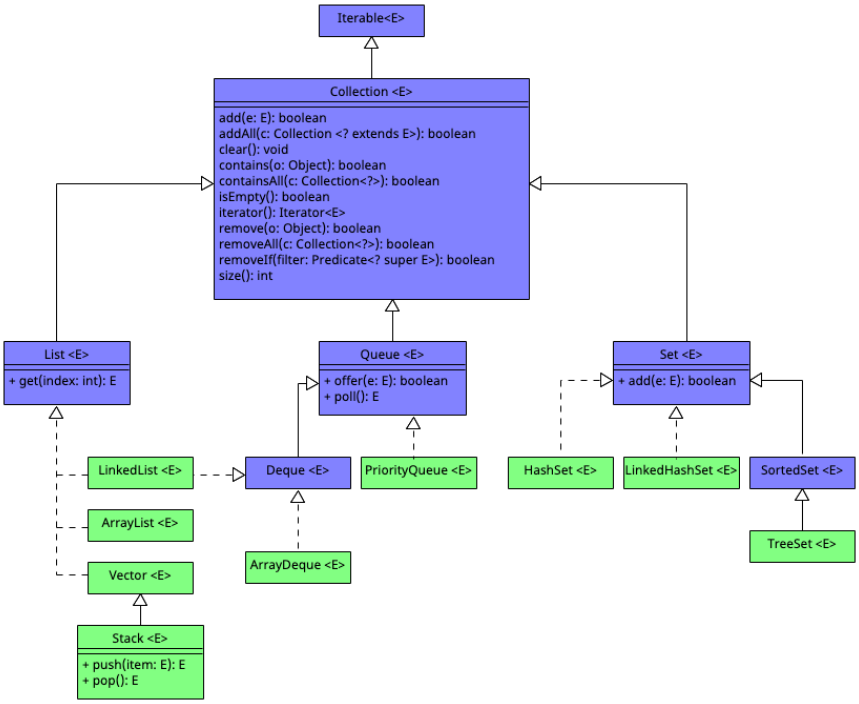
Die war ein reales Beispiel aus der Entwicklung des PM-Dungeon.

Es wurde eine `ArrayList<T>` zum Verwalten der Entitäten genutzt. Allerdings sollte jedes Element nur einmal in der Liste vorkommen, deshalb wurde beim Einfügen einer Entität geprüft, ob diese bereits in der Liste ist.

Hier wird die **falsche Datenstruktur** genutzt!

Eine Liste kann ein Objekt mehrfach enthalten, eine Menge (*Set*) hingegen kann ein Objekt nur *einmal* enthalten.

2.7.2.2 Collection-API in Java



Hinweis: Die abstrakten (Zwischen-) Klassen wurden im obigen UML aus Gründen der Übersichtlichkeit *nicht* aufgeführt. Aus den selben Gründen sind auch nur ausgewählte Methoden aufgenommen worden.

Hinweis: Blau = Interface, Grün = Klasse.

`Collection<T>` ist ein zentrales Interface im JDK und stellt die gemeinsame API der Collection-Klassen dar. Klassen, die `Collection<T>` implementieren, speichern und verwalten eine Menge an Objekten.

Unter anderem gibt es die aus dem Modul “ADS” bekannten Datentypen wie Listen, Sets, Queues etc.

Man unterscheidet zwischen “sorted” (geordnete) Collections, welche eine bestimmte Reihenfolge der Elemente halten (Reihenfolge des Einfügens, aufsteigende Werte etc.) und “unsorted” (ungeordnete) Collections, welche keine bestimmte Reihenfolge halten.

Eine Übersicht, welche Collection welche Datenstruktur implementiert, kann unter "Collection Implementations" eingesehen werden.

- `List<T>`-Collections sind eine geordnete Liste an Objekten. Per Index-Zugriff können Objekte an jeder Stelle der Liste zugegriffen (oder hinzugefügt) werden.
- `Queue<T>`-Collections sind eine geordnete Sammlung von Objekten. Objekte können nur am Ende der Queue hinzugefügt werden und nur am Anfang der Queue (der Head) gelesen oder entnommen werden ("first in first out").
- `Set<T>`-Collections sind eine (i.d.R.!) ungeordnete Menge an Objekten, die stets nur einmal in der Set enthalten sein können. In einem Set kann nicht direkt auf ein Objekt zugegriffen werden. Es kann aber geprüft werden, ob ein spezifisches Objekt in einer Set gespeichert ist.

Wichtig: `List<T>`, `Set<T>`, `Queue<T>` und `Map<K,V>` sind **Interfaces**, definieren also bestimmte Schnittstellen, die sich so wie aus ADS her bekannt verhalten. Diese können jeweils mit sehr unterschiedlichen Datenstrukturen implementiert werden und können dadurch auch intern ein anderes Verhalten haben (sortiert vs. nicht sortiert, Zugriffszeiten, …).

Siehe auch Interface Collection.

2.7.2.3 Listen: *ArrayList*

```
1 private List<Entity> entities = new ArrayList<>();
```



Link zu einer netten Animation

Eine `ArrayList<T>` ist von außen betrachtet ein sich dynamisch vergrößerndes Array.

Intern wird allerdings ein statisches(!) Array benutzt. Wenn dieses Array voll ist, wird es um 50% vergrößert und alle Inhalte in das neue Array kopiert. Davon merkt man als Nutzer aber nichts.

Dank es Arrays kann auf ein Element per Index mit O(1) zugegriffen werden.

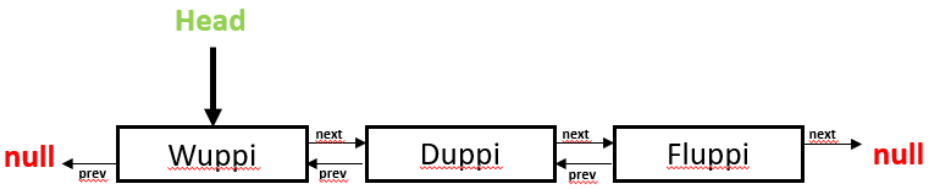
Wird ein Element aus der Liste gelöscht, rücken alle Nachfolgenden Einträge in der Liste einen Index auf (interner Kopiervorgang).

Deshalb ist eine `ArrayList<T>` effizient in der Abfrage und Manipulation von Einträgen, aber deutlich weniger effizient beim Hinzufügen und Löschen von Einträgen.

Per Default wird eine `ArrayList<T>` mit einem Array der Länge 10 angelegt, sobald das erste Element eingefügt wird. Man kann die Startgröße auch im Konstruktoraufruf der `ArrayList<T>` bestimmen: beispielsweise `new ArrayList<>(20)`.

Die Methoden einer `ArrayList<T>` sind nicht **`synchronized`**.

2.7.2.4 Listen: *LinkedList*



Link zu einer netten Animation

Eine `LinkedList<T>` ist eine Implementierung einer doppelt verketteten Liste (diese kennen Sie bereits aus ADS) in Java.

Jeder Eintrag wird als Knoten repräsentiert, der den eigentlichen Wert speichert und zusätzlich je einen Verweis auf den Vorgänger- und Nachfolger-Knoten hat.

Der Head der `LinkedList<T>` zeigt auf den Anfang der Liste, der Nachfolger des letzten Eintrag ist immer **`null`**.

Für den Zugriff auf ein Element muß man die `LinkedList<T>` traversieren und beginnt dabei am Anfang der Liste, deshalb ist ein Zugriff O(n).

Neue Elemente können effizient an das Ende der Liste eingefügt werden, indem der letzte Eintrag einen Verweis auf den neuen Knoten bekommt: O(1) (sofern man sich nicht nur den Start der Liste merkt, sondern auch das aktuelle Ende).

Wenn ein Element aus der Liste gelöscht wird, muss dieses zunächst gefundenen werden und die Liste danach neu verkettete werden: O(n).

Die Methoden einer `LinkedList<T>` sind nicht **`synchronized`**.

2.7.2.5 *Vector* und *Stack*

- `Vector<T>`:
  - Ein `Vector<T>` ähnelt einer `ArrayList<T>`
  - Das Array eines Vector wird jedoch verdoppelt, wenn es vergrößert wird
  - Die Methoden von `Vector<T>` sind **`synchronized`**

- `Stack<T>`:
  - Schnittstelle: “last in first out”-Prinzip
    - `push(T)`: Pushe Element oben auf den Stack
    - `pop(): T`: Hole oberstes Element vom Stack
  - Tatsächlich aber: **`class`** `Stack<E>` **`extends`** `Vector<E>`

2.7.2.6 Iterierbarkeit: *Iterable* und *Iterator*

```java
private List <Entity> entities = new ArrayList<>();

for (Entity e : entities) { ... }
entities.forEach(x -> ...);
```



Die Klassen aus der Collection-API implementieren das Interface `Iterable<T>` und sind damit iterierbar. Man kann sie darüber in einer klassischen **`for`**-Schleife nutzen, oder mit der Methode `forEach()` direkt über die Sammlung laufen.

Intern wird dabei ein passender `Iterator<T>` erzeugt, der die Elemente der Sammlung schrittweise mit der Methode `next()` zurückgibt. Mithilfe eines Cursor merkt sich der Iterator, bei welchem Eintrag der Datenstruktur er aktuell ist. Mit der Methode `hasNext()` kann geprüft werden, ob noch ein weiteres Element über den Iterator aus der Datenstruktur verfügbar ist.

Mit `remove()` kann das letzte zurückgegebene Element aus der Datenstruktur entfernt werden. Diese Methode ist im Interface als Default-Methode implementiert.

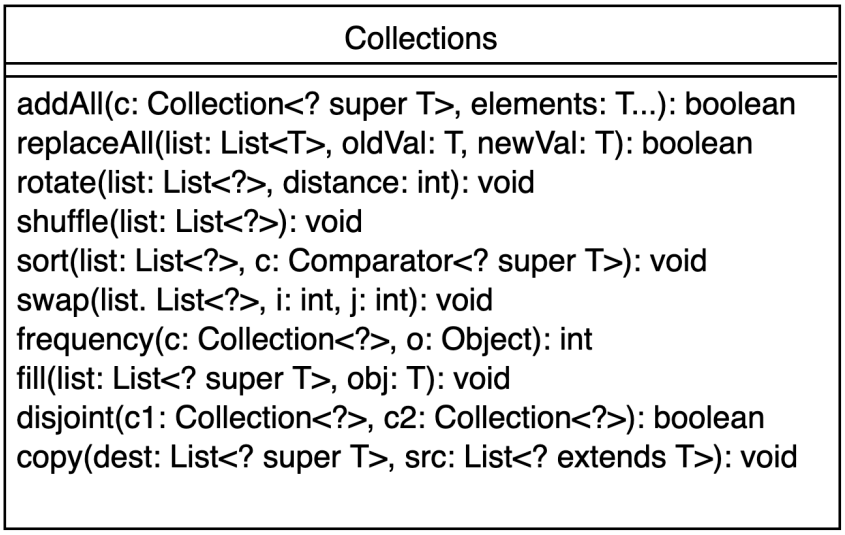
Damit kann man die Datenstrukturen auf eine von der Datenstruktur vorgegebene Weise ablaufen, beispielsweise einen Binärbaum.

Link zu einer netten Animation

Man kann auch selbst für eigene Klassen einen passenden `Iterator<T>` implementieren, der zum Ablaufen der Elemente der eigenen Klasse genutzt werden kann. Damit die eigene Klasse auch in einer **`for`**-Schleife genutzt werden kann, muss sie aber auch noch `Iterable<T>` implementieren.

Beispiel: iterator_example.*

2.7.2.7 Hilfsklasse *Collections*

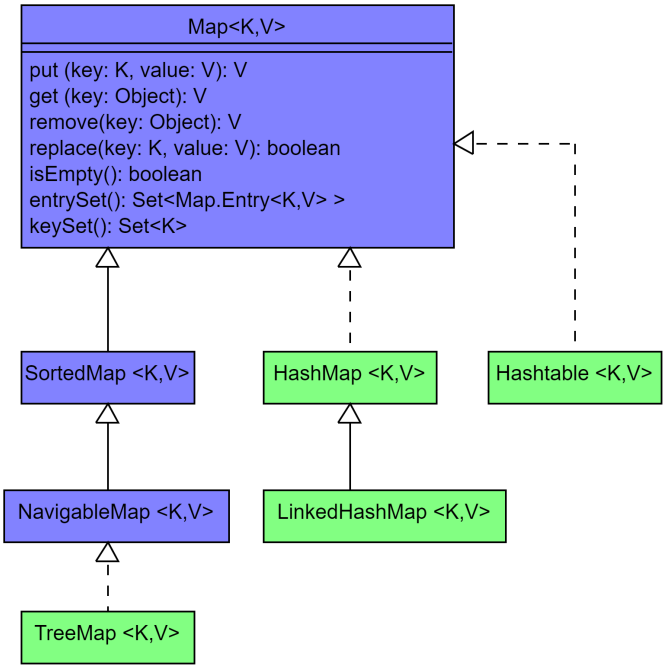


`Collections` ist eine Utility-Klasse mit statischen Methoden, die auf `Collection<T>`s ausgeführt werden. Diese Methoden nutzen das `Collection<T>`-Interface und/oder die `Iterable<T>`-Schnittstelle.

Siehe auch Class Collections.

Der Hintergrund für diese in Java nicht unübliche Aufsplittung in ein Interface und eine Utility-Klasse ist, dass bis vor kurzem Interface nur Schnittstellen definieren konnten. Erst seit einigen Java-Versionen kann in Interfaces auch Verhalten definiert werden (Default-Methoden). Aus heutiger Sicht würde man also vermutlich die statischen Methoden in der Klasse `Collections` eher direkt als Default-Methoden im Interface `Collection<T>` implementieren und bereitstellen, statt eine separate Utility-Klasse zu definieren.

2.7.2.8 *Map*



Hinweis: Die abstrakten (Zwischen-) Klassen wurden im obigen UML aus Gründen der Übersichtlichkeit *nicht* aufgeführt. Aus den selben Gründen sind auch nur ausgewählte Methoden aufgenommen worden.

Hinweis: Blau = Interface, Grün = Klasse.

Hinweis: Tatsächlich ist der Typ des Keys in den Methoden `get()` und `remove()` mit `Object` spezifiziert und nicht mit dem Typ-Parameter `K`. Das ist aus meiner Sicht eine Inkonsistenz in der API.

Eine `Map<K,V>` speichert Objekte als Key/Value-Paar mit den Typen `K` (Key) und `V` (Value).

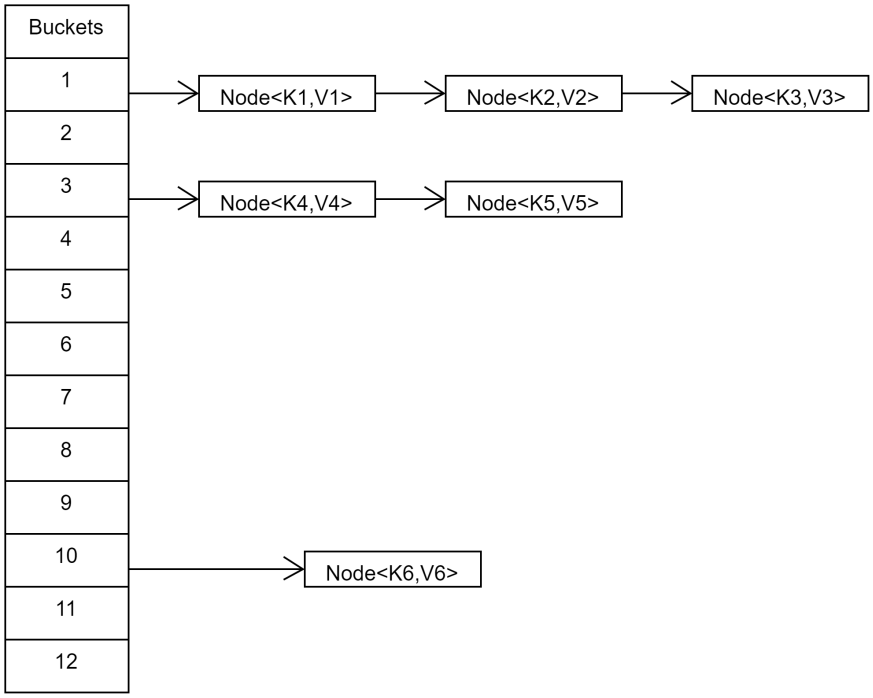
Dabei sind die Keys in einer Map einzigartig und werden verwendet, um auf das jeweilige Value zuzugreifen. Ein Value kann entsprechend (mit unterschiedlichen Keys) mehrfach im einer Map enthalten sein.

Es gibt eine Reihe verschiedener Implementierungen, die unterschiedliche Datenstrukturen einsetzen, beispielsweise:

- `HashMap<K,V>` hält keine Ordnung in den Einträgen. Verwendet den Hashwert, um Objekte zu speichern. Zugriff auf Einträge in einer `HashMap` ist O(1).
- `LinkedHashMap<K,V>` hält die Einträge in der Reihenfolge, in der sie eingefügt wurden.
- `TreeMap<K,V>` hält die Einträge in aufsteigender Reihenfolge.

Siehe auch Interface Map.

2.7.2.9 *HashMap*



Eine `HashMap<K,V>` speichert die Elemente in mehreren einfach verketteten Listen. Dafür verwendet sie die innere Klasse `Node<K,V>`.

Die Heads, die auf den Anfang einer Liste zeigen, werden in "Buckets" gespeichert. Initial besitzt eine HashMap 12 Buckets, diese werden bei Bedarf erweitert.

Um einen Eintrag hinzufügen, wird zunächst aus dem `hashCode()` des Key-Objektes mithilfe der Hash-Funktion der Index des Buckets berechnet. Ist der Bucket gefunden, wird geprüft, ob das Objekt dort schon vorkommt: Mit dem `hashCode()` des Key-Objektes werden alle Objekte in der Liste des Buckets verglichen. Wenn es Einträge mit dem selben `hashCode()` in der Liste gibt, wird mit `equals` geprüft, ob die Key-Objekte identisch sind. Ist dies der Fall, wird der existierende Eintrag überschrieben, anderenfalls wird der neue Eintrag an das Ende der Liste hinzugefügt.

Implementierungsdetail: Wenn die Listen zu groß werden, wird die Hashtabelle neu angelegt mit ungefähr der doppelten Anzahl der Einträge (Buckets) und die alten Einträge per *Re-Hash* neu verteilt (vgl. Class HashMap).

`HashMap<K,V>` Methoden sind nicht **`synchronized`**.

`HashMap<K,V>` unterstützt einen **`null`**-Key. Es darf beliebig viele **`null`**-Values geben.

Die Unterklasse `LinkedHashMap<K,V>` kann Ordnung zwischen den Elementen halten. Dafür wird eine doppelt verkettete Liste verwendet.

Beispiel: hash_example.HashCodeExample

2.7.2.10 *Hashtable*

- Nicht zu verwechseln mit der Datenstruktur: Hash-Tabellen (!)

- `Hashtable<K,V>` ist vergleichbar mit einer `HashMap<K,V>`
- `Hashtable<K,V>`-Methoden sind **`synchronized`**
- Kein Key oder Value darf **`null`** sein

2.7.2.11 Spielregeln für *equals()*, *hashCode()* und *compareTo()*

2.7.2.11.1 *equals()* **`boolean`** `equals(Object o)` ist eine Methode Klasse `Object` und wird genutzt, um Objekte auf Gleichheit zu prüfen. Die Default-Implementierung von `equals()` in `Object` vergleicht die beiden Objekte mit ==, gibt also nur dann **`true`** zurück, wenn die beiden zu vergleichenden Objekte die selbe Objekt-ID haben.

In der Praxis kann es sich anbieten, diese Methode zu überschreiben und eigene Kriterien für Gleichheit aufzustellen.

Dabei sind Spielregeln zu beachten (für nicht-**`null`** Objekte `x`, `y` und `z`):

1. Reflexivität: `x.equals(x)==` **`true`**
2. Symmetrie: `x.equals(y)== y.equals(x)`
3. Transitivität: Wenn `x.equals(y)==` **`true`** und `y.equals(z)==` **`true`**, dann auch `x.equals(z)==` **`true`**
4. Konsistenz: Mehrfache Aufrufe von `equals()` mit den selben Werten müssen immer das selbe Ergebnis liefern
5. `x.equals(`**`null`**`)==` **`false`**

2.7.2.11.2 *hashCode()* Die Methode **`int`** `hashCode()` gibt den Hash-Wert eines Objektes zurück. Der Hash-Wert eins Objektes wird genutzt, um dieses in einen Hash-basierten Container abzulegen bzw. zu finden.

Der Rückgabewert der Methode `hashCode()` für ein Objekt bleibt über die Laufzeit einer Anwendung immer identisch, solange sich die zur Prüfung der Gleichheit genutzten Attribute nicht ändern.

2.7.2.11.3 *compareTo()* Die Methode **`int`** `compareTo()` (Interface `Comparable<T>`) vergleicht Objekte und definiert damit eine Ordnung auf den Objekten. Während `equals()` für die Prüfung auf Gleichheit eingesetzt wird, wird `compareTo()` für die Sortierung von Objekten untereinander verwendet.

Spielregeln:

1. `x.compareTo(y)< 0` wenn `x` “kleiner” als `y` ist
2. `x.compareTo(y)> 0` wenn `x` “größer” als `y` ist
3. `x.compareTo(y)= 0` wenn `x` “gleich” als `y` ist
4. Symmetrie: `signum(x.compareTo(y))== -signum(y.compareTo(x))`
5. Transitivität: Wenn `x.compareTo(y)> 0` und `y.compareTo(z)> 0`, dann auch `x.compareTo(z)> 0`
6. Wenn `x.compareTo(y)== 0`, dann auch `signum(x.compareTo(z))== signum(y.compareTo(z))`

2.7.2.11.4 Der *equals()-hashCode()-compareTo()*-Vertrag Wird `equals()` überschrieben, sollte auch `hashCode()` (passend) überschrieben werden.

1. Wenn `x.equals(y)== true`, dann *muss* auch `x.hashCode()== y.hashCode()`
2. Wenn `x.equals(y)== false`, *sollte* `x.hashCode()!= y.hashCode()` sein (Unterschiedliche `hashCode()`-Werte für unterschiedliche Objekte verbessern allerdings die Leistung von Hash-Berechnungen, etwa in einer `HashMap<K,V>`!)
3. Es wird sehr empfohlen, dass `equals()` und `compareTo()` konsistente Ergebnisse liefern: `x.compareTo(y)== 0` gdw. `x.equals(y)== true` (Dies *muss* aber nicht zwingend eingehalten werden, sorgt dann aber u.U. für unerwartete Nebeneffekte beim Umgang mit `Collection<T>` und `Map<K,V>`!)

Beispiel: hash_example.HashCodeExample

2.7.2.12 Überblick

						Mutate (einfügen oder löschen) am		
Collection	**Sortiert?**	**Duplikate erlaubt?**	**synchronized?**	**null erlaubt?**	**Peek**	**Anfang**	**Mitte**	**Ende**
LinkedList	ja	ja	nein	ja	O(n)	O(1)	O(n)	O(1)
ArrayList	ja	ja	nein	ja	O(1)	O(n)	O(n)	O(1)
Vector	ja	ja	ja	ja	O(1)	O(n)	O(n)	O(1)
Stack	ja	ja	ja	ja	O(1)	O(1)	-	-
ArrayDeque	ja	ja	nein	Nein	O(1)	O(1)	-	O(1)
PriorityQueue	ja	ja	nein	Nein	O(1)	-	-	O(1)
HashSet	nein	nein	nein	ja	O(1)	O(1)	O(1)	O(1)
LinkedHashSet	ja	nein	nein	ja	O(1)	O(1)	O(1)	O(1)
TreeSet	ja	nein	nein	nein	O(log n)	O(log n)	O(log n)	O(log n)
Hashtable	nein	Keine doppelten Keys	ja	Nein	O(1)	O(1)	O(1)	O(1)
HashMap	nein	Keine doppelten Keys	nein	Einmal als Key, Mehrfach als Value	O(1)	O(1)	O(1)	O(1)
LinkedHashMap	ja	Keine doppelten Keys	nein	Einmal als Key, Mehrfach als Value	O(1)	O(1)	O(1)	O(1)
TreeMap	ja	nein	nein	nein	O(log n)	O(log n)	O(log n)	O(log n)

Komplexitätswerte beziehen sich auf den Regelfall. Sonderfälle wie das Vergrößern des Array einer `ArrayList<T>` können für temporär erhöhte Komplexität sorgen (das ist dem O-Kalkül aber egal).

2.7.2.13 Wrap-Up

- Interface `Collection<T>`: Schnittstelle für Datenstrukturen/Sammlungen zur Verwaltung einer Menge von Objekten
- Klasse `Collections`: Statische Hilfs-Methoden (anwendbar auf `Collection<T>`s)
- `Iterable<T>` liefert einen `Iterator<T>` zur Iteration über eine `Collection<T>`
- Interface `Map<K,V>`: Speichern von Key/Value-Paaren
- `equals()`-`hashCode()`-`compareTo()`-Vertrag beachten

Zum Nachlesen

- Oracle Corporation (2022b)

Lernziele

- k2: Was ist der Unterschied zwischen Collection<T> und List<T>?
- k2: Was ist der Unterschied zwischen einer List<T>, einer Queue<T> und einer Set<T>?
- k2: Nennen Sie charakteristische Merkmale von ArrayList<T>, LinkedList<T> und Vector<T>.
- k2: Was ist der Unterschied zwischen einer Queue<T> und einem Stack<T>?
- k2: Was ist eine Map<K,V>? Welche Vertreter kennen Sie?
- k3: Erklären Sie die 'Spielregeln' für die eigene Implementierung von equals().
- k3: Erklären Sie die 'Spielregeln' für die eigene Implementierung von hashCode().
- k3: Erklären Sie die 'Spielregeln' für die eigene Implementierung von compareTo().
- k3: Wie müssen und wie sollten equals(), hashCode() und compareTo() miteinander arbeiten?

Quizzes

- Quiz Collections (ILIAS)

2.7.3 Reguläre Ausdrücke

TL;DR
Mit Hilfe von regulären Ausdrücken kann man den Aufbau von Zeichenketten formal beschreiben. Dabei lassen sich direkt die gewünschten Zeichen einsetzen, oder man nutzt Zeichenklassen oder vordefinierte Ausdrücke. Teilausdrücke lassen sich gruppieren und über *Quantifier* kann definiert werden, wie oft ein Teilausdruck vorkommen soll. Die Quantifier sind per Default **greedy** und versuchen so viel wie möglich zu matchen.
Auf der Java-Seite stellt man reguläre Ausdrücke zunächst als `String` dar. Dabei muss darauf geachtet werden, dass ein Backslash im regulären Ausdruck im Java-String geschützt (*escaped*) werden muss, indem jeweils ein weiterer Backslash voran gestellt wird. Mit Hilfe der Klasse `java.util.regex.Pattern` lässt sich daraus ein Objekt mit dem kompilierten regulären Ausdruck erzeugen, was insbesondere bei mehrfacher Verwendung günstiger in der Laufzeit ist. Dem Pattern-Objekt kann man dann den Suchstring übergeben und bekommt ein Objekt der Klasse `java.util.regex.Matcher` (dort sind regulärer Ausdruck/Pattern und der Suchstring kombiniert). Mit den Methoden `Matcher#find` und `Matcher#matches` kann dann geprüft werden, ob das Pattern auf den Suchstring passt: `find` sucht dabei nach dem ersten Vorkommen des Patterns im Suchstring, `match` prüft, ob der gesamte String zum Pattern passt.

Videos

- VL RegExp
- Demo StringSplit
- Demo MatchFind
- Demo Quantifier
- Demo Groups
- Demo Backref

2.7.3.1 Suchen in Strings

Gesucht ist ein Programm zum Extrahieren von Telefonnummern aus E-Mails.

=> **Wie geht das?**

Leider gibt es unzählig viele Varianten, wie man eine Telefonnummer (samt Vorwahl und ggf. Ländervorwahl) aufschreiben kann:

```
030 - 123 456 789, 030-123456789, 030/123456789,
+49(30)123456-789, +49 (30) 123 456 - 789, ...
```

2.7.3.2 Definition Regulärer Ausdruck

> Ein **regulärer Ausdruck** ist eine Zeichenkette, die zur Beschreibung von Zeichenketten dient.

2.7.3.2.1 Anwendungen

- Finden von Bestandteilen in Zeichenketten
- Aufteilen von Strings in Tokens
- Validierung von textuellen Eingaben => “Eine Postleitzahl besteht aus 5 Ziffern”
- Compilerbau: Erkennen von Schlüsselwörtern und Strukturen und Syntaxfehlern

2.7.3.3 Einfachste reguläre Ausdrücke

Zeichenkette	Beschreibt
`x`	“x”
`.`	ein beliebiges Zeichen
`\t`	Tabulator
`\n`	Newline
`\r`	Carriage-return
`\\`	Backslash

2.7.3.3.1 Beispiel

- `abc` => “abc”
- `A.B` => “AAB” oder “A2B” oder …
- `a\\bc` => “a\bc”

2.7.3.3.2 Anmerkung In Java-Strings leitet der Backslash eine zu interpretierende Befehlssequenz ein. Deshalb muss der Backslash i.d.R. geschützt (“escaped”) werden. => Statt “`\n`” müssen Sie im Java-Code “`\\n`” schreiben!

2.7.3.4 Zeichenklassen

Zeichenkette	Beschreibt
`[abc]`	“a” oder “b” oder “c”
`[^abc]`	alles außer “a”, “b” oder “c” (Negation)
`[a-zA-Z]`	alle Zeichen von “a” bis “z” und “A” bis “Z” (Range)
`[a-z&&[def]]`	“d”,“e” oder “f” (Schnitt)
`[a-z&&[^bc]]`	“a” bis “z”, außer “b” und “c”: `[ad-z]` (Subtraktion)
`[a-z&&[^m-p]]`	“a” bis “z”, außer “m” bis “p”: `[a-lq-z]` (Subtraktion)

2.7.3.4.1 Beispiel

- `[abc]` => “a” oder “b” oder “c”
- `[a-c]` => “a” oder “b” oder “c”
- `[a-c][a-c]` => “aa”, “ab”, “ac”, “ba”, “bb”, “bc”, “ca”, “cb” oder “cc”
- `A[a-c]` => “Aa”, “Ab” oder “Ac”

2.7.3.5 Vordefinierte Ausdrücke

Zeichenkette	Beschreibt
`^`	Zeilenanfang
`$`	Zeilenende
`\d`	eine Ziffer: `[0-9]`
`\w`	beliebiges Wortzeichen: `[a-zA-Z_0-9]`
`\s`	Whitespace (Leerzeichen, Tabulator, Newline)
`\D`	jedes Zeichen außer Ziffern: `[^0-9]`
`\W`	jedes Zeichen außer Wortzeichen: `[^\w]`
`\S`	jedes Zeichen außer Whitespaces: `[^\s]`

2.7.3.5.1 Beispiel

- `\d\d\d\d\d` => “12345”
- `\w\wA` => “aaA”, “a0A”, “a_A”, …

2.7.3.6 Nutzung in Java

- `java.lang.String`:

```
1 public String[] split(String regex)
2 public boolean matches(String regex)
```

Demo: regexp.StringSplit

- `java.util.regex.Pattern`:

```
1 public static Pattern compile(String regex)
2 public Matcher matcher(CharSequence input)
```

  - Schritt 1: Ein Pattern compilieren (erzeugen) mit `Pattern#compile` => liefert ein Pattern-Objekt für den regulären Ausdruck zurück
  - Schritt 2: Dem Pattern-Objekt den zu untersuchenden Zeichenstrom übergeben mit `Pattern#matcher` => liefert ein Matcher-Objekt zurück, darin gebunden: Pattern (regulärer Ausdruck) und die zu untersuchende Zeichenkette

- `java.util.regex.Matcher`:

```
1 public boolean find()
2 public boolean matches()
3 public int groupCount()
4 public String group(int group)
```

  - Schritt 3: Mit dem Matcher-Objekt kann man die Ergebnisse der Anwendung des regulären Ausdrucks auf eine Zeichenkette auswerten

    Bedeutung der unterschiedlichen Methoden siehe folgende Folien

    `Matcher#group`: Liefert die Sub-Sequenz des Suchstrings zurück, die erfolgreich gematcht wurde (siehe unten “Fangende Gruppierungen”)

Hinweis:

In Java-Strings leitet der Backslash eine zu interpretierende Befehlssequenz ein. Deshalb muss der Backslash i.d.R. extra geschützt (“escaped”) werden.

=> Statt “`\n`” (regulärer Ausdruck) müssen Sie im Java-String “`\\n`” schreiben!

=> Statt “`a\\bc`” (regulärer Ausdruck, passt auf die Zeichenkette “a\bc”) müssen Sie im Java-String “`a\\\\bc`” schreiben!

Demo: regexp.MatchFind

2.7.3.7 Unterschied zw. Finden und Matchen

- `Matcher#find`:

  Regulärer Ausdruck muss im Suchstring **enthalten** sein.
  => Suche nach **erstem Vorkommen**

- `Matcher#matches`:

  Regulärer Ausdruck muss auf **kompletten** Suchstring passen.

2.7.3.7.1 Beispiel

- Regulärer Ausdruck: `abc`, Suchstring: “blah blah abc blub”
  - `Matcher#find`: erfolgreich
  - `Matcher#matches`: kein Match - Suchstring entspricht nicht dem Muster

2.7.3.8 Quantifizierung

Zeichenkette	Beschreibt
`X?`	ein oder kein “X”
`X*`	beliebig viele “X” (inkl. kein “X”)
`X+`	mindestens ein “X”, ansonsten beliebig viele “X”
`X{n}`	exakt n Vorkommen von “X”
`X{n,}`	mindestens n Vorkommen von “X”
`X{n,m}`	zwischen n und m Vorkommen von “X”

2.7.3.8.1 Beispiel

- `\d{5}` => “12345”
- `-?\d+\.\d*` => ???

2.7.3.9 Interessante Effekte

```
1 Pattern p = Pattern.compile("A.*A");
2 Matcher m = p.matcher("A 12 A 45 A");
3
4 if (m.matches())
5     String result = m.group(); // ???
```

Demo: regexp.Quantifier

`Matcher#group` liefert die Inputsequenz, auf die der Matcher angesprochen hat. Mit `Matcher#start` und `Matcher#end` kann man sich die Indizes des ersten und letzten Zeichens des Matches im Eingabezeichenstrom geben lassen. D.h. für einen Matcher `m` und eine Eingabezeichenkette `s` ist `m.group()` und `s.substring(m.start(), m.end())` äquivalent.

Da bei `Matcher#matches` das Pattern immer auf den gesamten Suchstring passen muss, verwundert das Ergebnis für `Matcher#group` nicht. Bei `Matcher#find` wird im Beispiel allerdings ebenfalls der gesamte Suchstring “gefunden” … Dies liegt am “*greedy*” Verhalten der Quantifizierer.

2.7.3.10 Nicht gierige Quantifizierung mit “?”

Zeichenkette	Beschreibt
`X*?`	non-greedy Variante von `X*`
`X+?`	non-greedy Variante von `X+`

2.7.3.10.1 Beispiel

- Suchstring "A 12 A 45 A":
  - `A.*A` findet/passt auf "A 12 A 45 A"
    normale **greedy** Variante
  - `A.*?A`
    - findet "A 12 A"
    - passt auf "A 12 A 45 A" (!)

    non-greedy Variante der Quantifizierung; `Matcher#matches` muss trotzdem auf den gesamten Suchstring passen!

2.7.3.11 (Fangende) Gruppierungen `Studi{2}` passt nicht auf "StudiStudi" (!)

Quantifizierung bezieht sich auf das direkt davor stehende Zeichen. Ggf. Gruppierungen durch Klammern verwenden!

Zeichenkette	Beschreibt
`X|Y`	X oder Y
`(C)`	Gruppierung

2.7.3.11.1 Beispiel

- `(A)(B(C))`
  - Gruppe 0: `ABC`
  - Gruppe 1: `A`
  - Gruppe 2: `BC`
  - Gruppe 3: `C`

Die Gruppen heißen auch "fangende" Gruppen (engl.: *"capturing groups"*).

Damit erreicht man eine Segmentierung des gesamten regulären Ausdrucks, der in seiner Wirkung aber nicht durch die Gruppierungen geändert wird. Durch die Gruppierungen von Teilen des regulären Ausdrucks erhält man die Möglichkeit, auf die entsprechenden Teil-Matches (der Unterausdrücke der einzelnen Gruppen) zuzugreifen:

- `Matcher#groupCount`: Anzahl der "fangenden" Gruppen im regulären Ausdruck

- `Matcher#group(i)`: Liefert die Subsequenz der Eingabezeichenkette zurück, auf die die jeweilige Gruppe gepasst hat. Dabei wird von links nach rechts durchgezählt, beginnend bei 1(!).

  Konvention: Gruppe 0 ist das gesamte Pattern, d.h. `m.group(0)== m.group();` …

Hinweis: Damit der Zugriff auf die Gruppen klappt, muss auch erst ein Match gemacht werden, d.h. das Erzeugen des Matcher-Objekts reicht noch nicht, sondern es muss auch noch ein `matcher.find()` oder `matcher.matches()` ausgeführt werden. Danach kann man bei Vorliegen eines Matches auf die Gruppen zugreifen.

`(Studi){2}` => "StudiStudi"

Demo: regexp.Groups

2.7.3.12 Gruppen und Backreferences

Matche zwei Ziffern, gefolgt von den selben zwei Ziffern

`(\d\d)\1`

- Verweis auf bereits gematchte Gruppen: `\num`

  `num` Nummer der Gruppe (1 … 9)

  => Verweist nicht auf regulären Ausdruck, sondern auf jeweiligen Match!

  Anmerkung: Laut Literatur/Doku nur 1 … 9, in Praxis geht auch mehr per Backreference …

- Benennung der Gruppe: `(?<name>X)`

  `X` ist regulärer Ausdruck für Gruppe, spitze Klammern wichtig

  => Backreference: `\k<name>`

Demo: regexp.Backref

2.7.3.13 Beispiel Gruppen und Backreferences

Regulärer Ausdruck: Namen einer Person matchen, wenn Vor- und Nachname identisch sind.

Lösung: `([A-Z][a-zA-Z]*)\s\1`

2.7.3.14 Umlaute und reguläre Ausdrücke

- Keine vordefinierte Abkürzung für Umlaute (wie etwa `\d`)
- Umlaute nicht in `[a-z]` enthalten, aber in `[a-ü]`

```
1 "helloüA".matches(".*?[ü]A");
2 "azäöüß".matches("[a-ä]");
3 "azäöüß".matches("[a-ö]");
4 "azäöüß".matches("[a-ü]");
5 "azäöüß".matches("[a-ß]");
```

- Strings sind Unicode-Zeichenketten

  => Nutzung der passenden Unicode Escape Sequence `\uFFFF`

```
1 System.out.println("\u0041 :: A");
2 System.out.println("helloüA".matches(".*?A"));
3 System.out.println("helloüA".matches(".*?\u0041"));
4 System.out.println("helloü\u0041".matches(".*?A"));
```

- RegExp vordefinieren und mit Variablen zusammenbauen ala Perl nicht möglich => Umweg String-Repräsentation

2.7.3.15 Wrap-Up

- RegExp: Zeichenketten, die andere Zeichenketten beschreiben
- `java.util.regex.Pattern` und `java.util.regex.Matcher`
- Unterschied zwischen `Matcher#find` und `Matcher#matches`!
- Quantifizierung ist möglich, aber **greedy** (Default)

Zum Nachlesen

- Oracle Corporation (2022c)

Lernziele

- k1: Wichtigste Methoden von java.util.regex.Pattern und java.util.regex.Matcher
- k2: Unterschied zwischen Matcher#find und Matcher#matches
- k2: Unterscheidung zwischen greedy und non-greedy Verhalten
- k3: Bildung einfacher regulärer Ausdrücke
- k3: Nutzung von Zeichenklassen und deren Negation
- k3: Nutzung der vordefinierten regulären Ausdrücke
- k3: Nutzung von Quantifizierern
- k3: Zusammenbauen von komplexen Ausdrücken (u.a. mit Gruppen)

Quizzes

- Quiz RegExp (ILIAS)

Challenges
In den Vorgaben finden Sie in der Klasse `Lexer` eine einfache Implementierung eines Lexers, worin ein einfaches Syntax-Highlighting für Java-Code realisiert ist.
Dazu arbeitet der Lexer mit sogenannten "Token" (Instanzen der Klasse `Token`). Diese haben einen regulären Ausdruck, um bestimmte Teile im Code zu erkennen, beispielsweise Keywords oder Kommentare und anderes. Der Lexer wendet alle Token auf den aktuellen Eingabezeichenstrom an (Methode `Token#test()`), und die Token prüfen mit "ihrem" regulären Ausdruck, ob die jeweils passende Eingabesequenz vorliegt. Die regulären Ausdrücke übergeben Sie dem `Token`-Konstruktor als entsprechendes `Pattern`-Objekt.
Neben dem jeweiligen Pattern kennt jedes Token noch eine `matchingGroup`: Dies ist ein Integer, der die relevante Matching-Group im regulären Ausdruck bezeichnet. Wenn Sie keine eigenen Gruppen in einem regulären Ausdruck eingebaut haben, nutzen Sie hier einfach den Wert 0.
Zusätzlich kennt jedes Token noch die Farbe für das Syntax-Highlighting in der von uns als Vorgabe realisierten Swing-GUI (Instanz von `Color`).
Erstellen Sie passende `Token`-Instanzen mit entsprechenden Pattern für die folgenden Token:

- Einzeiliger Kommentar: beginnend mit `//` bis zum Zeilenende
- Mehrzeiliger Kommentar: alles zwischen `/*` und dem nächsten `*/`
- Javadoc-Kommentar: alles zwischen `/**` und dem nächsten `*/`
- Strings: alles zwischen `"` und dem nächsten `"`
- Character: genau ein Zeichen zwischen `'` und `'`
- Keywords: **`package`**, **`import`**, **`class`**, **`public`**, **`private`**, **`final`**, **`return`**, **`null`**, **`new`** (jeweils freistehend, also nicht "newx" o.ä.)
- Annotation: beginnt mit `@`, enthält Buchstaben oder Minuszeichen

Die Token-Objekte fügen Sie im Konstruktor der Klasse `Lexer` durch den Aufruf der Methode `tokenizer.add(mytoken)` hinzu. Sie können Sich an den Kommentaren im `Lexer`-Konstruktor orientieren.
Sollten Token ineinander geschachtelt sein, erkennt der Lexer dies automatisch. Sie brauchen sich keine Gedanken dazu machen, in welcher Reihenfolge die Token eingefügt und abgearbeitet werden. Beispiel: Im regulären Ausdruck für den einzeiligen Kommentar brauchen Sie keine Keywords, Annotationen, Strings usw. erkennen.

2.7.4 Annotationen

TL;DR
Annotationen sind Metadaten zum Programm: Sie haben keinen (direkten) Einfluss auf die Ausführung des annotierten Codes, sondern enthalten Zusatzinformationen über ein Programm, die selbst nicht Teil des Programms sind. Verschiedene Tools werten Annotationen aus, beispielsweise der Compiler, Javadoc, JUnit, …
Annotationen können auf Deklarationen (Klassen, Felder, Methoden) angewendet werden und werden meist auf eine eigene Zeile geschrieben (Konvention).
Annotationen können relativ einfach selbst erstellt werden: Die Definition ist fast wie bei einem Interface. Zusätzlich kann man noch über Meta-Annotationen die Sichtbarkeit, Verwendbarkeit und Dokumentation einschränken. Annotationen können zur Übersetzungszeit mit einem Annotation-Processor verarbeitet werden oder zur Laufzeit über Reflection ausgewertet werden.

Videos

- VL Annotationen
- Demo Annotationen: `@Override`, `@Deprecated`
- Demo `@NotNull`
- Demo Annotationen selbst gebastelt
- Demo Annotation-Prozessor

2.7.4.1 Was passiert hier?

```java
public class A {
    public String getInfo() { return "Klasse A"; }
}

public class B extends A {
    public String getInfo(String s) { return s + "Klasse B"; }

    public static void main(String[] args) {
        B s = new B();
        System.out.println(s.getInfo("Info: "));
    }
}
```

Beispiel: annotations.B

Hilft `@Override`?

Tja, da sollte wohl die Methode `B#getInfo` die geerbte Methode `A#getInfo` **überschreiben**. Dummerweise wird hier die Methode aber nur **überladen** (mit entsprechenden Folgen beim Aufruf)!

Ein leider relativ häufiges Versehen, welches u.U. schwer zu finden ist. Annotationen (hier `@Override`) können dagegen helfen - der Compiler "weiß" dann, dass wir überschreiben wollen und meckert, wenn wir das nicht tun.

IDEs wie Eclipse können diese Annotation bereits beim Erstellen einer Klasse generieren: `Preferences > Java > Code Style > Add @Override annotation ...`.

2.7.4.2 Annotationen: Metadaten für Dritte

- **Zusatzinformationen** für Tools, Bibliotheken, …
- Kein direkter Einfluss auf die Ausführung des annotierten Codes

- Beispiele:
  - Compiler (JDK): `@Override`, `@Deprecated`, …
  - Javadoc: `@author`, `@version`, `@see`, `@param`, `@return`, …
  - JUnit: `@Test`, `@Before`, `@BeforeClass`, `@After`, `@AfterClass`
  - IntelliJ: `@NotNull`, `@Nullable`
  - Checker Framework: `@NonNull`, `@Nullable`, …

- Project Lombok: `@Getter`, `@Setter`, `@NonNull`, …
- Webservices: `@WebService`, `@WebMethod`
- …

Demo: annotations.B: @Override, @Deprecated

Jetzt schauen wir uns erst einmal die Auswirkungen von `@Override` und `@Deprecated` auf den Compiler (via Eclipse) an. Anschließend lernen Sie die Dokumentation mittels Javadoc-Annotationen kennen.

Das Thema JUnit ist in einer anderen VL dran. Webservices ereilen Sie dann in späteren Semestern :-)

2.7.4.2.1 @Override Die mit `@Override` annotierte Methode überschreibt eine Methode aus der Oberklasse oder implementiert eine Methode einer Schnittstelle. Dies wird durch den Compiler geprüft und ggf. mit einer Fehlermeldung quittiert.

`@Override` ist eine im JDK im Paket `java.lang` enthaltene Annotation.

2.7.4.2.2 @Deprecated Das mit `@Deprecated` markierte Element ist veraltet ("*deprecated*") und sollte nicht mehr benutzt werden. Typischerweise werden so markierte Elemente in zukünftigen Releases aus der API entfernt …

Die Annotation `@Deprecated` wird direkt im Code verwendet und entspricht der Annotation `@deprecated` im Javadoc. Allerdings kann letzteres nur von Javadoc ausgewertet werden.

`@Deprecated` ist eine im JDK im Paket `java.lang` enthaltene Annotation.

2.7.4.2.3 Weitere Annotationen aus *java.lang* Im Paket `java.lang` finden sich weitere Annotationen. Mit Hilfe von `@SuppressWarnings` lassen sich bestimmte Compilerwarnungen unterdrücken (**so etwas sollte man NIE tun!**), und mit `@FunctionalInterface` lassen sich Schnittstellen auszeichnen, die genau eine (abstrakte) Methode besitzen (Verweis auf spätere Vorlesung).

Weitere Annotationen aus dem JDK finden sich in den Paketen `java.lang.annotation` und `javax.annotation`.

2.7.4.3 Dokumentation mit Javadoc

```java
/**
 * Beschreibung Beschreibung Beschreibung
 *
 * @param date Tag, Wert zw. 1 .. 31
 * @return true, falls Datum gesetzt wurde; false sonst
 * @see java.util.Calendar
 * @deprecated As of JDK version 1.1
 */
public boolean setDate(int date) {
    setField(Calendar.DATE, date);
}
```

Die Dokumentation mit Javadoc hatten wir uns bereits in der Einheit "Javadoc" angesehen.

Hier noch einmal exemplarisch die wichtigsten Elemente, die an "**public**" sichtbaren Methoden verwendet werden.

Beispiel: annotations.B (Javadoc)

2.7.4.4 @NotNull mit IntelliJ

IntelliJ bietet im Paket `org.jetbrains.annotations` u.a. die Annotation `@NotNull` an.

Damit lassen sich Rückgabewerte von Methoden sowie Variablen (Attribute, lokale Variablen, Parameter) markieren: Diese dürfen nicht **null** werden.

IntelliJ prüft beim Compilieren, dass diese Elemente nicht **null** werden und warnt gegebenenfalls (zur Compilezeit). Zusätzlich baut IntelliJ entsprechende Assertions in den Code ein, die zur Laufzeit einen **null**-Wert abfangen und dann das Programm abbrechen.

Dadurch können entsprechende Dokumentationen im Javadoc und/oder manuelle Überprüfungen im Code entfallen. Außerdem hat man durch die Annotation gewissermaßen einen sichtbaren Vertrag (*Contract*) mit den Nutzern der Methode. Bei einem Aufruf mit **null** würde dieser Contract verletzt und eine entsprechende Exception geworfen (automatisch) statt einfach das Programm und die JVM "abzuschießen".

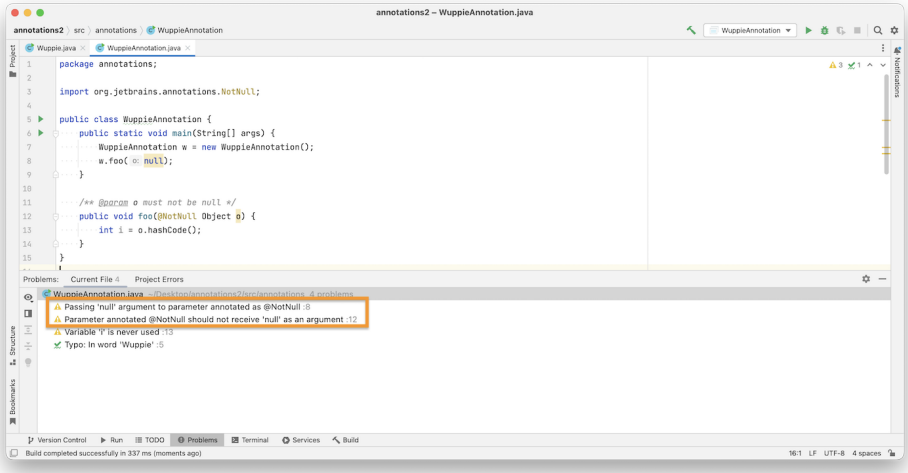
Nachteil: Die entsprechende Bibliothek muss bei allen Entwicklern vorhanden und in das Projekt eingebunden sein.

```java
/* o should not be null */
public void bar(Object o) {
    int i;
    if (o != null) {
        i = o.hashCode();
    }
}
```

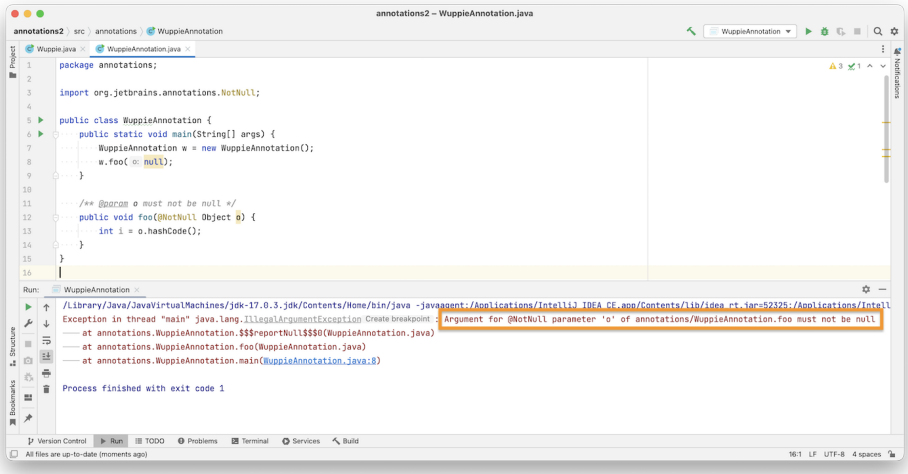
```java
/* o must not be null */
public void foo(@NotNull Object o) {
    // assert(o != null);  //  Wirkung (von IntelliJ eingefügt)
    int i = o.hashCode();
}
```

Demo: annotations.WuppieAnnotation: @NotNull

2.7.4.4.1 IntelliJ inferiert mit @NotNull mögliche *null*-Werte



2.7.4.4.2 IntelliJ baut bei @NotNull passende Assertions ein



2.7.4.5 Eigene Annotationen erstellen

```java
public @interface MyFirstAnnotation {}

public @interface MyThirdAnnotation {
    String author();

    int vl() default 1;
}

@MyFirstAnnotation
@MyThirdAnnotation(author = "Carsten Gips", vl = 3)
public class C {}
```

Demo: annotations.C

2.7.4.5.1 Definition einer Annotation Definition einer Annotation wie Interface, aber mit "@"-Zeichen vor dem `interface`-Schlüsselwort

2.7.4.5.2 Parameter für Annotation Parameter für Annotation werden über entsprechende Methoden-Deklaration realisiert

- "Rückgabetyp" der deklarierten "Methode" ist der erlaubte Typ der später verwendeten Parameter
- Name der "Methoden" wird bei der Belegung der Parameter verwendet, beispielsweise `author = ...`
- Vereinfachung: "Methodenname" `value` erlaubt das Weglassen des Schlüsselworts bei der Verwendung:

```
1 public @interface MySecondAnnotation {
2     String value();
3 }
4
5 @MySecondAnnotation("wuppie")
6 public class D {}
7
8 @MySecondAnnotation(value = "wuppie")
9 public class E {}
```

- Defaultwerte mit dem nachgestellten Schlüsselwort `default` sowie dem Defaultwert selbst

2.7.4.5.3 Javadoc Soll die Annotation in der Javadoc-Doku dargestellt werden, muss sie mit der Meta-Annotation `@Documented` ausgezeichnet werden (aus `java.lang.annotation.Documented`)

Hinweis: Die Annotation wird lediglich in die Doku aufgenommen, d.h. es erfolgt keine weitere Verarbeitung oder Hervorhebung o.ä.

2.7.4.5.4 Wann ist eine Annotation sichtbar (Beschränkung der Sichtbarkeit) Annotationen werden vom Compiler und/oder anderen Tools ausgewertet. Man kann entsprechend die Sichtbarkeit einer Annotation beschränken: Sie kann ausschließlich im Source-Code verfügbar sein, sie kann in der generierten Class-Datei eingebettet sein oder sie kann sogar zur Laufzeit (mittels *Reflection*, vgl. spätere Vorlesung) ausgelesen werden.

Beschränkung der Sichtbarkeit: Meta-Annotation `@Retention` aus `java.lang.annotation.Retention`

- `RetentionPolicy.SOURCE`: Nur Bestandteil der Source-Dateien, wird nicht in kompilierten Code eingebettet
- `RetentionPolicy.CLASS`: Wird vom Compiler in die Class-Datei eingebettet, steht aber zur Laufzeit *nicht* zur Verfügung (Standardwert, wenn nichts angegeben)
- `RetentionPolicy.RUNTIME`: Wird vom Compiler in die Class-Datei eingebettet und steht zur Laufzeit zur Verfügung und kann via *Reflection*[1] ausgelesen werden

Ohne explizite Angabe gilt für die selbst definierte Annotation die Einstellung `RetentionPolicy.CLASS`.

[1] Reflection ist Thema einer späteren Vorlesung

2.7.4.5.5 Wo darf eine Annotation verwendet werden

2.7.4.5.6 Anwendungsmöglichkeiten von Annotationen im Code

```java
@ClassAnnotation
public class Wuppie {
    @InstanceFieldAnnotation
    private String foo;

    @ConstructorAnnotation
    public Wuppie() {}

    @MethodAnnotation1
    @MethodAnnotation2
    @MethodAnnotation3
    public void fluppie(@ParameterAnnotation final Object arg1) {
        @VariableAnnotation
        final String bar = (@TypeAnnotation String) arg1;
    }
}
```

2.7.4.5.7 Einschränkung des Einsatzes eines Annotation Für jede Annotation kann eingeschränkt werden, wo (an welchen Java-Elementen) sie verwendet werden darf.

Beschränkung der Verwendung: Meta-Annotation `@Target` aus `java.lang.annotation.Target`

- `ElementType.TYPE`: alle Typdeklarationen: Klassen, Interfaces, Enumerations, …
- `ElementType.CONSTRUCTOR`: nur Konstruktoren
- `ElementType.METHOD`: nur Methoden
- `ElementType.FIELD`: nur statische Variablen und Objektvariablen
- `ElementType.PARAMETER`: nur Parametervariablen
- `ElementType.PACKAGE`: nur an Package-Deklarationen

Ohne explizite Angabe ist die selbst definierte Annotation für alle Elemente verwendbar.

2.7.4.6 Annotationen bei Compilieren bearbeiten: Java Annotation-Prozessoren Der dem `javac`-Compiler vorgelegte Source-Code wird eingelesen und in einen entsprechenden Syntax-Tree (*AST*) transformiert (dazu mehr im Master im Modul “Compilerbau” :)

Anschließend können sogenannte “Annotation Processors” über den AST laufen und ihre Analysen machen und/oder den AST modifizieren. (Danach kommen die üblichen weiteren Analysen und die Code-Generierung.)

(Vgl. OpenJDK: Compilation Overview.)

An dieser Stelle kann man sich einklinken und einen eigenen Annotation-Prozessor ausführen lassen. Zur Abgrenzung: Diese Auswertung der Annotationen findet zur **Compile-Zeit** statt! In einer späteren Vorlesung werden wir noch über die Auswertung zur **Laufzeit** sprechen: *Reflection*.

Im Prinzip muss man lediglich das Interface `javax.annotation.processing.Processor` implementieren oder die abstrakte Klasse `javax.annotation.processing.AbstractProcessor` erweitern.

Für die Registrierung im `javac` muss im Projekt (oder Jar-File) die Datei `META-INF/services/javax.annotation.processing.Processor` angelegt werden, die den vollständigen Namen des Annotation-Prozessors enthält. Dieser Annotation-Prozessor wird dann vom `javac` aufgerufen und läuft in einer eigenen JVM. Er kann die Annotationen, für die er registriert ist, auslesen und verarbeiten und neue Java-Dateien schreiben, die wiederum eingelesen und compiliert werden.

Im nachfolgenden Beispiel beschränke ich mich auf das Definieren und Registrieren eines einfachen Annotation-Prozessors, der lediglich die Annotationen liest.

```
1  @SupportedAnnotationTypes("annotations.MySecondAnnotation")
2  @SupportedSourceVersion(SourceVersion.RELEASE_17)
3  public class Foo extends AbstractProcessor {
4      @Override
5      public boolean process(Set<? extends TypeElement> as, RoundEnvironment re)
           {
6          for (TypeElement annot : as) {
7              for (Element el : re.getElementsAnnotatedWith(annot)) {
8                  processingEnv.getMessager().printMessage(Diagnostic.Kind.NOTE,
9                      "found @MySecondAnnotation at " + el);
10             }
11         }
12         return true;
13     }
14 }
```

Demo: annotations.C und annotations.Foo, META-INF

1. Der Annotation-Processor sollte von `AbstractProcessor` ableiten
2. Über `@SupportedAnnotationTypes` teilt man mit, für welche Annotationen sich der Prozessor interessiert (d.h. für welche er aufgerufen wird); "`*`" oder eine Liste mit String ist auch möglich
3. Mit `@SupportedSourceVersion` wird die (höchste) unterstützte Java-Version angegeben (neuere Versionen führen zu einer Warnung)
4. Die Methode `process` erledigt die Arbeit:
   - Der erste Parameter enthält alle gefundenen Annotationen, für die der Processor registriert ist
   - Der zweite Parameter enthält die damit annotierten Elemente
   - Iteration: Die äußere Schleife läuft über alle gefundenen Annotationen, die innere über die mit der jeweiligen Annotation versehenen Elemente
   - Jetzt könnte man mit den Elementen etwas sinnvolles anfangen, beispielsweise alle Attribute sammeln, die mit `@Getter` markiert sind und für diese neuen Code generieren
   - Im Beispiel wird lediglich der eigene Logger (`processingEnv.getMessager()`) aufgerufen, um beim Compiliervorgang eine Konsolenmeldung zu erzeugen …
5. Der Annotation-Processor darf keine Exception werfen, da sonst der Compiliervorgang abgebrochen würde. Zudem wäre der Stack-Trace der des Annotation-Processors und nicht der des compilierten Programms … Stattdessen wird ein Boolean zurückgeliefert, um anzudeuten, ob die Verarbeitung geklappt hat.

Für ein umfangreicheres Beispiel mit Code-Erzeugung vergleiche beispielsweise die Artikelserie unter cloudogu.com/en/blog/Java-Annotation-Processors_1-Intro. Siehe auch OpenJDK: Compilation Overview.

Im Projekt muss jetzt noch der Ordner `META-INF/services/` angelegt werden mit der Datei `javax.annotation.processing.Processor`. Deren Inhalt ist für das obige Beispiel die Zeile `annotations`

`.Foo`. Damit ist der Annotation-Processor `annotations.Foo` für das Übersetzen im eigenen Projekt registriert.

Zum Compilieren des Annotation-Processors selbst ruft man beispielsweise folgenden Befehl auf:

```
1 javac -cp . -proc:none annotations/Foo.java
```

Die Option `-proc:none` sorgt für das Beispiel dafür, dass beim Compilieren des Annotation-Processors dieser nicht bereits aufgerufen wird (was sonst wg. der Registrierung über `META-INF/services/javax.annotation.processing.Processor` passieren würde).

Zum Compilieren der Klasse `C` kann man wie sonst auch den Befehl nutzen:

```
1 javac -cp . annotations/C.java
```

Dabei läuft dann der Annotation-Processor `annotations.Foo` und erzeugt beim Verarbeiten von `annotations.C` die folgende Ausgabe:

```
1 Note: found @MySecondAnnotation at main(java.lang.String[])
```

2.7.4.7 Wrap-Up

- Annotationen: Metadaten zum Programm
  - **Zusatzinformationen** über ein Programm, aber nicht selbst Teil des Programms
  - **Kein (direkter) Einfluss** auf die **Ausführung** des annotierten Codes
- Typische Anwendungen: Compiler-Hinweise, Javadoc, Tests
  - Compiler: Erkennen von logischen Fehlern, Unterdrücken von Warnungen => `java.lang`: `@Override`, `@Deprecated`, `@SuppressWarnings`
  - Javadoc: Erkennen von Schlüsselwörtern (`@author`, `@return`, `@param`, ...)
  - JUnit: Erkennen von Tests-Methoden (`@Test`)
  - ...
- Annotationen können auf Deklarationen (Klassen, Felder, Methoden) angewendet werden
- Annotationen können relativ einfach selbst erstellt werden
  - Definition fast wie ein Interface
  - Einstellung der Sichtbarkeit und Verwendbarkeit und Dokumentation über Meta-Annotationen
- Verarbeitung von Annotationen zur Compilier-Zeit mit Annotation-Processor
- Verarbeitung von Annotationen zur Laufzeit mit Reflection (siehe spätere VL)

Zum Nachlesen

- Ullenboom (2021, 10.8 und 23.4)
- Oracle Corporation (2022c)
- Oracle Corporation (2022b)

Lernziele

- k2: Begriff der Annotation erklären können am Beispiel
- k3: Anwendung von `@Override` sowie der Javadoc-Annotationen
- k3: Erstellen eigener Annotationen sowie Einstellen der Sichtbarkeit und Verwendbarkeit
- k3: Erstellen eigener einfacher Annotation-Processors

Quizzes

- Quiz Annotationen (ILIAS)

Challenges

Schreiben Sie drei eigene Annotationen:

- `@MeineKlasse` darf nur an Klassendefinitionen stehen und speichert den Namen des Autoren ab.
- `@MeineMethode` darf nur an Methoden stehen.
- `@TODO` darf an Methoden und Klassen stehen, ist aber nur in den Source-Dateien sichtbar.

Implementieren Sie einen Annotation-Prozessor, welcher Ihren Quellcode nach der `@MeineKlasse`-Annotation durchsucht und dann den Namen der Klasse und den Namen des Autors ausgibt.
Zeigen Sie die Funktionen anhand einer Demo.

2.7.5 Reflection

TL;DR

Mit Hilfe der Reflection-API kann man Programme zur **Laufzeit** inspizieren und Eigenschaften von Elementen wie Klassen oder Methoden abfragen, aber auch Klassen instantiieren und Methoden aufrufen, die eigentlich auf `private` gesetzt sind oder die beispielsweise mit einer bestimmten Annotation markiert sind.
Die Laufzeitumgebung erzeugt zu jedem Typ ein Objekt der Klasse `java.lang.Class`. Über dieses `Class`-Objekt einer Klasse können dann Informationen über diese Klasse abgerufen werden, beispielsweise welche Konstruktoren, Methoden und Attribute es gibt.
Man kann über auch Klassen zur Laufzeit nachladen, die zur Compile-Zeit nicht bekannt waren. Dies bietet sich beispielsweise für User-definierte Plugins an.
Reflection ist ein mächtiges Werkzeug. Durch das Arbeiten mit Strings und die Interaktion/Inspektion zur *Laufzeit* verliert man aber viele Prüfungen, die der Compiler normalerweise zur Compile-Zeit vornimmt. Auch das Refactoring wird dadurch eher schwierig.

Videos

- VL Reflection
- Demo Reflection
- Demo Class-Loader

2.7.5.1 Ausgaben und Einblicke zur Laufzeit

```
1 public class FactoryBeispielTest {
2     @Test
```

```
3       public void testGetTicket() {
4           fail("not implemented");
5       }
6   }
```

```
1   @Target(value = ElementType.METHOD)
2   @Retention(value = RetentionPolicy.RUNTIME)
3   public @interface Wuppie {}
```

Reflection wird allgemein genutzt, um zur Laufzeit von Programmen Informationen über Klassen/Methoden/… zu bestimmen. Man könnte damit auch das Verhalten der laufenden Programme ändern oder Typen instantiieren und/oder Methoden aufrufen …

Wenn Sie nicht (mehr) wissen, wie man eigene Annotationen definiert, schauen Sie doch bitte einfach kurz im Handout zu Annotationen nach :-)

2.7.5.2 Wer bin ich? . . . Informationen über ein Programm (zur Laufzeit)

`java.lang.Class`: Metadaten über Klassen

```
1   // usual way of life
2   Studi heiner = new Studi();
3   heiner.hello();
4
5   // let's use reflection
6   try {
7       Object eve = Studi.class.getDeclaredConstructor().newInstance();
8       Method m = Studi.class.getDeclaredMethod("hello");
9       m.invoke(eve);
10  } catch (ReflectiveOperationException ignored) {}
```

Für jeden Typ instantiiert die JVM eine nicht veränderbare Instanz der Klasse `java.lang.Class`, über die Informationen zu dem Typ abgefragt werden können.

Dies umfasst u.a.:

- Klassenname
- Implementierte Interfaces
- Methoden
- Attribute
- Annotationen
- …

`java.lang.Class` bildet damit den Einstiegspunkt in die Reflection.

2.7.5.3 Vorgehen

1. Gewünschte Klasse über ein `Class`-Objekt laden

2. Informationen abrufen (welche Methoden, welche Annotationen, …)

3. Eine Instanz dieser Klasse erzeugen, und
4. Methoden aufrufen

Das Vorgehen umfasst vier Schritte: Zunächst die gewünschte Klasse über ein `Class`-Objekt laden und anschließend Informationen abrufen (etwa welche Methoden vorhanden sind, welche Annotationen annotiert wurden, …) und bei Bedarf eine Instanz dieser Klasse erzeugen sowie Methoden aufrufen.

Ein zweiter wichtiger Anwendungsfall (neben dem Abfragen von Informationen und Aufrufen von Methoden) ist das Laden von Klassen, die zur Compile-Zeit nicht mit dem eigentlichen Programm verbunden sind. Auf diesem Weg kann beispielsweise ein Bildbearbeitungsprogramm zur Laufzeit dynamisch Filter aus einem externen Ordner laden und nutzen, oder der Lexer kann die Tokendefinitionen zur Laufzeit einlesen (d.h. er könnte mit unterschiedlichen Tokensätzen arbeiten, die zur Compile-Zeit noch gar nicht definiert sind). Damit werden die Programme dynamischer.

2.7.5.4 Schritt 1: *Class*-Objekt erzeugen und Klasse laden

```
// Variante 1 (package.MyClass dynamisch zur Laufzeit laden)
Class<?> c = Class.forName("package.MyClass");


// Variante 2 (Objekt)
MyClass obj = new MyClass();
Class<?> c = obj.getClass();

// Variante 3 (Klasse)
Class<?> c = MyClass.class;
```

=> Einstiegspunkt der Reflection API

Eigentlich wird nur in **Variante 1 die über den String angegebene Klasse dynamisch von der Laufzeitumgebung (nach-) geladen** (muss also im gestarteten Programm nicht vorhanden sein). Die angegebene Klasse muss aber in Form von Byte-Code an der angegebenen Stelle (Ordner **`package`**, Dateiname `MyClass`**`.class`**) vorhanden sein.

Die anderen beiden Varianten setzen voraus, dass die jeweilige Klasse **bereits geladen** ist (also ganz normal mit den restlichen Sourcen zu Byte-Code (**`.class`**-Dateien) kompiliert wurde und mit dem Programm geladen wurde).

Alle drei Varianten ermöglichen die Introspektion der jeweiligen Klassen zur Laufzeit.

2.7.5.5 Schritt 2: In die Klasse reinschauen

```
// Studi-Klasse dynamisch (nach-) laden
Class<?> c = Class.forName("reflection.Studi");


// Parametersatz für Methode zusammenbasteln
Class<?>[] paramT = new Class<?>[] { String.class };

// public Methode aus dem **Class**-Objekt holen
Method pubMethod = c.getMethod("setName", paramT);
// beliebige Methode aus dem **Class**-Objekt holen
```

```
11 Method privMethod = c.getDeclaredMethod("setName", paramT);
12
13
14 Method[] publicMethods = c.getMethods();  // all public methods (incl.
       inherited)
15 Method[] allMethods = c.getDeclaredMethods();  // all methods (excl. inherited)
```

- **public** Methode laden (auch von Superklasse/Interface geerbt): `Class<?>.getMethod(String, Class<?>[])`
- Beliebige (auch **private**) Methoden (in der Klasse selbst deklariert): `Class<?>.getDeclaredMethod(...)`

Anmerkung: Mit `Class<?>.getDeclaredMethods()` erhalten Sie alle Methoden, die direkt in der Klasse deklariert werden (ohne geerbte Methoden!), unabhängig von deren Sichtbarkeit. Mit `Class<?>.getMethods()` erhalten Sie dagegen alle **public** Methoden, die in der Klasse selbst oder ihren Superklassen bzw. den implementierten Interfaces deklariert sind.

Vgl. Javadoc `getMethods` und `getDeclaredMethods`.

Die Methoden-Arrays können Sie nach bestimmten Eigenschaften durchsuchen, bzw. auf das Vorhandensein einer bestimmten Annotation prüfen (etwa mit `isAnnotationPresent()`) etc.

Analog können Sie weitere Eigenschaften einer Klasse abfragen, beispielsweise Attribute (`Class<?>.getDeclaredFields()`) oder Konstruktoren (`Class<?>.getDeclaredConstructors()`).

2.7.5.6 Schritt 3: Instanz der geladenen Klasse erzeugen

```
1  // Class-Objekt erzeugen
2  Class<?> c = Class.forName("reflection.Studi");
3
4
5  // Variante 1
6  Studi s = (Studi) c.newInstance();
7
8  // Variante 2
9  Constructor<?> ctor = c.getConstructor();
10 Studi s = (Studi) ctor.newInstance();
11
12 // Variante 3
13 Class<?>[] paramT = new Class<?>[] {String.class, int.class};
14 Constructor<?> ctor = c.getDeclaredConstructor(paramT);
15 Studi s = (Studi) ctor.newInstance("Beate", 42);
```

2.7.5.6.1 Parameterlose, öffentliche Konstruktoren:

- `Class<?>.newInstance()` (seit Java9 *deprecated*!)
- `Class<?>.getConstructor()` => `Constructor<?>.newInstance()`

2.7.5.6.2 Sonstige Konstruktoren: Passenden Konstruktor explizit holen: `Class<?>.getDeclaredConstructor(Class<?>[])`, Parametersatz zusammenbasteln (hier nicht dargestellt) und aufrufen `Constructor<?>.newInstance(...)`

2.7.5.6.3 Unterschied *new* und *Constructor.newInstance()*: `new` ist nicht identisch zu `Constructor.newInstance()`: `new` kann Dinge wie Typ-Prüfung oder Auto-Boxing mit erledigen, während man dies bei `Constructor.newInstance()` selbst explizit angeben oder erledigen muss.

Vgl. docs.oracle.com/javase/tutorial/reflect/member/ctorTrouble.html.

2.7.5.7 Schritt 4: Methoden aufrufen

```
// Studi-Klasse dynamisch (nach-) laden
Class<?> c = Class.forName("reflection.Studi");
// Studi-Objekt anlegen (Defaultkonstruktor)
Studi s = (Studi) c.newInstance();
// Parametersatz für Methode zusammenbasteln
Class<?>[] paramT = new Class<?>[] { String.class };
// Methode aus dem **Class**-Objekt holen
Method method = c.getMethod("setName", paramT);


// Methode auf dem **Studi**-Objekt aufrufen
method.invoke(s, "Holgi");
```

Die Reflection-API bietet neben dem reinen Zugriff auf (alle) Methoden noch viele weitere Möglichkeiten. Beispielsweise können Sie bei einer Methode nach der Anzahl der Parameter und deren Typ und Annotationen fragen etc. … Schauen Sie am besten einmal selbst in die API hinein.

Demo: reflection.ReflectionDemo

2.7.5.8 Hinweis: Klassen außerhalb des Classpath laden

```
File folder = new File("irgendwo");
URL[] ua = new URL[]{folder.toURI().toURL()};

URLClassLoader ucl = URLClassLoader.newInstance(ua);
Class<?> c1 = Class.forName("org.wuppie.Fluppie", true, ucl);
Class<?> c2 = ucl.loadClass("org.wuppie.Fluppie");
```

Bemerkung zu Ordnerstruktur und Classpath; Demo: reflection.ClassLoaderDemo

Mit `Class.forName("reflection.Studi")` können Sie die Klasse `Studi` im Package `reflection` laden. Dabei muss sich aber die entsprechende `.class`-Datei (samt der der Package-Struktur entsprechenden Ordnerstruktur darüber) **im Java-Classpath** befinden!

Mit einem weiteren `ClassLoader` können Sie auch aus Ordnern, die sich nicht im Classpath befinden, `.class`-Dateien laden. Dies geht dann entweder wie vorher über `Class.forName()`, wobei hier der neue Class-Loader als Parameter mitgegeben wird, oder direkt über den neuen Class-Loader mit dessen Methode `loadClass()`.

2.7.5.9 Licht und Schatten

Nützlich:

- Erweiterbarkeit: Laden von “externen” (zur Kompilierzeit unbekannter) Klassen in eine Anwendung
- Klassen-Browser, Debugger und Test-Tools

Nachteile:

- Verlust von Kapselung, Compiler-Unterstützung und Refactoring
- Performance: Dynamisches Laden von Klassen etc.
- Sicherheitsprobleme/-restriktionen

Reflection ist ein nützliches Werkzeug. Aber: **Gibt es eine Lösung ohne Reflection, wähle diese!**

2.7.5.10 Wrap-Up

- Inspektion von Programmen zur Laufzeit: **Reflection**
  - `java.lang.Class`: Metadaten über Klassen
  - Je Klasse ein `Class`-Objekt
  - Informationen über Konstruktoren, Methoden, Felder
  - Anwendung: Laden und Ausführen von zur Compile-Zeit unbekanntem Code
  - Vorsicht: Verlust von Refactoring und Compiler-Zusicherungen!

Zum Nachlesen

- Oracle Corporation (2022c)
- Inden (2013, Kap. 8)

Lernziele

- k2: Probleme beim Einsatz von Reflection
- k2: Bedeutung der verschiedenen Exceptions beim Aufruf von Methoden per Reflection
- k3: Inspection von Klassen zur Laufzeit mit Reflection
- k3: Einbindung von zur Compilezeit unbekannten Klassen, Aufruf von Konstruktoren und Methoden (mit und ohne Parameter/Rückgabewerte)

Quizzes

- Quiz Reflection (ILIAS)

Challenges
In den Vorgaben finden Sie eine einfache Implementierung für einen Taschenrechner mit Java-Swing. Dieser Taschenrechner kann nur mit `int`-Werten rechnen. Der Taschenrechner verfügt über keinerlei vordefinierte mathematische Operationen (Addieren, Subtrahieren etc.).
Erstellen Sie eigene mathematische Operationen, die `IOperation` implementieren. Jede Ihrer Klassen soll mit einer Annotation vermerkt werden, in welcher der Name der jeweiligen Operation gespeichert wird.
Der Taschenrechner lädt seine Operationen dynamisch über die statische Methode `OperationLoader.loadOperations` ein. In den Vorgaben ist diese Methode noch nicht ausimplementiert. Implementieren Sie die Funktion so, dass sie mit Hilfe von Reflection Ihre Operationen einliest. Geben Sie dazu den Ordner an, in dem die entsprechenden `.class`-Dateien liegen. (Dieser Ordner soll sich außerhalb Ihres Java-Projekts befinden!)
Verändern Sie nicht die Signatur der Methode.
Ihre Operation-Klassen dürfen Sie nicht vorher bekannt machen. Diese müssen in einem vom aktuellen Projekt separierten Ordner/Projekt liegen.

2.7.6 Exception-Handling

TL;DR
Man unterscheidet in Java zwischen **Exceptions** und **Errors**. Ein Error ist ein Fehler im System (OS, JVM), von dem man sich nicht wieder erholen kann. Eine Exception ist ein Fehlerfall innerhalb des Programmes, auf den man innerhalb des Programms reagieren kann.
Mit Hilfe von Exceptions lassen sich Fehlerfälle im Programmablauf deklarieren und behandeln. Methoden können/müssen mit dem Keyword `throws` gefolgt vom Namen der Exception deklarieren, dass sie im Fehlerfall diese spezifische Exception werfen (und nicht selbst behandeln).
Zum Exception-Handling werden die Keywords `try`, `catch` und `finally` verwendet. Dabei wird im `try`-Block der Code geschrieben, der einen potenziellen Fehler wirft. Im `catch`-Block wird das Verhalten implementiert, dass im Fehlerfall ausgeführt werden soll, und im `finally`-Block kann optional Code geschrieben werden, der sowohl im Erfolgs- als auch Fehlerfall ausgeführt wird.
Es wird zwischen **checked** Exceptions und **unchecked** Exceptions unterschieden. Checked Exceptions sind für erwartbare Fehlerfälle gedacht, die nicht vom Programm ausgeschlossen werden können, wie das Fehlen einer Datei, die eingelesen werden soll. Checked Exceptions müssen deklariert oder behandelt werden. Dies wird vom Compiler überprüft.
Unchecked Exceptions werden für Fehler in der Programmlogik verwendet, etwa das Teilen durch 0 oder Index-Fehler. Sie deuten auf fehlerhafte Programmierung, fehlerhafte Logik oder beispielsweise mangelhafte Eingabeprüfung in. Unchecked Exceptions müssen nicht deklariert oder behandelt werden. Unchecked Exceptions leiten von `RuntimeException` ab.
Als Faustregel gilt: Wenn der Aufrufer sich von einer Exception-Situation erholen kann, sollte man eine checked Exception nutzen. Wenn der Aufrufer vermutlich nichts tun kann, um sich von dem Problem zu erholen, dann sollte man eine unchecked Exception einsetzen.

Videos

- VL Exceptions

2.7.6.1 Fehlerfälle in Java

```
1 int div(int a, int b) {
```

```
    return a / b;
}


div(3, 0);
```

Problem: Programm wird abstürzen, da durch '0' geteilt wird ...

2.7.6.2 Lösung?

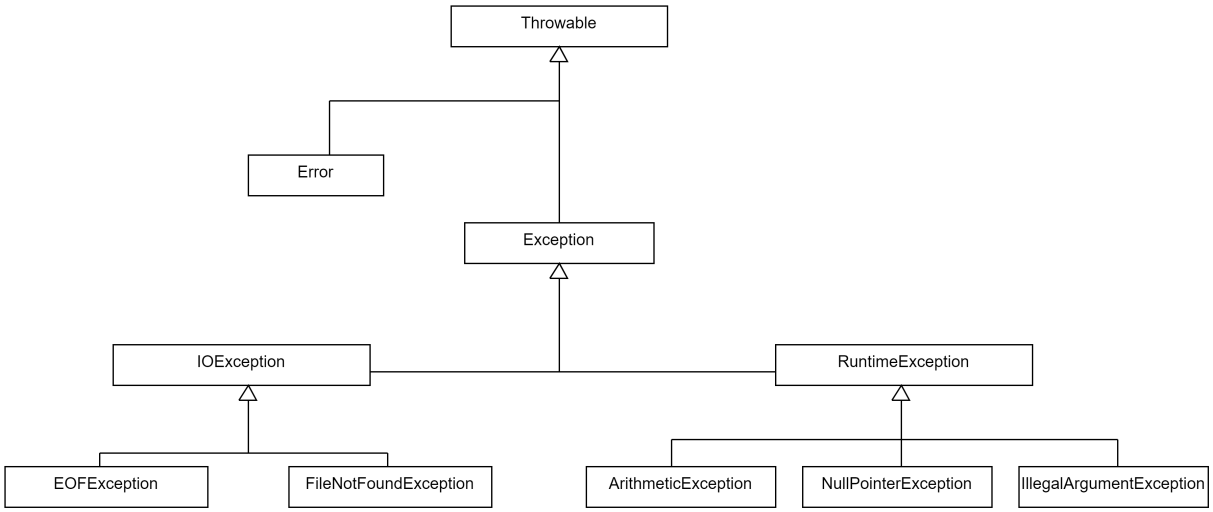
```
Optional<Integer> div(int a, int b) {
    if (b == 0) return Optional.empty();
    return Optional.of(a / b);
}


Optional<Integer> x = div(3, 0);
if (x.isPresent()) {
    // do something
} else {
    // do something else
}
```

Probleme:

- Da `int` nicht `null` sein kann, muss ein `Integer` Objekt erzeugt und zurückgegeben werden: Overhead wg. Auto-Boxing und -Unboxing!
- Der Aufrufer muss auf `null` prüfen.
- Es wird nicht kommuniziert, warum `null` zurückgegeben wird. Was ist das Problem?
- Was ist, wenn `null` ein gültiger Rückgabewert sein soll?

2.7.6.3 Vererbungsstruktur *Throwable*



2.7.6.3.1 *Exception* vs. *Error*

- `Error`:
  - Wird für Systemfehler verwendet (Betriebssystem, JVM, …)
    - `StackOverflowError`
    - `OutOfMemoryError`
  - Von einem Error kann man sich nicht erholen
  - Sollten nicht behandelt werden
- `Exception`:
  - Ausnahmesituationen bei der Abarbeitung eines Programms
  - Können "checked" oder "unchecked" sein
  - Von Exceptions kann man sich erholen

2.7.6.3.2 Unchecked vs. Checked Exceptions

- "Checked" Exceptions:
  - Für erwartbare Fehlerfälle, deren Ursprung nicht im Programm selbst liegt
    - `FileNotFoundException`
    - `IOException`
  - Alle nicht von `RuntimeException` ableitende Exceptions
  - Müssen entweder behandelt (**`try`**/**`catch`**) oder deklariert (**`throws`**) werden: Dies wird vom Compiler überprüft!
- "Unchecked" Exceptions:
  - Logische Programmierfehler ("Versagen" des Programmcodes)
    - `IndexOutOfBoundException`
    - `NullPointerException`
    - `ArithmeticException`
    - `IllegalArgumentException`
  - Leiten von `RuntimeException` oder Unterklassen ab
  - Müssen nicht deklariert oder behandelt werden

Beispiele checked Exception:

- Es soll eine Abfrage an eine externe API geschickt werden. Diese ist aber aktuell nicht zu erreichen. "Erholung": Anfrage noch einmal schicken.
- Es soll eine Datei geöffnet werden. Diese ist aber nicht unter dem angegebenen Pfad zu finden oder die Berechtigungen stimmen nicht. "Erholung": Aufrufer öffnet neuen File-Picker, um es noch einmal mit einer anderen Datei zu versuchen.

Beispiele unchecked Exception:

- Eine **`for`**-Loop über ein Array ist falsch programmiert und will auf einen Index im Array zugreifen, der nicht existiert. Hier kann der Aufrufer nicht Sinnvolles tun, um sich von dieser Situation zu erholen.

- Argumente oder Rückgabewerte einer Methode können **`null`** sein. Wenn man das nicht prüft, sondern einfach Methoden auf dem vermeintlichen Objekt aufruft, wird eine `NullPointerException` ausgelöst, die eine Unterklasse von `RuntimeException` ist und damit eine unchecked Exception. Auch hier handelt es sich um einen Fehler in der Programmlogik, von dem sich der Aufrufer nicht sinnvoll erholen kann.

2.7.6.4 *Throws*

```
int div(int a, int b) throws ArithmeticException {
    return a / b;
}
```

Alternativ:

```
int div(int a, int b) throws IllegalArgumentException {
    if (b == 0) throw new IllegalArgumentException("Can't divide by zero");
    return a / b;
}
```

Exception können an an den Aufrufer weitergeleitet werden oder selbst geworfen werden.

Wenn wie im ersten Beispiel bei einer Operation eine Exception entsteht und nicht gefangen wird, dann wird sie automatisch an den Aufrufer weitergeleitet. Dies wird über die **`throws`**-Klausel deutlich gemacht (Keyword **`throws`** plus den/die Namen der Exception(s), angefügt an die Methodensignatur). Bei unchecked Exceptions *kann* man das tun, bei checked Exceptions *muss* man dies tun.

Wenn man wie im zweiten Beispiel selbst eine neue Exception werfen will, erzeugt man mit **`new`** ein neues Objekt der gewünschten Exception und "wirft" diese mit **`throw`**. Auch diese Exception kann man dann entweder selbst fangen und bearbeiten (siehe nächste Folie) oder an den Aufrufer weiterleiten und dies dann entsprechend über die **`throws`**-Klausel deklarieren: nicht gefangene checked Exceptions *müssen* deklariert werden, nicht gefangene unchecked Exceptions *können* deklariert werden.

Wenn mehrere Exceptions an den Aufrufer weitergeleitet werden, werden sie in der **`throws`**-Klausel mit Komma getrennt: **`throws`** `Exception1, Exception2, Exception3`.

Anmerkung: In beiden obigen Beispielen wurde zur Verdeutlichung, dass die Methode `div()` eine Exception wirft, diese per **`throws`**-Klausel deklariert. Da es sich bei den beiden Beispielen aber jeweils um **unchecked Exceptions** handelt, ist dies im obigen Beispiel *nicht notwendig*. Der Aufrufer *muss* auch nicht ein passendes Exception-Handling einsetzen!

Wenn wir stattdessen eine **checked Exception** werfen würden oder in `div()` eine Methode aufrufen würden, die eine checked Exception deklariert hat, *muss* diese checked Exception entweder in `div()` gefangen und bearbeitet werden oder aber per **`throws`**-Klausel deklariert werden. Im letzteren Fall *muss* dann der Aufrufer analog damit umgehen (fangen oder selbst auch deklarieren). **Dies wird vom Compiler geprüft!**

2.7.6.5 *Try-Catch*

```
int a = getUserInput();
int b = getUserInput();

try {
    div(a, b);
```

```
6  } catch (IllegalArgumentException e) {
7      e.printStackTrace(); // Wird im Fehlerfall aufgerufen
8  }
9
10 // hier geht es normal weiter
```

- Im **`try`** Block wird der Code ausgeführt, der einen Fehler werfen könnte.
- Mit **`catch`** kann eine Exception gefangen und im **`catch`** Block behandelt werden.

Anmerkung: Das bloße Ausgeben des Stacktrace via `e.printStackTrace()` ist noch **kein sinnvolles Exception-Handling**! Hier sollte auf die jeweilige Situation eingegangen werden und versucht werden, den Fehler zu beheben oder dem Aufrufer geeignet zu melden!

2.7.6.6 *Try* und mehrstufiges *Catch*

```
1  try {
2      someMethod(a, b, c);
3  } catch (IllegalArgumentException iae) {
4      iae.printStackTrace();
5  } catch (FileNotFoundException | NullPointerException e) {
6      e.printStackTrace();
7  }
```

Eine im **`try`**-Block auftretende Exception wird der Reihe nach mit den **`catch`**-Blöcken gematcht (vergleichbar mit **`switch case`**).

Wichtig: Dabei muss die Vererbungshierarchie beachtet werden. Die spezialisierteste Klasse muss ganz oben stehen, die allgemeinste Klasse als letztes. Sonst wird eine Exception u.U. zu früh in einem nicht dafür gedachten **`catch`**-Zweig aufgefangen.

Wichtig: Wenn eine Exception nicht durch die **`catch`**-Zweige aufgefangen wird, dann wird sie an den Aufrufer weiter geleitet. Im Beispiel würde eine `IOException` nicht durch die **`catch`**-Zweige gefangen (`IllegalArgumentException` und `NullPointerException` sind im falschen Vererbungszweig, und `FileNotFoundException` ist spezieller als `IOException`) und entsprechend an den Aufrufer weiter gereicht. Da es sich obendrein um eine checked Exception handelt, müsste man diese per **`throws`** `IOException` an der Methode deklarieren.

2.7.6.7 *Finally*

```
1  Scanner myScanner = new Scanner(System.in);
2
3  try {
4      return 5 / myScanner.nextInt();
5  } catch (InputMismatchException ime) {
6      ime.printStackTrace();
7  } finally {
8      // wird immer aufgerufen
9      myScanner.close();
10 }
```

Der **`finally`** Block wird sowohl im Fehlerfall als auch im Normalfall aufgerufen. Dies wird beispielsweise für Aufräumarbeiten genutzt, etwa zum Schließen von Verbindungen oder Input-Streams.

2.7.6.8 *Try*-with-Resources

```
try (Scanner myScanner = new Scanner(System.in)) {
    return 5 / myScanner.nextInt();
} catch (InputMismatchException ime) {
    ime.printStackTrace();
}
```

Im `try`-Statement können Ressourcen deklariert werden, die am Ende sicher geschlossen werden. Diese Ressourcen müssen `java.io.Closeable` implementieren.

2.7.6.9 Eigene Exceptions

```
// Checked Exception
public class MyCheckedException extends Exception {
    public MyCheckedException(String errorMessage) {
        super(errorMessage);
    }
}
```

```
// Unchecked Exception
public class MyUncheckedException extends RuntimeException {
    public MyUncheckedException(String errorMessage) {
        super(errorMessage);
    }
}
```

Eigene Exceptions können durch Spezialisierung anderer Exception-Klassen realisiert werden. Dabei kann man direkt von `Exception` oder `RuntimeException` ableiten oder bei Bedarf von spezialisierteren Exception-Klassen.

Wenn die eigene Exception in der Vererbungshierarchie unter `RuntimeException` steht, handelt es sich um eine *unchecked Exception*, sonst um eine *checked Exception*.

In der Benutzung (werfen, fangen, deklarieren) verhalten sich eigene Exception-Klassen wie die Exceptions aus dem JDK.

2.7.6.10 Stilfrage: Wie viel Code im *Try*?

```
int getFirstLineAsInt(String pathToFile) {
    FileReader fileReader = new FileReader(pathToFile);
    BufferedReader bufferedReader = new BufferedReader(fileReader);
    String firstLine = bufferedReader.readLine();

    return Integer.parseInt(firstLine);
}
```

Zeigen: exceptions.HowMuchTry

Hier lassen sich verschiedene “Ausbaustufen” unterscheiden.

2.7.6.10.1 Handling an den Aufrufer übergeben

```
int getFirstLineAsIntV1(String pathToFile) throws FileNotFoundException,
    IOException {
    FileReader fileReader = new FileReader(pathToFile);
```

```
3       BufferedReader bufferedReader = new BufferedReader(fileReader);
4       String firstLine = bufferedReader.readLine();
5
6       return Integer.parseInt(firstLine);
7   }
```

Der Aufrufer hat den Pfad als String übergeben und ist vermutlich in der Lage, auf Probleme mit dem Pfad sinnvoll zu reagieren. Also könnte man in der Methode selbst auf ein **try/catch** verzichten und stattdessen die `FileNotFoundException` (vom `FileReader`) und die `IOException` (vom `bufferedReader.readLine()`) per **throws** deklarieren.

Anmerkung: Da `FileNotFoundException` eine Spezialisierung von `IOException` ist, reicht es aus, lediglich die `IOException` zu deklarieren.

2.7.6.10.2 Jede Exception einzeln fangen und bearbeiten

```
1   int getFirstLineAsIntV2(String pathToFile) {
2       FileReader fileReader = null;
3       try {
4           fileReader = new FileReader(pathToFile);
5       } catch (FileNotFoundException fnfe) {
6           fnfe.printStackTrace(); // Datei nicht gefunden
7       }
8
9       BufferedReader bufferedReader = new BufferedReader(fileReader);
10
11      String firstLine = null;
12      try {
13          firstLine = bufferedReader.readLine();
14      } catch (IOException ioe) {
15          ioe.printStackTrace(); // Datei kann nicht gelesen werden
16      }
17
18      try {
19          return Integer.parseInt(firstLine);
20      } catch (NumberFormatException nfe) {
21          nfe.printStackTrace(); // Das war wohl kein Integer
22      }
23
24      return 0;
25  }
```

In dieser Variante wird jede Operation, die eine Exception werfen kann, separat in ein **try/catch** verpackt und jeweils separat auf den möglichen Fehler reagiert.

Dadurch kann man die Fehler sehr einfach dem jeweiligen Statement zuordnen.

Allerdings muss man nun mit Behelfsinitialisierungen arbeiten und der Code wird sehr in die Länge gezogen und man erkennt die eigentlichen funktionalen Zusammenhänge nur noch schwer.

Anmerkung: Das “Behandeln” der Exceptions ist im obigen Beispiel kein gutes Beispiel für das Behandeln von Exceptions. Einfach nur einen Stacktrace zu printen und weiter zu machen, als ob nichts passiert wäre, ist **kein sinnvolles Exception-Handling**. Wenn Sie solchen Code schreiben oder sehen, ist das ein Anzeichen, dass auf dieser Ebene nicht sinnvoll mit dem Fehler umgegangen werden kann und dass man ihn besser an den Aufrufer weiter reichen sollte (siehe nächste Folie).

2.7.6.10.3 Funktionaler Teil in gemeinsames *Try* und mehrstufiges *Catch*

```java
int getFirstLineAsIntV3(String pathToFile) {
    try {
        FileReader fileReader = new FileReader(pathToFile);
        BufferedReader bufferedReader = new BufferedReader(fileReader);
        String firstLine = bufferedReader.readLine();
        return Integer.parseInt(firstLine);
    } catch (FileNotFoundException fnfe) {
        fnfe.printStackTrace(); // Datei nicht gefunden
    } catch (IOException ioe) {
        ioe.printStackTrace(); // Datei kann nicht gelesen werden
    } catch (NumberFormatException nfe) {
        nfe.printStackTrace(); // Das war wohl kein Integer
    }

    return 0;
}
```

Hier wurde der eigentliche funktionale Kern der Methode in ein gemeinsames **`try`**/**`catch`** verpackt und mit einem mehrstufigen **`catch`** auf die einzelnen Fehler reagiert. Durch die Art der Exceptions sieht man immer noch, wo der Fehler herkommt. Zusätzlich wird die eigentliche Funktionalität so leichter erkennbar.

Anmerkung: Auch hier ist das gezeigte Exception-Handling kein gutes Beispiel. Entweder man macht hier sinnvollere Dinge, oder man überlässt dem Aufrufer die Reaktion auf den Fehler.

2.7.6.11 Stilfrage: Wo fange ich die Exception?

```java
private static void methode1(int x) throws IOException {
    JFileChooser fc = new JFileChooser();
    fc.showDialog(null, "ok");
    methode2(fc.getSelectedFile().toString(), x, x * 2);
}

private static void methode2(String path, int x, int y) throws IOException {
    FileWriter fw = new FileWriter(path);
    BufferedWriter bw = new BufferedWriter(fw);
    bw.write("X:" + x + " Y: " + y);
}

public static void main(String... args) {
    try {
        methode1(42);
    } catch (IOException ioe) {
        ioe.printStackTrace();
    }
}
```

Prinzipiell steht es einem frei, wo man eine Exception fängt und behandelt. Wenn im `main()` eine nicht behandelte Exception auftritt (weiter nach oben geleitet wird), wird das Programm mit einem Fehler beendet.

Letztlich scheint es eine gute Idee zu sein, eine Exception so nah wie möglich am Ursprung der Fehlerursache zu behandeln. Man sollte sich dabei die Frage stellen: Wo kann ich sinnvoll auf den Fehler reagieren?

2.7.6.12 Stilfrage: Wann checked, wann unchecked

2.7.6.12.1 “Checked” Exceptions

- Für erwartbare Fehlerfälle, deren Ursprung nicht im Programm selbst liegt
- Aufrufer kann sich von der Exception erholen

2.7.6.12.2 “Unchecked” Exceptions

- Logische Programmierfehler (“Versagen” des Programmcodes)
- Aufrufer kann sich von der Exception vermutlich nicht erholen

Vergleiche “Unchecked Exceptions — The Controversy”.

2.7.6.13 Wrap-Up

- `Error` und `Exception`: System vs. Programm
- Checked und unchecked Exceptions: `Exception` vs. `RuntimeException`

- **`try`**: Versuche Code auszuführen
- **`catch`**: Verhalten im Fehlerfall
- **`finally`**: Verhalten im Erfolgs- und Fehlerfall

- **`throw`**: Wirft eine Exception
- **`throws`**: Deklariert eine Exception an Methode

- Eigene Exceptions durch Ableiten von anderen Exceptions (werden je nach Vererbungshierarchie automatisch checked oder unchecked)

Zum Nachlesen

- Oracle Corporation (2022b)
- Ullenboom (2021, Kap. 8)
- Oracle Corporation (2022c)

Lernziele

- k2: Unterschied zwischen Error und Exception
- k2: Unterschied zwischen checked und unchecked Exceptions
- k3: Umgang mit Exceptions
- k3: Eigene Exceptions schreiben

Quizzes

- Quiz Exceptions (ILIAS)

Challenges
Betrachten Sie die Vorgaben.

Verbessern Sie das Exception-Handling
Im package `better_try_catch` finden Sie die Klasse `BetterTryCatchMain`, in der verschiedene Methoden der Klasse `MyFunctions` aufgerufen werden.
Erklären Sie, warum das dort implementierte Exception-Handling nicht gut ist und verbessern Sie es.

Checked vs. unckecked Exceptions
Erklären Sie den Unterschied zwischen checked und unchecked Exceptions.
Im Folgenden werden verschiedene Exceptions beschrieben. Erklären Sie, ob diese jeweils "checked" oder "unchecked" sein sollten.

- `IntNotBetweenException` soll geworfen werden, wenn ein Integer-Parameter nicht im definierten Wertebereich liegt.
- `NoPicturesFoundException` soll geworfen werden, wenn in einem übergebenen Verzeichnis keine Bilddateien gefunden werden konnten.
- `NotAPrimeNumberException` soll geworfen werden, wenn eine vom User eingegebene Zahl keine Primzahl ist.

Freigeben von Ressourcen
Im Package `finally_resources` finden Sie die Klasse `MyResource`.
Rufen Sie die Methode `MyResource#doSomething` auf, im Anschluss müssen Sie **immer** die Methode `MyResource#close` aufrufen.

1. Zeigen Sie den Aufruf mit **`try-catch-finally`**.
2. Verändern Sie die Vorgaben so, dass Sie den Aufruf mit der "try-with-resources"-Technik ausführen können.

Where to catch?
Erklären Sie, wann und wo eine Exception gefangen und bearbeitet werden sollte.
Im Package `where_to_catch` finden Sie die Klasse `JustThrow`. Alle Methoden in der Klasse werfen aufkommende Exceptions bis zur `main` hoch.
Verändern Sie die Vorgaben so, dass die Exceptions an den passenden Stellen gefangen und sinnvoll bearbeitet werden. Begründen Sie Ihre Entscheidungen.

2.7.7 Aufzählungen (Enumerations)

TL;DR
Mit Hilfe von `enum` lassen sich Aufzählungstypen definieren (der Compiler erzeugt intern passende Klassen). Dabei wird den Konstanten eine fortlaufende Nummer zugeordnet, auf die mit `ordinal()` zugegriffen werden kann. Mit der Methode `values()` kann über die Konstanten iteriert werden, und mit `name()` kann eine Stringrepräsentation einer Konstanten erzeugt werden. Es sind keine Instanzen von Enum-Klassen erzeugbar, und die Enum-Konstanten sind implizit **`final`** und **`static`**.
Es lassen sich auch komplexe Enumerations analog zu Klassendefinition definieren, die eigene Konstruktoren, Felder und Methoden enthalten.

Videos

- VL Enumerations
- Demo Enumerations

2.7.7.1 Motivation

```java
public class Studi {
    public static final int IFM = 0;
    public static final int ELM = 1;
    public static final int ARC = 2;

    public Studi(String name, int credits, int studiengang) {
        // Wert für studiengang muss zwischen 0 und 2 liegen
        // Erwünscht: Konstanten nutzen
    }

    public static void main(String[] args) {
        Studi rainer = new Studi("Rainer", 10, Studi.IFM);
        Studi holger = new Studi("Holger", 3, 4);   // Laufzeit-Problem!
    }
}
```

Probleme:

- Keine Typsicherheit
- Konstanten gehören zur Klasse `Studi`, obwohl sie in anderem Kontext vermutlich auch interessant sind

Beispiel enums.v1.Studi

2.7.7.2 Verbesserung: Einfache Aufzählung

```java
public enum Fach {
    IFM, ELM, ARC
}


public class Studi {
    public Studi(String name, int credits, Fach studiengang) {
        // Typsicherheit für studiengang :-)
    }

    public static void main(String[] args) {
        Studi rainer = new Studi("Rainer", 10, Fach.IFM);
        Studi holger = new Studi("Holger", 3, 4);   // Syntax-Fehler!
    }
}
```

2.7.7.3 Einfache Aufzählungen: Eigenschaften

```java
public enum Fach {
    IFM, ELM, ARC
}
```

1. Enum-Konstanten (`IFM`, ...) sind implizit **`static`** und **`final`**

2. Enumerations (`Fach`) nicht instantiierbar
3. Enumerations stellen einen neuen Typ dar: hier der Typ `Fach`
4. Methoden: `name()`, `ordinal()`, `values()`, `toString()`

2.7.7.3.1 Wiederholung *static* **Attribute**:

- **`static`** Attribute sind Eigenschaften/Zustände der **Klasse**
- Gelten in jedem von der Klasse erzeugten Objekt
- Unterschiedliche Lebensdauer:
  - Objektattribute (Instanzvariablen): ab **`new`** bis zum Garbage Collector
  - Statische Variablen: Laufzeitumgebung (JVM) lädt und initialisiert die Klasse (**`static`** Attribute existieren, bis die JVM die Klasse entfernt)

Methoden:

- **`static`** deklarierte Methoden sind **Klassenmethoden**
- Können direkt auf der Klasse aufgerufen werden
- Beispiele: `Math.max()`, `Math.sin()`, `Integer.parseInt()`
- **Achtung**: In Klassenmethoden nur Klassenattribute nutzbar (keine Instanzattribute!), d.h. keine **`this`**-Referenz nutzbar

2.7.7.3.2 Wiederholung *final*: Attribute/Methoden/Klassen nicht änderbar

- Attribute: **`final`** Attribute können nur einmal gesetzt werden

```java
void foo() {
    int i = 2;
    final int j = 3;
    final int k;
    i = 3;
    j = 4;  // Compilerfehler
    k = 5;
    k = 6;  // Compilerfehler
}
```

Beispiel enums.FinalDemo

- Methoden: **`final`** deklarierte Methoden können bei Vererbung nicht überschrieben werden
- Klassen: von **`final`** deklarierten Klassen können keine Unterklassen gebildet werden

2.7.7.4 Einfache Aufzählungen: Eigenschaften (cnt.)

```java
// Referenzen auf Enum-Objekte können null sein
Fach f = null;
f = Fach.IFM;

// Vergleich mit == möglich
// equals() unnötig, da Vergleich mit Referenz auf statische Variable
if (f == Fach.IFM) {
    System.out.println("Richtiges Fach :-)");
```

```
}

// switch/case
switch (f) {
    case IFM:   // Achtung: *NICHT* Fach.IFM
        System.out.println("Richtiges Fach :-)");
        break;
    default:
        throw new IllegalArgumentException("FALSCHES FACH: " + f);
}
```

Außerdem können wir folgende Eigenschaften nutzen (u.a., s.u.):

- Enumerations haben Methode `String toString()` für die Konstanten
- Enumerations haben Methode `final T[] values()` für die Iteration über die Konstanten

Demo: enums.v2.Studi

2.7.7.5 Enum: Genauer betrachtet

```
public enum Fach {  IFM, ELM, ARC  }
```

Compiler sieht (*in etwa*):

```
public class Fach extends Enum {
    public static final Fach IFM = new Fach("IFM", 0);
    public static final Fach ELM = new Fach("ELM", 1);
    public static final Fach ARC = new Fach("ARC", 2);

    private Fach( String s, int i ) { super( s, i ); }
}
```

=> **Singleton-Pattern** für Konstanten

2.7.7.6 Enum-Klassen: Eigenschaften

```
public enum Fach {
    IFM,
    ELM("Elektrotechnik Praxisintegriert", 1, 30),
    ARC("Architektur", 4, 40),
    PHY("Physik", 3, 10);

    private final String description;
    private final int number;
    private final int capacity;

    Fach() { this("Informatik Bachelor", 0, 60); }
    Fach(String descr, int number, int capacity) {
        this.description = descr;  this.number = number;  this.capacity =
            capacity;
    }
    public String getDescription() {
        return "Konstante: " + name() + " (Beschreibung: " + description
                + ", Kapazitaet: " + capacity + ", Nummer: " + number
                + ", Ordinal: " + ordinal() + ")";
    }
```

```
20 }
```

2.7.7.6.1 Konstruktoren und Methoden für Enum-Klassen definierbar

- Kein eigener Aufruf von **`super`** (!)
- Konstruktoren implizit **`private`**

2.7.7.6.2 Compiler fügt automatisch folgende Methoden hinzu (Auswahl):

- Strings:
  - **`public final`** `String name()` => Name der Konstanten (**`final`**!)
  - **`public`** `String toString()` => Ruft `name()` auf, überschreibbar
- Konstanten:
  - **`public final`** `T[] values()` => Alle Konstanten der Aufzählung
  - **`public final int`** `ordinal()` => Interne Nummer der Konstanten (Reihenfolge des Anlegens der Konstanten!)
  - **`public static`** `T valueOf(String)` => Zum String passende Konstante (via `name()`)

Hinweis: Diese Methoden gibt es auch bei den “einfachen” Enumerationen (s.o.).

Demo: enums.v3

2.7.7.7 Wrap-Up

- Aufzählungen mit Hilfe von `enum` (Compiler erzeugt intern Klassen)
- Komplexe Enumerations analog zu Klassendefinition: Konstruktoren, Felder und Methoden (keine Instanzen von Enum-Klassen erzeugbar)
- Enum-Konstanten sind implizit **`final`** und **`static`**
- Compiler stellt Methoden `name()`, `ordinal()` und `values()` zur Verfügung
  - Name der Konstanten
  - Interne Nummer der Konstanten (Reihenfolge des Anlegens)
  - Array mit allen Konstanten der Enum-Klasse

Zum Nachlesen

- Oracle Corporation (2022c)
- Ullenboom (2021, 6.4.3 und 10.7)

Lernziele

- k2: Vorgänge beim Initialisieren von Enum-Klassen (Hinweis: static)
- k3: Erstellung komplexer Enumerationen mit Feldern und Konstruktoren
- k3: Nutzung von name(), ordinal() und values() in Enum-Klassen

Quizzes

- Quiz Enumerations (ILIAS)

2.7.8 Konfiguration eines Programms

TL;DR
Zu Konfiguration von Programmen kann man beim Aufruf Kommandozeilenparameter mitgeben. Diese sind in der über den Parameter`String[] args` in der `main(String[] args)`-Methode zugreifbar.
Es gibt oft eine Kurzversion ("-x") und/oder eine Langversion ("–breite"). Zusätzlich können Parameter noch ein Argument haben ("-x 12" oder "–breite=12"). Parameter können optional oder verpflichtend sein.
Um dies nicht manuell auswerten zu müssen, kann man beispielsweise die Bibliothkek Apache Commons CLI benutzen.
Ein anderer Weg zur Konfiguration sind Konfigurationsdateien, die man entsprechend einliest. Hier findet man häufig das "Ini-Format", also zeilenweise "Key=Value"-Paare. Diese kann man mit der Klasse `java.util.Properties` einlesen, bearbeiten und speichern (auch als XML).

Videos

- VL Konfiguration
- Demo Setzen von Kommandozeilenparametern (IDE)
- Demo Einbinden von Libs in IDE
- Demo Apache Commons CLI
- Demo Properties

2.7.8.1 Wie kann man Programme konfigurieren?

1. Parameter beim Start mitgeben: Kommandozeilenparameter (CLI)
2. Konfigurationsdatei einlesen und auswerten

2.7.8.2 Varianten von Kommandozeilenparameter

- Fixe Reihenfolge

```
java MyApp 10 20 hello debug
```

- Benannte Parameter I

```
java MyApp -x 10 -y 20 -answer hello -d
```

- Benannte Parameter II

```
java MyApp --breite=10 --hoehe=20 --answer=hello --debug
```

Häufig Mischung von Kurz- und Langformen

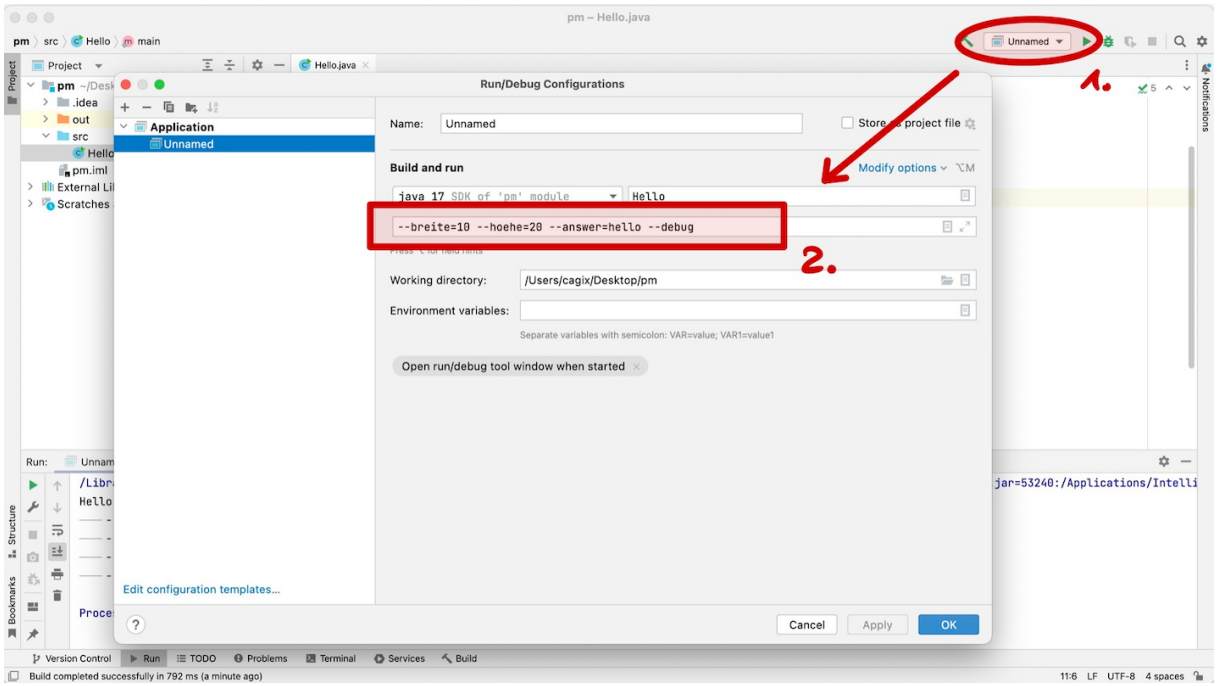
Häufig hat man eine Kurzform der Optionen, also etwa “-x”. Dabei ist der Name der Option in der Regel ein Zeichen lang. Es gibt aber auch Abweichungen von dieser Konvention, denken Sie beispielsweise an `java -version`.

In der Langform nutzt man dann einen aussagekräftigen Namen und stellt zwei Bindestriche voran, also beispielsweise “–breite” (als Alternative für “-x”).

Wenn Optionen Parameter haben, schreibt man in der Kurzform üblicherweise “-x 10” (trennt also den Parameter mit einem Leerzeichen von der Option) und in der Langform “–breite=10” (also mit einem “=” zwischen Option und Parameter). Das sind ebenfalls Konventionen, d.h. man kann prinzipiell auch in der Kurzform das “=” nutzen, also “-x=10”, oder in der Langform mit einem Leerzeichen trennen, also “–breite 10”.

Demo IDE und CLI

Hinweis IntelliJ: “`Edit Configurations`” => Kommandozeilenparameter unter “`Build and run`” im entsprechenden Feld eintragen



2.7.8.3 Auswertung Kommandozeilenparameter

- Kommandozeilenparameter werden als String-Array an `main()`-Methode übergeben:

```
1 public static void main(String[] args) { }
2 public static void main(String... argv) { }
```

=> Müssen “händisch” ausgewertet werden

Anmerkung: Nur Parameter! Nicht Programmname als erster Eintrag wie in C …

2.7.8.4 Beispiel Auswertung Kommandozeilenparameter

```
1  public static void main(String[] args) {
2      int x = 100;
3      String answer = "";
4      boolean debug = false;
5
6      // Parameter: -x=10 -answer=hello -debug
7      // => args = ["-x=10", "-answer=hello", "-debug"]
8      for (String param : args) {
9          if (param.startsWith("-x")) { x = Integer.parseInt(param.substring(3));
               }
10         if (param.startsWith("-a")) { answer = param.substring(8); }
11         if (param.startsWith("-d")) { debug = true; }
12     }
13 }
```

2.7.8.4.1 Kritik an manueller Auswertung Kommandozeilenparameter

- Umständlich und unübersichtlich
- Große `if-else`-Gebilde in `main()`
- Kurz- und Langform müssen getrennt realisiert werden
- Optionale Parameter müssen anders geprüft werden als Pflichtparameter
- Überlappende Parameternamen schwer aufzufinden
- Prüfung auf korrekten Typ nötig bei Parametern mit Werten
- Hilfe bei Fehlern muss separat realisiert und gepflegt werden

2.7.8.5 Apache Commons: CLI Rad nicht neu erfinden!

- Apache Commons bietet die CLI-Bibliothek zum Umgang mit Kommandozeilenparametern an: commons.apache.org/cli

Annäherung an fremde API:

- Lesen der verfügbaren Doku (PDF, HTML)
- Lesen der verfügbaren Javadoc
- Herunterladen der Bibliothek
- Einbinden ins Projekt

2.7.8.6 Exkurs: Einbinden fremder Bibliotheken/APIs

2.7.8.6.1 Eclipse

- Lib von commons.apache.org herunterladen und auspacken
- Neuen Unterordner im Projekt anlegen: `libs/`
- Bibliothek (`.jar`-Files) hinein kopieren
- Projektexplorer, Kontextmenü auf `.jar`-File: "`Add as Library`"
- Alternativ Menü-Leiste: "`Project` > `Properties` > `Java Build Path` > `Libraries` > `Add JARs`"

2.7.8.6.2 IntelliJ

- Variante 1:
  - Lib von commons.apache.org herunterladen und auspacken
  - Neuen Unterordner im Projekt anlegen: `libs/`
  - Bibliothek (`.jar`-Files) hinein kopieren
  - Variante 1 (a):Projektexplorer, Kontextmenü auf `.jar`-File: "`Build Path` > `Add to Build Path`"
  - Variante 1 (b): Projekteigenschaften, Eintrag "Libraries", "+", "New Project Library", "Java" und Jar-File auswählen
- Variante 2:
  - Projekteigenschaften, Eintrag "Libraries", "+", "New Project Library", "From Maven" und "commons-cli:commons-cli:1.5.0" als Suchstring eingeben und die Suche abschließen

2.7.8.6.3 Gradle oder Ant oder Maven

- Lib auf Maven Central suchen: "commons-cli:commons-cli" als Suchstring eingeben
- Passenden Dependency-Eintrag in das Build-Skript kopieren

2.7.8.6.4 Kommandozeilenaufruf

- Class-Path bei Aufruf setzen:
  - Unix: `java -cp .:<jarfile>:<jarfile> <mainclass>`
  - Windows: `java -cp .;<jarfile>;<jarfile> <mainclass>`

  Achtung: Unter Unix (Linux, MacOS) wird ein Doppelpunkt zum Trennen der Jar-Files eingesetzt, unter Windows ein Semikolon!

Beispiel: `java -classpath .:/home/user/wuppy.jar MyApp`

Vorgriff auf Build-Skripte (spätere VL): Im hier gezeigten Vorgehen werden die Abhängigkeiten manuell aufgelöst, d.h. die Jar-Files werden manuell heruntergeladen (oder selbst kompiliert) und dem Projekt hinzugefügt.

Alle später besprochenen Build-Skripte (Ant, Gradle) beherrschen die automatische Auflösung von Abhängigkeiten. Dazu muss im Skript die Abhängigkeit auf geeignete Weise beschrieben werden und wird dann beim Kompilieren des Programms automatisch von spezialisierten Servern in der im Skript definierten Version heruntergeladen. Dies funktioniert auch bei rekursiven Abhängigkeiten …

2.7.8.7 Überblick Umgang mit Apache Commons CLI **Paket**: `org.apache.commons.cli`

1. Definition der Optionen
   - Je Option eine Instanz der Klasse `Option`
   - Alle Optionen in Container `Options` sammeln
2. Parsen der Eingaben mit `DefaultParser`
3. Abfragen der Ergebnisse: `CommandLine`
4. Formatierte Hilfe ausgeben: `HelpFormatter`

Die Funktionsweise der einzelnen Klassen wird in der Demo kurz angerissen. Schauen Sie bitte zusätzlich in die Dokumentation.

Demo: Einbinden von Libs, cli.Args

2.7.8.8 Laden und Speichern von Konfigurationsdaten

```
#ola - ein Kommentar
hoehe=2
breite=9
gewicht=12
```

- Konfigurationsdaten sind i.d.R. Schlüssel-Wert-Paare (`String`/`String`)

  => `java.util.Properties`

  Tatsächlich verbirgt sich ein `Hashtable` dahinter:

```
public class Properties extends Hashtable<Object,Object>;
```

2.7.8.9 Laden und Speichern von Konfigurationsdaten (cnt.)

- Properties anlegen und modifizieren

```
Properties props = new Properties();
props.setProperty("breite", "9");
props.setProperty("breite", "99");
String value = props.getProperty("breite");
```

- Properties speichern: `Properties#store` und `Properties#storeToXML`

```
public void store(Writer writer, String comments)
public void store(OutputStream out, String comments)
public void storeToXML(OutputStream os, String comment, String encoding)
```

- Properties laden: `Properties#load` und `Properties#loadFromXML`

```
public void load(Reader reader)
public void load(InputStream inStream)
public void loadFromXML(InputStream in)
```

Demo: cli.Props, Hinweis auf “Apache Commons Configuration”

`java.util.Properties` sind eine einfache und im JDK bereits eingebaute Möglichkeit, mit Konfigurationsdateien zu hantieren. Deutlich umfangreichere Möglichkeiten bieten aber externe Bibliotheken, beispielsweise “Apache Commons Configuration” (commons.apache.org/configuration).

2.7.8.10 Wrap-Up

- Kommandozeilenparameter als `String[]` in `main()`-Methode
- Manuelle Auswertung komplex => *Apache Commons CLI*
- Schlüssel-Wert-Paare mit `java.util.Properties` aus/in Dateien laden/speichern

Zum Nachlesen

- Oracle Corporation (2022c)

Lernziele

- k3: Auswertung von Kommandozeilenparametern in einem Programm
- k3: Apache Commons CLI zur Verarbeitung von Kommandozeilenparametern
- k3: Laden von Konfigurationsdaten mit java.util.Properties

Quizzes

- Quiz Konfiguration (ILIAS)

2.8 Generics: Umgang mit parametrisierten Typen

2.8.1 Generische Klassen & Methoden

TL;DR
Generische Klassen und Methoden sind ein wichtiger Baustein in der Programmierung mit Java. Dabei werden Typ-Variablen eingeführt, die dann bei der Instantiierung der generischen Klassen oder beim Aufruf von generischen Methoden mit existierenden Typen konkretisiert werden (“Typ-Parameter”).
Syntaktisch definiert man die Typ-Variablen in spitzen Klammern hinter dem Klassennamen bzw. vor dem Rückgabetyp einer Methode: `public class Stack<E> { }` und `public <T> T foo(T m){ }`.

Videos

- VL Generische Klassen & Methoden
- Demo Generische Methoden

2.8.1.1 Generische Strukturen

```
1 Vector speicher = new Vector();
2 speicher.add(1); speicher.add(2); speicher.add(3);
```

```
3  speicher.add("huhu");
4
5  int summe = 0;
6  for (Object i : speicher) { summe += (Integer)i; }
```

Problem: Nutzung des "*raw*" Typs `Vector` ist nicht typsicher!

- Mögliche Fehler fallen erst zur Laufzeit und u.U. erst sehr spät auf: Offenbar werden im obigen Beispiel `int`-Werte erwartet, d.h. das Hinzufügen von `"huhu"` ist vermutlich ein Versehen (wird vom Compiler aber nicht bemerkt)
- Die Iteration über `speicher` kann nur allgemein als `Object` erfolgen, d.h. in der Schleife muss auf den vermuteten/gewünschten Typ gecastet werden: Hier würde dann der String `"huhu"` Probleme zur Laufzeit machen

---

```
1  Vector<Integer> speicher = new Vector<Integer>();
2  speicher.add(1); speicher.add(2); speicher.add(3);
3  speicher.add("huhu");
4
5  int summe = 0;
6  for (Integer i : speicher) { summe += i; }
```

Vorteile beim Einsatz von Generics:

- Datenstrukturen/Algorithmen nur einmal implementieren, aber für unterschiedliche Typen nutzen
- Keine Vererbungshierarchie nötig
- Nutzung ist typsicher, Casting unnötig
- Geht nur für Referenztypen
- Beispiel: Collections-API

2.8.1.2 Generische Klassen/Interfaces definieren

- **Definition**: "`<Typ>`" hinter Klassennamen

```
1  public class Stack<E> {
2      public E push(E item) {
3          addElement(item);
4          return item;
5      }
6  }
```

  - `Stack<E>` => Generische (parametrisierte) Klasse (auch: "*generischer Typ*")
  - `E` => Formaler Typ-Parameter (auch: "*Typ-Variable*")

- **Einsatz**:

```
1  Stack<Integer> stack = new Stack<Integer>();
```

  - `Integer` => Typ-Parameter
  - `Stack<Integer>` => Parametrisierter Typ

2.8.1.3 Generische Klassen instantiieren

- Typ-Parameter in spitzen Klammern hinter Klasse bzw. Interface

```
1  ArrayList<Integer> il = new ArrayList<Integer>();
2  ArrayList<Double>  dl = new ArrayList<Double>();
```

2.8.1.4 Beispiel I: Einfache generische Klassen

```
1  class Tutor<T> {
2      // T kann in Tutor *fast* wie Klassenname verwendet werden
3      private T x;
4      public T foo(T t) { ... }
5  }
```

```
1  Tutor<String>  a = new Tutor<String>();
2  Tutor<Integer> b = new Tutor<>();  // ab Java7: "Diamond Operator"
3
4  a.foo("wuppie");
5  b.foo(1);
6  b.foo("huhu");  // Fehlermeldung vom Compiler
```

Beispiel: classes.GenericClasses

2.8.1.4.1 Typ-Inferenz Typ-Parameter kann bei `new()` auf der rechten Seite oft weggelassen werden => **Typ-Inferenz**

```
1  Tutor<String> x = new Tutor<>();  // <>: "Diamantoperator"
```

(gilt seit Java 1.7)

2.8.1.5 Beispiel II: Vererbung mit Typparametern

```
1  interface Fach<T1, T2> {
2      public void machWas(T1 a, T2 b);
3  }
4
5  class SHK<T> extends Tutor<T> { ... }
6
7  class PM<X, Y, Z> implements Fach<X, Z> {
8      public void machWas(X a, Z b) { ... }
9      public Y getBla() { ... }
10 }
11
12 class Studi<A,B> extends Person { ... }
13 class Properties extends Hashtable<Object,Object> { ... }
```

Auch Interfaces und abstrakte Klassen können parametrisierbar sein.

Bei der Vererbung sind alle Varianten bzgl. der Typ-Variablen denkbar. Zu beachten ist dabei vor allem, dass die Typ-Variablen der Oberklasse (gilt analog für Interfaces) entweder durch Typ-Variablen der Unterklasse oder durch konkrete Typen spezifiziert sind. Die Typ-Variablen der Oberklasse dürfen nicht “in der Luft hängen” (siehe auch nächste Folie)!

2.8.1.6 Beispiel III: Überschreiben/Überladen von Methoden

```
class Mensch { ... }

class Studi<T extends Mensch> {
    public void f(T t) { ... }
}

class Prof<T> extends Mensch { ... }

class Tutor extends Studi<Mensch> {
    public void f(Mensch t) { ... }      // Ueberschreiben
    public void f(Tutor t) { ... }       // Ueberladen
}
```

2.8.1.7 Vorsicht: So geht es nicht!

```
class Foo<T> extends T { ... }

class Fluppie<T> extends Wuppie<S> { ... }
```

- Generische Klasse `Foo<T>` kann nicht selbst vom Typ-Parameter `T` ableiten (warum?)
- Bei Ableiten von generischer Klasse `Wuppie<S>` muss deren Typ-Parameter `S` bestimmt sein: etwa durch den Typ-Parameter der ableitenden Klasse, beispielsweise `Fluppie<S>` (statt `Fluppie<T>`)

2.8.1.8 Generische Methoden definieren

- "`<Typ>`" vor Rückgabetyp

```
public class Mensch {
    public <T> T myst(T m, T n) {
        return Math.random() > 0.5 ? m : n;
    }
}
```

- "Mischen possible":

```
public class Mensch<E> {
    public <T> T myst(T m, T n) { ... }
    public String myst(String m, String n) { ... }
}
```

2.8.1.9 Aufruf generischer Methoden

2.8.1.9.1 Aufruf

- Aufruf mit Typ-Parameter vor Methodennamen, oder
- Inferenz durch Compiler

2.8.1.9.2 Finden der richtigen Methode durch den Compiler

1. Zuerst Suche nach exakt passender Methode,
2. danach passend mit Konvertierungen => Compiler sucht gemeinsame Oberklasse in Typhierarchie

2.8.1.9.3 Beispiel

```
class Mensch {
    <T> T myst(T m, T n) { ... }
}
Mensch m = new Mensch();


m.<String>myst("Essen", "lecker");  // Angabe Typ-Parameter


m.myst("Essen", 1);           // String, Integer => T: Object
m.myst("Essen", "lecker");    // String, String  => T: String
m.myst(1.0, 1);               // Double, Integer => T: Number
```

Beispiel methods.GenericMethods

Reihenfolge der Suche nach passender Methode gilt auch für nicht-generisch überladene Methoden

```
class Mensch {
    public <T> T myst(T m, T n) {
        System.out.println("X#myst: T");
        return m;
    }

    // NICHT gleichzeitig erlaubt wg. Typ-Löschung (s.u.):
/*
    public <T1, T2> T1 myst(T1 m, T2 n) {
        System.out.println("X#myst: T");
        return m;
    }
*/

    public String myst(String m, String n) {
        System.out.println("X#myst: String");
        return m;
    }

    public int myst(int m, int n) {
        System.out.println("X#myst: int");
        return m;
    }
}


public class GenericMethods {
    public static void main(String[] args) {
        Mensch m = new Mensch();

        m.myst("Hello World", "m");
        m.myst("Hello World", 1);
        m.myst(3, 4);
```

```
34          m.myst(m, m);
35          m.<Mensch>myst(m, m);
36          m.myst(m, 1);
37          m.myst(3.0, 4);
38          m.<Double>myst(3, 4);
39      }
40  }
```

2.8.1.10 Wrap-Up

- Begriffe:
  - Generischer Typ: `Stack<T>`
  - Formaler Typ-Parameter: `T`
  - Parametrisierter Typ:`Stack<Long>`
  - Typ-Parameter: `Long`
  - Raw Type: `Stack`
- Generische Klassen: `public class Stack<E> { }`
  - "`<Typ>`" hinter Klassennamen
- Generische Methoden: `public <T> T foo(T m){ }`
  - "`<Typ>`" vor Rückgabewert

Zum Nachlesen

- Ullenboom (2021, 11.1)
- Oracle Corporation (2022b)
- Oracle Corporation (2022c)
- Bloch (2018)

Lernziele

- k1: Begriffe generischer Typ, parametrisierter Typ, formaler Typ-Parameter, Typ-Parameter
- k3: Erstellen und Nutzen von generischen Klassen und Interfaces
- k3: Erstellen und Nutzen von generischen Methoden

Quizzes

- Quiz Generics: Classes & Methods (ILIAS)

2.8.2 Bounds & Wildcards

TL;DR
Typ-Variablen können weiter eingeschränkt werden, in dem man einen verpflichtenden Ober- oder Untertyp angibt mit **`extends`** bzw. **`super`**. Damit muss der später bei der Instantiierung verwendete Typ-Parameter entweder die Oberklasse selbst sein oder davon ableiten (bei **`extends`**) bzw. der Typ-Parameter muss eine Oberklasse der angegebenen Schranke sein (**`super`**).
Durch die Einschränkung mit **`extends`** können in der Klasse/Methode auf der Typ-Variablen alle Methoden des angegebenen Obertyps verwendet werden.
Ein Wildcard (?) als Typ-Parameter steht für einen beliebigen Typ, wobei die Typ-Variable keinen Namen bekommt und damit innerhalb der Klasse/Methode nicht zugreifbar ist.

Videos

- VL Generics: Bounds & Wildcards
- Demo Wildcards

2.8.2.1 Bounds: Einschränken der generischen Typen

```java
public class Cps<E extends Number> {
    // Obere Schranke: E muss Number oder Subklasse sein
    // => Zugriff auf Methoden aus Number moeglich
}
Cps<Double> a;
Cps<Number> b;
Cps<String> c;  // Fehler!!!
```

- Schlüsselwort **`extends`** gilt hier auch für Interfaces
- Mehrere Interfaces: nach **`extends`** Klasse oder Interface, danach mit "&" getrennt die restlichen Interfaces:

```java
class Cps<E extends KlasseOderInterface & I1 & I2 & I3> {}
```

Anmerkung: Der Typ-Parameter ist analog auch mit **`super`** (nach unten) einschränkbar

Beispiel bounds.Cps

2.8.2.2 Wildcards: Dieser Typ ist mir nicht so wichtig

Wildcard mit "?" => steht für unbestimmten Typ

```java
public class Wuppie {
    public void m1(List<?> a) { ... }
    public void m2(List<? extends Number> b) { ... }
}
```

- `m1`: `List` beliebig parametrisierbar
  => In `m1` für Objekte in Liste `a` nur Methoden von `Object` nutzbar!
- `m2`: `List` muss mit `Number` oder Subklasse parametrisiert werden.
  => Dadurch für Objekte in Liste `b` alle Methoden von `Number` nutzbar ...

Weitere Eigenschaften:

- Durch Wildcard kein Zugriff auf den Typ
- Wildcard kann durch upper bound eingeschränkt werden
- Geht nicht bei Klassen-/Interface-Definitionen

Bloch (2018): Nur für Parameter und nicht für Rückgabewerte nutzen!

2.8.2.3 Hands-On: Ausgabe für generische Listen

Ausgabe für Listen gesucht, die sowohl Elemente der Klasse `A` als auch Elemente der Klasse `B` enthalten können

```
1  class A { void printInfo() { System.out.println("A"); } }
2  class B extends A { void printInfo() { System.out.println("B"); } }
3
4  public class X {
5      public static void main(String[] args) {
6          List<A> x = new ArrayList<A>();
7          x.add(new A());  x.add(new B());
8          printInfo(x);    // Klassenmethode in X, gesucht
9          List<B> y = new ArrayList<B>();
10         y.add(new B());  y.add(new B());
11         printInfo(y);    // Klassenmethode in X, gesucht
12     }
13 }
```

Hinweis: Dieses Beispiel beinhaltet auch Polymorphie bei/mit generischen Datentypen, bitte vorher auch das Video zum vierten Teil "Generics und Polymorphie" anschauen

2.8.2.3.1 Erster Versuch (*A* und *B* und *main()* wie oben)

```
1  public class X {
2      public static void printInfo(List<A> list) {
3          for (A a : list) { a.printInfo(); }
4      }
5  }
```

=> **So gehts nicht!** Eine `List<B>` ist **keine** `List<A>` (auch wenn ein `B` ein `A` ist, vgl. spätere Sitzung zu Generics und Vererbung ...)!

Beispiel wildcards.v1.X

2.8.2.3.2 Zweiter Versuch mit Wildcards (*A* und *B* und *main()* wie oben)

```
1 public class X {
2     public static void printInfo(List<?> list) {
3         for (Object a : list) { a.printInfo(); }
4     }
5 }
```

=> **So gehts auch nicht!** Im Prinzip passt das jetzt für `List<A>` und `List<B>`. Dummerweise hat man durch das Wildcard keinen Zugriff mehr auf den Typ-Parameter und muss für den Typ der Laufvariablen in der **`for`**-Schleife dann `Object` nehmen. Aber `Object` kennt unser `printInfo` nicht … Außerdem könnte man die Methode `X#printInfo` dank des Wildcards auch mit allen anderen Typen aufrufen …

Beispiel wildcards.v2.X

2.8.2.3.3 Dritter Versuch (Lösung) mit Wildcards und Bounds (*A* und *B* und *main()* wie oben)

```
1 public class X {
2     public static void printInfo(List<? extends A> list) {
3         for (A a : list) { a.printInfo(); }
4     }
5 }
```

Das ist die Lösung. Man erlaubt als Argument nur `List`-Objekte und fordert, dass sie mit `A` oder einer Unterklasse von `A` parametrisiert sind. D.h. in der Schleife kann man sich auf den gemeinsamen Obertyp `A` abstützen und hat dann auch wieder die `printInfo`-Methode zur Verfügung …

Konsole wildcards.v3.X

2.8.2.4 Wrap-Up

- Ein Wildcard (`?`) als Typ-Parameter steht für einen beliebigen Typ
  - Ist in Klasse oder Methode dann aber nicht mehr zugreifbar
- Mit Bounds kann man Typ-Parameter nach oben oder nach unten einschränken (im Sinne einer Vererbungshierarchie)
  - **`extends`**: Der Typ-Parameter muss eine Unterklasse eines bestimmten Typen sein
  - **`super`**: Der Typ-Parameter muss eine Oberklasse eines bestimmten Typen sein

Zum Nachlesen

- Ullenboom (2021, 11.3)
- Oracle Corporation (2022b)
- Oracle Corporation (2022c)
- Bloch (2018)

Lernziele

- k3: Umgang mit Wildcards und Bounds bei generischen Klassen/Methoden

Quizzes

- Quiz Generics: Bounds & Wildcards (ILIAS)

Challenges

Spieler, Mannschaften und Ligen Modellieren Sie in Java verschiedene Spielertypen sowie generische Mannschaften und Ligen, die jeweils bestimmte Spieler (-typen) bzw. Mannschaften aufnehmen können.

1. Implementieren Sie die Klasse `Spieler`, die das Interface `ISpieler` erfüllt.

```java
public interface ISpieler {
    String getName();
}
```

2. Implementieren Sie die beiden Klassen `FussballSpieler` und `BasketballSpieler` und sorgen Sie dafür, dass beide Klassen vom Compiler als Spieler betrachtet werden (geeignete Vererbungshierarchie).
3. Betrachten Sie das nicht-generische Interface `IMannschaft`. Erstellen Sie daraus ein generisches Interface `IMannschaft` mit einer Typ-Variablen. Stellen Sie durch geeignete Beschränkung der Typ-Variablen sicher, dass nur Mannschaften mit von `ISpieler` abgeleiteten Spielern gebildet werden können.

```java
public interface IMannschaft {
    boolean aufnehmen(ISpieler spieler);
    boolean rauswerfen(ISpieler spieler);
}
```

4. Betrachten Sie das nicht-generische Interface `ILiga`. Erstellen Sie daraus ein generisches Interface `ILiga` mit einer Typvariablen. Stellen Sie durch geeignete Beschränkung der Typvariablen sicher, dass nur Ligen mit von `IMannschaft` abgeleiteten Mannschaften angelegt werden können.

```java
public interface ILiga {
    boolean aufnehmen(IMannschaft mannschaft);
    boolean rauswerfen(IMannschaft mannschaft);
}
```

5. Leiten Sie von `ILiga` das **generische** Interface `IBundesLiga` ab. Stellen Sie durch geeignete Formulierung der Typvariablen sicher, dass nur Ligen mit Mannschaften angelegt werden können, deren Spieler vom Typ `FussballSpieler` (oder abgeleitet) sind.

   Realisieren Sie nun noch die Funktionalität von `IBundesLiga` als **nicht-generisches** Interface `IBundesLiga2`.

2.8.3 Type Erasure

TL;DR

Generics existieren eigentlich nur auf Quellcode-Ebene. Nach der Typ-Prüfung etc. entfernt der Compiler alle generischen Typ-Parameter und alle <...> (=> "Type-Erasure"), d.h. im Byte-Code stehen nur noch Raw-Typen bzw. die oberen Typ-Schranken der Typ-Parameter, in der Regel `Object`. Zusätzlich baut der Compiler die nötigen Casts ein. Als Anwender merkt man davon nichts, muss das "Type-Erasure" wegen der Auswirkungen aber auf dem Radar haben!

Videos

- VL Generics: Type Erasure

2.8.3.1 Typ-Löschung (*Type-Erasure*)

Der Compiler ersetzt nach Prüfung der Typen und ihrer Verwendung alle Typ-Parameter durch

1. deren obere (Typ-)Schranke und
2. passende explizite Cast-Operationen (im Byte-Code).

Die obere Typ-Schranke ist in der Regel der Typ der ersten Bounds-Klausel oder `Object`, wenn keine Einschränkungen formuliert sind.

Bei parametrisierten Typen wie `List<T>` wird der Typ-Parameter entfernt, es entsteht ein sogenannter *Raw*-Typ (`List`, quasi implizit mit `Object` parametrisiert).

=> Ergebnis: Nur **eine** (untypisierte) Klasse! Zur Laufzeit gibt es keine Generics mehr!

Hinweis: In C++ ist man den anderen möglichen Weg gegangen und erzeugt für jede Instantiierung die passende Klasse. Siehe Modul “Systemprogrammierung” :)

Beispiel: Aus dem folgenden harmlosen Code-Fragment:

```
class Studi<T> {
    T myst(T m, T n) { return n; }

    public static void main(String[] args) {
        Studi<Integer> a = new Studi<>();
        int i = a.myst(1, 3);
    }
}
```

---

wird nach der Typ-Löschung durch Compiler (das steht dann quasi im Byte-Code):

```
class Studi {
    Object myst(Object m, Object n) { return n; }

    public static void main(String[] args) {
        Studi a = new Studi();
        int i = (Integer) a.myst(1, 3);
    }
}
```

Die obere Schranke meist `Object` => `new T()` verboten/sinnfrei (s.u.)!

2.8.3.2 Type-Erasure bei Nutzung von Bounds

vor der Typ-Löschung durch Compiler:

```
class Cps<T extends Number> {
    T myst(T m, T n) {
        return n;
    }
```

```
5
6       public static void main(String[] args) {
7           Cps<Integer> a = new Cps<>();
8           int i = a.myst(1, 3);
9       }
10  }
```

nach der Typ-Löschung durch Compiler:

```
1   class Cps {
2       Number myst(Number m, Number n) {
3           return n;
4       }
5
6       public static void main(String[] args) {
7           Cps a = new Cps();
8           int i = (Integer) a.myst(1, 3);
9       }
10  }
```

2.8.3.3 Raw-Types: Ich mag meine Generics "well done" :-)

Raw-Types: Instanziierung ohne Typ-Parameter => `Object`

```
1   Stack s = new Stack(); // Stack von Object-Objekten
```

- Wegen Abwärtskompatibilität zu früheren Java-Versionen noch erlaubt.
- Nutzung wird nicht empfohlen! (Warum?)

2.8.3.3.1 Anmerkung

Raw-Types darf man zwar selbst im Quellcode verwenden (so wie im Beispiel hier), **sollte** die Verwendung aber vermeiden wegen der Typ-Unsicherheit: Der Compiler sieht im Beispiel nur noch einen Stack für `Object`, d.h. dort dürfen Objekte aller Typen abgelegt werden - es kann keine Typprüfung durch den Compiler stattfinden. Auf einem `Stack<String>` kann der Compiler prüfen, ob dort wirklich nur `String`-Objekte abgelegt werden und ggf. entsprechend Fehler melden.

Etwas anderes ist es, dass der Compiler im Zuge von Type-Erasure selbst Raw-Types in den Byte-Code schreibt. Da hat er vorher bereits die Typsicherheit geprüft und er baut auch die passenden Casts ein.

Das Thema ist eigentlich nur noch aus Kompatibilität zu Java5 oder früher da, weil es dort noch keine Generics gab (wurden erst mit Java6 eingeführt).

2.8.3.4 Folgen der Typ-Löschung: *new*

new mit parametrisierten Klassen ist nicht erlaubt!

```java
class Fach<T> {
    public T foo() {
        return new T();  // nicht erlaubt!!!
    }
}
```

Grund: Zur Laufzeit keine Klasseninformationen über `T` mehr

Im Code steht `return (CAST)new Object();`. Das neue Object kann man anlegen, aber ein Cast nach irgendeinem anderen Typ ist sinnfrei: Jede Klasse ist ein Untertyp von `Object`, aber eben nicht andersherum. Außerdem fehlt dem Objekt vom Typ `Object` auch sämtliche Information und Verhalten, die der Cast-Typ eigentlich mitbringt …

2.8.3.5 Folgen der Typ-Löschung: *static*

`static` mit generischen Typen ist nicht erlaubt!

```java
class Fach<T> {
    static T t;                    // nicht erlaubt!!!
    static Fach<T> c;              // nicht erlaubt!!!
    static void foo(T t) { ... };  // nicht erlaubt!!!
}

Fach<String>  a;
Fach<Integer> b;
```

Grund: Compiler generiert nur eine Klasse! Beide Objekte würden sich die statischen Attribute teilen (Typ zur Laufzeit unklar!).

Hinweis: Generische (statische) Methoden sind erlaubt.

2.8.3.6 Folgen der Typ-Löschung: *instanceof*

`instanceof` mit parametrisierten Klassen ist nicht erlaubt!

```java
class Fach<T> {
    void printType(Fach<?> p) {
        if (p instanceof Fach<Number>)
            ...
        else if (p instanceof Fach<String>)
            ...
    }
}
```

Grund: Unsinniger Code nach Typ-Löschung:

```
class Fach {
void printType(Fach p) {
    if (p instanceof Fach)
        ...
    else if (p instanceof Fach)
        ...
    }
}
```

2.8.3.7 Folgen der Typ-Löschung: *.class*

`.class` mit parametrisierten Klassen ist nicht erlaubt!

```
boolean x;
List<String>  a = new ArrayList<String>();
List<Integer> b = new ArrayList<Integer>();

x = (List<String>.class == List<Integer>.class);  // Compiler-Fehler
x = (a.getClass() == b.getClass());               // true
```

Grund: Es gibt nur `List.class` (und kein `List<String>.class` bzw. `List<Integer>.class`)!

2.8.3.8 Wrap-Up

- Generics existieren eigentlich nur auf Quellcode-Ebene
- “Type-Erasure”:
  - Compiler entfernt nach Typ-Prüfungen etc. generische Typ-Parameter etc. => im Byte-Code nur noch Raw-Typen bzw. die oberen Typ-Schranken der Typ-Parameter, in der Regel `Object`
  - Compiler baut passende Casts in Byte-Code ein
  - Transparent für User; Auswirkungen beachten!

Zum Nachlesen

- Ullenboom (2021, 11.2 und 11.6)
- Oracle Corporation (2022b)
- Oracle Corporation (2022c)
- Bloch (2018)

Lernziele

- k2: Typ-Löschung und Auswirkungen

Quizzes

- Quiz Generics: Type Erasure (ILIAS)

2.8.4 Generics und Polymorphie

TL;DR
Auch mit generischen Klassen stehen die Mechanismen Vererbung und Überladen zur Verfügung. Dabei muss aber beachtet werden, dass generische Klassen sich **"invariant"** verhalten: Der Typ selbst folgt der Vererbungsbeziehung, eine Vererbung des Typ-Parameters begründet *keine* Vererbungsbeziehung! D.h. aus `U extends O` folgt **nicht** `A<U> extends A<O>`.
Bei Arrays ist es genau anders herum: Wenn `U extends O` dann gilt auch `U[] extends O[]` … (Dies nennt man "*kovariantes*" Verhalten.)

Videos

- VL Generics und Polymorphie

2.8.4.1 Generische Polymorphie

`B<E> extends A<E>`

```
1  class A<E> { ... }
2  class B<E> extends A<E> { ... }
3
4  A<Double> ad = new B<Double>();
5  A<String> as = new B<String>();
```

```
1  class Vector<E> { ... }
2  class Stack<E> extends Vector<E> { ... }
3
4  Vector<Double> vd = new Stack<Double>();
5  Vector<String> vs = new Stack<String>();
```

=> Polymorphie bei Generics bezieht sich auf **Typ** (nicht Typ-Parameter)

Invarianz: Generics sind *invariant*, d.h. ein `HashSet<String>` ist ein Untertyp von `Set<String>`. Bei der Vererbung muss der Typ-Parameter identisch sein.

2.8.4.2 Polymorphie bei Generics bezieht sich nur auf Typ!

"`B extends A`" **bedeutet nicht** "`C<B> extends C<A>`"

```java
Stack<Number> s = new Stack<Integer>(); // DAS GEHT SO NICHT!

// Folgen (wenn obiges gehen wuerde):
s.push(new Integer(3)); // das ginge sowieso ...

// Folgen (wenn obiges gehen wuerde):
// Stack<Number> waere Oberklasse auch von Stack<Double>
s.push(new Double(2.0)); // waere dann auch erlaubt ...

// Das Objekt (Stack<Integer>) kann aber keine Double speichern!
// Zur Laufzeit keine Typ-Informationen mehr!
```

- Typ-Löschung => zur Laufzeit keine Typinformationen vorhanden
- Compiler muss Typen prüfen (können)!

2.8.4.3 Abgrenzung: Polymorphie bei Arrays

Wenn "`B extends A`" dann "`B[] extends A[]`"

```java
Object[] x = new String[] {"Hello", "World", ":-)"};
x[0] = "Hallo";
x[0] = new Double(2.0);  // Laufzeitfehler
String[] y = x;  // String[] ist KEIN Object[]!!!
```

- Arrays besitzen Typinformationen über gespeicherte Elemente
- Prüfung auf Typ-Kompatibilität zur **Laufzeit** (nicht Kompilierzeit!)

Hinweis auf Java-Geschichte (Java-Insel: "Type Erasure")

Arrays gab es sehr früh, Generics erst relativ spät (ab Java6) => bei Arrays fand man das Verhalten natürlich und pragmatisch (trotz der Laufzeit-Überprüfung).

Bei der Einführung von Generics musste man Kompatibilität sicherstellen (alter Code soll auch mit neuen Compilern übersetzt werden können - obwohl im alten Code Raw-Types verwendet werden). Außerdem wollte man von Laufzeit-Prüfung hin zu Compiler-Prüfung. Da würde das von Arrays bekannte Verhalten Probleme machen ...

Kovarianz: Arrays sind *kovariant*, d.h. ein Array vom Typ `String[]` ist wegen `String extends Object` ein Untertyp von `Object[]`.

Beispiel arrays.X

2.8.4.4 Arrays vs. parametrisierte Klassen

=> Keine Arrays mit parametrisierten Klassen!

```java
Foo<String>[] x = new Foo<String>[2];   // Compilerfehler

Foo<String[]> y = new Foo<String[]>();  // OK :)
```

Arrays mit parametrisierten Klassen sind nicht erlaubt! Arrays brauchen zur Laufzeit Typinformationen, die aber durch die Typ-Löschung entfernt werden.

2.8.4.5 Diskussion Vererbung vs. Generics **Vererbung:**

- *IS-A*-Beziehung
- Anwendung: Vererbungsbeziehung vorliegend, Eigenschaften verfeinern
- Beispiel: Ein Student *ist eine* Person

Generics:

- Schablone (Template) für viele Datentypen
- Anwendung: Identischer Code für unterschiedliche Typen
- Beispiel: Datenstrukturen, Algorithmen generisch realisieren

2.8.4.6 Wrap-Up

- Generics: Vererbung und Überladen möglich, aber:
  Aus "`U extends O`" folgt **nicht** "`A<U> extends A<O>`"
- Achtung: Bei Arrays gilt aber: Wenn "`U extends O`" dann gilt auch "`U[] extends O[]`" …

Zum Nachlesen

- Ullenboom (2021, 11.5)
- Oracle Corporation (2022b)
- Oracle Corporation (2022c)
- Bloch (2018)

Lernziele

- k3: Vererbungsbeziehungen mit generischen Klassen
- k3: Umgang mit Arrays und generischen Typen

Quizzes

- Quiz Generics und Polymorphie (ILIAS)

2.9 Multi-Threading: Parallelisierung von Programmen

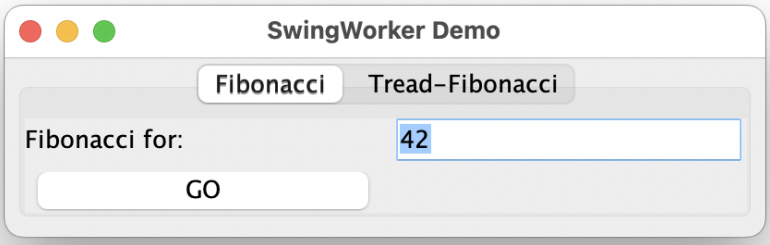
2.9.1 Einführung in die nebenläufige Programmierung mit Threads

TL;DR
Threads sind weitere Kontrollflussfäden, die von der Java-VM (oder (selten) vom OS) verwaltet werden. Damit ist sind sie leichtgewichtiger als der Start neuer Prozesse direkt auf Betriebssystem-Ebene.
Beim Start eines Java-Programms wird die `main()`-Methode automatisch in einem (Haupt-) Thread ausgeführt. Alle Anweisungen in einem Thread werden sequentiell ausgeführt.
Um einen neuen Thread zu erzeugen, leitet man von `Thread` ab oder implementiert das Interface `Runnable`. Von diesen eigenen Klassen kann man wie üblich ein neues Objekt anlegen. Die Methode `run()` enthält dabei den im Thread auszuführenden Code. Um einen Thread als neuen parallelen Kontrollfluss zu starten, muss man die geerbte Methode `start()` auf dem Objekt aufrufen. Im Fall der Implementierung von `Runnable` muss man das Objekt zuvor noch in den Konstruktor von `Thread` stecken und so ein neues `Thread`-Objekt erzeugen, auf dem man dann `start()` aufrufen kann.
Threads haben einen Lebenszyklus: Nach dem Erzeugen der Objekte mit **new** wird der Thread noch nicht ausgeführt. Durch den Aufruf der Methode `start()` gelangt der Thread in einen Zustand "ausführungsbereit". Sobald er vom Scheduler eine Zeitscheibe zugeteilt bekommt, wechselt er in den Zustand "rechnend". Von hier kann er nach Ablauf der Zeitscheibe durch den Scheduler wieder nach "ausführungsbereit" zurück überführt werden. Dieses Wechselspiel passiert automatisch und i.d.R. schnell, so dass selbst auf Maschinen mit nur einem Prozessor/Kern der Eindruck einer parallelen Verarbeitung entsteht. Nach Abarbeitung der `run()`-Methode wird der Thread beendet und kann nicht wieder neu gestartet werden. Bei Zugriff auf gesperrte Ressourcen oder durch `sleep()` oder `join()` kann ein Thread blockiert werden. Aus diesem Zustand gelangt er durch Interrupts oder nach Ablauf der Schlafzeit oder durch `notify` wieder zurück nach "ausführungsbereit".
Die Thread-Objekte sind normale Java-Objekte. Man kann hier Attribute und Methoden haben und diese entsprechend zugreifen/aufrufen. Das klappt auch, wenn der Thread noch nicht gestartet wurde oder bereits abgearbeitet wurde.

Videos

- VL Threads Intro
- Demo Threads Intro: Erzeugen von Threads
- Demo Threads Intro: Arbeiten mit Threads (`join()`)

2.9.1.1 42



Demo: misc.SwingWorkerDemo (GUI ausprobieren)

2.9.1.2 Einführung in nebenläufige Programmierung

2.9.1.2.1 Traditionelle Programmierung

- Aufruf einer Methode verlagert Kontrollfluss in diese Methode
- Code hinter Methodenaufruf wird erst **nach Beendigung** der Methode ausgeführt

```
1  public class Traditional {
2      public static void main(String... args) {
3          Traditional x = new Traditional();
4
5          System.out.println("main(): vor run()");
6          x.run();
7          System.out.println("main(): nach run()");
8      }
9
10     public void run() {
11         IntStream.range(0, 10).mapToObj(i -> "in run()").forEach(System.out::
               println);
12     }
13 }
```

Demo: intro.Traditional

2.9.1.2.2 Nebenläufige Programmierung

- Erzeugung eines neuen Kontrollflussfadens (Thread)
  - **Läuft (quasi-) parallel zu bisherigem Kontrollfluss**
- Threads können unabhängig von einander arbeiten
- Zustandsverwaltung durch Java-VM (oder Unterstützung durch Betriebssystem)
  - Aufruf einer bestimmten Methode erzeugt neuen Kontrollflussfaden

- Der neue Thread arbeitet “parallel” zum bisherigen Thread
- Kontrolle kehrt sofort wieder zurück: Code hinter dem Methodenaufruf wird ausgeführt ohne auf die Beendigung der aufgerufenen Methode zu warten
- Verteilung der Threads auf die vorhandenen Prozessorkerne abhängig von der Java-VM

```java
public class Threaded extends Thread {
    public static void main(String... args) {
        Threaded x = new Threaded();

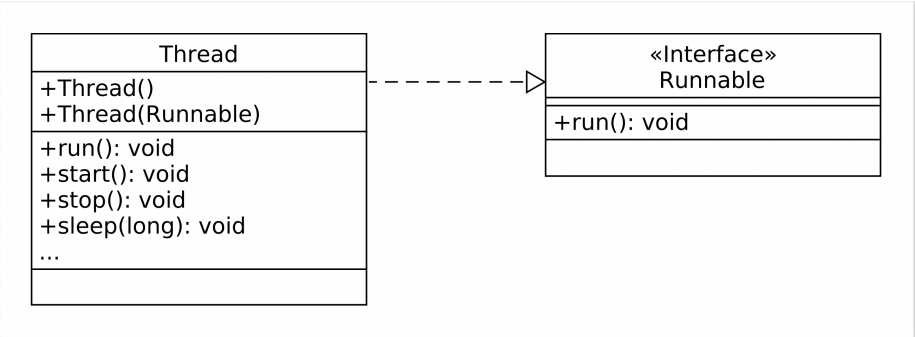
        System.out.println("main(): vor run()");
        x.start();
        System.out.println("main(): nach run()");
    }

    @Override
    public void run() {
        IntStream.range(0, 10).mapToObj(i -> "in run()").forEach(System.out::
            println);
    }
}
```

Demo: intro.Threaded

2.9.1.3 Erzeugen von Threads

- Ableiten von `Thread` oder Implementierung von `Runnable`



- Methode `run()` implementieren, aber nicht aufrufen
- Methode `start()` aufrufen, aber (i.d.R.) nicht implementieren

Demo: creation.*

2.9.1.3.1 Ableiten von *Thread*

- `start()` startet den Thread und sorgt für Ausführung von `run()`
- `start()` nur einmal aufrufen

2.9.1.3.2 Implementierung von *Runnable*

- Ebenfalls `run()` implementieren
- Neues `Thread`-Objekt erzeugen, Konstruktor das eigene Runnable übergeben
- Für Thread-Objekt die Methode `start()` aufrufen
  - Startet den Thread (das Runnable) und sorgt für Ausführung von `run()`

Vorteil von `Runnable`: Ist ein Interface, d.h. man kann noch von einer anderen Klasse erben

2.9.1.4 Zustandsmodell von Threads (vereinfacht)

Threads haben einen Lebenszyklus: Nach dem Erzeugen der Objekte mit **new** wird der Thread noch nicht ausgeführt. Er ist sozusagen in einem Zustand "erzeugt". Man kann bereits mit dem Objekt interagieren, also auf Attribute zugreifen und Methoden aufrufen.

Durch den Aufruf der Methode `start()` gelangt der Thread in einen Zustand "ausführungsbereit", er läuft also aus Nutzersicht. Allerdings hat er noch keine Ressourcen zugeteilt (CPU, ...), so dass er tatsächlich noch nicht rechnet. Sobald er vom Scheduler eine Zeitscheibe zugeteilt bekommt, wechselt er in den Zustand "rechnend" und führt den Inhalt der `run()`-Methode aus. Von hier kann er nach Ablauf der Zeitscheibe durch den Scheduler wieder nach "ausführungsbereit" zurück überführt werden. Dieses Wechselspiel passiert automatisch und i.d.R. schnell, so dass selbst auf Maschinen mit nur einem Prozessor/Kern der Eindruck einer parallelen Verarbeitung entsteht.

Nach der Abarbeitung der `run()`-Methode oder bei einer nicht gefangenen Exception wird der Thread beendet und kann nicht wieder neu gestartet werden. Auch wenn der Thread abgelaufen ist, kann man mit dem Objekt wie üblich interagieren (nur eben nicht mehr parallel).

Bei Zugriff auf gesperrte Ressourcen oder durch Aufrufe von Methoden wie `sleep()` oder `join()` kann ein Thread blockiert werden. Hier führt der Thread nichts aus, bekommt durch den Scheduler aber auch keine neue Zeitscheibe zugewiesen. Aus diesem Zustand gelangt der Thread wieder heraus, etwa durch Interrupts (Aufruf der Methode `interrupt()` auf dem Thread-Objekt) oder nach Ablauf der Schlafzeit (in `sleep()`) oder durch ein `notify`, und wird wieder zurück nach "ausführungsbereit" versetzt und wartet auf die Zuteilung einer Zeitscheibe durch den Scheduler.

Sie finden in (Boles 2008, 5.2 "Thread-Zustände") eine schöne ausführliche Darstellung.

2.9.1.4.1 Threads können wie normale Objekte kommunizieren

- Zugriff auf (**public**) Attribute (oder eben über Methoden)
- Aufruf von Methoden

2.9.1.4.2 Threads können noch mehr

- Eine Zeitlang schlafen: `Thread.sleep(<duration_ms>)`
  - Statische Methode der Klasse `Thread` (Klassenmethode)
  - Aufrufender Thread wird bis zum Ablauf der Zeit oder bis zum Aufruf der `interrupt()`-Methode des Threads blockiert
  - "Moderne" Alternative: `TimeUnit`, beispielsweise `TimeUnit.SECONDS.sleep( 2 );`

- Prozessor abgeben und hinten in Warteschlange einreihen: `yield()`
- Andere Threads stören: `otherThreadObj.interrupt()`
  - Die Methoden `sleep()`, `wait()` und `join()` im empfangenden Thread `otherThreadObj` lösen eine `InterruptedException` aus, wenn sie durch die Methode `interrupt()` unterbrochen werden. Das heißt, `interrupt()` beendet diese Methoden mit der Ausnahme.
  - Empfangender Thread verlässt ggf. den Zustand "blockiert" und wechselt in den Zustand "ausführungsbereit"
- Warten auf das Ende anderer Threads: `otherThreadObj.join()`
  - Ausführender Thread wird blockiert (also nicht `otherThreadObj`!)
  - Blockade des Aufrufers wird beendet, wenn der andere Thread (`otherThreadObj`) beendet wird.

Hinweis: Ein Thread wird beendet, wenn

- die `run()`-Methode normal endet, oder
- die `run()`-Methode durch eine nicht gefangene Exception beendet wird, oder
- von außen die Methode `stop()` aufgerufen wird (Achtung: Deprecated! Einen richtigen Ersatz gibt es aber auch nicht.).

Hinweis: Die Methoden `wait()`, `notify()`/`notifyAll()` und die "**`synchronized`**-Sperre" werden in der Sitzung ["Threads: Synchronisation"](threads-intro. besprochen.

Demo: intro.Join

2.9.1.5 Wrap-Up

Threads sind weitere Kontrollflussfäden, von Java-VM (oder (selten) von OS) verwaltet

- Ableiten von `Thread` oder implementieren von `Runnable`
- Methode `run` enthält den auszuführenden Code
- Starten des Threads mit `start` (nie mit `run`!)

Zum Nachlesen

- Ullenboom (2021, Kap. 16)
- Oracle Corporation (2022c)
- Boles (2008)

Lernziele

- k2: Grundsätzlicher Unterschied zw. Threads und Prozessen
- k2: Lebenszyklus von Threads
- k3: Erzeugen und Starten von Threads
- k3: Kommunikation mit Objekten

Quizzes

- Quiz Threads Intro (ILIAS)

2.9.2 Synchronisation: Verteilter Zugriff auf gemeinsame Ressourcen

TL;DR
Bei verteiltem Zugriff auf gemeinsame Ressourcen besteht Synchronisierungsbedarf, insbesondere sollten nicht mehrere Threads gleichzeitig geteilte Daten modifizieren. Dazu kommt das Problem, dass ein Thread in einer komplexen Folge von Aktionen die Zeitscheibe verlieren kann und dann später mit veralteten Daten weiter macht. Um den Zugriff auf gemeinsame Ressourcen oder den Eintritt in kritische Bereiche zu schützen und zu synchronisieren, kann man diese Zugriffe oder Bereiche in einen `synchronized`-Block legen. Dazu benötigt man noch ein beliebiges (gemeinsam sichtbares) Objekt, welches als Wächter- oder Sperr-Objekt fungiert. Beim Eintritt in den geschützten Block muss ein Thread einen Lock auf dem Sperr-Objekt erlangen. Hat bereits ein anderer Thread den Lock, wird der neue Thread so lange blockiert, bis der Lock wieder "frei" ist. Beim Eintritt in den Bereich wird dann durch den Thread auf dem Sperr-Objekt der Lock gesetzt und beim Austritt automatisch wieder aufgehoben. Dies nennt man auch **mehrseitige Synchronisierung** (mehrere Threads "stimmen" sich quasi untereinander über den Zugriff auf eine Ressource ab).
Um auf den Eintritt eines Ereignisses oder die Erfüllung einer Bedingung zu warten, kann man `wait` und `notify` nutzen. In einem `synchronized`-Block prüft man, ob die Bedingung erfüllt oder ein Ereignis eingetreten ist, und falls ja arbeitet man damit normal weiter. Falls die Bedingung nicht erfüllt ist oder das Ereignis nicht eingetreten ist, kann man auf dem im `synchronized`-Block genutzten Sperr-Objekt die Methode `wait()` aufrufen. Damit wird der Thread in die entsprechende Schlange auf dem Sperr-Objekt eingereiht und blockiert. Zusätzlich wird der Lock auf dem Sperr-Objekt freigegeben. Zum "Aufwecken" nutzt man an geeigneter Stelle auf dem **selben Sperr-Objekt** die Methode `notify()` oder `notifyALl()` (erstere weckt einen in der Liste des Sperr-Objekts wartenden Thread, die letztere alle). Nach dem Aufwachen macht der Thread nach seinem `wait()` weiter. Es ist also wichtig, dass die Bedingung, wegen der ursprünglich das `wait()` aufgerufen wurde, erneut abgefragt wird und ggf. erneut in das `wait()` gegangen wird. Dies nennt man **einseitige Synchronisierung**.
Es gibt darüber hinaus viele weitere Mechanismen und Probleme, die aber den Rahmen dieser Lehrveranstaltung deutlich übersteigen. Diese werden teilweise in den Veranstaltungen "Betriebssysteme" und/oder "Verteilte Systeme" besprochen.

Videos

- VL Threads Synchronisation
- Demo Teaser: Falscher Zugriff auf gemeinsame Ressourcen
- Demo Mehrseitige Synchronisation (Sperr-Objekt, synchronisierte Methode)
- Demo Mehrseitige Synchronisation: Deadlock
- Demo Einseitige Synchronisation

2.9.2.1 Motivation: Verteilter Zugriff auf gemeinsame Ressourcen

```java
public class Teaser implements Runnable {
    private int val = 0;

    public static void main(String... args) {
        Teaser x = new Teaser();
        new Thread(x).start();
        new Thread(x).start();
    }

    private void incrVal() {
        ++val;
        System.out.println(Thread.currentThread().getId() + ": " + val);
    }

    public void run() {
        IntStream.range(0, 5).forEach(i -> incrVal());
    }
}
```

Demo: synchronised.Teaser

2.9.2.2 Zugriff auf gemeinsame Ressourcen: Mehrseitige Synchronisierung

```java
synchronized (<Object reference>) {
    <statements (synchronized)>
}
```

=> **"Mehrseitige Synchronisierung"**

Fallunterscheidung: Thread T1 führt `synchronized`-Anweisung aus:

- Sperre im Sperr-Objekt nicht gesetzt:
  1. T1 setzt Sperre beim Eintritt,
  2. führt den Block aus, und
  3. löst Sperre beim Verlassen
- Sperre durch T1 gesetzt:
  1. T1 führt den Block aus, und
  2. löst Sperre beim Verlassen **nicht**
- Sperre durch T2 gesetzt: => T1 wird blockiert, bis T2 die Sperre löst

Anmerkung: Das für die Synchronisierung genutzte Objekt nennt man “Wächter-Objekt” oder auch “Sperr-Objekt” oder auch “Synchronisations-Objekt”.

Damit könnte man den relevanten Teil der Methode `incrVal()` beispielsweise in einen geschützten Bereich einschließen und als Sperr-Objekt das eigene Objekt (`this`) einsetzen:

```
1      private void incrVal() {
2          synchronized (this) { ++val; }
3      }
```

Demo: synchronised.ObjSync

2.9.2.3 Synchronisierte Methoden

```
1  void f() {
2      synchronized (this) {
3          ...
4      }
5  }
```

… ist äquivalent zu …

```
1  synchronized void f() {
2      ...
3  }
```

Kurzschreibweise: Man spart das separate Wächter-Objekt und synchronisiert auf sich selbst …

Die Methode `incrVal()` könnte entsprechend so umgeschrieben werden:

```
1      private synchronized void incrVal() {
2          ++val;
3      }
```

Demo: synchronised.MethodSync

2.9.2.4 Probleme bei der (mehrseitigen) Synchronisierung: Deadlocks

```
1  public class Deadlock {
2      private final String name;
3
4      public synchronized String getName() { return name; }
5      public synchronized void foo(Deadlock other) {
6          System.out.format("%s: %s.foo() \n", Thread.currentThread().getName(),
               name);
7          System.out.format("%s: %s.name()\n", Thread.currentThread().getName(),
               other.getName());
8      }
9
10     public static void main(String... args) {
11         final Deadlock a = new Deadlock("a");
12         final Deadlock b = new Deadlock("b");
13
14         new Thread(() -> a.foo(b)).start();
15         new Thread(() -> b.foo(a)).start();
```

```
16        }
17    }
```

Viel hilft hier nicht viel! Durch zu großzügige mehrseitige Synchronisierung kann es passieren, dass Threads gegenseitig aufeinander warten: Thread A belegt eine Ressource, die ein anderer Thread B haben möchte und Thread B belegt eine Ressource, die A gerne bekommen würde. Da es dann nicht weitergeht, nennt man diese Situation auch “Deadlock” (“Verklemmung”).

Im Beispiel ruft der erste Thread für das Objekt `a` die `foo()`-Methode auf und holt sich damit den Lock auf `a`. Um die Methode beenden zu können, muss noch die `getName()`-Methode vom Objekt `b` durch diesen ersten Thread aufgerufen werden. Dafür muss der erste Thread den Lock auf `b` bekommen.

Dummerweise hat parallel der zweite Thread auf dem Objekt `b` die `foo()`-Methode aufgerufen und sich damit den Lock auf `b` geholt. Damit muss der erste Thread so lange warten, bis der zweite Thread den Lock auf `b` freigibt.

Das wird allerdings nicht passieren, da der zweite Thread zur Beendigung der `foo()`-Methode noch `getName()` auf `a` ausführen muss und dazu den Lock auf `b` holen, den aber aktuell der erste Thread hält.

Und schon geht’s nicht mehr weiter :-)

Demo: synchronised.Deadlock

2.9.2.5 Warten auf andere Threads: Einseitige Synchronisierung

2.9.2.5.1 Problem

- Thread T1 wartet auf Arbeitsergebnis von T2
- T2 ist noch nicht fertig

2.9.2.5.2 Mögliche Lösungen

1. Aktives Warten (Polling): Permanente Abfrage
   - Kostet unnötig Rechenzeit
2. Schlafen mit `Thread.sleep()`
   - Etwas besser; aber wie lange soll man idealerweise schlafen?
3. Warten mit `T2.join()`
   - Macht nur Sinn, wenn T1 auf das *Ende* von T2 wartet
4. **Einseitige Synchronisierung** mit `wait()` und `notify()`
   - Das ist DIE Lösung für das Problem :)

2.9.2.6 Einseitige Synchronisierung mit *wait* und *notify*

- **wait**: Warten auf Erfüllung einer Bedingung (Thread blockiert):

```
1 synchronized (obj) {    // Geschützten Bereich betreten
2     while (!condition) {
3         try {
4             obj.wait(); // Thread wird blockiert
5         } catch (InterruptedException e) {}
6     }
7     ...     // Condition erfüllt: Tue Deine Arbeit
8 }
```

=> Bedingung nach Rückkehr von `wait` erneut prüfen!

2.9.2.6.1 Eigenschaften von *wait*

- Thread ruft auf Synchronisations-Objekt die Methode `wait` auf
- Prozessor wird entzogen, Thread blockiert
- Thread wird in interne Warteschlange des Synchronisations-Objekts eingetragen
- Sperre auf Synchronisations-Objekt wird freigegeben

=> Geht nur innerhalb der **`synchronized`**-Anweisung für das Synchronisations-Objekt!

2.9.2.7 Einseitige Synchronisierung mit *wait* und *notify* (cnt.)

- **notify**: Aufwecken von wartenden (blockierten) Threads:

```
1 synchronized (obj) {
2     obj.notify();       // einen Thread "in" obj aufwecken
3     obj.notifyAll();    // alle Threads "in" obj wecken
4 }
```

2.9.2.7.1 Eigenschaften von *notify* bzw. *notifyAll*

- Thread ruft auf einem Synchronisations-Objekt die Methode `notify` oder `notifyAll` auf
- Falls Thread(s) in Warteschlange des Objekts vorhanden, dann
  - `notify`: Ein *zufälliger* Thread wird aus Warteschlange entfernt und in den Zustand “ausführungsbereit” versetzt
  - `notifyAll`: Alle Threads werden aus Warteschlange entfernt und in den Zustand “ausführungsbereit” versetzt

=> Geht nur innerhalb der **`synchronized`**-Anweisung für das Synchronisations-Objekt!

Demo: synchronised.Staffel

2.9.2.8 Wrap-Up

Synchronisierungsbedarf bei verteiltem Zugriff auf gemeinsame Ressourcen:

- Vorsicht mit konkurrierendem Ressourcenzugriff:
  Synchronisieren mit `synchronized` => **Mehrseitige Synchronisierung**
- Warten auf Ereignisse mit `wait` und `notify`/`notifyAll` => **Einseitige Synchronisierung**

Zum Nachlesen

- Oracle Corporation (2022c)
- Boles (2008)

Lernziele

- k2: Notwendigkeit zur Synchronisation
- k2: Unterscheidung einseitige und mehrseitige Synchronisation
- k3: Synchronisation mit synchronized, wait, notify und notifyAll

Quizzes

- Quiz Threads Synchronisation (ILIAS)

Challenges

In den Vorgaben finden Sie eine Modellierung für ein Bankensystem.
Erweitern Sie die Vorgaben um Multithreading.
Erweitern Sie die Klasse `Kunde` so, dass sie in einem eigenen Thread ausgeführt werden kann. In der `run()`-Methode soll der `Kunde` eine `Rechnung` aus der Queue `offeneRechnungen` herausnehmen und sie bezahlen. Nutzen Sie dafür die statische Methode `Bank#ueberweisen`. Ist die Queue leer, soll der Thread so lange warten, bis eine neue Rechnung eingegangen ist. Nutzen Sie dafür einseitige Synchronisation.
Erweitern Sie die Klasse `Transaktion` so, dass sie in einem eigenen Thread ausgeführt werden kann. In der `run()`-Methode soll die `Transaktion` ausgeführt werden. Dabei soll vom Konto `von` der in der Rechnung hinterlegte Betrag abgezogen werden. Nutzen Sie dafür die Methode `Konto#sendeGeld`. Wenn das Geld erfolgreich abgezogen worden ist, soll das Geld auf das Empfängerkonto überwiesen werden. Nutzen Sie dafür die Methode `Konto#empfangeGeld`. Verwenden Sie mehrseitige Synchronisation.
Passen Sie die Methode `Bank#ueberweisen` so an, dass diese einen `Transaktion`-Thread erstellt und startet. Verwenden Sie dafür eine passende Struktur und setzen Sie die Executor-API ein.
Implementieren Sie die Klasse `Geldeintreiber`. Diese bekommt einen `Kunden` als Auftraggeber und eine Liste mit weiteren Kunden als Rechnungsempfänger übergeben. Implementieren Sie den `Geldeintreber` so, dass dieser in einem eigenen Thread ausgeführt werden kann. In der `run()`-Methode soll der `Geldeintreiber` eine Rechnung generieren und an einen der `Kunden` in der Liste schicken. Verwenden Sie dafür die Methode `Kunde#empfangeRechnung`. Das Ziel-`Konto` der `Rechnung` soll das `Konto` des Auftraggebers sein. Der `Geldeintreiber` macht nach jeder versendeten Rechnung fünf Sekunden Pause.
Hinweis: Achten Sie darauf, nur die nötigsten Ressourcen zu blockieren und auch nur so lange wie unbedingt nötig.

2.9.3 High-Level Concurrency

TL;DR
Das Erzeugen von Threads über die Klasse `Thread` oder das Interface `Runnable` und das Hantieren mit **`synchronized`** und `wait()`/`notify()` zählt zu den grundlegenden Dingen beim Multi-Threading mit Java. Auf diesen Konzepten bauen viele weitere Konzepte auf, die ein flexibleres Arbeiten mit Threads in Java ermöglichen.
Dazu zählt unter anderem das Arbeiten mit `Lock`-Objekten und dazugehörigen `Conditions`, was **`synchronized`** und `wait()`/`notify()` entspricht, aber feingranulareres und flexibleres Locking bietet.
Statt Threads immer wieder neu anzulegen (das Anlegen von Objekten bedeutet einen gewissen Aufwand zur Laufzeit), kann man Threads über sogenannte Thread-Pools wiederverwenden und über das Executor-Interface benutzen.
Schließlich bietet sich das Fork/Join-Framework zum rekursiven Zerteilen von Aufgaben und zur parallelen Bearbeitung der Teilaufgaben an.
Die in Swing integrierte Klasse `SwingWorker` ermöglicht es, in Swing Berechnungen in einen parallel ausgeführten Thread auszulagern.

Videos

- VL High-Level Concurrency
- Demo Lock-Objekte
- Demo Executor
- Demo Fork/Join
- Demo SwingWorker

2.9.3.1 Explizite Lock-Objekte Sie kennen bereits die Synchronisierung mit dem Schlüsselwort **`synchronized`**.

```
1 // Synchronisierung der gesamten Methode
2 public synchronized int incrVal() {
3     ...
4 }
```

```
1 // Synchronisierung eines Blocks (eines Teils einer Methode)
2 public int incrVal() {
3     ...
4     synchronized (someObj) {
5         ...
6     }
7     ...
8 }
```

Dabei wird implizit ein Lock über ein Objekt (das eigene Objekt im ersten Fall, das Sperrobjekt im zweiten Fall) benutzt.

Seit Java5 kann man alternativ auch explizite Lock-Objekte nutzen:

```
1 // Synchronisierung eines Teils einer Methode über ein
2 // Lock-Objekt (seit Java 5)
3 // Package `java.util.concurrent.locks`
```

```
4  public int incrVal() {
5      Lock waechter = new ReentrantLock();
6      ...
7      waechter.lock();
8      ... // Geschützter Bereich
9      waechter.unlock();
10     ...
11 }
```

Locks aus dem Paket `java.util.concurrent.locks` arbeiten analog zum impliziten Locken über **`synchronized`**. Sie haben darüber hinaus aber einige Vorteile:

- Methoden zum Abfragen, ob ein Lock möglich ist: `Lock#tryLock`
- Methoden zum Abfragen der aktuellen Warteschlangengröße: `Lock#getQueueLength`
- Verfeinerung `ReentrantReadWriteLock` mit Methoden `readLock` und `writeLock`
  - Locks nur zum Lesen bzw. nur zum Schreiben
- `Lock#newCondition` liefert ein Condition-Objekt zur Benachrichtigung ala `wait`/`notify`: `await`/`signal` => zusätzliches Timeout beim Warten möglich

Nachteile:

- Bei Exceptions werden implizite Locks durch **`synchronized`** automatisch durch das Verlassen der Methode freigegeben. Explizite Locks müssen **durch den Programmierer** freigegeben werden! => Nutzung des **`finally`**-Block!

Demo: lock.*

2.9.3.2 Thread-Management: Executor-Interface und Thread-Pools

2.9.3.2.1 Wiederverwendung von Threads

- Normale Threads sind immer Einmal-Threads: Man kann sie nur **einmal** in ihrem Leben starten (auch wenn das Objekt anschließend noch auf Nachrichten bzw. Methodenaufrufe reagiert)
- Zusätzliches Problem: Threads sind Objekte:
  - Threads brauchen relativ viel Arbeitsspeicher
  - Erzeugen und Entsorgen von Threads kostet Ressourcen
  - Zu viele Threads: Gesamte Anwendung hält an
- Idee: Threads wiederverwenden und Thread-Management auslagern => **Executor-Interface** und **Thread-Pool**

2.9.3.2.2 Executor-Interface

```
1 public interface Executor {
2     void execute(Runnable command);
3 }
```

- Neue Aufgaben als Runnable an einen Executor via `execute` übergeben

- Executor könnte damit sofort neuen Thread starten (oder alten wiederverwenden): `e.execute(r);` => entspricht in der Wirkung `(new Thread(r)).start();`

2.9.3.2.3 Thread-Pool hält Menge von "Worker-Threads"

- Statische Methoden von `java.util.concurrent.Executors` erzeugen Thread-Pools mit verschiedenen Eigenschaften:
  - `Executors#newFixedThreadPool` erzeugt ExecutorService mit spezifizierter Anzahl von Worker-Threads
  - `Executors#newCachedThreadPool` erzeugt Pool mit Threads, die nach 60 Sekunden Idle wieder entsorgt werden
- Rückgabe: `ExecutorService` (Thread-Pool)

```
1  public interface ExecutorService extends Executor { ... }
```

- `Executor#execute` übergibt Runnable dem nächsten freien Worker-Thread (oder erzeugt ggf. neuen Worker-Thread bzw. hängt Runnable in Warteschlange, je nach erzeugtem Pool)
- Methoden zum Beenden eines Thread-Pools (Freigabe): `shutdown()`, `isShutdown()`, ...

```
1  MyThread x = new MyThread();    // Runnable oder Thread
2
3  ExecutorService pool = Executors.newCachedThreadPool();
4
5  pool.execute(x);    // x.start()
6  pool.execute(x);    // x.start()
7  pool.execute(x);    // x.start()
8
9  pool.shutdown();    // Feierabend :)
```

Demo: executor.ExecutorDemo

2.9.3.2.4 Hintergrund (vereinfacht) Der Thread-Pool reserviert sich "nackten" Speicher, der der Größe von n Threads entspricht, und "prägt" die Objektstruktur durch einen Cast direkt auf (ohne wirkliche neue Objekte zu erzeugen). Dieses Vorgehen ist in der C-Welt wohlbekannt und schnell (vgl. Thema Speicherverwaltung in der LV "Systemprogrammierung"). In Java wird dies durch eine wohldefinierte Schnittstelle vor dem Nutzer verborgen.

2.9.3.2.5 Ausblick Hier haben wir nur die absoluten Grundlagen angerissen. Wir können auch `Callables` anstatt von `Runnables` übergeben, auf Ergebnisse aus der Zukunft warten (`Futures`), Dinge zeitgesteuert (immer wieder) starten, ...

Schauen Sie sich bei Interesse die weiterführende Literatur an, beispielsweise die Oracle-Dokumentation oder auch (Ullenboom 2021) (insbesondere den Abschnitt 16.4 "Der Ausführer (Executor) kommt").

2.9.3.3 Fork/Join-Framework: Teile und Herrsche Spezieller Thread-Pool zur rekursiven Bearbeitung parallelisierbarer Tasks

- `java.util.concurrent.ForkJoinPool#invoke` startet Task
- Task muss von `RecursiveTask<V>` erben:

```
public abstract class RecursiveTask<V> extends ForkJoinTask<V> {
    protected abstract V compute();
}
```

Prinzipieller Ablauf:

```
public class RecursiveTask extends ForkJoinTask<V> {
    protected V compute() {
        if (task klein genug) {
            berechne task sequentiell
        } else {
            teile task in zwei subtasks:
                left, right = new RecursiveTask(task)
            rufe compute() auf beiden subtasks auf:
                left.fork();          // starte neuen Thread
                r = right.compute();  // nutze aktuellen Thread
            warte auf ende der beiden subtasks: l = left.join()
            kombiniere die ergebnisse der beiden subtasks: l+r
        }
    }
}
```

Demo: forkjoin.ForkJoin

2.9.3.4 Swing und Threads

2.9.3.4.1 Lange Berechnungen in Listenern blockieren Swing-GUI

- Problem: Events werden durch **einen** *Event Dispatch Thread* (EDT) **sequentiell** bearbeitet
- Lösung: Berechnungen in neuen Thread auslagern
- **Achtung**: Swing ist **nicht Thread-safe**! Komponenten nicht durch verschiedene Threads manipulieren!

2.9.3.4.2 Lösung => `javax.swing.SwingWorker` ist eine spezielle Thread-Klasse, eng mit Swing/Event-Modell verzahnt.

- Implementieren:
  - `SwingWorker#doInBackground`: Für die langwierige Berechnung (muss man selbst implementieren)
  - `SwingWorker#done`: Wird vom EDT aufgerufen, wenn `doInBackground` fertig ist

- Aufrufen:

  - `SwingWorker#execute`: Started neuen Thread nach Anlegen einer Instanz und führt dann automatisch `doInBackground` aus
  - `SwingWorker#get`: Return-Wert von `doInBackground` abfragen

2.9.3.4.3 Anmerkungen

- `SwingWorker#done` ist optional: *kann* überschrieben werden
  - Beispielweise, wenn nach Beendigung der langwierigen Berechnung GUI-Bestandteile mit dem Ergebnis aktualisiert werden sollen
- `SwingWorker<T, V>` ist eine generische Klasse:
  - `T` Typ für das Ergebnis der Berechnung, d.h. Rückgabetyp für `doInBackground` und `get`
  - `V` Typ für Zwischenergebnisse

Demo: misc.SwingWorkerDemo

2.9.3.5 Letzte Worte :-)

- Viele weitere Konzepte
  - Semaphoren, Monitore, …
  - Leser-Schreiber-Probleme, Verklemmungen, …

  => Verweis auf LV “Betriebssysteme” und “Verteilte Systeme”

- **Achtung**: Viele Klassen sind nicht Thread-safe!

  Es gibt aber meist ein “Gegenstück”, welches Thread-safe ist.

  Beispiel Listen:

  - `java.util.ArrayList` ist **nicht** Thread-safe
  - `java.util.Vector` ist Thread-sicher

  => Siehe Javadoc in den JDK-Klassen!

- Thread-safe bedeutet **Overhead** (Synchronisierung)!

2.9.3.6 Wrap-Up Multi-Threading auf höherem Level: Thread-Pools und Fork/Join-Framework

- Feingranulareres und flexibleres Locking mit Lock-Objekten und Conditions
- Wiederverwendung von Threads: Thread-Management mit Executor-Interface und Thread-Pools
- Fork/Join-Framework zum rekursiven Zerteilen von Aufgaben und zur parallelen Bearbeitung der Teilaufgaben
- `SwingWorker` für die parallele Bearbeitung von Aufgaben in Swing

Zum Nachlesen

- Ullenboom (2021, Kap. 16)
- Oracle Corporation (2022c)
- Urma u. a. (2014, 7.2)

Lernziele

- k3: Umgang mit High-Level-Abstraktionen: Lock-Objekten und Conditions, Executor-Interface und Thread-Pools, Fork/Join-Framework, SwingWorker

Quizzes

- Quiz High-Level Concurrency (ILIAS)

3 Organisatorisches

3.1 Prüfungsvorbereitung

TL;DR

3.1.0.1 Durchführung: Präsenz oder Open-Book (je nach Corona-Lage) Die Klausur wird dieses Semester elektronisch stattfinden. Dazu werden wir den Prüfungs-ILIAS der HSBI nutzen.
Sofern die Situation dies zulässt, wird die Klausur in den Räumen der HSBI am Campus Minden unter Aufsicht durchgeführt. Hier werden Ihnen Rechner für den Zugang zum Prüfungs-ILIAS zur Verfügung gestellt, Sie benötigen nur Ihre HSBI-Zugangsdaten (User, Passwort), einen Studierendenausweis und Personalausweis sowie Ihren DIN-A4-Spickzettel.
Wenn die Corona-Lage eine Durchführung in Präsenz nicht erlaubt, wird die Klausur stattdessen als *Open-Book-Ausarbeitung* aus dem Home-Office durchgeführt. Sie benötigen dazu einen normalen Rechner oder Laptop mit einem Standardbrowser. Tablets und Handys können wg. der Mobil-Version der Browser problematisch sein. Sie müssen JavaScript aktivieren und Cookies zulassen, der Privacy-Modus ist bitte ebenfalls zu deaktivieren. Sie erreichen den Prüfungs-ILIAS eassessment.hsbi.de nur über VPN.
Die Entscheidung über die konkrete Durchführung wird spätestens zwei Wochen vor der Prüfung getroffen und Ihnen per EMail über das LSF mitgeteilt.

3.1.0.2 Ablauf der Klausur Die Prüfung (das ILIAS-Objekt) selbst schalte ich erst zum Start der Prüfung online. Bei der Durchführung als Open-Book-Ausarbeitung wird parallel zur Prüfung eine Zoom-Sitzung laufen, in der Sie Fragen stellen können.

3.1.0.3 Hilfsmittel und Themen Bei der Durchführung in Präsenz am Campus Minden ist ein Spickzettel (DIN A4, beidseitig beschrieben) als Hilfsmittel zugelassen.
Bei der Durchführung als “Open-Book-Ausarbeitung” im Home-Office sind alle Hilfsmittel zugelassen.
Die Unterstützung durch Dritte bzw. jegliche Kommunikation mit Dritten ist in keinem Fall zugelassen. Sie sollen die Prüfung selbstständig bearbeiten.
Es wird keines der behandelten Themen ausgeschlossen, allerdings eignen sich manche Themen etwas besser für Klausurfragen als andere ;-)

Videos

- Hinweise zur Prüfung: Fragetypen-Demo
- Hinweise zur Prüfung: Technische Vorbereitung

3.1.1 Elektronische Klausur: Termin, Materialien

3.1.1.1 Termin Die schriftliche Prüfung erfolgt durch eine Klausur, die als digitale Prüfung auf einem Prüfungs-ILIAS durchgeführt wird.

Es wird angestrebt, die Klausur in Präsenz in den Rechnerpools am Campus Minden durchzuführen. Falls dies wegen der Corona-Situation oder anderer Umstände nicht möglich sein sollte, wird die Klausur als “Open-Book-Ausarbeitung” im Home-Office durchgeführt.

Es wird in beiden Prüfungszeiträumen ein Termin angeboten. Die Termine werden vom Prüfungsamt bekannt gegeben.

Dauer jeweils 90 Minuten.

- Die konkrete Durchführungsform (in Präsenz am Campus Minden oder im Home-Office) wird Ihnen spätestens zwei Wochen vor der Prüfung über das LSF bekanntgegeben

3.1.1.2 Zugelassene Hilfsmittel **Präsenz (in Minden)**

Zugelassene Materialien: **DIN-A4-Spickzettel (beidseitig)**

Sie dürfen **einen** Spickzettel im **DIN-A4**-Format benutzen, der beidseitig beschrieben sein kann.

Ich möchte Sie hier noch einmal ermuntern, diesen Zettel tatsächlich manuell zu erstellen (also ganz traditionell zu **schreiben**), da bereits der Schreibvorgang einen gewissen Lerneffekt bewirkt!

Open-Book-Ausarbeitung (Homeoffice)

Falls die Prüfung als Open-Book-Ausarbeitung im Home-Office durchgeführt werden sollte, dürfen Sie alle Unterlagen benutzen.

- Ausnahme: **Keine Hilfe durch Dritte!** (insbesondere keine Zusammenarbeit, keine Kommunikation)

  Sie sollen die Prüfung eigenständig bearbeiten. Hilfe von Dritten sowie jegliche Kommunikation mit Dritten ist in keinem Fall zugelassen und wird als Täuschungsversuch gewertet.

3.1.1.3 Einsicht

- Prüfungseinsicht: Zeitnah; Bekanntgabe per Mail

3.1.2 Technische Vorbereitungen

Präsenz (in Minden)

Diese Bemerkungen betreffen die Durchführung als Präsenzprüfung in den Räumen am Campus Minden.

- **HSBI-Zugangsdaten**: Username, Passwort

  Bei der Durchführung der Prüfung am Campus Minden wird Ihnen ein Rechner zur Verfügung gestellt. Dort läuft voraussichtlich ein Browser im Kiosk-Mode, wo Sie sich am Prüfungs-ILIAS anmelden. Dazu benötigen Sie ihre HSBI-Zugangsdaten, mit denen Sie sich auch im “normalen” ILIAS anmelden.

- **Studierendenausweis** und Personalausweis

  An der Prüfung dürfen nur Personen teilnehmen, die dafür im LSF angemeldet sind. Es findet eine entsprechende Kontrolle statt. Halten Sie Ihren Studierendenausweis und Personalausweis bereit.

Open-Book-Ausarbeitung (Homeoffice)

Diese Bemerkungen betreffen die Durchführung aus dem Home-Office mit Ihrer Hardware. Bei der Durchführung in Präsenz in den Räumen am Campus Minden werden die technischen Details von uns für Sie vorbereitet sein.

- **Rechner**: Nutzen Sie für die Prüfung einen stationären Rechner oder ein Notebook.

  Vermeiden Sie die Verwendung von Tablets und Smartphones! Bei der Verwendung von Tablets kann es unter Umständen zu Darstellungsproblemen kommen. Smartphones sind aufgrund des kleinen Bildschirms für die Prüfungsdurchführung schlicht ungeeignet.

  Bei fehlendem Zugang zu einem entsprechenden Endgerät kontaktieren Sie bitte frühzeitig die Prüfungsverantwortlichen.

- **Netz**: Stabil genug? Belastbar genug?

  Wenn Sie keinen Zugang zu einer ausreichend stabilen Internetverbindung haben, setzen Sie sich frühzeitig mit Ihren Prüfungsverantwortlichen in Verbindung.

- **VPN**: Der Prüfungs-ILIAS ist nur im HSBI-VPN erreichbar.

  Installieren Sie den VPN-Client (Anleitung: hsbi.de/dvz/faq/cat/7) und testen Sie im Vorfeld der Prüfung bei aktivierter VPN-Verbindung den Zugang zur Prüfungsplattform eassessment.hsbi.de. Zugangsdaten wie im normalen ILIAS.

  Achtung: Auch wenn Sie sich in den Räumen der HSBI befinden, müssen Sie oft die VPN-Verbindung aktivieren, um Zugang zur Prüfungsplattform zu erhalten.

- **Browser**: Nutzen Sie einen der Standardbrowser (Edge, Firefox, Safari, Chrome/Chromium) in der Standardeinstellung: insbesondere JavaScript und Cookies müssen aktiviert/erlaubt sein.

  Deaktivieren Sie sämtliche Browser-Erweiterungen wie z.B. Ad-Blocker (AdBlockPlus, uBlock, …) oder JavaScript-Blocker (No-Script, Ghostery, …) für den Prüfungszeitraum.

  Nutzen Sie Ihren Browser nicht im Privacy-Modus!

- **HSBI-Zugangsdaten**: Username, Passwort

  Bei der Durchführung der Prüfung als Open-Book-Ausarbeitung führen Sie die Prüfung auf Ihrer eigenen Hardware im Home-Office durch. Auch hier müssen Sie sich am Prüfungs-ILIAS anmelden. Dazu benötigen Sie ihre HSBI-Zugangsdaten, mit denen Sie sich auch im “normalen” ILIAS anmelden.

3.1.3 Bearbeitung des E-Assessment

1. Lesen Sie sich die Hinweise auf der Startseite durch
2. Bearbeiten Sie die Aufgaben in **einem einzigen** Browser-Tab

   Öffnen Sie die Aufgaben *NICHT* in parallelen Tabs! Es kann sonst zu Fehlfunktionen von ILIAS kommen.

   Bewegen Sie sich nicht per Browser-Navigation (“vor”, “zurück” im Browser) durch die Aufgaben, sondern nutzen Sie dafür die Buttons “nächste Frage”, “Weiter” oder “Zurück” vom ILIAS!

3. Hinweis zu Anzeige der restlichen Bearbeitungsdauer

   Wenn Sie den Browser bzw. das Tab mit der Prüfung im Laufe der Prüfung verlassen, wird Ihnen bei der Rückkehr unter Umständen eine falsche restliche Bearbeitungsdauer angezeigt. Sie können die Anzeige korrigieren/aktualisieren, indem Sie einfach zu einer vorigen oder nächsten Aufgabe navigieren.

Hinweis: Die restliche Bearbeitungsdauer wird im Test nur dann angezeigt, wenn diese Funktion von den Prüfenden aktiviert wurde.

4. Parallel zum E-Assessment läuft eine Zoom-Session, dort können Sie Fragen stellen
5. Verbindungsprobleme (Home-Office):
   - Bei kurzzeitigen Verbindungsabbrüchen loggen Sie sich einfach wieder ein
   - Wenn die Probleme länger dauern, gilt der Versuch als nicht unternommen (außer Sie haben die Probleme aktiv herbeigeführt, dann kann das als Täuschungsversuch gewertet werden, vgl. RPO §22a (4))

3.1.4 Fragetypen-Demo

In Ihrem ILIAS-Kurs finden Sie eine **Fragetypen-Demo** mit den wichtigsten Fragetypen. Machen Sie sich mit der Mechanik der Fragetypen vertraut und schauen Sie sich die Kommentare bei den einzelnen Aufgaben an. Sie können die Demo bei Bedarf beliebig oft wiederholen.

3.1.5 Hinweise zu den Inhalten

- Klausurrelevant: Vorlesung und Praktikum
- Für Verständnis u.U. hilfreich: Studium der vertiefenden Literaturangaben

- **Fragen**:
  - Schauen Sie sich die Challenges und/oder Quizzes an …
  - Schauen Sie sich die Praktikumsaufgaben an …
  - Überlegen Sie sich, was zu einem Themengebiet im Rahmen einer Prüfung möglich ist und (wie) gefragt werden könnte :)

Können vor Kennen :-)

3.1.6 Beispiele für mögliche Fragen

3.1.6.1 Vererbung und Polymorphie

Betrachten Sie den folgenden Java-Code:

```
1  public class Person {
2      public String getInfo(Person p) { return "Person"; }
3  }
4
5  public class Studi extends Person {
6      public String getInfo(Studi s) { return "Studi"; }
7
8      public static void main(String[] args) {
9          Studi s = new Studi(); Person p = s;
10         System.out.println(s.getInfo(p));
11         System.out.println(s.getInfo(s));
12     }
13 }
```

Geben Sie alle Ausgaben, die das obige Programm produziert, an.

Begründen Sie Ihre Antwort kurz und stichhaltig (für *jede* Ausgabe!). Was geschieht, bzw. wieso kommt es zu der jeweiligen Ausgabe?

3.1.6.2 Multithreading und Synchronisierung

```
1  public class StaffelKaputt extends Thread {
2      private Object stab;
3      StaffelKaputt(Object stab) { this.stab = stab; }
4      public void run() {nimmStab(); laufen(); stabAbgeben();}
5      private void stabAbgeben() {
6          synchronized (stab) { stab.notifyAll(); }
7      }
8      private void nimmStab() {
9          synchronized (stab) {
10             try { stab.wait(); } catch (Exception e) { }
11         }}
12     void laufen() { System.out.println("laufe ... "); }
13     public static void main(String[] args) {
14         Object stab = new Object();
15         StaffelKaputt l1 = new StaffelKaputt(stab);
16         StaffelKaputt l2 = new StaffelKaputt(stab);
17         l1.start(); l2.start();
18     }}
```

Das Programm enthält einen Fehler, der sich zur Laufzeit offenbart. Welche Ausgabe erwarten Sie (angenommen, das Programm wäre fehlerfrei; eine mögliche Variante reicht)? Welche Ausgabe erhalten Sie stattdessen? Korrigieren Sie den Fehler.

3.1.6.3 Reguläre Ausdrücke

Auf welche Strings passt (im Sinne von "match") der folgende reguläre Ausdruck:
`\s*([a-zA-Z0-9_.\-]+)\s*=\s*(-?\d+\.?\d*)\s;?\s*`

3.1.6.4 Versionieren mit Git

- Erklären Sie, wie man mit Git die Unterschiede zwischen zwei bestimmten Versionsständen einer Datei herausfindet.
- Was ist der Unterschied zwischen einer Workingcopy und einem Repository?
- Worin liegt der Unterschied zwischen folgenden Arbeitsschritten:

  a. Editieren von Datei `A.txt`
  b. `git add A.txt`
  c. Editieren von Datei `A.txt`
  d. `git commit`

  versus

  a. Editieren von Datei `A.txt`
  b. Editieren von Datei `A.txt`
  c. `git add A.txt`

d. `git commit`

- Was würde `git diff` jeweils nach Schritt 2 anzeigen?

3.1.6.5 Kommandozeilenparameter Schreiben Sie ein Programm, welches auf zwei Kommandozeilenparameter reagieren kann. Die erkannten Parameter sollen auf der Konsole ausgegeben werden. Nutzen Sie Apache Commons CLI (API siehe Anhang).

- Beim Aufruf ohne Parameter soll eine Hilfe zum korrekten Aufruf ausgegeben werden und das Programm soll sich anschließend beenden.
- Das Programm soll den Parameter `-debug` erkennen.
- Das Programm soll den Parameter `-x=10` erkennen, wobei der Wert beim Aufruf variieren kann (Integer).
- Die Parameter können in unterschiedlicher Reihenfolge auftreten.
- Es kann auch nur ein Parameter angegeben werden.

3.1.6.6 Build mit Ant

- Was ist der Unterschied zwischen Ant-Targets und Ant-Tasks?
- Wie kann man Ant-Properties von außen (beim Aufruf) setzen?
- Schreiben Sie ein Ant-Target, welches alle `.class`-Dateien in einem Ordner umbenennt.
- Schreiben Sie ein Ant-Target, mit dem Sie die Javadoc-Dokumentation erzeugen, packen und das resultierende `.zip`-File in den Ordner `dist/` verschieben.
- Schreiben Sie Ant-Targets, mit denen Sie JUnit-Testfälle ausführen und auswerten können.

3.1.6.7 Generics Was kommt hier raus? Und warum?

```java
public class X {
    void methode(int a) {
        System.out.println("non-generic");
    }
    <T> void methode(T a) {
        System.out.println("generisch");
    }
    public static void main(String[] args) {
        X x = new X();
        x.methode(3);
        x.methode(new Integer(4));
        x.methode("huhu");
    }
}
```

3.1.6.8 Logging Erklären Sie den Code. Was passiert?

```java
class MyFormatter extends SimpleFormatter {
    public String format(LogRecord record) {
        return super.format(record) + "---- FAKE ----\n";
    }
}
```

```java
public class MoreLogging {
    public static void main(String[] argv) {
        Logger l = Logger.getLogger("MoreLogging");
        l.setLevel(Level.FINE);

        ConsoleHandler myHandler = new ConsoleHandler();
        myHandler.setFormatter(new MyFormatter());
        myHandler.setLevel(Level.FINER);
        l.addHandler(myHandler);

        l.info("Hello World :-)");
        l.fine("fine");
        l.finer("finer");
        l.finest("finest");
    }
}
```

3.1.6.9 Methodenreferenzen

- Was bedeutet der folgende Code?

```java
List<String> str = Arrays.asList("a", "b", "A", "B");
str.sort(String::compareToIgnoreCase);
```

4 Praktikum

Hier finden Sie die Übungsblätter.

4.1 Zyklus 1: Konzept

4.1.1 Zyklus 1 (erste Woche): Konzeptphase

Suchen Sie sich gemeinsam im Team die **Aufgaben** aus, die Sie in diesem Zyklus bearbeiten wollen, und erstellen Sie dafür ein **Lösungskonzept**.[2]

Sie können in diesem Zyklus Aufgaben im Umfang von 15 Punkten abgeben/vorstellen.

Die Konzeptskizze laden Sie bitte bis zur Deadline (s.o.) als **PDF** in die jeweilige ILIAS-Aufgabe hoch. **Wichtig**: Diese Abgabe muss aus technischen Gründen bitte **jedes Team-Mitglied einzeln** machen.[3]

Anschließend erfolgt die Peer-Feedback-Phase. ILIAS weist Ihnen mehrere Abgaben zu und Sie haben einen Tag Zeit, um Ihr Feedback im ILIAS abzugeben. Das Feedback muss **einzeln von jedem Team-Mitglied** bearbeitet werden.[4]

Im Praktikum am Freitag in dieser Zyklus-Woche stellen Sie im Team Ihr Konzept und das erhaltene Peer-Feedback vor. Dabei ist jedes Team-Mitglied anwesend und kann Auskunft geben.[5]

[2] vgl. Konzeptskizze in der FAQ
[3] vgl. Abgabe der Lösungen in der FAQ
[4] vgl. Peer-Feedback: How to Review in der FAQ
[5] vgl. Vorstellung der Lösungen in der FAQ

4.2 Zyklus 1: Implementierung

4.2.1 Zyklus 1 (zweite Woche): Implementierungsphase

Setzen Sie gemeinsam im Team Ihr Konzept aus der ersten Zyklushälfte in eine **Implementierung** um.

Den Quellcode und die sonstigen bei der Bearbeitung erstellten Artefakte (Texte, Texturen, …, aber bitte keine `.class`-Dateien) laden Sie bitte bis zur Deadline (s.o.) als **ZIP** in die jeweilige ILIAS-Aufgabe hoch. **Wichtig**: Diese Abgabe muss aus technischen Gründen bitte **jedes Team-Mitglied einzeln** machen.[6]

Anschließend erfolgt eine erneute Peer-Feedback-Phase. ILIAS weist Ihnen wieder mehrere Abgaben zu und Sie haben einen Tag Zeit, um Ihr Feedback im ILIAS abzugeben. Das Feedback muss wieder **einzeln von jedem Team-Mitglied** bearbeitet werden.[7]

Im Praktikum am Freitag in dieser Zyklus-Woche stellen Sie im Team Ihre Implementierung und das erhaltene Peer-Feedback vor. Dabei ist jedes Team-Mitglied anwesend und kann Auskunft geben.[8]

4.3 Zyklus 2: Konzept

4.3.1 Zyklus 2 (erste Woche): Konzeptphase

Suchen Sie sich gemeinsam im Team die **Aufgaben** aus, die Sie in diesem Zyklus bearbeiten wollen, und erstellen Sie dafür ein **Lösungskonzept**.[9]

Sie können in diesem Zyklus Aufgaben im Umfang von 15 Punkten abgeben/vorstellen.

Die Konzeptskizze laden Sie bitte bis zur Deadline (s.o.) als **PDF** in die jeweilige ILIAS-Aufgabe hoch. **Wichtig**: Diese Abgabe muss aus technischen Gründen bitte **jedes Team-Mitglied einzeln** machen.[10]

Anschließend erfolgt die Peer-Feedback-Phase. ILIAS weist Ihnen mehrere Abgaben zu und Sie haben einen Tag Zeit, um Ihr Feedback im ILIAS abzugeben. Das Feedback muss **einzeln von jedem Team-Mitglied** bearbeitet werden.[11]

Im Praktikum am Freitag in dieser Zyklus-Woche stellen Sie im Team Ihr Konzept und das erhaltene Peer-Feedback vor. Dabei ist jedes Team-Mitglied anwesend und kann Auskunft geben.[12]

4.4 Zyklus 2: Implementierung

4.4.1 Zyklus 2 (zweite Woche): Implementierungsphase

Setzen Sie gemeinsam im Team Ihr Konzept aus der ersten Zyklushälfte in eine **Implementierung** um.

[6] vgl. Abgabe der Lösungen in der FAQ
[7] vgl. Peer-Feedback: How to Review in der FAQ
[8] vgl. Vorstellung der Lösungen in der FAQ
[9] vgl. Konzeptskizze in der FAQ
[10] vgl. Abgabe der Lösungen in der FAQ
[11] vgl. Peer-Feedback: How to Review in der FAQ
[12] vgl. Vorstellung der Lösungen in der FAQ

Nutzen Sie bei der Umsetzung aktiv Git und einen Git-basierten Workflow für die Zusammenarbeit im Team.

Für die Abgabe erstellen Sie einen *öffentlich zugreifbaren* Pull- oder Merge-Request innerhalb Ihres Repos auf Ihrem Git-Server und geben die **URL zu diesem PR/MR** im ILIAS als Link ab. Achten Sie bitte darauf, dass der PR/MR nur die für dieses Blatt abzugebenden Dateien enthält und das der PR/MR bis zur Vorstellung im Praktikum **offen** bleibt (nicht vorher gemergt wird).[13] **Wichtig**: Diese Abgabe muss aus technischen Gründen bitte **jedes Team-Mitglied einzeln** machen.[14]

Anschließend erfolgt eine erneute Peer-Feedback-Phase. ILIAS weist Ihnen wieder mehrere Abgaben zu und Sie haben einen Tag Zeit, um Ihr Feedback im ILIAS abzugeben. Das Feedback muss wieder **einzeln von jedem Team-Mitglied** bearbeitet werden.[15]

Im Praktikum am Freitag in dieser Zyklus-Woche stellen Sie im Team Ihre Implementierung und das erhaltene Peer-Feedback vor. Dabei ist jedes Team-Mitglied anwesend und kann Auskunft geben.[16]

4.5 Zyklus 3: Konzept

4.5.1 Zyklus 3 (erste Woche): Konzeptphase

Suchen Sie sich gemeinsam im Team die **Aufgaben** aus, die Sie in diesem Zyklus bearbeiten wollen, und erstellen Sie dafür ein **Lösungskonzept**.[17]

Sie können in diesem Zyklus Aufgaben im Umfang von 15 Punkten abgeben/vorstellen.

Die Konzeptskizze laden Sie bitte bis zur Deadline (s.o.) als **PDF** in die jeweilige ILIAS-Aufgabe hoch. **Wichtig**: Diese Abgabe muss aus technischen Gründen bitte **jedes Team-Mitglied einzeln** machen.[18]

Anschließend erfolgt die Peer-Feedback-Phase. ILIAS weist Ihnen mehrere Abgaben zu und Sie haben einen Tag Zeit, um Ihr Feedback im ILIAS abzugeben. Das Feedback muss **einzeln von jedem Team-Mitglied** bearbeitet werden.[19]

Im Praktikum am Freitag in dieser Zyklus-Woche stellen Sie im Team Ihr Konzept und das erhaltene Peer-Feedback vor. Dabei ist jedes Team-Mitglied anwesend und kann Auskunft geben.[20]

4.6 Zyklus 3: Implementierung

4.6.1 Zyklus 3 (zweite Woche): Implementierungsphase

Setzen Sie gemeinsam im Team Ihr Konzept aus der ersten Zyklushälfte in eine **Implementierung** um.

Sie können in diesem Zyklus Aufgaben im Umfang von 15 Punkten abgeben/vorstellen. Zusätzlich bekommen Sie *bis* zu 5 Punkte für die Einhaltung einfacher Coding-Rules:

[13] vgl. Link zum Pull-/Merge-Request in der FAQ
[14] vgl. Abgabe der Lösungen in der FAQ
[15] vgl. Peer-Feedback: How to Review in der FAQ
[16] vgl. Vorstellung der Lösungen in der FAQ
[17] vgl. Konzeptskizze in der FAQ
[18] vgl. Abgabe der Lösungen in der FAQ
[19] vgl. Peer-Feedback: How to Review in der FAQ
[20] vgl. Vorstellung der Lösungen in der FAQ

- Einhaltung des PM-Coding-Styles: +2P
- Durchgängige Dokumentation mit Javadoc: +2P
- Durchgängiger Einsatz von Logging: +1P

Nutzen Sie bei der Umsetzung aktiv Git und einen Git-basierten Workflow für die Zusammenarbeit im Team.

Für die Abgabe erstellen Sie einen *öffentlich zugreifbaren* Pull- oder Merge-Request innerhalb Ihres Repos auf Ihrem Git-Server und geben die **URL zu diesem PR/MR** im ILIAS als Link ab. Achten Sie bitte darauf, dass der PR/MR nur die für dieses Blatt abzugebenden Dateien enthält und das der PR/MR bis zur Vorstellung im Praktikum **offen** bleibt (nicht vorher gemergt wird).[21] **Wichtig**: Diese Abgabe muss aus technischen Gründen bitte **jedes Team-Mitglied einzeln** machen.[22]

Anschließend erfolgt eine erneute Peer-Feedback-Phase. ILIAS weist Ihnen wieder mehrere Abgaben zu und Sie haben einen Tag Zeit, um Ihr Feedback im ILIAS abzugeben. Das Feedback muss wieder **einzeln von jedem Team-Mitglied** bearbeitet werden.[23]

Im Praktikum am Freitag in dieser Zyklus-Woche stellen Sie im Team Ihre Implementierung und das erhaltene Peer-Feedback vor. Dabei ist jedes Team-Mitglied anwesend und kann Auskunft geben.[24]

4.7 Zyklus 4: Konzept

4.7.1 Zyklus 4 (erste Woche): Konzeptphase

Suchen Sie sich gemeinsam im Team die **Aufgaben** aus, die Sie in diesem Zyklus bearbeiten wollen, und erstellen Sie dafür ein **Lösungskonzept**.[25]

Sie können in diesem Zyklus Aufgaben im Umfang von 15 Punkten abgeben/vorstellen.

Die Konzeptskizze laden Sie bitte bis zur Deadline (s.o.) als **PDF** in die jeweilige ILIAS-Aufgabe hoch. **Wichtig**: Diese Abgabe muss aus technischen Gründen bitte **jedes Team-Mitglied einzeln** machen.[26]

Anschließend erfolgt die Peer-Feedback-Phase. ILIAS weist Ihnen mehrere Abgaben zu und Sie haben einen Tag Zeit, um Ihr Feedback im ILIAS abzugeben. Das Feedback muss **einzeln von jedem Team-Mitglied** bearbeitet werden.[27]

Im Praktikum am Freitag in dieser Zyklus-Woche stellen Sie im Team Ihr Konzept und das erhaltene Peer-Feedback vor. Dabei ist jedes Team-Mitglied anwesend und kann Auskunft geben.[28]

[21] vgl. Link zum Pull-/Merge-Request in der FAQ
[22] vgl. Abgabe der Lösungen in der FAQ
[23] vgl. Peer-Feedback: How to Review in der FAQ
[24] vgl. Vorstellung der Lösungen in der FAQ
[25] vgl. Konzeptskizze in der FAQ
[26] vgl. Abgabe der Lösungen in der FAQ
[27] vgl. Peer-Feedback: How to Review in der FAQ
[28] vgl. Vorstellung der Lösungen in der FAQ

4.8 Zyklus 4: Implementierung

4.8.1 Zyklus 4 (zweite Woche): Implementierungsphase

Setzen Sie gemeinsam im Team Ihr Konzept aus der ersten Zyklushälfte in eine **Implementierung** um.

Sie können in diesem Zyklus Aufgaben im Umfang von 15 Punkten abgeben/vorstellen. Zusätzlich bekommen Sie *bis* zu 5 Punkte für die Einhaltung einfacher Coding-Rules:

- Einhaltung des PM-Coding-Styles: +2P
- Durchgängige Dokumentation mit Javadoc: +2P
- Durchgängiger Einsatz von Logging: +1P

Nutzen Sie bei der Umsetzung aktiv Git und einen Git-basierten Workflow für die Zusammenarbeit im Team.

Für die Abgabe erstellen Sie einen *öffentlich zugreifbaren* Pull- oder Merge-Request innerhalb Ihres Repos auf Ihrem Git-Server und geben die **URL zu diesem PR/MR** im ILIAS als Link ab. Achten Sie bitte darauf, dass der PR/MR nur die für dieses Blatt abzugebenden Dateien enthält und das der PR/MR bis zur Vorstellung im Praktikum **offen** bleibt (nicht vorher gemergt wird).[29] **Wichtig**: Diese Abgabe muss aus technischen Gründen bitte **jedes Team-Mitglied einzeln** machen.[30]

Anschließend erfolgt eine erneute Peer-Feedback-Phase. ILIAS weist Ihnen wieder mehrere Abgaben zu und Sie haben einen Tag Zeit, um Ihr Feedback im ILIAS abzugeben. Das Feedback muss wieder **einzeln von jedem Team-Mitglied** bearbeitet werden.[31]

Im Praktikum am Freitag in dieser Zyklus-Woche stellen Sie im Team Ihre Implementierung und das erhaltene Peer-Feedback vor. Dabei ist jedes Team-Mitglied anwesend und kann Auskunft geben.[32]

4.9 Zyklus 5: Konzept

4.9.1 Zyklus 5 (erste Woche): Konzeptphase

Suchen Sie sich gemeinsam im Team die **Aufgaben** aus, die Sie in diesem Zyklus bearbeiten wollen, und erstellen Sie dafür ein **Lösungskonzept**.[33]

Sie können in diesem Zyklus Aufgaben im Umfang von 15 Punkten abgeben/vorstellen.

Die Konzeptskizze laden Sie bitte bis zur Deadline (s.o.) als **PDF** in die jeweilige ILIAS-Aufgabe hoch. **Wichtig**: Diese Abgabe muss aus technischen Gründen bitte **jedes Team-Mitglied einzeln** machen.[34]

Anschließend erfolgt die Peer-Feedback-Phase. ILIAS weist Ihnen mehrere Abgaben zu und Sie haben einen Tag Zeit, um Ihr Feedback im ILIAS abzugeben. Das Feedback muss **einzeln von jedem Team-Mitglied** bearbeitet werden.[35]

[29] vgl. Link zum Pull-/Merge-Request in der FAQ
[30] vgl. Abgabe der Lösungen in der FAQ
[31] vgl. Peer-Feedback: How to Review in der FAQ
[32] vgl. Vorstellung der Lösungen in der FAQ
[33] vgl. Konzeptskizze in der FAQ
[34] vgl. Abgabe der Lösungen in der FAQ
[35] vgl. Peer-Feedback: How to Review in der FAQ

Im Praktikum am Freitag in dieser Zyklus-Woche stellen Sie im Team Ihr Konzept und das erhaltene Peer-Feedback vor. Dabei ist jedes Team-Mitglied anwesend und kann Auskunft geben.[36]

4.10 Zyklus 5: Implementierung

4.10.1 Zyklus 5 (zweite Woche): Implementierungsphase

Setzen Sie gemeinsam im Team Ihr Konzept aus der ersten Zyklushälfte in eine **Implementierung** um.

Sie können in diesem Zyklus Aufgaben im Umfang von 15 Punkten abgeben/vorstellen. Zusätzlich bekommen Sie *bis* zu 5 Punkte für die Einhaltung einfacher Coding-Rules:

- Einhaltung des PM-Coding-Styles: +2P
- Durchgängige Dokumentation mit Javadoc: +2P
- Durchgängiger Einsatz von Logging: +1P

Nutzen Sie bei der Umsetzung aktiv Git und einen Git-basierten Workflow für die Zusammenarbeit im Team.

Für die Abgabe erstellen Sie einen *öffentlich zugreifbaren* Pull- oder Merge-Request innerhalb Ihres Repos auf Ihrem Git-Server und geben die **URL zu diesem PR/MR** im ILIAS als Link ab. Achten Sie bitte darauf, dass der PR/MR nur die für dieses Blatt abzugebenden Dateien enthält und das der PR/MR bis zur Vorstellung im Praktikum **offen** bleibt (nicht vorher gemergt wird).[37] **Wichtig**: Diese Abgabe muss aus technischen Gründen bitte **jedes Team-Mitglied einzeln** machen.[38]

Anschließend erfolgt eine erneute Peer-Feedback-Phase. ILIAS weist Ihnen wieder mehrere Abgaben zu und Sie haben einen Tag Zeit, um Ihr Feedback im ILIAS abzugeben. Das Feedback muss wieder **einzeln von jedem Team-Mitglied** bearbeitet werden.[39]

Im Praktikum am Freitag in dieser Zyklus-Woche stellen Sie im Team Ihre Implementierung und das erhaltene Peer-Feedback vor. Dabei ist jedes Team-Mitglied anwesend und kann Auskunft geben.[40]

4.11 Sonderabgabe

4.11.1 Sondertermin (bei sonst 'nicht bestanden' oder Krankheit)

Dies ist ein Zusatzblatt für Studierende, die bis dahin unterhalb der Bestehensschwelle für die praktische Teilleistung liegen oder die wegen Krankheit einen Termin nicht wahrnehmen konnten.

Für diese Abgabe gibt es keine Konzeptphase und auch kein Peer-Feedback.

Suchen Sie sich die **Aufgaben** aus, die Sie für diese Abgabe bearbeiten wollen, und **implementieren** dafür Sie Ihre **Lösung**.

Sie können für diese Abgabe Aufgaben im Umfang von 15 Punkten abgeben/vorstellen. Zusätzlich bekommen Sie *bis* zu 5 Punkte für die Einhaltung einfacher Coding-Rules:

[36] vgl. Vorstellung der Lösungen in der FAQ
[37] vgl. Link zum Pull-/Merge-Request in der FAQ
[38] vgl. Abgabe der Lösungen in der FAQ
[39] vgl. Peer-Feedback: How to Review in der FAQ
[40] vgl. Vorstellung der Lösungen in der FAQ

- Einhaltung des PM-Coding-Styles: +2P
- Durchgängige Dokumentation mit Javadoc: +2P
- Durchgängiger Einsatz von Logging: +1P

Nutzen Sie bei der Umsetzung aktiv Git und einen Git-basierten Workflow für die Zusammenarbeit im Team.

Für die Abgabe erstellen Sie einen *öffentlich zugreifbaren* Pull- oder Merge-Request innerhalb Ihres Repos auf Ihrem Git-Server und geben die **URL zu diesem PR/MR** im ILIAS als Link ab. Achten Sie bitte darauf, dass der PR/MR nur die für dieses Blatt abzugebenden Dateien enthält und das der PR/MR bis zur Vorstellung im Praktikum **offen** bleibt (nicht vorher gemergt wird).[41] **Wichtig**: Diese Abgabe muss aus technischen Gründen bitte **jedes Team-Mitglied einzeln** machen.[42]

Im Praktikum am Freitag in dieser Zyklus-Woche stellen Sie Ihre Implementierung vor.[43]

4.12 Praktikum

Hier finden Sie die Praktikumsaufgaben. Die Aufgaben sind in vier Themenbereiche sortiert: “Basics”, “Monster”, “Loot” und “Bounty”.

Anregungen für **Spielideen** können Sie beispielsweise in den folgenden Videos finden:

- Shattered Pixel Dungeon Rogue Beginners Guide Playthrough
- Shattered Pixel Dungeon Duelist Update!

4.12.1 Gruppe “Basics”

Aufgabe	Punkte	Abhängig von
Fallen	5	
Fähigkeiten	10	
Charakterklassen	5	“Nahkampf” oder “Fernkampf” und “Fähigkeiten”
Speichern und Laden	5	
JUnit (Gruppe Basics)	5	“Fallen”
Freie Aufgabe	10	

4.12.2 Gruppe “Monster”

[41] vgl. Link zum Pull-/Merge-Request in der FAQ
[42] vgl. Abgabe der Lösungen in der FAQ
[43] vgl. Vorstellung der Lösungen in der FAQ

Aufgabe	Punkte	Abhängig von
Monster	5	
Fernkampf	5	“Monster”
Nahkampf	5	“Monster”
Boss-Monster	5	“Monster”
Game-Over	5	
JUnit (Gruppe Monster)	5	“Monster” oder “Fernkampf”
Freie Aufgabe)	10	

4.12.3 Gruppe “Loot”

Aufgabe	Punkte	Abhängig von
Items	5	
Freundlicher NPC-Geist	5	
Dialogsystem	5	
Shop	10	“Item” und “Dialogsystem”
Monster-Schatzkisten	5	“Monster” und “Item”
Quests	5	
Lockpicking	10	“Item”
JUnit (Gruppe Loot)	5	“Quests” oder “Dialogsystem”
Freie Aufgabe	10	

4.12.4 Gruppe “Bounty”

Aufgabe	Punkte
5P Bounty	5
10P Bounty	10

Hinweis: Die Liste der Bounty-Aufgaben wird sich dynamisch verändern. Es wird vorkommen, dass Aufgaben im Laufe des Semesters nicht mehr zur Verfügung stehen. Ebenso können im Laufe des Semesters neue Aufgaben hinzu kommen. Nicht mehr angebotene Aufgaben können Sie nicht mehr zur Bearbeitung wählen.

Hinweis: Für die Bounty-Aufgaben können Sie zusätzlich bis zu 3 Bonus-Punkte bekommen, wenn Sie Ihre Lösung als Pull-Request im Dungeon-Repo einreichen und Ihr PR vom Dungeon-Team akzeptiert (gemergt) wird. Es kann nur ein PR pro Bounty-Aufgabe gemergt werden, die Entscheidung liegt beim Dozenten.

4.13 Praktikumsaufgaben Gruppe Basics

Hier finden Sie die Praktikumsaufgaben aus der Gruppe "Basics". Sie benötigen diese Aufgaben, um einen rudimentären spielfähigen Dungeon zu erhalten.

4.13.1 Fallen

4.13.1.1 Ziel In dieser Aufgaben sollen Sie Fallen implementieren, die das Voranschreiten im Dungeon schwieriger machen.

4.13.1.2 Fallen Implementieren Sie Fallen, die Sie im Dungeon verteilen.

Eine Falle kann aktiviert werden, indem ein Held oder ein Monster darauf tritt oder ein Gegenstand darauf abgelegt wird.

Fallen können verschiedene Wirkungen haben: Sie können bei der Aktivierung Schadenspunkte verteilen oder einen Gegenstand/Held/Monster im aktuellen Spiel-Level teleportieren oder zufällige Monster "hervorzaubern".

Fallen können gut sichtbar sein oder auch versteckt sein und nur unter der Wirkung eines bestimmten Zaubertranks oder Zauberspruchs sichtbar sein.

Fallen können einmal aktiviert werden oder auch mehrfach wirken.

Zu einigen Fallen kann es Schalter geben, welche die Fallen deaktivieren.

Fallen und Schalter werden zu Beginn eines Level in diesem verteilt.

4.13.2 Fähigkeiten

4.13.2.1 Ziel In dieser Aufgabe sollen Sie den Levelaufstieg[44] und verschiedene Fähigkeiten implementieren, die dann vom Spieler verwendet werden können.

4.13.2.2 Fähigkeiten In den Vorgaben ist bereits ein Projectile-System und ein Feuerball-Skill implementiert. Führen Sie eine Codeanalyse durch und erklären Sie die Funktionalität.

Implementieren Sie ein Magie-Konzept. Der Spieler soll in der Lage sein, verschiedene Zauber zu verwenden.

Beachten Sie dabei, dass es sich nicht um Zauber handeln soll, die Schaden verursachen. Zauber, die Monster anderweitig manipulieren, sind erlaubt.

[44] "Levelaufstieg" bezieht sich nicht auf die Dungeon-Ebenen ("Dungeon-Level"), sondern auf die Erfahrungen und die (Level-) Einstufung des Spielers.

Überlegen Sie sich zwei verschiedene Zaubersprüche wie Gedankenkontrolle oder Telekinese oder Upgrade von Waffen/Rüstungen.

Eine der Fähigkeiten soll eine Form von Ressourcenkosten haben. Sie verbraucht also Lebenspunkte, Ausdauerpunkte, Mana-Punkte oder ähnliches.

4.13.2.3 Levelaufstieg In den Vorgaben gibt es bereits ein XP-System um Erfahrungspunkte zu sammeln und um Level aufzusteigen. Führen Sie eine Codeanalyse durch und erklären Sie die Funktionalität.

Verbessern Sie beim Levelaufstieg des Helden seine Charakterwerte und lassen Sie ihn die oben implementierten Zaubersprüche erlernen.

4.13.3 Charakterklassen

4.13.3.1 Ziel In Rollenspielen gibt die Charakterklasse an, welche Werte (Anzahl der Lebenspunkte, Kampfschaden, Mana-Punkte etc.) die Spielfigur hat, welche Waffen sie verwenden und welche Fähigkeiten sie erlernen kann. Typische Charakterklassen sind Krieger, Zauberer oder Schurke.

In dieser Aufgabe sollen Sie verschiedene Charakterklassen implementieren.

4.13.3.2 Voraussetzungen Um diese Aufgabe zu lösen, sollten Sie am besten bereits ein Kampfsystem (“Nahkampf” oder “Fernkampf” und “Fähigkeiten”) umgesetzt haben.

4.13.3.3 Charakterklassen Überlegen Sie sich zuerst, welche Charakterklassen Sie implementieren wollen. Sie sollen mindestens drei unterschiedliche Charakterklassen anbieten.

Überlegen UND notieren Sie dabei, was diese Charakterklassen gemeinsam haben und worin sie sich unterscheiden. Geben Sie jeder Charakterklasse eine passende Bezeichnung.

In den Vorgaben existiert bereits ein Stats-Component. Führen Sie eine Codeanalyse durch und erklären Sie die Funktionalität des Components.

Implementieren Sie nun die verschiedenen Charakterklassen im Sinne des ECS-Gedankens. Überlegen Sie auch, welche Pattern Sie ggf. verwenden können. Begründen Sie Ihre Entscheidungen schriftlich.

Zu Beginn des Spiels soll der Spieler die Klasse auswählen können (oder haben Sie eine bessere Idee?).

4.13.4 Speichern und Laden

4.13.4.1 Ziel In dieser Aufgaben sollen Sie ein Konzept zum Speichern und Laden des Spielzustandes implementieren.

4.13.4.2 Speichern und Laden Nutzen Sie Serializable, um im Spielverlauf Ihre Entitäts-Instanzen zu speichern und in einer Datei abzulegen. Beim Start des Spiels sollen alle Entitäten aus der Datei eingelesen und im neuen Level platziert werden.

4.13.5 JUnit (Gruppe Basics)

4.13.5.1 Ziel In dieser Aufgabe sollen Sie eine gründliche Testabdeckung für Ihre in der Aufgabe “Fallen” implementierten Klassen und Methoden erstellen.

4.13.5.2 JUnit

- Überlegen Sie sich, was Sie alles testen müssen und wie Sie dies tun können.
- Ermitteln Sie die Testfälle mit Äquivalenzklassenbildung und Grenzwertanalyse.
- Implementieren Sie Ihre Testfälle mit JUnit und Mockito.

4.13.6 Freie Aufgabe

4.13.6.1 Freie Aufgabe In dieser Aufgabe implementieren Sie ein großes oder mehrere kleinere Features für Ihren Dungeon. Sie dürfen sich diese(s) Feature(s) selbst aussuchen und konzipieren. Beachten Sie dabei, dass Ihr Feature nicht Teil einer anderen Dungeon-Aufgabe oder Teil der Vorgabe sein kann.

Formulieren Sie Ihr Vorhaben ausführlich in Ihrer Konzeptskizze. Machen Sie besonders deutlich, **was** Sie erreichen wollen.

Diese Aufgabe bringt Ihnen bis zu 10 Punkte ein. Wählen Sie den Umfang daher entsprechend so, dass Sie nicht zu wenig, aber auch nicht zu viel zu tun haben. Orientieren Sie sich dabei an dem Umfang der anderen Dungeon-Aufgaben.

Auch für diese Aufgabe gelten die allgemeinen Anforderungen an das Praktikum.

4.14 Praktikumsaufgaben Gruppe Monster

Hier finden Sie die Praktikumsaufgaben aus der Gruppe “Monster”. Sie implementieren hier verschiedene Monster und unterschiedliche Kampfsysteme.

4.14.1 Nahkampf

4.14.1.1 Ziel In dieser Aufgabe implementieren Sie ein einfaches Nahkampfkonzept, damit der Held gegen die Monster kämpfen kann.

4.14.1.2 Voraussetzung Um diese Aufgabe lösen zu können, müssen Sie vorher “Monster” implementiert haben.

4.14.1.3 Nahkampf In den Vorgaben ist ein Konzept für den Fernkampf mit Skills mit einen Projektil System implementiert. Führen Sie eine Codeanalyse durch und erklären Sie, wie das Konzept funktioniert.

Erweitern Sie das Konzept so, dass es auch für den Nahkampf genutzt werden kann. Implementieren Sie mindestens einen Nahkampfangriff.

Trifft der Angriff auf einen Gegner (oder den Helden), verursacht es Schaden.

Bei einem Treffer soll der Getroffene eine kurze Distanz zurückgeschleudert werden. Beachten Sie dabei, dass niemand durch eine Wand bewegt wird.

4.14.2 Fernkampf

4.14.2.1 Ziel In dieser Aufgabe implementieren Sie Projektile für den Fernkampf, damit der Held gegen die Monster auf Distanz kämpfen kann.

4.14.2.2 Voraussetzung Um diese Aufgabe lösen zu können, müssen Sie vorher "Monster" implementiert haben.

4.14.2.3 Fernkampf In den Vorgaben ist ein Konzept für den Fernkampf mit Skills mit einen Projektil System implementiert. Führen Sie eine Codeanalyse durch und erklären Sie, wie das Konzept funktioniert.

Implementieren Sie zwei unterschiedliche Projektile, die sich vom bereits implementierten Feuerball durch ihr Flugverhalten unterscheiden. Denkbar wäre ein Bumerang, der im Bogen zurück zum Spieler fliegt, oder ein Projektil, welches über die Distanz "nach unten fällt", oder ein Projektil, das an Wänden abprallt.

Trifft das Projektil auf einen Gegner (oder den Helden), verursacht es Schaden und fliegt nicht mehr weiter. Trifft das Projektil auf eine Wand, fliegt es nicht mehr weiter.

Bei einem Treffer soll der Getroffene eine kurze Distanz zurückgeschleudert werden. Beachten Sie dabei, dass niemand durch eine Wand bewegt wird.

4.14.3 Monster

4.14.3.1 Ziel In dieser Aufgabe implementieren Sie simple Gegner, welche sich selbstständig im Dungeon umherbewegen.

4.14.3.2 Monster Implementieren Sie analog zum Helden verschiedene Monster (mindestens drei) für das Dungeon. Die Monster sollen auch jeweils eigene Animationen haben.

Überlegen Sie sich, welche Components die Monster haben sollen.

In den Vorgaben finden Sie das AI-System. Führen Sie eine Codeanalyse durch und erklären Sie, wie AI im Dungeon umgesetzt wurde.

Lassen Sie die Monster sich zufällig im Dungeon bewegen. Sie können dafür die vorgegebenen Strategien nutzen. Implementieren Sie jedoch mindestens eine weitere IIdleAI-Strategie.

Verteilen Sie eine zufällige Anzahl von Monstern bei jedem Levelstart. Verteilen Sie mehr und stärkere Monster, je tiefer der Spieler im Dungeon vorangeschritten ist.

4.14.4 Boss-Monster

4.14.4.1 Ziel In dieser Aufgabe implementieren Sie ein einzigartiges Boss-Monster.

4.14.4.2 Voraussetzung Um diese Aufgabe lösen zu können, sollten Sie vorher "Monster" implementieren.

4.14.4.3 Boss-Monster Zu jedem guten Spiel gehört ein guter Boss-Gegner. Boss-Monster sind oft deutlich stärker als die sonstigen Monster im Spiel. Was sie jedoch besonders macht, ist das einzigartige Verhalten dieser Monster.

In dieser Aufgabe geht es nicht darum, ein wiederverwendbares Verhalten zu programmieren, sondern etwas Einzigartiges.

Konzipieren und implementieren Sie Ihren eigenen Boss-Gegner.

Folgende Kriterien müssen mindestens erfüllt sein:

- Eigene Texturen und Animationen
- Eigenes Bewegungsmuster
- Eigenes Angriffsmuster
- Hat unterschiedliche Angriffe, je nachdem wie nah/fern der Held zum Boss steht
- Eine zweite Phase, nachdem der Boss 50% seiner Lebenspunkte verloren hat: Der Boss soll dann andere Angriffe/Angriffstechniken haben und aggressiver werden

4.14.5 Game-Over

4.14.5.1 Ziel In dieser Aufgabe implementieren Sie den Spieler-Tod.

4.14.5.2 Game-Over Wenn der Held stirbt, soll "Game Over" auf dem UI angezeigt werden. Der Spieler soll dann gefragt werden, ob er das Spiel beenden oder neustarten möchte. Wenn der Spieler das Spiel beenden möchte, wird die Anwendung geschlossen. Recherchieren Sie, wie Anwendungen korrekt und sicher in Java beendet werden können. Wenn der Spieler weiterspielen möchte, wird ein neuer Run gestartet.

4.14.6 JUnit (Gruppe Monster)

4.14.6.1 Ziel In dieser Aufgabe sollen Sie eine gründliche Testabdeckung für Ihre in der Aufgabe "Monster" oder "Fernkampf" implementierten Klassen und Methoden erstellen.

4.14.6.2 JUnit

- Überlegen Sie sich, was Sie alles testen müssen und wie Sie dies tun können.
- Ermitteln Sie die Testfälle mit Äquivalenzklassenbildung und Grenzwertanalyse.
- Implementieren Sie Ihre Testfälle mit JUnit und Mockito.

4.14.7 Freie Aufgabe

4.14.7.1 Freie Aufgabe In dieser Aufgabe implementieren Sie ein großes oder mehrere kleinere Features für Ihren Dungeon. Sie dürfen sich diese(s) Feature(s) selbst aussuchen und konzipieren. Beachten Sie dabei, dass Ihr Feature nicht Teil einer anderen Dungeon-Aufgabe oder Teil der Vorgabe sein kann.

Formulieren Sie Ihr Vorhaben ausführlich in Ihrer Konzeptskizze. Machen Sie besonders deutlich, **was** Sie erreichen wollen.

Diese Aufgabe bringt Ihnen bis zu 10 Punkte ein. Wählen Sie den Umfang daher entsprechend so, dass Sie nicht zu wenig, aber auch nicht zu viel zu tun haben. Orientieren Sie sich dabei an dem Umfang der anderen Dungeon-Aufgaben.

Auch für diese Aufgabe gelten die allgemeinen Anforderungen an das Praktikum.

4.15 Praktikumsaufgaben Gruppe Loot

Hier finden Sie die Praktikumsaufgaben aus der Gruppe “Loot”. Sie implementieren hier verschiedene Items wie Tränke, Waffen, … und verschiedene Rucksäcke und Kisten sowie Händler und Quests.

4.15.1 Items

4.15.1.1 Ziel In dieser Aufgabe implementieren Sie verschiedene Gegenstände, die der Spieler verwenden kann.

4.15.1.2 Items In den Vorgaben sind bereits Items und ein Inventory-Component implementiert. Führen Sie eine Codeanalyse durch und erklären Sie die Funktionalität.

Implementieren Sie mindestens drei verschiedene Typen von Gegenständen (beispielsweise Tränke, Nahrung, Zaubersprüchen, Rüstungen, Waffen, …).

Jeder Gegenstand soll vom Spieler verwendbar sein. Überlegen Sie sich passende “Funktionen”, die die einzelnen Gegenstände erfüllen.

Jeder Gegenstand soll graphisch dargestellt werden.

4.15.1.3 Taschen Implementieren Sie Taschen, die das Inventar des Spielers erweitern.

Taschen nehmen selbst **einen** Inventarplatz ein, können aber wiederum selbst **mehrere** Gegenstände **eines** Typus (z.B. Waffen oder Tränke) aufbewahren. Taschen können nur eine begrenzte Anzahl an Gegenständen beinhalten.

4.15.2 Freundlicher NPC-Geist

4.15.2.1 Ziel In dieser Aufgabe implementieren Sie einen freundlichen Geist als NPC (*non player character*)[45].

4.15.2.2 Freundlicher NPC: Geist Überlegen Sie sich, welche Komponenten ein NPC braucht.

Implementieren Sie einen NPC, der aussieht wie ein Geist und sich in der Nähe des Spielers aufhält und diesen (mit Abstand) verfolgt. Der Geist soll auch manchmal verschwinden oder (zufällig) durch das Level wandern.

Implementieren Sie einen Grabstein, der im Level mit einem Geist platziert wird. Findet der Spieler den Grabstein (und der Geist ist dabei), soll dieser den Spieler belohnen/bestrafen.

4.15.3 Dialogsystem

4.15.3.1 Ziel In dieser Aufgabe implementieren Sie ein dynamisches Dialogsystem.

4.15.3.2 Dialogsystem Nutzen Sie reguläre Ausdrücke (*RegExp*), um ein Dialogsystem zu implementieren.

Implementieren Sie eine Möglichkeit, um den Spieler Texte eingeben zu lassen. Dabei sollen verschiedene Eingaben über reguläre Ausdrücke erkannt werden und entsprechend darauf reagiert werden.

Implementieren Sie ein Rätsel im Spiel, welches eine zu einem regulären Ausdruck passende Eingabe erfordert.

Anmerkung: Diese Aufgabe lässt sich hervorragend mit den Aufgaben “Shop” und “NPC” kombinieren.

4.15.4 Shop

4.15.4.1 Ziel In dieser Aufgabe implementieren Sie einen Shop, in dem der Spieler Dinge wie Items oder Verbesserungen kaufen und verkaufen kann.

4.15.4.2 Voraussetungen Um diese Aufgabe lösen zu können, sollten Sie vorher folgende Aufgaben gelöst haben:

- “Item”
- “Dialogsystem”

4.15.4.3 Shop Implementieren Sie einen Shop, in dem der Spieler Items kaufen und verkaufen kann.

- Items kosten etwas. Implementieren Sie daher eine Währung und ein System, um an Zahlungsmittel zu kommen.
- Items gibt es nur in begrenzter Anzahl im Shop.
- Items sind im Einkauf teurer als im Verkauf (aus Sicht des Spielers).

[45] Ein NPC ist wie ein “normales” Monster ein *non player character* mit bestimmten Eigenschaften, greift aber den Spieler nicht an.

Der Kauf bzw. Verkauf soll mithilfe von RexExp und dem Dialogsystem erfolgen. Implementieren Sie eine Mechanik zum Feilschen per Texteingabe. Preise sollen ebenfalls über das Dialogsystem angefragt/vorgeschlagen werden.

4.15.5 Monster-Schatzkisten

4.15.5.1 Ziel In dieser Aufgabe implementieren Sie eine Monster-Schatzkiste, die der Spieler im Level finden und plündern kann. Vor dem Plündern muss die Kiste aber erst besiegt werden!

4.15.5.2 Voraussetzung Um diese Aufgabe lösen zu können, müssen Sie vorher “Monster” und “Item” gelöst haben.

4.15.5.3 Monster-Schatzkiste In den Vorgaben finden Sie die Implementierung einer Schatzkiste. Führen Sie eine Codeanalyse durch und erklären Sie die Funktionalität.

Konzipieren und implementieren Sie nun die “Monster-Schatzkiste” als einen neuen Monster-Typ. Dieses Monster soll aussehen wie eine Schatzkiste, greift den Spieler jedoch beim Versuch sie zu plündern an. Wenn der Spieler das Monster besiegt hat, soll das Schatzkisten-Monster sich wie eine normale Schatzkiste verhalten und seine Beute preisgeben.

4.15.6 Quests

4.15.6.1 Ziel In dieser Aufgabe implementieren Sie verschiedene Quests, also Aufgaben, die der Spieler lösen kann, um Belohnungen zu erhalten.

4.15.6.2 Quests Implementieren Sie verschiedene Aufgaben (*Quests*), die der Held erfüllen kann. Dies könnte beispielsweise das Sammeln einer bestimmten Zahl von Gegenständen oder das Besiegen einer bestimmten Zahl einer bestimmten Art von Monstern sein.

Für die Erfüllung der Aufgaben soll der Held mit Erfahrungspunkten und/oder Items (etwa ein Zauberstab) belohnt werden.

Die Quests müssen dem Helden geeignet angezeigt werden, beispielsweise über eine bestimmte Textur einer Bodenkachel oder über einen Geist, der die Quest vorschlägt.

Eine präsentierte Quest kann durch den Helden angenommen werden und ist dann aktiv. Der Held kann das Angebot aber auch ablehnen.

Zeigen Sie aktive (angenommene) Quests an.

Implementieren Sie mindestens eine nicht kampfabhängige Quest.

4.15.7 Lockpicking

4.15.7.1 Ziel In dieser Aufgabe implementieren Sie ein Mini-Spiel, um verschlossene Schatzkiste zu knacken.

4.15.7.2 Voraussetzung Um diese Aufgabe lösen zu können, müssen Sie vorher "Item" gelöst haben.

4.15.7.3 Lockpicking In den Vorgaben finden Sie die Implementierung einer Schatzkiste. Führen Sie eine Codeanalyse durch und erklären Sie die Funktionalität.

Implementieren Sie eine Möglichkeit, dass Schatzkisten verschlossen sein können und nur mit einen bestimmten Schlüssel geöffnet werden können.

Recherchieren Sie nach "Lockpicking-Minigames" aus anderen Spielen und implementieren Sie eine Umsetzung für Ihr Spiel. Wenn der Spieler nicht den passenden Schlüssel zu einer Schatzkiste gefunden hat, soll er die Schatzkiste durch das Lösen des Mini-Spiels öffnen können.

4.15.8 JUnit (Gruppe Loot)

4.15.8.1 Ziel In dieser Aufgabe sollen Sie eine gründliche Testabdeckung für Ihre in der Aufgabe "Quests" oder "Dialogsystem" implementierten Klassen und Methoden erstellen.

4.15.8.2 JUnit

- Überlegen Sie sich, was Sie alles testen müssen und wie Sie dies tun können.
- Ermitteln Sie die Testfälle mit Äquivalenzklassenbildung und Grenzwertanalyse.
- Implementieren Sie Ihre Testfälle mit JUnit und Mockito.

4.15.9 Freie Aufgabe

4.15.9.1 Freie Aufgabe In dieser Aufgabe implementieren Sie ein großes oder mehrere kleinere Features für Ihren Dungeon. Sie dürfen sich diese(s) Feature(s) selbst aussuchen und konzipieren. Beachten Sie dabei, dass Ihr Feature nicht Teil einer anderen Dungeon-Aufgabe oder Teil der Vorgabe sein kann.

Formulieren Sie Ihr Vorhaben ausführlich in Ihrer Konzeptskizze. Machen Sie besonders deutlich, **was** Sie erreichen wollen.

Diese Aufgabe bringt Ihnen bis zu 10 Punkte ein. Wählen Sie den Umfang daher entsprechend so, dass Sie nicht zu wenig, aber auch nicht zu viel zu tun haben. Orientieren Sie sich dabei an dem Umfang der anderen Dungeon-Aufgaben.

Auch für diese Aufgabe gelten die allgemeinen Anforderungen an das Praktikum.

Quellen

Atlassian Pty Ltd. 2022. „Become a Git Guru.“ https://www.atlassian.com/git/tutorials.

Bloch, J. 2018. *Effective Java*. 3. Aufl. Addison-Wesley.

Boles, D. 2008. *Parallele Programmierung spielend gelernt mit dem Java-Hamster-Modell*. Vieweg+Teubne. http://www.boles.de/hamster/band3.html.

Chacon, S. 2013. „GitHub Flow“. https://githubflow.github.io/.

Chacon, S., und B. Straub. 2014. *Pro Git*. 2. Aufl. Apress. https://git-scm.com/book/en/v2.

„Documentation Gitlab CI“. o. J. Zugegriffen 10. März 2022. http://git03-ifm-min.ad.hsbi.de/help/ci/.

Driessen, V. 2010. „Git-Flow: A Successful Git Branching Model“. https://nvie.com/posts/a-successful-git-branching-model/.

Eilebrecht, K., und G. Starke. 2013. *Patterns kompakt*. Springer.

Fowler, M. 2011. *Refactoring*. Addison-Wesley.

Gamma, E., R. Helm, R. E. Johnson, und J. Vlissides. 2011. *Design Patterns*. Addison-Wesley.

GitHub Inc. 2022a. „Documentation GitHub CI“. https://resources.github.com/ci-cd/.

Github Inc. 2022. „Git Cheat Sheets“. https://training.github.com/.

GitHub Inc. 2022b. „GitHub Flow“. https://docs.github.com/en/get-started/quickstart/github-flow.

Google Open Source. 2022. „Google Java Style Guide“. https://google.github.io/styleguide/javaguide.html.

Gradle Inc. 2022. „Gradle Build Tool“. https://gradle.org/.

Inden, M. 2013. *Der Weg zum Java-Profi*. 2. Aufl. Dpunkt.verlag.

Kleuker, S. 2018. *Grundkurs Software-Engineering mit UML*. Springer Vieweg. https://doi.org/10.1007/978-3-658-19969-2.

Kleuker, S. 2019. *Qualitätssicherung durch Softwaretests*. Springer Vieweg. https://doi.org/10.1007/978-3-658-24886-4.

Martin, R. 2009. *Clean Code*. Mitp.

Miell, I., und A. H. Sayers. 2019. *Docker in Practice*. Manning Publications.

Nickoloff, D. 2019. *Docker in Action*. Manning Publications.

Nystrom, R. 2014. *Game Programming Patterns*. Genever Benning. https://github.com/munificent/game-programming-patterns.

Oracle Corporation. 2022a. „Java Core Libraries Developer Guide“. https://docs.oracle.com/en/java/javase/17/core/index.html.

Oracle Corporation. 2022b. „Learn Java“. https://dev.java/learn/.



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- “A Note About Git Commit Messages” by Tim Pope on tbaggery.com
- “*Three strikes…*”: (Fowler 2011, p. 58): “The Rule of Three”
- “*Any fool…*”: (Fowler 2011, p. 15)
- “356: Refactoring” by Andreas Bogk on Lutz Donnerhacke: “Fachbegriffe der Informatik”
- “*Refactoring*”: (Fowler 2011, p. 53)

> **Last modified:** 2e1985a Thu Mar 26 09:00:29 2026 +0100 tooling: fix settings for pandoc-lecture-zen